

ChArUco Board を使ったカメラキャリブレーション

0.0.2

構築: Doxygen 1.15.0

1 ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版.	1
2 REQUESTS.md - アプリケーションビルドおよび開発構成定義書	3
2.1 1. ソースコード	3
2.2 2. CMakeLists.txt の設定	3
2.2.1 2-2. 外部ライブラリの配置場所	4
2.2.2 2-3. OpenGL 関連のヘッダファイル	4
2.2.3 2-4. GLFW	4
2.2.4 2-5. Dear ImGui	4
2.2.5 2-6. Native File Dialog Extended	5
2.2.6 2-7. picojson	5
2.2.7 2-8. OpenCV	5
2.2.8 2-9. 実行ファイル	5
2.3 3. README.md, GEMINI.md, .gitignore	5
2.4 4. ビデオキャプチャデバイスの設計意図	6
2.4.1 4-1. UVC カメラ初期化におけるデコード初期化の分離	6
2.4.2 4-2. 手動デコードによる CPU 高速処理方式 (AVP自動変換の廃止)	6
2.4.3 4-3. 安全なフォーマットリスト動的取得設計 (生ポインタ管理の廃止)	6
2.5 5. UI (メニュー表示) の改善要請	6
2.5.1 5-1. ビデオフォーマット選択ドロップダウンの3分割表示と最適化	6
2.6 6. GPUゼロコピー化の断念とPBOの維持	7
2.6.1 6-1. OpenCVによる画像処理設計との整合性	7
3 名前空間索引	9
3.1 名前空間一覧	9
4 階層索引	11
4.1 クラス階層	11
5 クラス索引	13
5.1 クラス一覧	13
6 ファイル索引	15
6.1 ファイル一覧	15
7 名前空間詳解	17
7.1 gg 名前空間	17
7.1.1 詳解	19
7.1.2 列挙型詳解	20
7.1.2.1 BindingPoints	20
7.1.3 関数詳解	20
7.1.3.1 _ggError()	20
7.1.3.2 _ggFBOError()	20
7.1.3.3 ggArraysObj()	20

7.1.3.4 ggConjugate()	21
7.1.3.5 ggCreateComputeShader()	22
7.1.3.6 ggCreateNormalMap()	22
7.1.3.7 ggCreateShader()	23
7.1.3.8 ggCross() [1/2]	24
7.1.3.9 ggCross() [2/2]	25
7.1.3.10 ggDistance3() [1/2]	25
7.1.3.11 ggDistance3() [2/2]	26
7.1.3.12 ggDistance4() [1/2]	26
7.1.3.13 ggDistance4() [2/2]	27
7.1.3.14 ggDot3() [1/2]	28
7.1.3.15 ggDot3() [2/2]	28
7.1.3.16 ggDot4() [1/2]	29
7.1.3.17 ggDot4() [2/2]	29
7.1.3.18 ggElementsMesh()	30
7.1.3.19 ggElementsObj()	31
7.1.3.20 ggElementsSphere()	32
7.1.3.21 ggEllipse()	32
7.1.3.22 ggEulerQuaternion() [1/3]	33
7.1.3.23 ggEulerQuaternion() [2/3]	33
7.1.3.24 ggEulerQuaternion() [3/3]	34
7.1.3.25 ggFrustum()	35
7.1.3.26 ggIdentity()	36
7.1.3.27 ggIdentityQuaternion()	36
7.1.3.28 ggInit()	36
7.1.3.29 ggInvert() [1/2]	37
7.1.3.30 ggInvert() [2/2]	38
7.1.3.31 ggLength3() [1/2]	38
7.1.3.32 ggLength3() [2/2]	39
7.1.3.33 ggLength4() [1/2]	39
7.1.3.34 ggLength4() [2/2]	40
7.1.3.35 ggLoadComputeShader()	41
7.1.3.36 ggLoadHeight()	41
7.1.3.37 ggLoadImage()	42
7.1.3.38 ggLoadShader() [1/2]	43
7.1.3.39 ggLoadShader() [2/2]	43
7.1.3.40 ggLoadSimpleObj() [1/2]	44
7.1.3.41 ggLoadSimpleObj() [2/2]	45
7.1.3.42 ggLoadTexture()	45
7.1.3.43 ggLookat() [1/3]	46
7.1.3.44 ggLookat() [2/3]	47
7.1.3.45 ggLookat() [3/3]	47

7.1.3.46 ggMatrixQuaternion() [1/2]	48
7.1.3.47 ggMatrixQuaternion() [2/2]	49
7.1.3.48 ggNorm()	49
7.1.3.49 ggNormal()	50
7.1.3.50 ggNormalize()	50
7.1.3.51 ggNormalize3() [1/4]	51
7.1.3.52 ggNormalize3() [2/4]	51
7.1.3.53 ggNormalize3() [3/4]	52
7.1.3.54 ggNormalize3() [4/4]	53
7.1.3.55 ggNormalize4() [1/4]	53
7.1.3.56 ggNormalize4() [2/4]	54
7.1.3.57 ggNormalize4() [3/4]	54
7.1.3.58 ggNormalize4() [4/4]	55
7.1.3.59 ggOrthogonal()	55
7.1.3.60 ggPerspective()	56
7.1.3.61 ggPointsCube()	57
7.1.3.62 ggPointsSphere()	57
7.1.3.63 ggQuaternionMatrix()	57
7.1.3.64 ggQuaternionTransposeMatrix()	58
7.1.3.65 ggReadImage()	58
7.1.3.66 ggRectangle()	60
7.1.3.67 ggRotate() [1/5]	60
7.1.3.68 ggRotate() [2/5]	60
7.1.3.69 ggRotate() [3/5]	61
7.1.3.70 ggRotate() [4/5]	61
7.1.3.71 ggRotate() [5/5]	62
7.1.3.72 ggRotateQuaternion() [1/3]	63
7.1.3.73 ggRotateQuaternion() [2/3]	63
7.1.3.74 ggRotateQuaternion() [3/3]	63
7.1.3.75 ggRotateX()	64
7.1.3.76 ggRotateY()	65
7.1.3.77 ggRotateZ()	66
7.1.3.78 ggSaveColor()	66
7.1.3.79 ggSaveDepth()	67
7.1.3.80 ggSaveTga()	67
7.1.3.81 ggScale() [1/3]	68
7.1.3.82 ggScale() [2/3]	69
7.1.3.83 ggScale() [3/3]	69
7.1.3.84 ggSlerp() [1/4]	70
7.1.3.85 ggSlerp() [2/4]	70
7.1.3.86 ggSlerp() [3/4]	71
7.1.3.87 ggSlerp() [4/4]	71

7.1.3.88 ggTranslate() [1/3]	72
7.1.3.89 ggTranslate() [2/3]	73
7.1.3.90 ggTranslate() [3/3]	73
7.1.3.91 ggTranspose()	74
7.1.3.92 operator*()	75
7.1.3.93 operator+() [1/2]	75
7.1.3.94 operator+() [2/2]	75
7.1.3.95 operator-() [1/2]	76
7.1.3.96 operator-() [2/2]	76
7.1.3.97 operator/()	76
7.1.4 変数詳解	77
7.1.4.1 ggBufferAlignment	77
8 クラス詳解	79
8.1 Buffer クラス	79
8.1.1 詳解	80
8.1.2 構築子と解体子	80
8.1.2.1 Buffer() [1/4]	80
8.1.2.2 Buffer() [2/4]	81
8.1.2.3 Buffer() [3/4]	82
8.1.2.4 Buffer() [4/4]	82
8.1.2.5 ~Buffer()	83
8.1.3 関数詳解	83
8.1.3.1 bindBuffer()	83
8.1.3.2 channelsToFormat()	83
8.1.3.3 copy()	84
8.1.3.4 copyBuffer()	85
8.1.3.5 create()	86
8.1.3.6 discard()	87
8.1.3.7 formatToChannels()	87
8.1.3.8 getAspect()	88
8.1.3.9 getBufferName()	88
8.1.3.10 getChannels()	89
8.1.3.11 getFormat()	89
8.1.3.12 getHeight()	90
8.1.3.13 getSize()	90
8.1.3.14 getWidth()	91
8.1.3.15 map()	92
8.1.3.16 operator=() [1/2]	92
8.1.3.17 operator=() [2/2]	92
8.1.3.18 resize() [1/2]	93
8.1.3.19 resize() [2/2]	94

8.1.3.20 unbindBuffer()	95
8.1.3.21 unmap()	95
8.2 Calibration クラス	96
8.2.1 詳解	96
8.2.2 構築子と解体子	97
8.2.2.1 Calibration() [1/2]	97
8.2.2.2 Calibration() [2/2]	97
8.2.2.3 ~Calibration()	97
8.2.3 関数詳解	98
8.2.3.1 calibrate()	98
8.2.3.2 createBoard()	98
8.2.3.3 detectBoard()	98
8.2.3.4 detectMarkers()	99
8.2.3.5 discardCorners()	100
8.2.3.6 drawBoard()	100
8.2.3.7 finished()	100
8.2.3.8 getAllMarkerPoses()	100
8.2.3.9 getCameraMatrix()	101
8.2.3.10 getCornersCount()	101
8.2.3.11 getDistortionCoefficients()	101
8.2.3.12 getReprojectionError()	102
8.2.3.13 getSampleCount()	102
8.2.3.14 getTotalCount()	102
8.2.3.15 loadParameters()	102
8.2.3.16 operator=()	103
8.2.3.17 recordCorners()	103
8.2.3.18 RvecTvecToPose()	103
8.2.3.19 saveParameters()	104
8.2.3.20 setDictionary()	105
8.2.4 メンバ詳解	105
8.2.4.1 dictionaryList	105
8.3 CamCv クラス	106
8.3.1 詳解	108
8.3.2 構築子と解体子	108
8.3.2.1 CamCv()	108
8.3.2.2 ~CamCv()	108
8.3.3 関数詳解	108
8.3.3.1 close()	108
8.3.3.2 decreaseExposure()	108
8.3.3.3 decreaseGain()	109
8.3.3.4 getCodec() [1/2]	109
8.3.3.5 getCodec() [2/2]	109

8.3.3.6	getFps()	110
8.3.3.7	getPosition()	110
8.3.3.8	increaseExposure()	110
8.3.3.9	increaseGain()	110
8.3.3.10	open() [1/2]	111
8.3.3.11	open() [2/2]	111
8.3.3.12	setExposure()	112
8.3.3.13	setGain()	112
8.3.3.14	setPosition()	113
8.4	Camera クラス	113
8.4.1	詳解	115
8.4.2	構築子と解体子	115
8.4.2.1	Camera() [1/2]	115
8.4.2.2	Camera() [2/2]	115
8.4.2.3	~Camera()	115
8.4.3	関数詳解	116
8.4.3.1	capture()	116
8.4.3.2	close()	116
8.4.3.3	decreaseExposure()	117
8.4.3.4	decreaseGain()	117
8.4.3.5	getChannels()	117
8.4.3.6	getFps()	117
8.4.3.7	getFrames()	117
8.4.3.8	getHeight()	118
8.4.3.9	getSize()	118
8.4.3.10	getWidth()	118
8.4.3.11	increaseExposure()	118
8.4.3.12	increaseGain()	118
8.4.3.13	isRunning()	118
8.4.3.14	operator=()	119
8.4.3.15	start()	119
8.4.3.16	stop()	119
8.4.3.17	transmit() [1/3]	120
8.4.3.18	transmit() [2/3]	120
8.4.3.19	transmit() [3/3]	120
8.4.4	メンバ詳解	121
8.4.4.1	captured	121
8.4.4.2	channels	121
8.4.4.3	frame	121
8.4.4.4	height	121
8.4.4.5	image	121
8.4.4.6	in	121

8.4.4.7 interval	121
8.4.4.8 mtx	121
8.4.4.9 out	121
8.4.4.10 running	121
8.4.4.11 thr	122
8.4.4.12 total	122
8.4.4.13 width	122
8.5 CamImage クラス	122
8.5.1 詳解	124
8.5.2 構築子と解体子	124
8.5.2.1 CamImage() [1/2]	124
8.5.2.2 CamImage() [2/2]	124
8.5.2.3 ~CamImage()	125
8.5.3 関数詳解	125
8.5.3.1 isOpened()	125
8.5.3.2 load()	125
8.5.3.3 open()	126
8.5.3.4 save()	126
8.6 CamMf クラス	127
8.6.1 詳解	129
8.6.2 構築子と解体子	129
8.6.2.1 CamMf()	129
8.6.2.2 ~CamMf()	129
8.6.3 関数詳解	130
8.6.3.1 capture()	130
8.6.3.2 close()	130
8.6.3.3 getDeviceList()	131
8.6.3.4 getFormatList()	131
8.6.3.5 open()	131
8.6.3.6 select()	131
8.7 Capture クラス	132
8.7.1 詳解	132
8.7.2 構築子と解体子	133
8.7.2.1 Capture() [1/2]	133
8.7.2.2 Capture() [2/2]	133
8.7.2.3 ~Capture()	133
8.7.3 関数詳解	133
8.7.3.1 close()	133
8.7.3.2 getFps()	134
8.7.3.3 getSize()	134
8.7.3.4 isImage()	134
8.7.3.5 isOpend()	134

8.7.3.6 openDevice()	134
8.7.3.7 openImage()	135
8.7.3.8 openMovie()	135
8.7.3.9 operator bool()	136
8.7.3.10 retrieve()	136
8.7.3.11 start()	136
8.7.3.12 stop()	136
8.8 Compute クラス	137
8.8.1 詳解	137
8.8.2 構築子と解体子	137
8.8.2.1 Compute()	137
8.8.2.2 ~Compute()	137
8.8.3 関数詳解	138
8.8.3.1 execute()	138
8.8.3.2 get()	138
8.8.3.3 use()	138
8.9 Config クラス	138
8.9.1 詳解	139
8.9.2 構築子と解体子	139
8.9.2.1 Config()	139
8.9.2.2 ~Config()	139
8.9.3 関数詳解	140
8.9.3.1 getCheckerLength()	140
8.9.3.2 getDictionaryName()	140
8.9.3.3 getHeight()	140
8.9.3.4 getInitialImage()	141
8.9.3.5 getMarkerLength()	141
8.9.3.6 getTitle()	141
8.9.3.7 getWidth()	142
8.9.3.8 initialize()	142
8.9.3.9 load()	143
8.9.3.10 save()	143
8.9.4 フレンドと関連関数の詳解	144
8.9.4.1 Menu	144
8.10 Expand クラス	144
8.10.1 詳解	145
8.10.2 構築子と解体子	145
8.10.2.1 Expand() [1/2]	145
8.10.2.2 Expand() [2/2]	145
8.10.2.3 ~Expand()	146
8.10.3 関数詳解	146
8.10.3.1 getProgram()	146

8.10.3.2 operator=()	146
8.10.3.3 setup()	146
8.11 Framebuffer クラス	147
8.11.1 詳解	150
8.11.2 構築子と解体子	150
8.11.2.1 Framebuffer() [1/4]	150
8.11.2.2 Framebuffer() [2/4]	150
8.11.2.3 Framebuffer() [3/4]	151
8.11.2.4 Framebuffer() [4/4]	151
8.11.2.5 ~Framebuffer()	152
8.11.3 関数詳解	152
8.11.3.1 bindFramebuffer()	152
8.11.3.2 copy()	152
8.11.3.3 create()	153
8.11.3.4 discard()	154
8.11.3.5 getChannels()	154
8.11.3.6 getFramebufferName()	155
8.11.3.7 getSize()	155
8.11.3.8 operator=() [1/2]	155
8.11.3.9 operator=() [2/2]	156
8.11.3.10 show()	156
8.11.3.11 unbindFramebuffer()	157
8.11.3.12 update() [1/2]	157
8.11.3.13 update() [2/2]	158
8.12 GgApp クラス	159
8.12.1 詳解	159
8.12.2 構築子と解体子	159
8.12.2.1 GgApp() [1/3]	159
8.12.2.2 GgApp() [2/3]	160
8.12.2.3 GgApp() [3/3]	160
8.12.2.4 ~GgApp()	161
8.12.3 関数詳解	161
8.12.3.1 main()	161
8.12.3.2 operator=() [1/2]	163
8.12.3.3 operator=() [2/2]	163
8.13 gg::GgBuffer< T > クラステンプレート	164
8.13.1 詳解	164
8.13.2 構築子と解体子	165
8.13.2.1 GgBuffer() [1/3]	165
8.13.2.2 GgBuffer() [2/3]	165
8.13.2.3 GgBuffer() [3/3]	165
8.13.2.4 ~GgBuffer()	166

8.13.3 関数詳解	166
8.13.3.1 bind()	166
8.13.3.2 copy()	166
8.13.3.3 getBuffer()	167
8.13.3.4 getCount()	167
8.13.3.5 getStride()	167
8.13.3.6 getTarget()	168
8.13.3.7 map() [1/2]	168
8.13.3.8 map() [2/2]	169
8.13.3.9 operator=() [1/2]	169
8.13.3.10 operator=() [2/2]	170
8.13.3.11 read()	170
8.13.3.12 send()	171
8.13.3.13 unbind()	172
8.13.3.14 unmap()	172
8.14 gg::GgColorTexture クラス	172
8.14.1 詳解	172
8.14.2 構築子と解体子	173
8.14.2.1 GgColorTexture() [1/3]	173
8.14.2.2 GgColorTexture() [2/3]	173
8.14.2.3 GgColorTexture() [3/3]	173
8.14.2.4 ~GgColorTexture()	174
8.14.3 関数詳解	174
8.14.3.1 load() [1/2]	174
8.14.3.2 load() [2/2]	175
8.15 gg::GgElements クラス	176
8.15.1 詳解	177
8.15.2 構築子と解体子	177
8.15.2.1 GgElements() [1/2]	177
8.15.2.2 GgElements() [2/2]	178
8.15.2.3 ~GgElements()	178
8.15.3 関数詳解	178
8.15.3.1 draw()	178
8.15.3.2 getIndexBuffer()	179
8.15.3.3 getIndexCount()	179
8.15.3.4 load()	180
8.15.3.5 send()	181
8.16 gg::GgMatrix クラス	181
8.16.1 詳解	184
8.16.2 構築子と解体子	184
8.16.2.1 GgMatrix() [1/6]	184
8.16.2.2 GgMatrix() [2/6]	184

8.16.2.3 GgMatrix() [3/6]	185
8.16.2.4 GgMatrix() [4/6]	185
8.16.2.5 GgMatrix() [5/6]	186
8.16.2.6 GgMatrix() [6/6]	186
8.16.2.7 ~GgMatrix()	187
8.16.3 関数詳解	187
8.16.3.1 frustum()	187
8.16.3.2 get() [1/2]	187
8.16.3.3 get() [2/2]	188
8.16.3.4 invert()	189
8.16.3.5 loadFrustum()	189
8.16.3.6 loadIdentity()	190
8.16.3.7 loadInvert() [1/2]	190
8.16.3.8 loadInvert() [2/2]	191
8.16.3.9 loadLookat() [1/3]	191
8.16.3.10 loadLookat() [2/3]	192
8.16.3.11 loadLookat() [3/3]	193
8.16.3.12 loadNormal() [1/2]	194
8.16.3.13 loadNormal() [2/2]	195
8.16.3.14 loadOrthogonal()	195
8.16.3.15 loadPerspective()	196
8.16.3.16 loadRotate() [1/5]	197
8.16.3.17 loadRotate() [2/5]	197
8.16.3.18 loadRotate() [3/5]	198
8.16.3.19 loadRotate() [4/5]	198
8.16.3.20 loadRotate() [5/5]	199
8.16.3.21 loadRotateX()	200
8.16.3.22 loadRotateY()	201
8.16.3.23 loadRotateZ()	201
8.16.3.24 loadScale() [1/3]	201
8.16.3.25 loadScale() [2/3]	202
8.16.3.26 loadScale() [3/3]	203
8.16.3.27 loadTranslate() [1/3]	203
8.16.3.28 loadTranslate() [2/3]	204
8.16.3.29 loadTranslate() [3/3]	204
8.16.3.30 loadTranspose() [1/2]	205
8.16.3.31 loadTranspose() [2/2]	206
8.16.3.32 lookat() [1/3]	206
8.16.3.33 lookat() [2/3]	207
8.16.3.34 lookat() [3/3]	207
8.16.3.35 normal()	208
8.16.3.36 operator*() [1/3]	209

8.16.3.37 operator*() [2/3]	210
8.16.3.38 operator*() [3/3]	210
8.16.3.39 operator*=() [1/2]	211
8.16.3.40 operator*=() [2/2]	211
8.16.3.41 operator+() [1/2]	212
8.16.3.42 operator+() [2/2]	213
8.16.3.43 operator+=() [1/2]	214
8.16.3.44 operator+=() [2/2]	214
8.16.3.45 operator-() [1/2]	215
8.16.3.46 operator-() [2/2]	215
8.16.3.47 operator-=() [1/2]	216
8.16.3.48 operator-=() [2/2]	216
8.16.3.49 operator/() [1/2]	217
8.16.3.50 operator/() [2/2]	218
8.16.3.51 operator/=() [1/2]	219
8.16.3.52 operator/=() [2/2]	219
8.16.3.53 operator=() [1/3]	220
8.16.3.54 operator=() [2/3]	220
8.16.3.55 operator=() [3/3]	221
8.16.3.56 orthogonal()	221
8.16.3.57 perspective()	222
8.16.3.58 projection() [1/4]	222
8.16.3.59 projection() [2/4]	222
8.16.3.60 projection() [3/4]	223
8.16.3.61 projection() [4/4]	223
8.16.3.62 rotate() [1/5]	223
8.16.3.63 rotate() [2/5]	224
8.16.3.64 rotate() [3/5]	224
8.16.3.65 rotate() [4/5]	225
8.16.3.66 rotate() [5/5]	225
8.16.3.67 rotateX()	226
8.16.3.68 rotateY()	227
8.16.3.69 rotateZ()	228
8.16.3.70 scale() [1/3]	228
8.16.3.71 scale() [2/3]	229
8.16.3.72 scale() [3/3]	229
8.16.3.73 translate() [1/3]	230
8.16.3.74 translate() [2/3]	231
8.16.3.75 translate() [3/3]	231
8.16.3.76 transpose()	232
8.17 gg::GgNormalTexture クラス	233
8.17.1 詳解	233

8.17.2 構築子と解体子	233
8.17.2.1 GgNormalTexture() [1/3]	233
8.17.2.2 GgNormalTexture() [2/3]	233
8.17.2.3 GgNormalTexture() [3/3]	234
8.17.2.4 ~GgNormalTexture()	234
8.17.3 関数詳解	234
8.17.3.1 load() [1/2]	234
8.17.3.2 load() [2/2]	235
8.18 gg::GgPoints クラス	236
8.18.1 詳解	237
8.18.2 構築子と解体子	237
8.18.2.1 GgPoints() [1/2]	237
8.18.2.2 GgPoints() [2/2]	238
8.18.2.3 ~GgPoints()	238
8.18.3 関数詳解	238
8.18.3.1 draw()	238
8.18.3.2 getBuffer()	239
8.18.3.3 getCount()	239
8.18.3.4 load()	239
8.18.3.5 operator bool()	240
8.18.3.6 operator"!()"	240
8.18.3.7 send()	240
8.19 gg::GgPointShader クラス	241
8.19.1 詳解	241
8.19.2 構築子と解体子	242
8.19.2.1 GgPointShader() [1/3]	242
8.19.2.2 GgPointShader() [2/3]	242
8.19.2.3 GgPointShader() [3/3]	243
8.19.2.4 ~GgPointShader()	243
8.19.3 関数詳解	243
8.19.3.1 get()	243
8.19.3.2 load() [1/2]	244
8.19.3.3 load() [2/2]	245
8.19.3.4 loadMatrix() [1/2]	245
8.19.3.5 loadMatrix() [2/2]	246
8.19.3.6 loadModelviewMatrix() [1/2]	247
8.19.3.7 loadModelviewMatrix() [2/2]	247
8.19.3.8 loadProjectionMatrix() [1/2]	248
8.19.3.9 loadProjectionMatrix() [2/2]	249
8.19.3.10 unuse()	249
8.19.3.11 use() [1/5]	249
8.19.3.12 use() [2/5]	250

8.19.3.13 use() [3/5]	251
8.19.3.14 use() [4/5]	251
8.19.3.15 use() [5/5]	252
8.20 gg::GgQuaternion クラス	252
8.20.1 詳解	256
8.20.2 構築子と解体子	256
8.20.2.1 GgQuaternion() [1/7]	256
8.20.2.2 GgQuaternion() [2/7]	256
8.20.2.3 GgQuaternion() [3/7]	257
8.20.2.4 GgQuaternion() [4/7]	257
8.20.2.5 GgQuaternion() [5/7]	258
8.20.2.6 GgQuaternion() [6/7]	258
8.20.2.7 GgQuaternion() [7/7]	259
8.20.2.8 ~GgQuaternion()	259
8.20.3 関数詳解	259
8.20.3.1 add() [1/4]	259
8.20.3.2 add() [2/4]	259
8.20.3.3 add() [3/4]	260
8.20.3.4 add() [4/4]	260
8.20.3.5 conjugate()	261
8.20.3.6 divide() [1/4]	262
8.20.3.7 divide() [2/4]	263
8.20.3.8 divide() [3/4]	263
8.20.3.9 divide() [4/4]	264
8.20.3.10 euler() [1/3]	264
8.20.3.11 euler() [2/3]	265
8.20.3.12 euler() [3/3]	266
8.20.3.13 get()	266
8.20.3.14 getConjugateMatrix() [1/3]	267
8.20.3.15 getConjugateMatrix() [2/3]	267
8.20.3.16 getConjugateMatrix() [3/3]	268
8.20.3.17 getMatrix() [1/3]	268
8.20.3.18 getMatrix() [2/3]	269
8.20.3.19 getMatrix() [3/3]	270
8.20.3.20 invert()	270
8.20.3.21 loadAdd() [1/4]	271
8.20.3.22 loadAdd() [2/4]	271
8.20.3.23 loadAdd() [3/4]	272
8.20.3.24 loadAdd() [4/4]	272
8.20.3.25 loadConjugate() [1/2]	273
8.20.3.26 loadConjugate() [2/2]	274
8.20.3.27 loadDivide() [1/4]	274

8.20.3.28 loadDivide() [2/4]	275
8.20.3.29 loadDivide() [3/4]	275
8.20.3.30 loadDivide() [4/4]	276
8.20.3.31 loadEuler() [1/2]	277
8.20.3.32 loadEuler() [2/2]	277
8.20.3.33 loadIdentity()	278
8.20.3.34 loadInvert() [1/2]	279
8.20.3.35 loadInvert() [2/2]	279
8.20.3.36 loadMatrix() [1/2]	280
8.20.3.37 loadMatrix() [2/2]	281
8.20.3.38 loadMultiply() [1/4]	282
8.20.3.39 loadMultiply() [2/4]	282
8.20.3.40 loadMultiply() [3/4]	282
8.20.3.41 loadMultiply() [4/4]	283
8.20.3.42 loadNormalize() [1/2]	284
8.20.3.43 loadNormalize() [2/2]	284
8.20.3.44 loadRotate() [1/3]	285
8.20.3.45 loadRotate() [2/3]	285
8.20.3.46 loadRotate() [3/3]	286
8.20.3.47 loadRotateX()	287
8.20.3.48 loadRotateY()	288
8.20.3.49 loadRotateZ()	288
8.20.3.50 loadSlerp() [1/4]	289
8.20.3.51 loadSlerp() [2/4]	290
8.20.3.52 loadSlerp() [3/4]	290
8.20.3.53 loadSlerp() [4/4]	291
8.20.3.54 loadSubtract() [1/4]	292
8.20.3.55 loadSubtract() [2/4]	292
8.20.3.56 loadSubtract() [3/4]	293
8.20.3.57 loadSubtract() [4/4]	293
8.20.3.58 multiply() [1/4]	294
8.20.3.59 multiply() [2/4]	295
8.20.3.60 multiply() [3/4]	295
8.20.3.61 multiply() [4/4]	296
8.20.3.62 norm()	296
8.20.3.63 normalize()	297
8.20.3.64 operator*() [1/3]	298
8.20.3.65 operator*() [2/3]	298
8.20.3.66 operator*() [3/3]	298
8.20.3.67 operator*=() [1/3]	299
8.20.3.68 operator*=() [2/3]	299
8.20.3.69 operator*=() [3/3]	300

8.20.3.70 operator+() [1/3]	300
8.20.3.71 operator+() [2/3]	300
8.20.3.72 operator+() [3/3]	301
8.20.3.73 operator+=() [1/3]	301
8.20.3.74 operator+=() [2/3]	301
8.20.3.75 operator+=() [3/3]	302
8.20.3.76 operator-() [1/3]	302
8.20.3.77 operator-() [2/3]	302
8.20.3.78 operator-() [3/3]	302
8.20.3.79 operator-=() [1/3]	303
8.20.3.80 operator-=() [2/3]	303
8.20.3.81 operator-=() [3/3]	303
8.20.3.82 operator/() [1/3]	304
8.20.3.83 operator/() [2/3]	304
8.20.3.84 operator/() [3/3]	305
8.20.3.85 operator/=() [1/3]	305
8.20.3.86 operator/=() [2/3]	305
8.20.3.87 operator/=() [3/3]	306
8.20.3.88 operator=() [1/4]	306
8.20.3.89 operator=() [2/4]	307
8.20.3.90 operator=() [3/4]	307
8.20.3.91 operator=() [4/4]	308
8.20.3.92 rotate() [1/3]	308
8.20.3.93 rotate() [2/3]	309
8.20.3.94 rotate() [3/3]	309
8.20.3.95 rotateX()	310
8.20.3.96 rotateY()	311
8.20.3.97 rotateZ()	311
8.20.3.98 slerp() [1/2]	312
8.20.3.99 slerp() [2/2]	312
8.20.3.100 subtract() [1/4]	313
8.20.3.101 subtract() [2/4]	313
8.20.3.102 subtract() [3/4]	314
8.20.3.103 subtract() [4/4]	314
8.21 gg::GgShader クラス	315
8.21.1 詳解	316
8.21.2 構築子と解体子	316
8.21.2.1 GgShader() [1/4]	316
8.21.2.2 GgShader() [2/4]	317
8.21.2.3 GgShader() [3/4]	317
8.21.2.4 GgShader() [4/4]	318
8.21.2.5 ~GgShader()	318

8.21.3 関数詳解	318
8.21.3.1 get()	318
8.21.3.2 operator=() [1/2]	318
8.21.3.3 operator=() [2/2]	319
8.21.3.4 unuse()	319
8.21.3.5 use()	319
8.22 gg::GgShape クラス	320
8.22.1 詳解	320
8.22.2 構築子と解体子	320
8.22.2.1 GgShape()	320
8.22.2.2 ~GgShape()	321
8.22.3 関数詳解	321
8.22.3.1 draw()	321
8.22.3.2 get()	321
8.22.3.3 getMode()	322
8.22.3.4 operator bool()	322
8.22.3.5 operator"!()	323
8.22.3.6 setMode()	323
8.23 gg::GgSimpleObj クラス	323
8.23.1 詳解	323
8.23.2 構築子と解体子	323
8.23.2.1 GgSimpleObj()	323
8.23.2.2 ~GgSimpleObj()	324
8.23.3 関数詳解	324
8.23.3.1 draw()	324
8.23.3.2 get()	324
8.23.3.3 operator bool()	325
8.23.3.4 operator"!()	325
8.24 gg::GgSimpleShader クラス	325
8.24.1 詳解	327
8.24.2 構築子と解体子	327
8.24.2.1 GgSimpleShader() [1/4]	327
8.24.2.2 GgSimpleShader() [2/4]	328
8.24.2.3 GgSimpleShader() [3/4]	329
8.24.2.4 GgSimpleShader() [4/4]	329
8.24.2.5 ~GgSimpleShader()	329
8.24.3 関数詳解	329
8.24.3.1 load() [1/2]	329
8.24.3.2 load() [2/2]	330
8.24.3.3 loadMatrix() [1/4]	331
8.24.3.4 loadMatrix() [2/4]	332
8.24.3.5 loadMatrix() [3/4]	332

8.24.3.6 loadMatrix() [4/4]	333
8.24.3.7 loadModelviewMatrix() [1/4]	334
8.24.3.8 loadModelviewMatrix() [2/4]	334
8.24.3.9 loadModelviewMatrix() [3/4]	335
8.24.3.10 loadModelviewMatrix() [4/4]	335
8.24.3.11 operator=()	336
8.24.3.12 use() [1/13]	337
8.24.3.13 use() [2/13]	338
8.24.3.14 use() [3/13]	339
8.24.3.15 use() [4/13]	339
8.24.3.16 use() [5/13]	340
8.24.3.17 use() [6/13]	341
8.24.3.18 use() [7/13]	341
8.24.3.19 use() [8/13]	342
8.24.3.20 use() [9/13]	342
8.24.3.21 use() [10/13]	343
8.24.3.22 use() [11/13]	343
8.24.3.23 use() [12/13]	344
8.24.3.24 use() [13/13]	344
8.25 gg::GgTexture クラス	345
8.25.1 詳解	345
8.25.2 構築子と解体子	346
8.25.2.1 GgTexture() [1/3]	346
8.25.2.2 GgTexture() [2/3]	347
8.25.2.3 GgTexture() [3/3]	347
8.25.2.4 ~GgTexture()	348
8.25.3 関数詳解	348
8.25.3.1 bind()	348
8.25.3.2 getHeight()	348
8.25.3.3 getSize() [1/2]	348
8.25.3.4 getSize() [2/2]	349
8.25.3.5 getTexture()	349
8.25.3.6 getWidth()	349
8.25.3.7 operator=() [1/2]	350
8.25.3.8 operator=() [2/2]	350
8.25.3.9 swapRandB()	350
8.25.3.10 unbind()	351
8.26 gg::GgTrackball クラス	351
8.26.1 詳解	355
8.26.2 構築子と解体子	355
8.26.2.1 GgTrackball()	355
8.26.2.2 ~GgTrackball()	356

8.26.3 関数詳解	356
8.26.3.1 begin()	356
8.26.3.2 end()	356
8.26.3.3 get()	357
8.26.3.4 getMatrix()	357
8.26.3.5 getQuaternion()	357
8.26.3.6 getScale() [1/3]	358
8.26.3.7 getScale() [2/3]	358
8.26.3.8 getScale() [3/3]	358
8.26.3.9 getStart() [1/3]	358
8.26.3.10 getStart() [2/3]	358
8.26.3.11 getStart() [3/3]	359
8.26.3.12 motion()	359
8.26.3.13 operator=()	359
8.26.3.14 region() [1/2]	360
8.26.3.15 region() [2/2]	360
8.26.3.16 reset()	361
8.26.3.17 rotate()	362
8.27 gg::GgTriangles クラス	362
8.27.1 詳解	364
8.27.2 構築子と解体子	364
8.27.2.1 GgTriangles() [1/2]	364
8.27.2.2 GgTriangles() [2/2]	364
8.27.2.3 ~GgTriangles()	365
8.27.3 関数詳解	365
8.27.3.1 draw()	365
8.27.3.2 getBuffer()	366
8.27.3.3 getCount()	366
8.27.3.4 load()	366
8.27.3.5 send()	367
8.28 gg::GgUniformBuffer< T > クラステンプレート	367
8.28.1 詳解	368
8.28.2 構築子と解体子	368
8.28.2.1 GgUniformBuffer() [1/3]	368
8.28.2.2 GgUniformBuffer() [2/3]	369
8.28.2.3 GgUniformBuffer() [3/3]	369
8.28.2.4 ~GgUniformBuffer()	369
8.28.3 関数詳解	370
8.28.3.1 bind()	370
8.28.3.2 copy()	370
8.28.3.3 fill()	371
8.28.3.4 getBuffer()	372

8.28.3.5	getCount()	373
8.28.3.6	getStride()	373
8.28.3.7	getTarget()	374
8.28.3.8	load() [1/2]	374
8.28.3.9	load() [2/2]	375
8.28.3.10	map() [1/2]	376
8.28.3.11	map() [2/2]	376
8.28.3.12	read()	376
8.28.3.13	send()	377
8.28.3.14	unbind()	378
8.28.3.15	unmap()	378
8.29	gg::GgVector クラス	379
8.29.1	詳解	380
8.29.2	構築子と解体子	380
8.29.2.1	GgVector() [1/6]	380
8.29.2.2	GgVector() [2/6]	381
8.29.2.3	GgVector() [3/6]	382
8.29.2.4	GgVector() [4/6]	382
8.29.2.5	GgVector() [5/6]	383
8.29.2.6	GgVector() [6/6]	383
8.29.2.7	~GgVector()	383
8.29.3	関数詳解	383
8.29.3.1	distance3()	383
8.29.3.2	distance4()	384
8.29.3.3	dot3()	384
8.29.3.4	dot4()	385
8.29.3.5	length3()	386
8.29.3.6	length4()	386
8.29.3.7	normalize3()	386
8.29.3.8	normalize4()	387
8.29.3.9	operator*() [1/2]	387
8.29.3.10	operator*() [2/2]	387
8.29.3.11	operator*=() [1/2]	388
8.29.3.12	operator*=() [2/2]	388
8.29.3.13	operator+() [1/2]	389
8.29.3.14	operator+() [2/2]	389
8.29.3.15	operator+=() [1/2]	390
8.29.3.16	operator+=() [2/2]	390
8.29.3.17	operator-() [1/2]	391
8.29.3.18	operator-() [2/2]	391
8.29.3.19	operator-=() [1/2]	392
8.29.3.20	operator-=() [2/2]	392

8.29.3.21 operator/() [1/2]	393
8.29.3.22 operator/() [2/2]	393
8.29.3.23 operator/=() [1/2]	394
8.29.3.24 operator/=() [2/2]	394
8.29.3.25 operator=() [1/2]	395
8.29.3.26 operator=() [2/2]	395
8.30 gg::GgVertex 構造体	396
8.30.1 詳解	396
8.30.2 構築子と解体子	396
8.30.2.1 GgVertex() [1/4]	396
8.30.2.2 GgVertex() [2/4]	397
8.30.2.3 GgVertex() [3/4]	397
8.30.2.4 GgVertex() [4/4]	397
8.30.3 メンバ詳解	398
8.30.3.1 normal	398
8.30.3.2 position	398
8.31 gg::GgVertexArray クラス	398
8.31.1 詳解	398
8.31.2 構築子と解体子	399
8.31.2.1 GgVertexArray() [1/3]	399
8.31.2.2 GgVertexArray() [2/3]	399
8.31.2.3 GgVertexArray() [3/3]	400
8.31.2.4 ~GgVertexArray()	400
8.31.3 関数詳解	400
8.31.3.1 bind()	400
8.31.3.2 get()	400
8.31.3.3 operator=() [1/2]	401
8.31.3.4 operator=() [2/2]	401
8.32 Intrinsic 構造体	402
8.32.1 詳解	402
8.32.2 構築子と解体子	403
8.32.2.1 Intrinsic() [1/3]	403
8.32.2.2 Intrinsic() [2/3]	403
8.32.2.3 Intrinsic() [3/3]	404
8.32.3 関数詳解	404
8.32.3.1 setCenter()	404
8.32.3.2 setFov() [1/2]	404
8.32.3.3 setFov() [2/2]	405
8.32.3.4 setFps()	405
8.32.3.5 setSize()	406
8.32.4 メンバ詳解	406
8.32.4.1 center	406

8.32.4.2 fov	406
8.32.4.3 fps	406
8.32.4.4 sensorSize	406
8.32.4.5 size	406
8.33 gg::GgSimpleShader::Light 構造体	406
8.33.1 詳解	407
8.33.2 メンバ詳解	407
8.33.2.1 ambient	407
8.33.2.2 diffuse	407
8.33.2.3 position	408
8.33.2.4 specular	408
8.34 gg::GgSimpleShader::LightBuffer クラス	408
8.34.1 詳解	409
8.34.2 構築子と解体子	409
8.34.2.1 LightBuffer() [1/3]	409
8.34.2.2 LightBuffer() [2/3]	410
8.34.2.3 LightBuffer() [3/3]	410
8.34.2.4 ~LightBuffer()	411
8.34.3 関数詳解	411
8.34.3.1 load() [1/2]	411
8.34.3.2 load() [2/2]	412
8.34.3.3 loadAmbient() [1/3]	412
8.34.3.4 loadAmbient() [2/3]	413
8.34.3.5 loadAmbient() [3/3]	414
8.34.3.6 loadColor()	415
8.34.3.7 loadDiffuse() [1/3]	415
8.34.3.8 loadDiffuse() [2/3]	416
8.34.3.9 loadDiffuse() [3/3]	416
8.34.3.10 loadPosition() [1/4]	417
8.34.3.11 loadPosition() [2/4]	418
8.34.3.12 loadPosition() [3/4]	418
8.34.3.13 loadPosition() [4/4]	419
8.34.3.14 loadSpecular() [1/3]	420
8.34.3.15 loadSpecular() [2/3]	421
8.34.3.16 loadSpecular() [3/3]	421
8.34.3.17 select()	422
8.35 gg::GgSimpleShader::Material 構造体	423
8.35.1 詳解	424
8.35.2 メンバ詳解	424
8.35.2.1 ambient	424
8.35.2.2 diffuse	424
8.35.2.3 shininess	425

8.35.2.4 specular	425
8.36 gg::GgSimpleShader::MaterialBuffer クラス	425
8.36.1 詳解	426
8.36.2 構築子と解体子	427
8.36.2.1 MaterialBuffer() [1/3]	427
8.36.2.2 MaterialBuffer() [2/3]	427
8.36.2.3 MaterialBuffer() [3/3]	428
8.36.2.4 ~MaterialBuffer()	428
8.36.3 関数詳解	429
8.36.3.1 load() [1/2]	429
8.36.3.2 load() [2/2]	429
8.36.3.3 loadAmbient() [1/3]	430
8.36.3.4 loadAmbient() [2/3]	430
8.36.3.5 loadAmbient() [3/3]	431
8.36.3.6 loadAmbientAndDiffuse() [1/3]	432
8.36.3.7 loadAmbientAndDiffuse() [2/3]	433
8.36.3.8 loadAmbientAndDiffuse() [3/3]	433
8.36.3.9 loadDiffuse() [1/3]	434
8.36.3.10 loadDiffuse() [2/3]	435
8.36.3.11 loadDiffuse() [3/3]	435
8.36.3.12 loadShininess() [1/2]	436
8.36.3.13 loadShininess() [2/2]	437
8.36.3.14 loadSpecular() [1/3]	438
8.36.3.15 loadSpecular() [2/3]	438
8.36.3.16 loadSpecular() [3/3]	439
8.36.3.17 select()	440
8.37 Menu クラス	440
8.37.1 詳解	441
8.37.2 構築子と解体子	441
8.37.2.1 Menu() [1/2]	441
8.37.2.2 Menu() [2/2]	442
8.37.2.3 ~Menu()	442
8.37.3 関数詳解	442
8.37.3.1 draw()	442
8.37.3.2 getCheckerLength()	443
8.37.3.3 getMarkerLength()	443
8.37.3.4 getMenuBarHeight()	443
8.37.3.5 getPose()	444
8.37.3.6 operator bool()	444
8.37.3.7 operator=()	444
8.37.3.8 saveImage()	445
8.37.3.9 setSize()	445

8.37.3.10 setup()	446
8.37.4 メンバ詳解	446
8.37.4.1 detectBoard	446
8.37.4.2 detectMarker	446
8.38 Mesh クラス	447
8.38.1 詳解	447
8.38.2 構築子と解体子	447
8.38.2.1 Mesh() [1/2]	447
8.38.2.2 Mesh() [2/2]	447
8.38.2.3 ~Mesh()	448
8.38.3 関数詳解	448
8.38.3.1 draw()	448
8.38.3.2 drawMesh()	448
8.38.3.3 operator=()	449
8.39 Preference クラス	449
8.39.1 詳解	449
8.39.2 構築子と解体子	450
8.39.2.1 Preference() [1/3]	450
8.39.2.2 Preference() [2/3]	450
8.39.2.3 Preference() [3/3]	451
8.39.2.4 ~Preference()	451
8.39.3 関数詳解	451
8.39.3.1 buildShader()	451
8.39.3.2 getDescription()	451
8.39.3.3 getIntrinsics()	451
8.39.3.4 getShader()	451
8.39.3.5 setPreference()	452
8.40 Scene クラス	452
8.40.1 詳解	453
8.40.2 構築子と解体子	453
8.40.2.1 Scene()	453
8.40.2.2 ~Scene()	453
8.40.3 関数詳解	454
8.40.3.1 draw()	454
8.40.3.2 set()	454
8.40.4 メンバ詳解	454
8.40.4.1 light	454
8.41 Settings 構造体	454
8.41.1 詳解	455
8.41.2 構築子と解体子	455
8.41.2.1 Settings()	455
8.41.3 関数詳解	455

8.41.3.1 getFocal()	455
8.41.4 メンバ詳解	455
8.41.4.1 checkerLength	455
8.41.4.2 defaultEuler	455
8.41.4.3 defaultFocal	455
8.41.4.4 defaultFocalRange	456
8.41.4.5 dictionaryName	456
8.41.4.6 euler	456
8.41.4.7 focal	456
8.41.4.8 focalRange	456
8.41.4.9 markerLength	456
8.41.4.10 samples	456
8.42 Texture クラス	456
8.42.1 詳解	458
8.42.2 構築子と解体子	458
8.42.2.1 Texture() [1/4]	458
8.42.2.2 Texture() [2/4]	459
8.42.2.3 Texture() [3/4]	460
8.42.2.4 Texture() [4/4]	460
8.42.2.5 ~Texture()	461
8.42.3 関数詳解	461
8.42.3.1 bindTexture()	461
8.42.3.2 copy()	462
8.42.3.3 create()	463
8.42.3.4 discard()	464
8.42.3.5 draw()	465
8.42.3.6 drawPixels() [1/4]	465
8.42.3.7 drawPixels() [2/4]	466
8.42.3.8 drawPixels() [3/4]	467
8.42.3.9 drawPixels() [4/4]	467
8.42.3.10 getChannels()	468
8.42.3.11 getSize()	468
8.42.3.12 getTextureName()	468
8.42.3.13 operator=() [1/2]	469
8.42.3.14 operator=() [2/2]	469
8.42.3.15 readPixels() [1/3]	470
8.42.3.16 readPixels() [2/3]	470
8.42.3.17 readPixels() [3/3]	471
8.42.3.18 unbindTexture()	472
8.42.4 メンバ詳解	472
8.42.4.1 mesh	472
8.43 GgApp::Window クラス	472

8.43.1 詳解	474
8.43.2 構築子と解体子	474
8.43.2.1 Window() [1/3]	474
8.43.2.2 Window() [2/3]	475
8.43.2.3 Window() [3/3]	475
8.43.2.4 ~Window()	476
8.43.3 関数詳解	476
8.43.3.1 get()	476
8.43.3.2 getAltArrowX()	476
8.43.3.3 getAltArrowY()	477
8.43.3.4 getAltArrow()	477
8.43.3.5 getArrow() [1/2]	478
8.43.3.6 getArrow() [2/2]	478
8.43.3.7 getArrowX()	479
8.43.3.8 getArrowY()	480
8.43.3.9 getAspect()	480
8.43.3.10 getControlArrow()	481
8.43.3.11 getControlArrowX()	481
8.43.3.12 getControlArrowY()	482
8.43.3.13 getFboHeight()	482
8.43.3.14 getFboSize() [1/2]	483
8.43.3.15 getFboSize() [2/2]	483
8.43.3.16 getFboWidth()	483
8.43.3.17 getHeight()	484
8.43.3.18 getKey()	484
8.43.3.19 getLastKey()	484
8.43.3.20 getMouse() [1/3]	485
8.43.3.21 getMouse() [2/3]	485
8.43.3.22 getMouse() [3/3]	485
8.43.3.23 getMouseX()	485
8.43.3.24 getMouseY()	485
8.43.3.25 getRotation()	486
8.43.3.26 getRotationMatrix()	486
8.43.3.27 getScrollMatrix()	486
8.43.3.28 getShiftArrow()	486
8.43.3.29 getShiftArrowX()	487
8.43.3.30 getShiftArrowY()	487
8.43.3.31 getSize() [1/2]	488
8.43.3.32 getSize() [2/2]	488
8.43.3.33 getTranslation()	489
8.43.3.34 getTranslationMatrix()	489
8.43.3.35 getUserPointer()	489

8.43.3.36 getWheel() [1/3]	490
8.43.3.37 getWheel() [2/3]	490
8.43.3.38 getWheel() [3/3]	490
8.43.3.39 getWheelX()	490
8.43.3.40 getWheelY()	490
8.43.3.41 getWidth()	491
8.43.3.42 operator bool()	491
8.43.3.43 operator=() [1/2]	491
8.43.3.44 operator=() [2/2]	492
8.43.3.45 reset()	492
8.43.3.46 resetRotation()	493
8.43.3.47 resetTranslation()	493
8.43.3.48 restoreViewport()	493
8.43.3.49 selectInterface()	493
8.43.3.50 setClose()	494
8.43.3.51 setKeyboardFunc()	494
8.43.3.52 setMouseFunc()	494
8.43.3.53 setResizeFunc()	495
8.43.3.54 setUserPointer()	495
8.43.3.55 setVelocity()	495
8.43.3.56 setWheelFunc()	496
8.43.3.57 shouldClose()	496
8.43.3.58 swapBuffers()	496
8.43.3.59 updateViewport()	497
9 ファイル詳解	499
9.1 Buffer.cpp ファイル	499
9.1.1 詳解	499
9.2 Buffer.cpp	499
9.3 Buffer.h ファイル	502
9.3.1 詳解	503
9.3.2 マクロ定義詳解	503
9.3.2.1 USE_PIXEL_BUFFER_OBJECT	503
9.4 Buffer.h	503
9.5 calib.cpp ファイル	505
9.5.1 詳解	506
9.5.2 マクロ定義詳解	506
9.5.2.1 CONFIG_FILE	506
9.6 calib.cpp	506
9.7 Calibration.cpp ファイル	508
9.7.1 詳解	508
9.7.2 マクロ定義詳解	508

9.7.2.1 USE_RODRIGUES	508
9.8 Calibration.cpp	509
9.9 Calibration.h ファイル	514
9.9.1 詳解	515
9.10 Calibration.h	516
9.11 CamCv.h ファイル	517
9.11.1 詳解	518
9.12 CamCv.h	518
9.13 Camera.h ファイル	521
9.13.1 詳解	522
9.14 Camera.h	523
9.15 CamImage.h ファイル	525
9.15.1 詳解	526
9.16 CamImage.h	527
9.17 CamMf.cpp ファイル	528
9.17.1 詳解	529
9.17.2 関数詳解	529
9.17.2.1 HandleStreamChange()	529
9.17.2.2 SafeRelease()	529
9.17.2.3 SubTypeToName()	530
9.18 CamMf.cpp	530
9.19 CamMf.h ファイル	541
9.19.1 詳解	542
9.20 CamMf.h	542
9.21 Capture.cpp ファイル	544
9.21.1 詳解	544
9.22 Capture.cpp	544
9.23 Capture.h ファイル	547
9.23.1 詳解	548
9.24 Capture.h	548
9.25 Compute.h ファイル	549
9.25.1 詳解	549
9.26 Compute.h	550
9.27 Config.cpp ファイル	550
9.27.1 詳解	551
9.28 Config.cpp	551
9.29 Config.h ファイル	554
9.29.1 詳解	555
9.30 Config.h	555
9.31 Expand.cpp ファイル	557
9.32 Expand.cpp	557
9.33 Expand.h ファイル	559

9.33.1 詳解	559
9.34 Expand.h	560
9.35 Framebuffer.cpp ファイル	561
9.35.1 詳解	561
9.36 Framebuffer.cpp	561
9.37 Framebuffer.h ファイル	565
9.37.1 詳解	565
9.38 Framebuffer.h	566
9.39 gg.cpp ファイル	567
9.39.1 詳解	567
9.40 gg.cpp	567
9.41 gg.h ファイル	643
9.41.1 詳解	647
9.41.2 マクロ定義詳解	647
9.41.2.1 ggError	647
9.41.2.2 ggFBOError	648
9.41.3 型定義詳解	648
9.41.3.1 pathChar	648
9.41.3.2 pathString	648
9.41.4 関数詳解	648
9.41.4.1 TCharToUtf8()	648
9.41.4.2 Utf8ToTChar()	648
9.42 gg.h	649
9.43 GgApp.cpp ファイル	703
9.43.1 詳解	703
9.44 GgApp.cpp	704
9.45 GgApp.h ファイル	717
9.45.1 詳解	718
9.45.2 マクロ定義詳解	718
9.45.2.1 GG.BUTTON_COUNT	718
9.45.2.2 GG.INTERFACE_COUNT	718
9.45.2.3 GG.USE_IMGUI	718
9.46 GgApp.h	719
9.47 Intrinsics.cpp ファイル	727
9.47.1 詳解	727
9.48 Intrinsics.cpp	727
9.49 Intrinsics.h ファイル	728
9.49.1 詳解	729
9.50 Intrinsics.h	729
9.51 main.cpp ファイル	730
9.51.1 詳解	731
9.51.2 マクロ定義詳解	731

9.51.2.1 HEADER_STR	731
9.51.3 関数詳解	731
9.51.3.1 main()	731
9.52 main.cpp	732
9.53 Menu.cpp ファイル	733
9.53.1 詳解	734
9.53.2 関数詳解	734
9.53.2.1 getAnyList()	734
9.53.3 変数詳解	735
9.53.3.1 imageFilter	735
9.53.3.2 jsonFilter	735
9.53.3.3 movieFilter	735
9.54 Menu.cpp	735
9.55 Menu.h ファイル	749
9.55.1 詳解	749
9.56 Menu.h	750
9.57 Mesh.h ファイル	752
9.57.1 詳解	753
9.58 Mesh.h	753
9.59 opencv_link.h ファイル	754
9.59.1 詳解	755
9.60 opencv_link.h	755
9.61 parseconfig.h ファイル	756
9.61.1 詳解	757
9.61.2 関数詳解	757
9.61.2.1 getString() [1/3]	757
9.61.2.2 getString() [2/3]	758
9.61.2.3 getString() [3/3]	758
9.61.2.4 getValue() [1/2]	758
9.61.2.5 getValue() [2/2]	759
9.61.2.6 setString() [1/3]	760
9.61.2.7 setString() [2/3]	760
9.61.2.8 setString() [3/3]	760
9.61.2.9 setValue() [1/2]	761
9.61.2.10 setValue() [2/2]	761
9.62 parseconfig.h	762
9.63 Preference.cpp ファイル	764
9.63.1 詳解	765
9.64 Preference.cpp	765
9.65 Preference.h ファイル	766
9.65.1 詳解	767
9.66 Preference.h	767

9.67 REQUESTS.md ファイル	768
9.68 Scene.cpp ファイル	768
9.68.1 詳解	768
9.69 Scene.cpp	769
9.70 Scene.h ファイル	770
9.70.1 詳解	770
9.71 Scene.h	770
9.72 Texture.cpp ファイル	771
9.72.1 詳解	771
9.73 Texture.cpp	772
9.74 Texture.h ファイル	775
9.74.1 詳解	776
9.75 Texture.h	776
Index	779

Chapter 1

ゲームグラフィックス特論の宿題用補助 プログラム **GLFW3** 版.

著作権所有

Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies or substantial portions of the Software.

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 2

REQUESTS.md - アプリケーションビルドおよび開発構成定義書

このプログラムは、ChArUco Board を使ったカメラキャリブレーションアプリケーションである。

2.1 1. ソースコード

- C++ のバージョンは C++17、OpenGL のバージョンは 4.1 とする。
- C++ のソースファイル (.cpp .h) の文字コードは BOM 付き utf-8 とする。
- GLSL のシェーダのソースファイル (.vert .frag .geom .comp) の文字コードは BOM 無し utf-8 とする。
- 改行コードは CRLF とする。
- Windows の場合は、Visual Studio のソリューションエクスプローラーに、次のようにしてファイル名を表示する。
 - C++ のヘッダファイル (.h) は Header Files というフィルタに表示する。
 - GLSL のシェーダのソースファイル (.vert .frag .geom .comp) は Shader Files というフィルタに表示する。
 - 画像ファイルなど、その他のファイルは Resource Files というフィルタに表示する。
- macOS の場合は Xcode のナビゲーターエリアに、次のようにしてファイル名を表示する。
 - C++ のヘッダファイル (.h) は Header Files というグループ内に表示する。
 - GLSL のシェーダのソースファイル (.vert .frag .geom .comp) は Shader Files というグループ内に表示する。
 - 画像ファイルなど、その他のファイルは Resource Files というグループ内に表示する。

2.2 2. CMakeLists.txt の設定

- cmake を使って、このプロジェクトを Windows、macOS、Ubuntu Linux の環境でビルドできるようにする。
 - Windows の場合は、Visual Studio のソリューションファイルを作成する。
 - * ソリューションファイル名と同じプロジェクトを、単一のスタートアッププロジェクトに設定する。
 - macOS の場合は、Xcode のプロジェクトファイルを作成する。
 - Linux の場合は、Makefile を作成する。

2.2.1 2-2. 外部ライブラリの配置場所

- 外部ライブラリは `cmake` 時にダウンロードして、ソースのディレクトリの `libs` ディレクトリ以下に展開して配置する。
- ダウンロードには、ソースプログラムの配布先が `Python` を使用していないことを想定して、`Python` を使用せず、プラットフォームごとの標準的な手法を用いる。
- `Visual Studio` でこのディレクトリ以下のパスを指定する場合は、`$(SolutionDir)/libs` で始まるパスを使用する。

2.2.2 2-3. OpenGL 関連のヘッダファイル

- `Ubuntu Linux` の場合は `OpenGL` の開発パッケージ `libgl-dev` を用いる。
- `Windows` と `macOS` の場合は、以下の手順でダウンロードする。
 - `libs` の下に `include` というディレクトリを作る
 - `include` 中に `GL` および `KHR` というディレクトリを作る
 - `GL` の中に <https://registry.khronos.org/OpenGL/api/GL/> から必要なものをダウンロードして配置する。
 - `KHR` の中に <https://registry.khronos.org/EGL/api/KHR/> から必要なものをダウンロードして配置する。

2.2.3 2-4. GLFW

- フレームワークとして `GLFW` を用いる。
- `Ubuntu Linux` の場合は、`GLFW` バージョン 3 の開発パッケージ `libglfw3-dev` を用いる。
- `Windows` の場合は、最新バージョン (現時点で 3.4) のリリース版の `Windows 64bit` 用のバイナリファイル <https://github.com/glfw/glfw/releases/download/3.4/glfw-3.4.bin.WIN64.zip> をダウンロードして `libs` 以下に展開し、`Visual Studio` の最新のものに対応したスタティックライブラリファイル (現時点で `lib-vc2022`) をリンクする。
- `macOS` の場合は、最新バージョン (現時点で 3.4) のリリース版のバイナリファイル <https://github.com/glfw/glfw/releases/download/3.4/glfw-3.4.bin.MACOS.zip> をダウンロードして `libs` 以下に展開し、ビルドするシステムに対応したスタティックライブラリファイルをリンクする。
- `macOS` の場合は、`OpenGL.framework` のほか、`Cocoa.framework` など `OpenGL` を使ったプログラムの実行に必要なフレームワークをリンクする。

2.2.4 2-5. Dear ImGui

- GUI には `Dear ImGui` を用いる。
 - 最新リリース (現時点で `v1.92.8`) のソースファイル <https://github.com/ocornut/imgui/archive/refs/tags/v1.92.8.zip> をダウンロードし、`libs` 以下の `ImGui` というディレクトリに展開する。ただし、`OpenGL` バージョン 3 以降と `GLFW` の組み合わせに対応したものだけを配置すればよい。
 - 展開したファイルをプログラム本体と一緒にコンパイル・リンクする。
 - `Windows` の場合は、`Visual Studio` のソリューションエクスプローラーの `ImGui` というフィルタの中に `ImGui` のソースファイル名 (`.cpp`) を表示する。
 - `macOS` の場合は、`Xcode` のナビゲーターエリアの `ImGui` というグループ内に `ImGui` のソースファイル名 (`.cpp.m`) を表示する。

2.2.5 2-6. Native File Dialog Extended

- ファイルダイアログには Native File Dialog Extended を用いる
 - Windows と macOS の場合は、最新リリース (現時点では v1.3.0) のソースファイル <https://github.com/btzy/nativefiledialog-extended/archive/refs/tags/v1.3.0.zip> をダウンロードして、プラットフォームに合わせたソースファイルを ImGui と同じ libs ディレクトリ内の ImGui ディレクトリに配置する。
 - Windows の場合は、Visual Studio のソリューションエクスプローラーの ImGui というフィルタの中に、ImGui のソースファイル名と一緒に Native File Dialog Extended のソースファイル名を表示する。
 - macOS の場合は、Visual Studio のソリューションエクスプローラーの ImGui というグループ内に、ImGui のソースファイル名と一緒に Native File Dialog Extended のソースファイル名を表示する。
 - Ubuntu Linux の場合は、GTK+3 も必要になるので、開発パッケージ libgtk-3-dev および関連のパッケージを用いる。

2.2.6 2-7. picojson

- JSON ファイルの解析には picojson を用いる。
 - picojson の最新リリース (現時点では 1.3.0) のソースファイル <https://github.com/kazuho/picojson/archive/refs/tags/v1.3.0.zip> をダウンロードして、picojson.h を libs ディレクトリの中の include ディレクトリに配置する。

2.2.7 2-8. OpenCV

- OpenCV はバージョン 4 の最新リリース (現時点では 4.13.0) を使用する。
- Ubuntu Linux の場合は、OpenCV バージョン 4 の開発パッケージ libopencv-dev を用いる。
- Windows の場合は、リリース版の Windows 用バイナリインストーラ <https://github.com/opencv/opencv/releases/download/4.13.0/opencv-4.13.0-windows.exe> をダウンロードして libs 以下に展開してリンクする。
- macOS の場合は、ソースファイル <https://github.com/opencv/opencv/archive/refs/tags/4.13.0.zip> をダウンロードし、libs フォルダ内に展開してビルドし、作成されたライブラリファイルをこのプログラムにリンクする。
 - cmake のバージョンの違いによるエラーを回避するようにする。

2.2.8 2-9. 実行ファイル

- Windows および Ubuntu Linux では、実行ファイルと同じディレクトリに DLL や GLSL のソースファイル、フォントファイルなど、実行に必要なファイルをコピーする。
- Windows では、Visual Studio 内で実行 (デバッグ) できるように、実行時の作業ディレクトリを実行ファイルを置いたところに移すようにする。
- macOS では、アプリケーションバンドル (.app) を作成し、その中に実行に必要なファイルをコピーする。またこれを、Xcode 内から実行できるようにする。

2.3 3. README.md, GEMINI.md, .gitignore

- README.md を作成すること。
 - プログラムの構造を解説すること。
 - Mermaidによる構成図を添付すること。
 - カメラオンライン/画像オフラインでのキャリブレーション詳細手順を説明すること。

- プラットフォーム別のビルド手順を説明すること。
- GEMINI.md を作成すること。
 - アプリケーションのビルド・開発構成の定義を記録すること。
 - 各プラットフォームの依存ライブラリ構成を記録すること。
 - CMakeLists.txt の設計方針を記録すること。
- .gitignore を整備すること。
 - リポジトリに含める必要のないファイルやディレクトリを追加すること。
 - バイナリディレクトリ build に加え libs も Git 管理除外とすること。

2.4 4. ビデオキャプチャデバイスの設計意図

2.4.1 4-1. UVC カメラ初期化におけるデコード初期化の分離

- H.264 コーデックをサポートするカメラでのフリーズや起動失敗を避けるため、フォーマットリストの取得時およびキャプチャ「開始」ボタンを押す前のデバイスオープン時には、カメラフォーマットの適用（デコードのロードを含む setFormat）を一切行わないこと。
- フォーマットの適用は、ユーザーが画面上で形式を選択し、キャプチャを明示的に開始する瞬間にのみ 1 回だけ行われるように設計し、無駄なハードウェアデコードの初期化ロード（特に H.264 ハードウェア MFT の初期化）による物理遅延やエラー発生を防ぐこと。

2.4.2 4-2. 手動デコードによる CPU 高速処理方式（AVP自動変換の廃止）

- RICOH THETA V や Logi C615 など、H.264 や MJPG などの圧縮フォーマットを出力する Web カメラにおいて、OpenCV の AVP (Advanced Video Processing) 自動変換による物理遅延や列挙フリーズを回避するため、自前で IMFTransform デコード・カラーコンバータを構築して RGB32 にデコードする処理を移植し、OpenCV に依存しない Pure Media Foundation キャプチャ化を達成すること。

2.4.3 4-3. 安全なフォーマットリスト動的取得設計（生ポインタ管理の廃止）

- `Capture` クラス内でフォーマットリストの参照先を切り替える生ポインタ管理を廃止すること。カメラオブジェクトの有効状態を判定して安全にリストを返す `getFormatList()` を実装し、フォーマットリストが UI 上で空のまま更新されなくなる不具合を解消すること。

2.5 5. UI（メニュー表示）の改善要請

2.5.1 5-1. ビデオフォーマット選択ドロップダウンの3分割表示と最適化

- ビデオフォーマットの表示名（解像度 @ フレームレート fps (コーデック)）を、「解像度」「コマ数（フレームレート）」「符号化（コーデック）」の3つのドロップダウンリストに分割して表示すること。
- ドロップダウンリストの各項目（ユニークな解像度・フレームレート・コーデック）は毎フレーム再生成するのではなく、キャプチャデバイスの切り替え時および初期オープン時のみ作成するようにして、レンダリング負荷を最小限に抑えること。
- 存在しない組み合わせ（例えば、ある解像度で指定したフレームレートやコーデックがサポートされていないとき）が選択された場合は、「フォーマットが存在しません」という警告を表示し、キャプチャ「開始」ボタンを非表示にすること。

2.6 6. GPUゼロコピー化の断念とPBOの維持

2.6.1 6-1. OpenCVによる画像処理設計との整合性

- WGL DX Interop を用いた GPU ゼロコピー化（Direct3D11テクスチャからOpenGLテクスチャへの直接マッピング）は、画素データをGPU上に留めるため、OpenCVで行うChArUcoボードのコーナー検出や座標計算などのCPU上の画像処理設計と競合する。
- したがって、GPUゼロコピー化の実装は行わず、従来のPBO（Pixel Buffer Object）を用いた最速のCPU -> GPU画像転送機構を維持すること。

Chapter 3

名前空間索引

3.1 名前空間一覧

全名前空間の一覧です。

[gg](#) [17](#)

Chapter 4

階層索引

4.1 クラス階層

クラス階層一覧です。大雑把に文字符号順で並べられています。

std::array	
gg::GgMatrix	181
gg::GgVector	379
gg::GgQuaternion	252
gg::GgTrackball	351
Buffer	79
Texture	456
Framebuffer	147
Calibration	96
Camera	113
CamCv	106
CamImage	122
CamMf	127
Capture	132
Compute	137
Config	138
Expand	144
GgApp	159
gg::GgBuffer< T >	164
gg::GgColorTexture	172
gg::GgNormalTexture	233
gg::GgPointShader	241
gg::GgSimpleShader	325
gg::GgShader	315
gg::GgShape	320
gg::GgPoints	236
gg::GgTriangles	362
gg::GgElements	176
gg::GgSimpleObj	323
gg::GgTexture	345
gg::GgUniformBuffer< T >	367
gg::GgUniformBuffer< Light >	367
gg::GgSimpleShader::LightBuffer	408
gg::GgUniformBuffer< Material >	367
gg::GgSimpleShader::MaterialBuffer	425
gg::GgVertex	396
gg::GgVertexArray	398
Intrinsics	402

gg::GgSimpleShader::Light	406
gg::GgSimpleShader::Material	423
Menu	440
Mesh	447
Preference	449
Scene	452
Settings	454
GgApp::Window	472

Chapter 5

クラス索引

5.1 クラス一覧

クラス・構造体・共用体・インターフェースの一覧です。

Buffer	79
Calibration	96
CamCv	106
Camera	113
CamImage	122
CamMf	127
Capture	132
Compute	137
Config	138
Expand	144
Framebuffer	147
GgApp	159
gg::GgBuffer< T >	164
gg::GgColorTexture	172
gg::GgElements	176
gg::GgMatrix	181
gg::GgNormalTexture	233
gg::GgPoints	236
gg::GgPointShader	241
gg::GgQuaternion	252
gg::GgShader	315
gg::GgShape	320
gg::GgSimpleObj	323
gg::GgSimpleShader	325
gg::GgTexture	345
gg::GgTrackball	351
gg::GgTriangles	362
gg::GgUniformBuffer< T >	367
gg::GgVector	379
gg::GgVertex	396
gg::GgVertexArray	398
Intrinsics	402
gg::GgSimpleShader::Light	406
gg::GgSimpleShader::LightBuffer	408
gg::GgSimpleShader::Material	423
gg::GgSimpleShader::MaterialBuffer	425
Menu	440
Mesh	447
Preference	449
Scene	452

Settings	454
Texture	456
GgApp::Window	472

Chapter 6

ファイル索引

6.1 ファイル一覧

ファイル一覧です。

Buffer.cpp	499
Buffer.h	502
calib.cpp	505
Calibration.cpp	508
Calibration.h	514
CamCv.h	517
Camera.h	521
CamImage.h	525
CamMf.cpp	528
CamMf.h	541
Capture.cpp	544
Capture.h	547
Compute.h	549
Config.cpp	550
Config.h	554
Expand.cpp	557
Expand.h	559
Framebuffer.cpp	561
Framebuffer.h	565
gg.cpp	567
gg.h	643
GgApp.cpp	703
GgApp.h	717
Intrinsics.cpp	727
Intrinsics.h	728
main.cpp	730
Menu.cpp	733
Menu.h	749
Mesh.h	752
opencv_link.h	
MSVC環境用のOpenCV自動リンクおよびインクルード一括管理ヘッダー	754
parseconfig.h	756
Preference.cpp	764
Preference.h	766
Scene.cpp	768
Scene.h	770
Texture.cpp	771
Texture.h	775

Chapter 7

名前空間詳解

7.1 gg 名前空間

クラス

- class [GgVector](#)
- class [GgMatrix](#)
- class [GgQuaternion](#)
- class [GgTrackball](#)
- class [GgTexture](#)
- class [GgColorTexture](#)
- class [GgNormalTexture](#)
- class [GgBuffer](#)
- class [GgUniformBuffer](#)
- class [GgVertexArray](#)
- class [GgShape](#)
- class [GgPoints](#)
- struct [GgVertex](#)
- class [GgTriangles](#)
- class [GgElements](#)
- class [GgShader](#)
- class [GgPointShader](#)
- class [GgSimpleShader](#)
- class [GgSimpleObj](#)

列挙型

- enum [BindingPoints](#) { [LightBindingPoint](#) = 0 , [MaterialBindingPoint](#) }

関数

- void [ggInit](#) ()
- void [_ggError](#) (const std::string &name="", unsigned int line=0)
- void [_ggFBOError](#) (const std::string &name="", unsigned int line=0)
- void [ggCross](#) (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat [ggDot3](#) (const GLfloat *a, const GLfloat *b)
- GLfloat [ggLength3](#) (const GLfloat *a)
- GLfloat [ggDistance3](#) (const GLfloat *a, const GLfloat *b)
- void [ggNormalize3](#) (const GLfloat *a, GLfloat *b)
- void [ggNormalize3](#) (GLfloat *a)
- GLfloat [ggDot4](#) (const GLfloat *a, const GLfloat *b)
- GLfloat [ggLength4](#) (const GLfloat *a)
- GLfloat [ggDistance4](#) (const GLfloat *a, const GLfloat *b)

- void `ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `ggNormalize4` (GLfloat *a)
- const `GgVector` & `operator+` (const `GgVector` &v)
- `GgVector` `operator+` (GLfloat a, const `GgVector` &b)
- const `GgVector` `operator-` (const `GgVector` &v)
- `GgVector` `operator-` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator*` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator/` (GLfloat a, const `GgVector` &b)
- `GgVector` `ggCross` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggDot3` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength3` (const `GgVector` &a)
- GLfloat `ggDistance3` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize3` (const `GgVector` &a)
- void `ggNormalize3` (`GgVector` *a)
- GLfloat `ggDot4` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength4` (const `GgVector` &a)
- GLfloat `ggDistance4` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize4` (const `GgVector` &a)
- void `ggNormalize4` (`GgVector` *a)
- `GgMatrix` `ggIdentity` ()
- `GgMatrix` `ggTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggTranslate` (const GLfloat *t)
- `GgMatrix` `ggTranslate` (const `GgVector` &t)
- `GgMatrix` `ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggScale` (const GLfloat *s)
- `GgMatrix` `ggScale` (const `GgVector` &s)
- `GgMatrix` `ggRotateX` (GLfloat a)
- `GgMatrix` `ggRotateY` (GLfloat a)
- `GgMatrix` `ggRotateZ` (GLfloat a)
- `GgMatrix` `ggRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix` `ggRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r)
- `GgMatrix` `ggRotate` (const `GgVector` &r)
- `GgMatrix` `ggLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix` `ggLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix` `ggLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix` `ggOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggTranspose` (const `GgMatrix` &m)
- `GgMatrix` `ggInvert` (const `GgMatrix` &m)
- `GgMatrix` `ggNormal` (const `GgMatrix` &m)
- `GgQuaternion` `ggIdentityQuaternion` ()
- `GgQuaternion` `ggMatrixQuaternion` (const GLfloat *a)
- `GgQuaternion` `ggMatrixQuaternion` (const `GgMatrix` &m)
- `GgMatrix` `ggQuaternionMatrix` (const `GgQuaternion` &q)
- `GgMatrix` `ggQuaternionTransposeMatrix` (const `GgQuaternion` &q)
- `GgQuaternion` `ggRotateQuaternion` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgQuaternion` `ggRotateQuaternion` (const GLfloat *v, GLfloat a)
- `GgQuaternion` `ggRotateQuaternion` (const GLfloat *v)
- `GgQuaternion` `ggEulerQuaternion` (GLfloat heading, GLfloat pitch, GLfloat roll)
- `GgQuaternion` `ggEulerQuaternion` (const GLfloat *e)
- `GgQuaternion` `ggEulerQuaternion` (const `GgVector` &e)

- [GgQuaternion ggSlerp](#) (const GLfloat *a, const GLfloat *b, GLfloat t)
- [GgQuaternion ggSlerp](#) (const [GgQuaternion](#) &q, const [GgQuaternion](#) &r, GLfloat t)
- [GgQuaternion ggSlerp](#) (const [GgQuaternion](#) &q, const GLfloat *a, GLfloat t)
- [GgQuaternion ggSlerp](#) (const GLfloat *a, const [GgQuaternion](#) &q, GLfloat t)
- GLfloat [ggNorm](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggNormalize](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggConjugate](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggInvert](#) (const [GgQuaternion](#) &q)
- bool [ggSaveTga](#) (const std::string &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- bool [ggSaveColor](#) (const std::string &name)
- bool [ggSaveDepth](#) (const std::string &name)
- bool [ggReadImage](#) (const std::string &name, std::vector< GLubyte > &image, GLsizei *pWidth, GLsizei *pHeight, GLenum *pFormat)
- GLuint [ggLoadTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- GLuint [ggLoadImage](#) (const std::string &name, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE)
- void [ggCreateNormalMap](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format, GLfloat nz, GLenum internal, std::vector< [GgVector](#) > &nmap)
- GLuint [ggLoadHeight](#) (const std::string &name, GLfloat nz, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA)
- GLuint [ggCreateShader](#) (const std::string &vsr, const std::string &fsrc="", const std::string &gsr="", GLint nvarying=0, const char *const *varyings=nullptr, const std::string &vtext="vertex shader", const std::string &ftext="fragment shader", const std::string >ext="geometry shader")
- GLuint [ggLoadShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [ggLoadShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [ggCreateComputeShader](#) (const std::string &csr, const std::string &ctext="compute shader")
- GLuint [ggLoadComputeShader](#) (const std::string &comp)
- std::shared_ptr< [GgPoints](#) > [ggPointsCube](#) (GLsizei countv, GLfloat length=1.0f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgPoints](#) > [ggPointsSphere](#) (GLsizei countv, GLfloat radius=0.5f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgTriangles](#) > [ggRectangle](#) (GLfloat width=1.0f, GLfloat height=1.0f)
- std::shared_ptr< [GgTriangles](#) > [ggEllipse](#) (GLfloat width=1.0f, GLfloat height=1.0f, GLuint slices=16)
- std::shared_ptr< [GgTriangles](#) > [ggArraysObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [ggElementsObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [ggElementsMesh](#) (GLuint slices, GLuint stacks, const GLfloat(*pos)[3], const GLfloat(*norm)[3]=nullptr)
- std::shared_ptr< [GgElements](#) > [ggElementsSphere](#) (GLfloat radius=1.0f, int slices=16, int stacks=8)
- bool [ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, bool normalize=false)
- bool [ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint [ggBufferAlignment](#)
使用している GPU のバッファアライメント。

7.1.1 詳解

ゲームグラフィックス特論の宿題用補助プログラムの名前空間

7.1.2 列挙型詳解

7.1.2.1 BindingPoints

enum `gg::BindingPoints`

光源と材質の uniform buffer object の結合ポイント.

列挙値

<code>LightBindingPoint</code>	光源の uniform buffer object の結合ポイント.
<code>MaterialBindingPoint</code>	材質の uniform buffer object の結合ポイント.

`gg.h` の 1349 行目に定義があります.

7.1.3 関数詳解

7.1.3.1 _ggError()

```
void gg::_ggError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

OpenGL のエラーをチェックする.

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, <code>nullptr</code> なら何も出力しない.
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない.

覚え書き

OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する.

`gg.cpp` の 2615 行目に定義があります.

7.1.3.2 _ggFBOError()

```
void gg::_ggFBOError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

FBO のエラーをチェックする.

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, <code>nullptr</code> なら何も出力しない.
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない.

覚え書き

FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する.

`gg.cpp` の 2659 行目に定義があります.

7.1.3.3 ggArraysObj()

```
std::shared_ptr< gg::GgTriangles > gg::ggArraysObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイルを読み込む (Arrays 形式)

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

[GgTriangles](#) 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイルを読み込んで GgArrays 形式の三角形データを生成する.

[gg.cpp](#) の 5281 行目に定義があります。

呼び出し関係図:



7.1.3.4 ggConjugate()

```
GgQuaternion gg::ggConjugate (
    const GgQuaternion & q) [inline]
```

共役四元数を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

四元数 *q* の共役四元数.

[gg.h](#) の 4756 行目に定義があります。

呼び出し関係図:



7.1.3.5 ggCreateComputeShader()

```
GLuint gg::ggCreateComputeShader (
    const std::string & csrc,
    const std::string & ctext = "compute shader") [extern]
```

コンピュートシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する。

引数

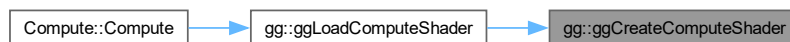
<i>csrc</i>	コンピュートシェーダのソースプログラムの文字列.
<i>ctext</i>	コンピュートシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 5054 行目に定義があります。

被呼び出し関係図:



7.1.3.6 ggCreateNormalMap()

```
void gg::ggCreateNormalMap (
    const GLubyte * hmap,
    GLsizei width,
    GLsizei height,
    GLenum format,
    GLfloat nz,
    GLenum internal,
    std::vector< GgVector > & nmap) [extern]
```

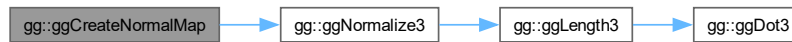
グレースケール画像 (8bit) から法線マップのデータを作成する。

引数

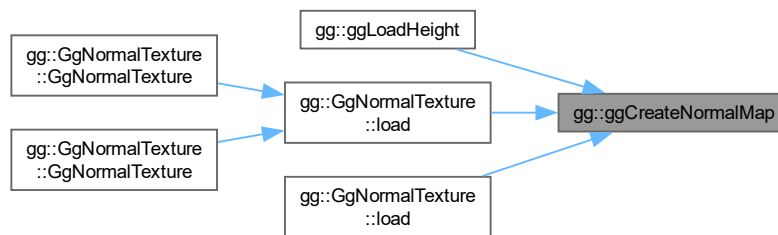
<i>hmap</i>	グレースケール画像のデータ.
<i>width</i>	高さマップのグレースケール画像 <i>hmap</i> の横の画素数.
<i>height</i>	高さマップのグレースケール画像 <i>hmap</i> の縦の画素数.
<i>format</i>	データの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線の z 成分の割合.
<i>internal</i>	法線マップを格納するテクスチャの内部フォーマット.
<i>nmap</i>	法線マップを格納する vector.

[gg.cpp](#) の 3881 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.7 ggCreateShader()

```

GLuint gg::ggCreateShader (
    const std::string & vsrc,
    const std::string & fsrc = "",
    const std::string & gsrc = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr,
    const std::string & vtext = "vertex shader",
    const std::string & ftext = "fragment shader",
    const std::string & gtext = "geometry shader") [extern]
  
```

シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する。

引数

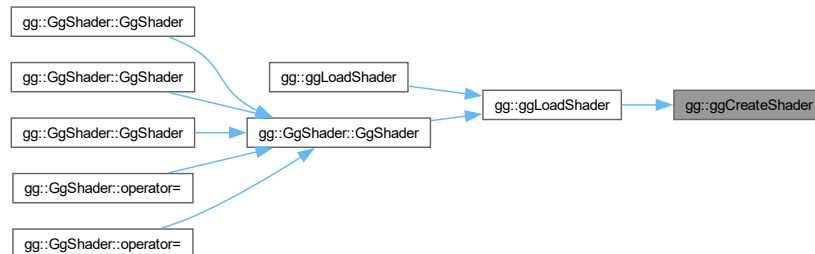
<i>vsrc</i>	バーテックスシェーダのソースプログラムの文字列.
<i>fsrc</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>gsrc</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).
<i>vtext</i>	バーテックスシェーダのコンパイル時のメッセージに追加する文字列.
<i>ftext</i>	フラグメントシェーダのコンパイル時のメッセージに追加する文字列.
<i>gtext</i>	ジオメトリシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 4878 行目に定義があります。

被呼び出し関係図:



7.1.3.8 ggCross() [1/2]

```
GgVector gg::ggCross (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の外積.

引数

a	GgVector 型のベクトル.
b	GgVector 型のベクトル.

戻り値

a と **b** の外積.

覚え書き

戻り値の **w** (第4) 要素は 0.

[gg.h](#) の 1994 行目に定義があります。

呼び出し関係図:



7.1.3.9 ggCross() [2/2]

```
void gg::ggCross (
    GLfloat * c,
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の外積.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.
<i>c</i>	結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1429 行目に定義があります。

被呼び出し関係図:



7.1.3.10 ggDistance3() [1/2]

```
GLfloat gg::ggDistance3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の距離.

引数

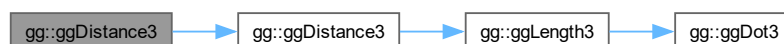
<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と b の距離.

gg.h の 2031 行目に定義があります。

呼び出し関係図:



7.1.3.11 ggDistance3() [2/2]

```
GLfloat gg::ggDistance3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の距離.

引数

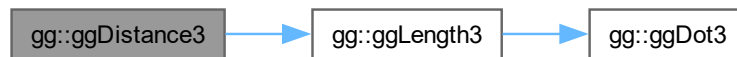
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.

戻り値

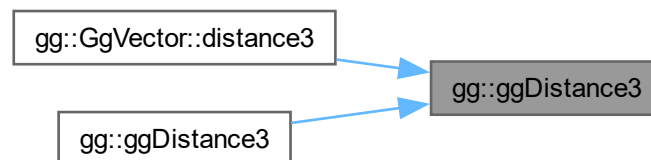
a と b の距離.

[gg.h](#) の 1466 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.12 ggDistance4() [1/2]

```
GLfloat gg::ggDistance4 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 4 要素の距離.

引数

<i>a</i>	GgVector 型の変数.
<i>b</i>	GgVector 型の変数.

戻り値

2 つの [GgVector](#) `a`, `b` の距離.

[gg.h](#) の 2095 行目に定義があります。

呼び出し関係図:



7.1.3.13 ggDistance4() [2/2]

```

GLfloat gg::ggDistance4 (
    const GLfloat * a,
    const GLfloat * b) [inline]
  
```

GLfloat 型の 4 要素の距離.

引数

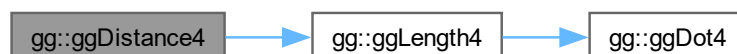
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	GLfloat 型の 4 要素の配列変数.

戻り値

`a` と `b` のそれぞれの 4 要素の距離.

[gg.h](#) の 1531 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.14 ggDot3() [1/2]

```
GLfloat gg::ggDot3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の内積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* のそれぞれの 3 要素の内積.

gg.h の 2008 行目に定義があります。

呼び出し関係図:



7.1.3.15 ggDot3() [2/2]

```
GLfloat gg::ggDot3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の内積.

引数

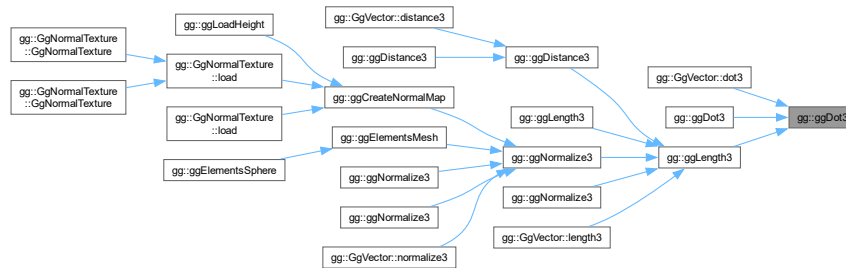
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.

戻り値

a と b のそれぞれの 3 要素の内積.

gg.h の 1443 行目に定義があります。

被呼び出し関係図:



7.1.3.16 ggDot4() [1/2]

```
GLfloat gg::ggDot4 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 4 要素の内積.

引数

a	GgVector 型の変数.
b	GgVector 型の変数.

戻り値

a と b のそれぞれの 4 要素の内積.

gg.h の 2072 行目に定義があります。

呼び出し関係図:



7.1.3.17 ggDot4() [2/2]

```
GLfloat gg::ggDot4 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

GLfloat 型の 4 要素の内積

引数

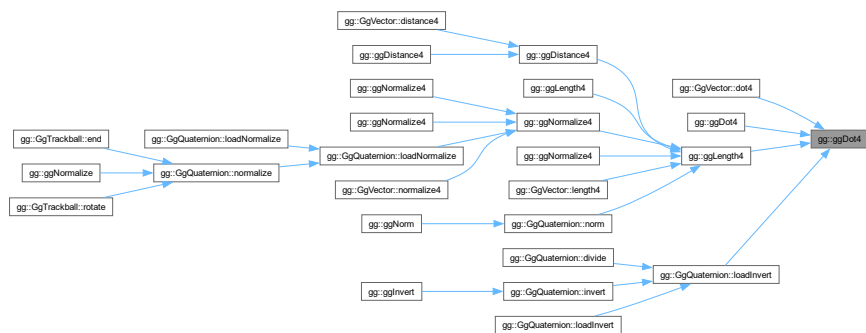
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	GLfloat 型の 4 要素の配列変数.

戻り値

a と *b* のそれぞれの 4 要素の内積.

[gg.h](#) の 1508 行目に定義があります。

被呼び出し関係図:



7.1.3.18 ggElementsMesh()

```
std::shared_ptr< gg::GgElements > gg::ggElementsMesh (
    GLuint slices,
    GLuint stacks,
    const GLfloat(*) pos[3],
    const GLfloat(*) norm[3] = nullptr) [extern]
```

メッシュ形状を作成する (Elements 形式).

引数

<i>slices</i>	メッシュの横方向の分割数.
<i>stacks</i>	メッシュの縦方向の分割数.
<i>pos</i>	メッシュの頂点の位置.
<i>norm</i>	メッシュの頂点の法線, nullptr なら頂点の位置から算出する.

戻り値

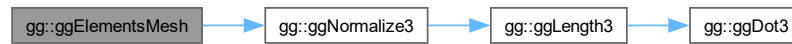
[GgElements](#) 型のポインタ.

覚え書き

メッシュ状に **GgElements** 形式の三角形データを生成する.

gg.cpp の 5315 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



7.1.3.19 ggElementsObj()

```
std::shared_ptr< gg::GgElements > gg::ggElementsObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイル を読み込む (Elements 形式).

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

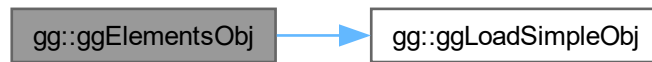
戻り値

GgElements 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイル を読み込んで **GgElements** 形式の三角形データを生成する。

gg.cpp の 5297 行目に定義があります。
呼び出し関係図:



7.1.3.20 ggElementsSphere()

```
std::shared_ptr< gg::GgElements > gg::ggElementsSphere (
    GLfloat radius = 1.0f,
    int slices = 16,
    int stacks = 8) [extern]
```

球状に三角形データを生成する (Elements 形式)。

引数

<i>radius</i>	球の半径.
<i>slices</i>	球の経度方向の分割数.
<i>stacks</i>	球の緯度方向の分割数.

戻り値

GgElements 型のポインタ.

覚え書き

球状に **GgElements** 形式の三角形データを生成する。

gg.cpp の 5410 行目に定義があります。
呼び出し関係図:



7.1.3.21 ggEllipse()

```
std::shared_ptr< gg::GgTriangles > gg::ggEllipse (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f,
    GLuint slices = 16) [extern]
```

楕円状に三角形を生成する.

引数

<i>width</i>	楕円の横幅.
<i>height</i>	楕円の高さ.
<i>slices</i>	楕円の分割数.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5256 行目に定義があります.

7.1.3.22 ggEulerQuaternion() [1/3]

```
GgQuaternion gg::ggEulerQuaternion (
    const GgVector & e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す.

引数

<i>e</i>	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
----------	--

戻り値

回転を表す四元数.

[gg.h](#) の 4670 行目に定義があります.

呼び出し関係図:



7.1.3.23 ggEulerQuaternion() [2/3]

```
GgQuaternion gg::ggEulerQuaternion (
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す.

引数

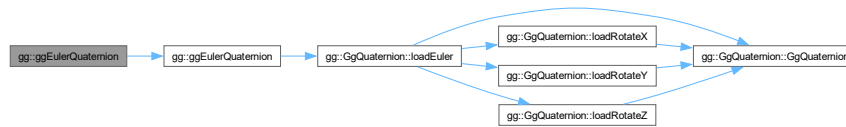
<i>e</i>	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
----------	---

戻り値

回転を表す四元数.

[gg.h](#) の 4659 行目に定義があります。

呼び出し関係図:



7.1.3.24 ggEulerQuaternion() [3/3]

```
GgQuaternion gg::ggEulerQuaternion (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) [inline]
```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を返す.

引数

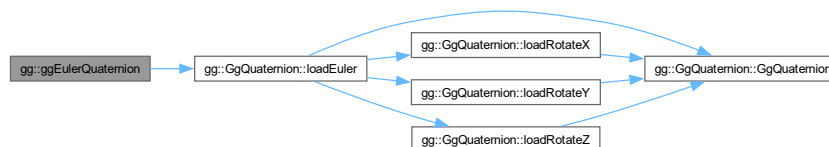
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

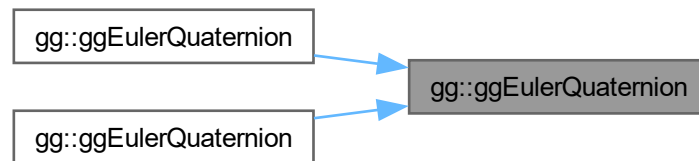
回転を表す四元数.

[gg.h](#) の 4647 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.25 ggFrustum()

```
GgMatrix gg::ggFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

透視透視投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

gg.h の 3388 行目に定義があります。

呼び出し関係図:



7.1.3.26 ggIdentity()

`GgMatrix gg::ggIdentity () [inline]`

単位行列を返す.

戻り値

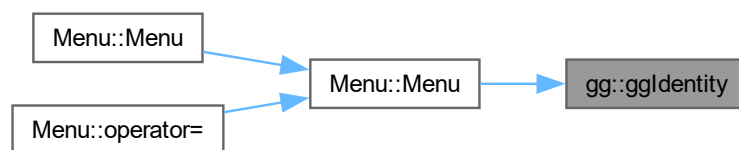
単位行列.

`gg.h` の 3113 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.27 ggIdentityQuaternion()

`GgQuaternion gg::ggIdentityQuaternion () [inline]`

単位四元数を返す.

戻り値

単位四元数.

`gg.h` の 4545 行目に定義があります。

呼び出し関係図:



7.1.3.28 ggInit()

`void gg::ggInit () [extern]`

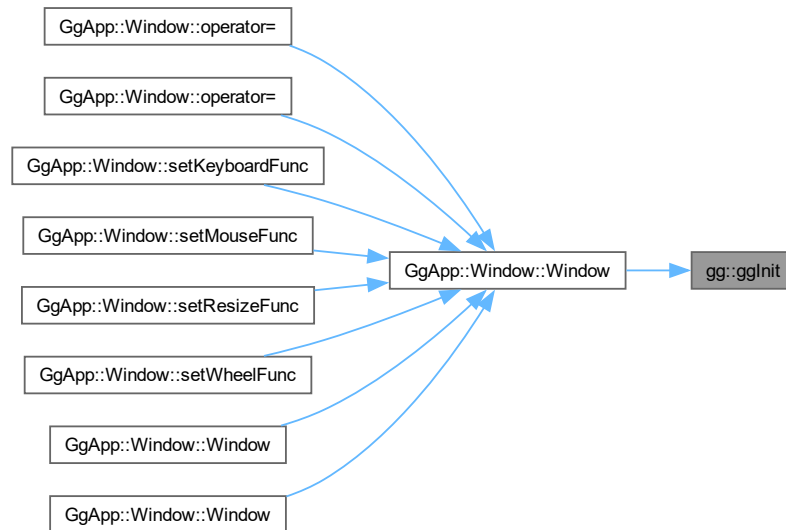
ゲームグラフィックス特論の都合にもとづく初期化を行う.

覚え書き

Windows において OpenGL 1.2 以降の API を有効化する.

gg.cpp の 1360 行目に定義があります。

被呼び出し関係図:



7.1.3.29 ggInvert() [1/2]

```
GgMatrix gg::ggInvert (
    const GgMatrix & m) [inline]
```

逆行列を返す.

引数

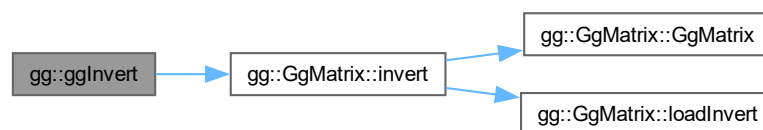
<i>m</i>	元の変換行列.
----------	---------

戻り値

m の逆行列.

gg.h の 3433 行目に定義があります。

呼び出し関係図:



7.1.3.30 ggInvert() [2/2]

```
GgQuaternion gg::ggInvert (
    const GgQuaternion & q) [inline]
```

四元数の逆元を求める。

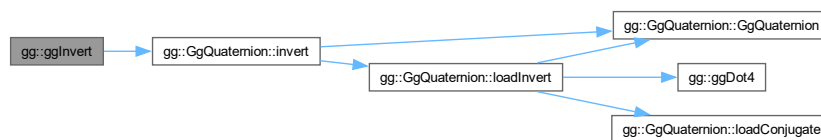
引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* の逆元.

gg.h の 4767 行目に定義があります。
呼び出し関係図:



7.1.3.31 ggLength3() [1/2]

```
GLfloat gg::ggLength3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の長さ.

引数

<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

a の 3 要素の長さ.

gg.h の 2019 行目に定義があります。
呼び出し関係図:



7.1.3.32 ggLength3() [2/2]

```
GLfloat gg::ggLength3 (
    const GLfloat * a) [inline]
```

3 要素の長さ.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
----------	-----------------------

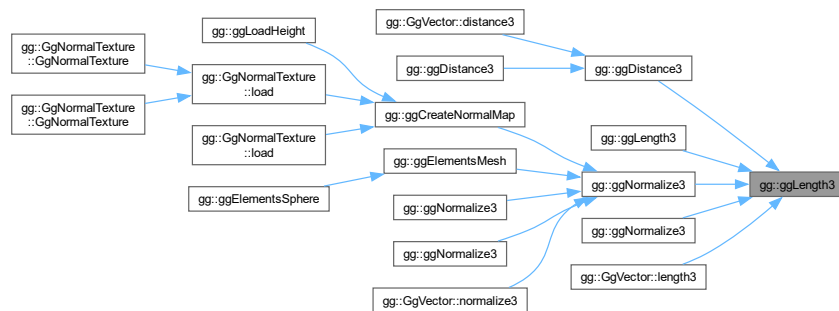
戻り値

a の 3 要素の長さ.

[gg.h](#) の 1454 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



7.1.3.33 ggLength4() [1/2]

```
GLfloat gg::ggLength4 (
    const GgVector & a) [inline]
```

[GgVector](#) 型の 4 要素の長さ.

引数

<i>a</i>	GgVector 型の変数.
----------	--------------------------------

戻り値

`a` の 4 要素の長さ.

[gg.h](#) の 2083 行目に定義があります。

呼び出し関係図:



7.1.3.34 ggLength4() [2/2]

```
GLfloat gg::ggLength4 (
    const GLfloat * a) [inline]
```

GLfloat 型の 4 要素の長さ.

引数

<code>a</code>	GLfloat 型の 4 要素の配列変数.
----------------	-----------------------

戻り値

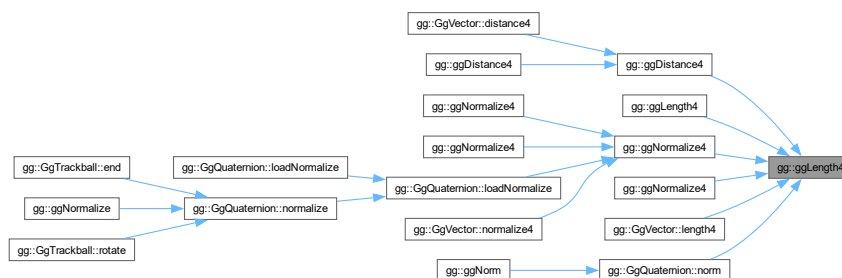
`a` の 4 要素の長さ.

[gg.h](#) の 1519 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.35 ggLoadComputeShader()

```
GLuint gg::ggLoadComputeShader (
    const std::string & comp) [extern]
```

コンピュートシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>comp</i>	コンピュートシェーダのソースファイル名.
-------------	----------------------

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 5095 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



7.1.3.36 ggLoadHeight()

```
GLuint gg::ggLoadHeight (
    const std::string & name,
    GLfloat nz,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA) [extern]
```

TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する.

引数

<i>name</i>	読み込むファイル名.
<i>nz</i>	法線の z 成分の割合.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).

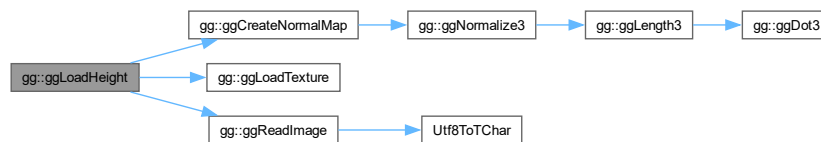
<i>internal</i>	<code>glTexImage2D()</code> に指定するテクスチャの内部フォーマット.
-----------------	--

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

`gg.cpp` の 3966 行目に定義があります。

呼び出し関係図:



7.1.3.37 ggLoadImage()

```

GLuint gg::ggLoadImage (
    const std::string & name,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE) [extern]
  
```

テクスチャを作成して TGA フォーマットの画像ファイルを読み込む。

引数

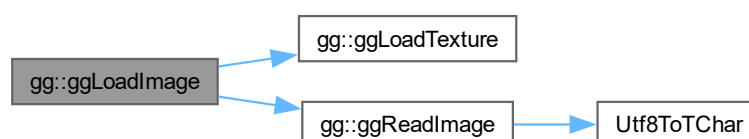
<i>name</i>	読み込むファイル名.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	テクスチャの内部フォーマット, デフォルトは GL_RGBA. 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

`gg.cpp` の 3832 行目に定義があります。

呼び出し関係図:



7.1.3.38 ggLoadShader() [1/2]

```
GLuint gg::ggLoadShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.h](#) の 5156 行目に定義があります。

呼び出し関係図:



7.1.3.39 ggLoadShader() [2/2]

```
GLuint gg::ggLoadShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [extern]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

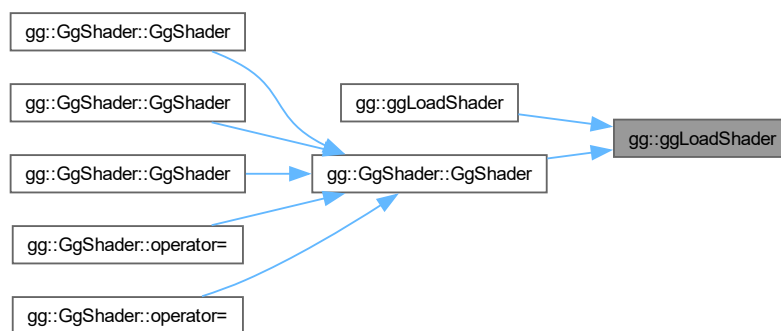
プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 5021 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.40 ggLoadSimpleObj() [1/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルと MTL ファイルを読み込む (Arrays 形式)

引数

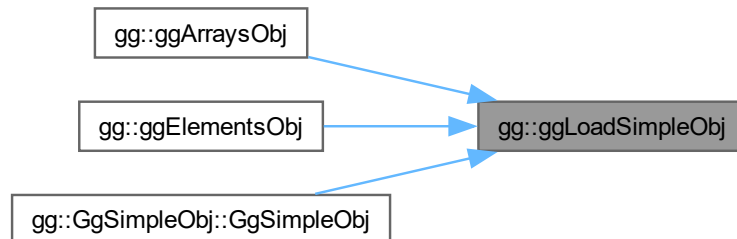
<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの <code>GgSimpleShader::Material</code> 型の材質.
<i>vert</i>	読み込んだデータの頂点属性.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら `true`.

`gg.cpp` の 4630 行目に定義があります。

被呼び出し関係図:



7.1.3.41 ggLoadSimpleObj() [2/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    std::vector< GLuint > & face,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルを読み込む (Elements 形式).

引数

<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの材質.
<i>vert</i>	読み込んだデータの頂点属性.
<i>face</i>	読み込んだデータの三角形の頂点インデックス.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら `true`.

`gg.cpp` の 4719 行目に定義があります。

7.1.3.42 ggLoadTexture()

```

GLuint gg::ggLoadTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
  
```

```

GGLenum format = GL_RGB,
GGLenum type = GL_UNSIGNED_BYTE,
GGLenum internal = GL_RGB,
GGLenum wrap = GL_CLAMP_TO_EDGE,
bool swizzle = false) [extern]

```

テクスチャを作成して確保して画像データをテクスチャとして読み込む。

引数

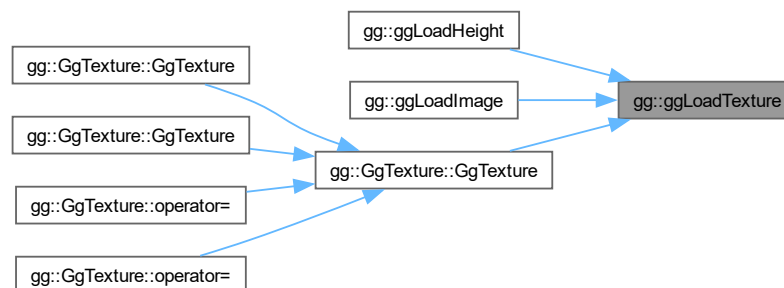
<i>image</i>	テクスチャとして読み込むデータ, nullptr ならテクスチャの作成のみを行う。
<i>width</i>	テクスチャとして読み込むデータ <i>image</i> の横の画素数。
<i>height</i>	テクスチャとして読み込むデータ <i>image</i> の縦の画素数。
<i>format</i>	<i>image</i> のフォーマット。
<i>type</i>	<i>image</i> のデータ型。
<i>internal</i>	テクスチャの内部フォーマット。
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE。
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false。

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0。

gg.cpp の 3781 行目に定義があります。

被呼び出し関係図:



7.1.3.43 ggLookat() [1/3]

```

GgMatrix gg::ggLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]

```

ビュー変換行列を返す。

引数

<i>e</i>	視点の位置を格納した GgVector 型の変数。
<i>t</i>	目標点の位置を格納した GgVector 型の変数。

u	上方向のベクトルを格納した GgVector 型の変数.
-----	-------------------------------------

戻り値

求めたビュー変換行列.

gg.h の 3348 行目に定義があります。

呼び出し関係図:



7.1.3.44 ggLookat() [2/3]

```

GgMatrix gg::ggLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
  
```

ビュー変換行列を返す.

引数

e	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
t	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
u	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

求めたビュー変換行列.

gg.h の 3330 行目に定義があります。

呼び出し関係図:



7.1.3.45 ggLookat() [3/3]

```

GgMatrix gg::ggLookat (
    GLfloat ex,
  
```

```

GLfloat ey,
GLfloat ez,
GLfloat tx,
GLfloat ty,
GLfloat tz,
GLfloat ux,
GLfloat uy,
GLfloat uz) [inline]

```

ビュー変換行列を返す。

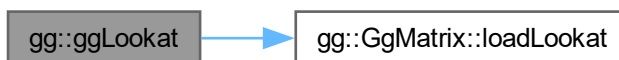
引数

<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

求めたビュー変換行列.

[gg.h](#) の 3312 行目に定義があります。
呼び出し関係図:



7.1.3.46 ggMatrixQuaternion() [1/2]

```

GgQuaternion gg::ggMatrixQuaternion (
    const GgMatrix & m) [inline]

```

回転の変換行列 *m* を表す四元数を返す。

引数

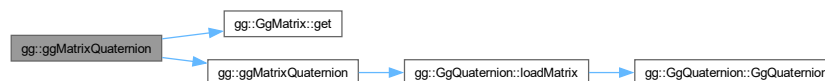
<i>m</i>	GgMatrix 型の変換行列.
----------	----------------------------------

戻り値

m による回転の変換に相当する四元数.

gg.h の 4569 行目に定義があります.

呼び出し関係図:



7.1.3.47 ggMatrixQuaternion() [2/2]

```
GgQuaternion gg::ggMatrixQuaternion (
    const GLfloat * a) [inline]
```

回転の変換行列 m を表す四元数を返す.

引数

a	GLfloat 型の 16 要素の配列変数.
---	------------------------

戻り値

a による回転の変換に相当する四元数.

gg.h の 4557 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



7.1.3.48 ggNorm()

```
GLfloat gg::ggNorm (
    const GgQuaternion & q) [inline]
```

四元数のノルムを返す.

引数

q	GgQuaternion 型の四元数.
-----	-------------------------------------

戻り値

四元数 q のノルム.

[gg.h](#) の 4734 行目に定義があります。

呼び出し関係図:



7.1.3.49 ggNormal()

```
GgMatrix gg::ggNormal (
    const GgMatrix & m) [inline]
```

法線変換行列を返す.

引数

m	元の変換行列.
-----	---------

戻り値

m の法線変換行列.

[gg.h](#) の 3444 行目に定義があります。

呼び出し関係図:



7.1.3.50 ggNormalize()

```
GgQuaternion gg::ggNormalize (
    const GgQuaternion & q) [inline]
```

正規化した四元数を返す.

引数

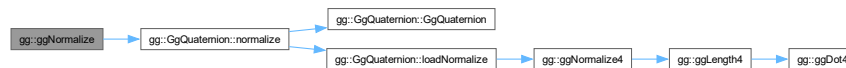
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* を正規化した四元数.

gg.h の 4745 行目に定義があります。

呼び出し関係図:



7.1.3.51 ggNormalize3() [1/4]

```
GgVector gg::ggNormalize3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の正規化.

引数

<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

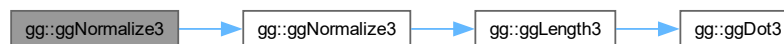
a の 3 要素を正規化したもの.

覚え書き

戻り値の *w* (第4) 要素は操作しない.

gg.h の 2045 行目に定義があります。

呼び出し関係図:



7.1.3.52 ggNormalize3() [2/4]

```
void gg::ggNormalize3 (
    const GLfloat * a,
    GLfloat * b) [inline]
```

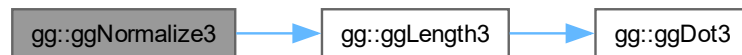
3 要素の正規化.

引数

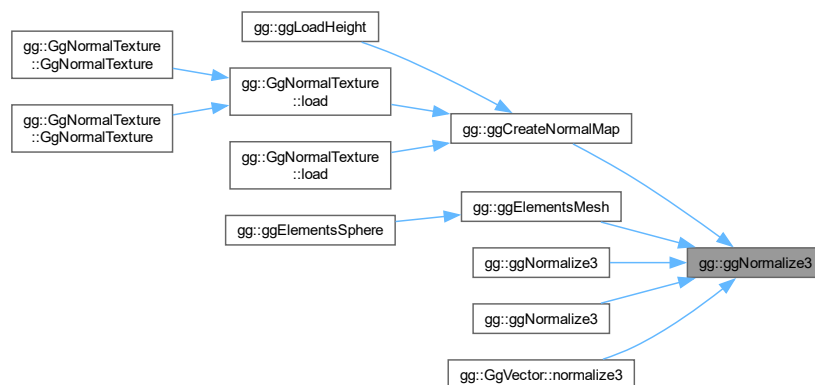
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	<i>a</i> の 3 要素を正規化した結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1478 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.53 ggNormalize3() [3/4]

```
void gg::ggNormalize3 (
    GgVector * a) [inline]
```

GgVector 型の 3 要素の正規化.

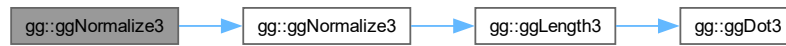
引数

<i>a</i>	GgVector 型の変数のポインタ.
----------	---------------------

覚え書き

`a` の `w` (第4) 要素は操作しない.

`gg.h` の 2060 行目に定義があります。
呼び出し関係図:



7.1.3.54 ggNormalize3() [4/4]

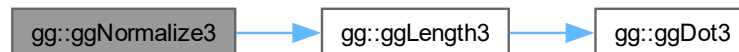
```
void gg::ggNormalize3 (
    GLfloat * a) [inline]
```

3 要素の正規化.

引数

<code>a</code>	GLfloat 型の 3 要素の配列変数.
----------------	-----------------------

`gg.h` の 1492 行目に定義があります。
呼び出し関係図:



7.1.3.55 ggNormalize4() [1/4]

```
GgVector gg::ggNormalize4 (
    const GgVector & a) [inline]
```

`GgVector` 型の 4 要素の正規化.

引数

<code>a</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

`a` の 4 要素を正規化した結果.

`gg.h` の 2106 行目に定義があります。

呼び出し関係図:



7.1.3.56 ggNormalize4() [2/4]

```
void gg::ggNormalize4 (
    const GLfloat * a,
    GLfloat * b) [inline]
```

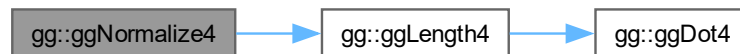
GLfloat 型の 4 要素の正規化.

引数

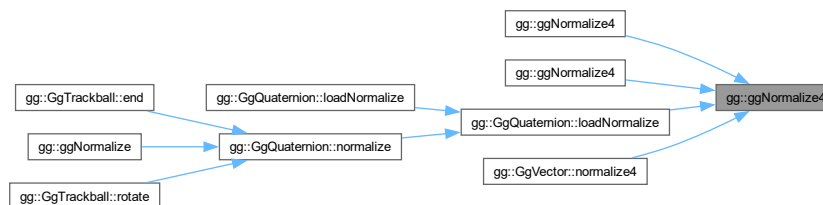
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	<i>a</i> を正規化した結果を格納する GLfloat 型の 4 要素の配列変数.

`gg.h` の 1543 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.57 ggNormalize4() [3/4]

```
void gg::ggNormalize4 (
    GgVector * a) [inline]
```

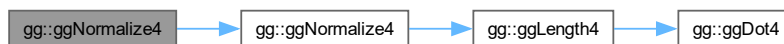
GgVector 型の 4 要素の正規化.

引数

<i>a</i>	GLfloat 型の 4 要素の配列変数.
----------	-----------------------

gg.h の 2118 行目に定義があります。

呼び出し関係図:



7.1.3.58 ggNormalize4() [4/4]

```
void gg::ggNormalize4 (
    GLfloat * a) [inline]
```

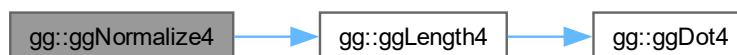
GLfloat 型の 4 要素の正規化.

引数

<i>a</i>	正規化する GLfloat 型の 4 要素の配列変数.
----------	-----------------------------

gg.h の 1558 行目に定義があります。

呼び出し関係図:



7.1.3.59 ggOrthogonal()

```
GgMatrix gg::ggOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

直交投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.

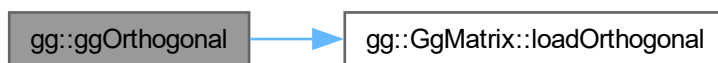
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた直交投影変換行列.

[gg.h](#) の 3369 行目に定義があります。

呼び出し関係図:



7.1.3.60 ggPerspective()

```

GgMatrix gg::ggPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) [inline]
  
```

画角を指定して透視投影変換行列を返す.

引数

<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

[gg.h](#) の 3407 行目に定義があります。

呼び出し関係図:



7.1.3.61 ggPointsCube()

```
std::shared_ptr< gg::GgPoints > gg::ggPointsCube (
    GLsizei countv,
    GLfloat length = 1.0f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を立方体状に生成する。

引数

<i>countv</i>	生成する点の数.
<i>length</i>	点群を生成する立方体の一辺の長さ.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5182 行目に定義があります。

7.1.3.62 ggPointsSphere()

```
std::shared_ptr< gg::GgPoints > gg::ggPointsSphere (
    GLsizei countv,
    GLfloat radius = 0.5f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を球状に生成する。

引数

<i>countv</i>	生成する点の数.
<i>radius</i>	点群を生成する半径.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5208 行目に定義があります。

7.1.3.63 ggQuaternionMatrix()

```
GgMatrix gg::ggQuaternionMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の変換行列を返す。

引数

q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する **GgMatrix** 型の変換行列.

gg.h の 4580 行目に定義があります。

呼び出し関係図:



7.1.3.64 ggQuaternionTransposeMatrix()

```
GgMatrix gg::ggQuaternionTransposeMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の転置した変換行列を返す.

引数

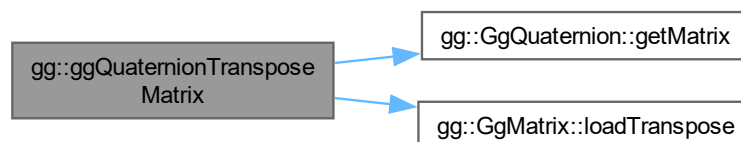
q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する転置した **GgMatrix** 型の変換行列.

gg.h の 4593 行目に定義があります。

呼び出し関係図:



7.1.3.65 ggReadImage()

```
bool gg::ggReadImage (
    const std::string & name,
    std::vector< GLubyte > & image,
```



```
GLsizei * pWidth,
GLsizei * pHeight,
GLenum * pFormat) [extern]
```

TGA ファイル (8/16/24/32bit) をメモリに読み込む。

引数

<i>name</i>	読み込むファイル名.
<i>image</i>	読み込んだデータを格納する <code>vector</code> .
<i>pWidth</i>	読み込んだ画像の横の画素数の格納先のポインタ, <code>nullptr</code> なら格納しない.
<i>pHeight</i>	読み込んだ画像の縦の画素数の格納先のポインタ, <code>nullptr</code> なら格納しない.
<i>pFormat</i>	読み込んだファイルの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA) の格納先のポインタ, <code>nullptr</code> なら格納しない.

戻り値

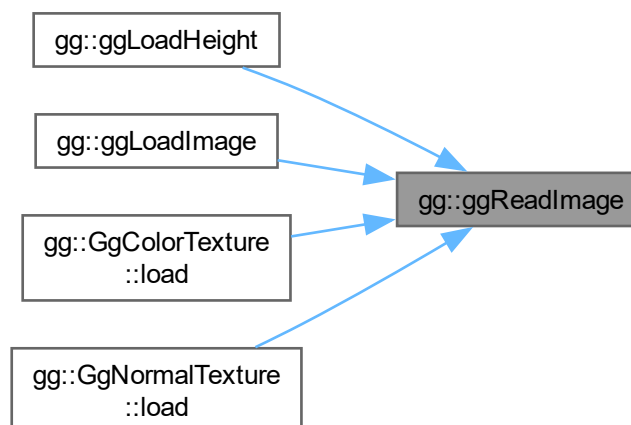
読み込みに成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3673 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.66 ggRectangle()

```
std::shared_ptr< gg::GgTriangles > gg::ggRectangle (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f) [extern]
```

矩形状に 2 枚の三角形を生成する.

引数

<i>width</i>	矩形の横幅.
<i>height</i>	矩形の高さ.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5235 行目に定義があります.

7.1.3.67 ggRotate() [1/5]

```
GgMatrix gg::ggRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>r</i>	回転軸のベクトルと回転角を表す GgVector 型の変数.
----------	--

戻り値

(*r*[0], *r*[1], *r*[2]) を軸に *r*[3] だけ回転する変換行列.

[gg.h](#) の 3292 行目に定義があります.

呼び出し関係図:



7.1.3.68 ggRotate() [2/5]

```
GgMatrix gg::ggRotate (
    const GgVector & r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

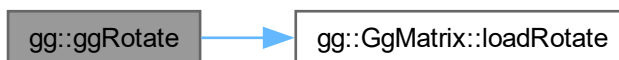
引数

<i>r</i>	回転軸のベクトルを表す GgVector 型の変数.
<i>a</i>	回転角.

戻り値

r を軸に *a* だけ回転する変換行列.

gg.h の 3268 行目に定義があります。
呼び出し関係図:



7.1.3.69 ggRotate() [3/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>r</i>	回転軸のベクトルと回転角を表す GLfloat 型の 4 要素の配列変数.
----------	---------------------------------------

戻り値

(*r*[0], *r*[1], *r*[2]) を軸に *r*[3] だけ回転する変換行列.

gg.h の 3280 行目に定義があります。
呼び出し関係図:



7.1.3.70 ggRotate() [4/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>r</i>	回転軸のベクトルを表す GLfloat 型の 3 要素の配列変数.
<i>a</i>	回転角.

戻り値

r を軸に *a* だけ回転する変換行列.

[gg.h](#) の 3255 行目に定義があります。

呼び出し関係図:



7.1.3.71 ggRotate() [5/5]

```
GgMatrix gg::ggRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

(x, y, z) を軸にさらに *a* 回転する変換行列.

[gg.h](#) の 3242 行目に定義があります。

呼び出し関係図:



7.1.3.72 ggRotateQuaternion() [1/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を返す.

引数

v	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

回転を表す四元数.

gg.h の 4634 行目に定義があります。
呼び出し関係図:



7.1.3.73 ggRotateQuaternion() [2/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を返す.

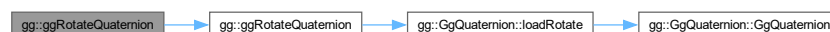
引数

v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

回転を表す四元数.

gg.h の 4623 行目に定義があります。
呼び出し関係図:



7.1.3.74 ggRotateQuaternion() [3/3]

```
GgQuaternion gg::ggRotateQuaternion (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
```

(x, y, z) を軸として角度 a 回転する四元数を返す.

引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

戻り値

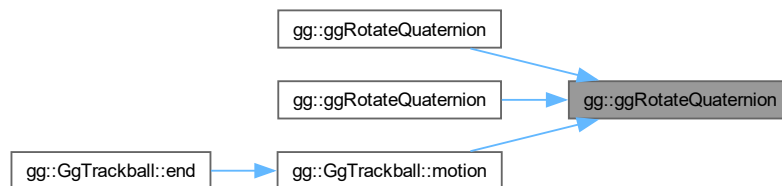
回転を表す四元数.

[gg.h](#) の 4610 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.75 ggRotateX()

```
GgMatrix gg::ggRotateX (
    GLfloat a) [inline]
```

x 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

x 軸中心に a だけ回転する変換行列.

gg.h の 3203 行目に定義があります。

呼び出し関係図:



7.1.3.76 ggRotateY()

```
GgMatrix gg::ggRotateY (  
    GLfloat a) [inline]
```

y 軸中心の回転の変換行列を返す.

引数

a	回転角.
---	------

戻り値

y 軸中心に a だけ回転する変換行列.

gg.h の 3215 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.77 ggRotateZ()

```
GgMatrix gg::ggRotateZ (
    GLfloat a) [inline]
```

z 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

z 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3227 行目に定義があります。
呼び出し関係図:



7.1.3.78 ggSaveColor()

```
bool gg::ggSaveColor (
    const std::string & name) [extern]
```

カラーバッファの内容を TGA ファイルに保存する.

引数

<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば `true`, 失敗すれば `false`.

[gg.cpp](#) の 3617 行目に定義があります。
呼び出し関係図:



7.1.3.79 ggSaveDepth()

```
bool gg::ggSaveDepth (
    const std::string & name) [extern]
```

デブスバッファの内容を TGA ファイルに保存する.

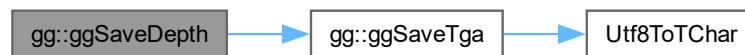
引数

<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば `true`, 失敗すれば `false`.

[gg.cpp](#) の 3643 行目に定義があります。
呼び出し関係図:



7.1.3.80 ggSaveTga()

```
bool gg::ggSaveTga (
    const std::string & name,
    const void * buffer,
    unsigned int width,
    unsigned int height,
    unsigned int depth) [extern]
```

配列の内容を TGA ファイルに保存する.

引数

<i>name</i>	保存するファイル名.
<i>buffer</i>	画像データを格納した配列.
<i>width</i>	画像の横の画素数.
<i>height</i>	画像の縦の画素数.
<i>depth</i>	1画素のバイト数.

戻り値

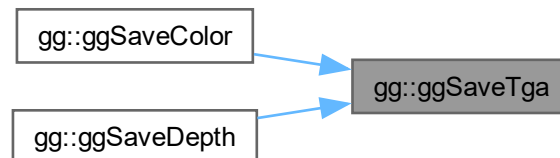
保存に成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3541 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.81 ggScale() [1/3]

```
GgMatrix gg::ggScale (
    const GgVector & s) [inline]
```

拡大縮小の変換行列を返す。

引数

<code>s</code>	拡大率の <code>GgVector</code> 型の変数.
----------------	----------------------------------

戻り値

拡大縮小の変換行列。

`gg.h` の 3191 行目に定義があります。

呼び出し関係図:



7.1.3.82 ggScale() [2/3]

```
GgMatrix gg::ggScale (
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を返す.

引数

s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

拡大縮小の変換行列.

[gg.h](#) の 3179 行目に定義があります。
呼び出し関係図:

**7.1.3.83 ggScale()** [3/3]

```
GgMatrix gg::ggScale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) [inline]
```

拡大縮小の変換行列を返す.

引数

x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	拡大率のスケールファクタ (= 1.0f).

戻り値

拡大縮小の変換行列.

[gg.h](#) の 3167 行目に定義があります。

呼び出し関係図:



7.1.3.84 ggSlerp() [1/4]

```

GgQuaternion gg::ggSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
  
```

二つの四元数の球面線形補間の結果を返す.

引数

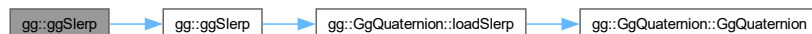
<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

q, *r* を *t* で内分した四元数.

[gg.h](#) の 4697 行目に定義があります。

呼び出し関係図:



7.1.3.85 ggSlerp() [2/4]

```

GgQuaternion gg::ggSlerp (
    const GgQuaternion & q,
    const GLfloat * a,
    GLfloat t) [inline]
  
```

二つの四元数の球面線形補間の結果を返す.

引数

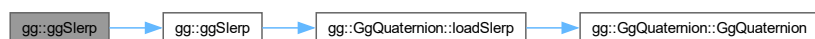
<i>q</i>	GgQuaternion 型の四元数.
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

q, *a* を *t* で内分した四元数.

[gg.h](#) の 4710 行目に定義があります。

呼び出し関係図:



7.1.3.86 ggSlerp() [3/4]

```

GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
  
```

二つの四元数の球面線形補間の結果を返す.

引数

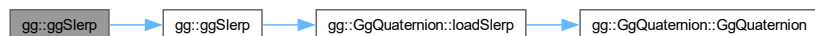
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

a, *q* を *t* で内分した四元数.

[gg.h](#) の 4723 行目に定義があります。

呼び出し関係図:



7.1.3.87 ggSlerp() [4/4]

```

GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
  
```

二つの四元数の球面線形補間の結果を返す.

引数

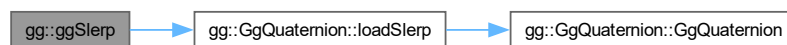
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>b</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

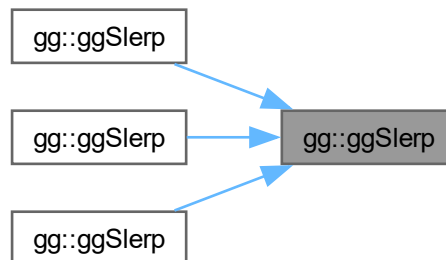
a, *b* を *t* で内分した四元数.

[gg.h](#) の 4683 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.88 ggTranslate() [1/3]

```
GgMatrix gg::ggTranslate (
    const GgVector & t) [inline]
```

平行移動の変換行列を返す.

引数

<i>t</i>	移動量の GgVector 型の変数.
----------	---------------------

戻り値

平行移動の変換行列.

[gg.h](#) の 3152 行目に定義があります。

呼び出し関係図:



7.1.3.89 ggTranslate() [2/3]

```
GgMatrix gg::ggTranslate (
    const GLfloat * t) [inline]
```

平行移動の変換行列を返す.

引数

<i>t</i>	移動量の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

平行移動の変換行列.

[gg.h](#) の 3140 行目に定義があります。

呼び出し関係図:



7.1.3.90 ggTranslate() [3/3]

```
GgMatrix gg::ggTranslate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) [inline]
```

平行移動の変換行列を返す.

引数

<i>x</i>	x 方向の移動量.
<i>y</i>	y 方向の移動量.
<i>z</i>	z 方向の移動量.
<i>w</i>	移動量のスケールファクタ (= 1.0f).

戻り値

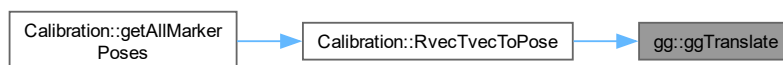
平行移動の変換行列.

[gg.h](#) の 3128 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.91 ggTranspose()

```
GgMatrix gg::ggTranspose (
    const GgMatrix & m) [inline]
```

転置行列を返す.

引数

<i>m</i>	元の変換行列.
----------	---------

戻り値

m の転置行列.

gg.h の 3422 行目に定義があります。

呼び出し関係図:



7.1.3.92 operator*()

```
GgVector gg::operator* (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに GgVector 型の各要素を乗じた積を返す.

引数

a	GLfloat 型の値.
b	GgVector 型のベクトル.

戻り値

a に b の各要素を乗じた積.

gg.h の 1967 行目に定義があります。

7.1.3.93 operator+() [1/2]

```
const GgVector & gg::operator+ (
    const GgVector & v) [inline]
```

単項の + 演算子なので何もしない.

引数

v	GgVector 型のベクトル.
---	------------------

戻り値

v の値.

gg.h の 1920 行目に定義があります。

7.1.3.94 operator+() [2/2]

```
GgVector gg::operator+ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに `GgVector` 型の各要素を足した和を返す.

引数

<code>a</code>	GLfloat 型の値.
<code>b</code>	<code>GgVector</code> 型のベクトル.

戻り値

`a` に `b` の各要素を足した和.

`gg.h` の 1932 行目に定義があります。

7.1.3.95 operator-() [1/2]

```
const GgVector gg::operator- (
    const GgVector & v) [inline]
```

符号の反転.

引数

<code>v</code>	<code>GgVector</code> 型のベクトル.
----------------	-------------------------------

戻り値

`v` の値の符号を反転した結果.

`gg.h` の 1943 行目に定義があります。

7.1.3.96 operator-() [2/2]

```
GgVector gg::operator- (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーから `GgVector` 型の各要素を引いた差を返す.

引数

<code>a</code>	GLfloat 型の値.
<code>b</code>	<code>GgVector</code> 型のベクトル.

戻り値

`a` から `b` の各要素を引いた差.

`gg.h` の 1955 行目に定義があります。

7.1.3.97 operator/()

```
GgVector gg::operator/ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーを `GgVector` 型の各要素で割った商を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a を *b* の各要素で割った商.

gg.h の 1979 行目に定義があります。

7.1.4 変数詳解

7.1.4.1 ggBufferAlignment

```
GLint gg::ggBufferAlignment [extern]
```

使用している GPU のバッファアライメント.

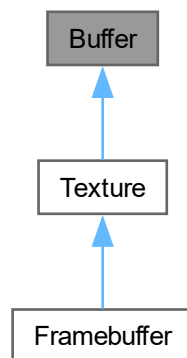
使用している GPU のバッファオブジェクトのアライメント, 初期化に取得される.

Chapter 8

クラス详解

8.1 Buffer クラス

`#include <Buffer.h>`
Buffer の継承関係図



公開メンバ関数

- `Buffer ()`
- `Buffer (GLsizei width, GLsizei height, int channels)`
- `Buffer (const Buffer &buffer)`
- `Buffer (Buffer &&buffer) noexcept`
- `virtual ~Buffer ()`
- `Buffer & operator= (const Buffer &buffer)`
- `Buffer & operator= (Buffer &&buffer) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `void resize (GLsizei width, GLsizei height, int channels)`
- `void resize (const Buffer &buffer)`
- `virtual void copy (const Buffer &buffer) noexcept`
- `virtual void discard ()`
- `auto getBufferName () const`
- `virtual const std::array< GLsizei, 2 > & getSize () const`
- `GLsizei getWidth () const`
- `GLsizei getHeight () const`

- virtual int `getChannels` () const
- auto `getFormat` () const
- auto `getAspect` () const
- void `bindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void `unbindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * `map` () const
- void `unmap` () const

限定公開メンバ関数

- auto `channelsToFormat` (int channels) const
- int `formatToChannels` (GLenum format) const
- void `copyBuffer` (const `Buffer` &buffer) noexcept

8.1.1 詳解

バッファクラス

@description フレームを格納するピクセルバッファオブジェクト
`Buffer.h` の 24 行目に定義があります。

8.1.2 構築子と解体子

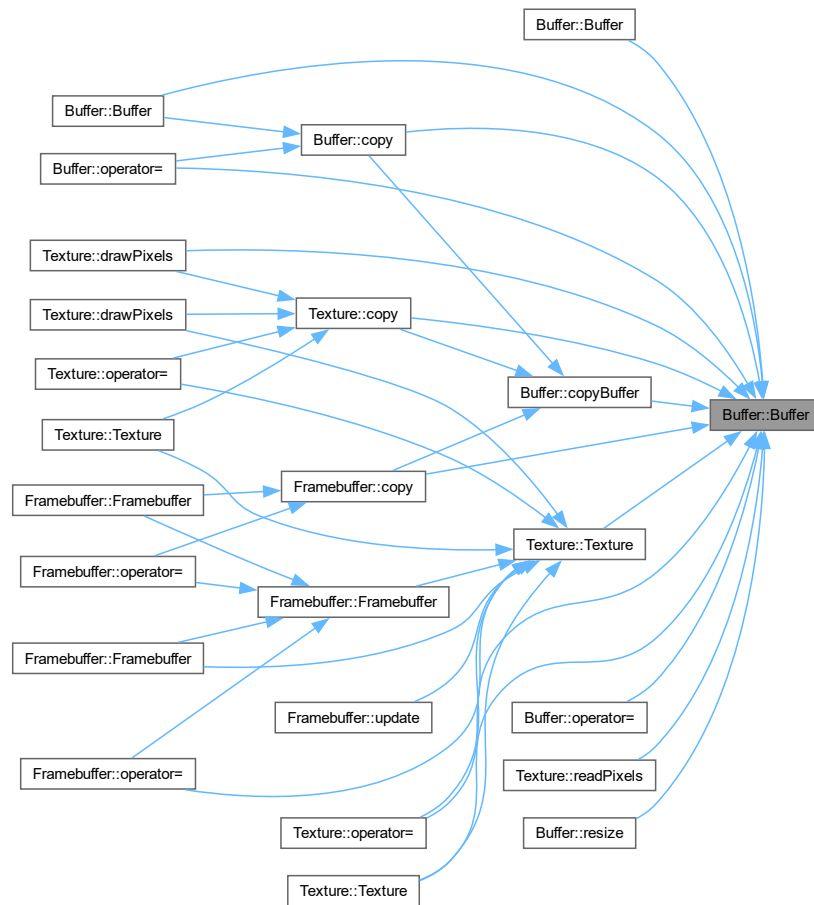
8.1.2.1 `Buffer()` [1/4]

```
Buffer::Buffer () [inline]
```

バッファのデフォルトコンストラクタ

`Buffer.h` の 81 行目に定義があります。

被呼び出し関係図:



8.1.2.2 Buffer() [2/4]

```
Buffer::Buffer (
    GLsizei width,
    GLsizei height,
    int channels) [inline]
バッファを作成するコンストラクタ
```

引数

<i>width</i>	格納するフレームの横の画素数
<i>height</i>	格納するフレームの縦の画素数
<i>channels</i>	格納するフレームのチャンネル数

覚え書き

ピクセルバッファオブジェクトを作成して、そこにフレームのデータを格納する。

[Buffer.h](#) の 102 行目に定義があります。

呼び出し関係図:



8.1.2.3 Buffer() [3/4]

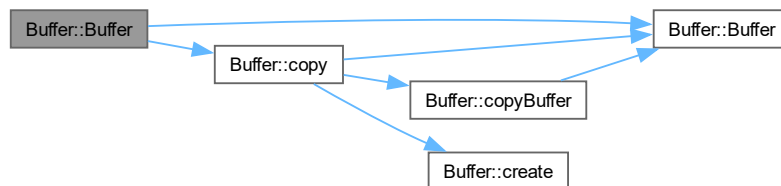
```
Buffer::Buffer (
    const Buffer & buffer) [inline]
コピーコンストラクタ
```

引数

<i>texture</i>	コピー元のバッファ
----------------	-----------

[Buffer.h](#) の 112 行目に定義があります。

呼び出し関係図:



8.1.2.4 Buffer() [4/4]

```
Buffer::Buffer (
    Buffer && buffer) [inline], [noexcept]
ムーブコンストラクタ
```

引数

<i>texture</i>	ムーブ元のバッファ
----------------	-----------

[Buffer.h](#) の 122 行目に定義があります。

呼び出し関係図:



8.1.2.5 ~Buffer()

```
virtual Buffer::~Buffer () [inline], [virtual]
```

デストラクタ

[Buffer.h](#) の 143 行目に定義があります。

呼び出し関係図:



8.1.3 関数詳解

8.1.3.1 bindBuffer()

```
void Buffer::bindBuffer (
    GLenum target = GL_PIXEL_PACK_BUFFER) const [inline]
```

バッファのピクセルバッファオブジェクトを結合する

引数

<i>target</i>	結合するターゲット
---------------	-----------

[Buffer.h](#) の 303 行目に定義があります。

8.1.3.2 channelsToFormat()

```
auto Buffer::channelsToFormat (
    int channels) const [inline], [protected]
```

チャンネル数からフォーマットを求める

引数

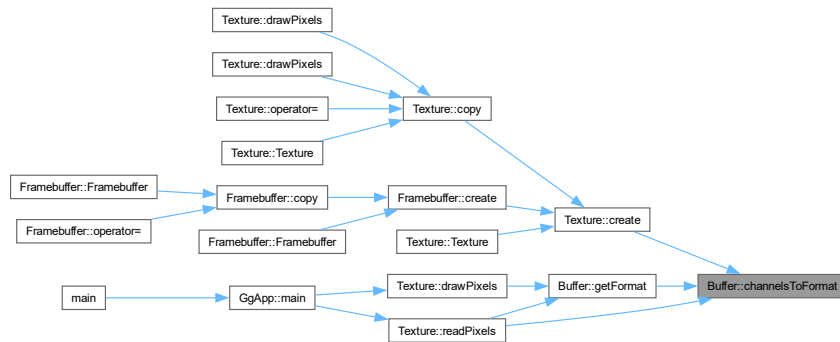
<i>channels</i>	フレームのチャンネル数
-----------------	-------------

戻り値

テクスチャのフォーマット

[Buffer.h](#) の 52 行目に定義があります。

被呼び出し関係図:



8.1.3.3 copy()

```
void Buffer::copy (
    const Buffer & buffer) [virtual], [noexcept]
```

バッファをコピーする

引数

<i>buffer</i>	コピー元のバッファ
---------------	-----------

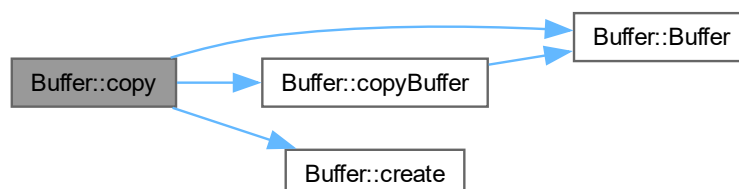
覚え書き

このバッファのサイズがコピー元のバッファのサイズと異なれば、このバッファを削除して新しいバッファを作り直してコピーする。

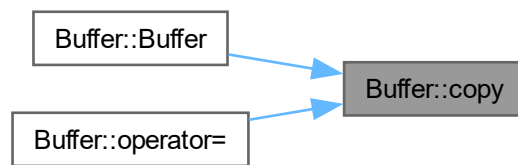
[Framebuffer](#), [Texture](#)で再実装されています。

[Buffer.cpp](#) の 93 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.4 copyBuffer()

```
void Buffer::copyBuffer (  
    const Buffer & buffer) [protected], [noexcept]
```

既存のバッファにコピーする

引数

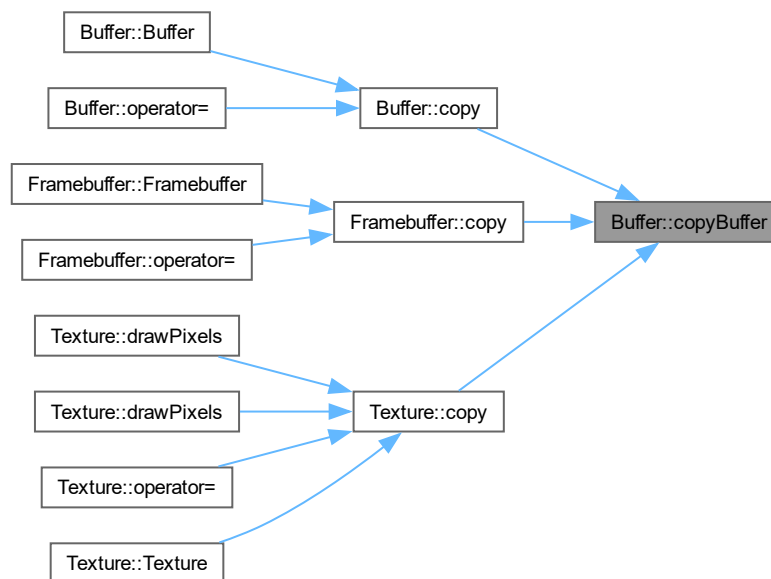
<i>buffer</i>	コピー元のバッファ
---------------	-----------

[Buffer.cpp](#) の 39 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.5 create()

```
void Buffer::create (
    GLsizei width,
    GLsizei height,
    int channels) [virtual]
```

バッファを作成する

引数

<i>width</i>	バッファに格納するフレームの横の画素数
<i>height</i>	バッファに格納するフレームの縦の画素数
<i>channels</i>	バッファに格納するフレームのチャンネル数

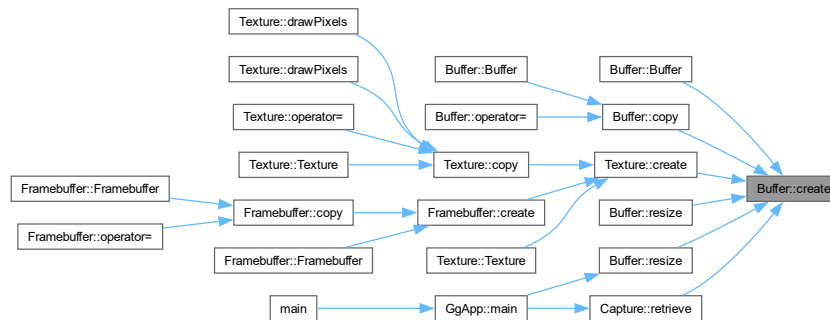
覚え書き

このバッファのサイズが引数で指定したサイズと異なれば、このバッファを削除して新しいバッファを作り直す。

[Framebuffer](#), [Texture](#)で再実装されています。

[Buffer.cpp](#) の 60 行目に定義があります。

被呼び出し関係図:



8.1.3.6 discard()

```
void Buffer::discard () [virtual]
```

バッファを破棄する

[Framebuffer](#), [Texture](#)で再実装されています。

[Buffer.cpp](#) の 187 行目に定義があります。

被呼び出し関係図:



8.1.3.7 formatToChannels()

```
int Buffer::formatToChannels (
    GLenum format) const [protected]
```

フォーマットからチャンネル数を求める

引数

<i>format</i>	テクスチャのフォーマット
---------------	--------------

戻り値

フレームのチャンネル数

[Buffer.cpp](#) の 16 行目に定義があります。

8.1.3.8 getAspect()

```
auto Buffer::getAspect () const [inline]
```

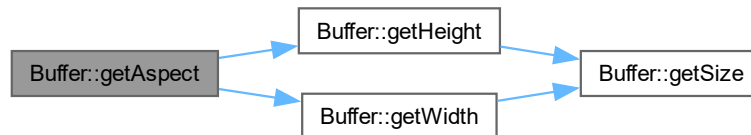
格納されているフレームの縦横比を得る

戻り値

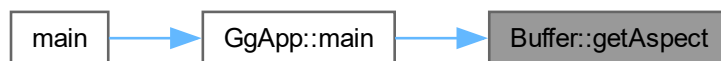
格納されているフレームの縦横比

[Buffer.h](#) の 293 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.9 getBufferName()

```
auto Buffer::getBufferName () const [inline]
```

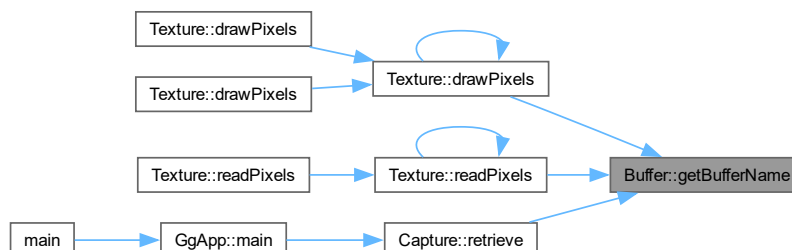
バッファのピクセルバッファオブジェクト名を得る

戻り値

ピクセルバッファオブジェクト名

[Buffer.h](#) の 230 行目に定義があります。

被呼び出し関係図:



8.1.3.10 getChannels()

```
virtual int Buffer::getChannels () const [inline], [virtual]
```

格納されているフレームのチャンネル数を得る

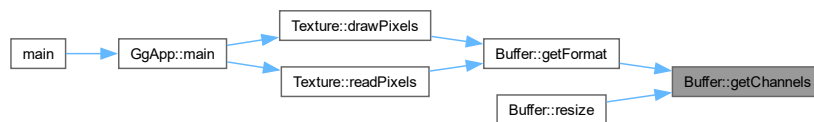
戻り値

格納されているフレームのチャンネル数

[Framebuffer](#), [Texture](#)で再実装されています。

[Buffer.h](#) の 273 行目に定義があります。

被呼び出し関係図:



8.1.3.11 getFormat()

```
auto Buffer::getFormat () const [inline]
```

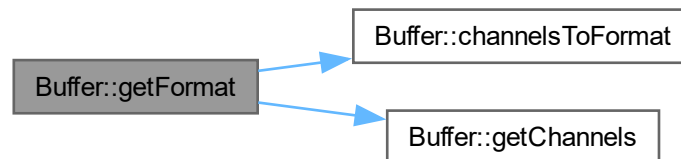
格納されているフレームのフォーマットを得る

戻り値

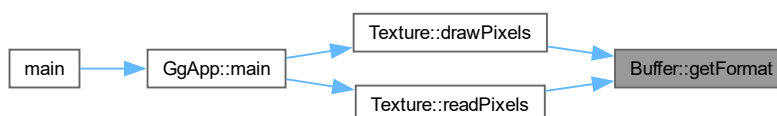
格納されているフレームのフォーマット

[Buffer.h](#) の 283 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.12 getHeight()

```
GLsizei Buffer::getHeight () const [inline]
```

格納されているフレームの縦の画素数を得る

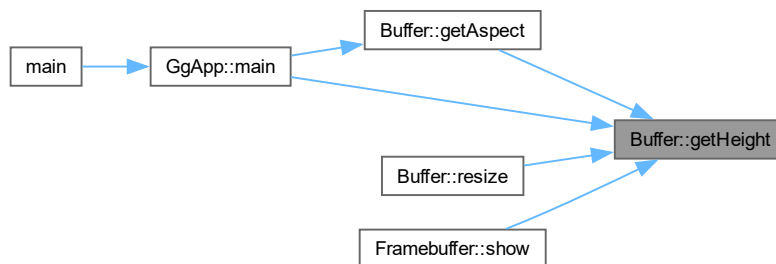
戻り値

格納されているフレームの縦の画素数

[Buffer.h](#) の 263 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.1.3.13 getSize()

```
virtual const std::array< GLsizei, 2 > & Buffer::getSize () const [inline], [virtual]
```

格納されているフレームのサイズを得る

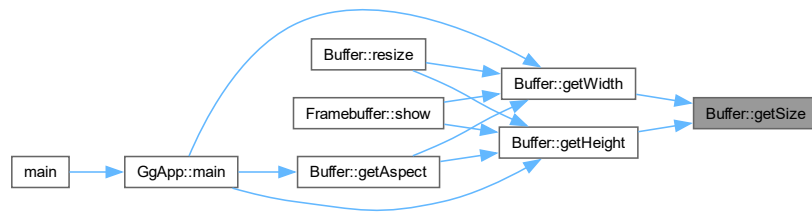
戻り値

格納されているフレームのサイズ

[Framebuffer](#), [Texture](#)で再実装されています。

[Buffer.h](#) の 243 行目に定義があります。

被呼び出し関係図:



8.1.3.14 getWidth()

```
GLsizei Buffer::getWidth () const [inline]
```

格納されているフレームの横の画素数を得る

戻り値

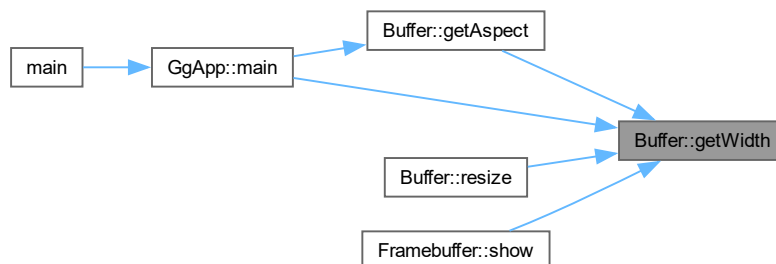
格納されているフレームの横の画素数

[Buffer.h](#) の 253 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.15 map()

```
GLvoid * Buffer::map () const
```

バッファのピクセルバッファオブジェクトをマップする

戻り値

ピクセルバッファオブジェクトをマップしたメモリ

[Buffer.cpp](#) の 152 行目に定義があります。

被呼び出し関係図:



8.1.3.16 operator=() [1/2]

```
Buffer & Buffer::operator= (
    Buffer && buffer) [noexcept]
```

ムーブ代入演算子

引数

<i>buffer</i>	ムーブ代入元のバッファ
---------------	-------------

戻り値

ムーブ代入結果のバッファの参照

[Buffer.cpp](#) の 121 行目に定義があります。

呼び出し関係図:



8.1.3.17 operator=() [2/2]

```
Buffer & Buffer::operator= (
    const Buffer & buffer)
```

代入演算子

引数

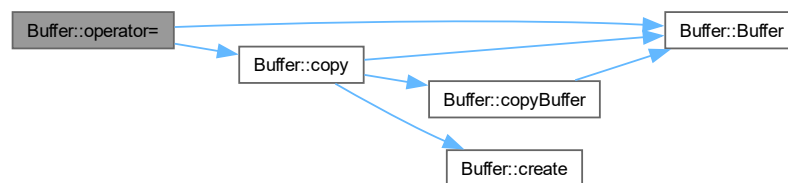
<i>buffer</i>	代入元のバッファ
---------------	----------

戻り値

代入結果のバッファの参照

[Buffer.cpp](#) の 106 行目に定義があります。

呼び出し関係図:



8.1.3.18 `resize()` [1/2]

```
void Buffer::resize (
    const Buffer & buffer) [inline]
```

オブジェクトが保持するフレームのサイズを引数に指定したオブジェクトと同じにする

引数

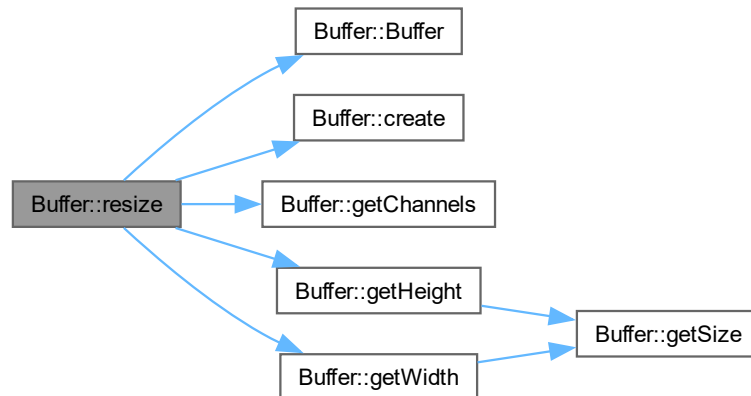
<i>buffer</i>	サイズの基準に用いるオブジェクト
<i>pixels</i>	作成するオブジェクトに格納するフレームのデータのポインタ

覚え書き

このオブジェクトのサイズが引数で指定したオブジェクトのサイズと異なれば、このオブジェクトを削除して新しいオブジェクトを作り直す。

[Buffer.h](#) の 203 行目に定義があります。

呼び出し関係図:

8.1.3.19 `resize()` [2/2]

```
void Buffer::resize (
    GLsizei width,
    GLsizei height,
    int channels) [inline]
```

オブジェクトが保持するフレームのサイズを変更する

引数

<i>width</i>	フレームの横の画素数
<i>height</i>	フレームの縦の画素数
<i>channels</i>	フレームのチャンネル数

覚え書き

このオブジェクトのサイズが引数で指定したオブジェクトのサイズと異なれば、このオブジェクトを削除して新しいオブジェクトを作り直す。

[Buffer.h](#) の 188 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.20 unbindBuffer()

```
void Buffer::unbindBuffer (
    GLenum target = GL_PIXEL_PACK_BUFFER) const [inline]
```

バッファのピクセルバッファオブジェクトの結合を解除する

引数

<i>target</i>	結合を解除するターゲット
---------------	--------------

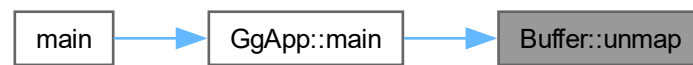
[Buffer.h](#) の 315 行目に定義があります。

8.1.3.21 unmap()

```
void Buffer::unmap () const
```

バッファのピクセルバッファオブジェクトをアンマップする
[Buffer.cpp](#) の 173 行目に定義があります。

被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Buffer.h](#)
- [Buffer.cpp](#)

8.2 Calibration クラス

```
#include <Calibration.h>
```

公開メンバ関数

- [Calibration](#) (const std::string &dictionaryName, const std::array< float, 2 > &length)
- [Calibration](#) (const [Calibration](#) &calibration)=delete
- virtual ~[Calibration](#) ()
- [Calibration](#) & operator= (const [Calibration](#) &calibration)=delete
- void [createBoard](#) (const std::array< float, 2 > &length)
- void [setDictionary](#) (const std::string &dictionaryName, const std::array< float, 2 > &length)
- void [drawBoard](#) (cv::Mat &boardImage, int width, int height)
- void [detectBoard](#) (cv::Mat &image)
- void [detectMarkers](#) (cv::Mat &image, float markerLength)
- void [recordCorners](#) ()
- void [discardCorners](#) ()
- bool [calibrate](#) ()
- auto [getCornersCount](#) () const
- auto [getSampleCount](#) () const
- auto [getTotalCount](#) () const
- const auto & [getCameraMatrix](#) () const
- const auto & [getDistortionCoefficients](#) () const
- auto [getReprojectionError](#) () const
- auto [finished](#) ()
- [GgMatrix RvecTvecToPose](#) (const cv::Vec3d &rvec, const cv::Vec3d &tvec)
- void [getAllMarkerPoses](#) (float markerLength, std::map< int, [GgMatrix](#) > &poses)
- bool [loadParameters](#) (const std::string &filename)
- bool [saveParameters](#) (const std::string &filename) const

静的公開変数類

- static const std::map< const std::string, const cv::aruco::PredefinedDictionaryType > [dictionaryList](#)
ArUco Marker 辞書のリスト

8.2.1 詳解

校正クラス

[Calibration.h](#) の 24 行目に定義があります。

8.2.2 構築子と解体子

8.2.2.1 Calibration() [1/2]

```
Calibration::Calibration (
    const std::string & dictionaryName,
    const std::array< float, 2 > & length)
```

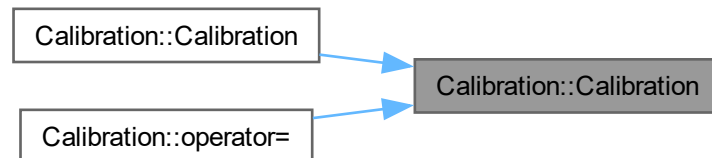
較正オブジェクトのコンストラクタ

引数

<i>dictionaryName</i>	ArUco Marker の辞書名
<i>length</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 28 行目に定義があります。

被呼び出し関係図:



8.2.2.2 Calibration() [2/2]

```
Calibration::Calibration (
    const Calibration & calibration) [delete]
```

コピーコンストラクタは使用しない

引数

<i>calibration</i>	コピー元
--------------------	------

呼び出し関係図:



8.2.2.3 ~Calibration()

```
Calibration::~~Calibration () [virtual]
```

デストラクタ

[Calibration.cpp](#) の 55 行目に定義があります。

8.2.3 関数詳解

8.2.3.1 `calibrate()`

```
bool Calibration::calibrate ()
```

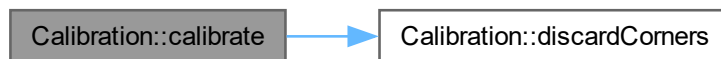
校正する

戻り値

校正に成功したら `true`

[Calibration.cpp](#) の 220 行目に定義があります。

呼び出し関係図:



8.2.3.2 `createBoard()`

```
void Calibration::createBoard (
    const std::array< float, 2 > & length)
```

ChArUco Board を作成する

引数

<i>length</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)
---------------	--

[Calibration.cpp](#) の 62 行目に定義があります。

被呼び出し関係図:



8.2.3.3 `detectBoard()`

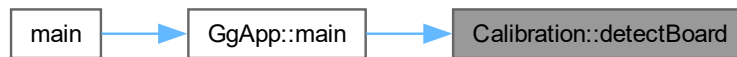
```
void Calibration::detectBoard (
    cv::Mat & image)
```

ChArUco Board を検出する

引数

<i>image</i>	ChArUco Board を検出する画像
--------------	-----------------------

[Calibration.cpp](#) の 111 行目に定義があります。
被呼び出し関係図:



8.2.3.4 detectMarkers()

```
void Calibration::detectMarkers (  
    cv::Mat & image,  
    float markerLength)
```

ArUco Marker を検出する

引数

<i>image</i>	ArUco Marker を検出する画像
<i>markerLength</i>	ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 129 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.2.3.5 discardCorners()

`void Calibration::discardCorners ()`
 取得した標本と校正結果を破棄する
[Calibration.cpp](#) の 200 行目に定義があります。
 被呼び出し関係図:



8.2.3.6 drawBoard()

`void Calibration::drawBoard (`
 `cv::Mat & boardImage,`
 `int width,`
 `int height)`

ChArUco Board を描く

引数

<i>boardImage</i>	ChArUco Board を描き込む画像
<i>width</i>	ChArUco Board の横の画素数
<i>height</i>	ChArUco Board の縦の画素数

[Calibration.cpp](#) の 103 行目に定義があります。

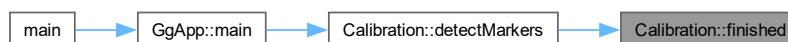
8.2.3.7 finished()

`auto Calibration::finished () [inline]`
 校正が完了したかどうかを調べる

戻り値

校正が完了していたら `true`

[Calibration.h](#) の 222 行目に定義があります。
 被呼び出し関係図:



8.2.3.8 getAllMarkerPoses()

`void Calibration::getAllMarkerPoses (`
 `float markerLength,`
 `std::map< int, GgMatrix > & poses)`

ArUco Marker の 3 次元姿勢の変換行列を求める

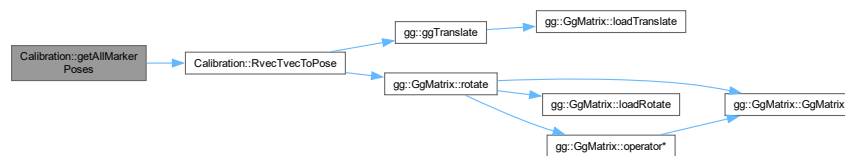
引数

<i>markerLength</i>	ArUco Marker の一辺の長さ (単位 cm)
<i>poses</i>	推定した ArUco Marker の 3 次元姿勢

覚え書き

これはキャリブレーション終了後に単独マーカの位置推定に用いる

[Calibration.cpp](#) の 313 行目に定義があります。
呼び出し関係図:



8.2.3.9 getCameraMatrix()

```
const auto & Calibration::getCameraMatrix () const [inline]
```

カメラ行列を取り出す

戻り値

カメラ行列

[Calibration.h](#) の 192 行目に定義があります。

8.2.3.10 getCornersCount()

```
auto Calibration::getCornersCount () const [inline]
```

検出数を取得する

戻り値

検出されたコーナーの数

[Calibration.h](#) の 162 行目に定義があります。

8.2.3.11 getDistortionCoefficients()

```
const auto & Calibration::getDistortionCoefficients () const [inline]
```

歪パラメータを取り出す

戻り値

歪パラメータ

[Calibration.h](#) の 202 行目に定義があります。

8.2.3.12 getReprojectionError()

```
auto Calibration::getReprojectionError () const [inline]
```

再投影誤差を得る

戻り値

再投影誤差

[Calibration.h](#) の 212 行目に定義があります。

8.2.3.13 getSampleCount()

```
auto Calibration::getSampleCount () const [inline]
```

標本数を取得する

戻り値

保存された標本の数

[Calibration.h](#) の 172 行目に定義があります。

8.2.3.14 getTotalCount()

```
auto Calibration::getTotalCount () const [inline]
```

検出数の合計を取得する

戻り値

検出されたコーナーの数

[Calibration.h](#) の 182 行目に定義があります。

8.2.3.15 loadParameters()

```
bool Calibration::loadParameters (
    const std::string & filename)
ファイルからキャリブレーションパラメータを読み込む
```

引数

<i>filename</i>	保存するファイル名
-----------------	-----------

戻り値

パラメータの読み込みに成功したら true

[Calibration.cpp](#) の 397 行目に定義があります。
呼び出し関係図:



8.2.3.16 operator=()

```
Calibration & Calibration::operator= (  
    const Calibration & calibration) [delete]
```

代入演算子は使用しない

引数

<i>calibration</i>	代入元
--------------------	-----

呼び出し関係図:



8.2.3.17 recordCorners()

```
void Calibration::recordCorners ()
```

標本を取得する

[Calibration.cpp](#) の 169 行目に定義があります。

8.2.3.18 RvecTvecToPose()

```
GgMatrix Calibration::RvecTvecToPose (  
    const cv::Vec3d & rvec,  
    const cv::Vec3d & tvec)
```

回転ベクトルと並進ベクトルから姿勢の変換行列を求める

引数

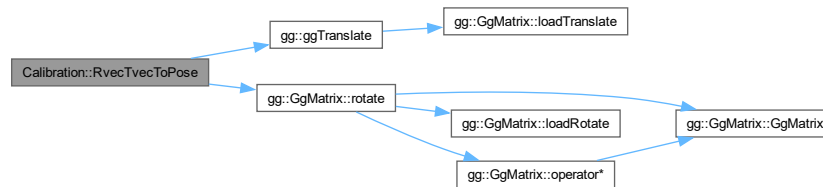
<i>rvec</i>	回転ベクトル
<i>tvec</i>	並進ベクトル

戻り値

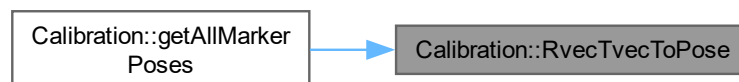
姿勢の変換行列

[Calibration.cpp](#) の 264 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.3.19 saveParameters()

```
bool Calibration::saveParameters (
    const std::string & filename) const
```

キャリブレーションパラメータをファイルに保存する

引数

<i>filename</i>	保存するファイル名
-----------------	-----------

戻り値

パラメータの書き込みに成功したら true

[Calibration.cpp](#) の 433 行目に定義があります。

呼び出し関係図:



8.2.3.20 setDictionary()

```
void Calibration::setDictionary (
    const std::string & dictionaryName,
    const std::array< float, 2 > & length)
```

ArUco Marker の辞書と検出器を設定する

引数

<i>dictionaryName</i>	ArUco Marker の辞書名
<i>length</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 81 行目に定義があります。
呼び出し関係図:



8.2.4 メンバ詳解

8.2.4.1 dictionaryList

```
const std::map< const std::string, const cv::aruco::PredefinedDictionaryType > Calibration←
::dictionaryList [static]
```

初期値:

```
{
    { "DICT_4X4_50", cv::aruco::DICT_4X4_50 },
    { "DICT_4X4_100", cv::aruco::DICT_4X4_100 },
    { "DICT_4X4_250", cv::aruco::DICT_4X4_250 },
    { "DICT_4X4_1000", cv::aruco::DICT_4X4_1000 },
    { "DICT_5X5_50", cv::aruco::DICT_5X5_50 },
    { "DICT_5X5_100", cv::aruco::DICT_5X5_100 },
    { "DICT_5X5_250", cv::aruco::DICT_5X5_250 },
    { "DICT_5X5_1000", cv::aruco::DICT_5X5_1000 },
    { "DICT_6X6_50", cv::aruco::DICT_6X6_50 },
    { "DICT_6X6_100", cv::aruco::DICT_6X6_100 },
    { "DICT_6X6_250", cv::aruco::DICT_6X6_250 },
    { "DICT_6X6_1000", cv::aruco::DICT_6X6_1000 },
    { "DICT_7X7_50", cv::aruco::DICT_7X7_50 },
    { "DICT_7X7_100", cv::aruco::DICT_7X7_100 },
    { "DICT_7X7_250", cv::aruco::DICT_7X7_250 },
    { "DICT_7X7_1000", cv::aruco::DICT_7X7_1000 },
    { "DICT_ARUCO_ORIGINAL", cv::aruco::DICT_ARUCO_ORIGINAL },
    { "DICT_APRILTAG_16h5", cv::aruco::DICT_APRILTAG_16h5 },
    { "DICT_APRILTAG_25h9", cv::aruco::DICT_APRILTAG_25h9 },
    { "DICT_APRILTAG_36h10", cv::aruco::DICT_APRILTAG_36h10 },
    { "DICT_APRILTAG_36h11", cv::aruco::DICT_APRILTAG_36h11 }
}
```

ArUco Marker 辞書のリスト

[Calibration.h](#) の 461 行目に定義があります。

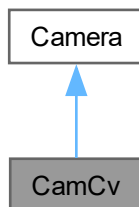
このクラス詳解は次のファイルから抽出されました:

- [Calibration.h](#)
- [Calibration.cpp](#)

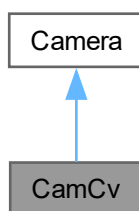
8.3 CamCv クラス

```
#include <CamCv.h>
```

CamCv の継承関係図



CamCv 連携図



公開メンバ関数

- `CamCv ()`
- `virtual ~CamCv ()`
- `auto open (int device, int width=0, int height=0, double fps=0.0, const char *fourcc="", int pref=cv::CAP_ANY)`
- `void close ()`
- `auto open (const std::string &file, int width=0, int height=0, double fps=0.0, const char *fourcc="", int pref=cv::CAP_ANY)`
- `virtual double getFps () const`
- `auto getCodec () const`
- `void getCodec (char *fourcc) const`
- `auto getPosition () const`
- `void setPosition (double frame)`
- `void setExposure (double exposure)`
- `void increaseExposure ()`
- `void decreaseExposure ()`
- `void setGain (double gain)`
- `void increaseGain ()`
- `void decreaseGain ()`

基底クラス **Camera** に属する継承公開メンバ関数

- **Camera** ()
- **Camera** (const **Camera** &camera)=delete
- virtual ~**Camera** ()
- **Camera** & **operator=** (const **Camera** &camera)=delete
- void **start** ()
- void **stop** ()
- void **transmit** (GLuint buffer)
- void **transmit** (std::vector< GLubyte > &buffer)
- template<typename MatType>
void **transmit** (MatType &buffer)
- auto **isRunning** () const
- std::array< int, 2 > **getSize** () const
- auto **getWidth** () const
- auto **getHeight** () const
- auto **getChannels** () const
- auto **getFrames** () const

その他の継承メンバ

基底クラス **Camera** に属する継承公開変数類

- double **in**
ムービーファイルのインポイント
- double **out**
ムービーファイルのアウトポイント

基底クラス **Camera** に属する継承限定公開変数類

- double **total**
ムービーファイルの総フレーム数
- double **interval**
キャプチャした画像のフレーム間隔
- int **width**
解像度 (幅)
- int **height**
解像度 (高さ)
- int **channels**
チャンネル数
- std::vector< GLubyte > **frame**
キャプチャデバイスから取得したフレーム
- std::vector< GLubyte > **image**
キャプチャしたフレームを GPU に送るために用いる一時メモリ
- bool **captured**
新しいフレームが取得されたら *true*
- std::thread **thr**
キャプチャを非同期に行うためのスレッド
- std::mutex **mtx**
キャプチャを非同期に行うためのミューテックス
- bool **running**
キャプチャスレッドが実行中なら *true*

8.3.1 詳解

OpenCV を使ってビデオをキャプチャするクラス
[CamCv.h](#) の 20 行目に定義があります。

8.3.2 構築子と解体子

8.3.2.1 CamCv()

```
CamCv::CamCv () [inline]
```

コンストラクタ

[CamCv.h](#) の 193 行目に定義があります。

8.3.2.2 ~CamCv()

```
virtual CamCv::~~CamCv () [inline], [virtual]
```

デストラクタ

[CamCv.h](#) の 202 行目に定義があります。

8.3.3 関数詳解

8.3.3.1 close()

```
void CamCv::close () [inline], [virtual]
```

キャプチャデバイスの使用を終了する

[Camera](#) を再実装しています。

[CamCv.h](#) の 225 行目に定義があります。

呼び出し関係図:



8.3.3.2 decreaseExposure()

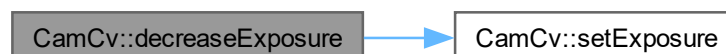
```
void CamCv::decreaseExposure () [inline], [virtual]
```

露出を一段階下げる

[Camera](#) を再実装しています。

[CamCv.h](#) の 327 行目に定義があります。

呼び出し関係図:



8.3.3.3 decreaseGain()

```
void CamCv::decreaseGain () [inline], [virtual]
```

利得を一段階下げる

[Camera](#) を再実装しています。

[CamCv.h](#) の 353 行目に定義があります。

呼び出し関係図:



8.3.3.4 getCodec() [1/2]

```
auto CamCv::getCodec () const [inline]
```

コーデックを調べる

戻り値

使用しているコーデックを表す 4 バイト

[CamCv.h](#) の 265 行目に定義があります。

被呼び出し関係図:



8.3.3.5 getCodec() [2/2]

```
void CamCv::getCodec (
    char * fourcc) const [inline]
```

コーデックを調べる

引数

<i>fourcc</i>	使用しているコーデックを表す 4 文字の格納先
---------------	-------------------------

[CamCv.h](#) の 275 行目に定義があります。

呼び出し関係図:



8.3.3.6 getFps()

```
virtual double CamCv::getFps () const [inline], [virtual]
```

キャプチャしたフレームのフレームレートを得る

戻り値

キャプチャしたフレームのフレームレート

[Camera](#)を再実装しています。

[CamCv.h](#) の 255 行目に定義があります。

8.3.3.7 getPosition()

```
auto CamCv::getPosition () const [inline]
```

ファイルから入力しているとき現在のフレーム番号を得る

戻り値

現在入力しているフレーム番号

[CamCv.h](#) の 291 行目に定義があります。

8.3.3.8 increaseExposure()

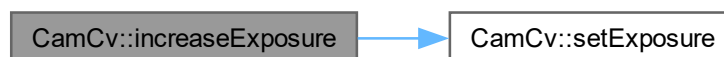
```
void CamCv::increaseExposure () [inline], [virtual]
```

露出を一段階上げる

[Camera](#)を再実装しています。

[CamCv.h](#) の 319 行目に定義があります。

呼び出し関係図:



8.3.3.9 increaseGain()

```
void CamCv::increaseGain () [inline], [virtual]
```

利得を一段階上げる

[Camera](#)を再実装しています。

CamCv.h の 345 行目に定義があります。
呼び出し関係図:



8.3.3.10 open() [1/2]

```
auto CamCv::open (
    const std::string & file,
    int width = 0,
    int height = 0,
    double fps = 0.0,
    const char * fourcc = "",
    int pref = cv::CAP_ANY) [inline]
```

ファイル / ネットワーク / GStreamer から入力する

引数

<i>device</i>	入力するファイルの名前
<i>width</i>	入力するファイルを開く際に期待するフレームの横の画素数, 0 ならお任せ
<i>height</i>	入力するファイルを開く際に期待するフレームの縦の画素数, 0 ならお任せ
<i>fps</i>	入力するファイルを開く際に期待するフレームフレームレート, 0 ならお任せ
<i>fourcc</i>	入力するファイルを開く際に期待するコーデックの 4 文字, "" ならお任せ

戻り値

入力するファイルが使用可能なら true

CamCv.h の 244 行目に定義があります。

8.3.3.11 open() [2/2]

```
auto CamCv::open (
    int device,
    int width = 0,
    int height = 0,
    double fps = 0.0,
    const char * fourcc = "",
    int pref = cv::CAP_ANY) [inline]
```

キャプチャデバイスを開く

引数

<i>device</i>	キャプチャデバイスの番号
<i>width</i>	キャプチャデバイスを開く際に期待するフレームの横の画素数, 0 ならお任せ

<i>height</i>	キャプチャデバイスを開く際に期待するフレームの縦の画素数, 0 ならお任せ
<i>fps</i>	キャプチャデバイスを開く際に期待するフレームフレームレート, 0 ならお任せ
<i>fourcc</i>	キャプチャデバイスを開く際に期待するコーデックの 4 文字, "" ならお任せ

戻り値

キャプチャデバイスが使用可能なら true

[CamCv.h](#) の 216 行目に定義があります。

8.3.3.12 setExposure()

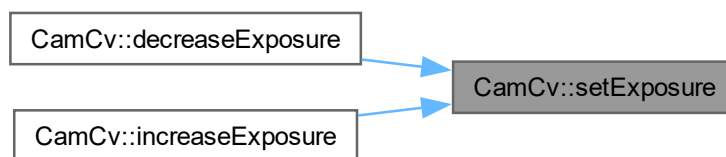
```
void CamCv::setExposure (
    double exposure) [inline]
```

露出を設定する

引数

<i>exposure</i>	設定する露出
-----------------	--------

[CamCv.h](#) の 311 行目に定義があります。
被呼び出し関係図:



8.3.3.13 setGain()

```
void CamCv::setGain (
    double gain) [inline]
```

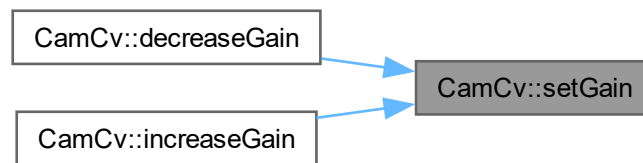
利得を設定する

引数

<i>gain</i>	設定する利得
-------------	--------

[CamCv.h](#) の 337 行目に定義があります。

被呼び出し関係図:



8.3.3.14 setPosition()

```
void CamCv::setPosition (
    double frame) [inline]
```

ファイルから入力しているとき再生位置を指定する

引数

<i>frame</i>	再生位置
--------------	------

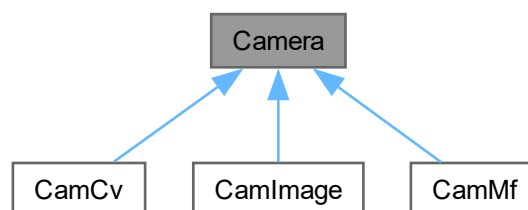
[CamCv.h](#) の 301 行目に定義があります。
このクラス詳解は次のファイルから抽出されました:

- [CamCv.h](#)

8.4 Camera クラス

```
#include <Camera.h>
```

Camera の継承関係図



公開メンバ関数

- [Camera](#) ()
- [Camera](#) (const [Camera](#) &camera)=delete
- virtual ~[Camera](#) ()
- [Camera](#) & operator= (const [Camera](#) &camera)=delete

- void `start` ()
- void `stop` ()
- void `transmit` (GLuint buffer)
- void `transmit` (std::vector< GLubyte > &buffer)
- template<typename MatType>
void `transmit` (MatType &buffer)
- virtual void `close` ()
- auto `isRunning` () const
- std::array< int, 2 > `getSize` () const
- auto `getWidth` () const
- auto `getHeight` () const
- auto `getChannels` () const
- auto `getFrames` () const
- virtual double `getFps` () const
- virtual void `increaseExposure` ()
- virtual void `decreaseExposure` ()
- virtual void `increaseGain` ()
- virtual void `decreaseGain` ()

公開変数類

- double `in`
ムービーファイルのインポイント
- double `out`
ムービーファイルのアウトポイント

限定公開メンバ関数

- virtual void `capture` ()

限定公開変数類

- double `total`
ムービーファイルの総フレーム数
- double `interval`
キャプチャした画像のフレーム間隔
- int `width`
解像度 (幅)
- int `height`
解像度 (高さ)
- int `channels`
チャンネル数
- std::vector< GLubyte > `frame`
キャプチャデバイスから取得したフレーム
- std::vector< GLubyte > `image`
キャプチャしたフレームを GPU に送るために用いる一時メモリ
- bool `captured`
新しいフレームが取得されたら `true`
- std::thread `thr`
キャプチャを非同期に行うためのスレッド
- std::mutex `mtx`
キャプチャを非同期に行うためのミューテックス
- bool `running`
キャプチャスレッドが実行中なら `true`

8.4.1 詳解

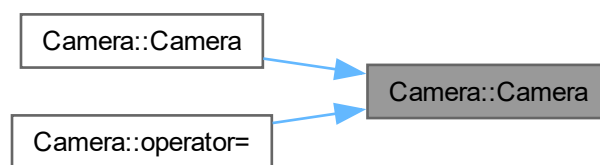
キャプチャデバイス関連の基底クラス
[Camera.h](#) の 22 行目に定義があります。

8.4.2 構築子と解体子

8.4.2.1 Camera() [1/2]

```
Camera::Camera () [inline]  
コンストラクタ
```

[Camera.h](#) の 82 行目に定義があります。
被呼び出し関係図:



8.4.2.2 Camera() [2/2]

```
Camera::Camera (  
    const Camera & camera) [delete]  
コピーコンストラクタは使用しない
```

引数

<i>camera</i>	コピー元
---------------	------

呼び出し関係図:

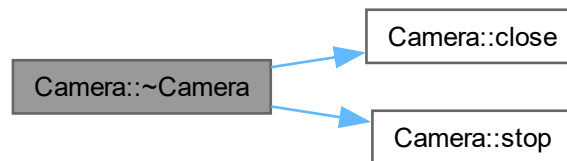


8.4.2.3 ~Camera()

```
virtual Camera::~Camera () [inline], [virtual]  
デストラクタ
```

[Camera.h](#) の 105 行目に定義があります。

呼び出し関係図:



8.4.3 関数詳解

8.4.3.1 capture()

```
virtual void Camera::capture () [inline], [protected], [virtual]
```

フレームをキャプチャする

@description スレッドを起動するための仮想関数。

[CamMf](#)で再実装されています。

[Camera.h](#) の 65 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.4.3.2 close()

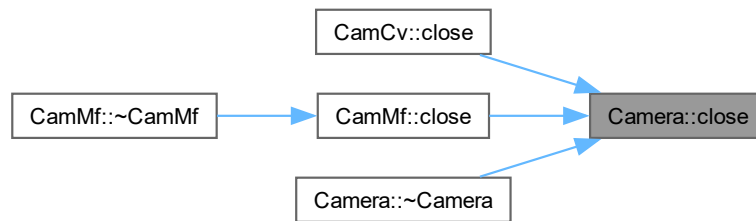
```
virtual void Camera::close () [inline], [virtual]
```

キャプチャデバイスの使用を終了する

[CamCv](#), [CamMf](#)で再実装されています。

[Camera.h](#) の 227 行目に定義があります。

被呼び出し関係図:



8.4.3.3 decreaseExposure()

```
virtual void Camera::decreaseExposure () [inline], [virtual]
```

露出を下げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 313 行目に定義があります。

8.4.3.4 decreaseGain()

```
virtual void Camera::decreaseGain () [inline], [virtual]
```

利得を下げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 327 行目に定義があります。

8.4.3.5 getChannels()

```
auto Camera::getChannels () const [inline]
```

キャプチャしたフレームのチャンネル数を調べる

戻り値

キャプチャしたフレームのチャンネル数

[Camera.h](#) の 278 行目に定義があります。

8.4.3.6 getFps()

```
virtual double Camera::getFps () const [inline], [virtual]
```

キャプチャデバイスのフレームレートを得る

戻り値

キャプチャデバイスのフレームレート

[CamCv](#)で再実装されています。

[Camera.h](#) の 298 行目に定義があります。

8.4.3.7 getFrames()

```
auto Camera::getFrames () const [inline]
```

ムービーファイルの総フレーム数を得る

戻り値

ムービーファイルの総フレーム数

[Camera.h](#) の 288 行目に定義があります。

8.4.3.8 getHeight()

```
auto Camera::getHeight () const [inline]
```

キャプチャしたフレームの縦の画素数を得る

戻り値

キャプチャ中のフレームの縦の画素数

[Camera.h](#) の 268 行目に定義があります。

8.4.3.9 getSize()

```
std::array< int, 2 > Camera::getSize () const [inline]
```

キャプチャしたフレームのサイズを得る

戻り値

キャプチャしたフレームのサイズ

[Camera.h](#) の 248 行目に定義があります。

8.4.3.10 getWidth()

```
auto Camera::getWidth () const [inline]
```

キャプチャしたフレームの横の画素数を得る

戻り値

キャプチャ中のフレームの横の画素数

[Camera.h](#) の 258 行目に定義があります。

8.4.3.11 increaseExposure()

```
virtual void Camera::increaseExposure () [inline], [virtual]
```

露出を上げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 306 行目に定義があります。

8.4.3.12 increaseGain()

```
virtual void Camera::increaseGain () [inline], [virtual]
```

利得を上げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 320 行目に定義があります。

8.4.3.13 isRunning()

```
auto Camera::isRunning () const [inline]
```

キャプチャスレッドが実行中かどうか調べる

戻り値

キャプチャ中なら true

[Camera.h](#) の 238 行目に定義があります。

8.4.3.14 operator=()

```
Camera & Camera::operator= (  
    const Camera & camera) [delete]
```

代入演算子は使用しない

引数

<i>camera</i>	代入元のカメラ
---------------	---------

戻り値

代入後のこのカメラの参照

呼び出し関係図:



8.4.3.15 start()

```
void Camera::start () [inline]
```

キャプチャスレッドを起動する
[Camera.h](#) の 128 行目に定義があります。

呼び出し関係図:

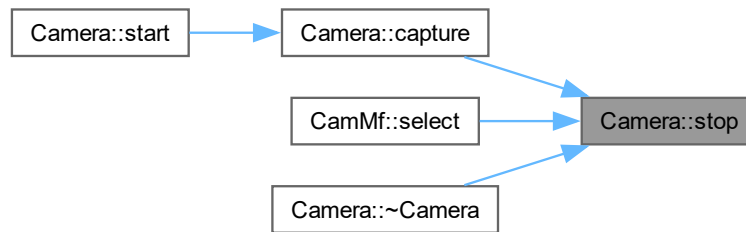


8.4.3.16 stop()

```
void Camera::stop () [inline]
```

キャプチャスレッドを停止する
[Camera.h](#) の 140 行目に定義があります。

被呼び出し関係図:



8.4.3.17 transmit() [1/3]

```
void Camera::transmit (
    GLuint buffer) [inline]
```

キャプチャデバイスをロックしてフレームをピクセルバッファオブジェクトに転送する

引数

<i>buffer</i>	転送先のピクセルバッファオブジェクト
---------------	--------------------

[Camera.h](#) の 158 行目に定義があります。

8.4.3.18 transmit() [2/3]

```
template<typename MatType>
void Camera::transmit (
    MatType & buffer) [inline]
```

キャプチャデバイスをロックしてフレームをメモリに転送する

引数

<i>buffer</i>	転送先のメモリ (cv::Matなど)
---------------	---------------------

[Camera.h](#) の 208 行目に定義があります。

8.4.3.19 transmit() [3/3]

```
void Camera::transmit (
    std::vector< GLubyte > & buffer) [inline]
```

キャプチャデバイスをロックしてフレームをメモリに転送する

引数

<i>buffer</i>	転送先のメモリ
---------------	---------

[Camera.h](#) の 182 行目に定義があります。

8.4.4 メンバ詳解

8.4.4.1 captured

`bool Camera::captured [protected]`
新しいフレームが取得されたら `true`
[Camera.h](#) の 48 行目に定義があります。

8.4.4.2 channels

`int Camera::channels [protected]`
チャンネル数
[Camera.h](#) の 39 行目に定義があります。

8.4.4.3 frame

`std::vector<GLubyte> Camera::frame [protected]`
キャプチャデバイスから取得したフレーム
[Camera.h](#) の 42 行目に定義があります。

8.4.4.4 height

`int Camera::height [protected]`
解像度 (高さ)
[Camera.h](#) の 36 行目に定義があります。

8.4.4.5 image

`std::vector<GLubyte> Camera::image [protected]`
キャプチャしたフレームを GPU に送るために用いる一時メモリ
[Camera.h](#) の 45 行目に定義があります。

8.4.4.6 in

`double Camera::in`
ムービーファイルのインポイント
[Camera.h](#) の 74 行目に定義があります。

8.4.4.7 interval

`double Camera::interval [protected]`
キャプチャした画像のフレーム間隔
[Camera.h](#) の 30 行目に定義があります。

8.4.4.8 mtx

`std::mutex Camera::mtx [protected]`
キャプチャを非同期に行うためのミューテックス
[Camera.h](#) の 54 行目に定義があります。

8.4.4.9 out

`double Camera::out`
ムービーファイルのアウトポイント
[Camera.h](#) の 77 行目に定義があります。

8.4.4.10 running

`bool Camera::running [protected]`
キャプチャスレッドが実行中なら `true`
[Camera.h](#) の 57 行目に定義があります。

8.4.4.11 thr

```
std::thread Camera::thr [protected]
```

キャプチャを非同期に行うためのスレッド
[Camera.h](#) の 51 行目に定義があります。

8.4.4.12 total

```
double Camera::total [protected]
```

ムービーファイルの総フレーム数
[Camera.h](#) の 27 行目に定義があります。

8.4.4.13 width

```
int Camera::width [protected]
```

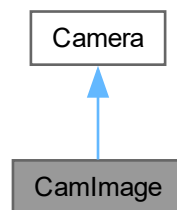
解像度 (幅)
[Camera.h](#) の 33 行目に定義があります。
このクラス詳解は次のファイルから抽出されました:

- [Camera.h](#)

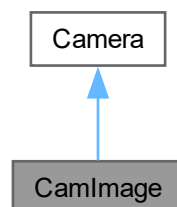
8.5 CamImage クラス

```
#include <CamImage.h>
```

CamImage の継承関係図



CamImage 連携図



公開メンバ関数

- `CamImage ()`
- `CamImage (std::string &filename, bool flip=false)`
- `virtual ~CamImage ()`
- `bool open (const std::string &filename, bool flip=false)`
- `bool isOpened () const`

基底クラス **Camera** に属する継承公開メンバ関数

- `Camera ()`
- `Camera (const Camera &camera)=delete`
- `virtual ~Camera ()`
- `Camera & operator= (const Camera &camera)=delete`
- `void start ()`
- `void stop ()`
- `void transmit (GLuint buffer)`
- `void transmit (std::vector< GLubyte > &buffer)`
- `template<typename MatType>`
`void transmit (MatType &buffer)`
- `virtual void close ()`
- `auto isRunning () const`
- `std::array< int, 2 > getSize () const`
- `auto getWidth () const`
- `auto getHeight () const`
- `auto getChannels () const`
- `auto getFrames () const`
- `virtual double getFps () const`
- `virtual void increaseExposure ()`
- `virtual void decreaseExposure ()`
- `virtual void increaseGain ()`
- `virtual void decreaseGain ()`

静的公開メンバ関数

- `static bool load (const std::string &filename, cv::Mat &frame)`
- `static bool save (const std::string &filename, const cv::Mat &image)`

その他の継承メンバ

基底クラス **Camera** に属する継承公開変数類

- `double in`
ムービーファイルのインポイント
- `double out`
ムービーファイルのアウトポイント

基底クラス **Camera** に属する継承限定公開メンバ関数

- `virtual void capture ()`

基底クラス **Camera** に属する継承限定公開変数類

- `double total`
ムービーファイルの総フレーム数
- `double interval`
キャプチャした画像のフレーム間隔
- `int width`
解像度（幅）
- `int height`
解像度（高さ）
- `int channels`
チャンネル数
- `std::vector< GLubyte > frame`
キャプチャデバイスから取得したフレーム
- `std::vector< GLubyte > image`
キャプチャしたフレームを *GPU* に送るために用いる一時メモリ
- `bool captured`
新しいフレームが取得されたら *true*
- `std::thread thr`
キャプチャを非同期に行うためのスレッド
- `std::mutex mtx`
キャプチャを非同期に行うためのミューテックス
- `bool running`
キャプチャスレッドが実行中なら *true*

8.5.1 詳解

OpenCV を使って画像ファイルを読み込むクラス [CamImage.h](#) の 23 行目に定義があります。

8.5.2 構築子と解体子

8.5.2.1 CamImage() [1/2]

```
CamImage::CamImage () [inline]
コンストラクタ
```

[CamImage.h](#) の 30 行目に定義があります。

8.5.2.2 CamImage() [2/2]

```
CamImage::CamImage (
    std::string & filename,
    bool flip = false) [inline]
```

画像ファイルを開いて読み込むコンストラクタ

引数

<i>filename</i>	画像ファイル名
<i>flip</i>	上下を反転するときは <i>true</i>

[CamImage.h](#) の 40 行目に定義があります。

呼び出し関係図:



8.5.2.3 ~CamImage()

```
virtual CamImage::~CamImage () [inline], [virtual]
デストラクタ
```

[CamImage.h](#) の 48 行目に定義があります。

8.5.3 関数詳解

8.5.3.1 isOpened()

```
bool CamImage::isOpened () const [inline]
画像ファイルが開けたかどうか調べる
```

戻り値

ファイルが開けていたら true

[CamImage.h](#) の 95 行目に定義があります。

8.5.3.2 load()

```
bool CamImage::load (
    const std::string & filename,
    cv::Mat & frame) [inline], [static]
画像ファイルを読み込む
```

引数

<i>filename</i>	読み込む画像ファイルのパス
<i>frame</i>	読み込んだ画像

戻り値

読み込みに成功したら true

[CamImage.h](#) の 108 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.5.3.3 open()

```
bool CamImage::open (
    const std::string & filename,
    bool flip = false) [inline]
```

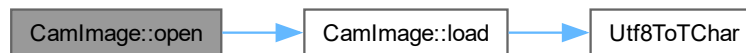
画像ファイルを開く

引数

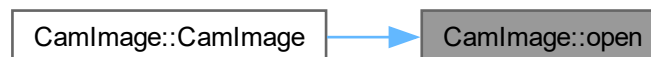
<i>filename</i>	画像ファイル名
<i>flip</i>	上下を反転するときは true

[CamImage.h](#) の 58 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.5.3.4 save()

```
bool CamImage::save (
    const std::string & filename,
    const cv::Mat & image) [inline], [static]
```

配列に格納されている画像をファイルに保存する

引数

<i>filename</i>	保存する画像ファイルのパス
<i>image</i>	保存する画像を保持している配列

戻り値

保存に成功したら true

[CamImage.h](#) の 149 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



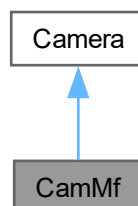
このクラス詳解は次のファイルから抽出されました:

- [CamImage.h](#)

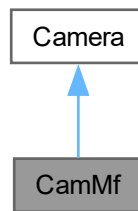
8.6 CamMf クラス

```
#include <CamMf.h>
```

CamMf の継承関係図



CamMf 連携図



公開メンバ関数

- `CamMf ()`
- `virtual ~CamMf ()`
- `bool open (int device, bool setupFormat=true)`
- `const auto & getFormatList () const`
- `bool select (int index)`
- `void capture ()`
- `void close ()`

基底クラス **Camera** に属する継承公開メンバ関数

- `Camera ()`
- `Camera (const Camera &camera)=delete`
- `virtual ~Camera ()`
- `Camera & operator= (const Camera &camera)=delete`
- `void start ()`
- `void stop ()`
- `void transmit (GLuint buffer)`
- `void transmit (std::vector< GLubyte > &buffer)`
- `template<typename MatType>`
`void transmit (MatType &buffer)`
- `auto isRunning () const`
- `std::array< int, 2 > getSize () const`
- `auto getWidth () const`
- `auto getHeight () const`
- `auto getChannels () const`
- `auto getFrames () const`
- `virtual double getFps () const`
- `virtual void increaseExposure ()`
- `virtual void decreaseExposure ()`
- `virtual void increaseGain ()`
- `virtual void decreaseGain ()`

静的公開メンバ関数

- `static const auto & getDeviceList ()`

その他の継承メンバ

基底クラス **Camera** に属する継承公開変数類

- double **in**
ムービーファイルのインポイント
- double **out**
ムービーファイルのアウトポイント

基底クラス **Camera** に属する継承限定公開変数類

- double **total**
ムービーファイルの総フレーム数
- double **interval**
キャプチャした画像のフレーム間隔
- int **width**
解像度 (幅)
- int **height**
解像度 (高さ)
- int **channels**
チャンネル数
- std::vector< GLubyte > **frame**
キャプチャデバイスから取得したフレーム
- std::vector< GLubyte > **image**
キャプチャしたフレームを *GPU* に送るために用いる一時メモリ
- bool **captured**
新しいフレームが取得されたら *true*
- std::thread **thr**
キャプチャを非同期に行うためのスレッド
- std::mutex **mtx**
キャプチャを非同期に行うためのミューテックス
- bool **running**
キャプチャスレッドが実行中なら *true*

8.6.1 詳解

Microsoft Media Foundation を使ってビデオをキャプチャするクラス **CamMf.h** の 25 行目に定義があります。

8.6.2 構築子と解体子

8.6.2.1 CamMf()

```
CamMf::CamMf () [inline]
コンストラクタ
```

CamMf.h の 210 行目に定義があります。

8.6.2.2 ~CamMf()

```
virtual CamMf::~~CamMf () [inline], [virtual]
デストラクタ
```

CamMf.h の 223 行目に定義があります。

呼び出し関係図:



8.6.3 関数詳解

8.6.3.1 capture()

```
void CamMf::capture () [virtual]
```

フレームをキャプチャする（別スレッドでループ実行される）

[Camera](#)を再実装しています。

[CamMf.cpp](#) の 675 行目に定義があります。

8.6.3.2 close()

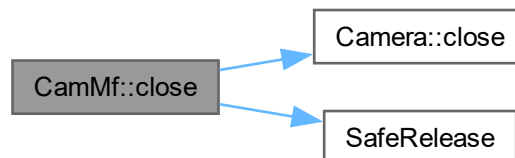
```
void CamMf::close () [virtual]
```

カメラを閉じる

[Camera](#)を再実装しています。

[CamMf.cpp](#) の 908 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.6.3.3 getDeviceList()

```
const auto & CamMf::getDeviceList () [inline], [static]
```

Media Foundation のビデオデバイスの一覧を作る

戻り値

デバイス名のリスト

[CamMf.h](#) の 234 行目に定義があります。

8.6.3.4 getFormatList()

```
const auto & CamMf::getFormatList () const [inline]
```

使用可能なビデオフォーマットの表示名のリストを返す

戻り値

フォーマット名のリスト

[CamMf.h](#) の 253 行目に定義があります。

8.6.3.5 open()

```
bool CamMf::open (
    int device,
    bool setupFormat = true)
```

カメラを開く

引数

<i>device</i>	デバイスの番号
<i>setupFormat</i>	最初のフォーマットを設定するかどうか (遅延初期化時は <code>false</code> を指定)

戻り値

開くことができたなら `true`

[CamMf.cpp](#) の 557 行目に定義があります。
呼び出し関係図:



8.6.3.6 select()

```
bool CamMf::select (
    int index)
```

フォーマットを選択して設定する

引数

<i>index</i>	選択するフォーマットのリストインデックス
--------------	----------------------

戻り値

成功したら true

[CamMf.cpp](#) の 605 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [CamMf.h](#)
- [CamMf.cpp](#)

8.7 Capture クラス

```
#include <Capture.h>
```

公開メンバ関数

- [Capture](#) ()
- [Capture](#) (const std::string &filename)
- virtual [~Capture](#) ()
- bool [openImage](#) (const std::string &filename)
- bool [openMovie](#) (const std::string &filename, cv::VideoCaptureAPIs backend=cv::CAP_FFMPEG)
- bool [openDevice](#) (int deviceNumber, std::array< int, 2 > &size, double &fps, cv::VideoCaptureAPIs backend, char *fourcc)
- void [start](#) ()
- void [stop](#) ()
- void [close](#) ()
- [operator bool](#) () const
- bool [isOpenend](#) () const
- bool [isImage](#) () const
- std::array< int, 2 > [getSize](#) () const
- double [getFps](#) () const
- void [retrieve](#) ([Buffer](#) &buffer)

8.7.1 詳解

キャプチャクラス

[Capture.h](#) の 28 行目に定義があります。

8.7.2 構築子と解体子

8.7.2.1 Capture() [1/2]

`Capture::Capture () [inline]`
キャプチャーオブジェクトのデフォルトコンストラクタ
[Capture.h](#) の 46 行目に定義があります。

8.7.2.2 Capture() [2/2]

`Capture::Capture (`
 `const std::string & filename) [inline]`
キャプチャするファイルを指定するコンストラクタ

引数

<code>filename</code>	キャプチャするファイルのパス名
-----------------------	-----------------

[Capture.h](#) の 58 行目に定義があります。

8.7.2.3 ~Capture()

`virtual Capture::~Capture () [inline], [virtual]`
デストラクタ
[Capture.h](#) の 69 行目に定義があります。
呼び出し関係図:



8.7.3 関数詳解

8.7.3.1 close()

`void Capture::close ()`
キャプチャデバイスを閉じる
[Capture.cpp](#) の 175 行目に定義があります。
被呼び出し関係図:



8.7.3.2 getFps()

```
double Capture::getFps () const
```

キャプチャデバイスのフレームレートを得る

戻り値

キャプチャデバイスのフレームレート

[Capture.cpp](#) の 197 行目に定義があります。

8.7.3.3 getSize()

```
std::array< int, 2 > Capture::getSize () const
```

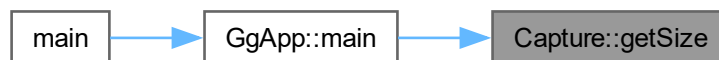
キャプチャデバイスのフレームの解像度を得る

戻り値

キャプチャデバイスのフレームの解像度

[Capture.cpp](#) の 188 行目に定義があります。

被呼び出し関係図:



8.7.3.4 isImage()

```
bool Capture::isImage () const [inline]
```

現在開いているのが静止画像かどうか

[Capture.h](#) の 179 行目に定義があります。

8.7.3.5 isOpenend()

```
bool Capture::isOpenend () const [inline]
```

キャプチャデバイスが有効かどうか

[Capture.h](#) の 171 行目に定義があります。

8.7.3.6 openDevice()

```
bool Capture::openDevice (
    int deviceNumber,
    std::array< int, 2 > & size,
    double & fps,
    cv::VideoCaptureAPIs backend,
    char * fourcc)
```

キャプチャデバイスを開く (Windows以外用: OpenCV)

引数

<i>deviceNumber</i>	開くデバイス番号
---------------------	----------

<i>size</i>	キャプチャデバイスのフレームの解像度
<i>fps</i>	キャプチャデバイスのフレームレート
<i>backend</i>	バックエンドの種類
<i>fourcc</i>	コーデックの 4 文字

戻り値

開くことができたなら `true`

[Capture.cpp](#) の 126 行目に定義があります。

8.7.3.7 openImage()

```
bool Capture::openImage (
    const std::string & filename)
```

画像ファイルを開く

引数

開く画像ファイル名	
-----------	--

戻り値

開くことができたなら `true`

[Capture.cpp](#) の 18 行目に定義があります。

被呼び出し関係図:



8.7.3.8 openMovie()

```
bool Capture::openMovie (
    const std::string & filename,
    cv::VideoCaptureAPIs backend = cv::CAP_FFMPEG)
```

動画ファイルを開く

引数

<i>filename</i>	開く動画ファイル名
<i>backend</i>	バックエンドの種類

戻り値

開くことができたなら `true`

[Capture.cpp](#) の 38 行目に定義があります。

8.7.3.9 operator bool()

```
Capture::operator bool () const [inline], [explicit]
```

キャプチャ中か否か

戻り値

キャプチャ中なら true

[Capture.h](#) の 163 行目に定義があります。

8.7.3.10 retrieve()

```
void Capture::retrieve (
    Buffer & buffer)
```

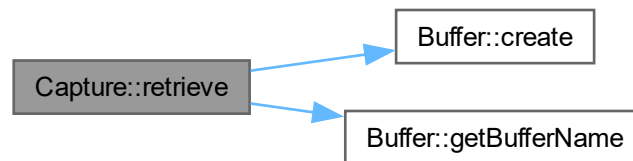
フレームを取得する

引数

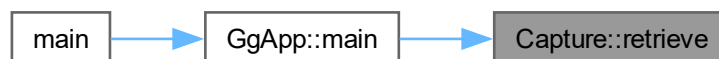
<i>buffer</i>	取得したフレームを格納するバッファ
---------------	-------------------

[Capture.cpp](#) の 206 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.7.3.11 start()

```
void Capture::start ()
```

キャプチャ開始

[Capture.cpp](#) の 157 行目に定義があります。

8.7.3.12 stop()

```
void Capture::stop ()
```

キャプチャ終了

[Capture.cpp](#) の 166 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Capture.h](#)
- [Capture.cpp](#)

8.8 Compute クラス

```
#include <Compute.h>
```

公開メンバ関数

- [Compute](#) (const char *comp)
- virtual [~Compute](#) ()
- auto [get](#) () const
- void [use](#) () const
- void [execute](#) (GLuint width, GLuint height, GLuint local_size_x=1, GLuint local_size_y=1) const

8.8.1 詳解

画像処理クラスの定義 (コンピュートシェーダ版)

[Compute.h](#) の 18 行目に定義があります。

8.8.2 構築子と解体子

8.8.2.1 Compute()

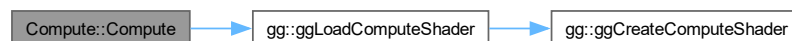
```
Compute::Compute (
    const char * comp) [inline]
コンストラクタ
```

引数

<i>comp</i>	コンピュートシェーダのソースファイル名
-------------	---------------------

[Compute.h](#) の 30 行目に定義があります。

呼び出し関係図:



8.8.2.2 ~Compute()

```
virtual Compute::~~Compute () [inline], [virtual]
デストラクタ
```

[Compute.h](#) の 38 行目に定義があります。

8.8.3 関数詳解

8.8.3.1 execute()

```
void Compute::execute (
    GLuint width,
    GLuint height,
    GLuint local_size_x = 1,
    GLuint local_size_y = 1) const [inline]
```

計算を実行する

引数

<i>width</i>	画像の横の画素数
<i>height</i>	画像の縦の画素数
<i>local_size_x</i>	ワークグループの横方向のスレッド数
<i>local_size_y</i>	ワークグループの縦方向のスレッド数

[Compute.h](#) の 70 行目に定義があります。

8.8.3.2 get()

```
auto Compute::get () const [inline]
```

シェーダプログラムを得る

戻り値

シェーダのプログラムオブジェクト

[Compute.h](#) の 49 行目に定義があります。

8.8.3.3 use()

```
void Compute::use () const [inline]
```

計算用のシェーダプログラムの使用を開始する
[Compute.h](#) の 57 行目に定義があります。
 このクラス詳解は次のファイルから抽出されました:

- [Compute.h](#)

8.9 Config クラス

```
#include <Config.h>
```

公開メンバ関数

- [Config](#) (const std::string &filename)
- virtual ~[Config](#) ()
- void [initialize](#) ()
- bool [load](#) (const [pathString](#) &filename)
- bool [save](#) (const [pathString](#) &filename) const
- const auto & [getTitle](#) () const
- auto [getWidth](#) () const
- auto [getHeight](#) () const
- const auto & [getInitialImage](#) () const

- const auto & [getDictionaryName](#) () const
- const auto & [getCheckerLength](#) () const
- auto [getMarkerLength](#) () const

フレンド

- class [Menu](#)

プライベートメンバは [Menu](#) クラスで設定する

8.9.1 詳解

構成データクラス

[Config.h](#) の 75 行目に定義があります。

8.9.2 構築子と解体子

8.9.2.1 Config()

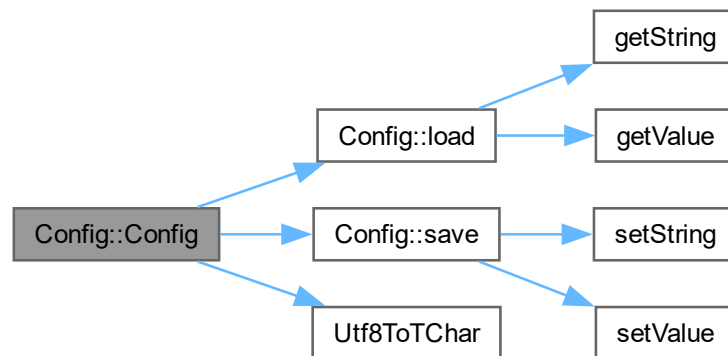
```
Config::Config (
    const std::string & filename)
コンストラクタ
```

引数

<i>filename</i>	構成データの初期化に用いる JSON 形式のファイル名
-----------------	-----------------------------

[Config.cpp](#) の 26 行目に定義があります。

呼び出し関係図:



8.9.2.2 ~Config()

```
Config::~Config () [virtual]
デストラクタ
```

[Config.cpp](#) の 73 行目に定義があります。

8.9.3 関数詳解

8.9.3.1 getCheckerLength()

```
const auto & Config::getCheckerLength () const [inline]
```

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さを得る

戻り値

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さ

[Config.h](#) の 204 行目に定義があります。

被呼び出し関係図:



8.9.3.2 getDictionaryName()

```
const auto & Config::getDictionaryName () const [inline]
```

使用中の ArUco Marker の辞書名を得る

戻り値

使用中の ArUco Marker の辞書名

[Config.h](#) の 194 行目に定義があります。

被呼び出し関係図:



8.9.3.3 getHeight()

```
auto Config::getHeight () const [inline]
```

ウィンドウの高さを得る

戻り値

ウィンドウの高さ

[Config.h](#) の 174 行目に定義があります。

被呼び出し関係図:



8.9.3.4 getInitialImage()

```
const auto & Config::getInitialImage () const [inline]
```

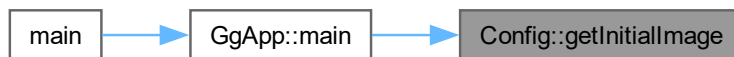
初期画像ファイル名を得る

戻り値

初期画像ファイル名

[Config.h](#) の 184 行目に定義があります。

被呼び出し関係図:



8.9.3.5 getMarkerLength()

```
auto Config::getMarkerLength () const [inline]
```

検出する ArUco Marker の一辺の長さを得る

戻り値

検出する ArUco Marker の一辺の長さ

[Config.h](#) の 214 行目に定義があります。

8.9.3.6 getTitle()

```
const auto & Config::getTitle () const [inline]
```

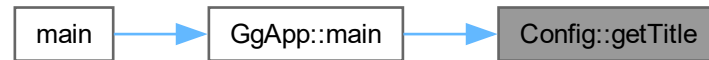
ウィンドウタイトルの文字列を得る

戻り値

ウィンドウのタイトル文字列

[Config.h](#) の 154 行目に定義があります。

被呼び出し関係図:



8.9.3.7 getWidth()

```
auto Config::getWidth () const [inline]
```

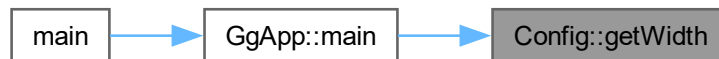
ウィンドウの横幅を得る

戻り値

ウィンドウの横幅

[Config.h](#) の 164 行目に定義があります。

被呼び出し関係図:



8.9.3.8 initialize()

```
void Config::initialize ()
```

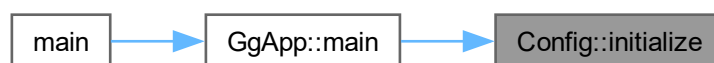
構成データを初期化する

覚え書き

OpenGL のレンダリングコンテキスト作成後に実行する

[Config.cpp](#) の 80 行目に定義があります。

被呼び出し関係図:



8.9.3.9 load()

```
bool Config::load (  
    const pathString & filename)
```

構成ファイルを読み込む

引数

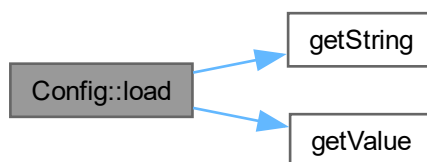
<i>filename</i>	構成データの JSON 形式のファイル名
-----------------	----------------------

戻り値

構成ファイルの読み込みに成功したら true

[Config.cpp](#) の 92 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.9.3.10 save()

```
bool Config::save (  
    const pathString & filename) const
```

構成ファイルを保存する

引数

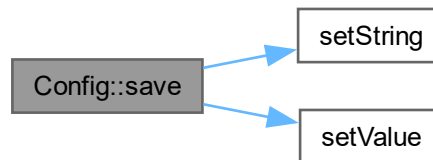
<i>filename</i>	構成データの JSON 形式のファイル名
-----------------	----------------------

戻り値

構成ファイルの保存に成功したら true

[Config.cpp](#) の 159 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.9.4 フレンドと関連関数の詳解

8.9.4.1 Menu

```
friend class Menu [friend]
```

プライベートメンバは [Menu](#) クラスで設定する

[Config.h](#) の 78 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Config.h](#)
- [Config.cpp](#)
- [Menu.cpp](#)

8.10 Expand クラス

```
#include <Expand.h>
```

公開メンバ関数

- [Expand](#) (const std::string &vert, const std::string &frag)
- [Expand](#) (const [Expand](#) &shader)=delete
- virtual ~[Expand](#) ()
- [Expand](#) & operator= (const [Expand](#) &shader)=delete
- auto [getProgram](#) () const
- std::array< GLsizei, 2 > [setup](#) (int samples, GLfloat aspect, const [gg::GgMatrix](#) &pose, const std::array< GLfloat, 2 > &fov, const std::array< GLfloat, 2 > ¢er, GLfloat focal, const std::array< GLfloat, 4 > &border, int unit=0) const

8.10.1 詳解

展開用シェーダクラス

[Expand.h](#) の 17 行目に定義があります。

8.10.2 構築子と解体子

8.10.2.1 Expand() [1/2]

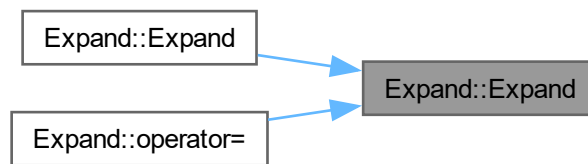
```
Expand::Expand (
    const std::string & vert,
    const std::string & frag)
コンストラクタ
```

引数

<i>vert</i>	バーテックスシェーダのソースファイル名
<i>frag</i>	フラグメントシェーダのソースファイル名

[Expand.cpp](#) の 12 行目に定義があります。

被呼び出し関係図:



8.10.2.2 Expand() [2/2]

```
Expand::Expand (
    const Expand & shader) [delete]
コピーコンストラクタは使用しない
```

引数

<i>shader</i>	コピー元のシェーダ
---------------	-----------

呼び出し関係図:



8.10.2.3 ~Expand()

```
Expand::~~Expand () [virtual]
```

デストラクタ

[Expand.cpp](#) の 29 行目に定義があります。

8.10.3 関数詳解

8.10.3.1 getProgram()

```
auto Expand::getProgram () const [inline]
```

展開用シェーダのプログラムオブジェクトを得る

戻り値

展開用シェーダのプログラムオブジェクト

[Expand.h](#) の 78 行目に定義があります。

8.10.3.2 operator=()

```
Expand & Expand::operator= (
    const Expand & shader) [delete]
```

代入演算子は使用しない

引数

<i>shader</i>	代入元のシェーダ
---------------	----------

戻り値

代入後のこのシェーダの参照

呼び出し関係図:



8.10.3.3 setup()

```
std::array< int, 2 > Expand::setup (
    int samples,
    GLfloat aspect,
    const gg::GgMatrix & pose,
    const std::array< GLfloat, 2 > & fov,
    const std::array< GLfloat, 2 > & center,
    GLfloat focal,
    const std::array< GLfloat, 4 > & border,
    int unit = 0) const
```

展開

引数

<i>samples</i>	展開するテクスチャをサンプリングする数
<i>aspect</i>	展開するテクスチャの縦横比
<i>pose</i>	サンプリングに用いるカメラの姿勢
<i>fov</i>	サンプリングに用いるカメラの相対画角 (単位は度)
<i>center</i>	サンプリングに用いるカメラの撮像面上の中心 (主点) 位置 @oaram focal サンプリングに用いるカメラの主点とスクリーンの距離
<i>border</i>	展開後のフレームの境界色
<i>unit</i>	テクスチャユニット番号

戻り値

描画すべきメッシュの横と縦の格子点数

覚え書き

展開するテクスチャをマッピングするメッシュの解像度を *samples* と *aspect* から個々のメッシュが正方形に近くなるよう決定し、それをもとに格子の間隔を求める

展開

[Expand.cpp](#) の 37 行目に定義があります。

呼び出し関係図:



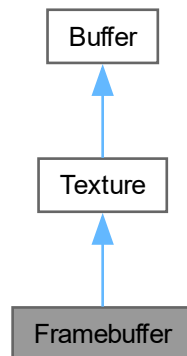
このクラス詳解は次のファイルから抽出されました:

- [Expand.h](#)
- [Expand.cpp](#)

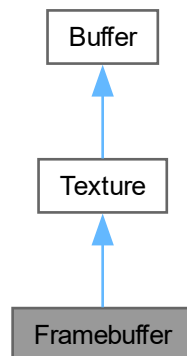
8.11 Framebuffer クラス

```
#include <Framebuffer.h>
```

Framebuffer の継承関係図



Framebuffer 連携図



公開メンバ関数

- `Framebuffer ()`
- `Framebuffer (GLsizei width, GLsizei height, int channels=3, GLenum attachment=GL_COLOR_ATTACHMENT0)`
- `Framebuffer (const Framebuffer &framebuffer)`
- `Framebuffer (Framebuffer &&framebuffer) noexcept`
- `virtual ~Framebuffer ()`
- `Framebuffer & operator= (const Framebuffer &framebuffer)`
- `Framebuffer & operator= (Framebuffer &&framebuffer) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `virtual void copy (const Buffer &framebuffer) noexcept`
- `virtual void discard ()`
- `auto getFramebufferName () const`

- virtual const std::array< GLsizei, 2 > & [getSize](#) () const
- virtual int [getChannels](#) () const
- void [bindFramebuffer](#) ()
- void [unbindFramebuffer](#) () const
- void [update](#) (const std::array< int, 2 > &size)
- void [update](#) (const std::array< int, 2 > &size, const [Texture](#) &frame, int unit=0)
- void [show](#) (GLsizei width, GLsizei height) const

基底クラス [Texture](#) に属する継承公開メンバ関数

- [Texture](#) ()
- [Texture](#) (GLsizei width, GLsizei height, int channels)
- [Texture](#) (const [Texture](#) &texture)
- [Texture](#) ([Texture](#) &&texture) noexcept
- virtual ~[Texture](#) ()
- [Texture](#) & [operator=](#) (const [Texture](#) &texture)
- [Texture](#) & [operator=](#) ([Texture](#) &&texture) noexcept
- auto [getTextureName](#) () const
- void [bindTexture](#) (int unit=0) const
- void [unbindTexture](#) () const
- void [draw](#) (GLsizei width, GLsizei height, int unit=0) const
- void [readPixels](#) (GLuint buffer) const
- void [readPixels](#) ([Buffer](#) &buffer) const
- void [readPixels](#) () const
- void [drawPixels](#) (GLuint buffer) const
- void [drawPixels](#) (const [Texture](#) &texture)
- void [drawPixels](#) (const [Buffer](#) &buffer)
- void [drawPixels](#) () const

基底クラス [Buffer](#) に属する継承公開メンバ関数

- [Buffer](#) ()
- [Buffer](#) (GLsizei width, GLsizei height, int channels)
- [Buffer](#) (const [Buffer](#) &buffer)
- [Buffer](#) ([Buffer](#) &&buffer) noexcept
- virtual ~[Buffer](#) ()
- [Buffer](#) & [operator=](#) (const [Buffer](#) &buffer)
- [Buffer](#) & [operator=](#) ([Buffer](#) &&buffer) noexcept
- void [resize](#) (GLsizei width, GLsizei height, int channels)
- void [resize](#) (const [Buffer](#) &buffer)
- auto [getBufferName](#) () const
- GLsizei [getWidth](#) () const
- GLsizei [getHeight](#) () const
- auto [getFormat](#) () const
- auto [getAspect](#) () const
- void [bindBuffer](#) (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void [unbindBuffer](#) (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * [map](#) () const
- void [unmap](#) () const

その他の継承メンバ

基底クラス [Buffer](#) に属する継承限定公開メンバ関数

- auto [channelsToFormat](#) (int channels) const
- int [formatToChannels](#) (GLenum format) const
- void [copyBuffer](#) (const [Buffer](#) &buffer) noexcept

基底クラス **Texture** に属する継承静的限定公開変数類

- static std::shared_ptr< **Mesh** > **mesh**
テクスチャ展開に用いるメッシュの頂点配列オブジェクト

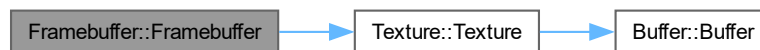
8.11.1 詳解

フレームバッファオブジェクトクラス
Framebuffer.h の 17 行目に定義があります。

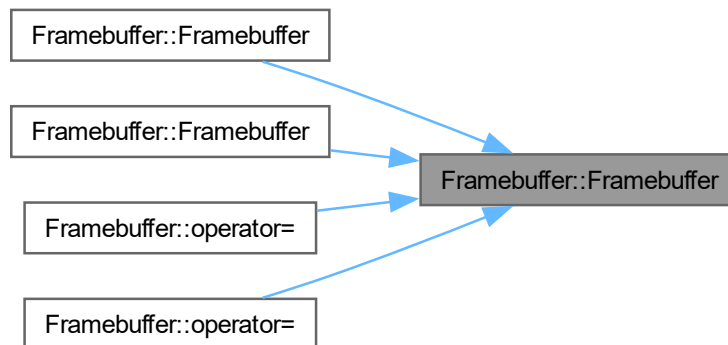
8.11.2 構築子と解体子

8.11.2.1 Framebuffer() [1/4]

Framebuffer::Framebuffer () [inline]
デフォルトコンストラクタ
Framebuffer.h の 36 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.11.2.2 Framebuffer() [2/4]

```

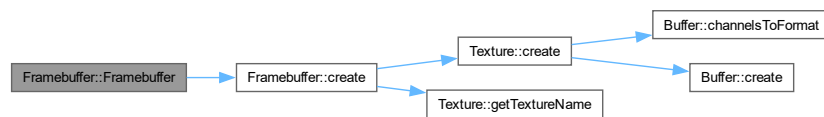
Framebuffer::Framebuffer (
    GLsizei width,
    GLsizei height,
    int channels = 3,
    GLenum attachment = GL_COLOR_ATTACHMENT0)
  
```

フレームバッファオブジェクトを作成するコンストラクタ

引数

<i>width</i>	作成するフレームバッファオブジェクトの横の画素数
<i>height</i>	作成するフレームバッファオブジェクトの縦の画素数
<i>channels</i>	作成するフレームバッファオブジェクトのチャンネル数
<i>attachment</i>	フレームバッファオブジェクトのカラーバッファのアタッチメント

Framebuffer.cpp の 13 行目に定義があります。
呼び出し関係図:



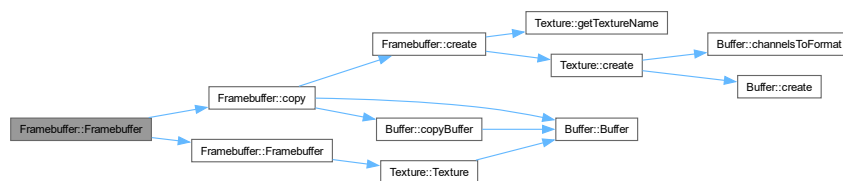
8.11.2.3 Framebuffer() [3/4]

```
Framebuffer::Framebuffer (
    const Framebuffer & framebuffer)
コピーコンストラクタ
```

引数

<i>framebuffer</i>	コピー元のフレームバッファオブジェクト
--------------------	---------------------

Framebuffer.cpp の 26 行目に定義があります。
呼び出し関係図:



8.11.2.4 Framebuffer() [4/4]

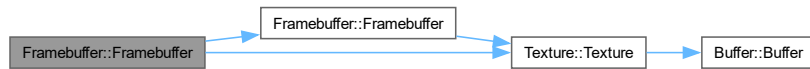
```
Framebuffer::Framebuffer (
    Framebuffer && framebuffer) [inline], [noexcept]
ムーブコンストラクタ
```

引数

<i>framebuffer</i>	ムーブ元のフレームバッファオブジェクト
--------------------	---------------------

Framebuffer.h の 68 行目に定義があります。

呼び出し関係図:



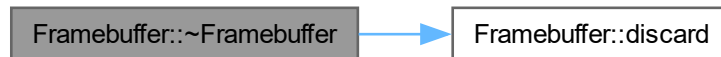
8.11.2.5 ~Framebuffer()

```
Framebuffer::~Framebuffer () [virtual]
```

デストラクタ

[Framebuffer.cpp](#) の 36 行目に定義があります。

呼び出し関係図:



8.11.3 関数詳解

8.11.3.1 bindFramebuffer()

```
void Framebuffer::bindFramebuffer ()
```

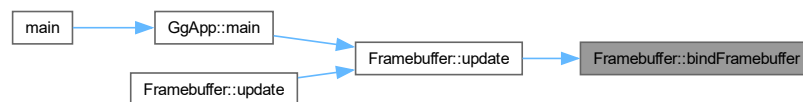
レンダリング先をフレームバッファオブジェクトに切り替える

覚え書き

この後から [unbindFramebuffer\(\)](#) の前まで、レンダリング先がフレームバッファオブジェクトになる。この時点のビューポートを保存する。この間でレンダリング処理を実行する。

[Framebuffer.cpp](#) の 154 行目に定義があります。

被呼び出し関係図:



8.11.3.2 copy()

```
void Framebuffer::copy (
```

```
    const Buffer & framebuffer) [virtual], [noexcept]
```

フレームバッファオブジェクトをコピーする

引数

<i>framebuffer</i>	コピー元のフレームバッファオブジェクト
--------------------	---------------------

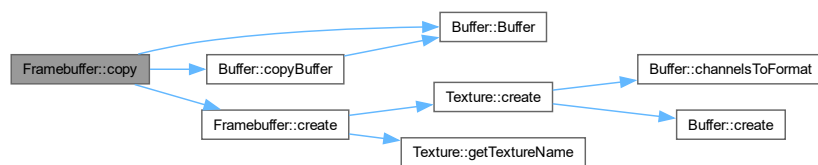
覚え書き

このフレームバッファオブジェクトのサイズがコピー元と異なれば、このフレームバッファオブジェクトを削除して、新しいフレームバッファオブジェクトを作り直してコピーする。
 引数の型が `Buffer` なのは、このオブジェクトの型が `Buffer` のとき `Buffer::copy()`、`Texture` のとき `Texture::copy()`、`Framebuffer` のとき `Framebuffer::copy()` を呼ぶようにするため。

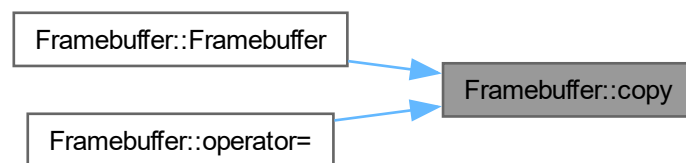
`Texture` を再実装しています。

`Framebuffer.cpp` の 141 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.11.3.3 create()

```
void Framebuffer::create (
    GLsizei width,
    GLsizei height,
    int channels) [virtual]
フレームバッファオブジェクトを作成する
```

引数

<i>width</i>	作成するフレームバッファオブジェクトの横の画素数
<i>height</i>	作成するフレームバッファオブジェクトの縦の画素数
<i>channels</i>	作成するフレームバッファオブジェクトのチャンネル数

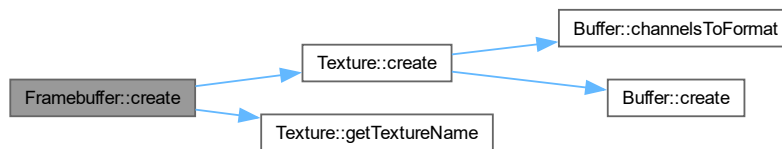
覚え書き

このフレームバッファオブジェクトのサイズが引数で指定したサイズと異なれば、このフレームバッファオブジェクトを削除して、新しいフレームバッファオブジェクトを作り直す。

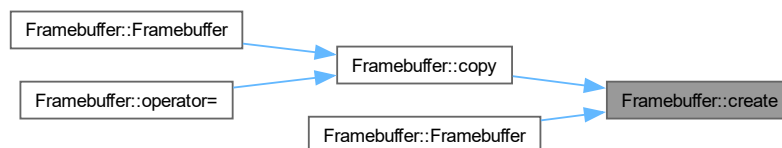
[Texture](#)を再実装しています。

[Framebuffer.cpp](#) の 110 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.11.3.4 discard()

```
void Framebuffer::discard () [virtual]
```

フレームバッファオブジェクトを破棄する

[Texture](#)を再実装しています。

[Framebuffer.cpp](#) の 92 行目に定義があります。

被呼び出し関係図:



8.11.3.5 getChannels()

```
virtual int Framebuffer::getChannels () const [inline], [virtual]
```

フレームバッファオブジェクトのチャンネル数を得る

戻り値

フレームバッファオブジェクトのチャンネル数

[Texture](#)を再実装しています。

[Framebuffer.h](#) の 163 行目に定義があります。

被呼び出し関係図:



8.11.3.6 getFramebufferName()

```
auto Framebuffer::getFramebufferName () const [inline]
```

フレームバッファオブジェクト名を得る

戻り値

フレームバッファオブジェクト名

[Framebuffer.h](#) の 143 行目に定義があります。

8.11.3.7 getSize()

```
virtual const std::array< GLsizei, 2 > & Framebuffer::getSize () const [inline], [virtual]
```

フレームバッファオブジェクトのサイズを得る

戻り値

フレームバッファオブジェクトのサイズ

[Texture](#)を再実装しています。

[Framebuffer.h](#) の 153 行目に定義があります。

8.11.3.8 operator=() [1/2]

```
Framebuffer & Framebuffer::operator= (
    const Framebuffer & framebuffer)
```

代入演算子

引数

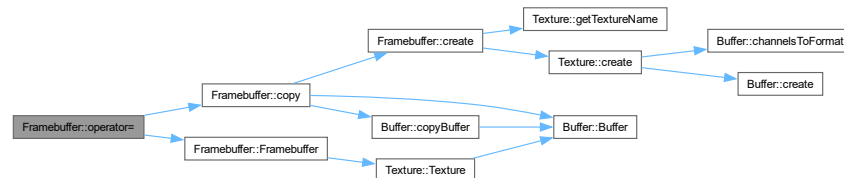
<i>framebuffer</i>	代入元のフレームバッファオブジェクト
--------------------	--------------------

戻り値

代入結果のテクスチャ

[Framebuffer.cpp](#) の 45 行目に定義があります。

呼び出し関係図:



8.11.3.9 operator=() [2/2]

```

Framebuffer & Framebuffer::operator= (
    Framebuffer && framebuffer) [noexcept]

```

ムーブ代入演算子

引数

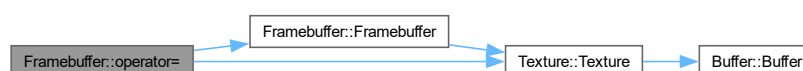
<i>framebuffer</i>	ムーブ代入元のフレームバッファオブジェクト
--------------------	-----------------------

戻り値

ムーブ代入結果のフレームバッファオブジェクト

[Framebuffer.cpp](#) の 61 行目に定義があります。

呼び出し関係図:



8.11.3.10 show()

```

void Framebuffer::show (
    GLsizei width,
    GLsizei height) const

```

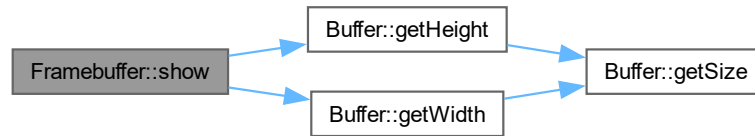
フレームバッファオブジェクトの内容を表示する

引数

<i>width</i>	フレームバッファオブジェクトの内容を表示する横の画素数
<i>height</i>	フレームバッファオブジェクトの内容を表示する縦の画素数

[Framebuffer.cpp](#) の 216 行目に定義があります。

呼び出し関係図:



8.11.3.11 unbindFramebuffer()

```
void Framebuffer::unbindFramebuffer () const
```

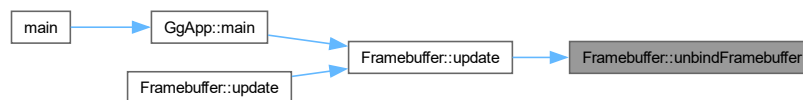
レンダリング先を通常のフレームバッファに戻す

覚え書き

`bindFramebuffer()` の後からこの前まで、レンダリング先がフレームバッファオブジェクトになる。保存したビューポートはここで復帰する。この間でレンダリング処理を実行する。

`Framebuffer.cpp` の 166 行目に定義があります。

被呼び出し関係図:



8.11.3.12 update() [1/2]

```
void Framebuffer::update (
    const std::array< int, 2 > & size)
```

テクスチャを展開してフレームバッファオブジェクトを更新する

引数

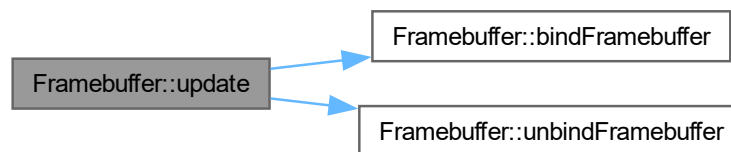
<code>size</code>	展開に用いるメッシュの分割数
-------------------	----------------

覚え書き

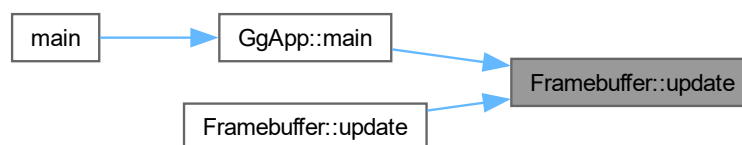
レンダリング先をフレームバッファオブジェクトに切り替えて、フレームバッファオブジェクト全体を覆うメッシュを描画する。これより前でシェードの選択やテクスチャの結合を行う。

[Framebuffer.cpp](#) の 186 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.11.3.13 update() [2/2]

```
void Framebuffer::update (
    const std::array< int, 2 > & size,
    const Texture & frame,
    int unit = 0)
```

テクスチャを展開してフレームバッファオブジェクトを更新する

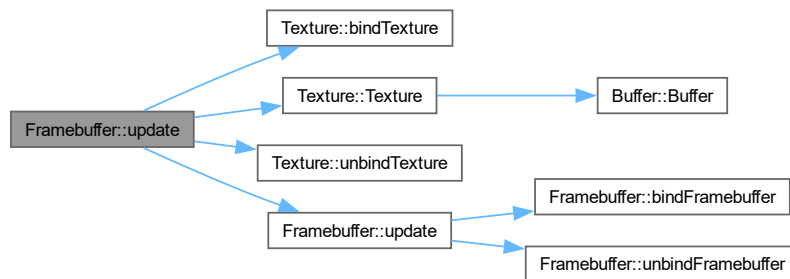
引数

<i>size</i>	展開に用いるメッシュの分割数
<i>frame</i>	展開するフレームを格納したテクスチャ
<i>unit</i>	テクスチャのマッピングに使うテクスチャユニットの番号

覚え書き

レンダリング先をフレームバッファオブジェクトに切り替えて、テクスチャユニット `unit` を指定して `frame` に格納されたテクスチャを結合してから、フレームバッファオブジェクト全体を覆うメッシュを描画する。これより前でシェードの選択を行う。 `unit` にはシェードにおいてテクスチャのサンプラの `uniform` 変数に設定する `GL_TEXTUREi` の `i` と一致させること。

[Framebuffer.cpp](#) の 201 行目に定義があります。
呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Framebuffer.h](#)
- [Framebuffer.cpp](#)

8.12 GgApp クラス

```
#include <GgApp.h>
```

クラス

- class [Window](#)

公開メンバ関数

- [GgApp](#) (int major=0, int minor=1)
- [GgApp](#) (const [GgApp](#) &w)=delete
- [GgApp](#) ([GgApp](#) &&w)=default
- virtual [~GgApp](#) ()
- [GgApp](#) & operator= (const [GgApp](#) &w)=delete
- [GgApp](#) & operator= ([GgApp](#) &&w)=default
- int [main](#) (int argc, const char *const *argv)

8.12.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラス。
[GgApp.h](#) の 88 行目に定義があります。

8.12.2 構築子と解体子

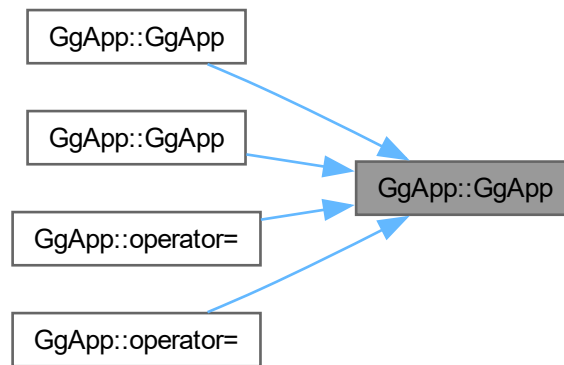
8.12.2.1 GgApp() [1/3]

```
GgApp::GgApp (
    int major = 0,
    int minor = 1)
コンストラクタ.
```

引数

<i>major</i>	使用する OpenGL の major 番号, 0 なら無指定.
<i>minor</i>	使用する OpenGL の minor 番号, major 番号が 0 なら無視.

[GgApp.cpp](#) の 48 行目に定義があります。
被呼び出し関係図:



8.12.2.2 GgApp() [2/3]

```
GgApp::GgApp (
    const GgApp & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のオブジェクト.
----------	--------------

呼び出し関係図:



8.12.2.3 GgApp() [3/3]

```
GgApp::GgApp (
    GgApp && w) [default]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ元のオブジェクト.
----------	--------------

呼び出し関係図:



8.12.2.4 ~GgApp()

`GgApp::~~GgApp () [virtual]`

デストラクタ.

[GgApp.cpp](#) の 95 行目に定義があります。

8.12.3 関数詳解

8.12.3.1 main()

```
int GgApp::main (  
    int argc,  
    const char *const * argv)
```

アプリケーション本体.

引数

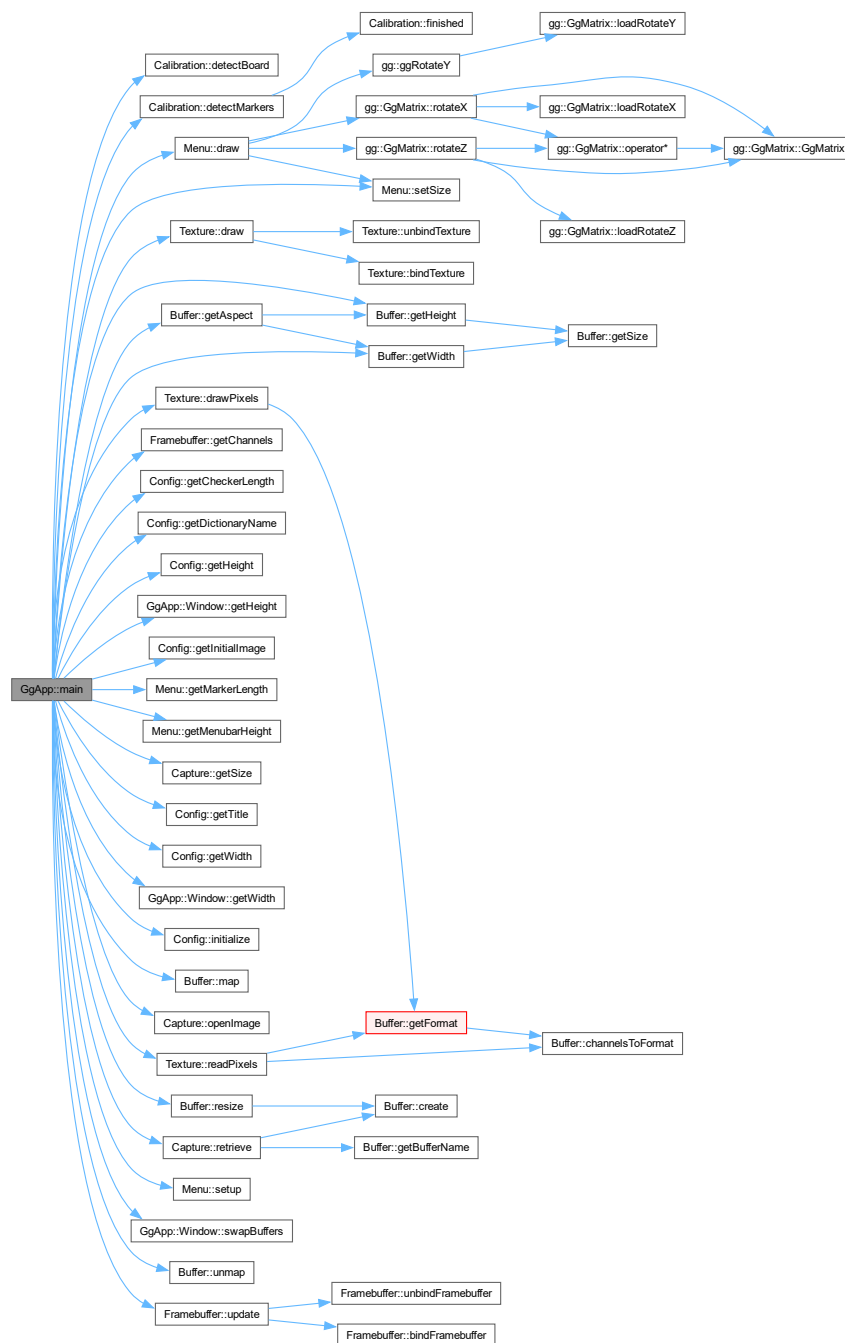
<i>argc</i>	コマンドライン引数の数.
<i>argv</i>	コマンドライン引数の配列.

戻り値

アプリケーションの終了ステータス。

`calib.cpp` の 33 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.12.3.2 operator=() [1/2]

```
GgApp & GgApp::operator= (
    const GgApp & w) [delete]
```

代入演算子は使用しない.

引数

<i>w</i>	代入元のオブジェクト.
----------	-------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



8.12.3.3 operator=() [2/2]

```
GgApp & GgApp::operator= (
    GgApp && w) [default]
```

ムーブ代入演算子.

引数

<i>w</i>	ムーブ代入元のオブジェクト.
----------	----------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [GgApp.h](#)
- [calib.cpp](#)
- [GgApp.cpp](#)

8.13 gg::GgBuffer< T > クラステンプレート

```
#include <gg.h>
```

公開メンバ関数

- [GgBuffer](#) (GLenum target, const T *data, GLsizei stride, GLsizei count, GLenum usage)
- [GgBuffer](#) (const GgBuffer< T > &buffer)=delete
- [GgBuffer](#) (GgBuffer< T > &&buffer)=default
- virtual [~GgBuffer](#) ()
- [GgBuffer](#)< T > & [operator=](#) (const [GgBuffer](#)< T > &buffer)=delete
- [GgBuffer](#)< T > & [operator=](#) ([GgBuffer](#)< T > &&buffer)=default
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLuint first, GLsizei count) const
- void [unmap](#) () const
- void [send](#) (const T *data, GLuint first, GLsizei count) const
- void [read](#) (T *data, GLuint first, GLsizei count) const
- void [copy](#) (GLuint src.buffer, GLuint src.first=0, GLuint dst.first=0, GLsizei count=0) const

8.13.1 詳解

```
template<typename T>
class gg::GgBuffer< T >
```

バッファオブジェクト.

覚え書き

頂点属性／頂点インデックス／ユニフォーム変数を格納するバッファオブジェクトの基底クラス.

[gg.h](#) の 5571 行目に定義があります。

8.13.2 構築子と解体子

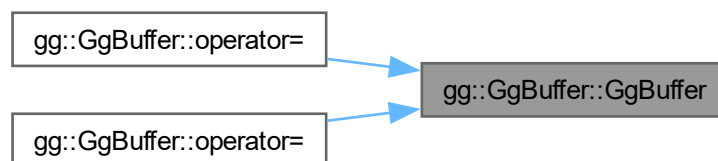
8.13.2.1 GgBuffer() [1/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GLenum target,
    const T * data,
    GLsizei stride,
    GLsizei count,
    GLenum usage) [inline]
コンストラクタ.
```

引数

<i>target</i>	バッファオブジェクトのターゲット.
<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>stride</i>	データの間隔.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5583 行目に定義があります。
被呼び出し関係図:



8.13.2.2 GgBuffer() [2/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    const GgBuffer< T > & buffer) [delete]
コピーコンストラクタは使用しない.
```

引数

<i>buffer</i>	コピー元のバッファ.
---------------	------------

8.13.2.3 GgBuffer() [3/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GgBuffer< T > && buffer) [default]
ムーブコンストラクタ.
```

引数

<i>buffer</i>	ムーブ元のバッファ.
---------------	------------

8.13.2.4 ~GgBuffer()

```
template<typename T>
virtual gg::GgBuffer< T >::~~GgBuffer () [inline], [virtual]
デストラクタ.
gg.h の 5583 行目に定義があります。
```

8.13.3 関数詳解

8.13.3.1 bind()

```
template<typename T>
void gg::GgBuffer< T >::bind () const [inline]
バッファオブジェクトを結合する.
gg.h の 5696 行目に定義があります。
```

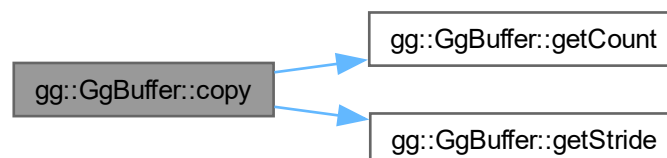
8.13.3.2 copy()

```
template<typename T>
void gg::GgBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
別のバッファオブジェクトからデータを複写する.
```

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (<i>src_buffer</i>) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 5801 行目に定義があります。
呼び出し関係図:



8.13.3.3 getBuffer()

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getBuffer () const [inline]
バッファオブジェクト名を取り出す。
```

戻り値

このバッファオブジェクト名。

gg.h の 5688 行目に定義があります。

8.13.3.4 getCount()

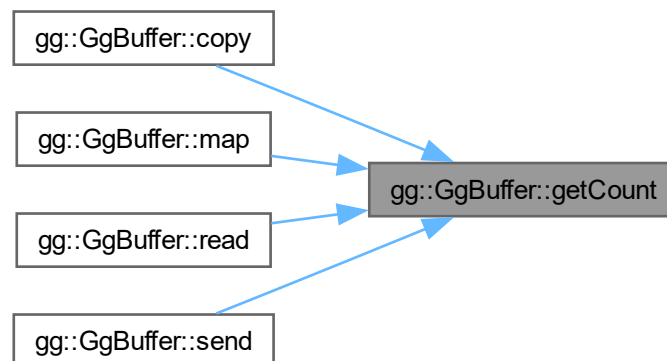
```
template<typename T>
const GLsizei & gg::GgBuffer< T >::getCount () const [inline]
バッファオブジェクトが保持するデータの数を取り出す。
```

戻り値

このバッファオブジェクトが保持するデータの数。

gg.h の 5678 行目に定義があります。

被呼び出し関係図:



8.13.3.5 getStride()

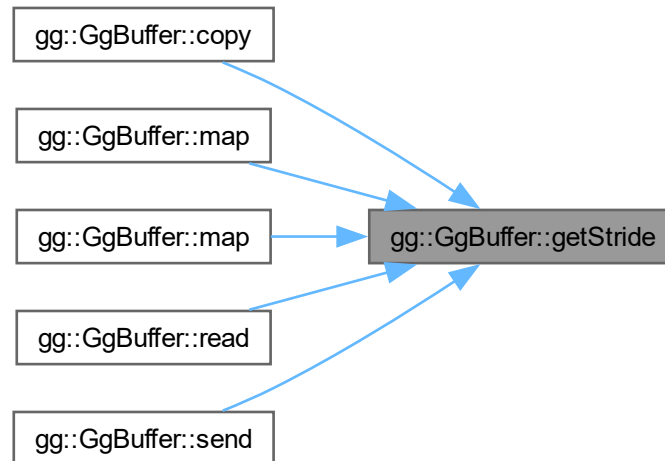
```
template<typename T>
GLsizeiptr gg::GgBuffer< T >::getStride () const [inline]
バッファオブジェクトのアライメントを考慮したデータの間隔を取り出す。
```

戻り値

このバッファオブジェクトのデータの間隔.

[gg.h](#) の 5668 行目に定義があります。

被呼び出し関係図:



8.13.3.6 getTarget()

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getTarget () const [inline]
```

バッファオブジェクトのターゲットを取り出す.

戻り値

このバッファオブジェクトのターゲット.

[gg.h](#) の 5658 行目に定義があります。

8.13.3.7 map() [1/2]

```
template<typename T>
void * gg::GgBuffer< T >::map () const [inline]
```

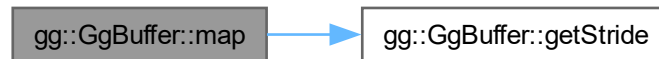
バッファオブジェクトをマップする.

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5714 行目に定義があります。

呼び出し関係図:



8.13.3.8 map() [2/2]

```
template<typename T>
void * gg::GgBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

バッファオブジェクトの指定した範囲をマップする.

引数

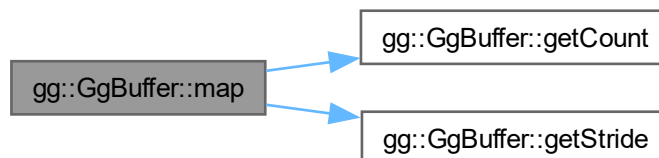
<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5731 行目に定義があります。

呼び出し関係図:



8.13.3.9 operator=() [1/2]

```
template<typename T>
GgBuffer< T > & gg::GgBuffer< T >::operator= (
    const GgBuffer< T > & buffer) [delete]
```

代入演算子は使用しない.

引数

<i>buffer</i>	代入元のバッファ.
---------------	-----------

戻り値

代入後のこのバッファの参照.

呼び出し関係図:



8.13.3.10 operator=() [2/2]

```
template<typename T>
GgBuffer< T > & gg::GgBuffer< T >::operator= (
    GgBuffer< T > && buffer) [default]
ムーブ代入演算子.
```

引数

<i>buffer</i>	ムーブ代入元のバッファ.
---------------	--------------

戻り値

ムーブ代入後のこのバッファの参照.

呼び出し関係図:



8.13.3.11 read()

```
template<typename T>
void gg::GgBuffer< T >::read (
    T * data,
```



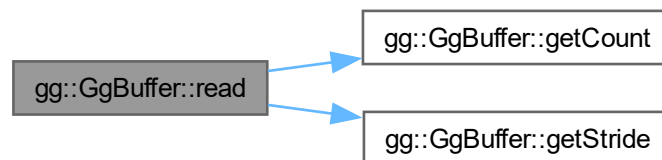
```
GLint first,
GLsizei count) const [inline]
```

バッファオブジェクトのデータから抽出する.

引数

<i>data</i>	抽出先の領域の先頭のポインタ.
<i>first</i>	抽出元のバッファオブジェクトの取り出すデータの領域の先頭の要素番号.
<i>count</i>	抽出するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 5774 行目に定義があります。
呼び出し関係図:



8.13.3.12 send()

```
template<typename T>
void gg::GgBuffer< T >::send (
    const T * data,
    GLint first,
    GLsizei count) const [inline]
```

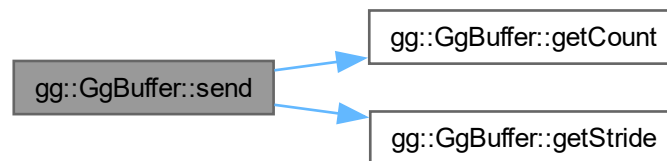
すでに確保したバッファオブジェクトにデータを転送する.

引数

<i>data</i>	転送元のデータが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 5756 行目に定義があります。

呼び出し関係図:



8.13.3.13 unbind()

```
template<typename T>
void gg::GgBuffer< T >::unbind () const [inline]
```

バッファオブジェクトを解放する。
[gg.h](#) の 5704 行目に定義があります。

8.13.3.14 unmap()

```
template<typename T>
void gg::GgBuffer< T >::unmap () const [inline]
```

バッファオブジェクトをアンマップする。
[gg.h](#) の 5744 行目に定義があります。
 このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.14 gg::GgColorTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgColorTexture](#) ()
- [GgColorTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- [GgColorTexture](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)
- virtual [~GgColorTexture](#) ()
- void [load](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- void [load](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)

8.14.1 詳解

カラーマップ。

覚え書き

カラー画像を読み込んでテクスチャを作成する。

[gg.h](#) の 5350 行目に定義があります。

8.14.2 構築子と解体子

8.14.2.1 GgColorTexture() [1/3]

gg::GgColorTexture::GgColorTexture () [inline]
 コンストラクタ.
[gg.h](#) の 5360 行目に定義があります。

8.14.2.2 GgColorTexture() [2/3]

```
gg::GgColorTexture::GgColorTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = true) [inline]
```

メモリ上のデータからカラーのテクスチャを作成するコンストラクタ.

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	読み込む画像の横の画素数.
<i>height</i>	読み込む画像の縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.

[gg.h](#) の 5376 行目に定義があります.
 呼び出し関係図:



8.14.2.3 GgColorTexture() [3/3]

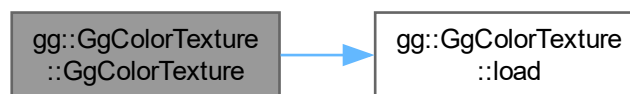
```
gg::GgColorTexture::GgColorTexture (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE) [inline]
```

TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成するコンストラクタ.

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, GL_TEXTURE_WRAP_S および GL_TEXTURE_WRAP_T に設定する値.

gg.h の 5397 行目に定義があります。
呼び出し関係図:



8.14.2.4 ~GgColorTexture()

virtual gg::GgColorTexture::~~GgColorTexture () [inline], [virtual]
デストラクタ.

gg.h の 5409 行目に定義があります。

8.14.3 関数詳解

8.14.3.1 load() [1/2]

```

void gg::GgColorTexture::load (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = true) [inline]
  
```

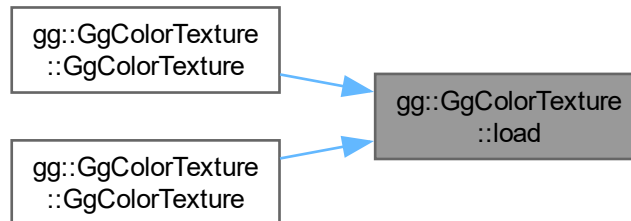
メモリ上のデータを読み込んでテクスチャを作成する。

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.
<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).

<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.
----------------	--------------------------------------

[gg.h](#) の 5425 行目に定義があります。
被呼び出し関係図:



8.14.3.2 load() [2/2]

```
void gg::GgColorTexture::load (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE)
```

TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する.

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 ならファイルの画像フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).

[gg.cpp](#) の 4028 行目に定義があります。
呼び出し関係図:



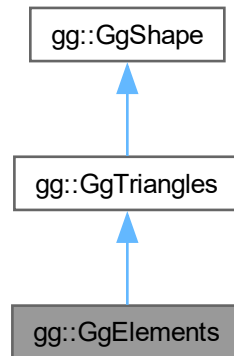
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

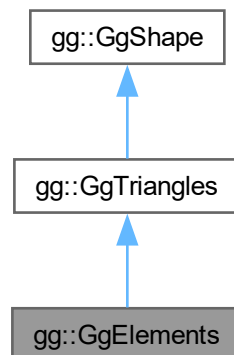
8.15 gg::GgElements クラス

```
#include <gg.h>
```

gg::GgElements の継承関係図



gg::GgElements 連携図



公開メンバ関数

- [GgElements](#) (GLenum mode=GL_TRIANGLES)
- [GgElements](#) (const [GgVertex](#) *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgElements](#) ()
- const GLsizei & [getIndexCount](#) () const
- const GLuint & [getIndexBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLuint firstv, GLsizei countv, const GLuint *face=nullptr, GLuint firstf=0, GLsizei countf=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum usage=GL_STATIC_DRAW)

- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

基底クラス [gg::GgTriangles](#) に属する継承公開メンバ関数

- [GgTriangles](#) (GLenum mode=GL_TRIANGLES)
- [GgTriangles](#) (const [GgVertex](#) *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgTriangles](#) ()
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLint first=0, GLsizei count=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

8.15.1 詳解

三角形で表した形状データ (Elements 形式).
[gg.h](#) の 6606 行目に定義があります。

8.15.2 構築子と解体子

8.15.2.1 GgElements() [1/2]

```
gg::GgElements::GgElements (
    GLenum mode = GL_TRIANGLES) [inline]
コンストラクタ.
```

引数

<i>mode</i>	描画する基本図形の種類.
-------------	--------------

[gg.h](#) の 6619 行目に定義があります。
 呼び出し関係図:



被呼び出し関係図:



8.15.2.2 GgElements() [2/2]

```

gg::GgElements::GgElements (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei countf,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

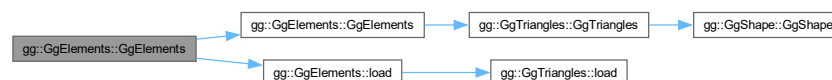
コンストラクタ.

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>face</i>	三角形の頂点インデックス.
<i>countf</i>	三角形の頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6634 行目に定義があります。

呼び出し関係図:



8.15.2.3 ~GgElements()

```

virtual gg::GgElements::~~GgElements () [inline], [virtual]
  
```

デストラクタ.

gg.h の 6650 行目に定義があります。

8.15.3 関数詳解

8.15.3.1 draw()

```

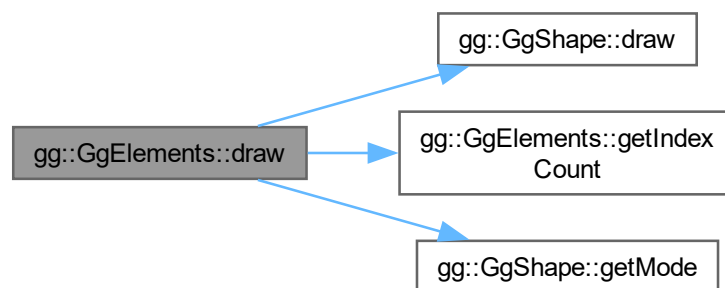
void gg::GgElements::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
  
```


インデックスを使った三角形の描画.

引数

<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

gg::GgTrianglesを再実装しています。
gg.cpp の 5169 行目に定義があります。
呼び出し関係図:



8.15.3.2 getIndexBuffer()

```
const GLuint & gg::GgElements::getIndexBuffer () const [inline]
```

三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名.

gg.h の 6668 行目に定義があります。

8.15.3.3 getIndexCount()

```
const GLsizei & gg::GgElements::getIndexCount () const [inline]
```

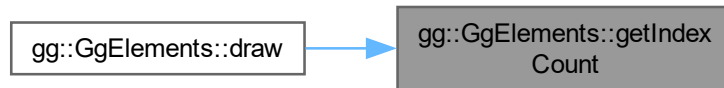
データの数を取り出す.

戻り値

この図形の三角形数.

[gg.h](#) の 6658 行目に定義があります。

被呼び出し関係図:



8.15.3.4 load()

```

void gg::GgElements::load (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei countf,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

バッファオブジェクトを確保して頂点属性と三角形の頂点インデックスデータを格納する.

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>countv</i>	頂点のデータの数 (頂点数).
<i>face</i>	三角形の頂点インデックスデータ.
<i>countf</i>	三角形の頂点数.
<i>usage</i>	バッファオブジェクトの使い方.

[gg.h](#) の 6705 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.15.3.5 send()

```

void gg::GgElements::send (
    const GgVertex * vert,
    GLuint firstv,
    GLsizei countv,
    const GLuint * face = nullptr,
    GLuint firstf = 0,
    GLsizei countf = 0) const [inline]
  
```

既存のバッファオブジェクトに頂点属性と三角形の頂点インデックスデータを転送する.

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>firstv</i>	頂点属性の転送先のバッファオブジェクトの先頭の要素番号.
<i>countv</i>	頂点のデータの数 (頂点数).
<i>face</i>	三角形の頂点インデックスデータ.
<i>firstf</i>	インデックスの転送先のバッファオブジェクトの先頭の要素番号.
<i>countf</i>	三角形の頂点数.

gg.h の 6683 行目に定義があります。

呼び出し関係図:



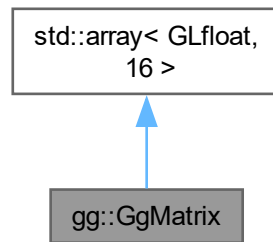
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

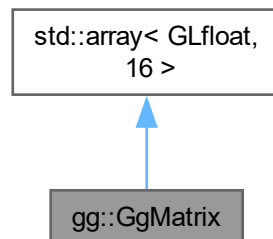
8.16 gg::GgMatrix クラス

```
#include <gg.h>
```

gg::GgMatrix の継承関係図



gg::GgMatrix 連携図



公開メンバ関数

- `GgMatrix` ()=default
- `constexpr GgMatrix` (GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03, GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13, GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23, GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33)
- `constexpr GgMatrix` (const GLfloat *a)
- `constexpr GgMatrix` (GLfloat c)
- `GgMatrix` (const GgMatrix &m)=default
- `GgMatrix` (GgMatrix &&m)=default
- `~GgMatrix` ()=default
- `GgMatrix & operator=` (const `GgMatrix` &m)=default
- `GgMatrix & operator=` (`GgMatrix` &&m)=default
- `GgMatrix & operator=` (const GLfloat *a)
- `GgMatrix operator+` (const GLfloat *a) const
- `GgMatrix operator+` (const `GgMatrix` &m) const
- `GgMatrix & operator+=` (const GLfloat *a)
- `GgMatrix & operator+=` (const `GgMatrix` &m)
- `GgMatrix operator-` (const GLfloat *a) const
- `GgMatrix operator-` (const `GgMatrix` &m) const
- `GgMatrix & operator-=` (const GLfloat *a)

- GgMatrix & operator-= (const GgMatrix &m)
- GgMatrix operator* (const GLfloat *a) const
- GgMatrix operator* (const GgMatrix &m) const
- GgMatrix & operator*= (const GLfloat *a)
- GgMatrix & operator*= (const GgMatrix &m)
- GgMatrix operator/ (const GLfloat *a) const
- GgMatrix operator/ (const GgMatrix &m) const
- GgMatrix & operator/= (const GLfloat *a)
- GgMatrix & operator/= (const GgMatrix &m)
- GgMatrix & loadIdentity ()
- GgMatrix & loadTranslate (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- GgMatrix & loadTranslate (const GLfloat *t)
- GgMatrix & loadTranslate (const GgVector &t)
- GgMatrix & loadScale (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- GgMatrix & loadScale (const GLfloat *s)
- GgMatrix & loadScale (const GgVector &s)
- GgMatrix & loadRotateX (GLfloat a)
- GgMatrix & loadRotateY (GLfloat a)
- GgMatrix & loadRotateZ (GLfloat a)
- GgMatrix & loadRotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- GgMatrix & loadRotate (const GLfloat *r, GLfloat a)
- GgMatrix & loadRotate (const GgVector &r, GLfloat a)
- GgMatrix & loadRotate (const GLfloat *r)
- GgMatrix & loadRotate (const GgVector &r)
- GgMatrix & loadLookat (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- GgMatrix & loadLookat (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- GgMatrix & loadLookat (const GgVector &e, const GgVector &t, const GgVector &u)
- GgMatrix & loadOrthogonal (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- GgMatrix & loadFrustum (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- GgMatrix & loadPerspective (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- GgMatrix & loadTranspose (const GLfloat *a)
- GgMatrix & loadTranspose (const GgMatrix &m)
- GgMatrix & loadInvert (const GLfloat *a)
- GgMatrix & loadInvert (const GgMatrix &m)
- GgMatrix & loadNormal (const GLfloat *a)
- GgMatrix & loadNormal (const GgMatrix &m)
- GgMatrix translate (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- GgMatrix translate (const GLfloat *t) const
- GgMatrix translate (const GgVector &t) const
- GgMatrix scale (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- GgMatrix scale (const GLfloat *s) const
- GgMatrix scale (const GgVector &s) const
- GgMatrix rotateX (GLfloat a) const
- GgMatrix rotateY (GLfloat a) const
- GgMatrix rotateZ (GLfloat a) const
- GgMatrix rotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- GgMatrix rotate (const GLfloat *r, GLfloat a) const
- GgMatrix rotate (const GgVector &r, GLfloat a) const
- GgMatrix rotate (const GLfloat *r) const
- GgMatrix rotate (const GgVector &r) const
- GgMatrix lookat (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz) const
- GgMatrix lookat (const GLfloat *e, const GLfloat *t, const GLfloat *u) const

- `GgMatrix` `lookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u) const
- `GgMatrix` `orthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix` `frustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix` `perspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar) const
- `GgMatrix` `transpose` () const
- `GgMatrix` `invert` () const
- `GgMatrix` `normal` () const
- void `projection` (GLfloat *c, const GLfloat *v) const
- void `projection` (GLfloat *c, const `GgVector` &v) const
- void `projection` (`GgVector` &c, const GLfloat *v) const
- void `projection` (`GgVector` &c, const `GgVector` &v) const
- `GgVector` `operator*` (const `GgVector` &v) const
- const GLfloat * `get` () const
- void `get` (GLfloat *a) const

8.16.1 詳解

変換行列.

`gg.h` の 2126 行目に定義があります。

8.16.2 構築子と解体子

8.16.2.1 `GgMatrix()` [1/6]

```
gg::GgMatrix::GgMatrix () [default]
コンストラクタ.
```

8.16.2.2 `GgMatrix()` [2/6]

```
gg::GgMatrix::GgMatrix (
    GLfloat m00,
    GLfloat m01,
    GLfloat m02,
    GLfloat m03,
    GLfloat m10,
    GLfloat m11,
    GLfloat m12,
    GLfloat m13,
    GLfloat m20,
    GLfloat m21,
    GLfloat m22,
    GLfloat m23,
    GLfloat m30,
    GLfloat m31,
    GLfloat m32,
    GLfloat m33) [inline], [constexpr]
コンストラクタ.
```

引数

<code>m00</code>	GLfloat 型の値.
<code>m01</code>	GLfloat 型の値.
<code>m02</code>	GLfloat 型の値.
<code>m03</code>	GLfloat 型の値.

<i>m10</i>	GLfloat 型の値.
<i>m11</i>	GLfloat 型の値.
<i>m12</i>	GLfloat 型の値.
<i>m13</i>	GLfloat 型の値.
<i>m20</i>	GLfloat 型の値.
<i>m21</i>	GLfloat 型の値.
<i>m22</i>	GLfloat 型の値.
<i>m23</i>	GLfloat 型の値.
<i>m30</i>	GLfloat 型の値.
<i>m31</i>	GLfloat 型の値.
<i>m32</i>	GLfloat 型の値.
<i>m33</i>	GLfloat 型の値.

gg.h の 2161 行目に定義があります。

8.16.2.3 GgMatrix() [3/6]

```
gg::GgMatrix::GgMatrix (
    const GLfloat * a) [inline], [constexpr]
コンストラクタ.
```

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

gg.h の 2176 行目に定義があります。
呼び出し関係図:



8.16.2.4 GgMatrix() [4/6]

```
gg::GgMatrix::GgMatrix (
    GLfloat c) [inline], [constexpr]
コンストラクタ.
```

引数

<i>c</i>	GLfloat 型の値.
----------	--------------

gg.h の 2186 行目に定義があります。

呼び出し関係図:



8.16.2.5 GgMatrix() [5/6]

```
gg::GgMatrix::GgMatrix (  
    const GgMatrix & m) [default]  
コピーコンストラクタ.
```

引数

<i>m</i>	コピー元の変換行列.
----------	------------

呼び出し関係図:



8.16.2.6 GgMatrix() [6/6]

```
gg::GgMatrix::GgMatrix (  
    GgMatrix && m) [default]  
ムーブコンストラクタ.
```

引数

<i>m</i>	ムーブ元の変換行列.
----------	------------

呼び出し関係図:



8.16.2.7 ~GgMatrix()

`gg::GgMatrix::~~GgMatrix () [default]`
デストラクタ.

8.16.3 関数詳解

8.16.3.1 frustum()

```
GgMatrix gg::GgMatrix::frustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) const [inline]
```

透視投影変換を乗じた結果を返す.

引数

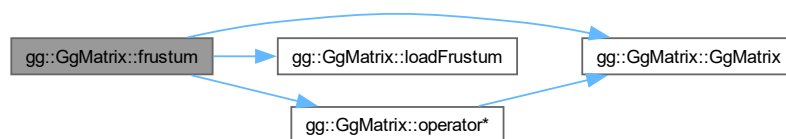
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

[gg.h](#) の 2969 行目に定義があります。

呼び出し関係図:



8.16.3.2 get() [1/2]

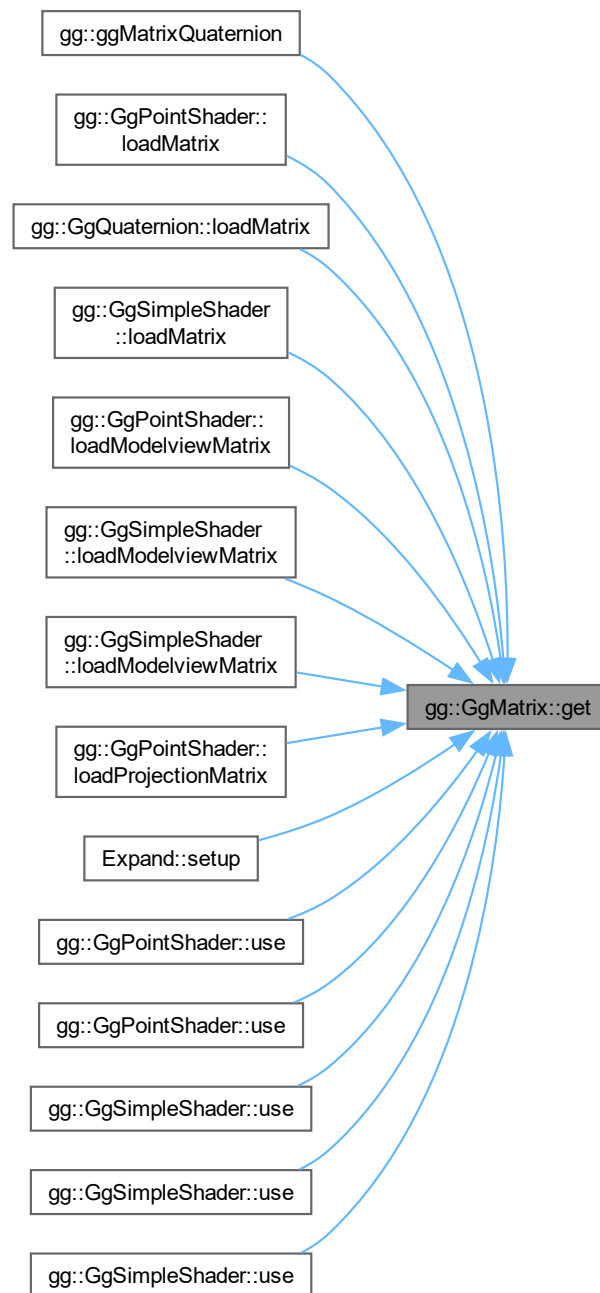
`const GLfloat * gg::GgMatrix::get () const [inline]`
変換行列を取り出す.

戻り値

変換行列を格納した GLfloat 型の 16 要素の配列変数.

[gg.h](#) の 3092 行目に定義があります。

被呼び出し関係図:



8.16.3.3 `get()` [2/2]

```
void gg::GgMatrix::get (
    GLfloat * a) const [inline]
```

変換行列を取り出す.

引数

a	変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

gg.h の 3102 行目に定義があります。

8.16.3.4 invert()

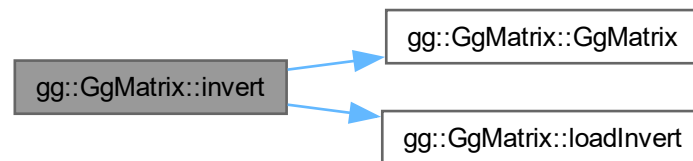
GgMatrix gg::GgMatrix::invert () const [inline]
逆行列を返す.

戻り値

逆行列.

gg.h の 3013 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.5 loadFrustum()

gg::GgMatrix & gg::GgMatrix::loadFrustum (
 GLfloat left,
 GLfloat right,
 GLfloat bottom,
 GLfloat top,
 GLfloat zNear,
 GLfloat zFar)

透視透視投影変換行列を格納する.

引数

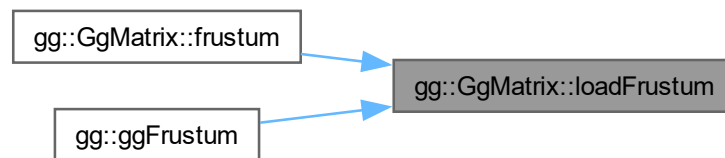
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

`gg.cpp` の 3129 行目に定義があります。

被呼び出し関係図:



8.16.3.6 loadIdentity()

`gg::GgMatrix & gg::GgMatrix::loadIdentity ()`

単位行列を格納する.

`gg.cpp` の 2740 行目に定義があります。

被呼び出し関係図:



8.16.3.7 loadInvert() [1/2]

`GgMatrix & gg::GgMatrix::loadInvert (const GgMatrix & m) [inline]`

逆行列を格納する.

引数

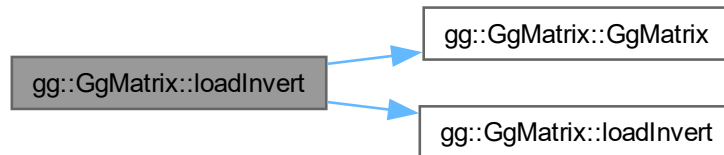
<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

設定した m の逆行列.

gg.h の 2692 行目に定義があります。

呼び出し関係図:



8.16.3.8 loadInvert() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadInvert (
    const GLfloat * a)
```

逆行列を格納する.

引数

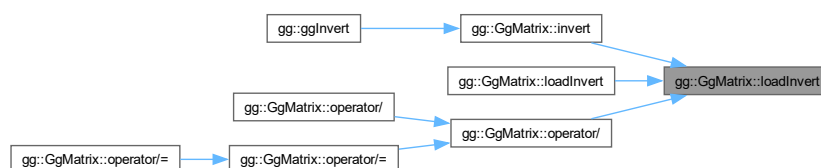
a	GLfloat 型の 16 要素の変換行列.
-----	------------------------

戻り値

設定した a の逆行列.

gg.cpp の 2915 行目に定義があります。

被呼び出し関係図:



8.16.3.9 loadLookat() [1/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]
```

ビュー変換行列を格納する.

引数

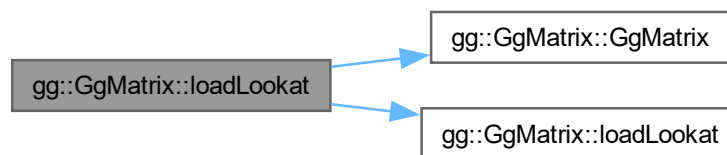
<i>e</i>	視点の位置の GgVector 型の変数.
<i>t</i>	目標点の位置の GgVector 型の変数.
<i>u</i>	上方向のベクトルの GgVector 型の変数.

戻り値

設定したビュー変換行列.

[gg.h](#) の 2612 行目に定義があります。

呼び出し関係図:



8.16.3.10 loadLookat() [2/3]

```
GgMatrix & gg::GgMatrix::loadLookat (  
    const GLfloat * e,  
    const GLfloat * t,  
    const GLfloat * u) [inline]
```

ビュー変換行列を格納する.

引数

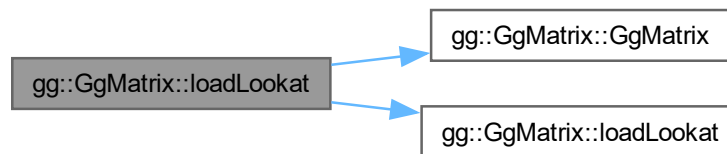
<i>e</i>	視点の位置の配列変数.
<i>t</i>	目標点の位置の配列変数.
<i>u</i>	上方向のベクトルの配列変数.

戻り値

設定したビュー変換行列.

gg.h の 2599 行目に定義があります。

呼び出し関係図:



8.16.3.11 loadLookat() [3/3]

```

gg::GgMatrix & gg::GgMatrix::loadLookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz)
  
```

ビュー変換行列を格納する.

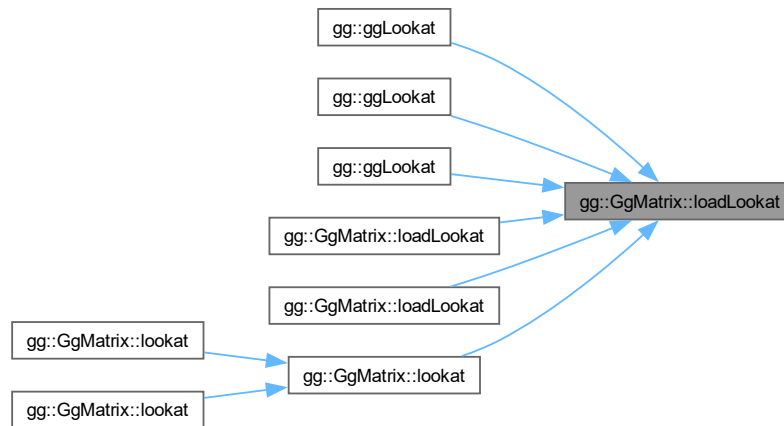
引数

<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

設定したビュー変換行列.

`gg.cpp` の 3029 行目に定義があります。
被呼び出し関係図:



8.16.3.12 loadNormal() [1/2]

```
GgMatrix & gg::GgMatrix::loadNormal (
    const GgMatrix & m) [inline]
```

法線変換行列を格納する.

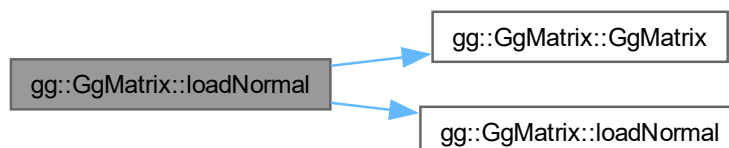
引数

<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

設定した *m* の法線変換行列.

`gg.h` の 2711 行目に定義があります。
呼び出し関係図:



8.16.3.13 loadNormal() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadNormal (
    const GLfloat * a)
```

法線変換行列を格納する。

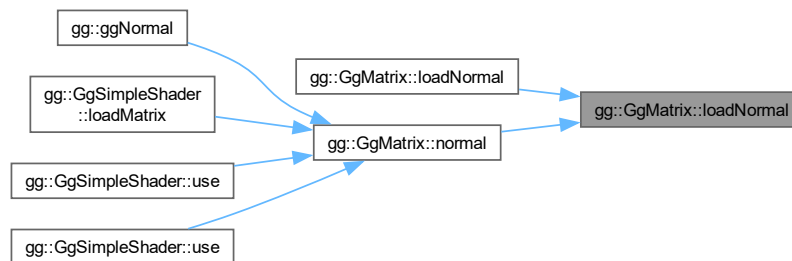
引数

<i>a</i>	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

設定した *m* の法線変換行列.

gg.cpp の 2998 行目に定義があります。
被呼び出し関係図:



8.16.3.14 loadOrthogonal()

```
gg::GgMatrix & gg::GgMatrix::loadOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)
```

直交投影変換行列を格納する。

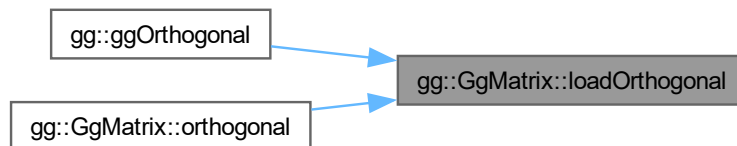
引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した直交投影変換行列.

`gg.cpp` の 3088 行目に定義があります。
被呼び出し関係図:



8.16.3.15 loadPerspective()

```

gg::GgMatrix & gg::GgMatrix::loadPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar)
  
```

画角を指定して透視投影変換行列を格納する.

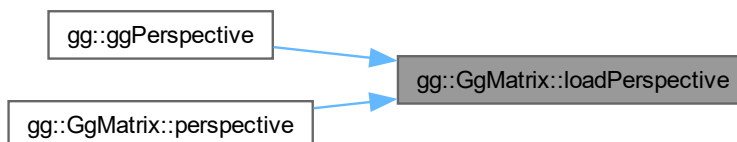
引数

<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

`gg.cpp` の 3170 行目に定義があります。
被呼び出し関係図:



8.16.3.16 loadRotate() [1/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

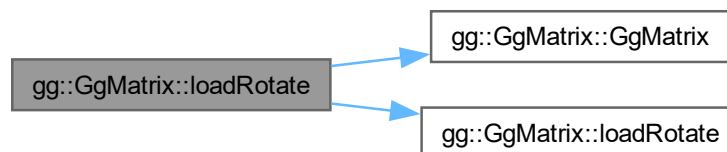
引数

r	回転軸の方向ベクトルと回転角を格納した GgVector 型のベクトル, 第 4 要素を回転角に用いる.
-----	--

戻り値

設定した変換行列.

[gg.h](#) の 2568 行目に定義があります。
呼び出し関係図:

**8.16.3.17 loadRotate() [2/5]**

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

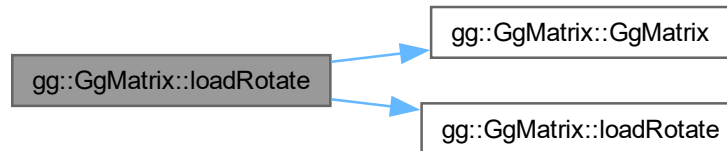
r	回転軸の方向ベクトルを格納した GgVector 型のベクトル, 第 4 要素は無視する.
a	回転角.

戻り値

設定した変換行列.

[gg.h](#) の 2546 行目に定義があります。

呼び出し関係図:



8.16.3.18 loadRotate() [3/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GLfloat * r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

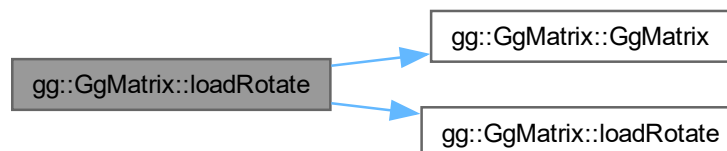
r	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数, 第 4 要素を回転角に用いる.
-----	---

戻り値

設定した変換行列.

[gg.h](#) の 2557 行目に定義があります。

呼び出し関係図:



8.16.3.19 loadRotate() [4/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

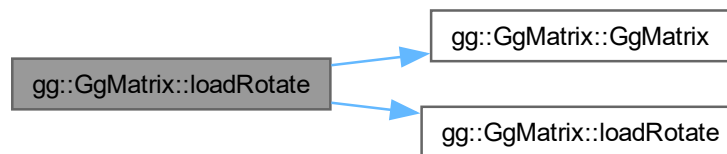
r	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
a	回転角.

戻り値

設定した変換行列.

gg.h の 2534 行目に定義があります。

呼び出し関係図:



8.16.3.20 loadRotate() [5/5]

```

gg::GgMatrix & gg::GgMatrix::loadRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a)
  
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を格納する.

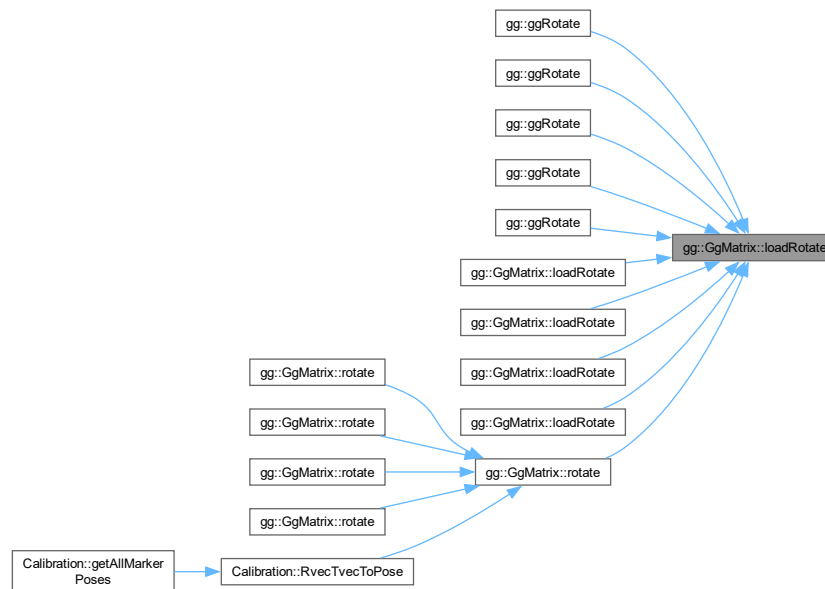
引数

x	回転軸の x 成分.
y	回転軸の y 成分.
z	回転軸の z 成分.
a	回転角.

戻り値

設定した変換行列.

`gg.cpp` の 2845 行目に定義があります。
被呼び出し関係図:



8.16.3.21 loadRotateX()

```
gg::GgMatrix & gg::GgMatrix::loadRotateX (
    GLfloat a)
```

x 軸中心の回転の変換行列を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

`gg.cpp` の 2788 行目に定義があります。
被呼び出し関係図:



8.16.3.22 loadRotateY()

```
gg::GgMatrix & gg::GgMatrix::loadRotateY (
    GLfloat a)
```

y 軸中心の回転の変換行列を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

gg.cpp の 2807 行目に定義があります。
被呼び出し関係図:



8.16.3.23 loadRotateZ()

```
gg::GgMatrix & gg::GgMatrix::loadRotateZ (
    GLfloat a)
```

z 軸中心の回転の変換行列を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

gg.cpp の 2826 行目に定義があります。
被呼び出し関係図:



8.16.3.24 loadScale() [1/3]

```
GgMatrix & gg::GgMatrix::loadScale (
    const GgVector & s) [inline]
```

拡大縮小の変換行列を格納する.

引数

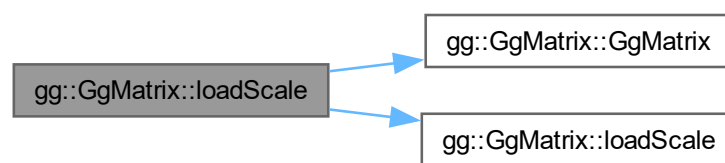
s	拡大率の GgVector 型の変数.
---	-------------------------------------

戻り値

設定した変換行列.

[gg.h](#) の 2487 行目に定義があります。

呼び出し関係図:



8.16.3.25 loadScale() [2/3]

```
GgMatrix & gg::GgMatrix::loadScale (  
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を格納する.

引数

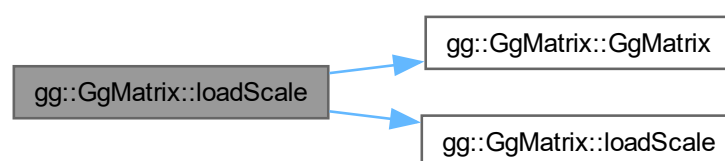
s	拡大率の GLfloat 型の配列 (x, y, z).
---	------------------------------

戻り値

設定した変換行列.

[gg.h](#) の 2476 行目に定義があります。

呼び出し関係図:



8.16.3.26 loadScale() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadScale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

拡大縮小の変換行列を格納する.

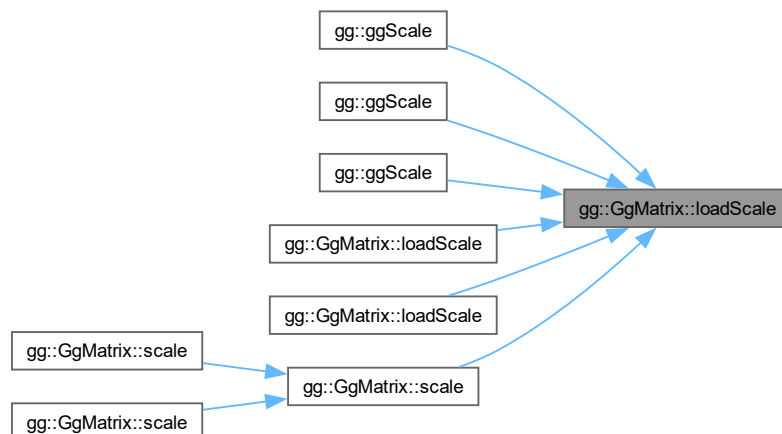
引数

<i>x</i>	x 方向の拡大率.
<i>y</i>	y 方向の拡大率.
<i>z</i>	z 方向の拡大率.
<i>w</i>	w 拡大率のスケールファクタ (= 1.0f).

戻り値

設定した変換行列.

gg.cpp の 2772 行目に定義があります。
被呼び出し関係図:



8.16.3.27 loadTranslate() [1/3]

```
GgMatrix & gg::GgMatrix::loadTranslate (
    const GgVector & t) [inline]
```

平行移動の変換行列を格納する.

引数

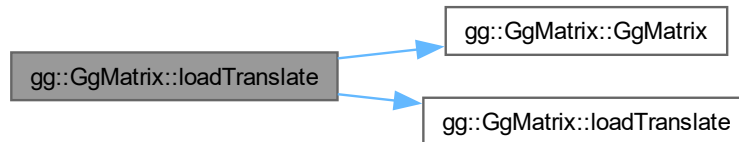
<i>t</i>	移動量の <code>GgVector</code> 型の変数.
----------	----------------------------------

戻り値

設定した変換行列.

[gg.h](#) の 2454 行目に定義があります。

呼び出し関係図:



8.16.3.28 loadTranslate() [2/3]

```
GgMatrix & gg::GgMatrix::loadTranslate (
    const GLfloat * t) [inline]
```

平行移動の変換行列を格納する.

引数

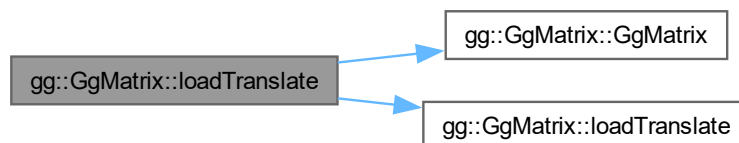
<i>t</i>	移動量の GLfloat 型の配列 (x, y, z).
----------	------------------------------

戻り値

設定した変換行列.

[gg.h](#) の 2443 行目に定義があります。

呼び出し関係図:



8.16.3.29 loadTranslate() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadTranslate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

平行移動の変換行列を格納する.

引数

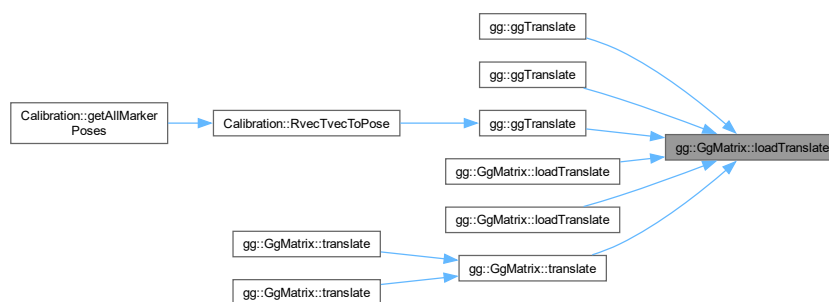
x	x 方向の移動量.
y	y 方向の移動量.
z	z 方向の移動量.
w	w 移動量のスケールファクタ (= 1.0f).

戻り値

設定した変換行列.

gg.cpp の 2756 行目に定義があります。

被呼び出し関係図:



8.16.3.30 loadTranspose() [1/2]

```
GgMatrix & gg::GgMatrix::loadTranspose (
    const GgMatrix & m) [inline]
```

転置行列を格納する.

引数

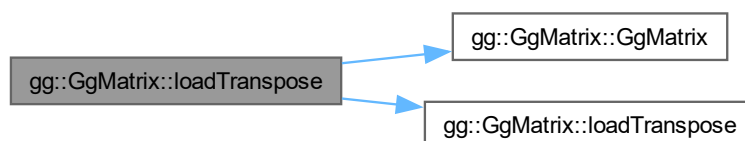
m	GgMatrix 型の変換行列.
-----	------------------

戻り値

設定した m の転置行列.

gg.h の 2673 行目に定義があります。

呼び出し関係図:



8.16.3.31 loadTranspose() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadTranspose (
    const GLfloat * a)
```

転置行列を格納する.

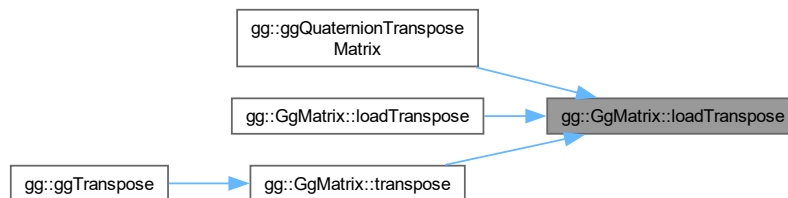
引数

<i>a</i>	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

設定した *a* の転置行列.

[gg.cpp](#) の 2887 行目に定義があります。
被呼び出し関係図:



8.16.3.32 lookout() [1/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

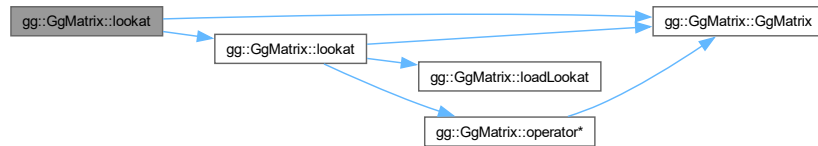
<i>e</i>	視点の位置を格納した GgVector 型の変数.
<i>t</i>	目標点の位置を格納した GgVector 型の変数.
<i>u</i>	上方向のベクトルを格納した GgVector 型の変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2932 行目に定義があります。

呼び出し関係図:



8.16.3.33 lookat() [2/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

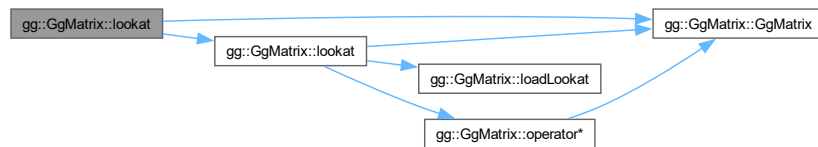
<i>e</i>	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>t</i>	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>u</i>	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2919 行目に定義があります。

呼び出し関係図:



8.16.3.34 lookat() [3/3]

```
GgMatrix gg::GgMatrix::lookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
```

```

    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz) const [inline]

```

ビュー変換を乗じた結果を返す。

引数

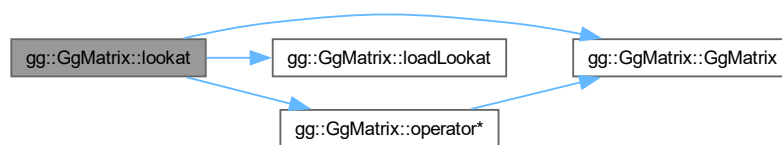
ex	視点の位置の x 座標値.
ey	視点の位置の y 座標値.
ez	視点の位置の z 座標値.
tx	目標点の位置の x 座標値.
ty	目標点の位置の y 座標値.
tz	目標点の位置の z 座標値.
ux	上方向のベクトルの x 成分.
uy	上方向のベクトルの y 成分.
uz	上方向のベクトルの z 成分.

戻り値

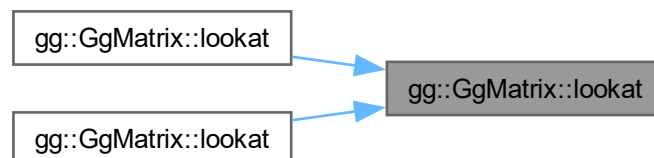
ビュー変換行列を乗じた変換行列。

[gg.h](#) の 2901 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.35 normal()

```

GgMatrix gg::GgMatrix::normal () const [inline]

```

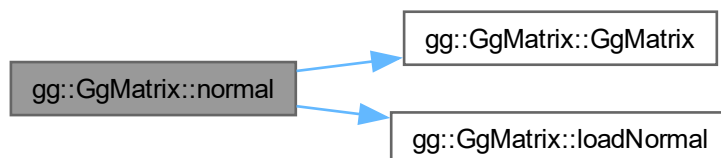
法線変換行列を返す.

戻り値

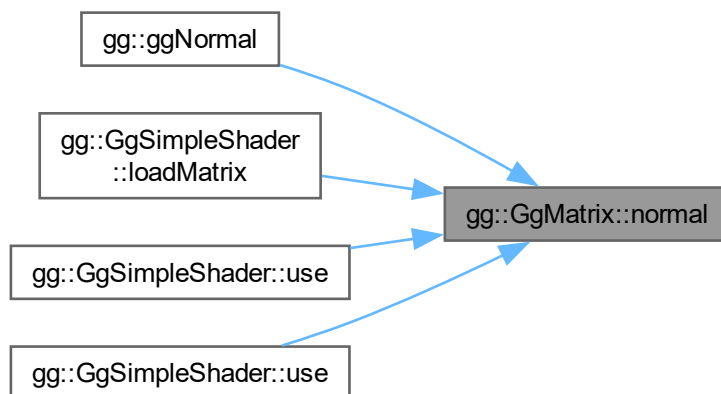
法線変換行列.

gg.h の 3024 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.16.3.36 operator*() [1/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を乗算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列に **a** を乗じた **GgMatrix** 型の変換行列.

gg.h の 2345 行目に定義があります。

呼び出し関係図:



8.16.3.37 operator*() [2/3]

```
GgVector gg::GgMatrix::operator* (
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

v	元のベクトルの GgVector 型の変数.
----------	-------------------------------

戻り値

v 変換結果の **GgVector** 型のベクトル.

gg.h の 3080 行目に定義があります。

8.16.3.38 operator*() [3/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を乗算した値を返す.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	---

戻り値

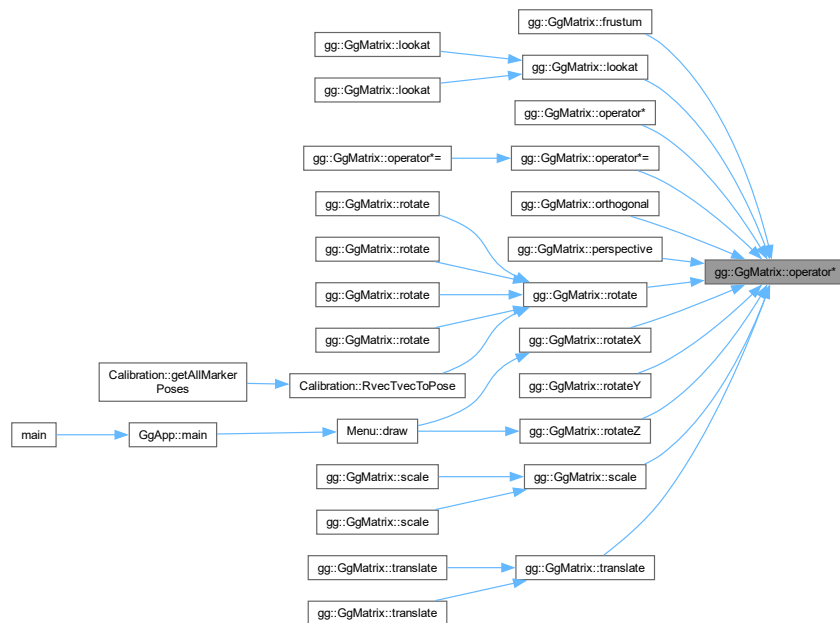
変換行列に **a** を乗じた **GgMatrix** 型の変換行列.

gg.h の 2332 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.39 operator*=() [1/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を乗算した結果を格納する.

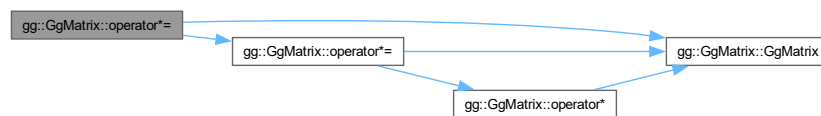
引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を掛けた変換行列の参照.

gg.h の 2368 行目に定義があります.
呼び出し関係図:



8.16.3.40 operator*=() [2/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を乗算した結果を格納する.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を掛けた変換行列の参照.

gg.h の 2356 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.16.3.41 operator+() [1/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を加算した値を返す.

引数

m	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列に **a** を加えた **GgMatrix** 型の変換行列.

gg.h の 2251 行目に定義があります。

呼び出し関係図:



8.16.3.42 operator+() [2/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を加算した値を返す.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

変換行列に **a** を加えた **GgMatrix** 型の変換行列.

gg.h の 2238 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.43 operator+=() [1/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を加算した結果を格納する.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を加えた変換行列の参照.

gg.h の 2274 行目に定義があります。
呼び出し関係図:



8.16.3.44 operator+=() [2/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を加算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を加えた変換行列の参照.

gg.h の 2262 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.16.3.45 operator-() [1/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GgMatrix & m) const [inline]
```

変換行列から別の変換行列を減算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列から *m* を引いた GgMatrix 型の変換行列.

gg.h の 2298 行目に定義があります.

呼び出し関係図:



8.16.3.46 operator-() [2/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GLfloat * a) const [inline]
```

変換行列から配列に格納した変換行列を減算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

変換行列から *a* を引いた GgMatrix 型の変換行列.

gg.h の 2285 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.16.3.47 operator-=() [1/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を減算した結果を格納する.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を引いた変換行列の参照.

gg.h の 2321 行目に定義があります。
呼び出し関係図:



8.16.3.48 operator-=() [2/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を減算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を引いた変換行列の参照.

[gg.h](#) の 2309 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.49 operator/() [1/2]

```
GgMatrix gg::GgMatrix::operator/ (  
    const GgMatrix & m) const [inline]
```

変換行列を変換行列で除算した値を返す.

引数

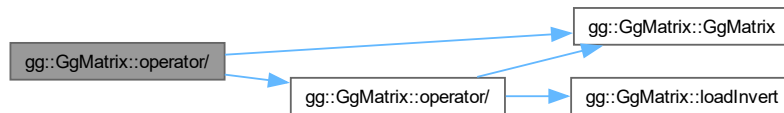
<i>m</i>	GgMatrix 型の変換行列.
----------	----------------------------------

戻り値

変換行列を **a** で割った **GgMatrix** 型の変換行列.

gg.h の 2393 行目に定義があります。

呼び出し関係図:



8.16.3.50 operator/() [2/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GLfloat * a) const [inline]
```

変換行列を配列に格納した変換行列で除算した値を返す.

引数

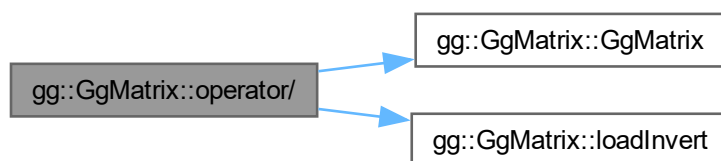
a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

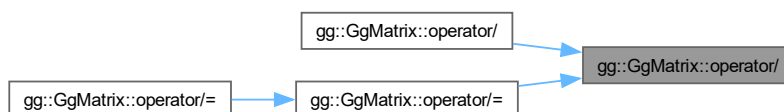
変換行列を **a** で割った **GgMatrix** 型の変換行列.

gg.h の 2379 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.51 operator/=() [1/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GgMatrix & m) [inline]
```

変換行列を別の変換行列で除算した結果を格納する.

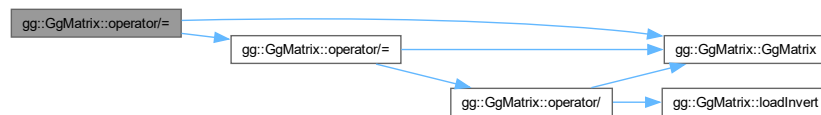
引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m で割った変換行列の参照.

gg.h の 2416 行目に定義があります。
呼び出し関係図:



8.16.3.52 operator/=() [2/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GLfloat * a) [inline]
```

変換行列を配列に格納した変換行列で除算した結果を格納する.

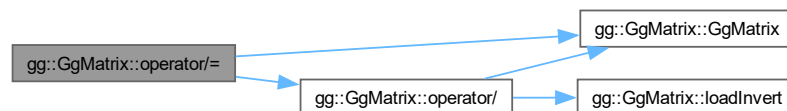
引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a で割った変換行列の参照.

gg.h の 2404 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.16.3.53 operator=() [1/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GgMatrix & m) [default]
```

代入演算子. 呼び出し関係図:



8.16.3.54 operator=() [2/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GLfloat * a) [inline]
```

配列変数の値を格納する.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

戻り値

a を代入したこのオブジェクトの参照.

[gg.h](#) の 2226 行目に定義があります。

呼び出し関係図:



8.16.3.55 operator=() [3/3]

```
GgMatrix & gg::GgMatrix::operator= (
    GgMatrix && m) [default]
```

ムーブ代入演算子. 呼び出し関係図:



8.16.3.56 orthogonal()

```
GgMatrix gg::GgMatrix::orthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) const [inline]
```

直交投影変換を乗じた結果を返す.

引数

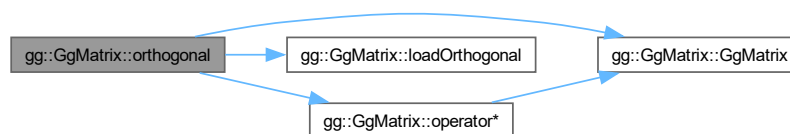
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

直交投影変換行列を乗じた変換行列.

gg.h の 2948 行目に定義があります。

呼び出し関係図:



8.16.3.57 perspective()

```
GgMatrix gg::GgMatrix::perspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) const [inline]
```

画角を指定して透視投影変換を乗じた結果を返す。

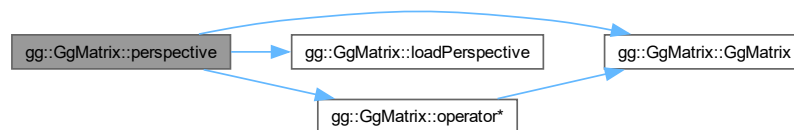
引数

<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

[gg.h](#) の 2988 行目に定義があります。
呼び出し関係図:



8.16.3.58 projection() [1/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<i>c</i>	変換結果を格納する GgVector 型の変数.
<i>v</i>	元のベクトルの GgVector 型の変数.

[gg.h](#) の 3069 行目に定義があります。

8.16.3.59 projection() [2/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<i>c</i>	変換結果を格納する GgVector 型の変数.
<i>v</i>	元のベクトルの <code>GLfloat</code> 型の 4 要素の配列変数.

[gg.h](#) の 3058 行目に定義があります。

8.16.3.60 projection() [3/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<i>c</i>	変換結果を格納する <code>GLfloat</code> 型の 4 要素の配列変数.
<i>v</i>	元のベクトルの GgVector 型の変数.

[gg.h](#) の 3047 行目に定義があります。

8.16.3.61 projection() [4/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<i>c</i>	変換結果を格納する <code>GLfloat</code> 型の 4 要素の配列変数.
<i>v</i>	元のベクトルの <code>GLfloat</code> 型の 4 要素の配列変数.

[gg.h](#) の 3036 行目に定義があります。

8.16.3.62 rotate() [1/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r) const [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す。

引数

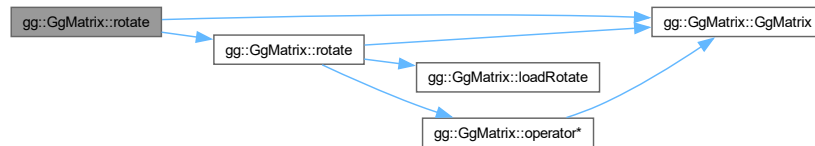
<i>r</i>	回転軸の方向ベクトルと回転角を格納した GgVector 型の変数).
----------	---

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに $r[3]$ 回転した変換行列.

[gg.h](#) の 2882 行目に定義があります.

呼び出し関係図:



8.16.3.63 rotate() [2/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

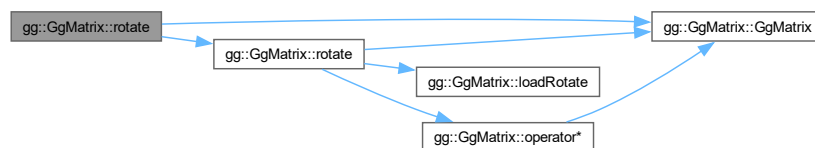
r	回転軸の方向ベクトルを格納した GgVector 型の変数.
a	回転角.

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに a 回転した変換行列.

[gg.h](#) の 2860 行目に定義があります.

呼び出し関係図:



8.16.3.64 rotate() [3/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r) const [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

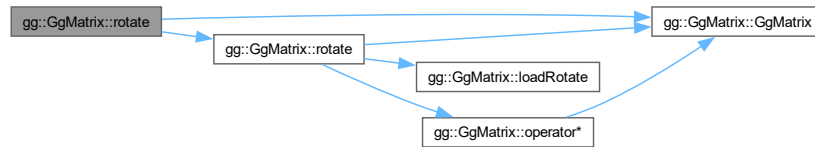
r	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数 (x , y , z , a).
-----	---

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに $r[3]$ 回転した変換行列.

gg.h の 2871 行目に定義があります。

呼び出し関係図:



8.16.3.65 rotate() [4/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

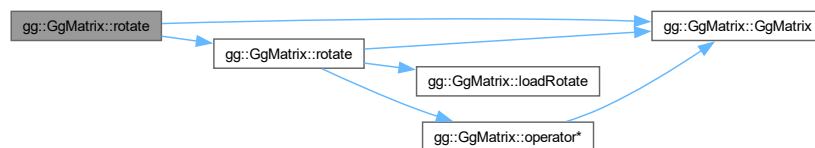
r	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
a	回転角.

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに a 回転した変換行列.

gg.h の 2848 行目に定義があります。

呼び出し関係図:



8.16.3.66 rotate() [5/5]

```
GgMatrix gg::GgMatrix::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]
```

(x, y, z) 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

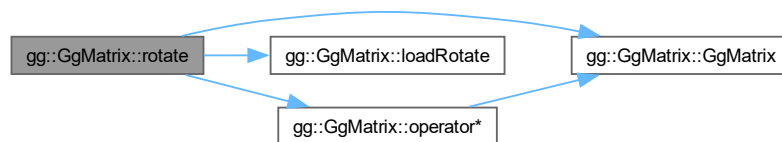
x	回転軸の x 成分.
y	回転軸の y 成分.
z	回転軸の z 成分.
a	回転角.

戻り値

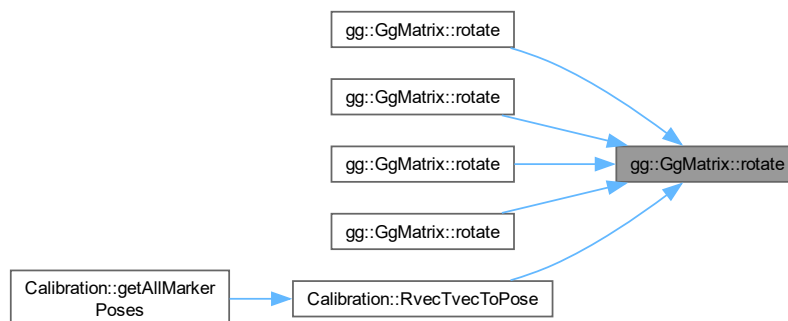
(x, y, z) を軸にさらに a 回転した変換行列.

[gg.h](#) の 2835 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.67 rotateX()

```
GgMatrix gg::GgMatrix::rotateX (
    GLfloat a) const [inline]
```

x 軸中心の回転変換を乗じた結果を返す.

引数

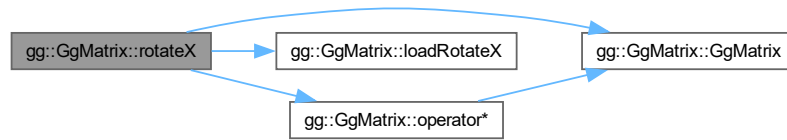
a	回転角.
-----	------

戻り値

x 軸中心にさらに a 回転した変換行列.

gg.h の 2796 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.68 rotateY()

```
GgMatrix gg::GgMatrix::rotateY (
    GLfloat a) const [inline]
```

y 軸中心の回転変換を乗じた結果を返す.

引数

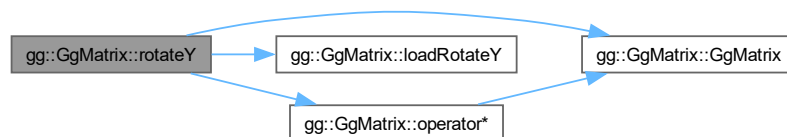
a	回転角.
---	------

戻り値

y 軸中心にさらに a 回転した変換行列.

gg.h の 2808 行目に定義があります。

呼び出し関係図:



8.16.3.69 rotateZ()

```
GgMatrix gg::GgMatrix::rotateZ (
    GLfloat a) const [inline]
```

z 軸中心の回転変換を乗じた結果を返す.

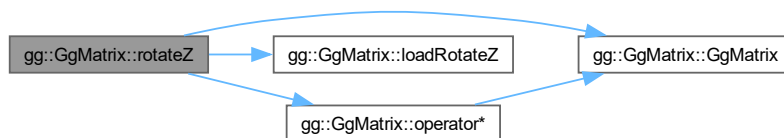
引数

<i>a</i>	回転角.
----------	------

戻り値

z 軸中心にさらに *a* 回転した変換行列.

[gg.h](#) の 2820 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.16.3.70 scale() [1/3]

```
GgMatrix gg::GgMatrix::scale (
    const GgVector & s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

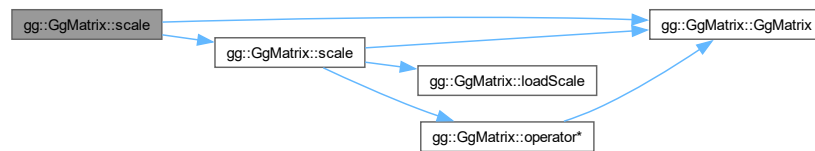
<i>s</i>	拡大率の <code>GgVector</code> 型の変数.
----------	----------------------------------

戻り値

拡大縮小した結果の変換行列.

gg.h の 2785 行目に定義があります。

呼び出し関係図:



8.16.3.71 scale() [2/3]

```
GgMatrix gg::GgMatrix::scale (
    const GLfloat * s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

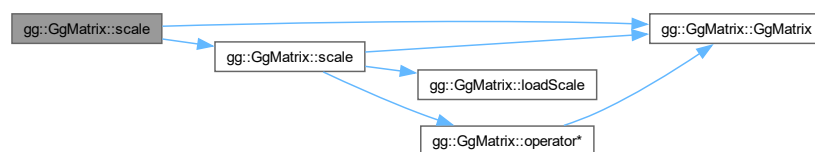
s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
---	--------------------------------------

戻り値

拡大縮小した結果の変換行列.

gg.h の 2774 行目に定義があります。

呼び出し関係図:



8.16.3.72 scale() [3/3]

```
GgMatrix gg::GgMatrix::scale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

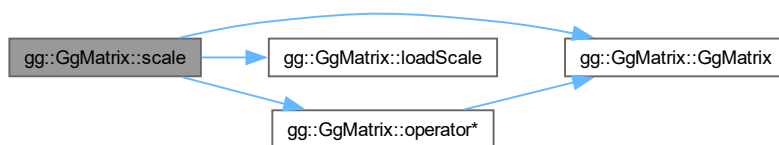
<i>x</i>	x 方向の拡大率.
<i>y</i>	y 方向の拡大率.
<i>z</i>	z 方向の拡大率.
<i>w</i>	w 移動量のスケールファクタ (= 1.0f).

戻り値

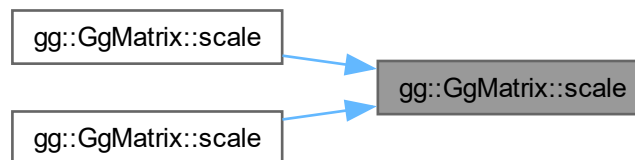
拡大縮小した結果の変換行列.

[gg.h](#) の 2762 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.73 translate() [1/3]

```
GgMatrix gg::GgMatrix::translate (
    const GgVector & t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

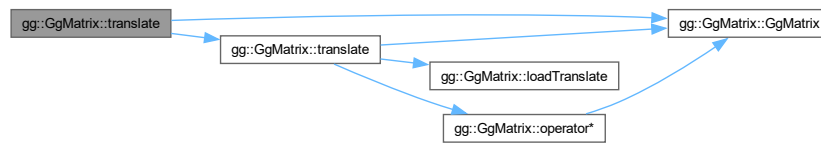
<i>t</i>	移動量の GgVector 型の変数.
----------	-------------------------------------

戻り値

平行移動した結果の変換行列.

[gg.h](#) の 2748 行目に定義があります。

呼び出し関係図:



8.16.3.74 translate() [2/3]

```
GgMatrix gg::GgMatrix::translate (
    const GLfloat * t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

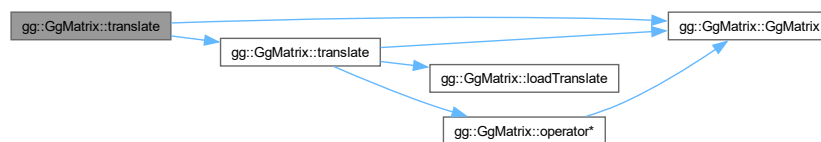
<i>t</i>	移動量の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

平行移動した結果の変換行列.

[gg.h](#) の 2737 行目に定義があります.

呼び出し関係図:



8.16.3.75 translate() [3/3]

```
GgMatrix gg::GgMatrix::translate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

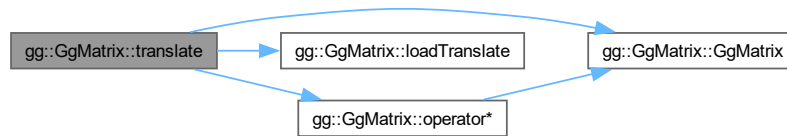
<i>x</i>	x 方向の移動量.
<i>y</i>	y 方向の移動量.
<i>z</i>	z 方向の移動量.
<i>w</i>	w 移動量のスケールファクタ (= 1.0f).

戻り値

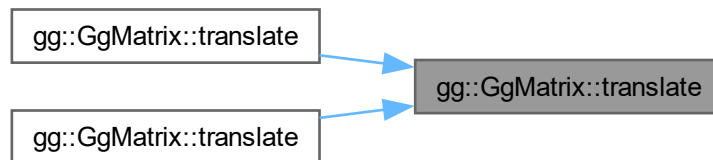
平行移動した結果の変換行列.

[gg.h](#) の 2725 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.76 transpose()

`GgMatrix gg::GgMatrix::transpose () const [inline]`

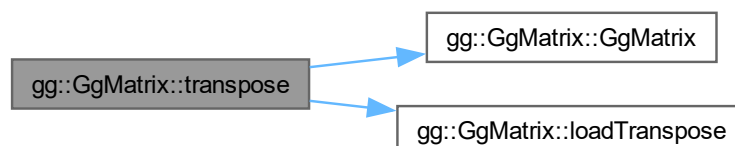
転置行列を返す.

戻り値

転置行列.

[gg.h](#) の 3002 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.17 gg::GgNormalTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgNormalTexture](#) ()
- [GgNormalTexture](#) (const GLubyte *image, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- [GgNormalTexture](#) (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- virtual [~GgNormalTexture](#) ()
- void [load](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- void [load](#) (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)

8.17.1 詳解

法線マップ.

覚え書き

高さマップ（グレイスケール画像）を読み込んで法線マップのテクスチャを作成する.

[gg.h](#) の 5460 行目に定義があります。

8.17.2 構築子と解体子

8.17.2.1 GgNormalTexture() [1/3]

```
gg::GgNormalTexture::GgNormalTexture () [inline]
コンストラクタ.
```

[gg.h](#) の 5470 行目に定義があります。

8.17.2.2 GgNormalTexture() [2/3]

```
gg::GgNormalTexture::GgNormalTexture (
    const GLubyte * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RED,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

メモリ上のデータから法線マップのテクスチャを作成するコンストラクタ.

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5484 行目に定義があります.

呼び出し関係図:



8.17.2.3 GgNormalTexture() [3/3]

```

gg::GgNormalTexture::GgNormalTexture (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
  
```

ファイルからデータを読み込んで法線マップのテクスチャを作成するコンストラクタ.

引数

<i>name</i>	画像ファイル名.
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5504 行目に定義があります.

呼び出し関係図:



8.17.2.4 ~GgNormalTexture()

```

virtual gg::GgNormalTexture::~~GgNormalTexture () [inline], [virtual]
  
```

デストラクタ.

gg.h の 5517 行目に定義があります.

8.17.3 関数詳解

8.17.3.1 load() [1/2]

```

void gg::GgNormalTexture::load (
  
```



```

const GLubyte * hmap,
GLsizei width,
GLsizei height,
GLenum format = GL_RED,
GLfloat nz = 1.0f,
GLenum internal = GL_RGBA) [inline]

```

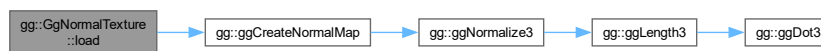
メモリ上のデータから法線マップのテクスチャを作成する.

引数

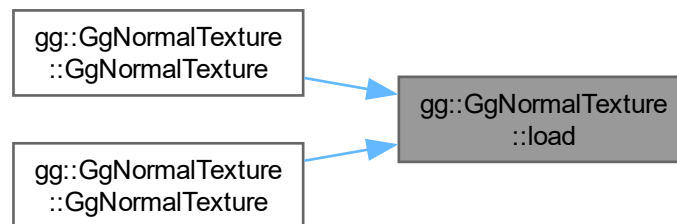
<i>hmap</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5531 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.2 load() [2/2]

```

void gg::GgNormalTexture::load (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA)

```

TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する.

引数

<i>name</i>	画像ファイル名 (1 チャンネルの TGA 画像).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.cpp の 4064 行目に定義があります。
呼び出し関係図:



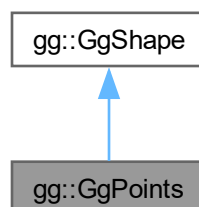
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

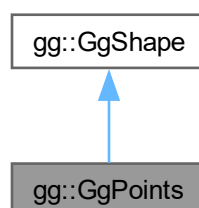
8.18 gg::GgPoints クラス

```
#include <gg.h>
```

gg::GgPoints の継承関係図



gg::GgPoints 連携図



公開メンバ関数

- [GgPoints](#) (GLenum mode=GL_POINTS)
- [GgPoints](#) (const [GgVector](#) *pos, GLsizei countv, GLenum mode=GL_POINTS, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgPoints](#) ()
- [operator bool](#) () const noexcept
- bool [operator!](#) () const noexcept
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVector](#) *pos, GLint first=0, GLsizei count=0) const
- void [load](#) (const [GgVector](#) *pos, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

8.18.1 詳解

点.

[gg.h](#) の 6333 行目に定義があります。

8.18.2 構築子と解体子

8.18.2.1 GgPoints() [1/2]

```
gg::GgPoints::GgPoints (
    GLenum mode = GL_POINTS) [inline]
```

コンストラクタ.

[gg.h](#) の 6344 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.18.2.2 GgPoints() [2/2]

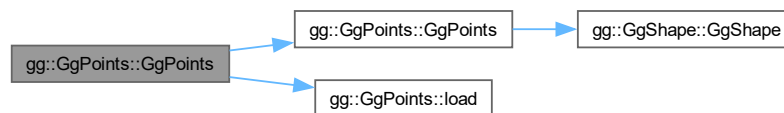
```
gg::GgPoints::GgPoints (
    const GgVector * pos,
    GLsizei countv,
    GLenum mode = GL_POINTS,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

コンストラクタ.

引数

<i>pos</i>	この図形の頂点の位置のデータの配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6357 行目に定義があります。
呼び出し関係図:



8.18.2.3 ~GgPoints()

```
virtual gg::GgPoints::~GgPoints () [inline], [virtual]
```

デストラクタ.

gg.h の 6371 行目に定義があります。

8.18.3 関数詳解

8.18.3.1 draw()

```
void gg::GgPoints::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

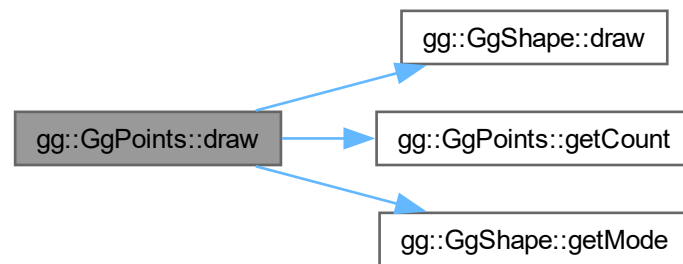
点の描画.

引数

<i>first</i>	描画を開始する最初の点の番号.
<i>count</i>	描画する点の数, 0 なら全部の点を描く.

gg::GgShape を再実装しています。
gg.cpp の 5126 行目に定義があります。

呼び出し関係図:



8.18.3.2 getBuffer()

```
const GLuint & gg::GgPoints::getBuffer () const [inline]
```

頂点の位置データを格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の頂点の位置データを格納した頂点バッファオブジェクト名.

[gg.h](#) の 6410 行目に定義があります。

8.18.3.3 getCount()

```
const GLsizei & gg::GgPoints::getCount () const [inline]
```

データの数を取り出す.

戻り値

この図形の頂点の位置データの数 (頂点数).

[gg.h](#) の 6400 行目に定義があります。

被呼び出し関係図:



8.18.3.4 load()

```
void gg::GgPoints::load (
    const GgVector * pos,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点の位置データを格納する.

引数

<i>pos</i>	頂点の位置データが格納されている領域の先頭のポインタ.
<i>count</i>	頂点のデータの数 (頂点数).
<i>usage</i>	バッファオブジェクトの使い方.

gg.cpp の 5113 行目に定義があります。
被呼び出し関係図:



8.18.3.5 operator bool()

```
gg::GgPoints::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

バッファが有効かどうか調べる.

戻り値

バッファが有効なら true

gg::GgShape を再実装しています。
gg.h の 6380 行目に定義があります。

8.18.3.6 operator"!()

```
bool gg::GgPoints::operator! () const [inline], [virtual], [noexcept]
```

バッファが有効かどうかの結果を反転する.

戻り値

バッファが有効なら false, 無効なら true.

gg::GgShape を再実装しています。
gg.h の 6390 行目に定義があります。

8.18.3.7 send()

```
void gg::GgPoints::send (
    const GgVector * pos,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点の位置データを転送する.

引数

<i>pos</i>	転送元の頂点の位置データが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

gg.h の 6422 行目に定義があります。

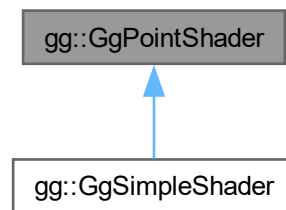
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.19 gg::GgPointShader クラス

```
#include <gg.h>
```

gg::GgPointShader の継承関係図



公開メンバ関数

- [GgPointShader \(\)](#)
- [GgPointShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgPointShader](#) (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual [~GgPointShader \(\)](#)
- bool [load](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool [load](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void [loadProjectionMatrix](#) (const GLfloat *mp) const
- virtual void [loadProjectionMatrix](#) (const [GgMatrix](#) &mp) const
- virtual void [loadModelviewMatrix](#) (const GLfloat *mv) const
- virtual void [loadModelviewMatrix](#) (const [GgMatrix](#) &mv) const
- virtual void [loadMatrix](#) (const GLfloat *mp, const GLfloat *mv) const
- virtual void [loadMatrix](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- virtual void [use](#) () const
- void [use](#) (const GLfloat *mp) const
- void [use](#) (const [GgMatrix](#) &mp) const
- void [use](#) (const GLfloat *mp, const GLfloat *mv) const
- void [use](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- void [unuse](#) () const
- GLuint [get](#) () const

8.19.1 詳解

点のシェーダ.

gg.h の 6977 行目に定義があります。

8.19.2 構築子と解体子

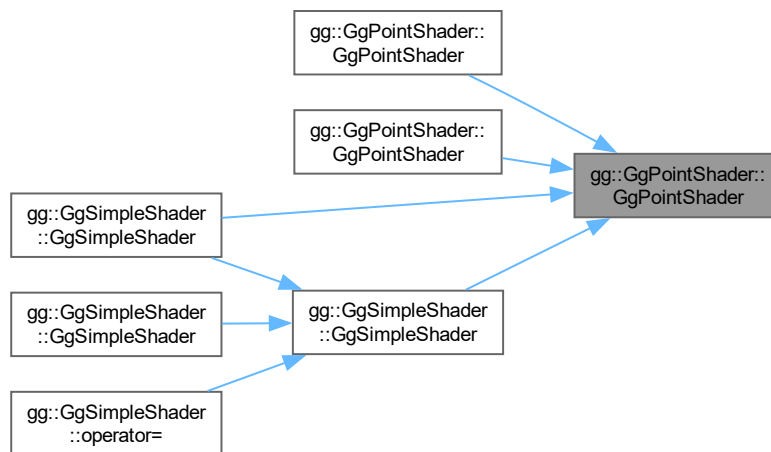
8.19.2.1 GgPointShader() [1/3]

`gg::GgPointShader::GgPointShader () [inline]`

コンストラクタ.

[gg.h](#) の 6993 行目に定義があります。

被呼び出し関係図:



8.19.2.2 GgPointShader() [2/3]

```

gg::GgPointShader::GgPointShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

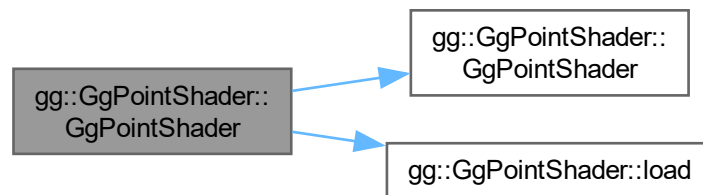
コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 7008 行目に定義があります。

呼び出し関係図:



8.19.2.3 GgPointShader() [3/3]

```
gg::GgPointShader::GgPointShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

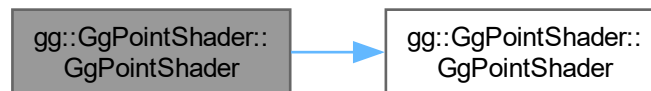
コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

[gg.h](#) の 7027 行目に定義があります。

呼び出し関係図:



8.19.2.4 ~GgPointShader()

```
virtual gg::GgPointShader::~GgPointShader () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 7039 行目に定義があります。

8.19.3 関数詳解

8.19.3.1 get()

```
GLuint gg::GgPointShader::get () const [inline]
```

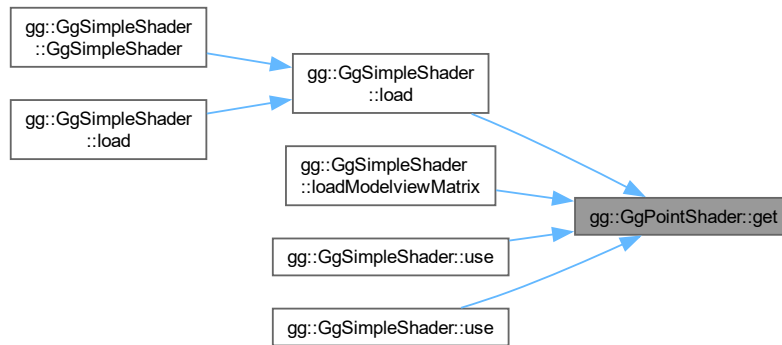
シェーダのプログラム名を得る.

戻り値

シェーダのプログラム名.

gg.h の 7223 行目に定義があります。

被呼び出し関係図:



8.19.3.2 load() [1/2]

```

bool gg::GgPointShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら true.

gg.h の 7086 行目に定義があります。

呼び出し関係図:



8.19.3.3 load() [2/2]

```
bool gg::GgPointShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込む。

引数

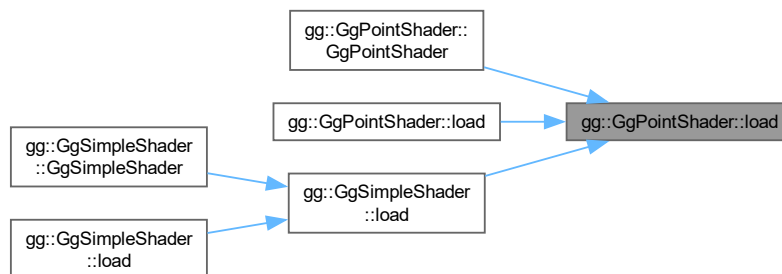
<i>vert</i>	バーテックスシェーダのソースファイル名。
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用)。
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用)。
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用)。
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト。

戻り値

プログラムオブジェクトが作成できれば true。

gg.h の 7053 行目に定義があります。

被呼び出し関係図:



8.19.3.4 loadMatrix() [1/2]

```
virtual void gg::GgPointShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する。

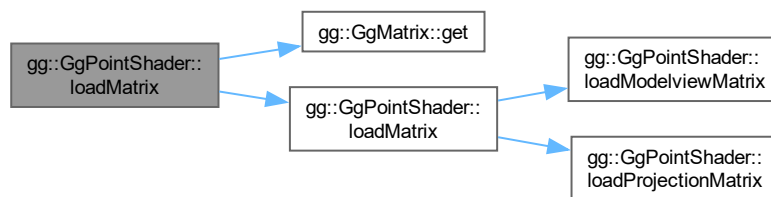
引数

<i>mp</i>	GgMatrix 型の投影変換行列。
<i>mv</i>	GgMatrix 型のモデルビュー変換行列。

gg::GgSimpleShader で再実装されています。

gg.h の 7153 行目に定義があります。

呼び出し関係図:



8.19.3.5 loadMatrix() [2/2]

```
virtual void gg::GgPointShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する.

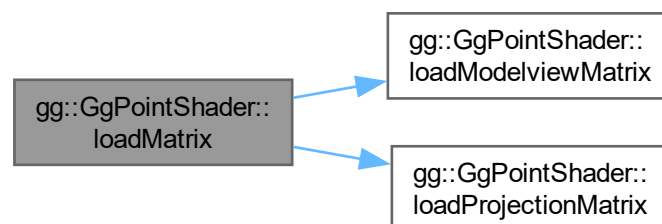
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

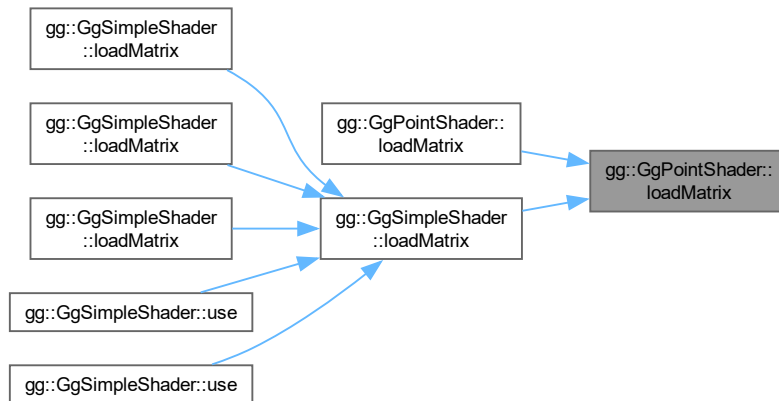
[gg::GgSimpleShader](#)で再実装されています。

[gg.h](#) の 7141 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.19.3.6 loadModelviewMatrix() [1/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

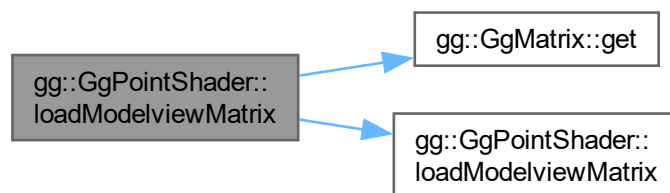
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	------------------------

gg::GgSimpleShaderで再実装されています。

gg.h の 7130 行目に定義があります。

呼び出し関係図:



8.19.3.7 loadModelviewMatrix() [2/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

8.19.3.9 loadProjectionMatrix() [2/2]

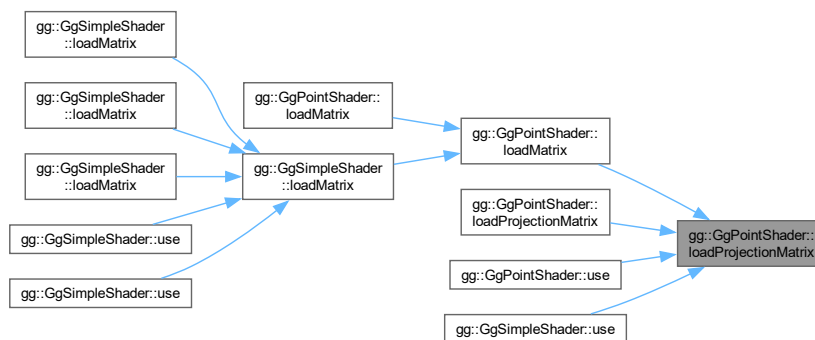
```
virtual void gg::GgPointShader::loadProjectionMatrix (
    const GLfloat * mp) const [inline], [virtual]
```

投影変換行列を設定する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列。
-----------	------------------------------------

gg.h の 7100 行目に定義があります。
被呼び出し関係図:



8.19.3.10 unuse()

```
void gg::GgPointShader::unuse () const [inline]
```

シェーダプログラムの使用を終了する。

gg.h の 7213 行目に定義があります。

8.19.3.11 use() [1/5]

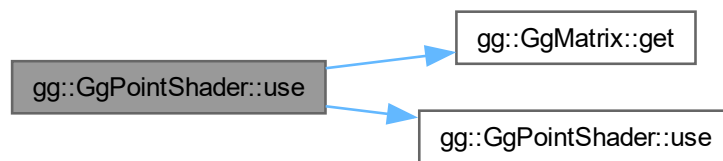
```
virtual void gg::GgPointShader::use () const [inline], [virtual]
```

シェーダプログラムの使用を開始する。

gg::GgSimpleShader で再実装されています。

gg.h の 7161 行目に定義があります。

呼び出し関係図:



8.19.3.13 use() [3/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

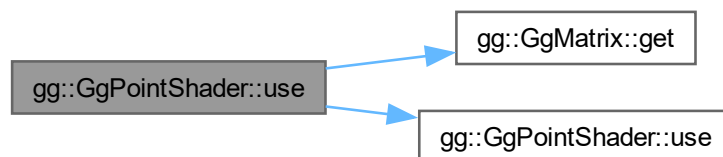
投影変換行列とモデルビューを設定してシェーダプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg.h の 7205 行目に定義があります。

呼び出し関係図:



8.19.3.14 use() [4/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp) const [inline]
```

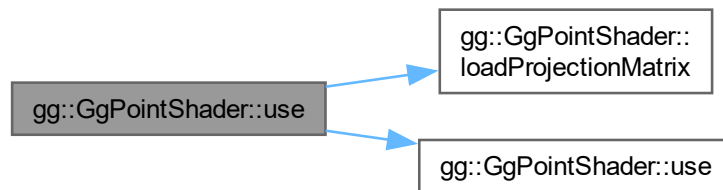
投影変換行列を設定してシェーダプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

gg.h の 7171 行目に定義があります。

呼び出し関係図:



8.19.3.15 use() [5/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

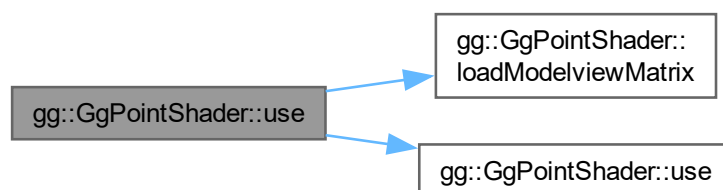
投影変換行列とモデルビューを設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

[gg.h](#) の 7193 行目に定義があります。

呼び出し関係図:



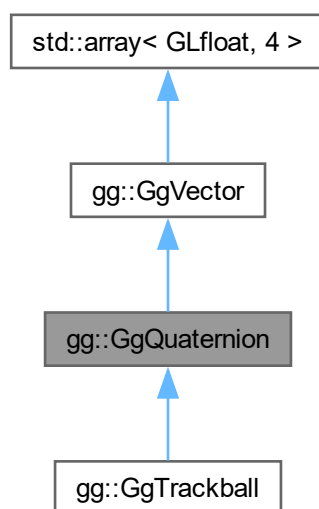
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

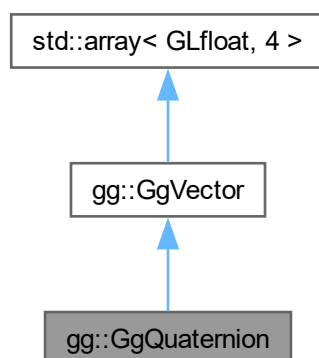
8.20 gg::GgQuaternion クラス

```
#include <gg.h>
```

gg::GgQuaternion の継承関係図



gg::GgQuaternion 連携図



公開メンバ関数

- `GgQuaternion()`=default
- `constexpr GgQuaternion (GLfloat x, GLfloat y, GLfloat z, GLfloat w)`
- `constexpr GgQuaternion (GLfloat c)`
- `GgQuaternion (const GLfloat *a)`
- `GgQuaternion (const GgVector &v)`
- `GgQuaternion (const GgQuaternion &q)`=default
- `GgQuaternion (GgQuaternion &&q)`=default
- `~GgQuaternion()`=default

- `GgQuaternion & operator= (const GgQuaternion &q)=default`
- `GgQuaternion & operator= (GgQuaternion &&q)=default`
- `GLfloat norm () const`
- `GgQuaternion & loadAdd (GLfloat x, GLfloat y, GLfloat z, GLfloat w)`
- `GgQuaternion & loadAdd (const GLfloat *a)`
- `GgQuaternion & loadAdd (const GgVector &v)`
- `GgQuaternion & loadAdd (const GgQuaternion &q)`
- `GgQuaternion & loadSubtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w)`
- `GgQuaternion & loadSubtract (const GLfloat *a)`
- `GgQuaternion & loadSubtract (const GgVector &v)`
- `GgQuaternion & loadSubtract (const GgQuaternion &q)`
- `GgQuaternion & loadMultiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w)`
- `GgQuaternion & loadMultiply (const GLfloat *a)`
- `GgQuaternion & loadMultiply (const GgVector &v)`
- `GgQuaternion & loadMultiply (const GgQuaternion &q)`
- `GgQuaternion & loadDivide (GLfloat x, GLfloat y, GLfloat z, GLfloat w)`
- `GgQuaternion & loadDivide (const GLfloat *a)`
- `GgQuaternion & loadDivide (const GgVector &v)`
- `GgQuaternion & loadDivide (const GgQuaternion &q)`
- `GgQuaternion add (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const`
- `GgQuaternion add (const GLfloat *a) const`
- `GgQuaternion add (const GgVector &v) const`
- `GgQuaternion add (const GgQuaternion &q) const`
- `GgQuaternion subtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const`
- `GgQuaternion subtract (const GLfloat *a) const`
- `GgQuaternion subtract (const GgVector &v) const`
- `GgQuaternion subtract (const GgQuaternion &q) const`
- `GgQuaternion multiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const`
- `GgQuaternion multiply (const GLfloat *a) const`
- `GgQuaternion multiply (const GgVector &v) const`
- `GgQuaternion multiply (const GgQuaternion &q) const`
- `GgQuaternion divide (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const`
- `GgQuaternion divide (const GLfloat *a) const`
- `GgQuaternion divide (const GgVector &v) const`
- `GgQuaternion divide (const GgQuaternion &q) const`
- `GgQuaternion & operator= (const GLfloat *a)`
- `GgQuaternion & operator= (const GgVector &v)`
- `GgQuaternion & operator+= (const GLfloat *a)`
- `GgQuaternion & operator+= (const GgVector &v)`
- `GgQuaternion & operator+= (const GgQuaternion &q)`
- `GgQuaternion & operator-= (const GLfloat *a)`
- `GgQuaternion & operator-= (const GgVector &v)`
- `GgQuaternion & operator-= (const GgQuaternion &q)`
- `GgQuaternion & operator*= (const GLfloat *a)`
- `GgQuaternion & operator*= (const GgVector &v)`
- `GgQuaternion & operator*= (const GgQuaternion &q)`
- `GgQuaternion & operator/= (const GLfloat *a)`
- `GgQuaternion & operator/= (const GgVector &v)`
- `GgQuaternion & operator/= (const GgQuaternion &q)`
- `GgQuaternion operator+ (const GLfloat *a) const`
- `GgQuaternion operator+ (const GgVector &v) const`
- `GgQuaternion operator+ (const GgQuaternion &q) const`
- `GgQuaternion operator- (const GLfloat *a) const`
- `GgQuaternion operator- (const GgVector &v) const`
- `GgQuaternion operator- (const GgQuaternion &q) const`

- [GgQuaternion operator*](#) (const GLfloat *a) const
- [GgQuaternion operator*](#) (const [GgVector](#) &v) const
- [GgQuaternion operator*](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator/](#) (const GLfloat *a) const
- [GgQuaternion operator/](#) (const [GgVector](#) &v) const
- [GgQuaternion operator/](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion & loadMatrix](#) (const GLfloat *a)
- [GgQuaternion & loadMatrix](#) (const [GgMatrix](#) &m)
- [GgQuaternion & loadIdentity](#) ()
- [GgQuaternion & loadRotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- [GgQuaternion & loadRotate](#) (const GLfloat *v, GLfloat a)
- [GgQuaternion & loadRotate](#) (const GLfloat *v)
- [GgQuaternion & loadRotateX](#) (GLfloat a)
- [GgQuaternion & loadRotateY](#) (GLfloat a)
- [GgQuaternion & loadRotateZ](#) (GLfloat a)
- [GgQuaternion rotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- [GgQuaternion rotate](#) (const GLfloat *v, GLfloat a) const
- [GgQuaternion rotate](#) (const GLfloat *v) const
- [GgQuaternion rotateX](#) (GLfloat a) const
- [GgQuaternion rotateY](#) (GLfloat a) const
- [GgQuaternion rotateZ](#) (GLfloat a) const
- [GgQuaternion & loadEuler](#) (GLfloat heading, GLfloat pitch, GLfloat roll)
- [GgQuaternion & loadEuler](#) (const GLfloat *e)
- [GgQuaternion euler](#) (GLfloat heading, GLfloat pitch, GLfloat roll) const
- [GgQuaternion euler](#) (const GLfloat *e) const
- [GgQuaternion euler](#) (const [GgVector](#) &e) const
- [GgQuaternion & loadSlerp](#) (const GLfloat *a, const GLfloat *b, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const [GgQuaternion](#) &q, const [GgQuaternion](#) &r, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const [GgQuaternion](#) &q, const GLfloat *a, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const GLfloat *a, const [GgQuaternion](#) &q, GLfloat t)
- [GgQuaternion & loadNormalize](#) (const GLfloat *a)
- [GgQuaternion & loadNormalize](#) (const [GgQuaternion](#) &q)
- [GgQuaternion & loadConjugate](#) (const GLfloat *a)
- [GgQuaternion & loadConjugate](#) (const [GgQuaternion](#) &q)
- [GgQuaternion & loadInvert](#) (const GLfloat *a)
- [GgQuaternion & loadInvert](#) (const [GgQuaternion](#) &q)
- [GgQuaternion slerp](#) (GLfloat *a, GLfloat t) const
- [GgQuaternion slerp](#) (const [GgQuaternion](#) &q, GLfloat t) const
- [GgQuaternion normalize](#) () const
- [GgQuaternion conjugate](#) () const
- [GgQuaternion invert](#) () const
- void [get](#) (GLfloat *a) const
- void [getMatrix](#) (GLfloat *a) const
- void [getMatrix](#) ([GgMatrix](#) &m) const
- [GgMatrix getMatrix](#) () const
- void [getConjugateMatrix](#) (GLfloat *a) const
- void [getConjugateMatrix](#) ([GgMatrix](#) &m) const
- [GgMatrix getConjugateMatrix](#) () const

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & operator= (const **GgVector** &v)=default
- **GgVector** & operator= (**GgVector** &&v)=default
- **GgVector** operator+ (const **GgVector** &v) const
- **GgVector** operator+ (GLfloat c) const
- **GgVector** & operator+= (const **GgVector** &v)
- **GgVector** & operator+= (GLfloat c)
- **GgVector** operator- (const **GgVector** &v) const
- **GgVector** operator- (GLfloat c) const
- **GgVector** & operator-= (const **GgVector** &v)
- **GgVector** & operator-= (GLfloat c)
- **GgVector** operator* (const **GgVector** &v) const
- **GgVector** operator* (GLfloat c) const
- **GgVector** & operator*= (const **GgVector** &v)
- **GgVector** & operator*= (GLfloat c)
- **GgVector** operator/ (const **GgVector** &v) const
- **GgVector** operator/ (GLfloat c) const
- **GgVector** & operator/= (**GgVector** &v)
- **GgVector** & operator/= (GLfloat c)
- GLfloat dot3 (const **GgVector** &v) const
- GLfloat length3 () const
- GLfloat distance3 (const **GgVector** &v) const
- **GgVector** normalize3 () const
- GLfloat dot4 (const **GgVector** &v) const
- GLfloat length4 () const
- GLfloat distance4 (const **GgVector** &v) const
- **GgVector** normalize4 () const

8.20.1 詳解

四元数.

gg.h の 3452 行目に定義があります。

8.20.2 構築子と解体子

8.20.2.1 GgQuaternion() [1/7]

```
gg::GgQuaternion::GgQuaternion () [default]
コンストラクタ.
```

8.20.2.2 GgQuaternion() [2/7]

```
gg::GgQuaternion::GgQuaternion (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline], [constexpr]
コンストラクタ.
```

引数

x	四元数の x 要素.
y	四元数の y 要素.
z	四元数の z 要素.
w	四元数の w 要素.

gg.h の 3481 行目に定義があります。
呼び出し関係図:



8.20.2.3 GgQuaternion() [3/7]

```
gg::GgQuaternion::GgQuaternion (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

c	GLfloat 型の値.
-----	--------------

gg.h の 3491 行目に定義があります。
呼び出し関係図:



8.20.2.4 GgQuaternion() [4/7]

```
gg::GgQuaternion::GgQuaternion (
    const GLfloat * a) [inline]
```

コンストラクタ.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

gg.h の 3501 行目に定義があります。

呼び出し関係図:



8.20.2.5 GgQuaternion() [5/7]

```
gg::GgQuaternion::GgQuaternion (
    const GgVector & v) [inline]
```

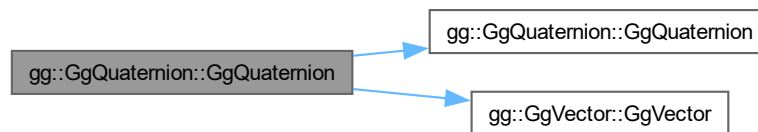
コンストラクタ.

引数

<i>v</i>	四元数を格納した GgVector 型の変数.
----------	--------------------------------

[gg.h](#) の [3511](#) 行目に定義があります。

呼び出し関係図:



8.20.2.6 GgQuaternion() [6/7]

```
gg::GgQuaternion::GgQuaternion (
    const GgQuaternion & q) [default]
```

コピーコンストラクタ.

引数

<i>q</i>	コピー元の四元数.
----------	-----------

呼び出し関係図:



8.20.2.7 GgQuaternion() [7/7]

```
gg::GgQuaternion::GgQuaternion (
    GgQuaternion && q) [default]
ムーブコンストラクタ.
```

引数

<i>q</i>	ムーブ元の四元数.
----------	-----------

呼び出し関係図:



8.20.2.8 ~GgQuaternion()

```
gg::GgQuaternion::~~GgQuaternion () [default]
デストラクタ.
```

8.20.3 関数詳解

8.20.3.1 add() [1/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgQuaternion & q) const [inline]
四元数に別の四元数を加算した結果を返す.
```

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

q を加えた四元数.

[gg.h](#) の 3804 行目に定義があります。

呼び出し関係図:



8.20.3.2 add() [2/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgVector & v) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

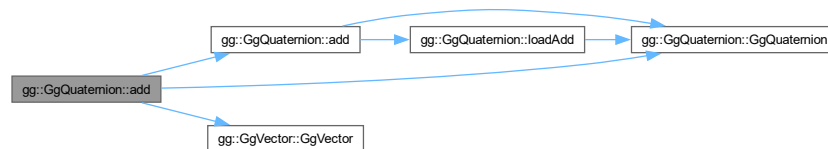
v	四元数を格納した GgVector 型の変数.
----------	--------------------------------

戻り値

v を加えた四元数.

gg.h の 3793 行目に定義があります.

呼び出し関係図:



8.20.3.3 add() [3/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GLfloat * a) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	---------------------------------------

戻り値

a を加えた四元数.

gg.h の 3782 行目に定義があります.

呼び出し関係図:



8.20.3.4 add() [4/4]

```
GgQuaternion gg::GgQuaternion::add (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

x	加える四元数の x 要素.
y	加える四元数の y 要素.
z	加える四元数の z 要素.
w	加える四元数の w 要素.

戻り値

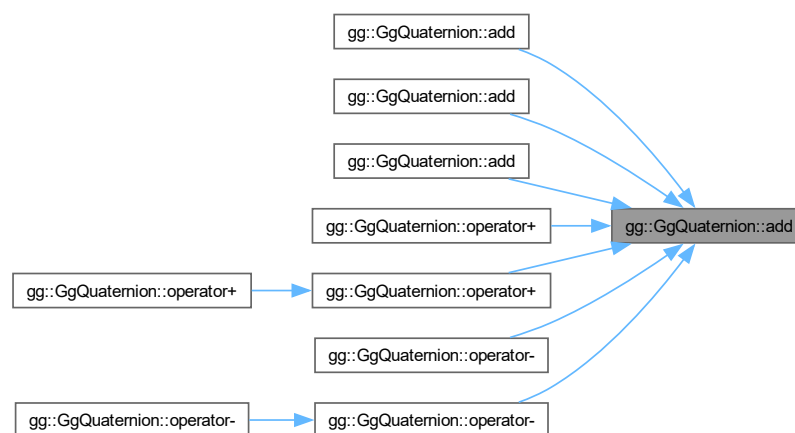
(x, y, z, w) を加えた四元数.

[gg.h](#) の 3770 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.5 conjugate()

`GgQuaternion gg::GgQuaternion::conjugate () const [inline]`

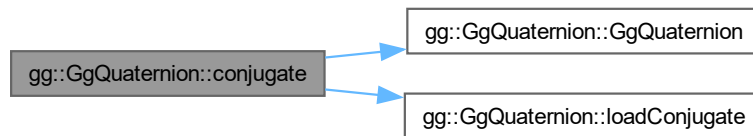
共役四元数に変換する。

戻り値

共役四元数.

[gg.h](#) の 4441 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.6 divide() [1/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GgQuaternion & q) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

q で割った四元数.

[gg.h](#) の 3953 行目に定義があります。

呼び出し関係図:



8.20.3.7 divide() [2/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GgVector & v) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

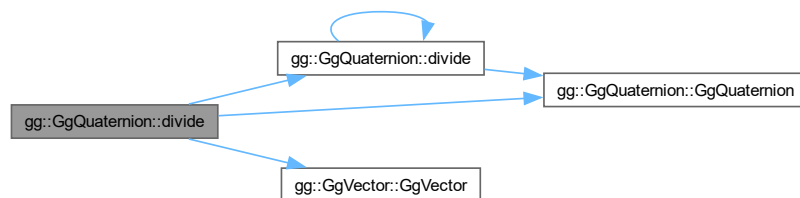
v	四元数を格納した GgVector 型の変数.
---	-------------------------

戻り値

v で割った四元数.

gg.h の 3942 行目に定義があります。

呼び出し関係図:



8.20.3.8 divide() [3/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GLfloat * a) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

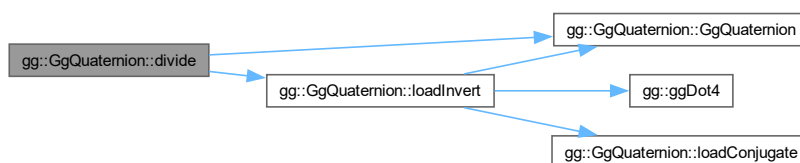
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

a で割った四元数.

gg.h の 3928 行目に定義があります。

呼び出し関係図:



8.20.3.9 divide() [4/4]

```
GgQuaternion gg::GgQuaternion::divide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数を別の四元数で除算した結果を返す。

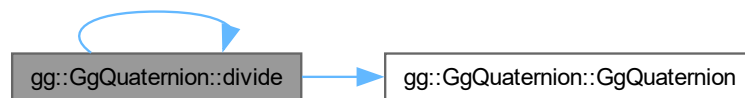
引数

x	割る四元数の x 要素.
y	割る四元数の y 要素.
z	割る四元数の z 要素.
w	割る四元数の w 要素.

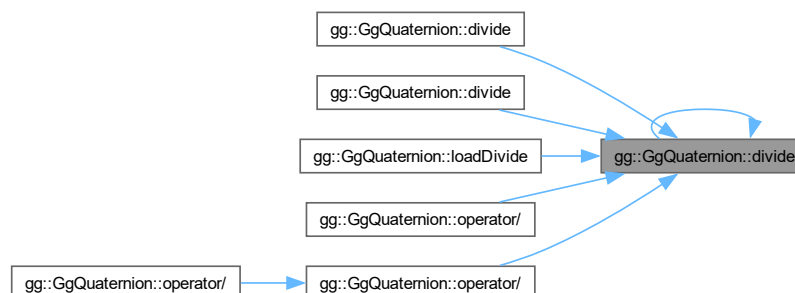
戻り値

(x, y, z, w) を割った四元数.

[gg.h](#) の 3916 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.20.3.10 euler() [1/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GgVector & e) const [inline]
```

四元数をオイラー角 ($e[0], e[1], e[2]$) で回転した四元数を返す。

引数

e	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
---	--

戻り値

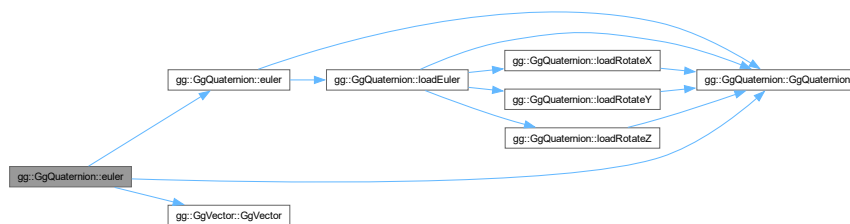
回転した四元数.

覚え書き

第 4 要素は無視する.

[gg.h](#) の 4281 行目に定義があります.

呼び出し関係図:



8.20.3.11 euler() [2/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GLfloat * e) const [inline]
```

四元数をオイラー角 (e[0], e[1], e[2]) で回転した四元数を返す.

引数

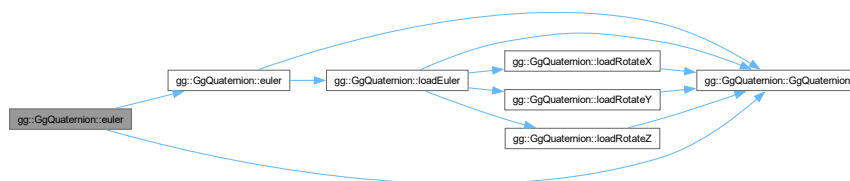
e	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
---	---

戻り値

回転した四元数.

[gg.h](#) の 4267 行目に定義があります.

呼び出し関係図:



8.20.3.12 euler() [3/3]

```
GgQuaternion gg::GgQuaternion::euler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) const [inline]
```

四元数をオイラー角 (heading, pitch, roll) で回転した四元数を返す.

引数

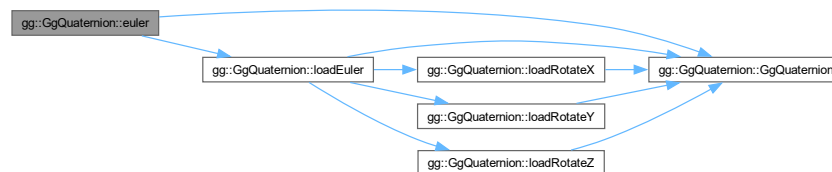
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

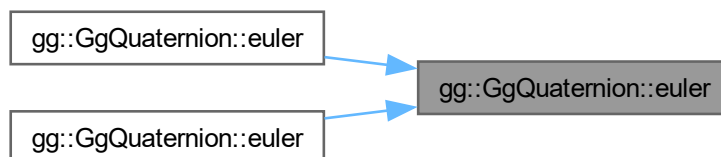
回転した四元数.

[gg.h](#) の 4255 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.20.3.13 get()

```
void gg::GgQuaternion::get (
    GLfloat * a) const [inline]
```

四元数を取り出す.

引数

<i>a</i>	四元数を格納する GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

gg.h の 4465 行目に定義があります。

8.20.3.14 getConjugateMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getConjugateMatrix () const [inline]
```

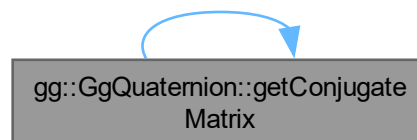
四元数の共役が表す回転の変換行列を取り出す.

戻り値

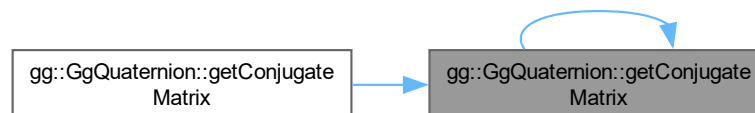
回転の変換を表す GgMatrix 型の変換行列.

gg.h の 4532 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.15 getConjugateMatrix() [2/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GgMatrix & m) const [inline]
```

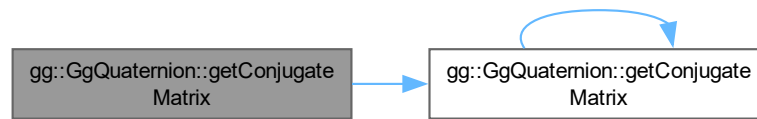
四元数の共役が表す回転の変換行列を m に求める.

引数

<i>m</i>	回転の変換行列を格納する GgMatrix 型の変数.
----------	-----------------------------

gg.h の 4522 行目に定義があります。

呼び出し関係図:



8.20.3.16 getConjugateMatrix() [3/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GLfloat * a) const [inline]
```

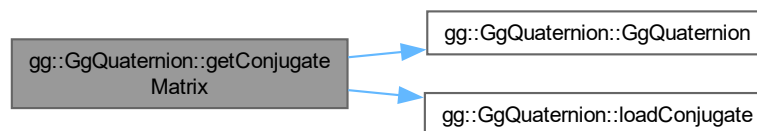
四元数の共役が表す回転の変換行列を **a** に求める.

引数

a	回転の変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	-------------------------------------

[gg.h](#) の 4510 行目に定義があります。

呼び出し関係図:



8.20.3.17 getMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getMatrix () const [inline]
```

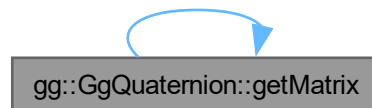
四元数が表す回転の変換行列を取り出す.

戻り値

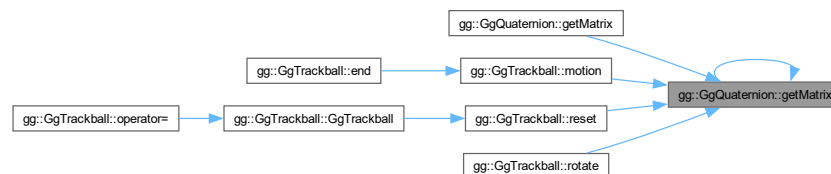
回転の変換を表す **GgMatrix** 型の変換行列.

gg.h の 4498 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.18 getMatrix() [2/3]

```
void gg::GgQuaternion::getMatrix (
    GgMatrix & m) const [inline]
```

四元数が表す回転の変換行列を m に求める。

引数

<i>m</i>	回転の変換行列を格納する GgMatrix 型の変数.
----------	------------------------------------

gg.h の 4488 行目に定義があります。

呼び出し関係図:



8.20.3.19 getMatrix() [3/3]

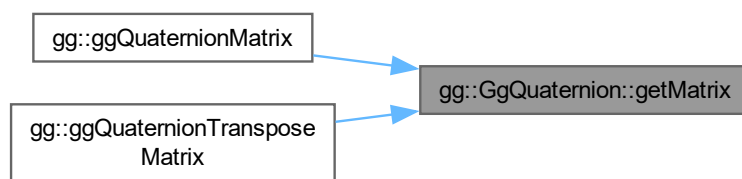
```
void gg::GgQuaternion::getMatrix (
    GLfloat * a) const [inline]
```

四元数が表す回転の変換行列を **a** に求める。

引数

a	回転の変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	-------------------------------------

[gg.h](#) の 4478 行目に定義があります。
被呼び出し関係図:



8.20.3.20 invert()

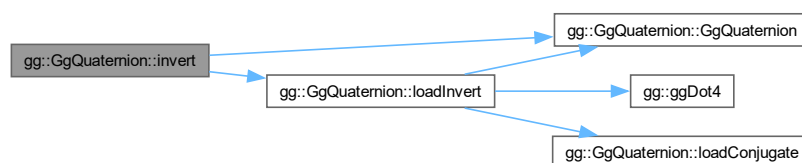
```
GgQuaternion gg::GgQuaternion::invert () const [inline]
```

逆元に変換する。

戻り値

四元数の逆元。

[gg.h](#) の 4453 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.20.3.21 loadAdd() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q を加えた四元数.

gg.h の 3608 行目に定義があります。
呼び出し関係図:



8.20.3.22 loadAdd() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgVector & v) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

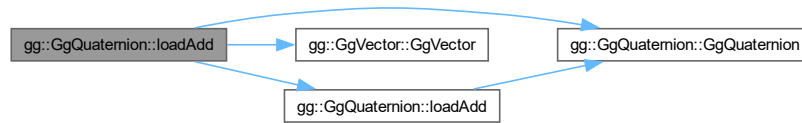
<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

戻り値

v を加えた四元数.

gg.h の 3597 行目に定義があります。

呼び出し関係図:



8.20.3.23 loadAdd() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GLfloat * a) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を加えた四元数.

[gg.h](#) の 3586 行目に定義があります。

呼び出し関係図:



8.20.3.24 loadAdd() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>x</i>	加える四元数の <i>x</i> 要素.
<i>y</i>	加える四元数の <i>y</i> 要素.
<i>z</i>	加える四元数の <i>z</i> 要素.
<i>w</i>	加える四元数の <i>w</i> 要素.

戻り値

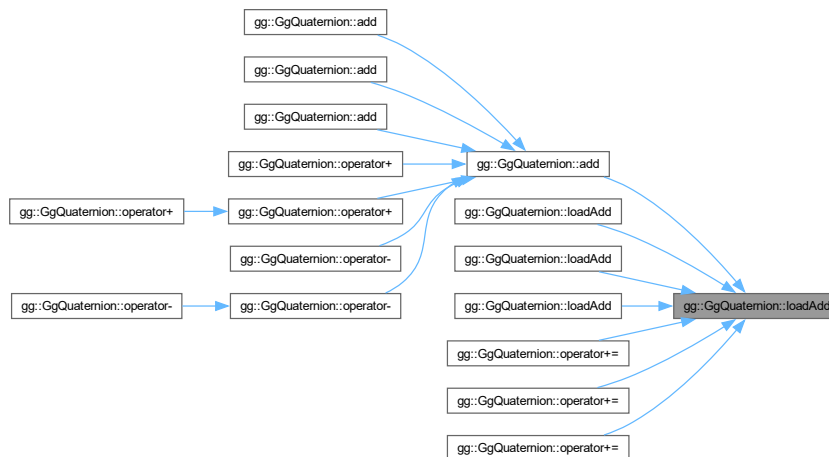
(x, y, z, w) を加えた四元数.

gg.h の 3570 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.25 loadConjugate() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadConjugate (
    const GgQuaternion & q) [inline]
```

引数に指定した四元数の共役四元数を格納する。

引数

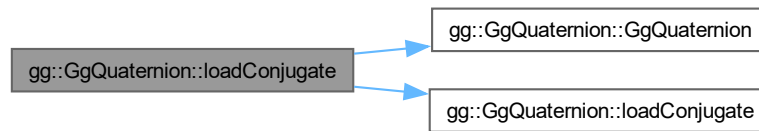
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

共役四元数.

[gg.h](#) の 4372 行目に定義があります。

呼び出し関係図:



8.20.3.26 loadConjugate() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadConjugate (
    const GLfloat * a)
```

引数に指定した四元数の共役四元数を格納する.

引数

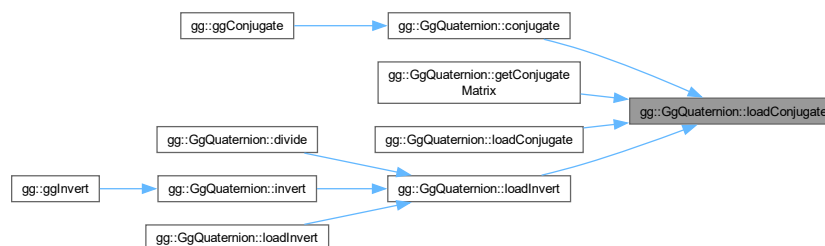
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

共役四元数.

[gg.cpp](#) の 3384 行目に定義があります。

被呼び出し関係図:



8.20.3.27 loadDivide() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgQuaternion & q) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

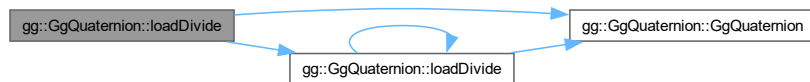
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

q で割った四元数.

gg.h の 3756 行目に定義があります。

呼び出し関係図:



8.20.3.28 loadDivide() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgVector & v) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

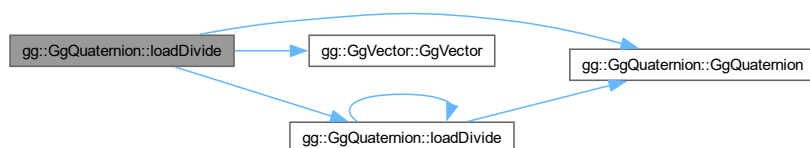
v	四元数を格納した GgVector 型の変数.
-----	-------------------------

戻り値

v で割った四元数.

gg.h の 3745 行目に定義があります。

呼び出し関係図:



8.20.3.29 loadDivide() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GLfloat * a) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a で割った四元数.

gg.h の 3734 行目に定義があります。

呼び出し関係図:



8.20.3.30 loadDivide() [4/4]

```

GgQuaternion & gg::GgQuaternion::loadDivide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
  
```

四元を別の四元数で除算した結果を格納する.

引数

x	割る四元数の x 要素.
y	割る四元数の y 要素.
z	割る四元数の z 要素.
w	割る四元数の w 要素.

戻り値

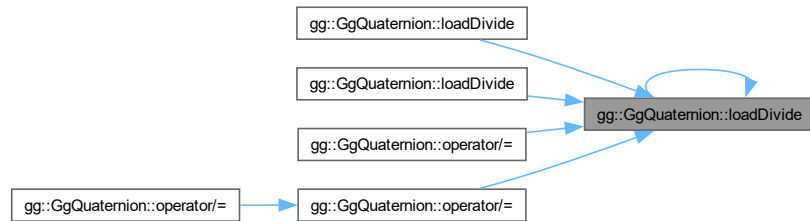
(x, y, z, w) を割った四元数.

gg.h の 3722 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.31 loadEuler() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadEuler (
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を格納する.

引数

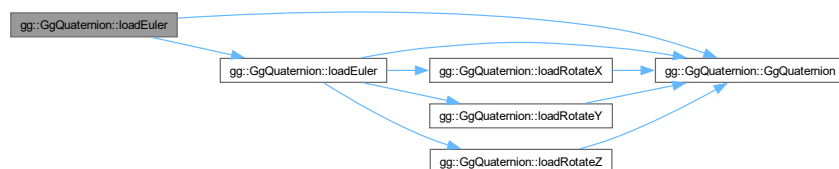
e	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
---	---

戻り値

格納した回転を表す四元数.

gg.h の 4242 行目に定義があります。

呼び出し関係図:



8.20.3.32 loadEuler() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadEuler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll)
```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を格納する.

引数

heading	y 軸中心の回転角.
pitch	x 軸中心の回転角.

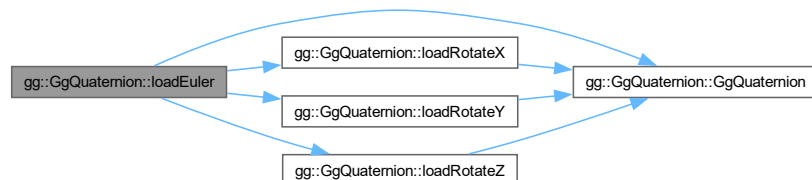
<i>roll</i>	z 軸中心の回転角.
-------------	------------

戻り値

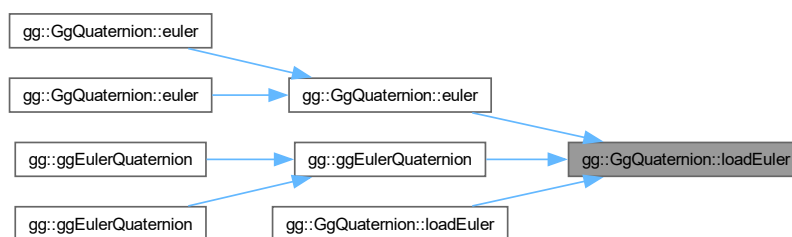
格納した回転を表す四元数.

[gg.cpp](#) の 3356 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.33 loadIdentity()

[GgQuaternion](#) & [gg::GgQuaternion::loadIdentity](#) () [inline]

単位元を格納する.

戻り値

格納された単位元.

[gg.h](#) の 4092 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.34 loadInvert() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadInvert (
    const GgQuaternion & q) [inline]
```

引数に指定した四元数の逆元を格納する.

引数

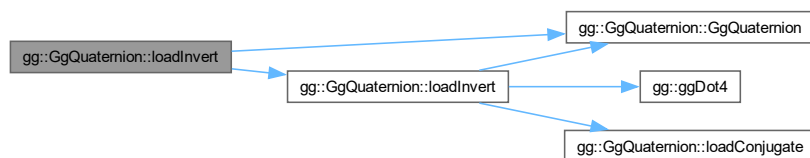
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数の逆元.

gg.h の 4391 行目に定義があります。

呼び出し関係図:



8.20.3.35 loadInvert() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadInvert (
    const GLfloat * a)
```

引数に指定した四元数の逆元を格納する.

引数

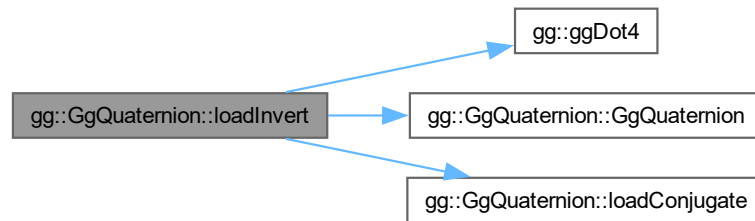
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

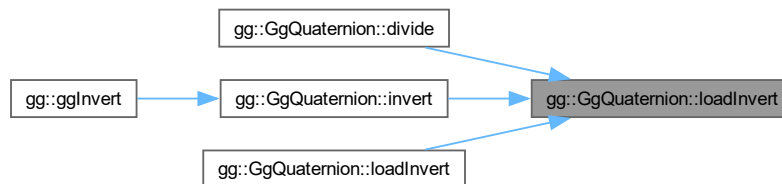
四元数の逆元.

`gg.cpp` の 3398 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.36 loadMatrix() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GgMatrix & m) [inline]
```

回転の変換行列 `m` を表す四元数を格納する.

引数

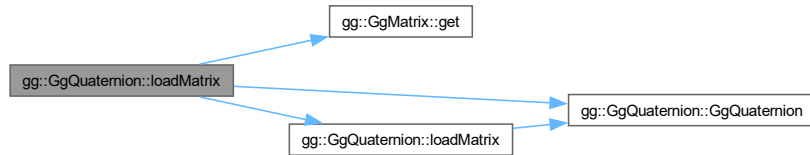
<code>m</code>	Ggmatrix 型の変換行列.
----------------	------------------

戻り値

m による回転の変換に相当する四元数.

gg.h の 4082 行目に定義があります。

呼び出し関係図:



8.20.3.37 loadMatrix() [2/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GLfloat * a) [inline]
```

回転の変換行列を表す四元数を格納する.

引数

a	GLfloat 型の 16 要素の変換行列.
---	------------------------

戻り値

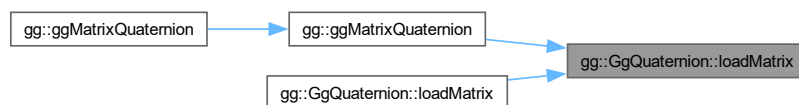
a による回転の変換に相当する四元数.

gg.h の 4070 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.38 loadMultiply() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

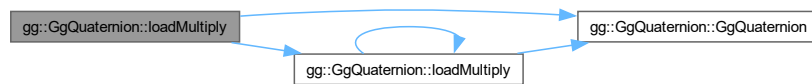
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q を乗じた四元数.

gg.h の 3708 行目に定義があります。

呼び出し関係図:



8.20.3.39 loadMultiply() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgVector & v) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

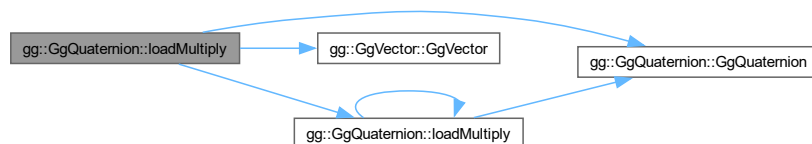
<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

戻り値

v を乗じた四元数.

gg.h の 3697 行目に定義があります。

呼び出し関係図:



8.20.3.40 loadMultiply() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GLfloat * a) [inline]
```


四元数に別の四元数を乗算した結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を乗じた四元数.

gg.h の 3686 行目に定義があります。
呼び出し関係図:



8.20.3.41 loadMultiply() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

<i>x</i>	掛ける四元数の x 要素.
<i>y</i>	掛ける四元数の y 要素.
<i>z</i>	掛ける四元数の z 要素.
<i>w</i>	掛ける四元数の w 要素.

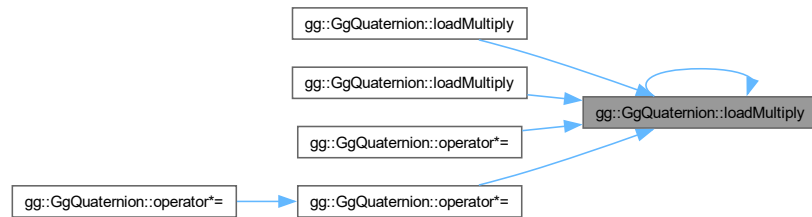
戻り値

(*x*, *y*, *z*, *w*) を掛けた四元数.

gg.h の 3674 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.20.3.42 loadNormalize() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadNormalize (
    const GgQuaternion & q) [inline]
```

引数に指定した四元数を正規化して格納する.

引数

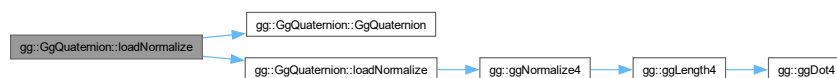
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

正規化された四元数.

gg.h の 4353 行目に定義があります.

呼び出し関係図:



8.20.3.43 loadNormalize() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadNormalize (
    const GLfloat * a)
```

引数に指定した四元数を正規化して格納する.

引数

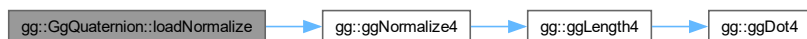
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

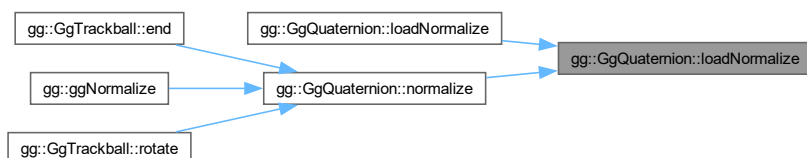
正規化された四元数.

gg.cpp の 3372 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.44 loadRotate() [1/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を格納する.

引数

v	軸ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------------

戻り値

格納された回転を表す四元数.

gg.h の 4126 行目に定義があります。

呼び出し関係図:



8.20.3.45 loadRotate() [2/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を格納する.

引数

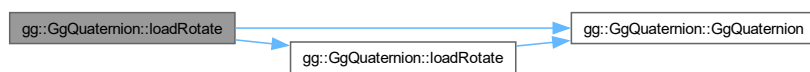
v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

格納された回転を表す四元数.

gg.h の 4115 行目に定義があります.

呼び出し関係図:



8.20.3.46 loadRotate() [3/3]

```

gg::GgQuaternion & gg::GgQuaternion::loadRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a)
  
```

(x, y, z) を軸として角度 a 回転する四元数を格納する.

引数

x	軸ベクトルの x 成分.
y	軸ベクトルの y 成分.
z	軸ベクトルの z 成分.
a	回転角.

戻り値

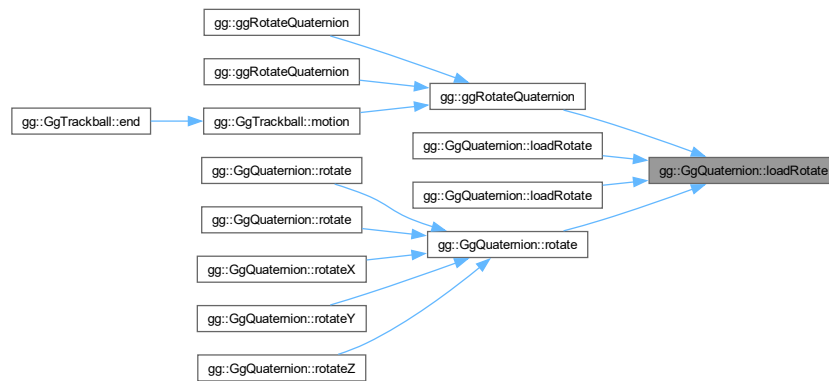
格納された回転を表す四元数.

gg.cpp の 3298 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.20.3.47 loadRotateX()

`gg::GgQuaternion & gg::GgQuaternion::loadRotateX (GLfloat a)`

x 軸中心に角度 *a* 回転する四元数を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

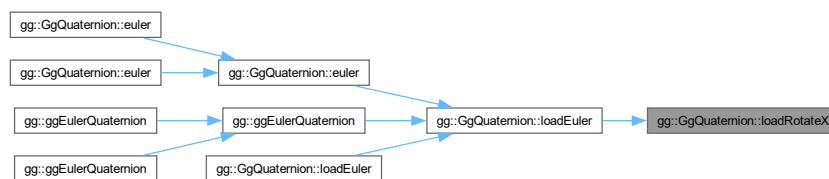
格納された回転を表す四元数.

`gg.cpp` の 3320 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.20.3.48 loadRotateY()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateY (
    GLfloat a)
```

y 軸中心に角度 **a** 回転する四元数を格納する.

引数

a	回転角.
----------	------

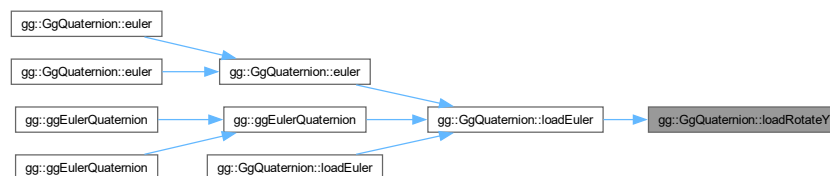
戻り値

格納された回転を表す四元数.

[gg.cpp](#) の 3332 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.20.3.49 loadRotateZ()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateZ (
    GLfloat a)
```

z 軸中心に角度 **a** 回転する四元数を格納する.

引数

a	回転角.
----------	------

戻り値

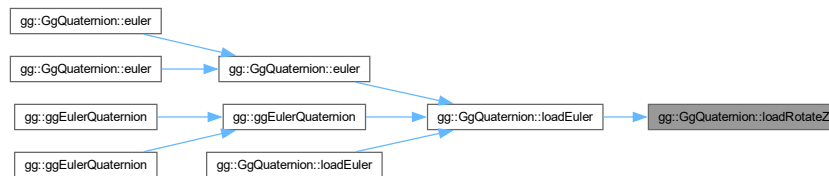
格納された回転を表す四元数.

gg.cpp の 3344 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.20.3.50 loadSlerp() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した *q*, *r* を *t* で内分した四元数.

gg.h の 4308 行目に定義があります.

呼び出し関係図:



8.20.3.51 loadSlerp() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GLfloat * a,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

格納した *q*, *a* を *t* で内分した四元数.

[gg.h](#) の 4321 行目に定義があります.

呼び出し関係図:



8.20.3.52 loadSlerp() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した a, q を t で内分した四元数.

gg.h の 4334 行目に定義があります。

呼び出し関係図:



8.20.3.53 loadSlerp() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
b	四元数を格納した GLfloat 型の 4 要素の配列変数.
t	補間パラメータ.

戻り値

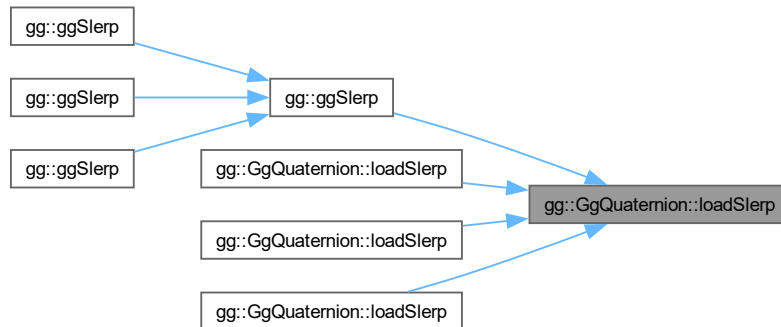
格納した a, b を t で内分した四元数.

gg.h の 4294 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.54 loadSubtract() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgQuaternion & q) [inline]
```

四元数から別の四元数を減算した結果を格納する。

引数

<i>q</i>	<code>GgQuaternion</code> 型の四元数.
----------	----------------------------------

戻り値

q を引いた四元数.

`gg.h` の 3660 行目に定義があります。

呼び出し関係図:



8.20.3.55 loadSubtract() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgVector & v) [inline]
```

四元数から別の四元数を減算した結果を格納する。

引数

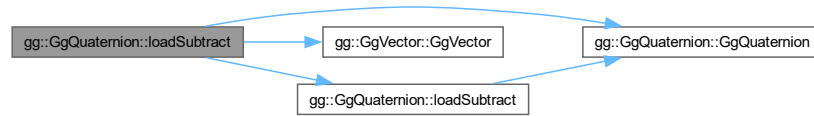
<i>v</i>	四元数を格納した <code>GgVector</code> 型の変数.
----------	--------------------------------------

戻り値

v を引いた四元数.

gg.h の 3649 行目に定義があります。

呼び出し関係図:



8.20.3.56 loadSubtract() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GLfloat * a) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

a を引いた四元数.

gg.h の 3638 行目に定義があります。

呼び出し関係図:



8.20.3.57 loadSubtract() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

x	引く四元数の x 要素.
y	引く四元数の y 要素.
z	引く四元数の z 要素.

<code>w</code>	引く四元数の <code>w</code> 要素.
----------------	---------------------------

戻り値

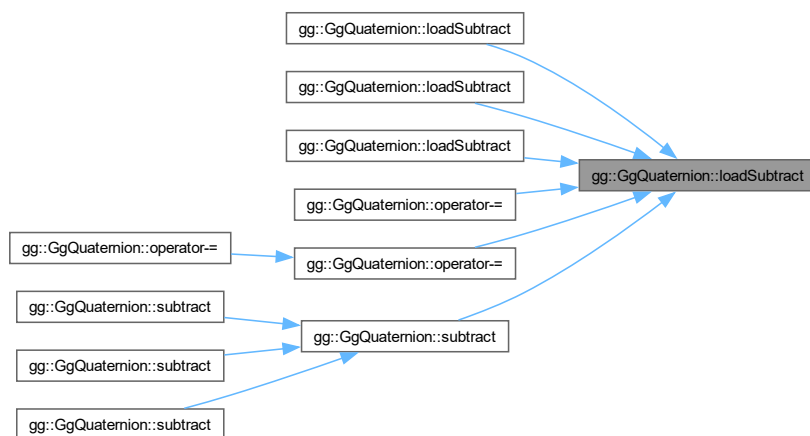
(`x`, `y`, `z`, `w`) を引いた四元数.

`gg.h` の 3622 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.20.3.58 multiply() [1/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GgQuaternion & q) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

<code>q</code>	<code>GgQuaternion</code> 型の四元数.
----------------	----------------------------------

戻り値

`q` を掛けた四元数.

`gg.h` の 3902 行目に定義があります.

呼び出し関係図:



8.20.3.59 multiply() [2/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GgVector & v) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

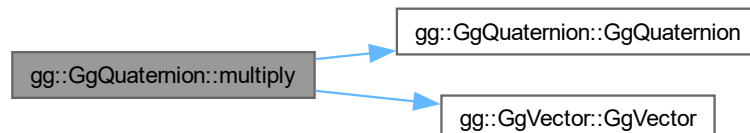
v	四元数を格納した GgVector 型の変数.
----------	--------------------------------

戻り値

v を掛けた四元数.

gg.h の 3891 行目に定義があります.

呼び出し関係図:



8.20.3.60 multiply() [3/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GLfloat * a) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	---------------------------------------

戻り値

a を掛けた四元数.

gg.h の 3878 行目に定義があります。

呼び出し関係図:



8.20.3.61 multiply() [4/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

x	掛ける四元数の x 要素.
y	掛ける四元数の y 要素.
z	掛ける四元数の z 要素.
w	掛ける四元数の w 要素.

戻り値

(x, y, z, w) を掛けた四元数.

gg.h の 3866 行目に定義があります。

呼び出し関係図:



8.20.3.62 norm()

```
GLfloat gg::GgQuaternion::norm () const [inline]
```

四元数のノルムを求める.

戻り値

四元数のノルム.

gg.h の 3556 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.63 normalize()

```
GgQuaternion gg::GgQuaternion::normalize () const [inline]
```

正規化する.

戻り値

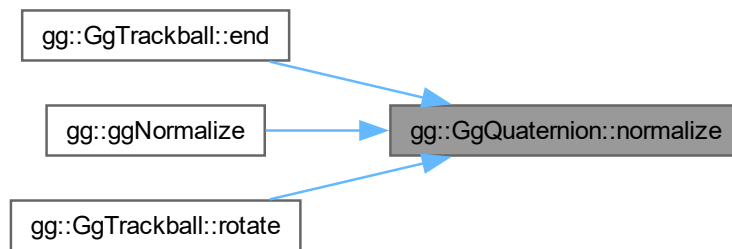
正規化された四元数.

gg.h の 4429 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.64 `operator*()` [1/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GgQuaternion & q) const [inline]
```

`gg.h` の 4047 行目に定義があります。

呼び出し関係図:

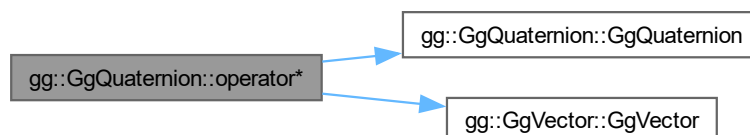


8.20.3.65 `operator*()` [2/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GgVector & v) const [inline]
```

`gg.h` の 4043 行目に定義があります。

呼び出し関係図:



8.20.3.66 `operator*()` [3/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GLfloat * a) const [inline]
```


gg.h の 4039 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.20.3.67 operator*=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgQuaternion & q) [inline]
```

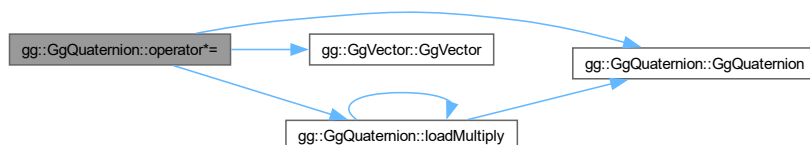
gg.h の 3999 行目に定義があります。
呼び出し関係図:



8.20.3.68 operator*=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgVector & v) [inline]
```

gg.h の 3995 行目に定義があります。
呼び出し関係図:

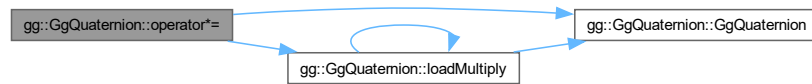


8.20.3.69 operator*=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GLfloat * a) [inline]
```

gg.h の 3991 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.70 operator+() [1/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgQuaternion & q) const [inline]
```

gg.h の 4023 行目に定義があります。

呼び出し関係図:

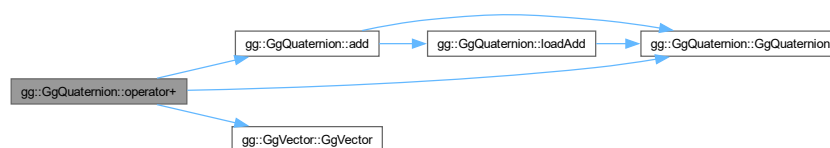


8.20.3.71 operator+() [2/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgVector & v) const [inline]
```

gg.h の 4019 行目に定義があります。

呼び出し関係図:



8.20.3.72 operator+() [3/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GLfloat * a) const [inline]
```

gg.h の 4015 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:

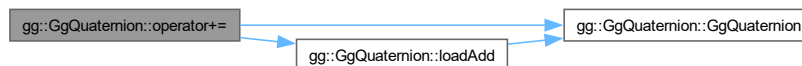


8.20.3.73 operator+=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgQuaternion & q) [inline]
```

gg.h の 3975 行目に定義があります。

呼び出し関係図:

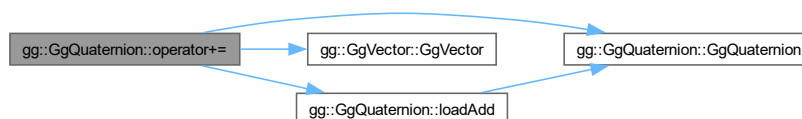


8.20.3.74 operator+=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgVector & v) [inline]
```

gg.h の 3971 行目に定義があります。

呼び出し関係図:

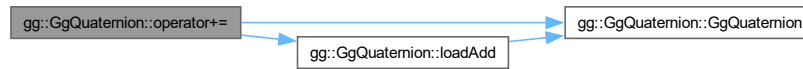


8.20.3.75 operator+=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GLfloat * a) [inline]
```

gg.h の 3967 行目に定義があります。

呼び出し関係図:

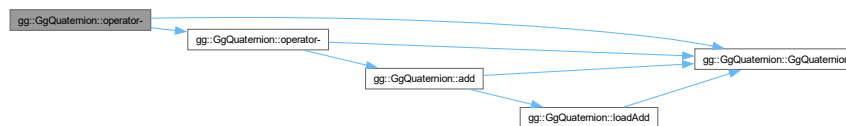


8.20.3.76 operator-() [1/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgQuaternion & q) const [inline]
```

gg.h の 4035 行目に定義があります。

呼び出し関係図:

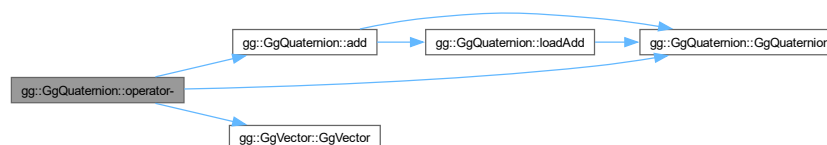


8.20.3.77 operator-() [2/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgVector & v) const [inline]
```

gg.h の 4031 行目に定義があります。

呼び出し関係図:



8.20.3.78 operator-() [3/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GLfloat * a) const [inline]
```

gg.h の 4027 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.79 operator-=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgQuaternion & q) [inline]
```

gg.h の 3987 行目に定義があります。

呼び出し関係図:

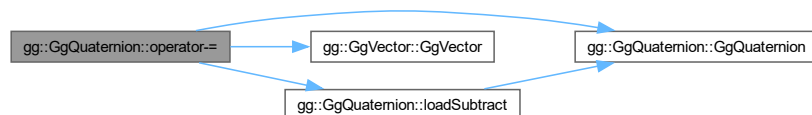


8.20.3.80 operator-=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgVector & v) [inline]
```

gg.h の 3983 行目に定義があります。

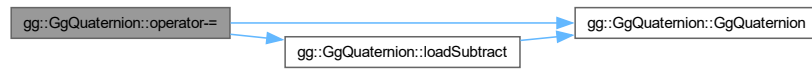
呼び出し関係図:



8.20.3.81 operator-=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GLfloat * a) [inline]
```

gg.h の 3979 行目に定義があります。
呼び出し関係図:



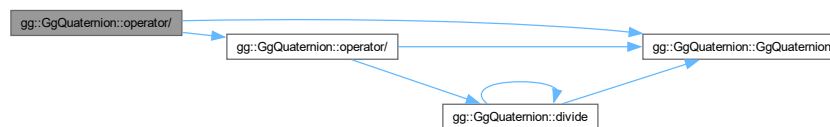
被呼び出し関係図:



8.20.3.82 operator/() [1/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgQuaternion & q) const [inline]
```

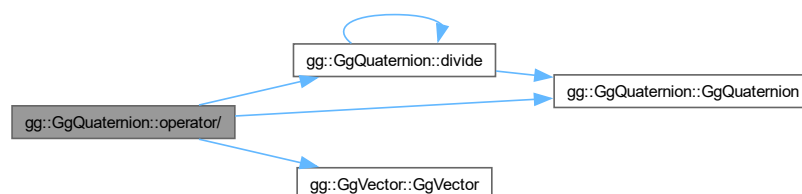
gg.h の 4059 行目に定義があります。
呼び出し関係図:



8.20.3.83 operator/() [2/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgVector & v) const [inline]
```

gg.h の 4055 行目に定義があります。
呼び出し関係図:

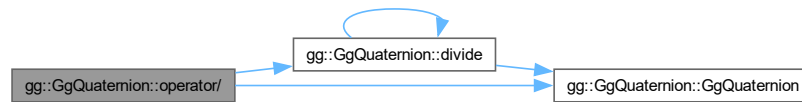


8.20.3.84 operator/() [3/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GLfloat * a) const [inline]
```

gg.h の 4051 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.85 operator/=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgQuaternion & q) [inline]
```

gg.h の 4011 行目に定義があります。

呼び出し関係図:

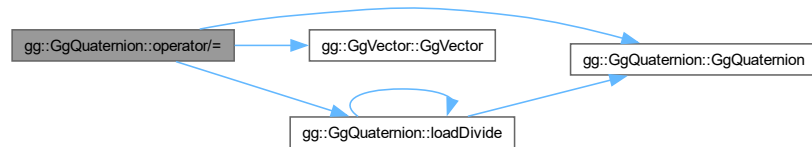


8.20.3.86 operator/=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgVector & v) [inline]
```

gg.h の 4007 行目に定義があります。

呼び出し関係図:



8.20.3.87 operator/=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GLfloat * a) [inline]
```

gg.h の 4003 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.88 operator=() [1/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GgQuaternion & q) [default]
```

代入演算子.

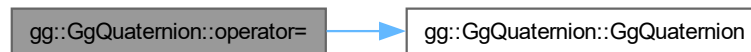
引数

q	代入元の四元数.
-----	----------

戻り値

代入後のこの四元数の参照.

呼び出し関係図:



被呼び出し関係図:

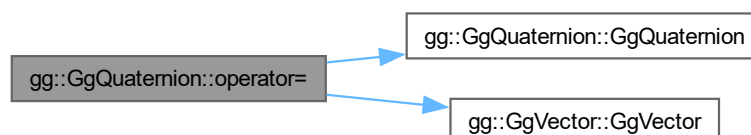


8.20.3.89 operator=() [2/4]

```
GgQuaternion & gg::GgQuaternion::operator= (  
    const GgVector & v) [inline]
```

gg.h の 3963 行目に定義があります。

呼び出し関係図:

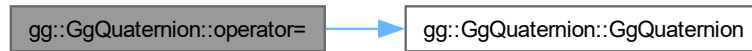


8.20.3.90 operator=() [3/4]

```
GgQuaternion & gg::GgQuaternion::operator= (  
    const GLfloat * a) [inline]
```

gg.h の 3959 行目に定義があります。

呼び出し関係図:



8.20.3.91 operator=() [4/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    GgQuaternion && q) [default]
```

ムーブ代入演算子.

引数

<i>q</i>	ムーブ代入元の四元数.
----------	-------------

戻り値

ムーブ代入後のこの四元数の参照.

呼び出し関係図:



8.20.3.92 rotate() [1/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 v[3] 回転した四元数を返す.

引数

<i>v</i>	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

回転した四元数.

gg.h の 4188 行目に定義があります。

呼び出し関係図:



8.20.3.93 rotate() [2/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v,
    GLfloat a) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 a 回転した四元数を返す.

引数

<i>v</i>	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
<i>a</i>	回転角.

戻り値

回転した四元数.

gg.h の 4177 行目に定義があります。

呼び出し関係図:



8.20.3.94 rotate() [3/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]
```

四元数を (x, y, z) を軸として角度 a 回転した四元数を返す.

引数

<i>x</i>	軸ベクトルの x 成分.
----------	--------------

<i>y</i>	軸ベクトルの <i>y</i> 成分.
<i>z</i>	軸ベクトルの <i>z</i> 成分.
<i>a</i>	回転角.

戻り値

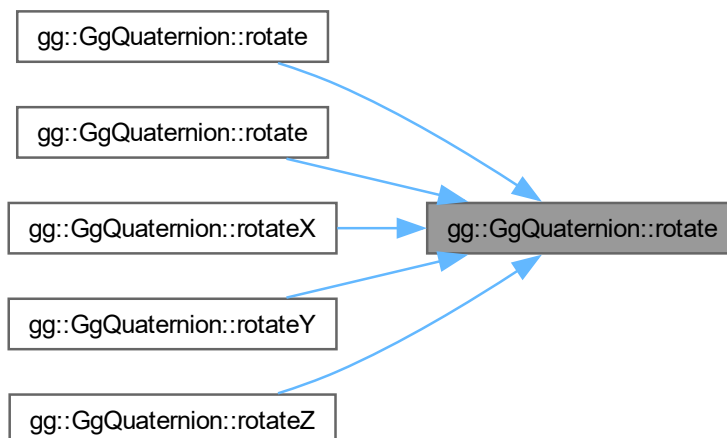
回転した四元数.

[gg.h](#) の 4164 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.95 rotateX()

```
GgQuaternion gg::GgQuaternion::rotateX (
    GLfloat a) const [inline]
```

四元数を *x* 軸中心に角度 *a* 回転した四元数を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

回転した四元数.

gg.h の 4199 行目に定義があります。

呼び出し関係図:



8.20.3.96 rotateY()

```
GgQuaternion gg::GgQuaternion::rotateY (
    GLfloat a) const [inline]
```

四元数を y 軸中心に角度 a 回転した四元数を返す.

引数

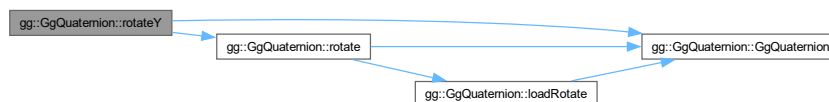
a	回転角.
---	------

戻り値

回転した四元数.

gg.h の 4210 行目に定義があります。

呼び出し関係図:



8.20.3.97 rotateZ()

```
GgQuaternion gg::GgQuaternion::rotateZ (
    GLfloat a) const [inline]
```

四元数を z 軸中心に角度 a 回転した四元数を返す.

引数

a	回転角.
---	------

戻り値

回転した四元数.

[gg.h](#) の 4221 行目に定義があります。

呼び出し関係図:



8.20.3.98 slerp() [1/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    const GgQuaternion & q,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

四元数を *q* に対して *t* で内分した結果.

[gg.h](#) の 4417 行目に定義があります。

呼び出し関係図:



8.20.3.99 slerp() [2/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    GLfloat * a,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

四元数を **a** に対して **t** で内分した結果.

[gg.h](#) の 4403 行目に定義があります。

呼び出し関係図:



8.20.3.100 subtract() [1/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgQuaternion & q) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

q を引いた四元数.

[gg.h](#) の 3852 行目に定義があります。

呼び出し関係図:



8.20.3.101 subtract() [2/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgVector & v) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>v</i>	四元数を格納した GgVector 型の変数.
----------	---

戻り値

v を引いた四元数.

gg.h の 3841 行目に定義があります。

呼び出し関係図:



8.20.3.102 subtract() [3/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GLfloat * a) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を引いた四元数.

gg.h の 3830 行目に定義があります。

呼び出し関係図:



8.20.3.103 subtract() [4/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>x</i>	引く四元数の x 要素.
<i>y</i>	引く四元数の y 要素.
<i>z</i>	引く四元数の z 要素.

<code>w</code>	引く四元数の <code>w</code> 要素.
----------------	---------------------------

戻り値

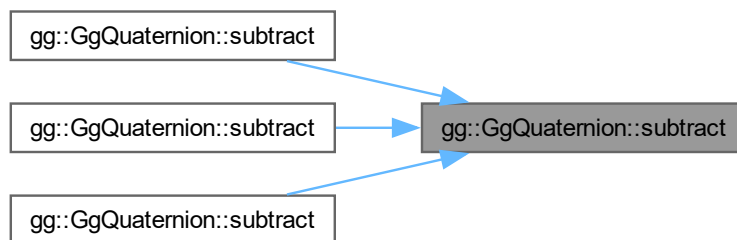
(`x`, `y`, `z`, `w`) を引いた四元数.

[gg.h](#) の 3818 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.21 gg::GgShader クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", int nvarying=0, const char *const *varyings=nullptr)
- [GgShader](#) (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- [GgShader](#) (const GgShader &shader)=delete
- [GgShader](#) (GgShader &&shader)=default
- virtual ~[GgShader](#) ()
- [GgShader](#) & operator= (const [GgShader](#) &shader)=delete
- [GgShader](#) & operator= ([GgShader](#) &&shader)=default
- void [use](#) () const
- void [unuse](#) () const
- GLuint [get](#) () const

8.21.1 詳解

シェーダの基底クラス.

覚え書き

シェーダのクラスはこのクラスを派生して作る.

[gg.h](#) の 6864 行目に定義があります。

8.21.2 構築子と解体子

8.21.2.1 GgShader() [1/4]

```
gg::GgShader::GgShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

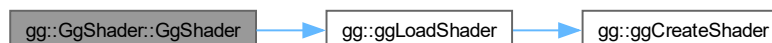
コンストラクタ.

引数

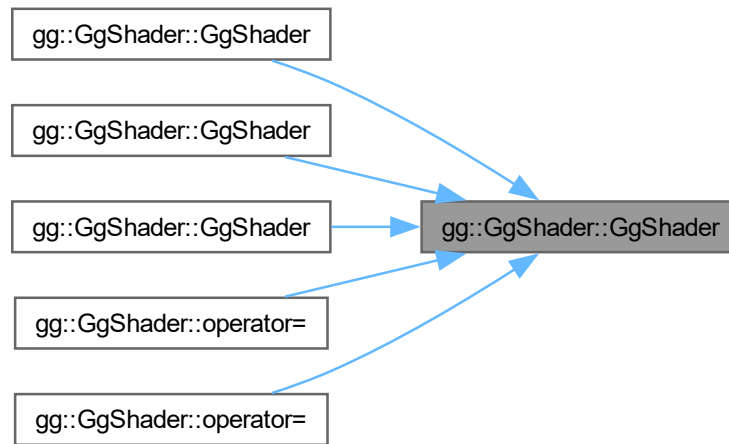
<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 6880 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.2.2 GgShader() [2/4]

```
gg::GgShader::GgShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

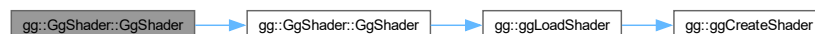
コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

gg.h の 6898 行目に定義があります。

呼び出し関係図:



8.21.2.3 GgShader() [3/4]

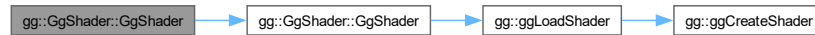
```
gg::GgShader::GgShader (
    const GgShader & shader) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>shader</i>	コピー元のシェーダ.
---------------	------------

呼び出し関係図:



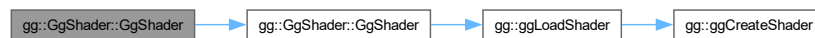
8.21.2.4 GgShader() [4/4]

```
gg::GgShader::GgShader (
    GgShader && shader) [default]
ムーブコンストラクタ.
```

引数

<i>shader</i>	ムーブ元のシェーダ.
---------------	------------

呼び出し関係図:



8.21.2.5 ~GgShader()

```
virtual gg::GgShader::~GgShader () [inline], [virtual]
デストラクタ.
gg.h の 6924 行目に定義があります。
```

8.21.3 関数詳解

8.21.3.1 get()

```
GLuint gg::GgShader::get () const [inline]
シェーダのプログラム名を得る.
```

戻り値

シェーダのプログラム名.

[gg.h](#) の [6968](#) 行目に定義があります。

8.21.3.2 operator=() [1/2]

```
GgShader & gg::GgShader::operator= (
    const GgShader & shader) [delete]
代入演算子は使用しない.
```

引数

<i>shader</i>	代入元のシェーダ.
---------------	-----------

戻り値

代入後のこのシェーダの参照.

呼び出し関係図:



8.21.3.3 operator=() [2/2]

`GgShader` & `gg::GgShader::operator=` (
`GgShader` && *shader*) [default]
 ムーブ代入演算子.

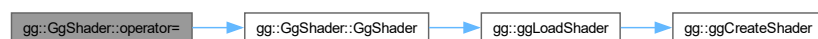
引数

<i>shader</i>	ムーブ代入元のシェーダ.
---------------	--------------

戻り値

ムーブ代入後のこのシェーダの参照.

呼び出し関係図:



8.21.3.4 unuse()

`void gg::GgShader::unuse () const [inline]`
 シェーダプログラムの使用を終了する.
[gg.h](#) の 6958 行目に定義があります。

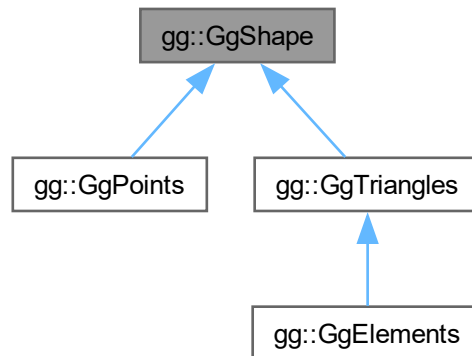
8.21.3.5 use()

`void gg::GgShader::use () const [inline]`
 シェーダプログラムの使用を開始する.
[gg.h](#) の 6950 行目に定義があります。
 このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.22 gg::GgShape クラス

#include <gg.h>
gg::GgShape の継承関係図



公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

8.22.1 詳解

形状データの基底クラス.

覚え書き

形状データのクラスはこのクラスを派生して作る. 基本図形の種類と頂点配列オブジェクトを保持する.

[gg.h](#) の 6240 行目に定義があります。

8.22.2 構築子と解体子

8.22.2.1 GgShape()

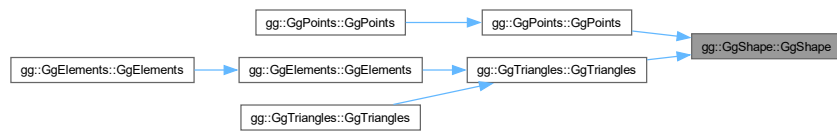
```
gg::GgShape::GgShape (
    GLenum mode = 0) [inline]
```

コンストラクタ.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

gg.h の 6255 行目に定義があります。
被呼び出し関係図:



8.22.2.2 ~GgShape()

virtual gg::GgShape::~~GgShape () [inline], [virtual]
デストラクタ.

gg.h の 6264 行目に定義があります。

8.22.3 関数詳解

8.22.3.1 draw()

```
virtual void gg::GgShape::draw (
    GLint first = 0,
    GLsizei count = 0) const [inline], [virtual]
```

図形の描画, 派生クラスでこの手続きをオーバーライドする.

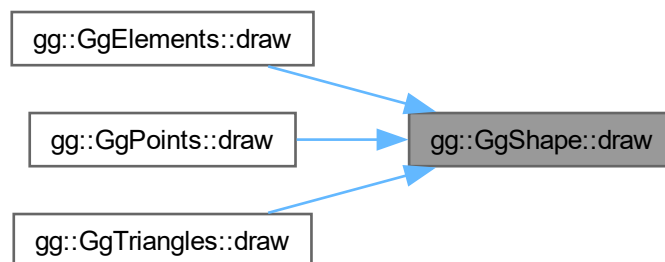
引数

<i>first</i>	描画する最初のアイテム.
<i>count</i>	描画するアイテムの数, 0 なら全部のアイテムを描画する.

gg::GgElements, gg::GgPoints, gg::GgTriangles で再実装されています。

gg.h の 6324 行目に定義があります。

被呼び出し関係図:



8.22.3.2 get()

```
const GLuint & gg::GgShape::get () const [inline]
```

頂点配列オブジェクト名を取り出す.

戻り値

頂点配列オブジェクト名.

gg.h の 6293 行目に定義があります。

被呼び出し関係図:



8.22.3.3 getMode()

```
const GLenum & gg::GgShape::getMode () const [inline]
```

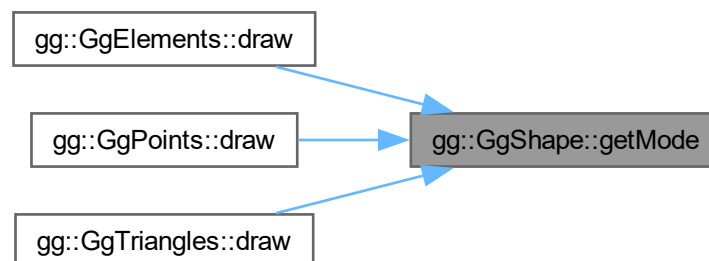
基本図形の検査.

戻り値

この頂点配列オブジェクトの基本図形の種類.

gg.h の 6313 行目に定義があります。

被呼び出し関係図:



8.22.3.4 operator bool()

```
virtual gg::GgShape::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうか調べる.

戻り値

頂点配列オブジェクトが有効なら true

gg::GgPoints で再実装されています。

gg.h の 6273 行目に定義があります。

8.22.3.5 operator!()

virtual bool gg::GgShape::operator! () const [inline], [virtual], [noexcept]
頂点配列オブジェクトが有効かどうかの結果を反転する。

戻り値

頂点配列オブジェクトが有効なら false, 無効なら true.

gg::GgPointsで再実装されています。
gg.h の 6283 行目に定義があります。

8.22.3.6 setMode()

void gg::GgShape::setMode (
GLenum mode) [inline]

基本図形の設定.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

gg.h の 6303 行目に定義があります。
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.23 gg::GgSimpleObj クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgSimpleObj](#) (const std::string &name, bool normalize=false)
- virtual ~[GgSimpleObj](#) ()
デストラクタ.
- operator bool () const noexcept
- bool operator! () const noexcept
- const [GgTriangles](#) * [get](#) () const
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

8.23.1 詳解

Wavefront OBJ 形式のファイル (Arrays 形式).
[gg.h](#) の 8244 行目に定義があります。

8.23.2 構築子と解体子

8.23.2.1 GgSimpleObj()

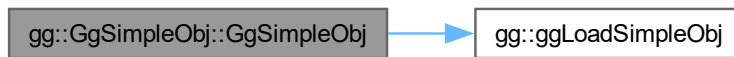
```
gg::GgSimpleObj::GgSimpleObj (   
    const std::string & name,   
    bool normalize = false)
```

コンストラクタ.

引数

<i>name</i>	三角形分割された Alias OBJ 形式のファイルのファイル名.
<i>normalize</i>	true なら図形のサイズを [-1, 1] に正規化する.

gg.cpp の 5990 行目に定義があります。
呼び出し関係図:



8.23.2.2 ~GgSimpleObj()

virtual gg::GgSimpleObj::~~GgSimpleObj () [inline], [virtual]
デストラクタ.

gg.h の 8267 行目に定義があります。

8.23.3 関数詳解

8.23.3.1 draw()

```
void gg::GgSimpleObj::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

Wavefront OBJ 形式のデータを描画する手続き。

引数

<i>first</i>	描画する最初のパーツ番号.
<i>count</i>	描画するパーツの数, 0 なら全部のパーツを描く.

gg.cpp の 6018 行目に定義があります。

8.23.3.2 get()

const GgTriangles * gg::GgSimpleObj::get () const [inline]
形状データの取り出し.

戻り値

[GgTriangles](#) 型の形状データのポインタ。

[gg.h](#) の 8296 行目に定義があります。

呼び出し関係図:



8.23.3.3 operator bool()

```
gg::GgSimpleObj::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる。

戻り値

オブジェクトが有効なら true

[gg.h](#) の 8276 行目に定義があります。

8.23.3.4 operator"!()"

```
bool gg::GgSimpleObj::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する。

戻り値

オブジェクトが有効なら false, 無効なら true.

[gg.h](#) の 8286 行目に定義があります。

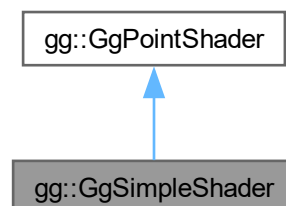
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

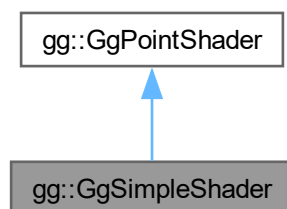
8.24 gg::GgSimpleShader クラス

```
#include <gg.h>
```

gg::GgSimpleShader の継承関係図



gg::GgSimpleShader 連携図



クラス

- struct [Light](#)
- class [LightBuffer](#)
- struct [Material](#)
- class [MaterialBuffer](#)

公開メンバ関数

- [GgSimpleShader](#) ()
- [GgSimpleShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const GgSimpleShader &o)
- virtual [~GgSimpleShader](#) ()
- [GgSimpleShader & operator=](#) (const [GgSimpleShader](#) &o)
- bool [load](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool [load](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void [loadModelviewMatrix](#) (const GLfloat *mv, const GLfloat *mn) const
- virtual void [loadModelviewMatrix](#) (const [GgMatrix](#) &mv, const [GgMatrix](#) &mn) const
- virtual void [loadModelviewMatrix](#) (const GLfloat *mv) const
- virtual void [loadModelviewMatrix](#) (const [GgMatrix](#) &mv) const
- virtual void [loadMatrix](#) (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- virtual void [loadMatrix](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv, const [GgMatrix](#) &mn) const
- virtual void [loadMatrix](#) (const GLfloat *mp, const GLfloat *mv) const
- virtual void [loadMatrix](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- void [use](#) () const
- void [use](#) (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- void [use](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv, const [GgMatrix](#) &mn) const
- void [use](#) (const GLfloat *mp, const GLfloat *mv) const
- void [use](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- void [use](#) (const [LightBuffer](#) *light, GLint i=0) const
- void [use](#) (const [LightBuffer](#) &light, GLint i=0) const
- void [use](#) (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn, const [LightBuffer](#) *light, GLint i=0) const
- void [use](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv, const [GgMatrix](#) &mn, const [LightBuffer](#) &light, GLint i=0) const
- void [use](#) (const GLfloat *mp, const GLfloat *mv, const [LightBuffer](#) *light, GLint i=0) const
- void [use](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv, const [LightBuffer](#) &light, GLint i=0) const
- void [use](#) (const GLfloat *mp, const [LightBuffer](#) *light, GLint i=0) const
- void [use](#) (const [GgMatrix](#) &mp, const [LightBuffer](#) &light, GLint i=0) const

基底クラス **gg::GgPointShader** に属する継承公開メンバ関数

- **GgPointShader** ()
- **GgPointShader** (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- **GgPointShader** (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual **~GgPointShader** ()
- bool **load** (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool **load** (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void **loadProjectionMatrix** (const GLfloat *mp) const
- virtual void **loadProjectionMatrix** (const **GgMatrix** &mp) const
- void **use** (const GLfloat *mp) const
- void **use** (const **GgMatrix** &mp) const
- void **use** (const GLfloat *mp, const GLfloat *mv) const
- void **use** (const **GgMatrix** &mp, const **GgMatrix** &mv) const
- void **unuse** () const
- GLuint **get** () const

8.24.1 詳解

三角形に単純な陰影付けを行うシェーダ。
gg.h の 7232 行目に定義があります。

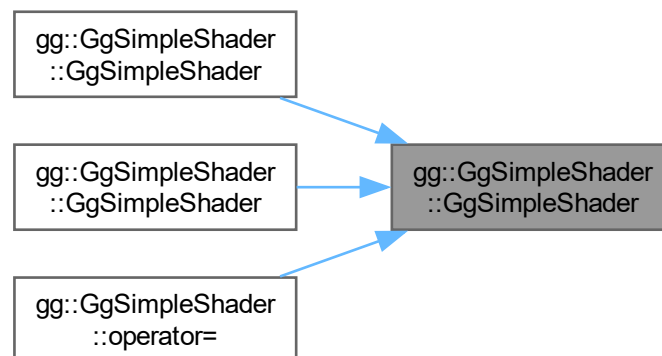
8.24.2 構築子と解体子

8.24.2.1 GgSimpleShader() [1/4]

`gg::GgSimpleShader::GgSimpleShader () [inline]`
 コンストラクタ。
gg.h の 7243 行目に定義があります。
 呼び出し関係図:



被呼び出し関係図:



8.24.2.2 GgSimpleShader() [2/4]

```

gg::GgSimpleShader::GgSimpleShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 7258 行目に定義があります。

呼び出し関係図:



8.24.2.3 GgSimpleShader() [3/4]

```
gg::GgSimpleShader::GgSimpleShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

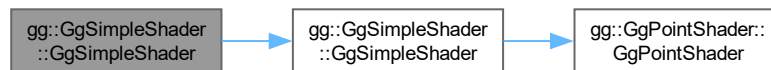
コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

gg.h の 7276 行目に定義があります。

呼び出し関係図:



8.24.2.4 GgSimpleShader() [4/4]

```
gg::GgSimpleShader::GgSimpleShader (
    const GgSimpleShader & o) [inline]
```

コピーコンストラクタ.

gg.h の 7288 行目に定義があります。

呼び出し関係図:



8.24.2.5 ~GgSimpleShader()

```
virtual gg::GgSimpleShader::~~GgSimpleShader () [inline], [virtual]
```

デストラクタ.

gg.h の 7297 行目に定義があります。

8.24.3 関数詳解

8.24.3.1 load() [1/2]

```
bool gg::GgSimpleShader::load (
    const std::array< std::string, 3 > & files,
```

```
GLint nvarying = 0,
const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら true.

[gg.h](#) の 7344 行目に定義があります。

呼び出し関係図:



8.24.3.2 load() [2/2]

```
bool gg::GgSimpleShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr)
```

シェーダのソースプログラムの文字列からプログラムオブジェクトを作成する。

引数

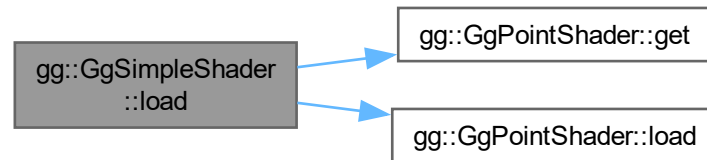
<i>vert</i>	バーテックスシェーダのソースプログラムの文字列.
<i>frag</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用) .
<i>geom</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用) .
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

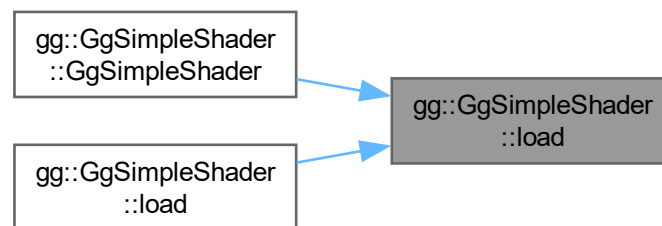
プログラムオブジェクトの作成に成功したら true.

gg.cpp の 5973 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.24.3.3 loadMatrix() [1/4]

```
virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する.

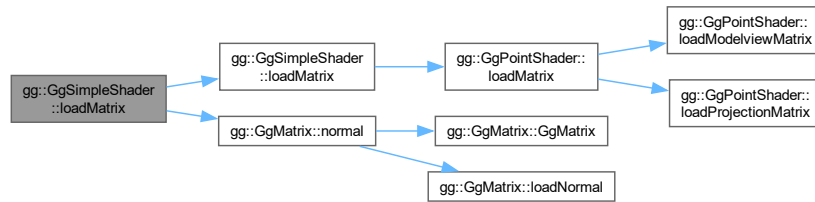
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg::GgPointShaderを再実装しています。

gg.h の 7438 行目に定義があります。

呼び出し関係図:



8.24.3.4 loadMatrix() [2/4]

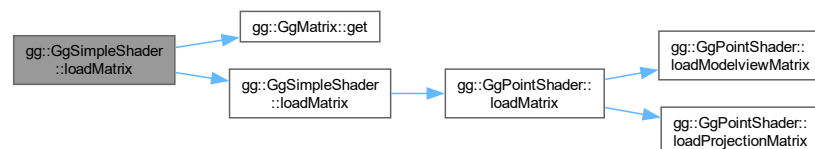
```
virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列と法線変換行列を設定する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7416 行目に定義があります。
呼び出し関係図:



8.24.3.5 loadMatrix() [3/4]

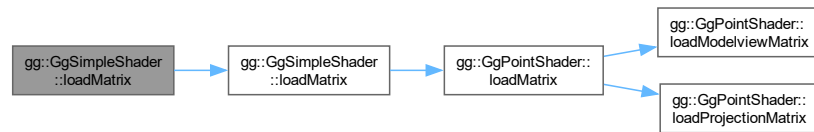
```
virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

gg::GgPointShaderを再実装しています。
 gg.h の 7427 行目に定義があります。
 呼び出し関係図:



8.24.3.6 loadMatrix() [4/4]

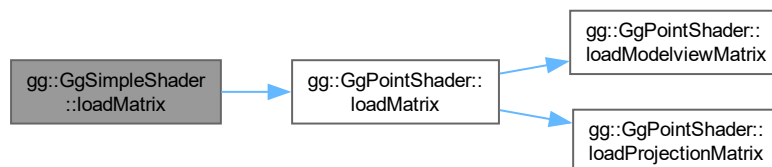
```
virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列と法線変換行列を設定する。

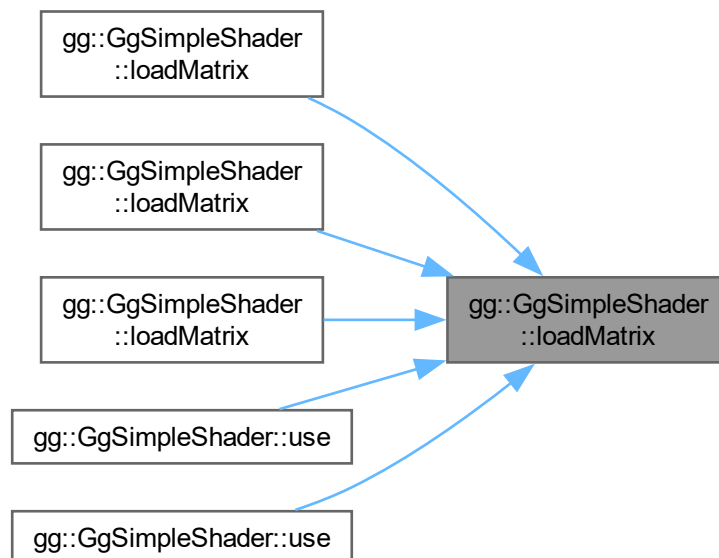
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 7403 行目に定義があります。
 呼び出し関係図:



被呼び出し関係図:



8.24.3.7 loadModelviewMatrix() [1/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する.

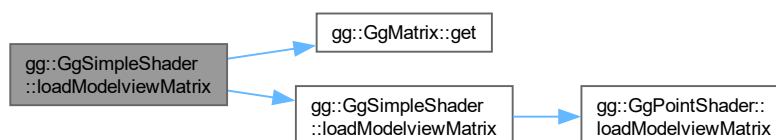
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	--

[gg::GgPointShader](#)を再実装しています。

[gg.h](#) の 7391 行目に定義があります。

呼び出し関係図:



8.24.3.8 loadModelviewMatrix() [2/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
```

```
const GgMatrix & mv,
const GgMatrix & mn) const [inline], [virtual]
```

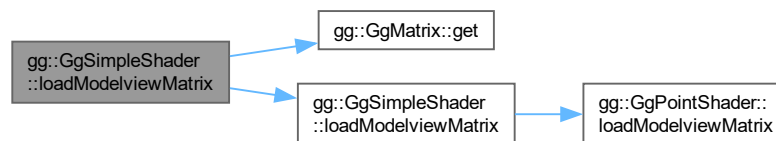
モデルビュー変換行列と法線変換行列を設定する。

引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7371 行目に定義があります。

呼び出し関係図:



8.24.3.9 loadModelviewMatrix() [3/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する。

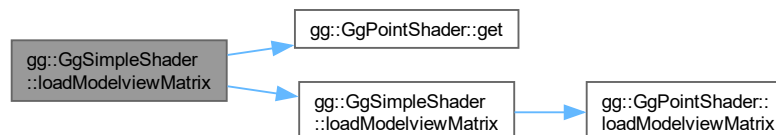
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgPointShader を再実装しています。

gg.h の 7381 行目に定義があります。

呼び出し関係図:



8.24.3.10 loadModelviewMatrix() [4/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
const GLfloat * mv,
const GLfloat * mn) const [inline], [virtual]
```

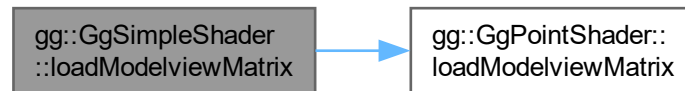
モデルビュー変換行列と法線変換行列を設定する。

引数

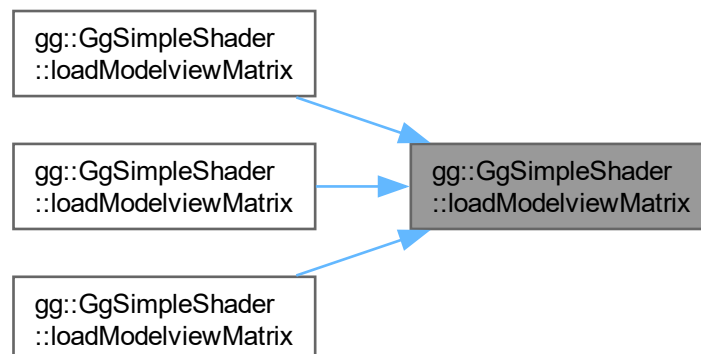
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 7359 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.24.3.11 operator=()

```
GgSimpleShader & gg::GgSimpleShader::operator= (
    const GgSimpleShader & o) [inline]
```

代入演算子.

引数

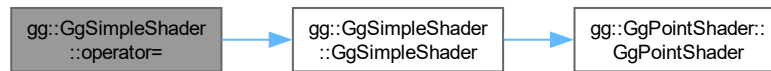
<i>o</i>	代入する GgSimpleShader 型のオブジェクト.
----------	-------------------------------

戻り値

代入後の [GgSimpleShader](#) 型のオブジェクトの参照.

[gg.h](#) の 7307 行目に定義があります。

呼び出し関係図:



8.24.3.12 use() [1/13]

```
void gg::GgSimpleShader::use () const [inline], [virtual]
```

シェーダプログラムの使用を開始する.

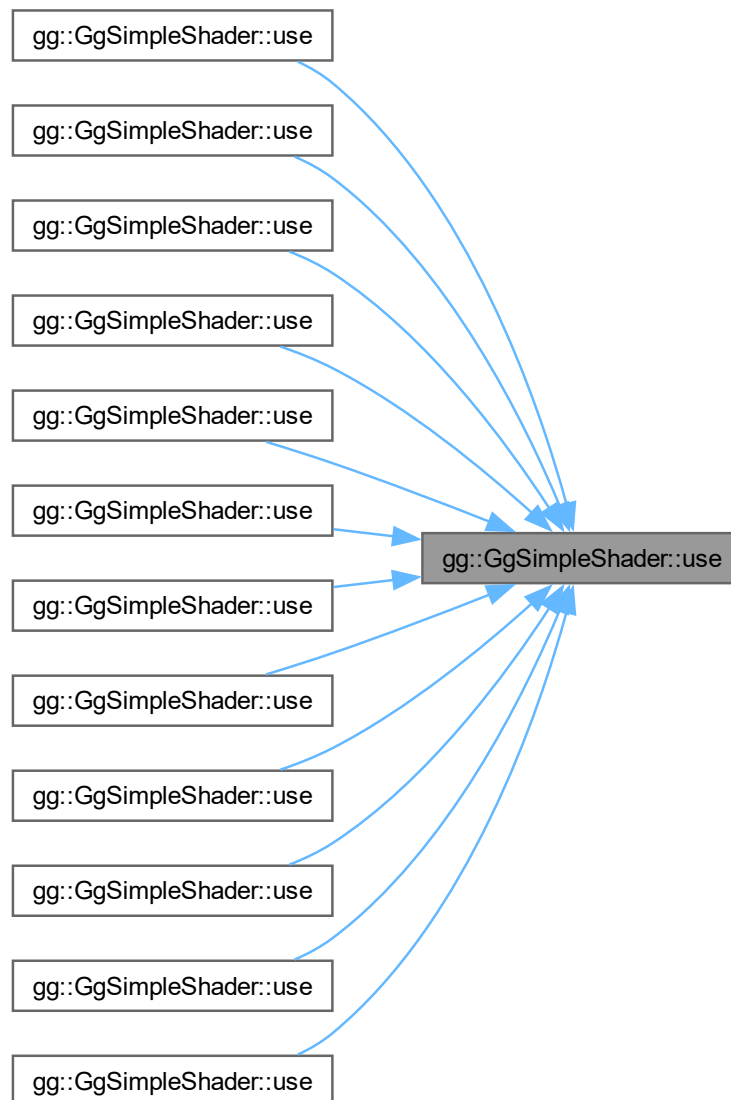
[gg::GgPointShader](#)を再実装しています。

[gg.h](#) の 8011 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.24.3.13 use() [2/13]

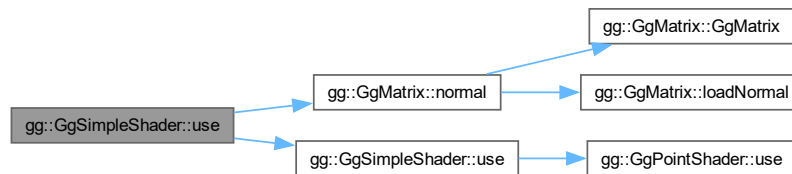
```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg.h の 8062 行目に定義があります。
呼び出し関係図:



8.24.3.14 use() [3/13]

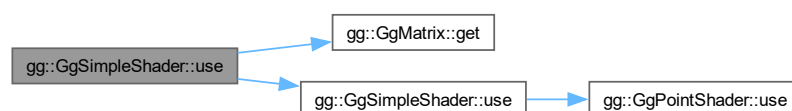
```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline]
```

投影変換行列とモデルビュー変換行列と法線変換行列を指定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 8040 行目に定義があります。
呼び出し関係図:



8.24.3.15 use() [4/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

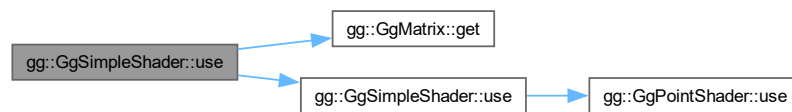
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの <code>uniform block</code> のインデックス.

[gg.h](#) の 8126 行目に定義があります。

呼び出し関係図:



8.24.3.16 use() [5/13]

```

void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const LightBuffer & light,
    GLint i = 0) const [inline]
  
```

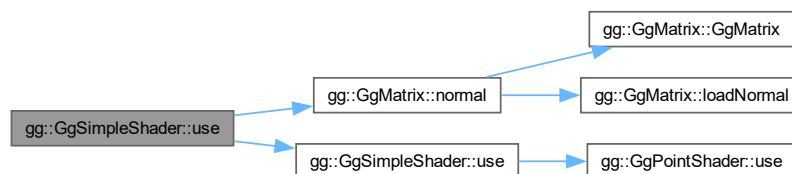
光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの <code>uniform block</code> のインデックス.

[gg.h](#) の 8163 行目に定義があります。

呼び出し関係図:



8.24.3.17 use() [6/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

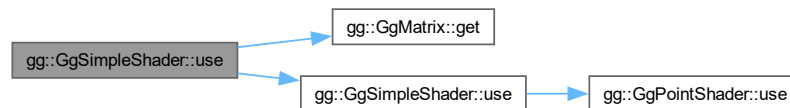
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8196 行目に定義があります。

呼び出し関係図:



8.24.3.18 use() [7/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

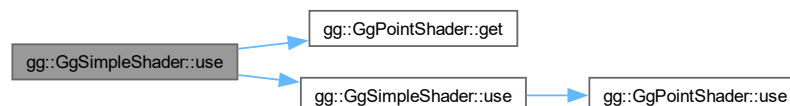
投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

gg.h の 8051 行目に定義があります。

呼び出し関係図:



8.24.3.19 use() [8/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline]
```

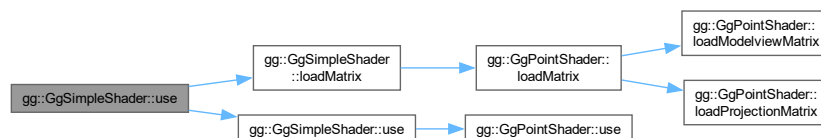
投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 8024 行目に定義があります。

呼び出し関係図:



8.24.3.20 use() [9/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

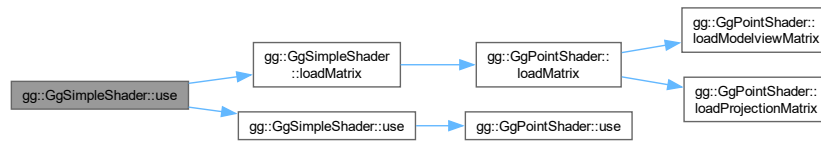
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8102 行目に定義があります。

呼び出し関係図:



8.24.3.21 use() [10/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

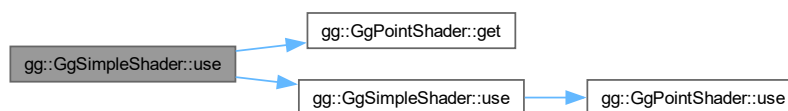
光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8145 行目に定義があります。

呼び出し関係図:



8.24.3.22 use() [11/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

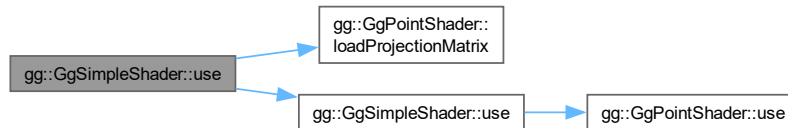
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.

<i>i</i>	光源データの uniform block のインデックス.
----------	--------------------------------------

[gg.h](#) の 8180 行目に定義があります。
呼び出し関係図:



8.24.3.23 use() [12/13]

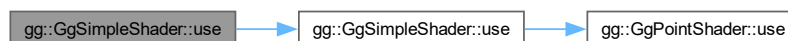
```
void gg::GgSimpleShader::use (
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

光源を指定してシェーダプログラムの使用を開始する.

引数

<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

[gg.h](#) の 8088 行目に定義があります。
呼び出し関係図:



8.24.3.24 use() [13/13]

```
void gg::GgSimpleShader::use (
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

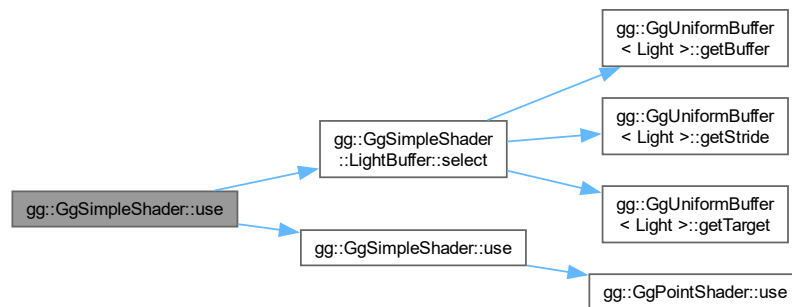
光源を指定してシェーダプログラムの使用を開始する.

引数

<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

[gg.h](#) の 8073 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.25 gg::GgTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- [GgTexture](#) (const GgTexture &texture)=delete
- [GgTexture](#) (GgTexture &&texture)=default
- virtual ~[GgTexture](#) ()
- [GgTexture](#) & operator= (const [GgTexture](#) &texture)=delete
- [GgTexture](#) & operator= ([GgTexture](#) &&texture)=default
- void [bind](#) () const
- void [unbind](#) () const
- void [swapRandB](#) (bool swizzle) const
- const GLsizei & [getWidth](#) () const
- const GLsizei & [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- const GLuint & [getTexture](#) () const

8.25.1 詳解

テクスチャ.

覚え書き

画像データを読み込んでテクスチャマップを作成する.

[gg.h](#) の 5193 行目に定義があります。

8.25.2 構築子と解体子

8.25.2.1 GgTexture() [1/3]

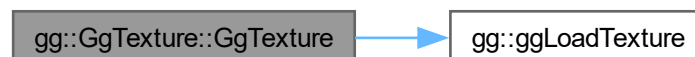
```
gg::GgTexture::GgTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = false) [inline]
```

メモリ上のデータからテクスチャを作成するコンストラクタ.

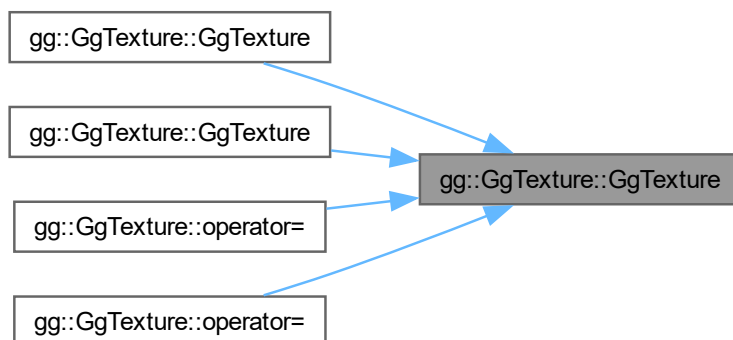
引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.
<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false.

gg.h の 5215 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.25.2.2 GgTexture() [2/3]

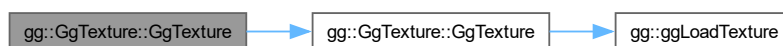
```
gg::GgTexture::GgTexture (
    const GgTexture & texture) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>texture</i>	コピー元のテクスチャ.
----------------	-------------

呼び出し関係図:



8.25.2.3 GgTexture() [3/3]

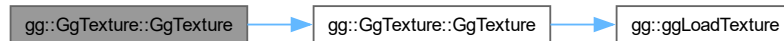
```
gg::GgTexture::GgTexture (
    GgTexture && texture) [default]
```

ムーブコンストラクタ.

引数

<i>texture</i>	ムーブ元のテクスチャ.
----------------	-------------

呼び出し関係図:



8.25.2.4 ~GgTexture()

```
virtual gg::GgTexture::~~GgTexture () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 5247 行目に定義があります。

8.25.3 関数詳解

8.25.3.1 bind()

```
void gg::GgTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

[gg.h](#) の 5272 行目に定義があります。

8.25.3.2 getHeight()

```
const GLsizei & gg::GgTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す.

戻り値

テクスチャの縦の画素数.

[gg.h](#) の 5307 行目に定義があります。

被呼び出し関係図:



8.25.3.3 getSize() [1/2]

```
const GLsizei * gg::GgTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す.

戻り値

テクスチャのサイズを格納した配列へのポインタ.

[gg.h](#) の 5328 行目に定義があります。

8.25.3.4 getSize() [2/2]

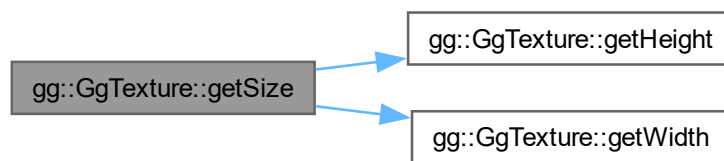
```
void gg::GgTexture::getSize (
    GLsizei * size) const [inline]
```

使用しているテクスチャのサイズを取り出す.

引数

size	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数.
-------------	--------------------------------------

gg.h の 5317 行目に定義があります。
呼び出し関係図:



8.25.3.5 getTexture()

```
const GLuint & gg::GgTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る.

戻り値

テクスチャ名.

gg.h の 5338 行目に定義があります。

8.25.3.6 getWidth()

```
const GLsizei & gg::GgTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す.

戻り値

テクスチャの横の画素数.

gg.h の 5297 行目に定義があります。
被呼び出し関係図:



8.25.3.7 operator=() [1/2]

```
GgTexture & gg::GgTexture::operator= (
    const GgTexture & texture) [delete]
```

代入演算子は使用しない.

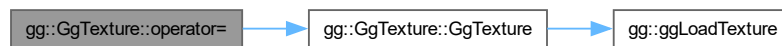
引数

<i>texture</i>	代入元のテクスチャ.
----------------	------------

戻り値

代入後のこのテクスチャの参照.

呼び出し関係図:



8.25.3.8 operator=() [2/2]

```
GgTexture & gg::GgTexture::operator= (
    GgTexture && texture) [default]
```

ムーブ代入演算子.

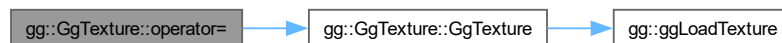
引数

<i>texture</i>	ムーブ代入元のテクスチャ.
----------------	---------------

戻り値

ムーブ代入後のこのテクスチャの参照.

呼び出し関係図:



8.25.3.9 swapRandB()

```
void gg::GgTexture::swapRandB (
    bool swizzle) const
```

テクスチャの赤と青を交換する

引数

<code>swizzle</code>	赤と青を交換するなら <code>true</code>
----------------------	------------------------------

`gg.cpp` の 4006 行目に定義があります。

8.25.3.10 unbind()

```
void gg::GgTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

`gg.h` の 5280 行目に定義があります。

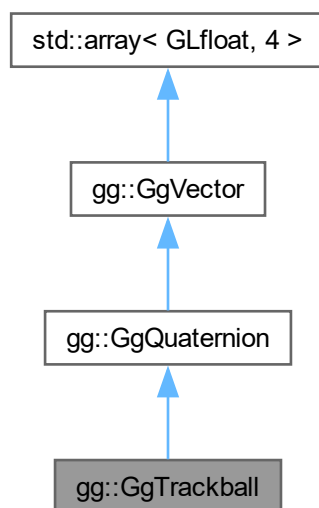
このクラス詳解は次のファイルから抽出されました:

- `gg.h`
- `gg.cpp`

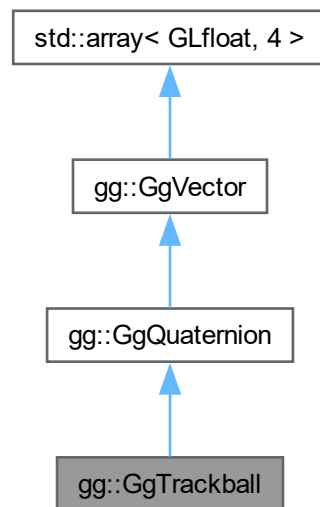
8.26 gg::GgTrackball クラス

```
#include <gg.h>
```

`gg::GgTrackball` の継承関係図



gg::GgTrackball 連携図



公開メンバ関数

- `GgTrackball` (`const GgQuaternion &q=gglIdentityQuaternion()`)
- `~GgTrackball` ()=default
- `GgTrackball & operator=` (`const GgQuaternion &q`)
- `void region` (`GLfloat w, GLfloat h`)
- `void region` (`int w, int h`)
- `void begin` (`GLfloat x, GLfloat y`)
- `void motion` (`GLfloat x, GLfloat y`)
- `void rotate` (`const GgQuaternion &q`)
- `void end` (`GLfloat x, GLfloat y`)
- `void reset` (`const GgQuaternion &q=gglIdentityQuaternion()`)
- `const GLfloat * getStart` () const
- `const GLfloat & getStart` (`int direction`) const
- `void getStart` (`GLfloat *position`) const
- `const GLfloat * getScale` () const
- `const GLfloat getScale` (`int direction`) const
- `void getScale` (`GLfloat *factor`) const
- `const GgQuaternion & getQuaternion` () const
- `const GgMatrix & getMatrix` () const
- `const GLfloat * get` () const

基底クラス `gg::GgQuaternion` に属する継承公開メンバ関数

- `GgQuaternion` ()=default
- `constexpr GgQuaternion` (`GLfloat x, GLfloat y, GLfloat z, GLfloat w`)
- `constexpr GgQuaternion` (`GLfloat c`)
- `GgQuaternion` (`const GLfloat *a`)
- `GgQuaternion` (`const GgVector &v`)
- `GgQuaternion` (`const GgQuaternion &q`)=default

- [GgQuaternion](#) ([GgQuaternion](#) &&q)=default
- [~GgQuaternion](#) ()=default
- [GgQuaternion](#) & [operator=](#) (const [GgQuaternion](#) &q)=default
- [GgQuaternion](#) & [operator=](#) ([GgQuaternion](#) &&q)=default
- [GLfloat](#) [norm](#) () const
- [GgQuaternion](#) & [loadAdd](#) ([GLfloat](#) x, [GLfloat](#) y, [GLfloat](#) z, [GLfloat](#) w)
- [GgQuaternion](#) & [loadAdd](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [loadAdd](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadAdd](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadSubtract](#) ([GLfloat](#) x, [GLfloat](#) y, [GLfloat](#) z, [GLfloat](#) w)
- [GgQuaternion](#) & [loadSubtract](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [loadSubtract](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadSubtract](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadMultiply](#) ([GLfloat](#) x, [GLfloat](#) y, [GLfloat](#) z, [GLfloat](#) w)
- [GgQuaternion](#) & [loadMultiply](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [loadMultiply](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadMultiply](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadDivide](#) ([GLfloat](#) x, [GLfloat](#) y, [GLfloat](#) z, [GLfloat](#) w)
- [GgQuaternion](#) & [loadDivide](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [loadDivide](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadDivide](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) [add](#) ([GLfloat](#) x, [GLfloat](#) y, [GLfloat](#) z, [GLfloat](#) w) const
- [GgQuaternion](#) [add](#) (const [GLfloat](#) *a) const
- [GgQuaternion](#) [add](#) (const [GgVector](#) &v) const
- [GgQuaternion](#) [add](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) [subtract](#) ([GLfloat](#) x, [GLfloat](#) y, [GLfloat](#) z, [GLfloat](#) w) const
- [GgQuaternion](#) [subtract](#) (const [GLfloat](#) *a) const
- [GgQuaternion](#) [subtract](#) (const [GgVector](#) &v) const
- [GgQuaternion](#) [subtract](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) [multiply](#) ([GLfloat](#) x, [GLfloat](#) y, [GLfloat](#) z, [GLfloat](#) w) const
- [GgQuaternion](#) [multiply](#) (const [GLfloat](#) *a) const
- [GgQuaternion](#) [multiply](#) (const [GgVector](#) &v) const
- [GgQuaternion](#) [multiply](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) [divide](#) ([GLfloat](#) x, [GLfloat](#) y, [GLfloat](#) z, [GLfloat](#) w) const
- [GgQuaternion](#) [divide](#) (const [GLfloat](#) *a) const
- [GgQuaternion](#) [divide](#) (const [GgVector](#) &v) const
- [GgQuaternion](#) [divide](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) & [operator=](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [operator=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator+=](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [operator+=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator+=](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [operator-=](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [operator-=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator-=](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [operator*=](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [operator*=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator*=](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [operator/=](#) (const [GLfloat](#) *a)
- [GgQuaternion](#) & [operator/=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator/=](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) [operator+](#) (const [GLfloat](#) *a) const
- [GgQuaternion](#) [operator+](#) (const [GgVector](#) &v) const
- [GgQuaternion](#) [operator+](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) [operator-](#) (const [GLfloat](#) *a) const

- `GgQuaternion operator-` (const `GgVector` &v) const
- `GgQuaternion operator-` (const `GgQuaternion` &q) const
- `GgQuaternion operator*` (const `GLfloat` *a) const
- `GgQuaternion operator*` (const `GgVector` &v) const
- `GgQuaternion operator*` (const `GgQuaternion` &q) const
- `GgQuaternion operator/` (const `GLfloat` *a) const
- `GgQuaternion operator/` (const `GgVector` &v) const
- `GgQuaternion operator/` (const `GgQuaternion` &q) const
- `GgQuaternion & loadMatrix` (const `GLfloat` *a)
- `GgQuaternion & loadMatrix` (const `GgMatrix` &m)
- `GgQuaternion & loadIdentity` ()
- `GgQuaternion & loadRotate` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` a)
- `GgQuaternion & loadRotate` (const `GLfloat` *v, `GLfloat` a)
- `GgQuaternion & loadRotate` (const `GLfloat` *v)
- `GgQuaternion & loadRotateX` (`GLfloat` a)
- `GgQuaternion & loadRotateY` (`GLfloat` a)
- `GgQuaternion & loadRotateZ` (`GLfloat` a)
- `GgQuaternion rotate` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` a) const
- `GgQuaternion rotate` (const `GLfloat` *v, `GLfloat` a) const
- `GgQuaternion rotate` (const `GLfloat` *v) const
- `GgQuaternion rotateX` (`GLfloat` a) const
- `GgQuaternion rotateY` (`GLfloat` a) const
- `GgQuaternion rotateZ` (`GLfloat` a) const
- `GgQuaternion & loadEuler` (`GLfloat` heading, `GLfloat` pitch, `GLfloat` roll)
- `GgQuaternion & loadEuler` (const `GLfloat` *e)
- `GgQuaternion euler` (`GLfloat` heading, `GLfloat` pitch, `GLfloat` roll) const
- `GgQuaternion euler` (const `GLfloat` *e) const
- `GgQuaternion euler` (const `GgVector` &e) const
- `GgQuaternion & loadSlerp` (const `GLfloat` *a, const `GLfloat` *b, `GLfloat` t)
- `GgQuaternion & loadSlerp` (const `GgQuaternion` &q, const `GgQuaternion` &r, `GLfloat` t)
- `GgQuaternion & loadSlerp` (const `GgQuaternion` &q, const `GLfloat` *a, `GLfloat` t)
- `GgQuaternion & loadSlerp` (const `GLfloat` *a, const `GgQuaternion` &q, `GLfloat` t)
- `GgQuaternion & loadNormalize` (const `GLfloat` *a)
- `GgQuaternion & loadNormalize` (const `GgQuaternion` &q)
- `GgQuaternion & loadConjugate` (const `GLfloat` *a)
- `GgQuaternion & loadConjugate` (const `GgQuaternion` &q)
- `GgQuaternion & loadInvert` (const `GLfloat` *a)
- `GgQuaternion & loadInvert` (const `GgQuaternion` &q)
- `GgQuaternion slerp` (`GLfloat` *a, `GLfloat` t) const
- `GgQuaternion slerp` (const `GgQuaternion` &q, `GLfloat` t) const
- `GgQuaternion normalize` () const
- `GgQuaternion conjugate` () const
- `GgQuaternion invert` () const
- void `get` (`GLfloat` *a) const
- void `getMatrix` (`GLfloat` *a) const
- void `getMatrix` (`GgMatrix` &m) const
- `GgMatrix getMatrix` () const
- void `getConjugateMatrix` (`GLfloat` *a) const
- void `getConjugateMatrix` (`GgMatrix` &m) const
- `GgMatrix getConjugateMatrix` () const

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & **operator=** (const **GgVector** &v)=default
- **GgVector** & **operator=** (**GgVector** &&v)=default
- **GgVector** **operator+** (const **GgVector** &v) const
- **GgVector** **operator+** (GLfloat c) const
- **GgVector** & **operator+=** (const **GgVector** &v)
- **GgVector** & **operator+=** (GLfloat c)
- **GgVector** **operator-** (const **GgVector** &v) const
- **GgVector** **operator-** (GLfloat c) const
- **GgVector** & **operator-=** (const **GgVector** &v)
- **GgVector** & **operator-=** (GLfloat c)
- **GgVector** **operator*** (const **GgVector** &v) const
- **GgVector** **operator*** (GLfloat c) const
- **GgVector** & **operator*=** (const **GgVector** &v)
- **GgVector** & **operator*=** (GLfloat c)
- **GgVector** **operator/** (const **GgVector** &v) const
- **GgVector** **operator/** (GLfloat c) const
- **GgVector** & **operator/=** (**GgVector** &v)
- **GgVector** & **operator/=** (GLfloat c)
- GLfloat **dot3** (const **GgVector** &v) const
- GLfloat **length3** () const
- GLfloat **distance3** (const **GgVector** &v) const
- **GgVector** **normalize3** () const
- GLfloat **dot4** (const **GgVector** &v) const
- GLfloat **length4** () const
- GLfloat **distance4** (const **GgVector** &v) const
- **GgVector** **normalize4** () const

8.26.1 詳解

簡易トラックボール処理.
[gg.h](#) の 4775 行目に定義があります。

8.26.2 構築子と解体子

8.26.2.1 GgTrackball()

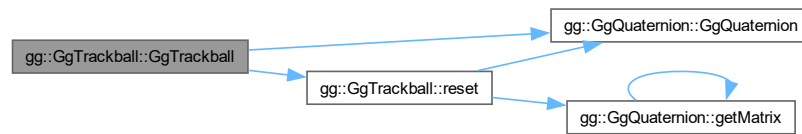
```
gg::GgTrackball::GgTrackball (
    const GgQuaternion & q = ggIdentityQuaternion()) [inline]
コンストラクタ.
```

引数

<i>q</i>	トラックボールの回転の初期値の四元数.
----------	---------------------

[gg.h](#) の 4790 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.26.2.2 ~GgTrackball()

`gg::GgTrackball::~~GgTrackball () [default]`
デストラクタ.

8.26.3 関数詳解

8.26.3.1 begin()

```
void gg::GgTrackball::begin (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を開始する.

引数

<i>x</i>	現在のマウスの <i>x</i> 座標.
<i>y</i>	現在のマウスの <i>y</i> 座標.

覚え書き

マウスのドラッグ開始時 (マウスボタンを押したとき) に呼び出す.

[gg.cpp](#) の [3453](#) 行目に定義があります.

8.26.3.2 end()

```
void gg::GgTrackball::end (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を停止する.

引数

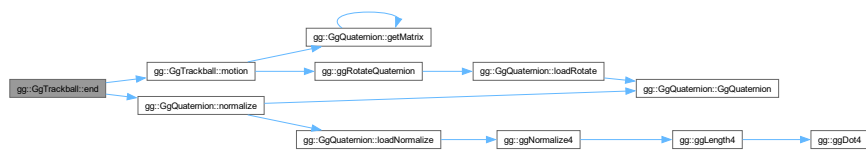
x	現在のマウスの x 座標.
y	現在のマウスの y 座標.

覚え書き

マウスのドラッグ終了時 (マウスボタンを離したとき) に呼び出す.

gg.cpp の 3519 行目に定義があります.

呼び出し関係図:



8.26.3.3 get()

```
const GLfloat * gg::GgTrackball::get () const [inline]
```

現在の回転の変換行列を取り出す.

戻り値

回転の変換を表す GLfloat 型の 16 要素の配列.

gg.h の 4968 行目に定義があります.

8.26.3.4 getMatrix()

```
const GgMatrix & gg::GgTrackball::getMatrix () const [inline]
```

現在の回転の変換行列を取り出す.

戻り値

回転の変換を表す GgMatrix 型の変換行列.

gg.h の 4958 行目に定義があります.

8.26.3.5 getQuaternion()

```
const GgQuaternion & gg::GgTrackball::getQuaternion () const [inline]
```

現在の回転の四元数を取り出す.

戻り値

回転の変換を表す Quaternion 型の四元数.

gg.h の 4948 行目に定義があります.

呼び出し関係図:



8.26.3.6 `getScale()` [1/3]

```
const GLfloat * gg::GgTrackball::getScale () const [inline]
```

トラックボール処理の換算係数を取り出す。

戻り値

トラックボールの換算係数のポインタ。

[gg.h](#) の 4917 行目に定義があります。

8.26.3.7 `getScale()` [2/3]

```
void gg::GgTrackball::getScale (
    GLfloat * factor) const [inline]
```

トラックボール処理の換算係数を取り出す。

引数

<i>factor</i>	トラックボールの換算係数を格納する 2 要素の配列。
---------------	----------------------------

[gg.h](#) の 4937 行目に定義があります。

8.26.3.8 `getScale()` [3/3]

```
const GLfloat gg::GgTrackball::getScale (
    int direction) const [inline]
```

トラックボール処理の換算係数を取り出す。

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向。
------------------	-----------------------

[gg.h](#) の 4927 行目に定義があります。

8.26.3.9 `getStart()` [1/3]

```
const GLfloat * gg::GgTrackball::getStart () const [inline]
```

トラックボール処理の開始位置を取り出す。

戻り値

トラックボールの開始位置のポインタ。

[gg.h](#) の 4888 行目に定義があります。

8.26.3.10 `getStart()` [2/3]

```
void gg::GgTrackball::getStart (
    GLfloat * position) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>position</i>	トラックボールの開始位置を格納する 2 要素の配列。
-----------------	----------------------------

[gg.h](#) の 4906 行目に定義があります。

8.26.3.11 `getStart()` [3/3]

```
const GLfloat & gg::GgTrackball::getStart (
    int direction) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向.
------------------	-----------------------

[gg.h](#) の 4896 行目に定義があります。

8.26.3.12 `motion()`

```
void gg::GgTrackball::motion (
    GLfloat x,
    GLfloat y)
```

回転の変換行列を計算する。

引数

<i>x</i>	現在のマウスの x 座標.
<i>y</i>	現在のマウスの y 座標.

覚え書き

マウスのドラッグ中に呼び出す。

[gg.cpp](#) の 3469 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:

8.26.3.13 `operator=()`

```
GgTrackball & gg::GgTrackball::operator= (
    const GgQuaternion & q) [inline]
```

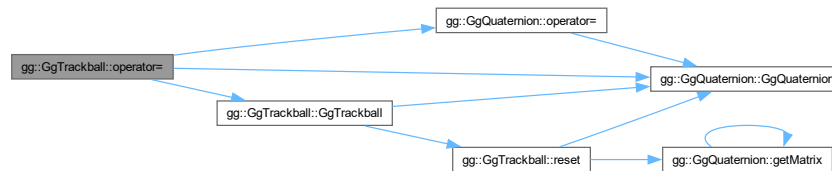
代入.

引数

<i>q</i>	トラックボールの回転の初期値の四元数.
----------	---------------------

[gg.h](#) の 4805 行目に定義があります。

呼び出し関係図:



8.26.3.14 region() [1/2]

```
void gg::GgTrackball::region (
    GLfloat w,
    GLfloat h)
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す.

[gg.cpp](#) の 3440 行目に定義があります。

被呼び出し関係図:



8.26.3.15 region() [2/2]

```
void gg::GgTrackball::region (
    int w,
    int h) [inline]
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す.

gg.h の 4831 行目に定義があります.

呼び出し関係図:



8.26.3.16 reset()

```
void gg::GgTrackball::reset (
    const GgQuaternion & q = ggIdentityQuaternion())
```

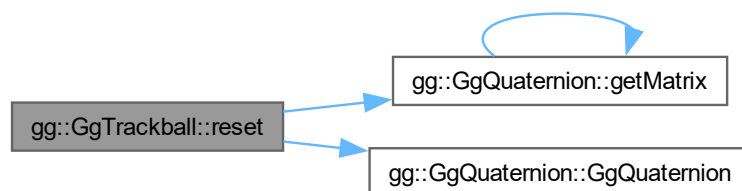
トラックボールをリセットする.

引数

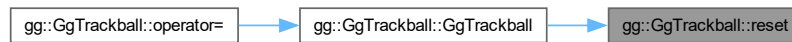
<i>q</i>	トラックボールの回転の初期値の四元数.
----------	---------------------

gg.cpp の 3422 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



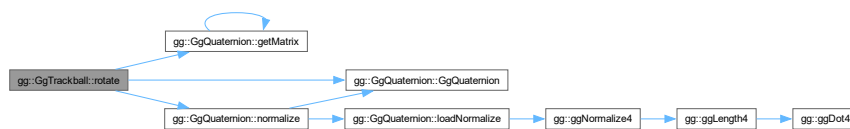
8.26.3.17 rotate()

```
void gg::GgTrackball::rotate (
    const GgQuaternion & q)
    トラックボールの回転角を修正する.
```

引数

<i>q</i>	修正分の回転角の四元数.
----------	--------------

[gg.cpp](#) の 3498 行目に定義があります。
呼び出し関係図:



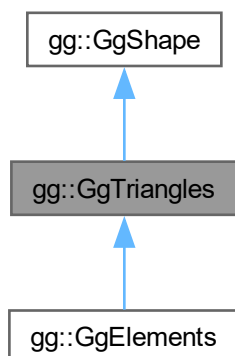
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

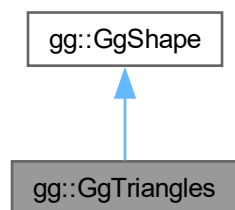
8.27 gg::GgTriangles クラス

```
#include <gg.h>
```


gg::GgTriangles の継承関係図



gg::GgTriangles 連携図



公開メンバ関数

- [GgTriangles](#) (GLenum mode=GL_TRIANGLES)
- [GgTriangles](#) (const [GgVertex](#) *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgTriangles](#) ()
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLint first=0, GLsizei count=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const

- void `setMode` (GLenum mode)
- const GLenum & `getMode` () const

8.27.1 詳解

三角形で表した形状データ (Arrays 形式).
[gg.h](#) の 6509 行目に定義があります。

8.27.2 構築子と解体子

8.27.2.1 GgTriangles() [1/2]

```
gg::GgTriangles::GgTriangles (
    GLenum mode = GL_TRIANGLES) [inline]
コンストラクタ.
```

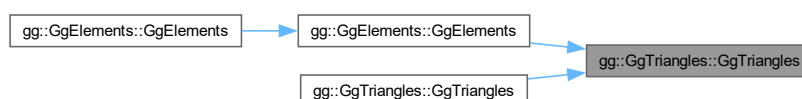
引数

<i>mode</i>	描画する基本図形の種類.
-------------	--------------

[gg.h](#) の 6522 行目に定義があります。
 呼び出し関係図:



被呼び出し関係図:



8.27.2.2 GgTriangles() [2/2]

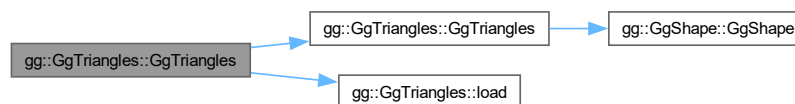
```
gg::GgTriangles::GgTriangles (
    const GgVertex * vert,
    GLsizei count,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
コンストラクタ.
```

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
-------------	-------------------------------------

<i>count</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6535 行目に定義があります。
呼び出し関係図:



8.27.2.3 ~GgTriangles()

```
virtual gg::GgTriangles::~~GgTriangles () [inline], [virtual]
```

デストラクタ.
gg.h の 6549 行目に定義があります。

8.27.3 関数詳解

8.27.3.1 draw()

```
void gg::GgTriangles::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

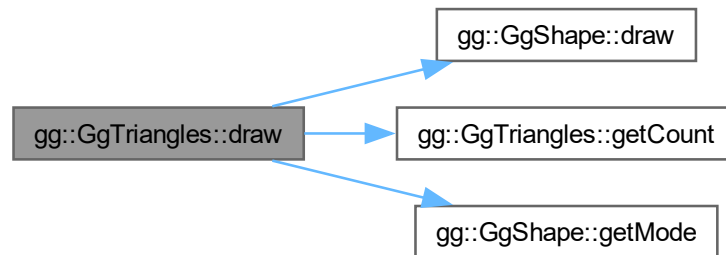
三角形の描画.

引数

<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

gg::GgShape を再実装しています。
gg::GgElements で再実装されています。
gg.cpp の 5157 行目に定義があります。

呼び出し関係図:



8.27.3.2 getBuffer()

```
const GLuint & gg::GgTriangles::getBuffer () const [inline]
```

頂点属性を格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の頂点属性を格納した頂点バッファオブジェクト名.

[gg.h](#) の 6568 行目に定義があります。

8.27.3.3 getCount()

```
const GLsizei & gg::GgTriangles::getCount () const [inline]
```

データの数を取り出す.

戻り値

この図形の頂点属性の数 (頂点数).

[gg.h](#) の 6558 行目に定義があります。

被呼び出し関係図:



8.27.3.4 load()

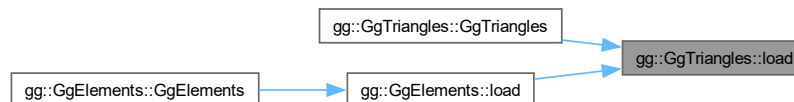
```
void gg::GgTriangles::load (
    const GgVertex * vert,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点属性を格納する.

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>count</i>	頂点のデータの数 (頂点数).
<i>usage</i>	バッファオブジェクトの使い方.

gg.cpp の 5138 行目に定義があります。
被呼び出し関係図:



8.27.3.5 send()

```
void gg::GgTriangles::send (
    const GgVertex * vert,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点属性を転送する.

引数

<i>vert</i>	転送元の頂点属性が格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

gg.h の 6580 行目に定義があります。
被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- gg.h
- gg.cpp

8.28 gg::GgUniformBuffer< T > クラステンプレート

```
#include <gg.h>
```

公開メンバ関数

- [GgUniformBuffer](#) ()
- [GgUniformBuffer](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- [GgUniformBuffer](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgUniformBuffer](#) ()
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLuint first, GLsizei count) const
- void [unmap](#) () const
- void [load](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [load](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [send](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [fill](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [read](#) (GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.28.1 詳解

```
template<typename T>
class gg::GgUniformBuffer< T >
```

ユニフォームバッファオブジェクト。

覚え書き

ユニフォーム変数を格納するバッファオブジェクトの基底クラス。

[gg.h](#) の 5827 行目に定義があります。

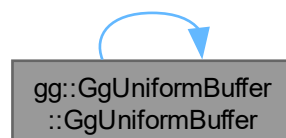
8.28.2 構築子と解体子

8.28.2.1 GgUniformBuffer() [1/3]

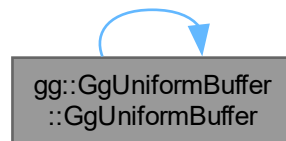
```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer () [inline]
コンストラクタ。
```

[gg.h](#) の 5830 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.28.2.2 GgUniformBuffer() [2/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

ユニフォームバッファオブジェクトのブロックごとにデータを転送するコンストラクタ.

引数

<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5830 行目に定義があります。

8.28.2.3 GgUniformBuffer() [3/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T & data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

ユニフォームバッファオブジェクトの全ブロックに同じデータを格納するコンストラクタ.

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5830 行目に定義があります。

8.28.2.4 ~GgUniformBuffer()

```
template<typename T>
virtual gg::GgUniformBuffer< T >::~GgUniformBuffer () [inline], [virtual]
デストラクタ.
```

gg.h の 5830 行目に定義があります。

8.28.3 関数詳解

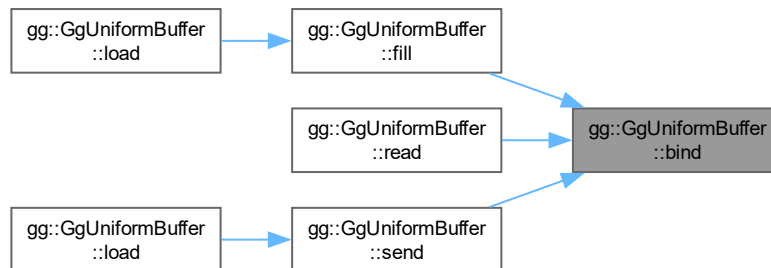
8.28.3.1 bind()

```
template<typename T>
void gg::GgUniformBuffer< T >::bind () const [inline]
```

ユニフォームバッファオブジェクトを結合する。

gg.h の 5915 行目に定義があります。

被呼び出し関係図:



8.28.3.2 copy()

```
template<typename T>
void gg::GgUniformBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
```

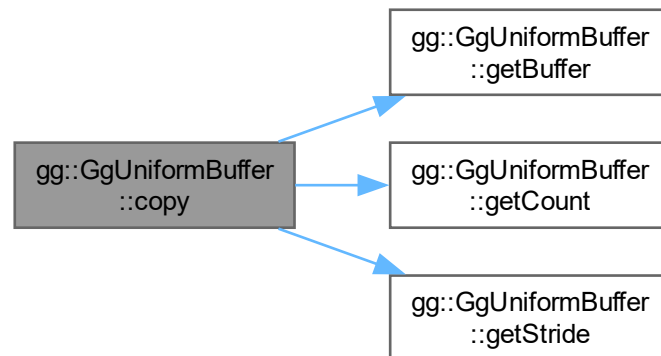
別のバッファオブジェクトからデータを複写する。

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (buffer) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6126 行目に定義があります。

呼び出し関係図:



8.28.3.3 fill()

```

template<typename T>
void gg::GgUniformBuffer< T >::fill (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
  
```

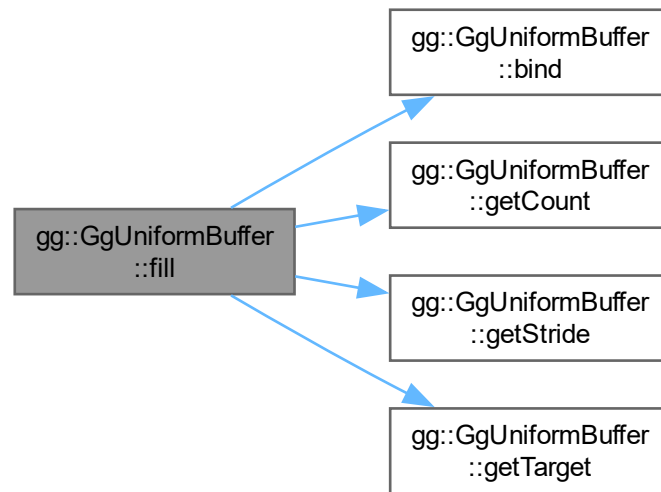
ユニフォームバッファオブジェクトの全ブロックのメンバーを同じデータを格納する.

引数

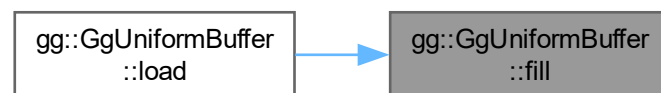
<i>data</i>	格納するデータ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

[gg.h](#) の 6043 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.28.3.4 getBuffer()

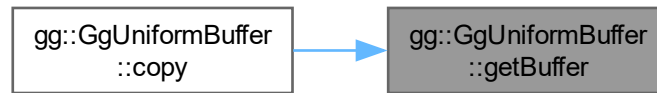
```
template<typename T>
const GLuint & gg::GgUniformBuffer< T >::getBuffer () const [inline]
ユニフォームバッファオブジェクト名を取り出す.
```

戻り値

このユニフォームバッファオブジェクト名.

gg.h の 5907 行目に定義があります。

被呼び出し関係図:



8.28.3.5 getCount()

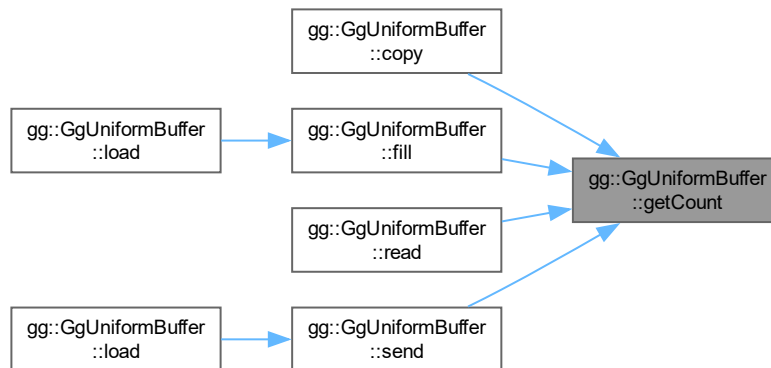
```
template<typename T>
const GLsizei & gg::GgUniformBuffer< T >::getCount () const [inline]
データの数を取り出す.
```

戻り値

このユニフォームバッファオブジェクトのデータの数.

gg.h の 5897 行目に定義があります。

被呼び出し関係図:



8.28.3.6 getStride()

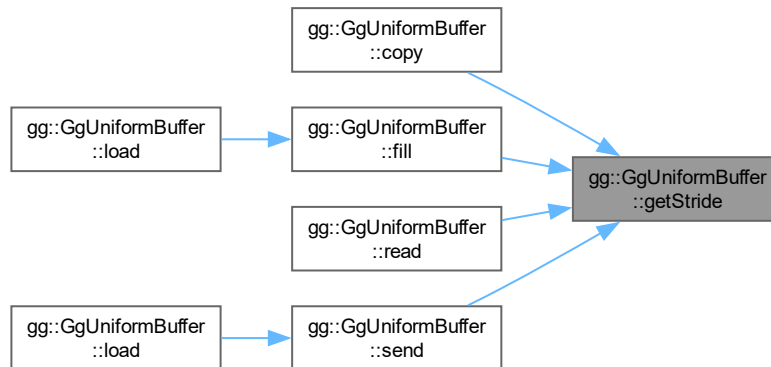
```
template<typename T>
GLsizeiptr gg::GgUniformBuffer< T >::getStride () const [inline]
ユニフォームバッファオブジェクトのアライメントを考慮したデータの間隔を取り出す.
```

戻り値

このユニフォームバッファオブジェクトのデータの間隔.

gg.h の 5887 行目に定義があります。

被呼び出し関係図:



8.28.3.7 getTarget()

```
template<typename T>
```

```
const GLuint & gg::GgUniformBuffer< T >::getTarget () const [inline]
```

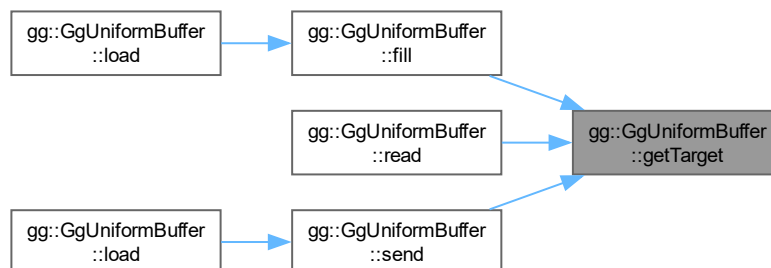
ユニフォームバッファオブジェクトのターゲットを取り出す。

戻り値

このユニフォームバッファオブジェクトのターゲット.

gg.h の 5877 行目に定義があります。

被呼び出し関係図:



8.28.3.8 load() [1/2]

```
template<typename T>
```

```
void gg::GgUniformBuffer< T >::load (
    const T & data,
```

```
GLsizei count,
GLenum usage = GL_STATIC_DRAW) [inline]
```

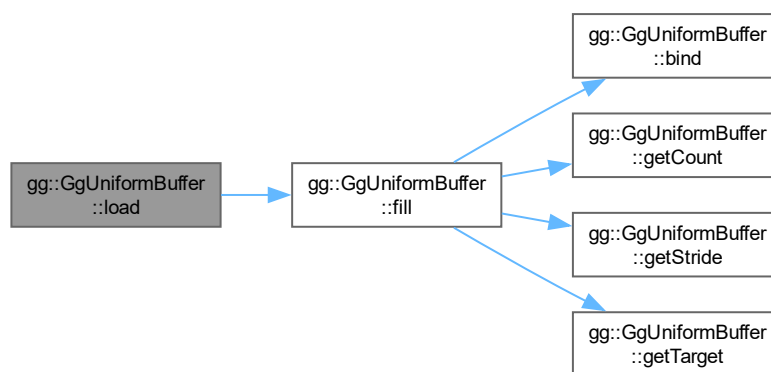
ユニフォームバッファオブジェクトを確保して全てのブロックに同じデータを格納する.

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5984 行目に定義があります。

呼び出し関係図:



8.28.3.9 load() [2/2]

```
template<typename T>
void gg::GgUniformBuffer< T >::load (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

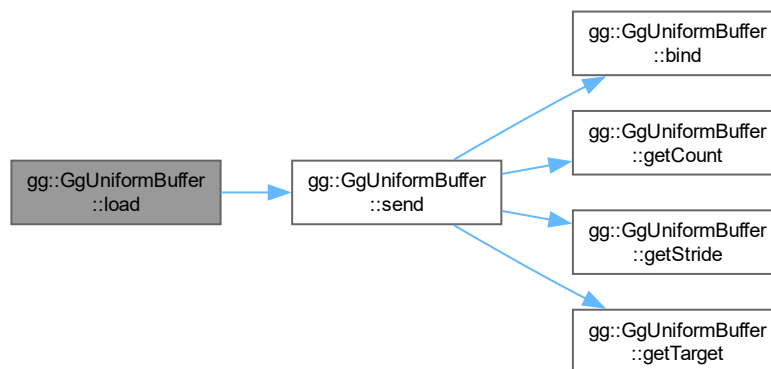
ユニフォームバッファオブジェクトを確保してブロックごとにデータを転送する.

引数

<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5965 行目に定義があります。

呼び出し関係図:



8.28.3.10 map() [1/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map () const [inline]
```

ユニフォームバッファオブジェクトをマップする.

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5933 行目に定義があります。

8.28.3.11 map() [2/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

ユニフォームバッファオブジェクトの指定した範囲をマップする.

引数

<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5945 行目に定義があります。

8.28.3.12 read()

```
template<typename T>
void gg::GgUniformBuffer< T >::read (
    GLvoid * data,
    GLint offset = 0,
```

```

    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]

```

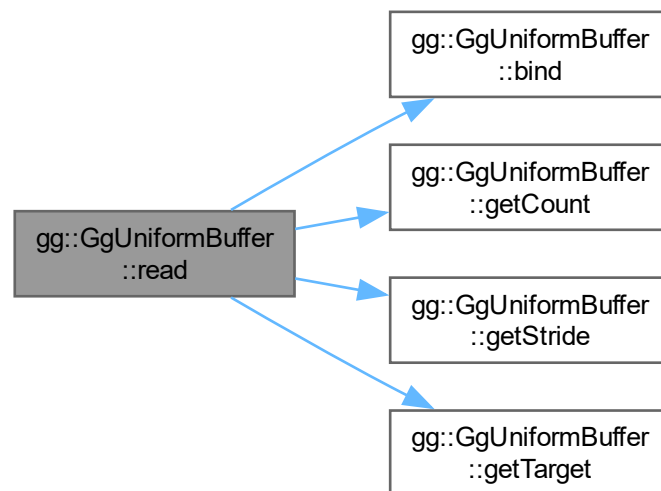
ユニフォームバッファオブジェクトからデータを抽出する。

引数

<i>data</i>	抽出先の領域の先頭のポインタ。
<i>offset</i>	抽出元のユニフォームバッファオブジェクトのメンバのブロックの先頭からのバイトオフセット。
<i>size</i>	抽出するデータの一個あたりのバイト数。
<i>first</i>	抽出元のユニフォームバッファオブジェクトのブロックの先頭の番号。
<i>count</i>	抽出するデータの数 (0 ならユニフォームバッファオブジェクト全体)。

gg.h の 6078 行目に定義があります。

呼び出し関係図:



8.28.3.13 send()

```

template<typename T>
void gg::GgUniformBuffer< T >::send (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]

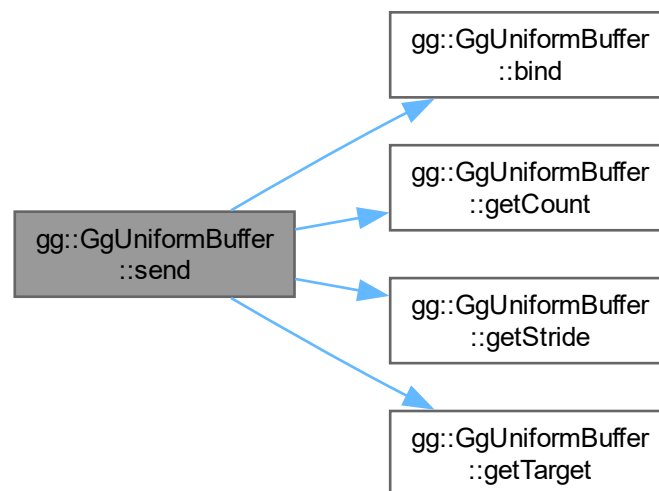
```

ユニフォームバッファオブジェクトを確保してユニフォームバッファオブジェクトのブロックごとのメンバを同じデータで埋める。

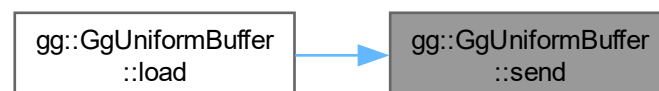
引数

<i>data</i>	データが格納されている領域の先頭のポインタ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

gg.h の 6005 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.28.3.14 unbind()

```
template<typename T>
void gg::GgUniformBuffer< T >::unbind () const [inline]
```

ユニフォームバッファオブジェクトを解放する。
gg.h の 5923 行目に定義があります。

8.28.3.15 unmap()

```
template<typename T>
```



```
void gg::GgUniformBuffer< T >::unmap () const [inline]
```

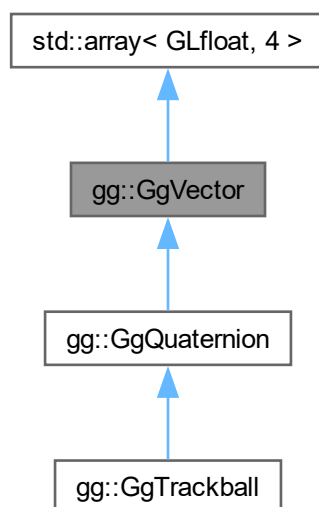
バッファオブジェクトをアンマップする。
[gg.h](#) の 5953 行目に定義があります。
 このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

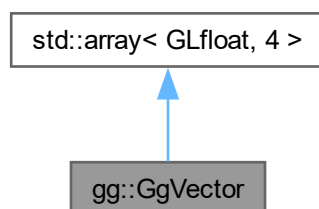
8.29 gg::GgVector クラス

```
#include <gg.h>
```

gg::GgVector の継承関係図



gg::GgVector 連携図



公開メンバ関数

- [GgVector](#) ()=default
- constexpr [GgVector](#) (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr [GgVector](#) (const GLfloat *a)

- constexpr `GgVector` (GLfloat c)
- `GgVector` (const `GgVector` &v)=default
- `GgVector` (`GgVector` &&v)=default
- `~GgVector` ()=default
- `GgVector` & `operator=` (const `GgVector` &v)=default
- `GgVector` & `operator=` (`GgVector` &&v)=default
- `GgVector operator+` (const `GgVector` &v) const
- `GgVector operator+` (GLfloat c) const
- `GgVector` & `operator+=` (const `GgVector` &v)
- `GgVector` & `operator+=` (GLfloat c)
- `GgVector operator-` (const `GgVector` &v) const
- `GgVector operator-` (GLfloat c) const
- `GgVector` & `operator-=` (const `GgVector` &v)
- `GgVector` & `operator-=` (GLfloat c)
- `GgVector operator*` (const `GgVector` &v) const
- `GgVector operator*` (GLfloat c) const
- `GgVector` & `operator*=` (const `GgVector` &v)
- `GgVector` & `operator*=` (GLfloat c)
- `GgVector operator/` (const `GgVector` &v) const
- `GgVector operator/` (GLfloat c) const
- `GgVector` & `operator/=` (`GgVector` &v)
- `GgVector` & `operator/=` (GLfloat c)
- GLfloat `dot3` (const `GgVector` &v) const
- GLfloat `length3` () const
- GLfloat `distance3` (const `GgVector` &v) const
- `GgVector normalize3` () const
- GLfloat `dot4` (const `GgVector` &v) const
- GLfloat `length4` () const
- GLfloat `distance4` (const `GgVector` &v) const
- `GgVector normalize4` () const

8.29.1 詳解

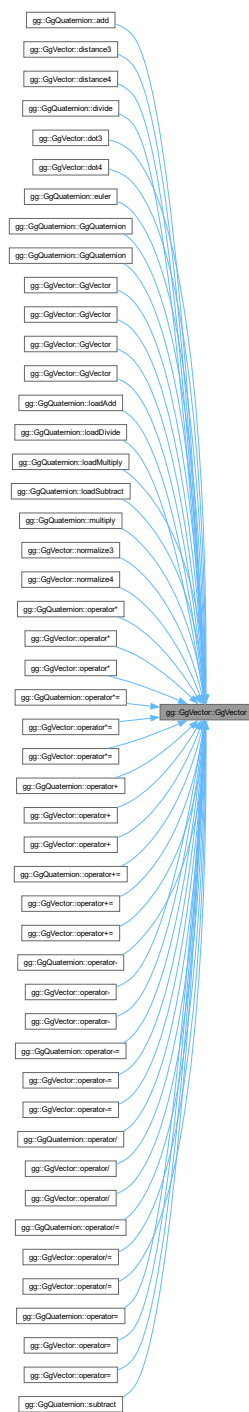
単精度実数の 4 要素のベクトル。
[gg.h](#) の 1571 行目に定義があります。

8.29.2 構築子と解体子

8.29.2.1 `GgVector()` [1/6]

```
gg::GgVector::GgVector () [default]
```

コンストラクタ. 被呼び出し関係図:



8.29.2.2 GgVector() [2/6]

```
gg::GgVector::GgVector (
    GLfloat v0,
    GLfloat v1,
    GLfloat v2,
    GLfloat v3) [inline], [constexpr]
コンストラクタ.
```

引数

<i>v0</i>	GLfloat 型の値.
<i>v1</i>	GLfloat 型の値.
<i>v2</i>	GLfloat 型の値.
<i>v3</i>	GLfloat 型の値.

[gg.h](#) の 1588 行目に定義があります。

8.29.2.3 GgVector() [3/6]

```
gg::GgVector::GgVector (
    const GLfloat * a) [inline], [constexpr]
コンストラクタ.
```

引数

<i>a</i>	GLfloat 型の 4 要素の配列変数.
----------	-----------------------

[gg.h](#) の 1598 行目に定義があります。
呼び出し関係図:



8.29.2.4 GgVector() [4/6]

```
gg::GgVector::GgVector (
    GLfloat c) [inline], [constexpr]
コンストラクタ.
```

引数

<i>c</i>	GLfloat 型の値.
----------	--------------

[gg.h](#) の 1608 行目に定義があります。
呼び出し関係図:



8.29.2.5 GgVector() [5/6]

```
gg::GgVector::GgVector (
    const GgVector & v) [default]
コピーコンストラクタ.
```

引数

<i>v</i>	コピー元のベクトル.
----------	------------

呼び出し関係図:

**8.29.2.6 GgVector()** [6/6]

```
gg::GgVector::GgVector (
    GgVector && v) [default]
ムーブコンストラクタ.
```

引数

<i>v</i>	ムーブ元のベクトル.
----------	------------

呼び出し関係図:

**8.29.2.7 ~GgVector()**

```
gg::GgVector::~~GgVector () [default]
デストラクタ.
```

8.29.3 関数詳解**8.29.3.1 distance3()**

```
GLfloat gg::GgVector::distance3 (
    const GgVector & v) const [inline]
```

GgVector 型の 3 要素の距離.

引数

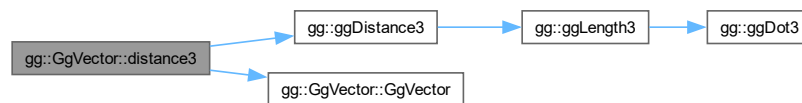
v	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと **v** の 3 要素の距離.

gg.h の 1852 行目に定義があります。

呼び出し関係図:



8.29.3.2 distance4()

```
GLfloat gg::GgVector::distance4 (
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の距離.

引数

v	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと **v** の 4 要素の距離.

gg.h の 1896 行目に定義があります。

呼び出し関係図:



8.29.3.3 dot3()

```
GLfloat gg::GgVector::dot3 (
    const GgVector & v) const [inline]
```

GgVector 型の 3 要素の内積.

引数

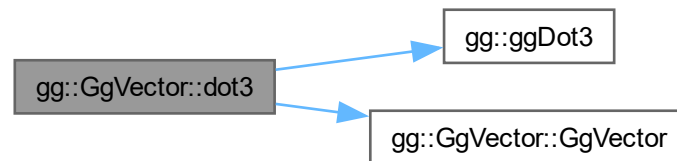
v	GgVector 型のベクトル.
-----	------------------

戻り値

オブジェクトと v のそれぞれの 3 要素の内積.

gg.h の 1831 行目に定義があります。

呼び出し関係図:



8.29.3.4 dot4()

```
GLfloat gg::GgVector::dot4 (  
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の内積.

引数

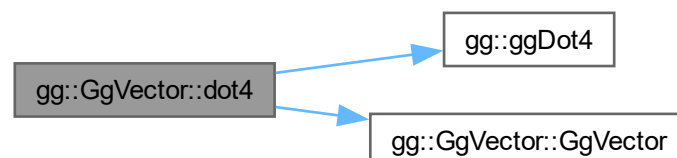
v	GgVector 型のベクトル.
-----	------------------

戻り値

オブジェクトと v のそれぞれの 4 要素の内積.

gg.h の 1875 行目に定義があります。

呼び出し関係図:



8.29.3.5 length3()

GLfloat gg::GgVector::length3 () const [inline]

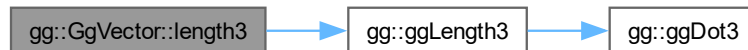
GgVector 型の 3 要素の長さ.

戻り値

オブジェクトの 4 要素の長さ.

gg.h の 1841 行目に定義があります。

呼び出し関係図:



8.29.3.6 length4()

GLfloat gg::GgVector::length4 () const [inline]

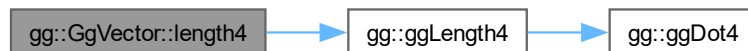
GgVector 型の 4 要素の長さ.

戻り値

オブジェクトの 4 要素の長さ.

gg.h の 1885 行目に定義があります。

呼び出し関係図:



8.29.3.7 normalize3()

GgVector gg::GgVector::normalize3 () const [inline]

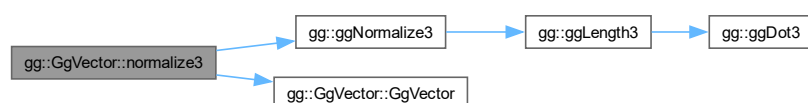
GgVector 型の 4 要素の正規化.

戻り値

GLfloat 型の 4 要素の配列変数.

gg.h の 1862 行目に定義があります。

呼び出し関係図:



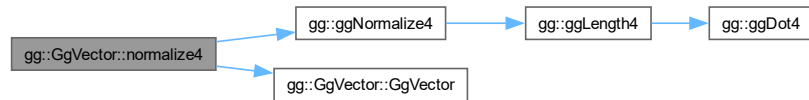
8.29.3.8 normalize4()

`GgVector` `gg::GgVector::normalize4 () const [inline]`
`GgVector` 型の 4 要素の正規化.

戻り値

GLfloat 型の 4 要素の配列変数.

`gg.h` の 1906 行目に定義があります。
 呼び出し関係図:



8.29.3.9 operator*() [1/2]

`GgVector` `gg::GgVector::operator* (`
 `const GgVector & v) const [inline]`
`GgVector` 型の積を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの積のオブジェクト.

`gg.h` の 1740 行目に定義があります。
 呼び出し関係図:



8.29.3.10 operator*() [2/2]

`GgVector` `gg::GgVector::operator* (`
 `GLfloat c) const [inline]`
`GgVector` 型の各要素にスカラーを乗じた積を返す.

引数

<code>c</code>	GLfloat 型の変数.
----------------	---------------

戻り値

オブジェクトの各要素に `c` を乗じたオブジェクト.

`gg.h` の 1751 行目に定義があります.

呼び出し関係図:



8.29.3.11 operator*=() [1/2]

```
GgVector & gg::GgVector::operator*= (
    const GgVector & v) [inline]
```

`GgVector` 型を乗算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ乗算したオブジェクトの参照.

`gg.h` の 1761 行目に定義があります.

呼び出し関係図:



8.29.3.12 operator*=() [2/2]

```
GgVector & gg::GgVector::operator*= (
    GLfloat c) [inline]
```

`GgVector` 型の各要素にスカラーを乗じる.

引数

c	GgVector 型のベクトル.
---	------------------

戻り値

オブジェクトの各要素に c を乗じたオブジェクトの参照.

gg.h の 1773 行目に定義があります.

呼び出し関係図:

gg::GgVector::operator*=

gg::GgVector::GgVector

8.29.3.13 operator+() [1/2]

```
GgVector gg::GgVector::operator+ (  
    const GgVector & v) const [inline]
```

GgVector 型の和を返す.

引数

v	GgVector 型のベクトル.
---	------------------

戻り値

オブジェクトの各要素と v の各要素の要素ごとの和のオブジェクト.

gg.h の 1648 行目に定義があります.

呼び出し関係図:

gg::GgVector::operator+

gg::GgVector::GgVector

8.29.3.14 operator+() [2/2]

```
GgVector gg::GgVector::operator+ (  
    GLfloat c) const [inline]
```

GgVector 型の各要素にスカラーを足した和を返す.

引数

c	GLfloat 型の値.
---	--------------

戻り値

a の各要素に b を足した和のオブジェクト.

gg.h の 1659 行目に定義があります。

呼び出し関係図:



8.29.3.15 operator+=() [1/2]

```
GgVector & gg::GgVector::operator+= (  
    const GgVector & v) [inline]
```

GgVector 型を加算する.

引数

v	GgVector 型の変数.
---	----------------

戻り値

オブジェクトの各要素に v の各要素をそれぞれ加算したオブジェクトの参照.

gg.h の 1670 行目に定義があります。

呼び出し関係図:



8.29.3.16 operator+=() [2/2]

```
GgVector & gg::GgVector::operator+= (  
    GLfloat c) [inline]
```

GgVector 型の各要素にスカラーを加算する.

引数

c	GLfloat 型の値.
---	--------------

戻り値

オブジェクトの各要素に c を足したオブジェクトの参照.

gg.h の 1682 行目に定義があります.

呼び出し関係図:



8.29.3.17 operator-() [1/2]

```
GgVector gg::GgVector::operator- (
    const GgVector & v) const [inline]
```

GgVector 型の差を返す.

引数

v	GgVector 型の変数.
---	----------------

戻り値

オブジェクトの各要素と v の各要素の要素ごとの差のオブジェクト.

gg.h の 1694 行目に定義があります.

呼び出し関係図:



8.29.3.18 operator-() [2/2]

```
GgVector gg::GgVector::operator- (
    GLfloat c) const [inline]
```

GgVector 型の各要素からスカラーを引いた差を返す.

引数

c	GLfloat 型の変数.
---	---------------

戻り値

オブジェクトの各要素から c を引いたオブジェクト.

gg.h の 1705 行目に定義があります.

呼び出し関係図:



8.29.3.19 operator-=() [1/2]

```
GgVector & gg::GgVector::operator-= (
    const GgVector & v) [inline]
```

GgVector 型を減算する.

引数

v	GgVector 型の変数.
---	----------------

戻り値

オブジェクトの各要素に v の各要素をそれぞれ減算したオブジェクトの参照.

gg.h の 1716 行目に定義があります.

呼び出し関係図:



8.29.3.20 operator-=() [2/2]

```
GgVector & gg::GgVector::operator-= (
    GLfloat c) [inline]
```

GgVector 型の各要素からスカラーを引く.

引数

c	GLfloat 型の変数.
---	---------------

戻り値

オブジェクトの各要素から c を引いたオブジェクトの参照.

gg.h の 1728 行目に定義があります.

呼び出し関係図:



8.29.3.21 operator/() [1/2]

```
GgVector gg::GgVector::operator/ (
    const GgVector & v) const [inline]
```

GgVector 型の商を返す.

引数

v	GgVector 型の変数.
---	----------------

戻り値

オブジェクトの各要素を v の各要素で要素ごとに割った結果のオブジェクト.

gg.h の 1785 行目に定義があります.

呼び出し関係図:



8.29.3.22 operator/() [2/2]

```
GgVector gg::GgVector::operator/ (
    GLfloat c) const [inline]
```

GgVector 型の各要素をスカラーで割った商を返す.

引数

c	GLfloat 型の変数.
---	---------------

戻り値

オブジェクトの各要素を c で割ったオブジェクト.

gg.h の 1796 行目に定義があります.

呼び出し関係図:



8.29.3.23 operator/=() [1/2]

```
GgVector & gg::GgVector::operator/= (
    GgVector & v) [inline]
```

GgVector 型を除算する.

引数

v	GgVector 型の変数.
---	----------------

戻り値

オブジェクトの各要素に v の各要素をそれぞれ乗算したオブジェクトの参照.

gg.h の 1807 行目に定義があります.

呼び出し関係図:



8.29.3.24 operator/=() [2/2]

```
GgVector & gg::GgVector::operator/= (
    GLfloat c) [inline]
```

GgVector 型の各要素をスカラーで割る.

引数

c	GLfloat 型の変数.
---	---------------

戻り値

オブジェクトの各要素を c で割ったオブジェクトの参照.

gg.h の 1819 行目に定義があります。

呼び出し関係図:



8.29.3.25 operator=() [1/2]

```
GgVector & gg::GgVector::operator= (  
    const GgVector & v) [default]
```

代入演算子. 呼び出し関係図:



8.29.3.26 operator=() [2/2]

```
GgVector & gg::GgVector::operator= (  
    GgVector && v) [default]
```

ムーブ代入演算子. 呼び出し関係図:

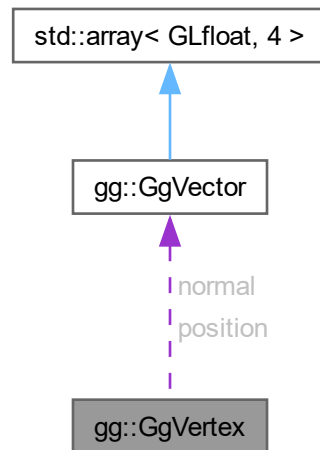


このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.30 gg::GgVertex 構造体

```
#include <gg.h>
gg::GgVertex 連携図
```



公開メンバ関数

- `GgVertex ()`
- `GgVertex (const GgVector &pos, const GgVector &norm)`
- `GgVertex (GLfloat px, GLfloat py, GLfloat pz, GLfloat nx, GLfloat ny, GLfloat nz)`
- `GgVertex (const GLfloat *pos, const GLfloat *norm)`

公開変数類

- `GgVector position`
位置.
- `GgVector normal`
法線.

8.30.1 詳解

三角形の頂点データ.
[gg.h](#) の 6448 行目に定義があります。

8.30.2 構築子と解体子

8.30.2.1 GgVertex() [1/4]

`gg::GgVertex::GgVertex () [inline]`
 コンストラクタ.
[gg.h](#) の 6459 行目に定義があります。

被呼び出し関係図:



8.30.2.2 GgVertex() [2/4]

```
gg::GgVertex::GgVertex (
    const GgVector & pos,
    const GgVector & norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	GgVector 型の位置データ.
<i>norm</i>	GgVector 型の法線データ.

gg.h の 6469 行目に定義があります。

8.30.2.3 GgVertex() [3/4]

```
gg::GgVertex::GgVertex (
    GLfloat px,
    GLfloat py,
    GLfloat pz,
    GLfloat nx,
    GLfloat ny,
    GLfloat nz) [inline]
```

コンストラクタ.

引数

<i>px</i>	GgVector 型の位置データの x 成分.
<i>py</i>	GgVector 型の位置データの y 成分.
<i>pz</i>	GgVector 型の位置データの z 成分.
<i>nx</i>	GgVector 型の法線データの x 成分.
<i>ny</i>	GgVector 型の法線データの y 成分.
<i>nz</i>	GgVector 型の法線データの z 成分.

gg.h の 6485 行目に定義があります。

8.30.2.4 GgVertex() [4/4]

```
gg::GgVertex::GgVertex (
    const GLfloat * pos,
    const GLfloat * norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	3 要素の GLfloat 型の位置データのポインタ.
<i>norm</i>	3 要素の GLfloat 型の法線データのポインタ.

[gg.h](#) の 6500 行目に定義があります。

呼び出し関係図:



8.30.3 メンバ詳解

8.30.3.1 normal

[GgVector](#) [gg::GgVertex::normal](#)

法線.

[gg.h](#) の 6454 行目に定義があります。

8.30.3.2 position

[GgVector](#) [gg::GgVertex::position](#)

位置.

[gg.h](#) の 6451 行目に定義があります。

この構造体詳解は次のファイルから抽出されました:

- [gg.h](#)

8.31 gg::GgVertexArray クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgVertexArray](#) (GLenum mode=0)
- [GgVertexArray](#) (const [GgVertexArray](#) &array)=delete
- [GgVertexArray](#) ([GgVertexArray](#) &&array)=default
- virtual ~[GgVertexArray](#) ()
- [GgVertexArray](#) & operator= (const [GgVertexArray](#) &array)=delete
- [GgVertexArray](#) & operator= ([GgVertexArray](#) &&array)=default
- const GLuint & [get](#) () const
- void [bind](#) () const

8.31.1 詳解

頂点配列クラス.

[gg.h](#) の 6157 行目に定義があります。

8.31.2 構築子と解体子

8.31.2.1 GgVertexArray() [1/3]

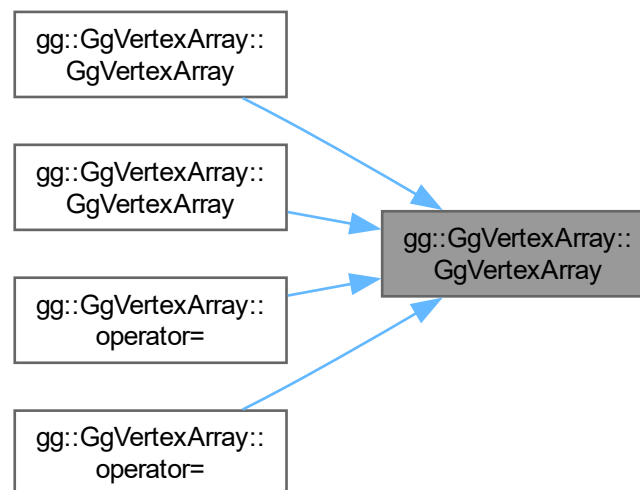
```
gg::GgVertexArray::GgVertexArray (
    GLenum mode = 0) [inline]
```

コンストラクタ.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

gg.h の 6169 行目に定義があります。
被呼び出し関係図:



8.31.2.2 GgVertexArray() [2/3]

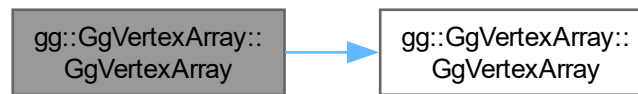
```
gg::GgVertexArray::GgVertexArray (
    const GgVertexArray & array) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>array</i>	コピー元の頂点配列オブジェクト.
--------------	------------------

呼び出し関係図:



8.31.2.3 GgVertexArray() [3/3]

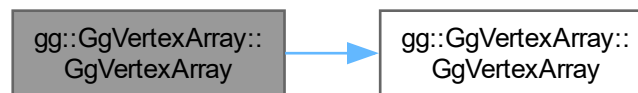
```
gg::GgVertexArray::GgVertexArray (
    GgVertexArray && array) [default]
```

ムーブコンストラクタ.

引数

<i>array</i>	ムーブ元の頂点配列オブジェクト.
--------------	------------------

呼び出し関係図:



8.31.2.4 ~GgVertexArray()

```
virtual gg::GgVertexArray::~~GgVertexArray () [inline], [virtual]
```

デストラクタ.
[gg.h](#) の 6192 行目に定義があります。

8.31.3 関数詳解

8.31.3.1 bind()

```
void gg::GgVertexArray::bind () const [inline]
```

頂点配列オブジェクトを結合する.
[gg.h](#) の 6227 行目に定義があります。

8.31.3.2 get()

```
const GLuint & gg::GgVertexArray::get () const [inline]
```

頂点配列オブジェクト名を取り出す.

戻り値

頂点配列オブジェクト名.

gg.h の 6219 行目に定義があります。

8.31.3.3 operator=() [1/2]

```
GgVertexArray & gg::GgVertexArray::operator= (
    const GgVertexArray & array) [delete]
```

代入演算子は使用しない.

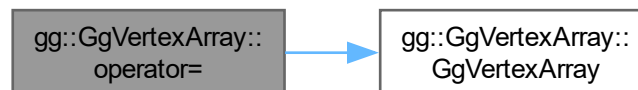
引数

<code>array</code>	代入元の頂点配列オブジェクト.
--------------------	-----------------

戻り値

代入後のこの頂点配列オブジェクトの参照.

呼び出し関係図:



8.31.3.4 operator=() [2/2]

```
GgVertexArray & gg::GgVertexArray::operator= (
    GgVertexArray && array) [default]
```

ムーブ代入演算子.

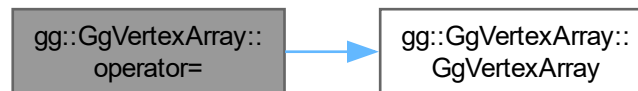
引数

<code>array</code>	ムーブ代入元の頂点配列オブジェクト.
--------------------	--------------------

戻り値

ムーブ代入後のこの頂点配列オブジェクトの参照.

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.32 Intrinsic 構造体

```
#include <Intrinsic.h>
```

公開メンバ関数

- [Intrinsic](#) ()
- [Intrinsic](#) (const std::array< float, 2 > &[fov](#), const std::array< float, 2 > &[center](#), const std::array< int, 2 > [size](#), double [fps](#))
- [Intrinsic](#) (const picojson::object &object)
- void [setFov](#) (float focal)
- void [setFov](#) (float fovx, float fovy)
- void [setCenter](#) (float x, float y)
- void [setSize](#) (int width, int height)
- void [setFps](#) (double frequency)

公開変数類

- std::array< float, 2 > [fov](#)
キャプチャデバイスのレンズの縦横の画角
- std::array< float, 2 > [center](#)
キャプチャデバイスのレンズの中心 (主点) の位置
- std::array< int, 2 > [size](#)
キャプチャデバイスの解像度
- double [fps](#)
キャプチャデバイスのフレームレート

静的公開変数類

- static constexpr auto [sensorSize](#) { 35.0f }
展開時のセンサーサイズ

8.32.1 詳解

キャプチャデバイス固有のパラメータ
[Intrinsic.h](#) の 20 行目に定義があります。

8.32.2 構築子と解体子

8.32.2.1 Intrinsics() [1/3]

`Intrinsics::Intrinsics () [inline]`
 キャプチャデバイス固有のパラメータの構造体のデフォルトコンストラクタ

覚え書き

構成ファイルが読めなかったときにしか使わない

[Intrinsics.h](#) の 39 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.32.2.2 Intrinsics() [2/3]

```

Intrinsics::Intrinsics (
    const std::array< float, 2 > & fov,
    const std::array< float, 2 > & center,
    const std::array< int, 2 > size,
    double fps) [inline]
  
```

パラメータを指定するときに使う キャプチャデバイス固有のパラメータの構造体のコンストラクタ

引数

<i>fov</i>	キャプチャデバイスのレンズの縦横の画角
<i>center</i>	キャプチャデバイスのレンズの中心 (主点) の位置
<i>resolution</i>	キャプチャデバイスの解像度
<i>fps</i>	キャプチャデバイスのフレームレート

[Intrinsics.h](#) の 53 行目に定義があります。

8.32.2.3 Intrinsic() [3/3]

```
Intrinsic::Intrinsic (
    const picojson::object & object)
```

構成ファイルを読み込むときに使う キャプチャデバイス固有のパラメータの構造体のコンストラクタ [Intrinsic.cpp](#) の 14 行目に定義があります。

呼び出し関係図:



8.32.3 関数詳解

8.32.3.1 setCenter()

```
void Intrinsic::setCenter (
    float x,
    float y) [inline]
```

キャプチャデバイスのレンズの中心の位置を変更する

引数

<i>x</i>	キャプチャデバイスのレンズの中心の位置の <i>x</i> 座標
<i>y</i>	キャプチャデバイスのレンズの中心の位置の <i>y</i> 座標

[Intrinsic.h](#) の 97 行目に定義があります。

8.32.3.2 setFov() [1/2]

```
void Intrinsic::setFov (
    float focal)
```

スクリーンの高さをもとにしてキャプチャデバイスのレンズの縦横の画角を変更する

引数

<i>focal</i>	撮像面の対角線長が <i>sensorSize</i> のときの焦点距離
--------------	--------------------------------------

[Intrinsic.cpp](#) の 32 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.32.3.3 setFov() [2/2]

```
void Intrinsic::setFov (  
    float fovx,  
    float fovy) [inline]
```

キャプチャデバイスのレンズの縦横の画角を変更する

引数

<i>fovx</i>	キャプチャデバイスのレンズの <i>fovx</i> 方向の画角
<i>fovy</i>	キャプチャデバイスのレンズの <i>fovy</i> 方向の画角

[Intrinsic.h](#) の 84 行目に定義があります。

8.32.3.4 setFps()

```
void Intrinsic::setFps (  
    double frequency) [inline]
```

キャプチャデバイスのフレームレートを変更する

引数

<i>frequency</i>	フレームレート
------------------	---------

[Intrinsic.h](#) の 124 行目に定義があります。

8.32.3.5 setSize()

```
void Intrinsics::setSize (
    int width,
    int height) [inline]
```

キャプチャデバイスの解像度を変更する

引数

<i>width</i>	キャプチャデバイスのフレームの横の画素数
<i>height</i>	キャプチャデバイスのフレームの縦の画素数

[Intrinsics.h](#) の 111 行目に定義があります。

8.32.4 メンバ詳解

8.32.4.1 center

```
std::array<float, 2> Intrinsics::center
```

キャプチャデバイスのレンズの中心 (主点) の位置
[Intrinsics.h](#) の 26 行目に定義があります。

8.32.4.2 fov

```
std::array<float, 2> Intrinsics::fov
```

キャプチャデバイスのレンズの縦横の画角
[Intrinsics.h](#) の 23 行目に定義があります。

8.32.4.3 fps

```
double Intrinsics::fps
```

キャプチャデバイスのフレームレート
[Intrinsics.h](#) の 32 行目に定義があります。

8.32.4.4 sensorSize

```
auto Intrinsics::sensorSize { 35.0f } [static], [constexpr]
```

展開時のセンサーサイズ
[Intrinsics.h](#) の 69 行目に定義があります。

8.32.4.5 size

```
std::array<int, 2> Intrinsics::size
```

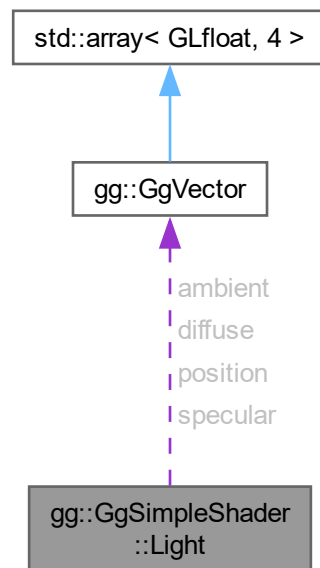
キャプチャデバイスの解像度
[Intrinsics.h](#) の 29 行目に定義があります。
 この構造体詳解は次のファイルから抽出されました:

- [Intrinsics.h](#)
- [Intrinsics.cpp](#)

8.33 gg::GgSimpleShader::Light 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Light 連携図



公開変数類

- `GgVector ambient`
光源強度の環境光成分.
- `GgVector diffuse`
光源強度の拡散反射光成分.
- `GgVector specular`
光源強度の鏡面反射光成分.
- `GgVector position`
光源の位置.

8.33.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データ.
`gg.h` の 7446 行目に定義があります。

8.33.2 メンバ詳解

8.33.2.1 ambient

`GgVector gg::GgSimpleShader::Light::ambient`
光源強度の環境光成分.
`gg.h` の 7448 行目に定義があります。

8.33.2.2 diffuse

`GgVector gg::GgSimpleShader::Light::diffuse`
光源強度の拡散反射光成分.
`gg.h` の 7449 行目に定義があります。

8.33.2.3 position

`GgVector` `gg::GgSimpleShader::Light::position`

光源の位置.

`gg.h` の 7451 行目に定義があります。

8.33.2.4 specular

`GgVector` `gg::GgSimpleShader::Light::specular`

光源強度の鏡面反射光成分.

`gg.h` の 7450 行目に定義があります。

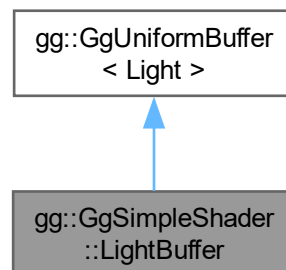
この構造体詳解は次のファイルから抽出されました:

- `gg.h`

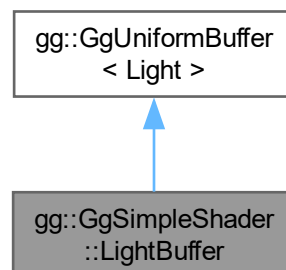
8.34 gg::GgSimpleShader::LightBuffer クラス

```
#include <gg.h>
```

`gg::GgSimpleShader::LightBuffer` の継承関係図



`gg::GgSimpleShader::LightBuffer` 連携図



公開メンバ関数

- `LightBuffer` (`const Light *light=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW`)

- `LightBuffer` (const `Light` &light, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `LightBuffer` (`GgVector` ambient, `GgVector` diffuse, `GgVector` specular, `GgVector` position, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual `~LightBuffer` ()
- void `loadAmbient` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const `GgVector` &ambient, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const `GgVector` &diffuse, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const `GgVector` &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadColor` (const `Light` &color, GLint first=0, GLsizei count=1) const
- void `loadPosition` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const `GgVector` &position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GLfloat *position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const `GgVector` *position, GLint first=0, GLsizei count=1) const
- void `load` (const `Light` *light, GLint first=0, GLsizei count=1) const
- void `load` (const `Light` &light, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Light >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const `Light` *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.34.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データのユニフォームバッファオブジェクト。`gg.h` の 7457 行目に定義があります。

8.34.2 構築子と解体子

8.34.2.1 LightBuffer() [1/3]

```
gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light * light = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

デフォルトコンストラクタ。

引数

<i>light</i>	GgSimpleShader::Light 型の光源データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, glBufferData() の第 4 引数の usage に渡される.

gg.h の 7469 行目に定義があります。

呼び出し関係図:



8.34.2.2 LightBuffer() [2/3]

```

gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light & light,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

同じデータで埋めるコンストラクタ.

引数

<i>light</i>	GgSimpleShader::Light 型の光源データ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, glBufferData() の第 4 引数の usage に渡される.

gg.h の 7485 行目に定義があります。

呼び出し関係図:



8.34.2.3 LightBuffer() [3/3]

```

gg::GgSimpleShader::LightBuffer::LightBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
  
```



```
GgVector position,
GLsizei count = 1,
GLenum usage = GL_STATIC_DRAW) [inline]
```

同じ引数で埋めるコンストラクタ.

引数

<i>ambient</i>	光源強度の環境光成分.
<i>diffuse</i>	光源強度の拡散反射光成分.
<i>specular</i>	光源強度の鏡面反射光成分.
<i>position</i>	光源の位置.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7504 行目に定義があります。
呼び出し関係図:



8.34.2.4 ~LightBuffer()

```
virtual gg::GgSimpleShader::LightBuffer::~~LightBuffer () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 7519 行目に定義があります。

8.34.3 関数詳解

8.34.3.1 load() [1/2]

```
void gg::GgSimpleShader::LightBuffer::load (
    const Light & light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の色と位置を設定する.

引数

<i>light</i>	光源の特性の GgSimpleShader::Light 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7711 行目に定義があります。

呼び出し関係図:



8.34.3.2 load() [2/2]

```
void gg::GgSimpleShader::LightBuffer::load (
    const Light * light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の色と位置を設定する。

引数

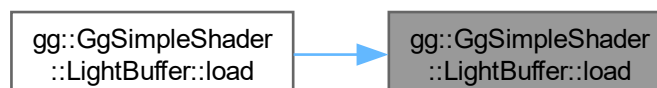
<i>light</i>	光源の特性の GgSimpleShader::Light 構造体のポインタ。
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7699 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.34.3.3 loadAmbient() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
```

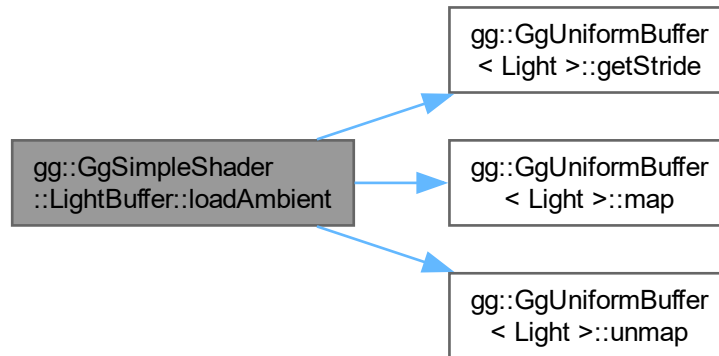
```
const GgVector & ambient,
GLint first = 0,
GLsizei count = 1) const
```

光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5483 行目に定義があります。
呼び出し関係図:



8.34.3.4 loadAmbient() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

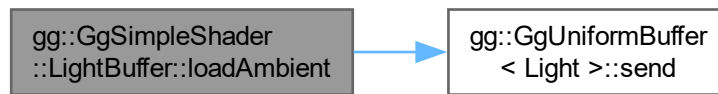
光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7554 行目に定義があります。

呼び出し関係図:



8.34.3.5 loadAmbient() [3/3]

```

void gg::GgSimpleShader::LightBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

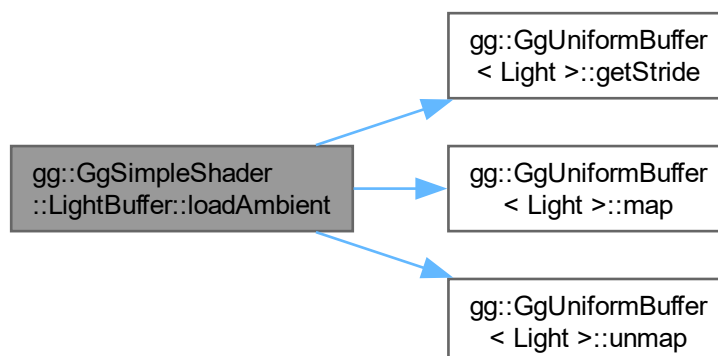
光源の強度の環境光成分を設定する。

引数

<i>r</i>	光源の強度の環境光成分の赤成分.
<i>g</i>	光源の強度の環境光成分の緑成分.
<i>b</i>	光源の強度の環境光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5457 行目に定義があります。

呼び出し関係図:



8.34.3.6 loadColor()

```
void gg::GgSimpleShader::LightBuffer::loadColor (
    const Light & color,
    GLint first = 0,
    GLsizei count = 1) const
```

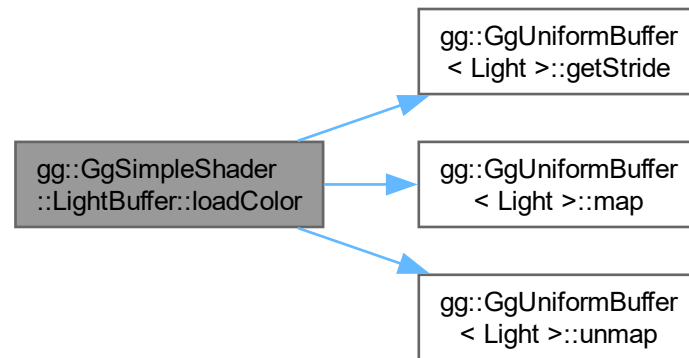
光源の色を設定するか位置は変更しない。

引数

<i>color</i>	光源の特性の GgSimpleShader::Light 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5610 行目に定義があります。

呼び出し関係図:



8.34.3.7 loadDiffuse() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    const GgVector & diffuse,
    GLint first = 0,
    GLsizei count = 1) const
```

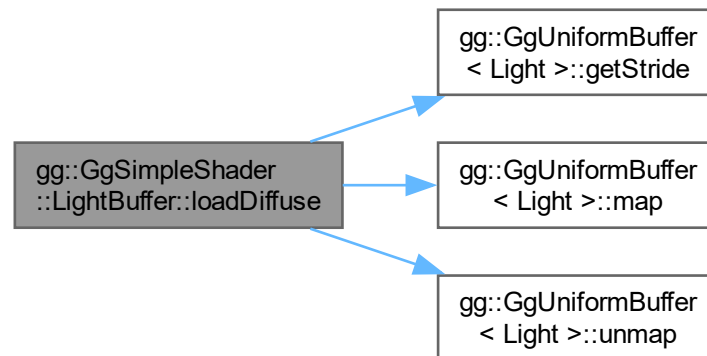
光源の強度の拡散反射光成分を設定する。

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5535 行目に定義があります。

呼び出し関係図:



8.34.3.8 loadDiffuse() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の拡散反射光成分を設定する.

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7591 行目に定義があります。

呼び出し関係図:



8.34.3.9 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
```

```

GLfloat b,
GLfloat a = 1.0f,
GLint first = 0,
GLsizei count = 1) const

```

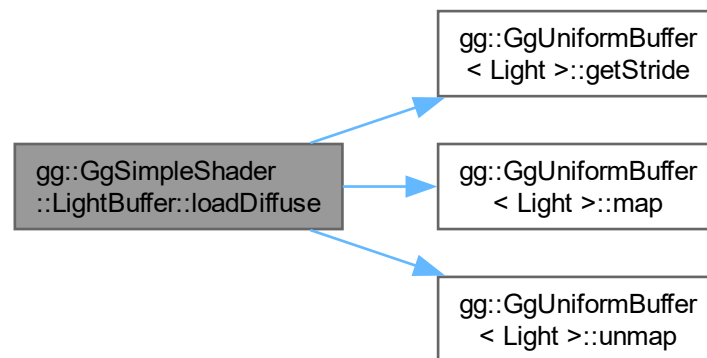
光源の強度の拡散反射光成分を設定する。

引数

<i>r</i>	光源の強度の拡散反射光成分の赤成分.
<i>g</i>	光源の強度の拡散反射光成分の緑成分.
<i>b</i>	光源の強度の拡散反射光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5509 行目に定義があります。

呼び出し関係図:



8.34.3.10 loadPosition() [1/4]

```

void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector & position,
    GLint first = 0,
    GLsizei count = 1) const

```

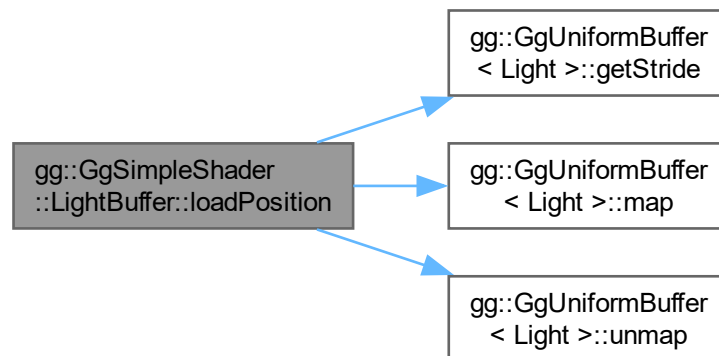
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5664 行目に定義があります。

呼び出し関係図:



8.34.3.11 loadPosition() [2/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector * position,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

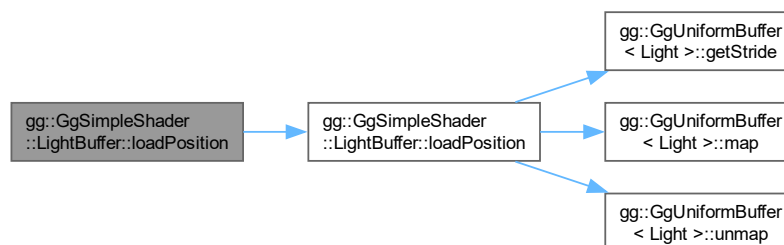
光源の位置を設定する.

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の配列.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7687 行目に定義があります。

呼び出し関係図:



8.34.3.12 loadPosition() [3/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
```



```
const GLfloat * position,
GLint first = 0,
GLsizei count = 1) const [inline]
```

光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7674 行目に定義があります。

呼び出し関係図:



8.34.3.13 loadPosition() [4/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

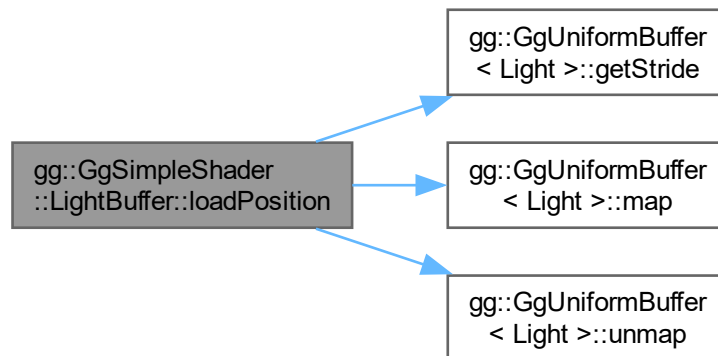
光源の位置を設定する。

引数

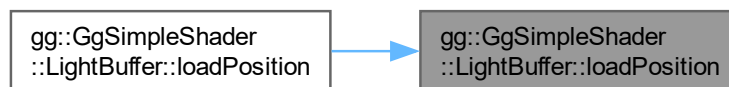
<i>x</i>	光源の位置の x 座標.
<i>y</i>	光源の位置の y 座標.
<i>z</i>	光源の位置の z 座標.
<i>w</i>	光源の位置の w 座標, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5638 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.34.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

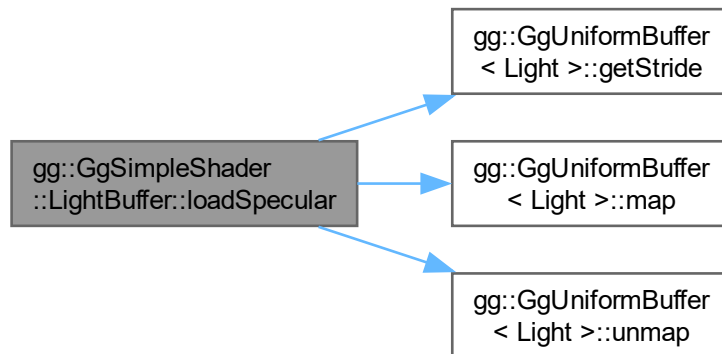
光源の強度の鏡面反射光成分を設定する.

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

`gg.cpp` の 5587 行目に定義があります。

呼び出し関係図:



8.34.3.15 loadSpecular() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の鏡面反射光成分を設定する。

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7628 行目に定義があります。

呼び出し関係図:



8.34.3.16 loadSpecular() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
```

```

GLfloat b,
GLfloat a = 1.0f,
GLint first = 0,
GLsizei count = 1) const

```

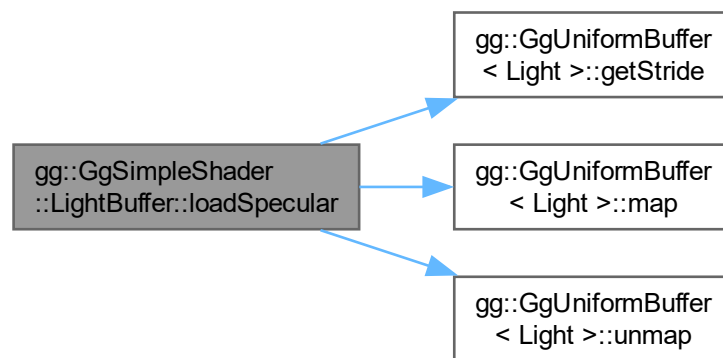
光源の強度の鏡面反射光成分を設定する。

引数

<i>r</i>	光源の強度の鏡面反射光成分の赤成分.
<i>g</i>	光源の強度の鏡面反射光成分の緑成分.
<i>b</i>	光源の強度の鏡面反射光成分の青成分.
<i>a</i>	光源の強度の鏡面反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5561 行目に定義があります。

呼び出し関係図:



8.34.3.17 select()

```

void gg::GgSimpleShader::LightBuffer::select (
    GLint i = 0) const [inline]

```

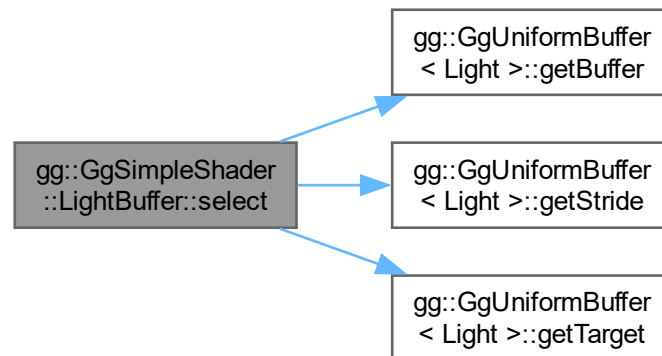
光源を選択する。

引数

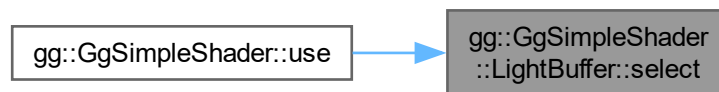
<i>i</i>	光源データの uniform block のインデックス.
----------	--------------------------------------

[gg.h](#) の 7721 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



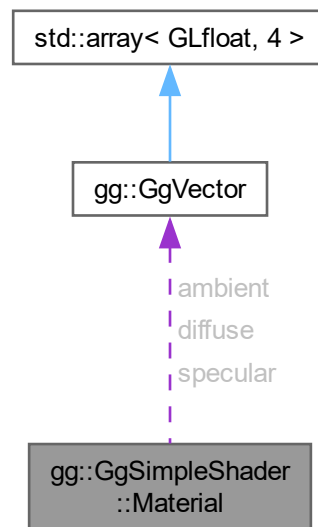
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.35 gg::GgSimpleShader::Material 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Material 連携図



公開変数類

- [GgVector ambient](#)
環境光に対する反射係数.
- [GgVector diffuse](#)
拡散反射係数.
- [GgVector specular](#)
鏡面反射係数.
- [GLfloat shininess](#)
輝き係数.

8.35.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データ.
[gg.h](#) の 7732 行目に定義があります。

8.35.2 メンバ詳解

8.35.2.1 ambient

[GgVector](#) gg::GgSimpleShader::Material::ambient
環境光に対する反射係数.
[gg.h](#) の 7734 行目に定義があります。

8.35.2.2 diffuse

[GgVector](#) gg::GgSimpleShader::Material::diffuse
拡散反射係数.
[gg.h](#) の 7735 行目に定義があります。

8.35.2.3 shininess

GLfloat gg::GgSimpleShader::Material::shininess
 輝き係数.

gg.h の 7737 行目に定義があります。

8.35.2.4 specular

GgVector gg::GgSimpleShader::Material::specular
 鏡面反射係数.

gg.h の 7736 行目に定義があります。

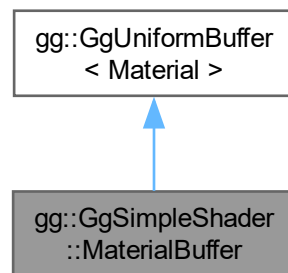
この構造体詳解は次のファイルから抽出されました:

- gg.h

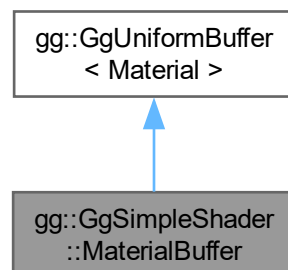
8.36 gg::GgSimpleShader::MaterialBuffer クラス

```
#include <gg.h>
```

gg::GgSimpleShader::MaterialBuffer の継承関係図



gg::GgSimpleShader::MaterialBuffer 連携図



公開メンバ関数

- **MaterialBuffer** (const **Material** *material=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)

- `MaterialBuffer` (const `Material` &material, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `MaterialBuffer` (`GgVector` ambient, `GgVector` diffuse, `GgVector` specular, GLfloat shininess, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual `~MaterialBuffer` ()
- void `loadAmbient` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const `GgVector` &ambient, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const `GgVector` &diffuse, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (const `GgVector` &color, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (const GLfloat *color, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const `GgVector` &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadShininess` (GLfloat shininess, GLint first=0, GLsizei count=1) const
- void `loadShininess` (const GLfloat *shininess, GLint first=0, GLsizei count=1) const
- void `load` (const `Material` *material, GLint first=0, GLsizei count=1) const
- void `load` (const `Material` &material, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Material >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const `Material` *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.36.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データのユニフォームバッファオブジェクト。`gg.h` の 7743 行目に定義があります。

8.36.2 構築子と解体子

8.36.2.1 MaterialBuffer() [1/3]

```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material * material = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

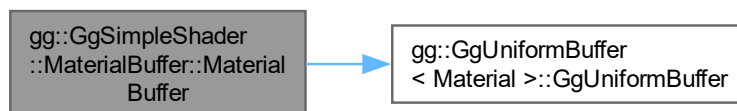
デフォルトコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7755 行目に定義があります。

呼び出し関係図:



8.36.2.2 MaterialBuffer() [2/3]

```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material & material,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

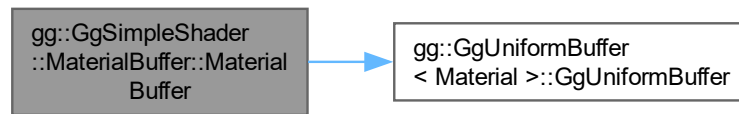
同じデータで埋めるコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7771 行目に定義があります。

呼び出し関係図:



8.36.2.3 MaterialBuffer() [3/3]

```

gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GLfloat shininess,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

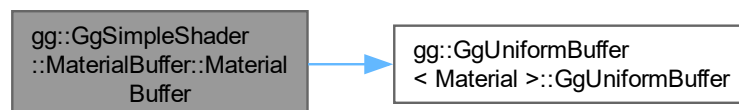
同じ引数で埋めるコンストラクタ.

引数

<i>ambient</i>	環境光に対する反射係数.
<i>diffuse</i>	拡散反射係数.
<i>specular</i>	鏡面反射係数.
<i>shininess</i>	輝き 係数.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7790 行目に定義があります。

呼び出し関係図:



8.36.2.4 ~MaterialBuffer()

```

virtual gg::GgSimpleShader::MaterialBuffer::~MaterialBuffer () [inline], [virtual]
デストラクタ.
  
```

[gg.h](#) の 7805 行目に定義があります。

8.36.3 関数詳解

8.36.3.1 load() [1/2]

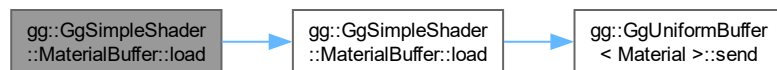
```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material & material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

材質を設定する.

引数

<i>material</i>	光源の特性の GgSimpleShader::Material 構造体.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 7990 行目に定義があります。
呼び出し関係図:



8.36.3.2 load() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material * material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

材質を設定する.

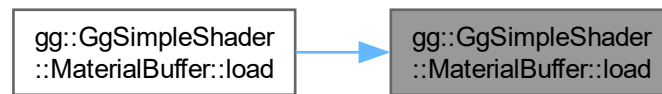
引数

<i>material</i>	光源の特性の GgSimpleShader::Material 構造体のポインタ.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 7978 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.36.3.3 loadAmbient() [1/3]

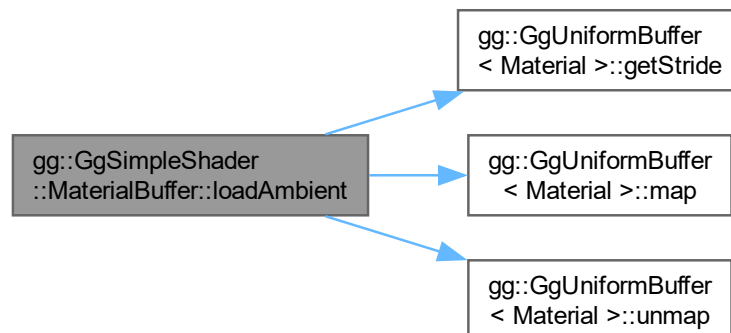
```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GgVector & ambient,
    GLint first = 0,
    GLsizei count = 1) const
```

三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5716 行目に定義があります。
呼び出し関係図:



8.36.3.4 loadAmbient() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

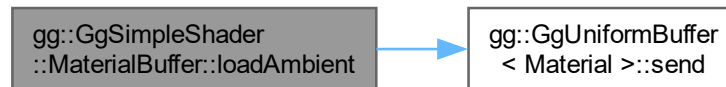
環境光に対する反射係数を設定する。

引数

<i>ambient</i>	環境光に対する反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.h の 7840 行目に定義があります。

呼び出し関係図:



8.36.3.5 loadAmbient() [3/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

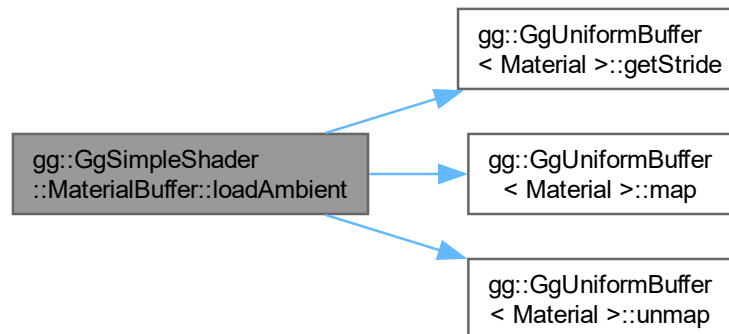
環境光に対する反射係数を設定する。

引数

<i>r</i>	環境光に対する反射係数の赤成分.
<i>g</i>	環境光に対する反射係数の緑成分.
<i>b</i>	環境光に対する反射係数の青成分.
<i>a</i>	環境光に対する反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5690 行目に定義があります。

呼び出し関係図:



8.36.3.6 loadAmbientAndDiffuse() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GgVector & color,
    GLint first = 0,
    GLsizei count = 1) const
```

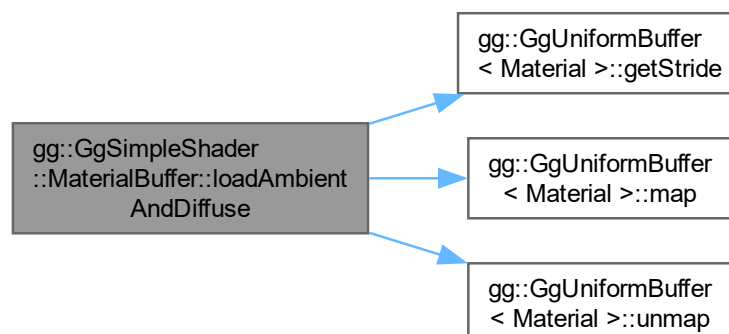
三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5820 行目に定義があります。

呼び出し関係図:



8.36.3.7 loadAmbientAndDiffuse() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GLfloat * color,
    GLint first = 0,
    GLsizei count = 1) const
```

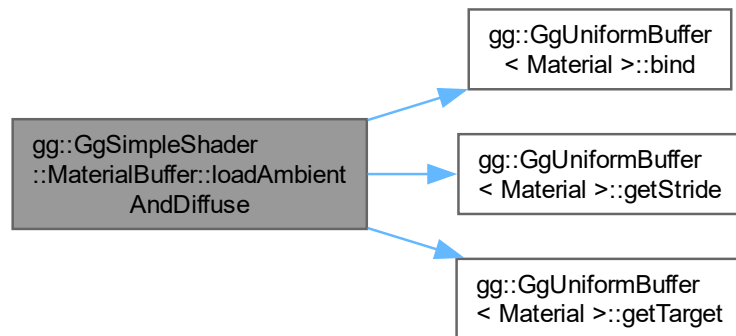
環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5843 行目に定義があります。

呼び出し関係図:



8.36.3.8 loadAmbientAndDiffuse() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

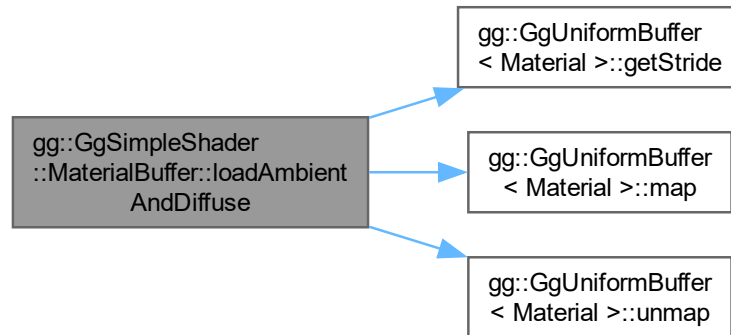
環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>r</i>	環境光に対する反射係数と拡散反射係数の赤成分.
<i>g</i>	環境光に対する反射係数と拡散反射係数の緑成分.
<i>b</i>	環境光に対する反射係数と拡散反射係数の青成分.
<i>a</i>	環境光に対する反射係数と拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.

<i>count</i>	値を設定する材質データの数, デフォルトは 1.
--------------	--------------------------

gg.cpp の 5794 行目に定義があります。
呼び出し関係図:



8.36.3.9 loadDiffuse() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GgVector & diffuse,
    GLint first = 0,
    GLsizei count = 1) const
```

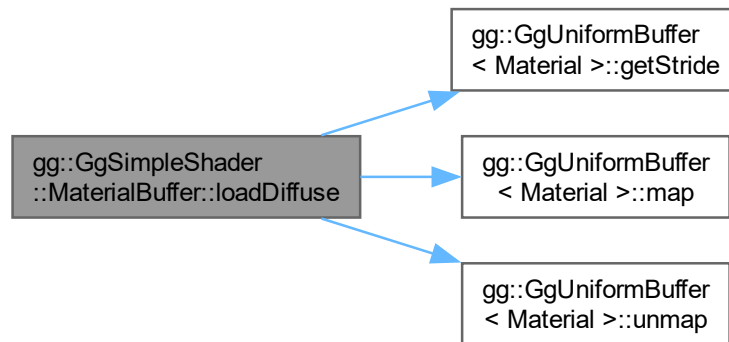
三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する。

引数

<i>ambient</i>	光源の強度の拡散反射光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5768 行目に定義があります。

呼び出し関係図:



8.36.3.10 loadDiffuse() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

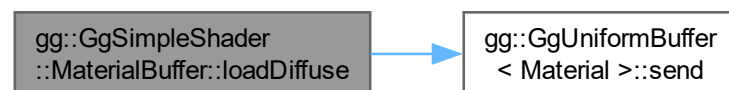
拡散反射係数を設定する。

引数

<i>diffuse</i>	拡散反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 7877 行目に定義があります。

呼び出し関係図:



8.36.3.11 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
```

```

GLfloat a = 1.0f,
GLint first = 0,
GLsizei count = 1) const

```

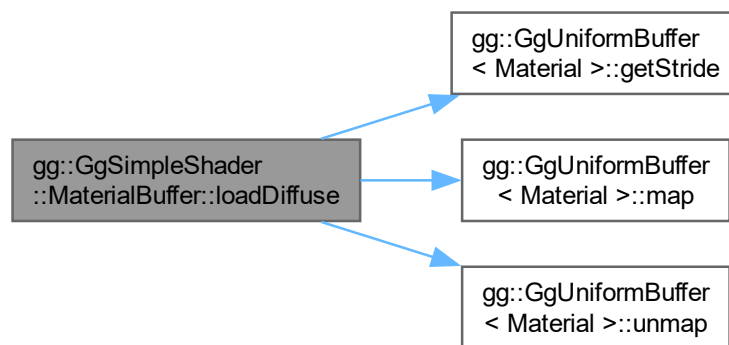
拡散反射係数を設定する。

引数

<i>r</i>	拡散反射係数の赤成分.
<i>g</i>	拡散反射係数の緑成分.
<i>b</i>	拡散反射係数の青成分.
<i>a</i>	拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5742 行目に定義があります。

呼び出し関係図:



8.36.3.12 loadShininess() [1/2]

```

void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    const GLfloat * shininess,
    GLint first = 0,
    GLsizei count = 1) const

```

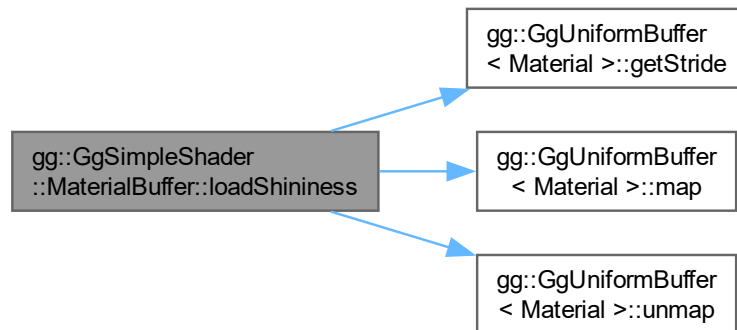
輝き係数を設定する。

引数

<i>shininess</i>	輝き係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5956 行目に定義があります。

呼び出し関係図:



8.36.3.13 loadShininess() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    GLfloat shininess,
    GLint first = 0,
    GLsizei count = 1) const
```

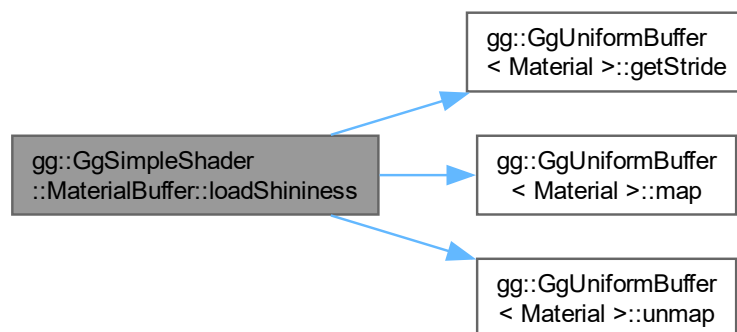
輝き係数を設定する.

引数

<i>shininess</i>	輝き係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5935 行目に定義があります。

呼び出し関係図:



8.36.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

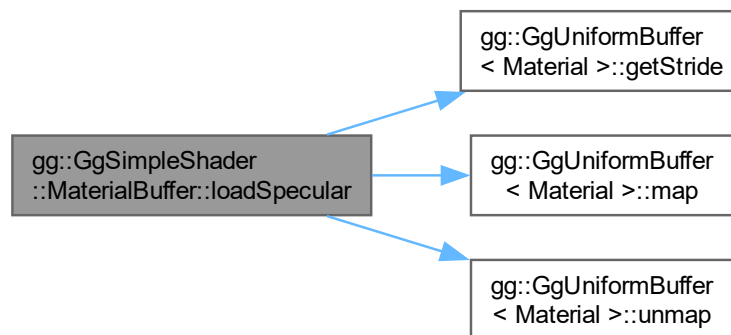
三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する。

引数

<i>ambient</i>	鏡面反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5912 行目に定義があります。

呼び出し関係図:



8.36.3.15 loadSpecular() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

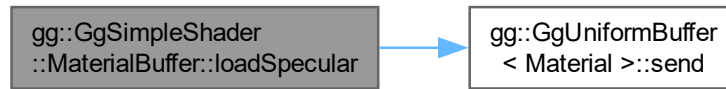
鏡面反射係数を設定する。

引数

<i>specular</i>	鏡面反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 7947 行目に定義があります。

呼び出し関係図:



8.36.3.16 loadSpecular() [3/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

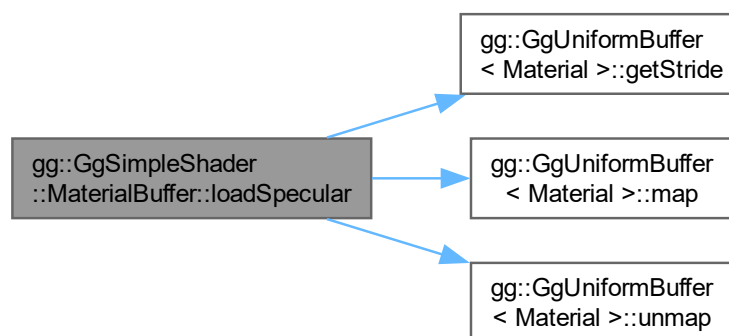
鏡面反射係数を設定する。

引数

<i>r</i>	鏡面反射係数の赤成分.
<i>g</i>	鏡面反射係数の緑成分.
<i>b</i>	鏡面反射係数の青成分.
<i>a</i>	鏡面反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5886 行目に定義があります。

呼び出し関係図:



8.36.3.17 select()

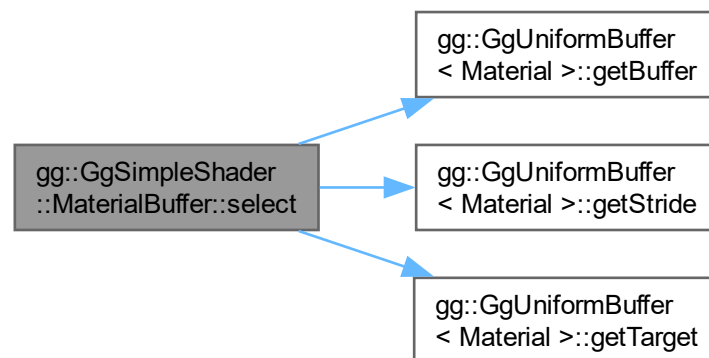
```
void gg::GgSimpleShader::MaterialBuffer::select (
    GLint i = 0) const [inline]
```

材質を選択する.

引数

<i>i</i>	材質データの uniform block のインデックス.
----------	-------------------------------

[gg.h](#) の 8000 行目に定義があります。
呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.37 Menu クラス

```
#include <Menu.h>
```

公開メンバ関数

- [Menu](#) (const [Config](#) &config, [Capture](#) &capture, [Calibration](#) &calibration)
- [Menu](#) (const [Menu](#) &menu)=delete
- virtual [~Menu](#) ()
- [Menu](#) & [operator=](#) (const [Menu](#) &menu)=delete
- [operator bool](#) () const
- const auto & [getPose](#) () const
- auto [getMenuubarHeight](#) () const
- void [setSize](#) (const std::array< int, 2 > &size)
- const auto & [getCheckerLength](#) () const
- auto [getMarkerLength](#) () const
- std::array< GLsizei, 2 > [setup](#) (GLfloat aspect) const
- void [draw](#) ()
- void [saveImage](#) (const cv::Mat &image, const std::string &filename="*.jpg") const

公開変数類

- bool `detectMarker`
ArUco Marker を検出するなら *true*
- bool `detectBoard`
ChArUco Board を検出するなら *true*

8.37.1 詳解

メニューの描画

`Menu.h` の 23 行目に定義があります。

8.37.2 構築子と解体子

8.37.2.1 `Menu()` [1/2]

```
Menu::Menu (
    const Config & config,
    Capture & capture,
    Calibration & calibration)
```

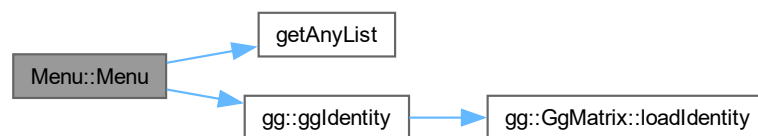
コンストラクタ

引数

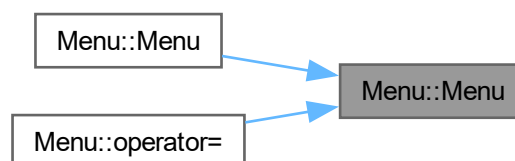
<i>config</i>	構成データ
<i>capture</i>	入力フレームを取得するキャプチャデバイス
<i>calibration</i>	校正オブジェクト

`Menu.cpp` の 460 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.37.2.2 Menu() [2/2]

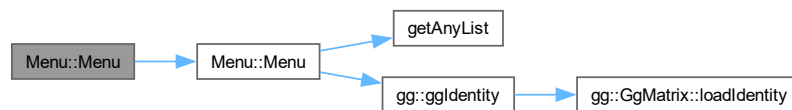
```
Menu::Menu (
    const Menu & menu) [delete]
```

コピーコンストラクタは使用しない

引数

<code>menu</code>	コピー元
-------------------	------

呼び出し関係図:



8.37.2.3 ~Menu()

```
Menu::~Menu () [virtual]
```

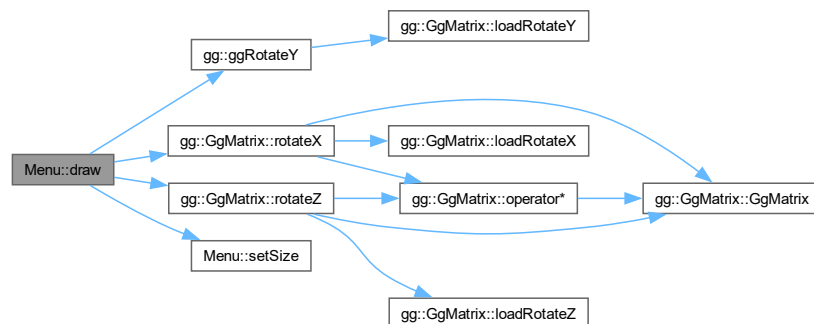
デストラクタ
[Menu.cpp](#) の 530 行目に定義があります。

8.37.3 関数詳解

8.37.3.1 draw()

```
void Menu::draw ()
```

メニューを描画する
[Menu.cpp](#) の 623 行目に定義があります。
 呼び出し関係図:



被呼び出し関係図:



8.37.3.2 getCheckerLength()

```
const auto & Menu::getCheckerLength () const [inline]
```

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さを得る

戻り値

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さ

[Menu.h](#) の 308 行目に定義があります。

8.37.3.3 getMarkerLength()

```
auto Menu::getMarkerLength () const [inline]
```

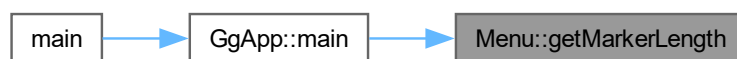
検出する ArUco Marker の一辺の長さを得る

戻り値

検出する ArUco Marker の一辺の長さ

[Menu.h](#) の 318 行目に定義があります。

被呼び出し関係図:



8.37.3.4 getMenubarHeight()

```
auto Menu::getMenubarHeight () const [inline]
```

メニューバーの高さを得る

戻り値

メニューバーの高さ

[Menu.h](#) の 286 行目に定義があります。
被呼び出し関係図:



8.37.3.5 getPose()

```
const auto & Menu::getPose () const [inline]
```

正規化デバイス座標系における焦点距離を求める

戻り値

図形の姿勢

[Menu.h](#) の 276 行目に定義があります。

8.37.3.6 operator bool()

```
Menu::operator bool () const [inline], [explicit]
```

処理を継続するかどうか調べる

戻り値

処理を継続するなら true

[Menu.h](#) の 266 行目に定義があります。

8.37.3.7 operator=()

```
Menu & Menu::operator= (
    const Menu & menu) [delete]
```

代入演算子を使用しない

引数

<i>menu</i>	代入元のメニュー
-------------	----------

戻り値

代入後のこのメニューの参照

呼び出し関係図:



8.37.3.8 saveImage()

```
void Menu::saveImage (
    const cv::Mat & image,
    const std::string & filename = "*.jpg") const
```

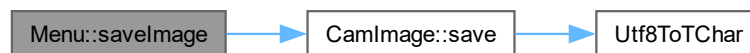
画像の保存

引数

<i>image</i>	保存する画像データ
<i>filename</i>	保存する画像ファイル名のテンプレート

[Menu.cpp](#) の 1175 行目に定義があります。

呼び出し関係図:



8.37.3.9 setSize()

```
void Menu::setSize (
    const std::array< int, 2 > & size)
```

解像度初期値を設定する

引数

<i>size</i>	キャプチャされるフレームの解像度
-------------	------------------

覚え書き

解像度と現在の焦点距離 *focal* をもとに、撮像系の縦横の画角の初期値も設定する。投影面の中心位置も設定する。

[Menu.cpp](#) の 545 行目に定義があります。
被呼び出し関係図:



8.37.3.10 setup()

```
std::array< GLsizei, 2 > Menu::setup (
    GLfloat aspect) const
```

シェーダを設定する

引数

<i>aspect</i>	表示領域の縦横比
---------------	----------

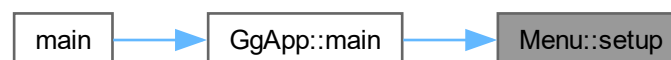
戻り値

描画すべきメッシュの横と縦の格子点数

覚え書き

格子点数は画角 *aspect* と展開用メッシュのサンプル点数 *samples* から求める。

[Menu.cpp](#) の 613 行目に定義があります。
被呼び出し関係図:



8.37.4 メンバ詳解

8.37.4.1 detectBoard

```
bool Menu::detectBoard
```

ChArUco Board を検出するなら true
[Menu.h](#) の 230 行目に定義があります。

8.37.4.2 detectMarker

```
bool Menu::detectMarker
```

ArUco Marker を検出するなら true

[Menu.h](#) の 227 行目に定義があります。
このクラス詳解は次のファイルから抽出されました:

- [Menu.h](#)
- [Menu.cpp](#)

8.38 Mesh クラス

```
#include <Mesh.h>
```

公開メンバ関数

- [Mesh\(\)](#)
- [Mesh\(const Mesh &mesh\)=delete](#)
- [virtual ~Mesh\(\)](#)
- [Mesh & operator= \(const Mesh &mesh\)=delete](#)
- [void draw \(const std::array< GLfloat, 2 > &scale, GLint unit=0\) const](#)
- [void drawMesh \(const std::array< GLsizei, 2 > &size\) const](#)

8.38.1 詳解

メッシュの描画クラス
[Mesh.h](#) の 20 行目に定義があります。

8.38.2 構築子と解体子

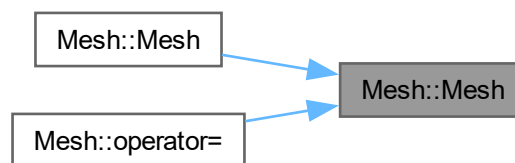
8.38.2.1 Mesh() [1/2]

```
Mesh::Mesh () [inline]
```

コンストラクタ

[Mesh.h](#) の 39 行目に定義があります。

被呼び出し関係図:



8.38.2.2 Mesh() [2/2]

```
Mesh::Mesh (
    const Mesh & mesh) [delete]
```

コピーコンストラクタは使用しない呼び出し関係図:



8.38.2.3 ~Mesh()

```
virtual Mesh::~Mesh () [inline], [virtual]
```

デストラクタ

[Mesh.h](#) の 60 行目に定義があります。

8.38.3 関数詳解

8.38.3.1 draw()

```
void Mesh::draw (
    const std::array< GLfloat, 2 > & scale,
    GLint unit = 0) const [inline]
```

シェーダを指定して矩形を描画する

引数

<i>scale</i>	マッピングするテクスチャのスケール
<i>unit</i>	使用するテクスチャユニット

覚え書き

この前にテクスチャユニット `GL_TEXTURE0 + unit` を指定し、テクスチャを結合しておく必要がある。

[Mesh.h](#) の 84 行目に定義があります。

8.38.3.2 drawMesh()

```
void Mesh::drawMesh (
    const std::array< GLsizei, 2 > & size) const [inline]
```

メッシュを描画する

引数

<i>size</i>	描画するメッシュの横と縦の格子点数
-------------	-------------------

覚え書き

この前に別途シェーダを指定し、メッシュの格子点数を得ておく必要がある。

[Mesh.h](#) の 116 行目に定義があります。

8.38.3.3 operator=()

```
Mesh & Mesh::operator= (
    const Mesh & mesh) [delete]
```

代入演算子は使用しない

引数

<i>menu</i>	代入元のオブジェクト
-------------	------------

戻り値

代入後のこのオブジェクトの参照

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Mesh.h](#)

8.39 Preference クラス

```
#include <Preference.h>
```

公開メンバ関数

- [Preference](#) ()
- [Preference](#) (const std::string &description, const std::string &vert, const std::string &frag, const [Intrinsics](#) &intrinsics=[Intrinsics](#){})
- [Preference](#) (const picojson::object &object)
- virtual ~[Preference](#) ()
- void [buildShader](#) ()
- const auto & [getDescription](#) () const
- const auto & [getIntrinsics](#) () const
- const auto & [getShader](#) () const
- void [setPreference](#) (picojson::object &object) const

8.39.1 詳解

キャプチャデバイスの構成データ

キャプチャデバイスごとの特性を記述する. キャプチャデバイス固有のパラメータはカメラの内部パラメータなどである. キャプチャデバイスの投影方式は平面展開用のシェードとして記述する.

[Preference.h](#) の 24 行目に定義があります.

8.39.2 構築子と解体子

8.39.2.1 Preference() [1/3]

`Preference::Preference ()`

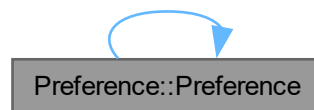
キャプチャデバイスの構成データのデフォルトコンストラクタ

覚え書き

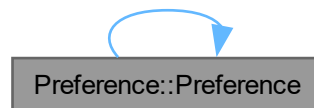
構成ファイルが読めなかったときにしか使わない。

`Preference.cpp` の 13 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.39.2.2 Preference() [2/3]

```

Preference::Preference (
    const std::string & description,
    const std::string & vert,
    const std::string & frag,
    const Intrinsic & intrinsics = Intrinsic{})
  
```

シェーダのソースファイルを指定するときに使う キャプチャデバイスの構成データのコンストラクタ

引数

<i>description</i>	このキャプチャデバイスの説明の文字列
<i>vert</i>	展開用のバーテックスシェーダのソースファイル名
<i>frag</i>	展開用のフラグメントシェーダのソースファイル名
<i>intrinsics</i>	キャプチャデバイス固有のパラメータ

`Preference.cpp` の 22 行目に定義があります。

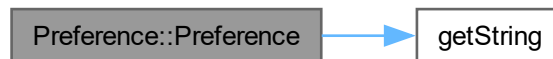
8.39.2.3 Preference() [3/3]

```
Preference::Preference (
    const picojson::object & object)
```

構成ファイルを読み込むときに使う キャプチャデバイスの構成データのコンストラクタ

[Preference.cpp](#) の 36 行目に定義があります。

呼び出し関係図:



8.39.2.4 ~Preference()

```
Preference::~~Preference () [virtual]
```

デストラクタ

[Preference.cpp](#) の 50 行目に定義があります。

8.39.3 関数詳解

8.39.3.1 buildShader()

```
void Preference::buildShader ()
```

展開用のシェーダをビルドする

[Preference.cpp](#) の 59 行目に定義があります。

8.39.3.2 getDescription()

```
const auto & Preference::getDescription () const [inline]
```

説明の文字列を取り出す

戻り値

構成の説明文

[Preference.h](#) の 84 行目に定義があります。

8.39.3.3 getIntrinsics()

```
const auto & Preference::getIntrinsics () const [inline]
```

キャプチャデバイス固有のパラメータを取り出す

戻り値

キャプチャデバイス固有のパラメータ

[Preference.h](#) の 94 行目に定義があります。

8.39.3.4 getShader()

```
const auto & Preference::getShader () const [inline]
```

シェーダを取り出す

戻り値

この構成のシェーダの参照

[Preference.h](#) の 104 行目に定義があります。

8.39.3.5 setPreference()

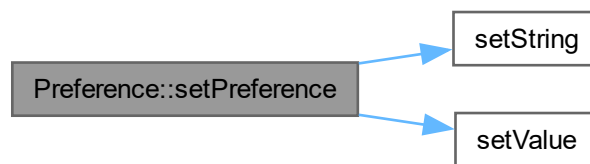
```
void Preference::setPreference (
    picojson::object & object) const
```

JSON オブジェクトに構成を格納する

引数

<i>object</i>	格納先の JSON オブジェクト
---------------	------------------

[Preference.cpp](#) の 71 行目に定義があります。
呼び出し関係図:



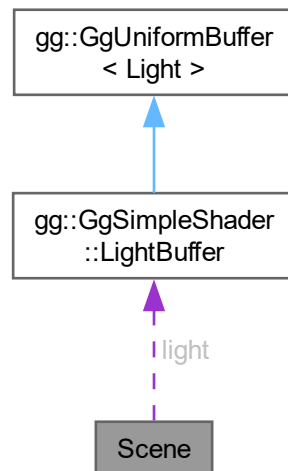
このクラス詳解は次のファイルから抽出されました:

- [Preference.h](#)
- [Preference.cpp](#)

8.40 Scene クラス

```
#include <Scene.h>
```

Scene 連携図



公開メンバ関数

- `Scene()`
- `virtual ~Scene()`
- `void set(const std::string &filename)`
- `void draw(const GgMatrix &mp, const GgMatrix &mv) const`

公開変数類

- `GgSimpleShader::LightBuffer light`

8.40.1 詳解

シーンの描画クラス

`Scene.h` の 18 行目に定義があります。

8.40.2 構築子と解体子

8.40.2.1 Scene()

`Scene::Scene()`

コンストラクタ

`Scene.cpp` の 25 行目に定義があります。

8.40.2.2 ~Scene()

`Scene::~Scene()` [virtual]

デストラクタ

`Scene.cpp` の 40 行目に定義があります。

8.40.3 関数詳解

8.40.3.1 draw()

```
void Scene::draw (
    const GgMatrix & mp,
    const GgMatrix & mv) const
```

シーンを描画する

引数

<i>mp</i>	投影変換行列
<i>mv</i>	モデルビュー変換行列

[Scene.cpp](#) の 55 行目に定義があります。

8.40.3.2 set()

```
void Scene::set (
    const std::string & filename)
```

モデルを選択する

引数

<i>filename</i>	Wavefront OBJ 形式の形状データファイルのファイル名
-----------------	----------------------------------

[Scene.cpp](#) の 47 行目に定義があります。

8.40.4 メンバ詳解

8.40.4.1 light

```
GgSimpleShader::LightBuffer Scene::light
```

[Scene.h](#) の 29 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Scene.h](#)
- [Scene.cpp](#)

8.41 Settings 構造体

```
#include <Config.h>
```

公開メンバ関数

- [Settings](#) (const std::string &[dictionaryName](#))
- auto [getFocal](#) () const

公開変数類

- int [samples](#)
展開に用いるメッシュのサンプル数
- std::array< float, 3 > [euler](#)
キャプチャデバイスの姿勢のオイラー角
- float [focal](#)
描画時の焦点距離

- `std::array< float, 2 >` `focalRange`
描画時の焦点距離の範囲
- `std::string` `dictionaryName`
使用中の *ArUco Marker* 辞書名
- `std::array< float, 2 >` `checkerLength`
検出する *ChArUco Board* のマス目一辺の長さ と *ArUco Marker* の一辺の長さ (単位 *cm*)
- `float` `markerLength`
検出する *ArUco Marker* の一辺の長さ (単位 *cm*)

静的公開変数類

- `static constexpr decltype(euler) defaultEuler { 0.0f, 0.0f, 0.0f }`
キャプチャデバイスの姿勢のデフォルト値
- `static constexpr decltype(focal) defaultFocal { 50.0f }`
展開時の焦点距離のデフォルト値
- `static constexpr decltype(focalRange) defaultFocalRange { 10.0f, 200.0f }`
描画時の焦点距離の範囲のデフォルト値

8.41.1 詳解

表示関連の設定データ

[Config.h](#) の 17 行目に定義があります。

8.41.2 構築子と解体子

8.41.2.1 Settings()

```
Settings::Settings (
    const std::string & dictionaryName) [inline]
```

コンストラクタ

[Config.h](#) の 52 行目に定義があります。

8.41.3 関数詳解

8.41.3.1 getFocal()

```
auto Settings::getFocal () const [inline]
```

正規化デバイス座標系における焦点距離を求める

[Config.h](#) の 65 行目に定義があります。

8.41.4 メンバ詳解

8.41.4.1 checkerLength

```
std::array<float, 2> Settings::checkerLength
```

検出する *ChArUco Board* のマス目一辺の長さ と *ArUco Marker* の一辺の長さ (単位 *cm*)

[Config.h](#) の 44 行目に定義があります。

8.41.4.2 defaultEuler

```
decltype(euler) Settings::defaultEuler { 0.0f, 0.0f, 0.0f } [static], [constexpr]
```

キャプチャデバイスの姿勢のデフォルト値

[Config.h](#) の 26 行目に定義があります。

8.41.4.3 defaultFocal

```
decltype(focal) Settings::defaultFocal { 50.0f } [static], [constexpr]
```

展開時の焦点距離のデフォルト値

[Config.h](#) の 32 行目に定義があります。

8.41.4.4 defaultFocalRange

```
decltype(focalRange) Settings::defaultFocalRange { 10.0f, 200.0f } [static], [constexpr]
```

描画時の焦点距離の範囲のデフォルト値

[Config.h](#) の 38 行目に定義があります。

8.41.4.5 dictionaryName

```
std::string Settings::dictionaryName
```

使用中の ArUco Marker 辞書名

[Config.h](#) の 41 行目に定義があります。

8.41.4.6 euler

```
std::array<float, 3> Settings::euler
```

キャプチャデバイスの姿勢のオイラー角

[Config.h](#) の 23 行目に定義があります。

8.41.4.7 focal

```
float Settings::focal
```

描画時の焦点距離

[Config.h](#) の 29 行目に定義があります。

8.41.4.8 focalRange

```
std::array<float, 2> Settings::focalRange
```

描画時の焦点距離の範囲

[Config.h](#) の 35 行目に定義があります。

8.41.4.9 markerLength

```
float Settings::markerLength
```

検出する ArUco Marker の一辺の長さ (単位 cm)

[Config.h](#) の 47 行目に定義があります。

8.41.4.10 samples

```
int Settings::samples
```

展開に用いるメッシュのサンプル数

[Config.h](#) の 20 行目に定義があります。

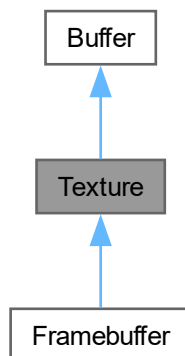
この構造体詳解は次のファイルから抽出されました:

- [Config.h](#)

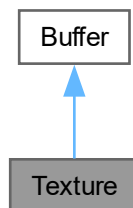
8.42 Texture クラス

```
#include <Texture.h>
```

Texture の継承関係図



Texture 連携図



公開メンバ関数

- `Texture ()`
- `Texture (GLsizei width, GLsizei height, int channels)`
- `Texture (const Texture &texture)`
- `Texture (Texture &&texture) noexcept`
- `virtual ~Texture ()`
- `Texture & operator= (const Texture &texture)`
- `Texture & operator= (Texture &&texture) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `virtual void copy (const Buffer &texture) noexcept`
- `virtual void discard ()`
- `auto getTextureName () const`
- `virtual const std::array< GLsizei, 2 > & getSize () const`
- `virtual int getChannels () const`
- `void bindTexture (int unit=0) const`
- `void unbindTexture () const`
- `void draw (GLsizei width, GLsizei height, int unit=0) const`
- `void readPixels (GLuint buffer) const`

- void `readPixels` (`Buffer` &buffer) const
- void `readPixels` () const
- void `drawPixels` (GLuint buffer) const
- void `drawPixels` (const `Texture` &texture)
- void `drawPixels` (const `Buffer` &buffer)
- void `drawPixels` () const

基底クラス `Buffer` に属する継承公開メンバ関数

- `Buffer` ()
- `Buffer` (GLsizei width, GLsizei height, int channels)
- `Buffer` (const `Buffer` &buffer)
- `Buffer` (`Buffer` &&buffer) noexcept
- virtual `~Buffer` ()
- `Buffer` & `operator=` (const `Buffer` &buffer)
- `Buffer` & `operator=` (`Buffer` &&buffer) noexcept
- void `resize` (GLsizei width, GLsizei height, int channels)
- void `resize` (const `Buffer` &buffer)
- auto `getBufferName` () const
- GLsizei `getWidth` () const
- GLsizei `getHeight` () const
- auto `getFormat` () const
- auto `getAspect` () const
- void `bindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void `unbindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * `map` () const
- void `unmap` () const

静的限定公開変数類

- static std::shared_ptr< `Mesh` > `mesh`
テクスチャ展開に用いるメッシュの頂点配列オブジェクト

その他の継承メンバ

基底クラス `Buffer` に属する継承限定公開メンバ関数

- auto `channelsToFormat` (int channels) const
- int `formatToChannels` (GLenum format) const
- void `copyBuffer` (const `Buffer` &buffer) noexcept

8.42.1 詳解

テクスチャクラス

`Texture.h` の 20 行目に定義があります。

8.42.2 構築子と解体子

8.42.2.1 `Texture()` [1/4]

`Texture::Texture () [inline]`

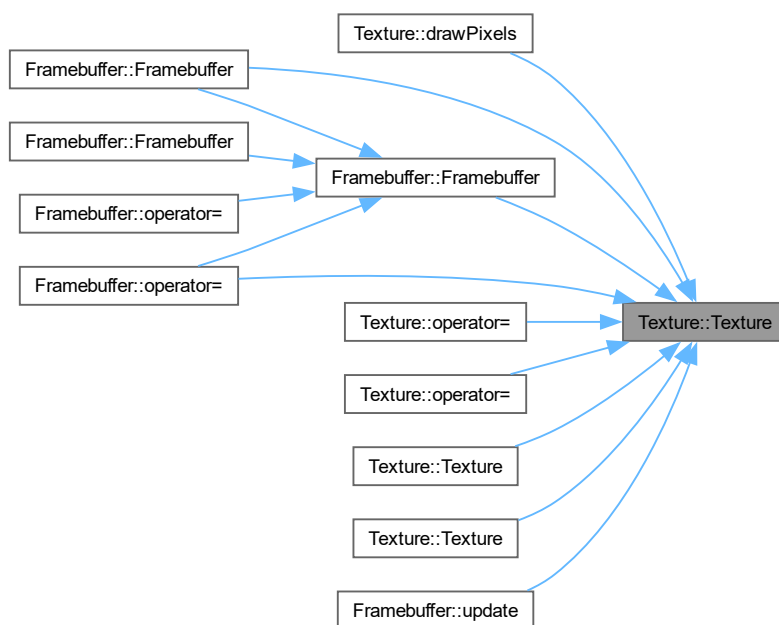
テクスチャのデフォルトコンストラクタ

`Texture.h` の 41 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.2.2 Texture() [2/4]

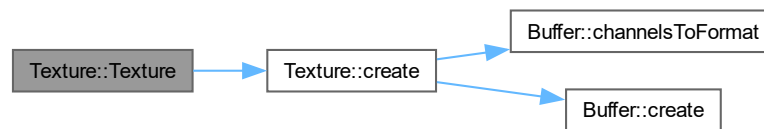
```
Texture::Texture (
    GLsizei width,
    GLsizei height,
    int channels)
テクスチャを作成するコンストラクタ
```

引数

<i>width</i>	作成するテクスチャの横の画素数
<i>height</i>	作成するテクスチャの縦の画素数
<i>channels</i>	作成するテクスチャのチャンネル数

Texture.cpp の 13 行目に定義があります。

呼び出し関係図:



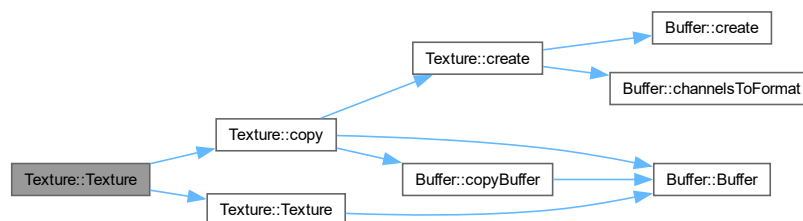
8.42.2.3 Texture() [3/4]

```
Texture::Texture (
    const Texture & texture)
コピーコンストラクタ
```

引数

<i>texture</i>	コピー元のテクスチャ
----------------	------------

[Texture.cpp](#) の 22 行目に定義があります。
呼び出し関係図:



8.42.2.4 Texture() [4/4]

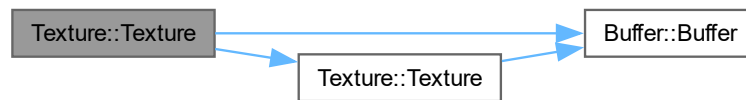
```
Texture::Texture (
    Texture && texture) [inline], [noexcept]
ムーブコンストラクタ
```

引数

<i>texture</i>	ムーブ元のテクスチャ
----------------	------------

[Texture.h](#) の 70 行目に定義があります。

呼び出し関係図:



8.42.2.5 ~Texture()

```
Texture::~Texture () [virtual]
```

デストラクタ

[Texture.cpp](#) の 31 行目に定義があります。

呼び出し関係図:



8.42.3 関数詳解

8.42.3.1 bindTexture()

```
void Texture::bindTexture (
    int unit = 0) const [inline]
```

テクスチャユニットを指定してテクスチャを結合する

引数

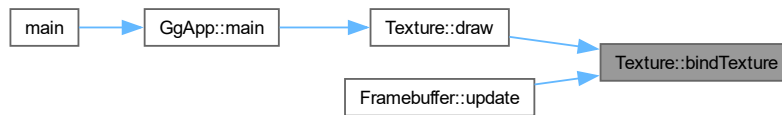
<i>unit</i>	テクスチャユニット番号
-------------	-------------

覚え書き

`unit` にはシェーダにおいてテクスチャのサンプラの `uniform` 変数に設定する `GL_TEXTUREi` の `i` と一致させること。

`Texture.h` の 171 行目に定義があります。

被呼び出し関係図:



8.42.3.2 copy()

```
void Texture::copy (
    const Buffer & texture) [virtual], [noexcept]
```

テクスチャをコピーする

引数

<code>texture</code>	コピー元のテクスチャ
----------------------	------------

覚え書き

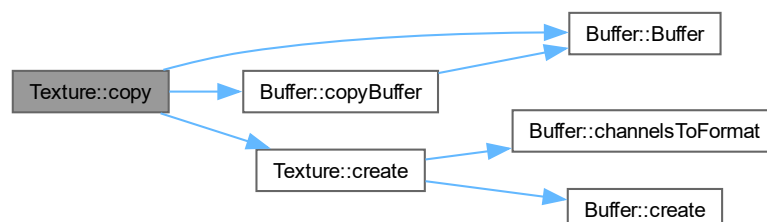
このテクスチャのサイズがコピー元と異なれば、このテクスチャを削除して新しいテクスチャを作り直してコピーする。引数の型が `Buffer` なのは、このオブジェクトの型が `Buffer` のとき `Buffer::copy()`、`Texture` のとき `Texture::copy()`、`Framebuffer` のとき `Framebuffer::copy()` を呼ぶようにするため。

`Buffer` を再実装しています。

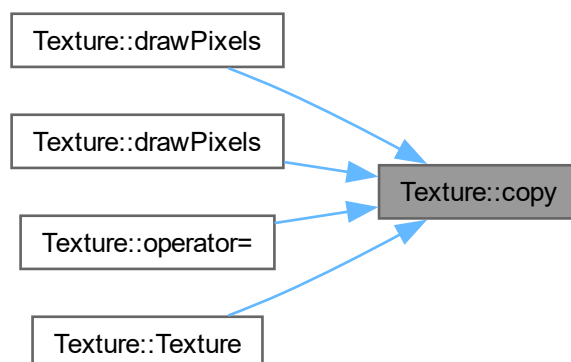
`Framebuffer` で再実装されています。

`Texture.cpp` の 121 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.3 create()

```
void Texture::create (  
    GLsizei width,  
    GLsizei height,  
    int channels) [virtual]
```

テクスチャを作成する

引数

<i>width</i>	作成するテクスチャの横の画素数
<i>height</i>	作成するテクスチャの縦の画素数
<i>channels</i>	作成するテクスチャのチャンネル数

覚え書き

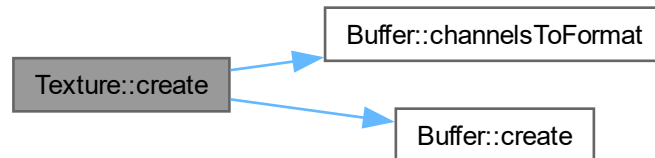
このテクスチャのサイズが引数で指定したサイズと異なれば、このテクスチャを削除して新しいテクスチャを作り直す。

[Buffer](#)を再実装しています。

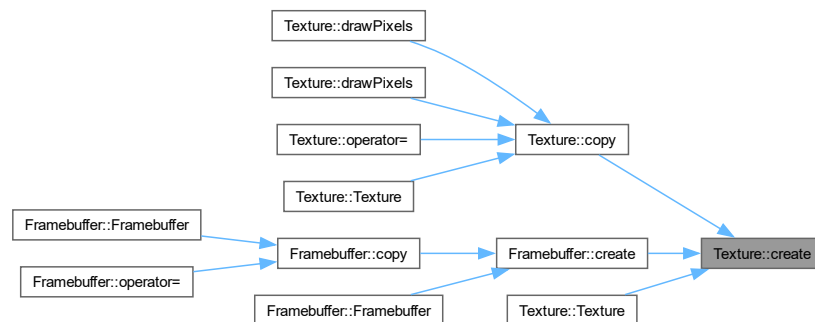
[Framebuffer](#)で再実装されています。

[Texture.cpp](#) の 85 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.4 discard()

```
void Texture::discard () [virtual]
```

テクスチャを破棄する

[Buffer](#)を再実装しています。

[Framebuffer](#)で再実装されています。

[Texture.cpp](#) の 134 行目に定義があります。

被呼び出し関係図:



8.42.3.5 draw()

```
void Texture::draw (
    GLsizei width,
    GLsizei height,
    int unit = 0) const
```

このテクスチャをマッピングして矩形を描画する

引数

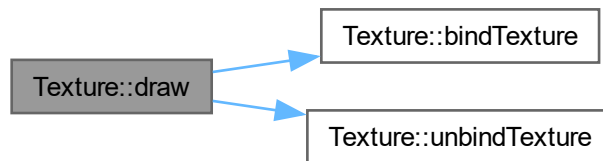
<i>width</i>	表示領域の横の画素数
<i>height</i>	表示領域の縦の画素数
<i>unit</i>	使用するテクスチャユニット番号

覚え書き

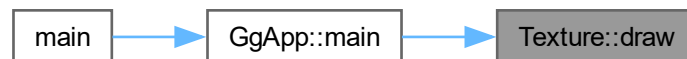
フレームバッファ全体を覆うメッシュを描画する。unit にはシェードにおいてテクスチャのサンプルの uniform 変数に設定する GL_TEXTURE*i* の *i* と一致させること。

[Texture.cpp](#) の 151 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



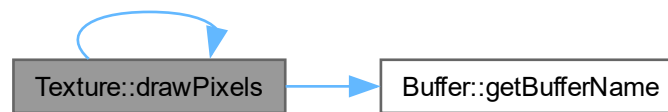
8.42.3.6 drawPixels() [1/4]

```
void Texture::drawPixels () const [inline]
```

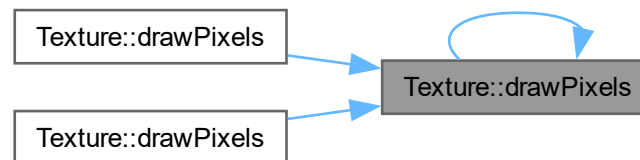
ピクセルバッファオブジェクトからテクスチャにデータをコピーする

[Texture.h](#) の 307 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.7 drawPixels() [2/4]

```
void Texture::drawPixels (
    const Buffer & buffer) [inline]
```

指定したオブジェクトからテクスチャにデータをコピーする

引数

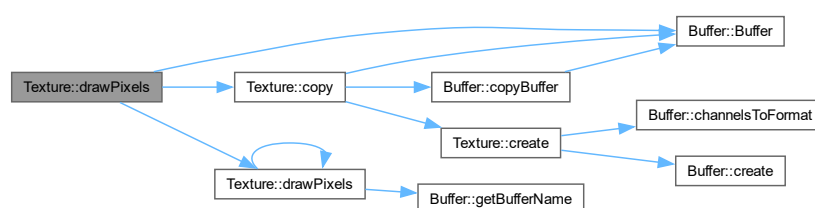
<i>buffer</i>	テクスチャをコピーする先のオブジェクト
---------------	---------------------

覚え書き

テクスチャのサイズをコピー元のオブジェクトのバッファに合わせる。

[Texture.h](#) の 295 行目に定義があります。

呼び出し関係図:



8.42.3.8 drawPixels() [3/4]

```
void Texture::drawPixels (
    const Texture & texture) [inline]
```

指定したテクスチャからからデータをコピーする

引数

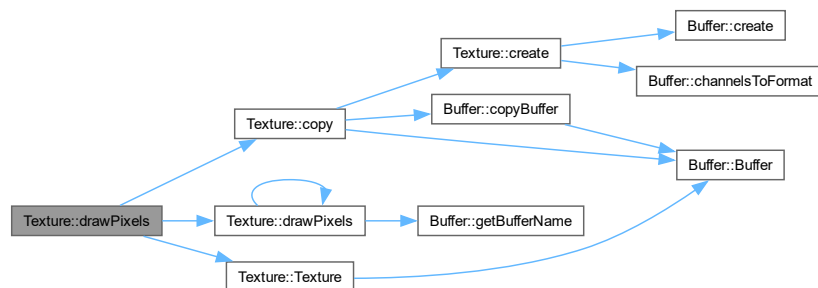
<i>texture</i>	コピー元のテクスチャ
----------------	------------

覚え書き

テクスチャのサイズをコピー元のテクスチャに合わせる。

[Texture.h](#) の 278 行目に定義があります。

呼び出し関係図:



8.42.3.9 drawPixels() [4/4]

```
void Texture::drawPixels (
    GLuint buffer) const
```

指定したピクセルバッファオブジェクトからテクスチャにデータをコピーする

引数

<i>buffer</i>	コピーするテクスチャを格納したピクセルバッファオブジェクト名
---------------	--------------------------------

覚え書き

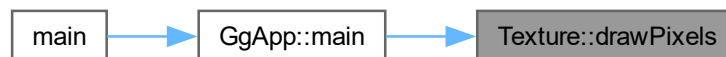
コピーするデータのサイズが一致している必要がある。

[Texture.cpp](#) の 237 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.10 getChannels()

```
virtual int Texture::getChannels () const [inline], [virtual]
```

テクスチャのチャンネル数を得る

[Buffer](#)を再実装しています。

[Framebuffer](#)で再実装されています。

[Texture.h](#) の 157 行目に定義があります。

8.42.3.11 getSize()

```
virtual const std::array< GLsizei, 2 > & Texture::getSize () const [inline], [virtual]
```

テクスチャのサイズを得る

[Buffer](#)を再実装しています。

[Framebuffer](#)で再実装されています。

[Texture.h](#) の 149 行目に定義があります。

8.42.3.12 getTextureName()

```
auto Texture::getTextureName () const [inline]
```

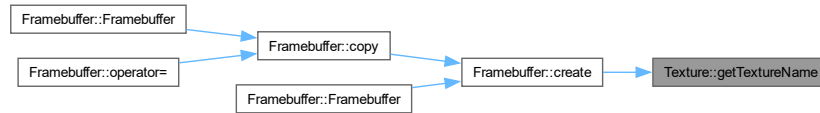
テクスチャ名を得る

戻り値

テクスチャ名

[Texture.h](#) の 141 行目に定義があります。

被呼び出し関係図:



8.42.3.13 operator=() [1/2]

```
Texture & Texture::operator= (
    const Texture & texture)
```

代入演算子

引数

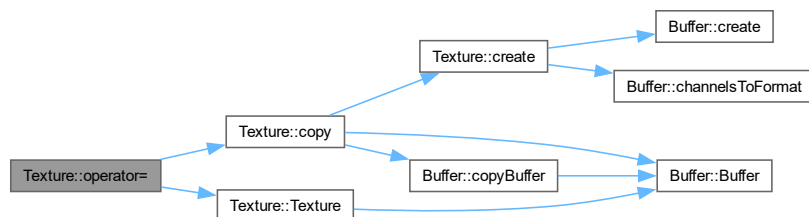
<i>texture</i>	代入元のテクスチャ
----------------	-----------

戻り値

代入結果のテクスチャ

[Texture.cpp](#) の 40 行目に定義があります。

呼び出し関係図:



8.42.3.14 operator=() [2/2]

```
Texture & Texture::operator= (
    Texture && texture) [noexcept]
```

ムーブ代入演算子

引数

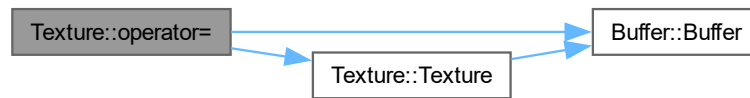
<i>texture</i>	ムーブ代入元のテクスチャ
----------------	--------------

戻り値

ムーブ代入結果のテクスチャ

[Texture.cpp](#) の 55 行目に定義があります。

呼び出し関係図:



8.42.3.15 readPixels() [1/3]

```
void Texture::readPixels () const [inline]
```

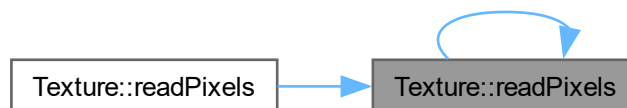
テクスチャからピクセルバッファオブジェクトにデータをコピーする

[Texture.h](#) の 245 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.16 readPixels() [2/3]

```
void Texture::readPixels (
    Buffer & buffer) const [inline]
```

テクスチャから指定したオブジェクトのバッファにデータをコピーする

引数

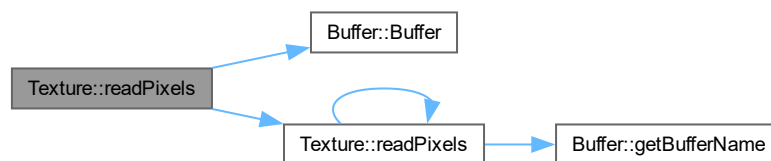
<i>buffer</i>	テクスチャをコピーする先のオブジェクト
---------------	---------------------

覚え書き

コピーする先のオブジェクトのサイズをテクスチャに合わせる。

[Texture.h](#) の 230 行目に定義があります。

呼び出し関係図:



8.42.3.17 readPixels() [3/3]

```
void Texture::readPixels (
    GLuint buffer) const
```

テクスチャから指定したピクセルバッファオブジェクトにデータをコピーする

引数

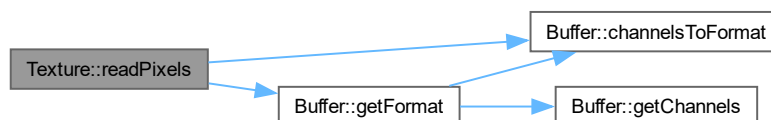
<i>buffer</i>	テクスチャをコピーする先のピクセルバッファオブジェクト名
---------------	------------------------------

覚え書き

コピーするデータのサイズが一致している必要がある。

[Texture.cpp](#) の 178 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



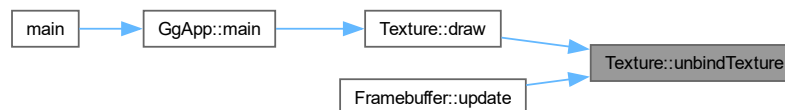
8.42.3.18 unbindTexture()

```
void Texture::unbindTexture () const [inline]
```

テクスチャの結合を解除する

[Texture.h](#) の 183 行目に定義があります。

被呼び出し関係図:



8.42.4 メンバ詳解

8.42.4.1 mesh

```
std::shared_ptr< Mesh > Texture::mesh [static], [protected]
```

テクスチャ展開に用いるメッシュの頂点配列オブジェクト

[Texture.h](#) の 34 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Texture.h](#)
- [Texture.cpp](#)

8.43 GgApp::Window クラス

```
#include <GgApp.h>
```

公開メンバ関数

- [Window](#) (const std::string &title="GLFW Window", int width=640, int height=480, int fullscreen=0, GLFWwindow *share=nullptr)
- [Window](#) (const Window &w)=delete
- [Window](#) (Window &&w)=default
- virtual [~Window](#) ()
- [Window](#) & operator= (const Window &w)=delete
- [Window](#) & operator= (Window &&w)=default
- auto * [get](#) () const
- void [setClose](#) (int flag=GLFW_TRUE) const

- bool `shouldClose` () const
- operator bool ()
- void `swapBuffers` () const
- void `restoreViewport` () const
- void `updateViewport` ()
- auto `getWidth` () const
- auto `getHeight` () const
- auto `getFboWidth` () const
- auto `getFboHeight` () const
- const auto & `getSize` () const
- void `getSize` (GLsizei *size) const
- const auto & `getFboSize` () const
- void `getFboSize` (GLsizei *fboSize) const
- auto `getAspect` () const
- bool `getKey` (int key) const
- void `selectInterface` (int no)
- void `setVelocity` (GLfloat vx, GLfloat vy, GLfloat vz=0.1f)
- int `getLastKey` ()
- auto `getArrow` (int direction=0, int mods=0) const
- auto `getArrowX` (int mods=0) const
- auto `getArrowY` (int mods=0) const
- void `getArrow` (GLfloat *arrow, int mods=0) const
- auto `getShiftArrowX` () const
- auto `getShiftArrowY` () const
- void `getShiftArrow` (GLfloat *shift_arrow) const
- auto `getControlArrowX` () const
- auto `getControlArrowY` () const
- void `getControlArrow` (GLfloat *control_arrow) const
- auto `getAltArrowX` () const
- auto `getAltArrowY` () const
- void `getAltArrow` (GLfloat *alt_arrow) const
- const auto * `getMouse` () const
- void `getMouse` (GLfloat *position) const
- auto `getMouse` (int direction) const
- auto `getMouseX` () const
- auto `getMouseY` () const
- const auto * `getWheel` () const
- void `getWheel` (GLfloat *rotation) const
- auto `getWheel` (int direction) const
- auto `getWheelX` () const
- auto `getWheelY` () const
- const auto & `getTranslation` (int button=GLFW_MOUSE_BUTTON_1) const
- auto `getTranslationMatrix` (int button=GLFW_MOUSE_BUTTON_1) const
- auto `getScrollMatrix` (int button=GLFW_MOUSE_BUTTON_1) const
- auto `getRotation` (int button=GLFW_MOUSE_BUTTON_1) const
- auto `getRotationMatrix` (int button=GLFW_MOUSE_BUTTON_1) const
- void `resetRotation` ()
- void `resetTranslation` ()
- void `reset` ()
- void * `getUserPointer` () const
- void `setUserPointer` (void *pointer)
- void `setResizeFunc` (void(*func)(const `Window` *window, int width, int height))
- void `setKeyboardFunc` (void(*func)(const `Window` *window, int key, int scancode, int action, int mods))
- void `setMouseFunc` (void(*func)(const `Window` *window, int button, int action, int mods))
- void `setWheelFunc` (void(*func)(const `Window` *window, double x, double y))

8.43.1 詳解

ウィンドウ関連の処理.

覚え書き

GLFW を使って OpenGL のウィンドウを操作するラッパークラス.

[GgApp.h](#) の 150 行目に定義があります。

8.43.2 構築子と解体子

8.43.2.1 Window() [1/3]

```
GgApp::Window::Window (  
    const std::string & title = "GLFW Window",  
    int width = 640,  
    int height = 480,  
    int fullscreen = 0,  
    GLFWwindow * share = nullptr)
```

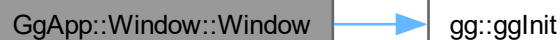
コンストラクタ.

引数

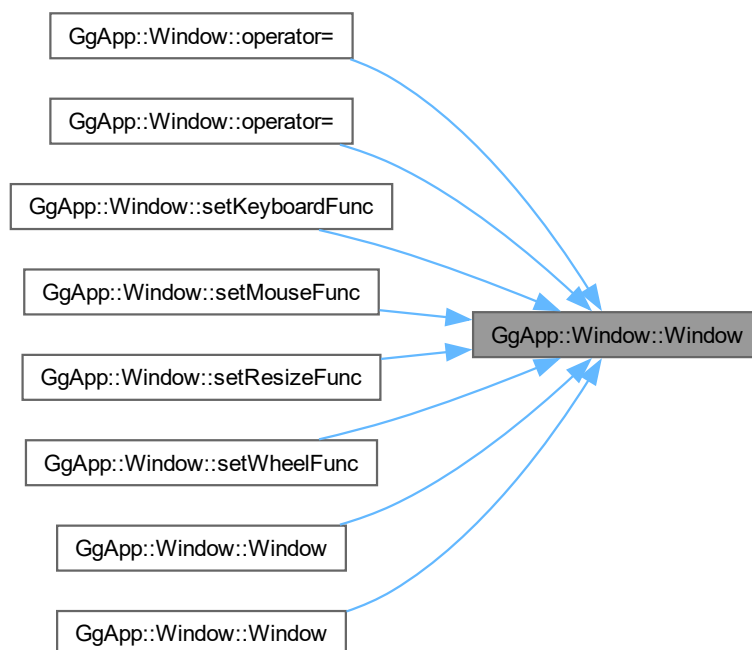
<i>title</i>	ウィンドウタイトルの文字列.
<i>width</i>	開くウィンドウの幅, フルスクリーン時は無視され実際のディスプレイの幅が使われる.
<i>height</i>	開くウィンドウの高さ, フルスクリーン時は無視され実際のディスプレイの高さが使われる.
<i>fullscreen</i>	フルスクリーン表示を行うディスプレイ番号, 0 ならフルスクリーン表示を行わない.
<i>share</i>	共有するコンテキスト, nullptr ならコンテキストを共有しない.

[GgApp.cpp](#) の 348 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.2.2 Window() [2/3]

```
GgApp::Window::Window (
    const Window & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のウィンドウ.
----------	-------------

呼び出し関係図:



8.43.2.3 Window() [3/3]

```
GgApp::Window::Window (
    Window && w) [default]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

呼び出し関係図:



8.43.2.4 ~Window()

```
virtual GgApp::Window::~~Window () [inline], [virtual]
```

デストラクタ.

[GgApp.h](#) の 284 行目に定義があります。

8.43.3 関数詳解

8.43.3.1 get()

```
auto * GgApp::Window::get () const [inline]
```

ウィンドウの識別子のポインタを取得する.

戻り値

GLFWwindow 型のウィンドウ識別子のポインタ.

[GgApp.h](#) の 314 行目に定義があります。

8.43.3.2 getAltArrowX()

```
auto GgApp::Window::getAltArrowX () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の X 値.

[GgApp.h](#) の 638 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.3 getAltArrowY()

```
auto GgApp::Window::getAltArrowY () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 648 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.43.3.4 getAltArrow()

```
void GgApp::Window::getAltArrow (
    GLfloat * alt_arrow) const [inline]
```

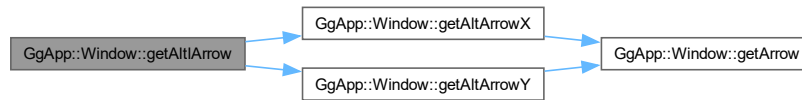
ALT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>alt_arrow</i>	ALT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
------------------	---

[GgApp.h](#) の 658 行目に定義があります.

呼び出し関係図:



8.43.3.5 getArrow() [1/2]

```
void GgApp::Window::getArrow (
    GLfloat * arrow,
    int mods = 0) const [inline]
```

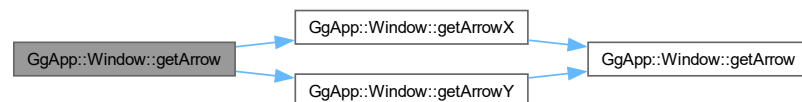
矢印キーの現在の値を得る.

引数

<i>arrow</i>	矢印キーの値を格納する GLfloat[2] の配列.
<i>mods</i>	修飾キーの状態 (0: なし, 1: SHIFT, 2: CTRL, 3: ALT).

[GgApp.h](#) の 565 行目に定義があります。

呼び出し関係図:



8.43.3.6 getArrow() [2/2]

```
auto GgApp::Window::getArrow (
    int direction = 0,
    int mods = 0) const [inline]
```

矢印キーの現在の値を得る.

引数

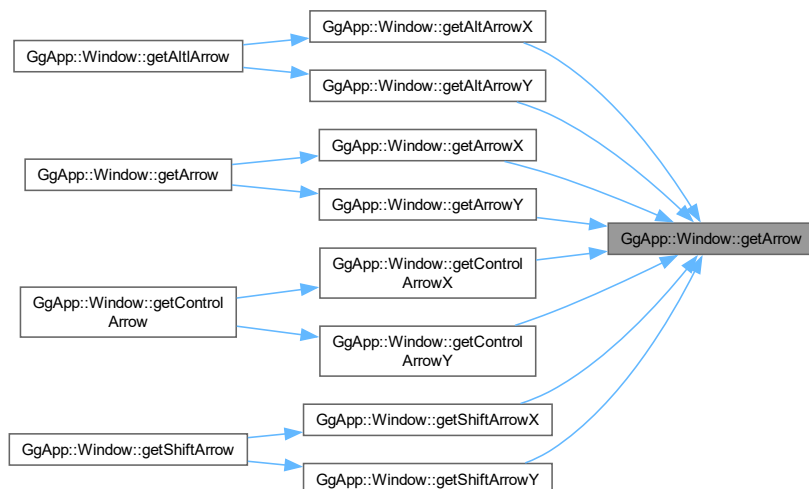
<i>direction</i>	方向 (0: X, 1: Y).
<i>mods</i>	修飾キーの状態 (0: なし, 1: SHIFT, 2: CTRL, 3: ALT).

戻り値

矢印キーの値.

[GgApp.h](#) の 531 行目に定義があります。

被呼び出し関係図:



8.43.3.7 getArrowX()

```
auto GgApp::Window::getArrowX (
    int mods = 0) const [inline]
```

矢印キーの現在の X 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0: なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	---

戻り値

矢印キーの X 値.

[GgApp.h](#) の 543 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.8 getArrowY()

```
auto GgApp::Window::getArrowY (  
    int mods = 0) const [inline]
```

矢印キーの現在の Y 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの Y 値.

[GgApp.h](#) の 554 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.9 getAspect()

```
auto GgApp::Window::getAspect () const [inline]
```

ウィンドウの縦横比を得る.

戻り値

ウィンドウの縦横比.

[GgApp.h](#) の 468 行目に定義があります。

8.43.3.10 getControlArrow()

```
void GgApp::Window::getControlArrow (
    GLfloat * control_arrow) const [inline]
```

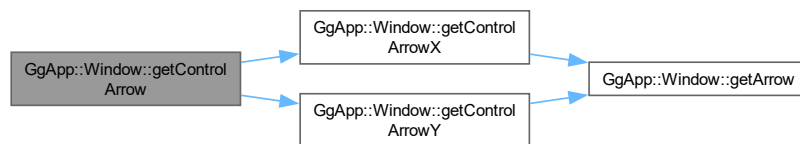
CTRL キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>control_arrow</i>	CTRL キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
----------------------	--

[GgApp.h](#) の 627 行目に定義があります。

呼び出し関係図:



8.43.3.11 getControlArrowX()

```
auto GgApp::Window::getControlArrowX () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

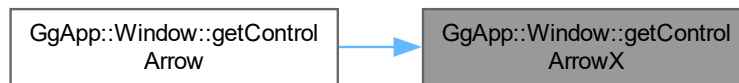
CTRL キーを押しながら矢印キーを押したときの現在の X 値.

[GgApp.h](#) の 607 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.12 getControlArrowY()

```
auto GgApp::Window::getControlArrowY () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

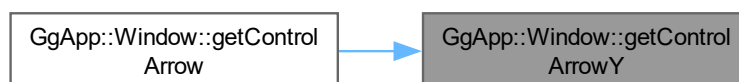
CTRL キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 617 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.13 getFboHeight()

```
auto GgApp::Window::getFboHeight () const [inline]
```

FBO の高さを得る.

戻り値

FBO の高さ.

GgApp.h の 416 行目に定義があります。

被呼び出し関係図:



8.43.3.14 getFboSize() [1/2]

```
const auto & GgApp::Window::getFboSize () const [inline]
```

FBO のサイズを得る.

戻り値

FBO の幅と高さを格納した GLsizei 型の 2 要素の配列.

GgApp.h の 447 行目に定義があります。

8.43.3.15 getFboSize() [2/2]

```
void GgApp::Window::getFboSize (
    GLsizei * fboSize) const [inline]
```

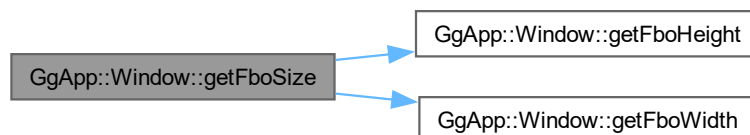
FBO のサイズを得る.

引数

size	FBO の幅と高さを格納した GLsizei 型の 2 要素の配列.
------	------------------------------------

GgApp.h の 457 行目に定義があります。

呼び出し関係図:



8.43.3.16 getFboWidth()

```
auto GgApp::Window::getFboWidth () const [inline]
```

FBO の横幅を得る.

戻り値

FBO の横幅.

[GgApp.h](#) の 406 行目に定義があります。

被呼び出し関係図:



8.43.3.17 getHeight()

```
auto GgApp::Window::getHeight () const [inline]
```

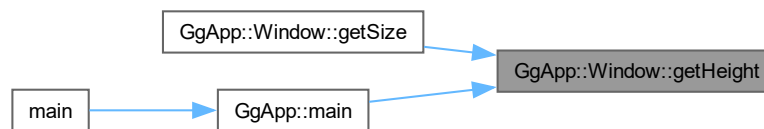
ウィンドウの高さを得る.

戻り値

ウィンドウの高さ.

[GgApp.h](#) の 396 行目に定義があります。

被呼び出し関係図:



8.43.3.18 getKey()

```
bool GgApp::Window::getKey (
    int key) const [inline]
```

キーが押されているかどうかを判定する.

戻り値

キーが押されていれば `true`.

[GgApp.h](#) の 478 行目に定義があります。

8.43.3.19 getLastKey()

```
int GgApp::Window::getLastKey () [inline]
```

最後にタイプしたキーを得る.

戻り値

最後にタイプしたキーの文字.

[GgApp.h](#) の 516 行目に定義があります。

8.43.3.20 getMouse() [1/3]

```
const auto * GgApp::Window::getMouse () const [inline]
```

マウスカーソルの現在位置を得る.

戻り値

マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.

[GgApp.h](#) の 669 行目に定義があります。

8.43.3.21 getMouse() [2/3]

```
void GgApp::Window::getMouse (
    GLfloat * position) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>position</i>	マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.
-----------------	---------------------------------------

[GgApp.h](#) の 680 行目に定義があります。

8.43.3.22 getMouse() [3/3]

```
auto GgApp::Window::getMouse (
    int direction) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスカーソルの現在位置.

[GgApp.h](#) の 693 行目に定義があります。

8.43.3.23 getMouseX()

```
auto GgApp::Window::getMouseX () const [inline]
```

マウスカーソルの現在位置の X 座標を得る.

戻り値

direction 方向のマウスカーソルの X 方向の現在位置.

[GgApp.h](#) の 704 行目に定義があります。

8.43.3.24 getMouseY()

```
auto GgApp::Window::getMouseY () const [inline]
```

マウスカーソルの現在位置の Y 座標を得る.

戻り値

direction 方向のマウスカーソルの Y 方向の現在位置.

[GgApp.h](#) の 715 行目に定義があります。

8.43.3.25 getRotation()

```
auto GgApp::Window::getRotation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	--

戻り値

回転を行う [GgQuaternion](#) 型の四元数.

[GgApp.h](#) の 843 行目に定義があります.

8.43.3.26 getRotationMatrix()

```
auto GgApp::Window::getRotationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	--

戻り値

回転を行う [GgMarix](#) 型の変換行列.

[GgApp.h](#) の 856 行目に定義があります.

8.43.3.27 getScrollMatrix()

```
auto GgApp::Window::getScrollMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによってオブジェクトの平行移動の変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

クリッピング座標系で平行移動を行う [GgMatrix](#) 型の変換行列.

[GgApp.h](#) の 820 行目に定義があります.

8.43.3.28 getShiftArrow()

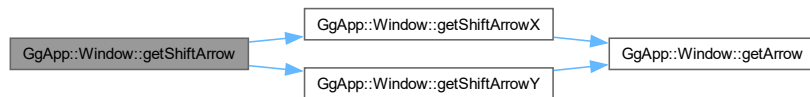
```
void GgApp::Window::getShiftArrow (
    GLfloat * shift_arrow) const [inline]
```

SHIFT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>shift_arrow</i>	SHIFT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
--------------------	---

GgApp.h の 596 行目に定義があります。
呼び出し関係図:



8.43.3.29 getShiftArrowX()

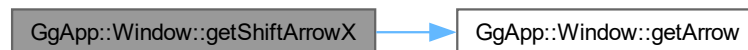
```
auto GgApp::Window::getShiftArrowX () const [inline]
```

SHIFT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の X 値.

GgApp.h の 576 行目に定義があります。
呼び出し関係図:



被呼び出し関係図:



8.43.3.30 getShiftArrowY()

```
auto GgApp::Window::getShiftArrowY () const [inline]
```

SHIFT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の Y 値.

GgApp.h の 586 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.31 getSize() [1/2]

```
const auto & GgApp::Window::getSize () const [inline]
```

ウィンドウのサイズを得る.

戻り値

ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列の参照.

GgApp.h の 426 行目に定義があります。

8.43.3.32 getSize() [2/2]

```
void GgApp::Window::getSize (
    GLsizei * size) const [inline]
```

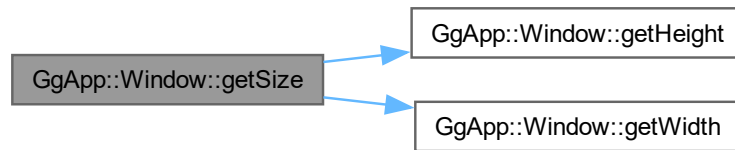
ウィンドウのサイズを得る.

引数

<code>size</code>	ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列.
-------------------	-------------------------------------

GgApp.h の 436 行目に定義があります。

呼び出し関係図:



8.43.3.33 getTranslation()

```
const auto & GgApp::Window::getTranslation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボール処理を考慮したマウスによるスクロールの変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

平行移動量を格納した GLfloat[3] の配列のポインタ.

[GgApp.h](#) の 784 行目に定義があります。

8.43.3.34 getTranslationMatrix()

```
auto GgApp::Window::getTranslationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによって視点の平行移動の変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

視点座標系で平行移動を行う [GgMatrix](#) 型の変換行列.

[GgApp.h](#) の 797 行目に定義があります。

8.43.3.35 getUserPointer()

```
void * GgApp::Window::getUserPointer () const [inline]
```

ユーザーポインタを取り出す.

戻り値

保存されているユーザポインタ.

[GgApp.h](#) の 898 行目に定義があります。

8.43.3.36 getWheel() [1/3]

```
const auto * GgApp::Window::getWheel () const [inline]
```

マウスホイールの回転量を得る.

戻り値

マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.

[GgApp.h](#) の 726 行目に定義があります。

8.43.3.37 getWheel() [2/3]

```
void GgApp::Window::getWheel (
    GLfloat * rotation) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>rotation</i>	マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.
-----------------	--------------------------------------

[GgApp.h](#) の 737 行目に定義があります。

8.43.3.38 getWheel() [3/3]

```
auto GgApp::Window::getWheel (
    int direction) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスホイールの回転量.

[GgApp.h](#) の 750 行目に定義があります。

8.43.3.39 getWheelX()

```
auto GgApp::Window::getWheelX () const [inline]
```

マウスホイールの X 方向の回転量を得る.

戻り値

マウスホイールの X 方向の回転量.

[GgApp.h](#) の 761 行目に定義があります。

8.43.3.40 getWheelY()

```
auto GgApp::Window::getWheelY () const [inline]
```

マウスホイールの Y 方向の回転量を得る.

戻り値

マウスホイールの Y 方向の回転量.

[GgApp.h](#) の 772 行目に定義があります。

8.43.3.41 getWidth()

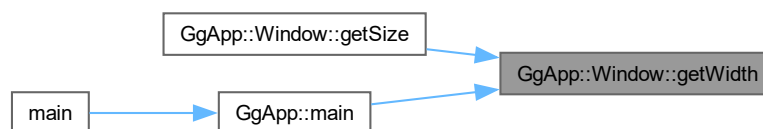
```
auto GgApp::Window::getWidth () const [inline]
```

ウィンドウの横幅を得る.

戻り値

ウィンドウの横幅.

[GgApp.h](#) の 386 行目に定義があります。
被呼び出し関係図:



8.43.3.42 operator bool()

```
GgApp::Window::operator bool () [explicit]
```

イベントを取得してループを継続すべきかどうか調べる.

戻り値

ループを継続すべきなら `true`.

[GgApp.cpp](#) の 441 行目に定義があります。
呼び出し関係図:



8.43.3.43 operator=() [1/2]

```
Window & GgApp::Window::operator= (
    const Window & w) [delete]
```

代入演算子は使用しない.

引数

<i>w</i>	代入元のウィンドウ.
----------	------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



8.43.3.44 operator=() [2/2]

```
Window & GgApp::Window::operator= (
    Window && w) [default]
```

ムーブ代入演算子.

引数

<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

呼び出し関係図:

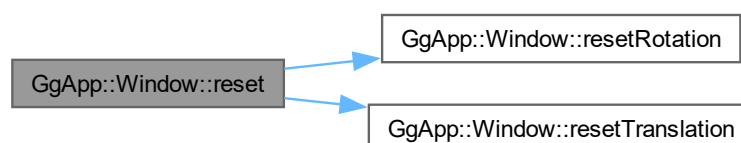


8.43.3.45 reset()

```
void GgApp::Window::reset () [inline]
```

トラックボール・マウスホイール・矢印キーの値を初期化する.
[GgApp.h](#) の 884 行目に定義があります.

呼び出し関係図:



8.43.3.46 resetRotation()

```
void GgApp::Window::resetRotation () [inline]
```

トラックボール処理を初期化する.

[GgApp.h](#) の 866 行目に定義があります.

被呼び出し関係図:



8.43.3.47 resetTranslation()

```
void GgApp::Window::resetTranslation () [inline]
```

平行移動量を初期化する.

[GgApp.h](#) の 875 行目に定義があります.

被呼び出し関係図:



8.43.3.48 restoreViewport()

```
void GgApp::Window::restoreViewport () const [inline]
```

ビューポートを元のサイズに復帰する.

[GgApp.h](#) の 355 行目に定義があります.

被呼び出し関係図:



8.43.3.49 selectInterface()

```
void GgApp::Window::selectInterface (  
    int no) [inline]
```

インタフェースを選択する.

引数

<i>no</i>	インターフェース番号.
-----------	-------------

[GgApp.h](#) の 493 行目に定義があります。

8.43.3.50 setClose()

```
void GgApp::Window::setClose (
    int flag = GLFW_TRUE) const [inline]
```

ウィンドウのクローズフラグを設定する.

引数

<i>flag</i>	クローズフラグ, 0 (GLFW_FALSE) 以外ならウィンドウを閉じる.
-------------	--

[GgApp.h](#) の 324 行目に定義があります。

8.43.3.51 setKeyboardFunc()

```
void GgApp::Window::setKeyboardFunc (
    void(* func) (const Window *window, int key, int scancode, int action, int mods))
[inline]
```

ユーザ定義の keyboard 関数を設定する.

引数

<i>func</i>	ユーザ定義の keyboard 関数, キーボードの操作時に呼び出される.
-------------	---------------------------------------

[GgApp.h](#) の 928 行目に定義があります。
呼び出し関係図:



8.43.3.52 setMouseFunc()

```
void GgApp::Window::setMouseFunc (
    void(* func) (const Window *window, int button, int action, int mods)) [inline]
```

ユーザ定義の mouse 関数を設定する.

引数

<i>func</i>	ユーザ定義の mouse 関数, マウスボタンの操作時に呼び出される.
-------------	-------------------------------------

[GgApp.h](#) の 938 行目に定義があります。

呼び出し関係図:



8.43.3.53 setResizeFunc()

```
void GgApp::Window::setResizeFunc (
    void(* func )(const Window *window, int width, int height)) [inline]
ユーザ定義の resize 関数を設定する.
```

引数

<i>func</i>	ユーザ定義の <code>resize</code> 関数, ウィンドウのサイズ変更時に呼び出される.
-------------	---

[GgApp.h](#) の 918 行目に定義があります。

呼び出し関係図:



8.43.3.54 setUserPointer()

```
void GgApp::Window::setUserPointer (
    void * pointer) [inline]
任意のユーザポインタを保存する.
```

引数

<i>pointer</i>	保存するユーザポインタ.
----------------	--------------

[GgApp.h](#) の 908 行目に定義があります。

8.43.3.55 setVelocity()

```
void GgApp::Window::setVelocity (
    GLfloat vx,
    GLfloat vy,
    GLfloat vz = 0.1f) [inline]
マウスの移動速度を設定する.
```

引数

<i>vx</i>	x 方向の移動速度.
-----------	------------

<code>vy</code>	y 方向の移動速度.
<code>vz</code>	z 方向の移動速度.

[GgApp.h](#) の 506 行目に定義があります。

8.43.3.56 setWheelFunc()

```
void GgApp::Window::setWheelFunc (
    void(* func )(const Window *window, double x, double y)) [inline]
```

ユーザ定義の wheel 関数を設定する.

引数

<code>func</code>	ユーザ定義の wheel 関数, マウスホイールの操作時に呼び出される.
-------------------	--------------------------------------

[GgApp.h](#) の 948 行目に定義があります。
呼び出し関係図:



8.43.3.57 shouldClose()

```
bool GgApp::Window::shouldClose () const [inline]
```

ウィンドウを閉じるべきかどうか調べる.

戻り値

ウィンドウを閉じるべきなら true.

[GgApp.h](#) の 334 行目に定義があります。
被呼び出し関係図:



8.43.3.58 swapBuffers()

```
void GgApp::Window::swapBuffers () const
```

カラーバッファを入れ替える.
[GgApp.cpp](#) の 492 行目に定義があります。

被呼び出し関係図:



8.43.3.59 updateViewport()

`void GgApp::Window::updateViewport ()`
ビューポートのサイズを更新する。
[GgApp.cpp](#) の 511 行目に定義があります。
呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [GgApp.h](#)
- [GgApp.cpp](#)

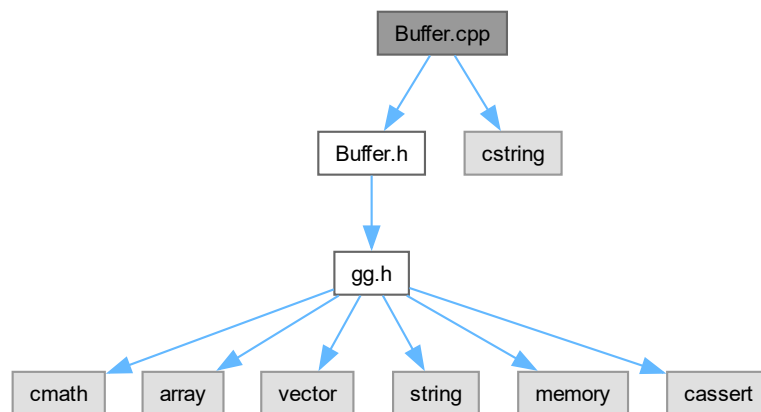
Chapter 9

ファイル詳解

9.1 Buffer.cpp ファイル

```
#include "Buffer.h"
#include <cstring>
```

Buffer.cpp の依存先関係図:



9.1.1 詳解

バッファクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Buffer.cpp](#) に定義があります。

9.2 Buffer.cpp

[\[詳解\]](#)

```

00001
00008 #include "Buffer.h"
00009
00010 // 標準ライブラリ
00011 #include <cstring>
00012
00013 //
00014 // フォーマットからチャンネル数を求める
00015 //
00016 int Buffer::formatToChannels(GLenum format) const
00017 {
00018     switch (format)
00019     {
00020     case GL_RED:
00021         return 1;
00022     case GL_RG:
00023         return 2;
00024     case GL_RGB:
00025         return 3;
00026     case GL_RGBA:
00027         return 4;
00028     default:
00029         assert(false);
00030         break;
00031     }
00032
00033     return 0;
00034 };
00035
00036 //
00037 // 既存のバッファにコピーする
00038 //
00039 void Buffer::copyBuffer(const Buffer& buffer) noexcept
00040 {
00041     #if defined(USE_PIXEL_BUFFER_OBJECT)
00042         // コピー元のピクセルバッファオブジェクト
00043         glBindBuffer(GL_COPY_READ_BUFFER, buffer.bufferName);
00044
00045         // コピー先のピクセルバッファオブジェクト
00046         glBindBuffer(GL_COPY_WRITE_BUFFER, bufferName);
00047
00048         // バッファオブジェクトをコピーする
00049         glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
00050             0, 0, std::min(bufferLength, buffer.bufferLength));
00051     #else
00052         // フレームのデータをコピーする
00053         memcpy(bufferName.data(), buffer.bufferName.data(), bufferName.size());
00054     #endif
00055 }
00056
00057 //
00058 // バッファを作成する
00059 //
00060 void Buffer::create(GLsizei width, GLsizei height, int channels)
00061 {
00062     // 指定したサイズが保持しているフレームのサイズと同じなら何もしない
00063     if (width == bufferSize[0] && height == bufferSize[1]
00064         && channels == bufferChannels) return;
00065
00066     // フレームのサイズとチャンネル数を記録する
00067     bufferSize = std::array<int, 2>{ width, height };
00068     bufferChannels = channels;
00069
00070     #if defined(USE_PIXEL_BUFFER_OBJECT)
00071         // フレームの保存に必要なメモリ量を求める
00072         bufferLength = width * height * channels;
00073
00074         // 以前のピクセルバッファオブジェクトを破棄する
00075         glDeleteBuffers(1, &bufferName);
00076
00077         // 新しいピクセルバッファオブジェクトを作成する
00078         glGenBuffers(1, &bufferName);
00079
00080         // ピクセルバッファオブジェクトのメモリを確保してデータを転送する
00081         glBindBuffer(GL_PIXEL_PACK_BUFFER, bufferName);
00082         glBufferData(GL_PIXEL_PACK_BUFFER, bufferLength, nullptr, GL_DYNAMIC_COPY);
00083         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00084     #else
00085         // メモリを確保する
00086         bufferName.resize(width * height * channels);
00087     #endif
00088 }
00089
00090 //
00091 // バッファをコピーする
00092 //
00093 void Buffer::copy(const Buffer& buffer) noexcept

```

```

00094 {
00095     // コピー元と同じサイズの空のバッファを作る
00096     Buffer::create(buffer.bufferSize[0], buffer.bufferSize[1],
00097         buffer.bufferChannels);
00098
00099     // 作成されたバッファにコピーする
00100     copyBuffer(buffer);
00101 }
00102
00103 //
00104 // 代入演算子
00105 //
00106 Buffer& Buffer::operator=(const Buffer& buffer)
00107 {
00108     // 代入元と代入先が同じなら何もしない
00109     if (&buffer == this) return *this;
00110
00111     // 引数のバッファをバッファをこのバッファにコピーする
00112     Buffer::copy(buffer);
00113
00114     // このバッファを返す
00115     return *this;
00116 }
00117
00118 //
00119 // ムーブ代入演算子
00120 //
00121 Buffer& Buffer::operator=(Buffer&& buffer) noexcept
00122 {
00123     // ムーブ代入元とムーブ代入先が同じなら何もしない
00124     if (&buffer == this) return *this;
00125
00126     // ムーブ代入元のバッファのメンバをムーブする
00127     bufferSize = buffer.bufferSize;
00128     buffer.bufferSize = { 0, 0 };
00129     bufferChannels = buffer.bufferChannels;
00130     buffer.bufferChannels = 0;
00131 #if defined(USE_PIXELBUFFER_OBJECT)
00132     bufferLength = buffer.bufferLength;
00133     buffer.bufferLength = 0;
00134
00135     // ピクセルバッファオブジェクトを削除する
00136     glDeleteBuffers(1, &bufferName);
00137
00138     // ムーブ代入元のピクセルバッファオブジェクト名をムーブ代入先に移す
00139     bufferName = buffer.bufferName;
00140
00141     // ムーブ代入元のデストラクタでバッファが削除されないよう 0 にする
00142     buffer.bufferName = 0;
00143 #endif
00144
00145     // このバッファを返す
00146     return *this;
00147 }
00148
00149 //
00150 // ピクセルバッファオブジェクトをマップする
00151 //
00152 GLvoid* Buffer::map()
00153 #if defined(USE_PIXELBUFFER_OBJECT)
00154 const
00155 #endif
00156 {
00157     #if defined(USE_PIXELBUFFER_OBJECT)
00158         // ピクセルバッファオブジェクトを参照する
00159         glBindBuffer(GL_PIXEL_PACK_BUFFER, bufferName);
00160
00161         // ピクセルバッファオブジェクトをマップする
00162         return glMapBufferRange(GL_PIXEL_PACK_BUFFER, 0, bufferLength,
00163             GL_MAP_READ_BIT | GL_MAP_WRITE_BIT);
00164     #else
00165         // データの格納場所を返す
00166         return bufferName.data();
00167     #endif
00168 }
00169
00170 //
00171 // ピクセルバッファオブジェクトをアンマップする
00172 //
00173 void Buffer::unmap() const
00174 {
00175     #if defined(USE_PIXELBUFFER_OBJECT)
00176         // ピクセルバッファオブジェクトのマップを解除する
00177         glUnmapBuffer(GL_PIXEL_PACK_BUFFER);
00178
00179         // アップロード先のピクセルバッファオブジェクトの結合を解除する
00180         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);

```

```

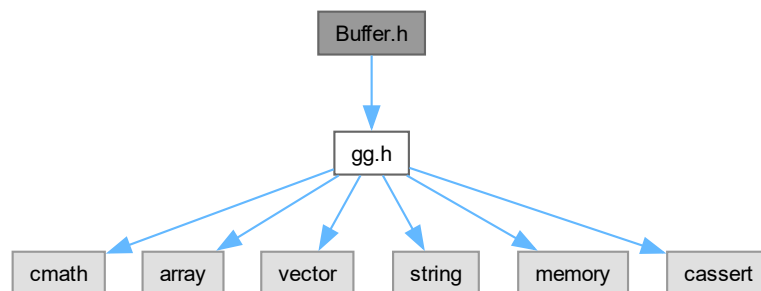
00181 #endif
00182 }
00183
00184 //
00185 // バッファを破棄する
00186 //
00187 void Buffer::discard()
00188 {
00189     #if defined(USE_PIXEL_BUFFER_OBJECT)
00190         // ピクセルバッファオブジェクトの結合を解除する
00191         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00192         glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
00193
00194         // ピクセルバッファオブジェクトを削除する
00195         glDeleteBuffers(1, &bufferName);
00196         bufferName = 0;
00197     #else
00198         // メモリを消去する
00199         bufferName.clear();
00200     #endif
00201
00202     // バッファのサイズを 0 にする
00203     bufferSize = std::array<int, 2>{ 0, 0 };
00204     bufferChannels = 0;
00205 }

```

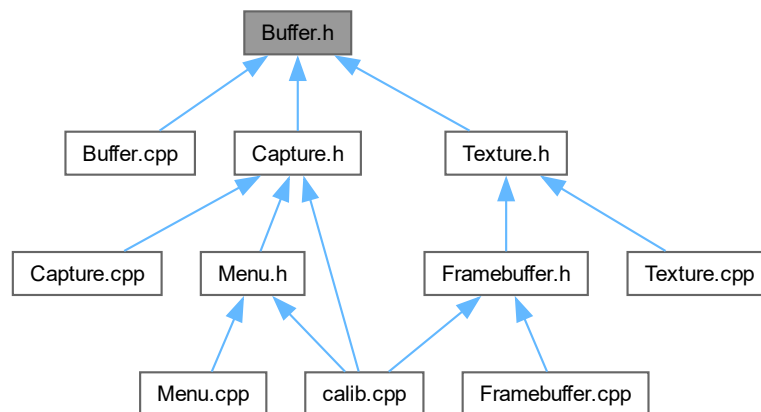
9.3 Buffer.h ファイル

```
#include "gg.h"
```

Buffer.h の依存先関係図:



被依存関係図:



クラス

- class [Buffer](#)

マクロ定義

- `#define` [USE_PIXEL_BUFFER_OBJECT](#)

9.3.1 詳解

バッファクラスの定義

著者

Kohe Tokoi

日付

April 3, 2023

[Buffer.h](#) に定義があります。

9.3.2 マクロ定義詳解

9.3.2.1 USE_PIXEL_BUFFER_OBJECT

`#define USE_PIXEL_BUFFER_OBJECT`
[Buffer.h](#) の 16 行目に定義があります。

9.4 Buffer.h

[[詳解](#)]

```

00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014

```

```

00015 // ピクセルバッファオブジェクトを使うとき
00016 #define USE_PIXEL_BUFFER_OBJECT
00017
00024 class Buffer
00025 {
00027     std::array<int, 2> bufferSize;
00028
00030     int bufferChannels;
00031
00032     #if defined(USE_PIXEL_BUFFER_OBJECT)
00034         GLsizei bufferLength;
00035
00037         GLuint
00038     #else
00040         std::vector<GLubyte>
00041     #endif
00042         bufferName;
00043
00044 protected:
00045
00052     auto channelsToFormat(int channels) const
00053     {
00054         // OpenGL のテクスチャフォーマットのリスト
00055         static constexpr GLenum toFormat[] { GL_RED, GL_RG, GL_RGB, GL_RGBA };
00056
00057         // フォーマットを返す
00058         return toFormat[(channels - 1) & 3];
00059     }
00060
00067     int formatToChannels(GLenum format) const;
00068
00074     void copyBuffer(const Buffer& buffer) noexcept;
00075
00076 public:
00077
00081     Buffer()
00082         : bufferSize{ 0, 0 }
00083         , bufferChannels{ 0 }
00084     #if defined(USE_PIXEL_BUFFER_OBJECT)
00085         , bufferLength{ 0 }
00086         , bufferName{ 0 }
00087     #endif
00088     {
00089     }
00090
00102     Buffer(GLsizei width, GLsizei height, int channels)
00103     {
00104         Buffer::create(width, height, channels);
00105     }
00106
00112     Buffer(const Buffer& buffer)
00113     {
00114         Buffer::copy(buffer);
00115     }
00116
00122     Buffer(Buffer&& buffer) noexcept
00123         : bufferSize{ buffer.bufferSize }
00124         , bufferChannels{ buffer.bufferChannels }
00125     #if defined(USE_PIXEL_BUFFER_OBJECT)
00126         , bufferLength{ buffer.bufferLength }
00127         , bufferName{ buffer.bufferName }
00128     #endif
00129     {
00130         bufferSize = { 0, 0 };
00131         bufferChannels = 0;
00132     #if defined(USE_PIXEL_BUFFER_OBJECT)
00133         bufferLength = 0;
00134
00135         // ムーブ元のデストラクタでバッファが削除されないよう 0 にする
00136         bufferName = 0;
00137     #endif
00138     }
00139
00143     virtual ~Buffer()
00144     {
00145         Buffer::discard();
00146     }
00147
00154     Buffer& operator=(const Buffer& buffer);
00155
00162     Buffer& operator=(Buffer&& buffer) noexcept;
00163
00175     virtual void create(GLsizei width, GLsizei height, int channels);
00176
00188     void resize(GLsizei width, GLsizei height, int channels)
00189     {
00190         create(width, height, channels);

```

```

00191     }
00192
00203     void resize(const Buffer& buffer)
00204     {
00205         create(buffer.getWidth(), buffer.getHeight(), buffer.getChannels());
00206     }
00207
00217     virtual void copy(const Buffer& buffer) noexcept;
00218
00222     virtual void discard();
00223
00229     #if defined(USE_PIXEL_BUFFER_OBJECT)
00230     auto getBufferName() const
00231     #else
00232     auto& getBufferName()
00233     #endif
00234     {
00235         return bufferName;
00236     }
00237
00243     virtual const std::array<GLsizei, 2>& getSize() const
00244     {
00245         return bufferSize;
00246     }
00247
00253     GLsizei getWidth() const
00254     {
00255         return getSize()[0];
00256     }
00257
00263     GLsizei getHeight() const
00264     {
00265         return getSize()[1];
00266     }
00267
00273     virtual int getChannels() const
00274     {
00275         return bufferChannels;
00276     }
00277
00283     auto getFormat() const
00284     {
00285         return channelsToFormat(getChannels());
00286     }
00287
00293     auto getAspect() const
00294     {
00295         return static_cast<GLfloat>(getWidth()) / static_cast<GLfloat>(getHeight());
00296     }
00297
00303     void bindBuffer(GLenum target = GL_PIXEL_PACK_BUFFER) const
00304     {
00305         #if defined(USE_PIXEL_BUFFER_OBJECT)
00306             glBindBuffer(target, bufferName);
00307         #endif
00308     }
00309
00315     void unbindBuffer(GLenum target = GL_PIXEL_PACK_BUFFER) const
00316     {
00317         #if defined(USE_PIXEL_BUFFER_OBJECT)
00318             glBindBuffer(target, 0);
00319         #endif
00320     }
00321
00327     GLvoid* map()
00328     #if defined(USE_PIXEL_BUFFER_OBJECT)
00329         const
00330     #endif
00331     ;
00332
00336     void unmap() const;
00337 };

```

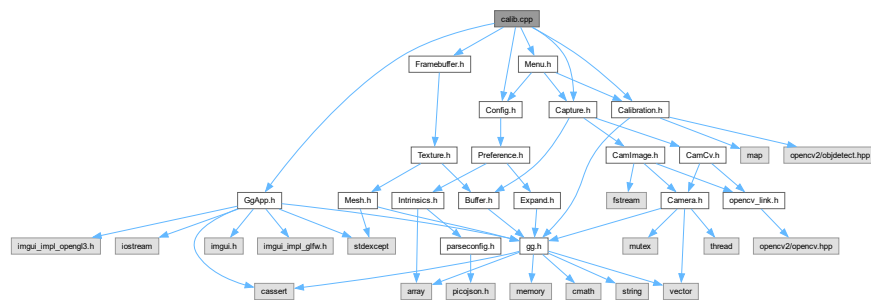
9.5 calib.cpp ファイル

```

#include "GgApp.h"
#include "Config.h"
#include "Capture.h"
#include "Calibration.h"
#include "Menu.h"
#include "Framebuffer.h"

```

calib.cpp の依存先関係図:



マクロ定義

- `#define CONFIG_FILE PROJECT_NAME "_config.json"`

9.5.1 詳解

ChArUco Board によるカメラキャリブレーション

著者

Kohe Tokoi

日付

March 6, 2024

[calib.cpp](#) に定義があります。

9.5.2 マクロ定義詳解

9.5.2.1 CONFIG_FILE

`#define CONFIG_FILE PROJECT_NAME "_config.json"`
[calib.cpp](#) の 28 行目に定義があります。

9.6 calib.cpp

[詳解]

```

00001
00008
00009 // ウィンドウ関連の処理
00010 #include "GgApp.h"
00011
00012 // 構成データ
00013 #include "Config.h"
00014
00015 // キャプチャデバイス
00016 #include "Capture.h"
00017
00018 // 校正
00019 #include "Calibration.h"
00020
00021 // メニュー
00022 #include "Menu.h"
00023
00024 // フレームバッファオブジェクト
00025 #include "Framebuffer.h"
00026
00027 // 構成ファイル名
00028 #define CONFIG_FILE PROJECT_NAME "_config.json"
00029

```



```

00030 //
00031 // アプリケーション本体
00032 //
00033 int GgApp::main(int argc, const char* const* argv)
00034 {
00035     // 構成ファイルを読み込む
00036     Config config{ CONFIG_FILE };
00037
00038     // 構成にもとづいてウィンドウを作成する
00039     GgApp::Window window{ config.getTitle(), config.getWidth(), config.getHeight() };
00040
00041     // 開いたウィンドウに対して初期化処理を実行する
00042     config.initialize();
00043
00044     // キャプチャデバイスを作る
00045     Capture capture;
00046
00047     // 校正オブジェクトを作成する
00048     Calibration calibration{ config.getDictionaryName(), config.getCheckerLength() };
00049
00050     // メニューを作る
00051     Menu menu{ config, capture, calibration };
00052
00053     // キャプチャデバイスで初期画像を開く
00054     if (!capture.openImage(config.getInitialImage())) throw std::runtime_error("Cannot open initial
image.");
00055
00056     // 解像度と画角の調整値の初期値を初期画像に合わせる
00057     menu.setSize(capture.getSize());
00058
00059     // キャプチャしたフレームを保持するテクスチャ
00060     Texture frame;
00061
00062     // 画像の展開に用いるフレームバッファオブジェクトのサイズを初期ウィンドウに合わせる
00063     Framebuffer framebuffer{ config.getWidth(), config.getHeight() };
00064
00065     // ウィンドウが開いている間繰り返す
00066     while (window && menu)
00067     {
00068         // メニューを表示して設定を更新する
00069         menu.draw();
00070
00071         // 選択しているキャプチャデバイスから1フレーム取得する
00072         capture.retrieve(frame);
00073
00074         // ピクセルバッファオブジェクトの内容をテクスチャに転送する
00075         frame.drawPixels();
00076
00077         // フレームバッファオブジェクトのサイズをキャプチャしたフレームに合わせる
00078         framebuffer.resize(frame);
00079
00080         // シェーダの設定を行う
00081         const auto&& size{ menu.setup(framebuffer.getAspect()) };
00082
00083         // フレームバッファオブジェクトにフレームを展開する
00084         framebuffer.update(size, frame);
00085
00086         // ArUco Marker を検出するなら
00087         if (menu.detectMarker || menu.detectBoard)
00088         {
00089             // フレームバッファオブジェクトの内容をピクセルバッファオブジェクトに転送する
00090             framebuffer.readPixels();
00091
00092             // 入力画像のサイズを調べる
00093             const auto size{ cv::Size{ framebuffer.getWidth(), framebuffer.getHeight() } };
00094
00095             // ピクセルバッファオブジェクトを CPU のメモリ空間にマップする
00096             cv::Mat image{ size, CV_8UC(framebuffer.getChannels()), framebuffer.map() };
00097
00098             // ChArUco Board を認識するなら
00099             if (menu.detectBoard)
00100             {
00101                 // ChArUco Board を検出する
00102                 calibration.detectBoard(image);
00103             }
00104             else
00105             {
00106                 // ArUco Marker を検出する
00107                 calibration.detectMarkers(image, menu.getMarkerLength());
00108             }
00109
00110             // ピクセルバッファオブジェクトのマップを解除する
00111             framebuffer.unmap();
00112
00113             // ピクセルバッファオブジェクトの内容をフレームバッファオブジェクトに書き戻す
00114             framebuffer.drawPixels();
00115         }

```

```

00116
00117 // 表示するウィンドウのビューポートを再設定する
00118 window.setMenuBarHeight(menu.getMenuBarHeight());
00119
00120 // フレームバッファオブジェクトの内容を表示する
00121 // framebuffer.show(window.getWidth(), window.getHeight());
00122 framebuffer.draw(window.getWidth(), window.getHeight());
00123
00124 // カラーバッファを入れ替えてイベントを取り出す
00125 window.swapBuffers();
00126 }
00127
00128 return 0;
00129 }

```

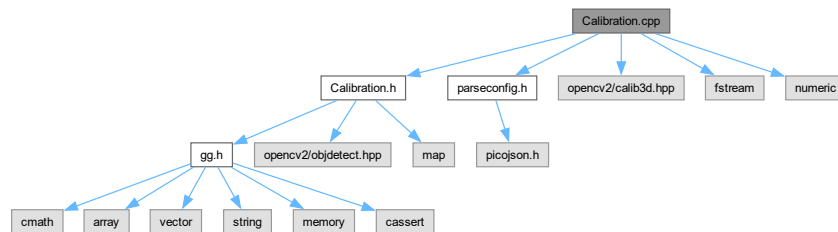
9.7 Calibration.cpp ファイル

```

#include "Calibration.h"
#include "parseconfig.h"
#include <opencv2/calib3d.hpp>
#include <fstream>
#include <numeric>

```

Calibration.cpp の依存先関係図:



マクロ定義

- `#define` [USE_RODRIGUES](#)

9.7.1 詳解

校正用フレームバッファオブジェクトクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Calibration.cpp](#) に定義があります。

9.7.2 マクロ定義詳解

9.7.2.1 USE_RODRIGUES

```
#define USE_RODRIGUES
```

[Calibration.cpp](#) の 22 行目に定義があります。

9.8 Calibration.cpp

[詳解]

```

00001
00008 #include "Calibration.h"
00009
00010 // 構成ファイルの読み取り補助
00011 #include "parseconfig.h"
00012
00013 // OpenCV
00014 #pragma warning(disable:4819)
00015 #include <opencv2/calib3d.hpp>
00016
00017 // 標準ライブラリ
00018 #include <fstream>
00019 #include <numeric>
00020
00021 // cv::Rodrigues() を使う
00022 #define USE_RODRIGUES
00023
00024
00025 //
00026 // コンストラクタ
00027 //
00028 Calibration::Calibration(const std::string& dictionaryName, const std::array<float, 2>& length)
00029 : size{ 0, 0 }
00030 , repError{ 0.0 }
00031 , totalCorners{ 0 }
00032 , calibrationFlags
00033 {
00034     0
00035     // cv::CALIB_USE_INTRINSIC_GUESS // cameraMatrix contains valid initial values of fx, fy, cx, cy
    that are optimized further. Otherwise, (cx, cy) is initially set to the image center (imageSize is
    used), and focal distances are computed in a least - squares fashion. Note, that if intrinsic
    parameters are known, there is no need to use this function just to estimate extrinsic parameters. Use
    solvePnP instead.
00036     // cv::CALIB_FIX_PRINCIPAL_POINT // The principal point is not changed during the global
    optimization. It stays at the center or at a different location specified when CALIB_USE_INTRINSIC_GUESS
    is set too.
00037     // cv::CALIB_FIX_ASPECT_RATIO // The functions consider only fy as a free parameter. The ratio
    fx / fy stays the same as in the input cameraMatrix. When CALIB_USE_INTRINSIC_GUESS is not set, the
    actual input values of fx and fy are ignored, only their ratio is computed and used further.
00038     // cv::CALIB_ZERO_TANGENT_DIST // Tangential distortion coefficients (p1, p2) are set to zeros
    and stay zero.
00039     // cv::CALIB_FIX_FOCAL_LENGTH // The focal length is not changed during the global optimization
    if CALIB_USE_INTRINSIC_GUESS is set.
00040     // cv::CALIB_FIX_K1 // , ..., CALIB_FIX_K6 The corresponding radial distortion
    coefficient is not changed during the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient
    from the supplied distCoeffs matrix is used. Otherwise, it is set to 0.
00041     // cv::CALIB_RATIONAL_MODEL // Coefficients k4, k5, and k6 are enabled. To provide the
    backward compatibility, this extra flag should be explicitly specified to make the calibration
    function use the rational model and return 8 coefficients or more.
00042     // cv::CALIB_THIN_PRISM_MODEL // Coefficients s1, s2, s3 and s4 are enabled. To provide the
    backward compatibility, this extra flag should be explicitly specified to make the calibration
    function use the thin prism model and return 12 coefficients or more.
00043     // cv::CALIB_FIX_S1_S2_S3_S4 // The thin prism distortion coefficients are not changed during
    the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient from the supplied distCoeffs
    matrix is used. Otherwise, it is set to 0.
00044     // cv::CALIB_TILTED_MODEL // Coefficients tauX and tauY are enabled. To provide the backward
    compatibility, this extra flag should be explicitly specified to make the calibration function use the
    tilted sensor model and return 14 coefficients.
00045     // cv::CALIB_FIX_TAUX_TAUY // The coefficients of the tilted sensor model are not changed
    during the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient from the supplied
    distCoeffs matrix is used. Otherwise, it is set to 0.
00046 }
00047 {
00048     // ArUco Marker の辞書を選択する
00049     setDictionary(dictionaryName, length);
00050 }
00051
00052 //
00053 // デストラクタ
00054 //
00055 Calibration::~Calibration()
00056 {
00057 }
00058
00059 //
00060 // Charuco Board を作成する
00061 //
00062 void Calibration::createBoard(const std::array<float, 2>& length)
00063 {
00064     // キャリブレーション用の Charuco Board を作成する
00065     board = new cv::aruco::CharucoBoard(cv::Size{ 10, 7 },
00066         length[0] * 0.01f, length[1] * 0.01f, dictionary);
00067 }

```

```

00068 // キャリブレーション用の ChArUco Board の検出器を作成する
00069 boardDetector = new cv::aruco::CharucoDetector(*board);
00070
00071 // board は boardDetector->getBoard() で取り出すことができるが
00072 // 実行中に board を作り直すことがあるので cv::Ptr に持たせる
00073
00074 // 校正結果を再利用しない
00075 calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00076 }
00077
00078 //
00079 // ArUco Marker の辞書と検出器を設定する
00080 //
00081 void Calibration::setDictionary(const std::string& dictionaryName, const std::array<float, 2>& length)
00082 {
00083     // ArUco Marker の辞書を検索する
00084     auto dictionaryItem{ dictionaryList.find(dictionaryName) };
00085
00086     // ArUco Marker の辞書が見つからなかったら辞書リストの最初の辞書を使う
00087     if (dictionaryItem == dictionaryList.end()) dictionaryItem = dictionaryList.begin();
00088
00089     // ArUco Marker の辞書を設定する
00090     dictionary = cv::aruco::getPredefinedDictionary(dictionaryItem->second);
00091
00092     // ArUco Marker の検出器を作成する
00093     cv::aruco::DetectorParameters detectorParams = cv::aruco::DetectorParameters();
00094     detector = new cv::aruco::ArucoDetector(dictionary, detectorParams);
00095
00096     // キャリブレーション用の ChArUco Board を作成する
00097     createBoard(length);
00098 }
00099
00100 //
00101 // ChArUco Board を描く
00102 //
00103 void Calibration::drawBoard(cv::Mat& boardImage, int width, int height)
00104 {
00105     board->generateImage(cv::Size{ width, height }, boardImage, 10, 1);
00106 }
00107
00108 //
00109 // ChArUco Board を検出する
00110 //
00111 void Calibration::detectBoard(cv::Mat& image)
00112 {
00113     // 画像のサイズを保存しておく
00114     size = image.size();
00115
00116     // ChArUco Board のコーナーを検出する
00117     boardDetector->detectBoard(image, charucoCorners, charucoIds);
00118
00119     // コーナーが見つからなかったら何もしない
00120     if (charucoCorners.empty()) return;
00121
00122     // ChArUco Board のコーナーの位置を表示に描き込む
00123     cv::aruco::drawDetectedCornersCharuco(image, charucoCorners, charucoIds, cv::Scalar(0, 0, 255));
00124 }
00125
00126 //
00127 // ArUco Marker を検出する
00128 //
00129 void Calibration::detectMarkers(cv::Mat& image, float markerLength)
00130 {
00131     // ArUco Marker のコーナーを検出する
00132     detector->detectMarkers(image, corners, ids, rejected);
00133
00134     // コーナーが見つからなければ戻る
00135     if (corners.empty()) return;
00136
00137     // キャリブレーションが完了していれば
00138     if (finished())
00139     {
00140         // 各マーカーに対応する3次元空間の点
00141         std::vector<float> markerCenter{ markerLength * 0.5f };
00142         std::vector<cv::Point3f> markerObjPoints;
00143         markerObjPoints.push_back(cv::Point3f(-markerCenter, markerCenter, 0.0f));
00144         markerObjPoints.push_back(cv::Point3f(markerCenter, markerCenter, 0.0f));
00145         markerObjPoints.push_back(cv::Point3f(markerCenter, -markerCenter, 0.0f));
00146         markerObjPoints.push_back(cv::Point3f(-markerCenter, -markerCenter, 0.0f));
00147
00148         // 個々のマーカーについて
00149         for (size_t i = 0; i < corners.size(); ++i)
00150         {
00151             // マーカーのコーナー検出位置から3次元姿勢(回転・平行移動)を推定する
00152             cv::Vec3d rvec, tvec;
00153             cv::solvePnP(markerObjPoints, corners[i], cameraMatrix, distCoeffs, rvec, tvec, false,
00154                 cv::SOLVEPNP_IPPE_SQUARE);

```

```

00154
00155     // 座標軸を描く
00156     cv::drawFrameAxes(image, cameraMatrix, distCoeffs, rvec, tvec, markerLength);
00157 }
00158 }
00159 else
00160 {
00161     // ArUco Marker の場所に矩形と番号を描き込む
00162     cv::aruco::drawDetectedMarkers(image, corners, ids);
00163 }
00164 }
00165
00166 //
00167 // 標本を取得する
00168 //
00169 void Calibration::recordCorners()
00170 {
00171     // ChArUco Board のコーナーが4 つ以上見つければ
00172     if (charucoCorners.size() >= 4)
00173     {
00174         // ChArUco Board のレイアウトと検出されたコーナーから
00175         // ChArUco Board 上の点と対応する画像上の点を求める
00176         board->matchImagePoints(charucoCorners, charucoIds, objectPoints, imagePoints);
00177
00178         // ChArUco Board 上の点と対応する画像上の点が見つければ
00179         if (!imagePoints.empty() && !objectPoints.empty())
00180         {
00181             // ChArUco Board のコーナーを記録する
00182             allCorners.push_back(charucoCorners);
00183             allIds.push_back(charucoIds);
00184             allImagePoints.push_back(imagePoints);
00185             allObjectPoints.push_back(objectPoints);
00186
00187             // 記録したコーナーの数の合計を求める
00188             totalCorners += static_cast<int>(charucoCorners.size());
00189         }
00190     }
00191     #if defined(DEBUG)
00192     std::cerr << "charucoCorners = " << charucoCorners.size()
00193     << ", allCorners = " << allCorners.size() << "\n";
00194     #endif
00195 }
00196
00197 //
00198 // 標本と校正結果を破棄する
00199 //
00200 void Calibration::discardCorners()
00201 {
00202     // 記録した標本を消去する
00203     allCorners.clear();
00204     allIds.clear();
00205
00206     // 校正結果を消去する
00207     cameraMatrix.release();
00208     distCoeffs.release();
00209
00210     // 検出したコーナー数の合計を 0 にする
00211     totalCorners = 0;
00212
00213     // 校正の計算結果を再利用しない
00214     calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00215 }
00216
00217 //
00218 // 校正する
00219 //
00220 bool Calibration::calibrate()
00221 {
00222     // 再投影誤差はとりあえず 0 にしておく
00223     repError = 0.0f;
00224
00225     // コーナーを合計 6 つ以上検出できていれば
00226     if (allCorners.size() >= 6) try
00227     {
00228         // CALIB_USE_INTRINSIC_GUESS が設定されていない場合に、
00229         // fx と fy をしてしたアスペクト比に強制する
00230         if (calibrationFlags & cv::CALIB_FIX_ASPECT_RATIO)
00231         {
00232             const auto aspect{ static_cast<double>(size.width) / size.height };
00233             cameraMatrix = cv::Mat::eye(3, 3, CV_64F);
00234             cameraMatrix.at<double>(0, 0) = aspect;
00235         }
00236
00237         // ChArUco Board の姿勢 (使わないので捨ててしまう)
00238         std::vector<cv::Mat> boardRvecs, boardTvecs;
00239
00240         // 取得した全てのコーナーからカメラパラメータを推定する

```

```

00241     repError = cv::calibrateCamera(allObjectPoints, allImagePoints, size,
00242     cameraMatrix, distCoeffs, boardRvecs, boardTvecs, cv::noArray(),
00243     cv::noArray(), cv::noArray(), calibrationFlags);
00244 }
00245 catch (const cv::Exception&)
00246 {
00247     // 校正に失敗した場合は計算結果を捨てる
00248     discardCorners();
00249
00250     // 校正に失敗したことを報告する
00251     return false;
00252 }
00253 // 校正の計算結果を再利用する
00254 calibrationFlags |= cv::CALIB_USE_INTRINSIC_GUESS;
00255 // 校正に成功したことを報告する
00256 return true;
00257 }
00258 //
00259 // 回転ベクトルから姿勢の変換行列を求める
00260 //
00261 //
00262 //
00263 //
00264 GgMatrix Calibration::RvecTvecToPose(const cv::Vec3d& rvec, const cv::Vec3d& tvec)
00265 {
00266     #if defined(USE_RODRIGUES)
00267         // 回転軸と回転角から回転の変換行列を求める
00268         cv::Mat_<double> r(3, 3);
00269         cv::Rodrigues(rvec, r);
00270
00271         // 姿勢の変換行列
00272         return GgMatrix
00273         {
00274             static_cast<GLfloat>(r[0][0]),
00275             static_cast<GLfloat>(r[1][0]),
00276             static_cast<GLfloat>(r[2][0]),
00277             0.0f,
00278             static_cast<GLfloat>(r[0][1]),
00279             static_cast<GLfloat>(r[1][1]),
00280             static_cast<GLfloat>(r[2][1]),
00281             0.0f,
00282             static_cast<GLfloat>(r[0][2]),
00283             static_cast<GLfloat>(r[1][2]),
00284             static_cast<GLfloat>(r[2][2]),
00285             0.0f,
00286             static_cast<GLfloat>(tvec[0]),
00287             static_cast<GLfloat>(tvec[1]),
00288             static_cast<GLfloat>(tvec[2]),
00289             1.0f
00290         };
00291     #else
00292         // 回転角
00293         const auto d{ cv::norm(rvec) };
00294
00295         // 回転軸ベクトル
00296         const auto rx{ static_cast<GLfloat>(rvec[0] / d) };
00297         const auto ry{ static_cast<GLfloat>(rvec[1] / d) };
00298         const auto rz{ static_cast<GLfloat>(rvec[2] / d) };
00299
00300         // 平行移動量
00301         const auto tx{ static_cast<GLfloat>(tvec[0]) };
00302         const auto ty{ static_cast<GLfloat>(tvec[1]) };
00303         const auto tz{ static_cast<GLfloat>(tvec[2]) };
00304
00305         // 姿勢の変換行列
00306         return ggTranslate(tx, ty, tz).rotate(rx, ry, rz, static_cast<GLfloat>(d));
00307     #endif
00308 }
00309 //
00310 //
00311 // ArUco Marker の3次元姿勢の変換行列を求める
00312 //
00313 void Calibration::getAllMarkerPoses(float markerLength, std::map<int, GgMatrix>& poses)
00314 {
00315     // 各マーカに対応する3次元空間の点
00316     const float markerCenter = markerLength * 0.5f;
00317     std::vector<cv::Point3f> markerObjPoints;
00318     markerObjPoints.push_back(cv::Point3f(-markerCenter, markerCenter, 0.0f));
00319     markerObjPoints.push_back(cv::Point3f(markerCenter, markerCenter, 0.0f));
00320     markerObjPoints.push_back(cv::Point3f(markerCenter, -markerCenter, 0.0f));
00321     markerObjPoints.push_back(cv::Point3f(-markerCenter, -markerCenter, 0.0f));
00322
00323     // 個々のマーカについて
00324     for (size_t i = 0; i < corners.size(); ++i)
00325     {
00326         // マーカーのコーナー検出位置から3次元姿勢（回転・平行移動）を推定する
00327         cv::Vec3d rvec, tvec;

```

```

00328     cv::solvePnP(markerObjPoints, corners[i], cameraMatrix, distCoeffs, rvec, tvec, false,
00329                 cv::SOLVEPNP_IPPE_SQUARE);
00329
00330     // 各マーカの姿勢の変換行列を求める
00331     poses[ids[i]] = RvecTvecToPose(rvec, tvec);
00332 }
00333 }
00334
00335 //
00336 // カメラパラメータの JSON オブジェクトから数値の配列を取得する
00337 //
00338 static bool getMatrix(const picojson::object& object,
00339                      const std::string& key, cv::Mat& mat, int cols, int rows)
00340 {
00341     // key に一致するオブジェクトを探す
00342     const auto&& value{ object.find(key) };
00343
00344     // オブジェクトが無いか配列でなかったら戻る
00345     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00346
00347     // 配列を取り出す
00348     const auto& array{ value->second.get<picojson::array>() };
00349
00350     // 配列の要素数とデータの格納先の行列の要素数が一致していなければ戻る
00351     if (static_cast<size_t>(cols) * rows != array.size()) return false;
00352
00353     // カメラ行列の要素を確保する
00354     mat = cv::Mat::zeros(rows, cols, CV_64F);
00355
00356     // 配列の要素について
00357     for (size_t i = 0; i < array.size(); ++i)
00358     {
00359         // 行列の要素の位置
00360         const auto x{ static_cast<int>(i % cols) };
00361         const auto y{ static_cast<int>(i / cols) };
00362
00363         // 要素が数値なら格納する
00364         if (array[i].is<double>()) mat.at<double>(y, x) = array[i].get<double>();
00365     }
00366
00367     return true;
00368 }
00369
00370 //
00371 // カメラパラメータの JSON オブジェクトから数値の配列を取得する
00372 //
00373 static void setMatrix(picojson::object& object,
00374                      const std::string& key, const cv::Mat& mat)
00375 {
00376     // picojson の配列
00377     picojson::array array;
00378
00379     // 配列の要素について
00380     for (size_t i = 0; i < mat.total(); ++i)
00381     {
00382         // 行列の要素の位置
00383         const auto x{ static_cast<int>(i % mat.cols) };
00384         const auto y{ static_cast<int>(i / mat.cols) };
00385
00386         // 要素を picojson::array に追加する
00387         array.emplace_back(picojson::value(mat.at<double>(y, x)));
00388     }
00389
00390     // オブジェクトに追加する
00391     object.emplace(key, array);
00392 }
00393
00394 //
00395 // ファイルからキャリブレーションパラメータを読み込む
00396 //
00397 bool Calibration::loadParameters(const std::string& filename)
00398 {
00399     // パラメータファイルの読み込み
00400     std::ifstream json{ filename };
00401     if (!json) return false;
00402
00403     // JSON の読み込み
00404     picojson::value value;
00405     json >> value;
00406     json.close();
00407
00408     // 構成内容の取り出し
00409     const auto& object{ value.get<picojson::object>() };
00410
00411     // オブジェクトが空だったらエラー
00412     if (object.empty()) return false;
00413

```

```

00414 // カメラ行列
00415 if (!getMatrix(object, "camera matrix", cameraMatrix, 3, 3)) return false;
00416
00417 // 歪み定数
00418 if (!getMatrix(object, "distortion", distCoeffs, 5, 1)) return false;
00419
00420 // 再投影誤差
00421 getValue(object, "error", repError);
00422
00423 // 較正の計算結果を再利用しない
00424 calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00425
00426 // キャリブレーションパラメータの読み込み
00427 return true;
00428 }
00429
00430 //
00431 // キャリブレーションパラメータをファイルに保存する
00432 //
00433 bool Calibration::saveParameters(const std::string& filename) const
00434 {
00435     // 設定値を保存する
00436     std::ofstream config{ filename };
00437     if (!config) return false;
00438
00439     // オブジェクト
00440     picojson::object object;
00441
00442     // カメラ行列
00443     setMatrix(object, "camera matrix", cameraMatrix);
00444
00445     // 歪み定数
00446     setMatrix(object, "distortion", distCoeffs);
00447
00448     // 再投影誤差
00449     setValue(object, "error", repError);
00450
00451     // 構成をシリアライズして保存
00452     picojson::value v{ object };
00453     config << v.serialize(true);
00454     config.close();
00455
00456     // キャリブレーションパラメータの書き込み
00457     return true;
00458 }
00459
00460 // ArUco Marker 辞書のリスト
00461 const std::map<const std::string, const cv::aruco::PredefinedDictionaryType>
    Calibration::dictionaryList
00462 {
00463     { "DICT_4X4_50", cv::aruco::DICT_4X4_50 },
00464     { "DICT_4X4_100", cv::aruco::DICT_4X4_100 },
00465     { "DICT_4X4_250", cv::aruco::DICT_4X4_250 },
00466     { "DICT_4X4_1000", cv::aruco::DICT_4X4_1000 },
00467     { "DICT_5X5_50", cv::aruco::DICT_5X5_50 },
00468     { "DICT_5X5_100", cv::aruco::DICT_5X5_100 },
00469     { "DICT_5X5_250", cv::aruco::DICT_5X5_250 },
00470     { "DICT_5X5_1000", cv::aruco::DICT_5X5_1000 },
00471     { "DICT_6X6_50", cv::aruco::DICT_6X6_50 },
00472     { "DICT_6X6_100", cv::aruco::DICT_6X6_100 },
00473     { "DICT_6X6_250", cv::aruco::DICT_6X6_250 },
00474     { "DICT_6X6_1000", cv::aruco::DICT_6X6_1000 },
00475     { "DICT_7X7_50", cv::aruco::DICT_7X7_50 },
00476     { "DICT_7X7_100", cv::aruco::DICT_7X7_100 },
00477     { "DICT_7X7_250", cv::aruco::DICT_7X7_250 },
00478     { "DICT_7X7_1000", cv::aruco::DICT_7X7_1000 },
00479     { "DICT_ARUCO_ORIGINAL", cv::aruco::DICT_ARUCO_ORIGINAL },
00480     { "DICT_APRILTAG_16h5", cv::aruco::DICT_APRILTAG_16h5 },
00481     { "DICT_APRILTAG_25h9", cv::aruco::DICT_APRILTAG_25h9 },
00482     { "DICT_APRILTAG_36h10", cv::aruco::DICT_APRILTAG_36h10 },
00483     { "DICT_APRILTAG_36h11", cv::aruco::DICT_APRILTAG_36h11 }
00484 };

```

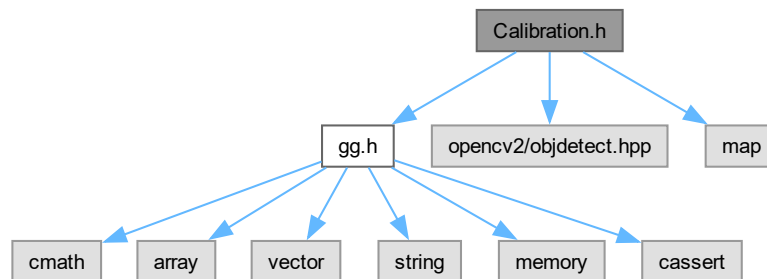
9.9 Calibration.h ファイル

```

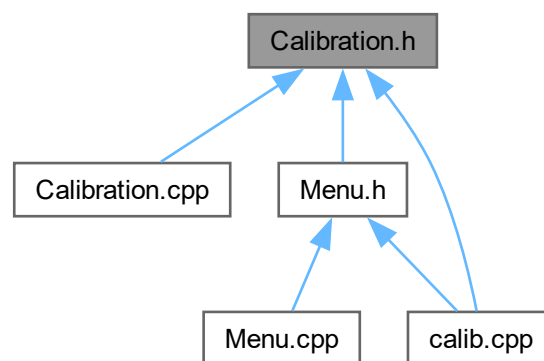
#include "gg.h"
#include <opencv2/objdetect.hpp>
#include <map>

```


Calibration.h の依存先関係図:



被依存関係図:



クラス

- class [Calibration](#)

9.9.1 詳解

較正クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Calibration.h](#) に定義があります。

9.10 Calibration.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00015 // OpenCV ArUco & ChArUco (modern OpenCV 4.7+)
00016 #include <opencv2/objdetect.hpp>
00017
00018 // 標準ライブラリ
00019 #include <map>
00020
00024 class Calibration
00025 {
00027     cv::aruco::Dictionary dictionary;
00028
00030     cv::Ptr<cv::aruco::ArucoDetector> detector;
00031
00033     cv::Ptr<cv::aruco::CharucoBoard> board;
00034
00036     cv::Ptr<cv::aruco::CharucoDetector> boardDetector;
00037
00039     std::vector<std::vector<cv::Point2f>> corners, rejected;
00040     std::vector<int> ids;
00041
00043     std::vector<cv::Point2f> charucoCorners;
00044     std::vector<int> charucoIds;
00045     std::vector<cv::Point3f> objectPoints;
00046     std::vector<cv::Point2f> imagePoints;
00047
00049     std::vector<std::vector<cv::Point2f>> allCorners;
00050     std::vector<std::vector<int>> allIds;
00051     std::vector<std::vector<cv::Point2f>> allImagePoints;
00052     std::vector<std::vector<cv::Point3f>> allObjectPoints;
00053
00055     cv::Mat cameraMatrix;
00056
00058     cv::Mat distCoeffs;
00059
00061     cv::Size size;
00062
00064     double repError;
00065
00067     int totalCorners;
00068
00070     int calibrationFlags;
00071
00072 public:
00073
00080     Calibration(const std::string& dictionaryName, const std::array<float, 2>& length);
00081
00087     Calibration(const Calibration& calibration) = delete;
00088
00092     virtual ~Calibration();
00093
00099     Calibration& operator=(const Calibration& calibration) = delete;
00100
00106     void createBoard(const std::array<float, 2>& length);
00107
00114     void setDictionary(const std::string& dictionaryName, const std::array<float, 2>& length);
00115
00123     void drawBoard(cv::Mat& boardImage, int width, int height);
00124
00130     void detectBoard(cv::Mat& image);
00131
00138     void detectMarkers(cv::Mat& image, float markerLength);
00139
00143     void recordCorners();
00144
00148     void discardCorners();
00149
00155     bool calibrate();
00156
00162     auto getCornersCount() const
00163     {
00164         return static_cast<int>(corners.size());
00165     }
00166
00172     auto getSampleCount() const
00173     {
00174         return static_cast<int>(allCorners.size());
00175     }

```

```

00176
00182 auto getTotalCount() const
00183 {
00184     return totalCorners;
00185 }
00186
00192 const auto& getCameraMatrix() const
00193 {
00194     return cameraMatrix;
00195 }
00196
00202 const auto& getDistortionCoefficients() const
00203 {
00204     return distCoeffs;
00205 }
00206
00212 auto getReprojectionError() const
00213 {
00214     return repError;
00215 }
00216
00222 auto finished()
00223 {
00224     return cameraMatrix.total() == 9 && distCoeffs.total() == 5;
00225 }
00226
00234 GgMatrix RvecTvecToPose(const cv::Vec3d& rvec, const cv::Vec3d& tvec);
00235
00245 void getAllMarkerPoses(float markerLength, std::map<int, GgMatrix>& poses);
00246
00253 bool loadParameters(const std::string& filename);
00254
00261 bool saveParameters(const std::string& filename) const;
00262
00264 static const std::map<const std::string, const cv::aruco::PredefinedDictionaryType> dictionaryList;
00265 };

```

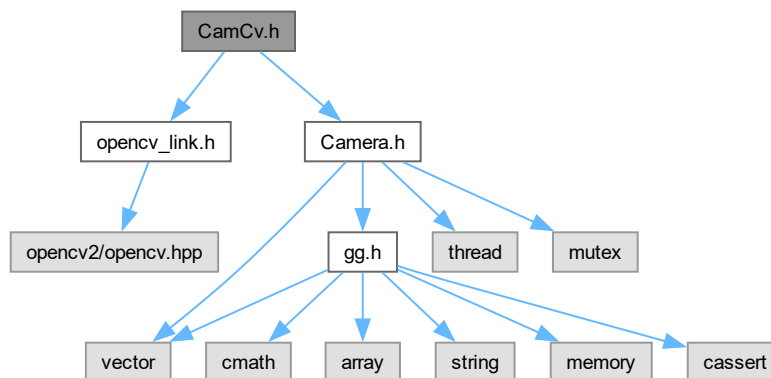
9.11 CamCv.h ファイル

```

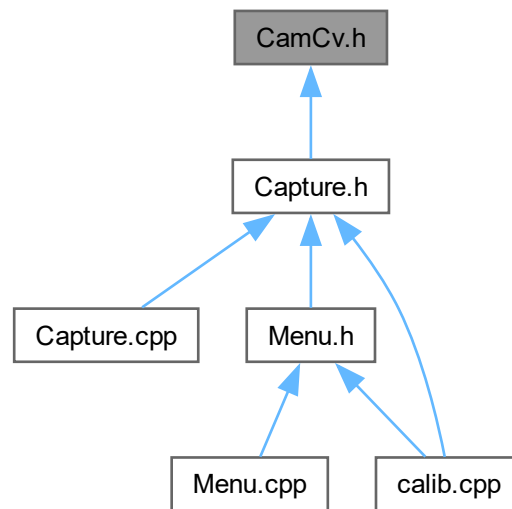
#include "opencv_link.h"
#include "Camera.h"

```

CamCv.h の依存先関係図:



被依存関係図:



クラス

- class [CamCv](#)

9.11.1 詳解

OpenCV を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

December 27, 2022

[CamCv.h](#) に定義があります。

9.12 CamCv.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // OpenCV へのリンクとインクルード
00012 #include "opencv_link.h"
00013
00014 // カメラ関連 of 処理
00015 #include "Camera.h"
00016
00020 class CamCv : public Camera
00021 {
00023     cv::VideoCapture camera;
00024
00026     cv::Mat cvFrame;
00027
00029     cv::Mat cvImage;
00030
```

```

00032 double elapsedTime;
00033
00035 int exposure, gain;
00036
00046 bool init(int initialWidth, int initialHeight, double initialFps, const char* fourcc = "")
00047 {
00048     // カメラのコーデック・解像度・フレームレートを設定する
00049     if (fourcc[0] != '\\0') camera.set(cv::CAP_PROP_FOURCC,
00050         cv::VideoWriter::fourcc(fourcc[0], fourcc[1], fourcc[2], fourcc[3]));
00051     if (initialWidth > 0) camera.set(cv::CAP_PROP_FRAME_WIDTH, initialWidth);
00052     if (initialHeight > 0) camera.set(cv::CAP_PROP_FRAME_HEIGHT, initialHeight);
00053     if (initialFps > 0.0) camera.set(cv::CAP_PROP_FPS, initialFps);
00054
00055     // fps が 0 なら逆数をカメラの遅延に使う
00056     const auto fps{ camera.get(cv::CAP_PROP_FPS) };
00057     if (fps > 0.0) interval = 1000.0 / fps;
00058
00059     // ムービーファイルのインポイント・アウトポイントの初期値とフレーム数
00060     in = camera.get(cv::CAP_PROP_POS_FRAMES);
00061     out = total = camera.get(cv::CAP_PROP_FRAME_COUNT);
00062
00063     // 経過時間
00064     elapsedTime = 0.0;
00065
00066     // カメラから最初のフレームをキャプチャできなかったらカメラは使えない
00067     if (!camera.grab()) return false;
00068
00069     // カメラの利得と露出を取得する
00070     gain = static_cast<GLsizei>(camera.get(cv::CAP_PROP_GAIN));
00071     exposure = static_cast<GLsizei>(camera.get(cv::CAP_PROP_EXPOSURE) * 10.0);
00072
00073     // フレームを取り出してキャプチャ用のメモリを確保する
00074     camera.retrieve(cvFrame);
00075
00076     #if defined(DEBUG)
00077     char codec[5]{ 0, 0, 0, 0, 0 };
00078     getCodec(codec);
00079     std::cerr << "in:" << in << ", out:" << out
00080         << ", width:" << cvFrame.cols << ", height:" << cvFrame.rows
00081         << ", fourcc: " << codec << "\n";
00082     #endif
00083
00084     // キャプチャしたデータを一時メモリにコピーする
00085     cvFrame.copyTo(cvImage);
00086
00087     // 基底クラスのバッファとメンバを更新
00088     width = cvFrame.cols;
00089     height = cvFrame.rows;
00090     channels = cvFrame.channels();
00091     {
00092         const size_t size = cvFrame.total() * cvFrame.elemSize();
00093         frame.resize(size);
00094         memcpy(frame.data(), cvFrame.data, size);
00095         image.resize(size);
00096         memcpy(image.data(), cvImage.data, size);
00097     }
00098
00099     // フレームがキャプチャされたことを記録する
00100     captured = true;
00101
00102     // カメラが使える
00103     return true;
00104 }
00105
00109 void capture()
00110 {
00111     // 再生開始時刻
00112     auto startTime{ glfwGetTime() };
00113
00114     // スレッドが実行可の間
00115     while (running)
00116     {
00117         // フレームを取り出せたら true
00118         auto status{ (total <= 0.0 || camera.get(cv::CAP_PROP_POS_FRAMES) < out) && camera.grab() };
00119
00120         // ムービーファイルでないかムービーファイルの終端でなければ次のフレームを取り出して
00121         if (status && camera.retrieve(cvFrame))
00122         {
00123             // 一時メモリをロックしてから
00124             std::lock_guard<std::mutex> lock{ mtx };
00125
00126             // キャプチャしたデータを一時メモリにコピーして
00127             cvFrame.copyTo(cvImage);
00128
00129             // 基底クラスのバッファとメンバを更新
00130             width = cvFrame.cols;
00131             height = cvFrame.rows;

```

```

00132         channels = cvFrame.channels();
00133     {
00134         const size_t size = cvFrame.total() * cvFrame.elemSize();
00135         frame.resize(size);
00136         memcpy(frame.data(), cvFrame.data, size);
00137         image.resize(size);
00138         memcpy(image.data(), cvImage.data, size);
00139     }
00140
00141     // 新しいフレームがキャプチャされたことを通知する
00142     captured = true;
00143 }
00144
00145 // 遅延時間
00146 auto deferred{ 0.0 };
00147
00148 // ムービーファイルから入力しているとき
00149 if (total > 0.0)
00150 {
00151     // ムービーファイルの終端に到達していたら
00152     if (!status)
00153     {
00154         // インポイントまで巻き戻す
00155         camera.set(cv::CAP_PROP_POS_FRAMES, in);
00156
00157         // 経過時間を戻す
00158         elapsedTime = 0.0;
00159
00160         // 開始時間を更新する
00161         startTime = glfwGetTime();
00162     }
00163     else
00164     {
00165         // 現在のフレームのインポイントからの経過時間
00166         const auto pos{ camera.get(cv::CAP_PROP_POS_MSEC) - in };
00167
00168         // 再生位置の次のフレームの時刻に対する経過時間
00169         const auto now{ (elapsedTime + glfwGetTime() - startTime) * 1000.0 + interval };
00170
00171         // 遅延時間はフレームの経過時間と現在の経過時間の差
00172         deferred = pos - now;
00173     }
00174 }
00175
00176 // 遅延時間あれば
00177 if (deferred > 0.0)
00178 {
00179     // 待つ
00180     std::this_thread::sleep_for(std::chrono::milliseconds(static_cast<int>(deferred)));
00181 }
00182 }
00183
00184 // 再生停止までの経過時間を積算する
00185 elapsedTime += glfwGetTime() - startTime;
00186 }
00187
00188 public:
00189
00190     CamCv()
00191         : elapsedTime{ 0.0 }
00192         , exposure{ 0 }
00193         , gain{ 0 }
00194     {}
00195
00196     virtual ~CamCv()
00197     {
00198     }
00199
00200     auto open(int device, int width = 0, int height = 0, double fps = 0.0, const char* fourcc = "", int
pref = cv::CAP_ANY)
00201     {
00202         // カメラを開いて初期化する
00203         return camera.open(device, pref) && init(width, height, fps, fourcc);
00204     }
00205
00206     void close()
00207     {
00208         // キャプチャデバイスを開放する
00209         camera.release();
00210
00211         // キャプチャデバイスを閉じる
00212         Camera::close();
00213     }
00214
00215     auto open(const std::string& file, int width = 0, int height = 0, double fps = 0.0, const char*
fourcc = "", int pref = cv::CAP_ANY)
00216     {

```

```

00246     // ファイル／ネットワークを開いて初期化する
00247     return camera.open(file, pref) && init(width, height, fps, fourcc);
00248 }
00249
00255 virtual double getFps() const
00256 {
00257     return camera.get(cv::CAP_PROP_FPS);
00258 }
00259
00265 auto getCodec() const
00266 {
00267     return static_cast<unsigned int>(camera.get(cv::CAP_PROP_FOURCC));
00268 }
00269
00275 void getCodec(char* fourcc) const
00276 {
00277     auto cc{ getCodec() };
00278     for (int i = 0; i < 4; ++i)
00279     {
00280         fourcc[i] = static_cast<char>(cc & 0x7f);
00281         if (!isalnum(fourcc[i])) fourcc[i] = '?';
00282         cc >>= 8;
00283     }
00284 }
00285
00291 auto getPosition() const
00292 {
00293     return camera.get(cv::CAP_PROP_POS_FRAMES);
00294 }
00295
00301 void setPosition(double frame)
00302 {
00303     camera.set(cv::CAP_PROP_POS_FRAMES, frame);
00304 }
00305
00311 void setExposure(double exposure)
00312 {
00313     if (camera.isOpened()) camera.set(cv::CAP_PROP_EXPOSURE, exposure);
00314 }
00315
00319 void increaseExposure()
00320 {
00321     setExposure(++exposure * 0.1);
00322 }
00323
00327 void decreaseExposure()
00328 {
00329     setExposure(--exposure * 0.1);
00330 }
00331
00337 void setGain(double gain)
00338 {
00339     if (camera.isOpened()) camera.set(cv::CAP_PROP_GAIN, gain);
00340 }
00341
00345 void increaseGain()
00346 {
00347     setGain(++gain);
00348 }
00349
00353 void decreaseGain()
00354 {
00355     setGain(--gain);
00356 }
00357 };

```

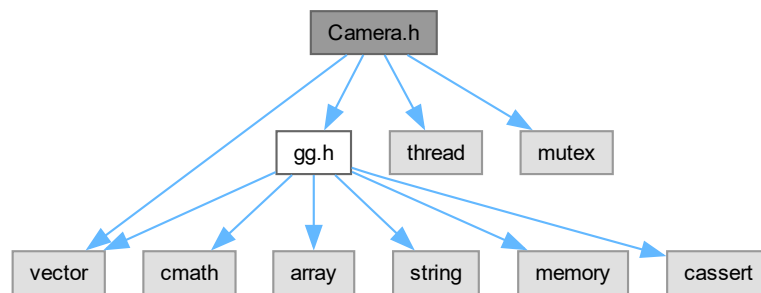
9.13 Camera.h ファイル

```

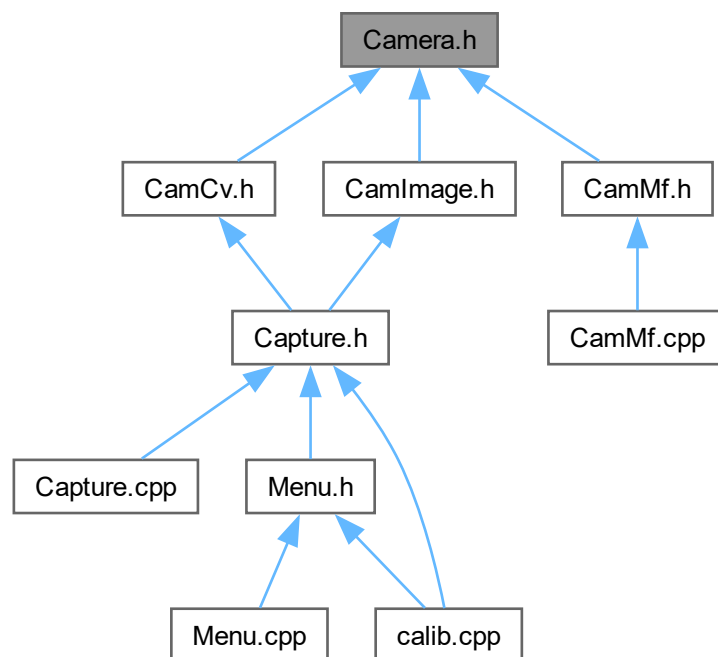
#include "gg.h"
#include <vector>
#include <thread>
#include <mutex>

```

Camera.h の依存先関係図:



被依存関係図:



クラス

- class [Camera](#)

9.13.1 詳解

キャプチャデバイス関連の基底クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Camera.h](#) に定義があります。

9.14 Camera.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 #include <vector>
00014
00015 // 非同期処理
00016 #include <thread>
00017 #include <mutex>
00018
00022 class Camera
00023 {
00024 protected:
00025
00027     double total;
00028
00030     double interval;
00031
00033     int width;
00034
00036     int height;
00037
00039     int channels;
00040
00042     std::vector<GLubyte> frame;
00043
00045     std::vector<GLubyte> image;
00046
00048     bool captured;
00049
00051     std::thread thr;
00052
00054     std::mutex mtx;
00055
00057     bool running;
00058
00065     virtual void capture()
00066     {
00067         // 継承されていなければスレッドを起動しない
00068         stop();
00069     }
00070
00071 public:
00072
00074     double in;
00075
00077     double out;
00078
00082     Camera()
00083         : total{ -1.0 }
00084         , interval{ 10.0 }
00085         , width{ 0 }
00086         , height{ 0 }
00087         , channels{ 0 }
00088         , captured{ false }
00089         , running{ false }
00090         , in{ -1.0 }
00091         , out{ -1.0 }
00092     {
00093     }
00094
00100     Camera(const Camera& camera) = delete;
00101
00105     virtual ~Camera()
00106     {
00107         // キャプチャスレッドを停止する
00108         stop();
```

```

00109
00110 // キャプチャデバイスの使用を終了する
00111 close();
00112
00113 // 一時メモリを空にする
00114 image.clear();
00115 }
00116
00123 Camera& operator=(const Camera& camera) = delete;
00124
00128 void start()
00129 {
00130 // スレッドが起動状態であることを記録しておく
00131 running = true;
00132
00133 // スレッドを起動する
00134 thr = std::thread([&] { this->capture(); });
00135 }
00136
00140 void stop()
00141 {
00142 // キャプチャスレッドが実行中なら
00143 if (running)
00144 {
00145 // キャプチャスレッドのループを止めて
00146 running = false;
00147
00148 // 合流する
00149 thr.join();
00150 }
00151 }
00152
00158 void transmit(GLuint buffer)
00159 {
00160 // 新しいフレームが取得されているときカメラのロックが成功したら
00161 std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);
00162 if (captured && lock.owns_lock())
00163 {
00164 // データの長さを計算して
00165 const auto length{ image.size() * sizeof(GLubyte) };
00166
00167 // フレームをピクセルバッファオブジェクトに転送して
00168 glBindBuffer(GL_PIXEL_PACK_BUFFER, buffer);
00169 glBufferSubData(GL_PIXEL_PACK_BUFFER, 0, length, image.data());
00170 glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00171
00172 // 次のフレームの取得を待つ
00173 captured = false;
00174 }
00175 }
00176
00182 void transmit(std::vector<GLubyte>& buffer)
00183 {
00184 // 新しいフレームが取得されているときカメラのロックが成功したら
00185 std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);
00186 if (captured && lock.owns_lock())
00187 {
00188 // データの長さを計算して
00189 const auto length{ image.size() };
00190
00191 // 呼び出し先のサイズを合わせてから
00192 buffer.resize(length);
00193
00194 // フレームを呼び出し元にコピーして
00195 memcpy(buffer.data(), image.data(), length);
00196
00197 // 次のフレームの取得を待つ
00198 captured = false;
00199 }
00200 }
00201
00207 template <typename MatType>
00208 void transmit(MatType& buffer)
00209 {
00210 // 新しいフレームが取得されているときカメラのロックが成功したら
00211 std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);
00212 if (captured && lock.owns_lock())
00213 {
00214 // 呼び出し元にコピーして
00215 // cv::Mat の型番号 (CV_8UC1~CV_8UC4) は (channels - 1) << 3 で表されます
00216 buffer.create(height, width, ((channels - 1) << 3));
00217 memcpy(buffer.data, image.data(), image.size());
00218
00219 // 次のフレームの取得を待つ
00220 captured = false;
00221 }
00222 }

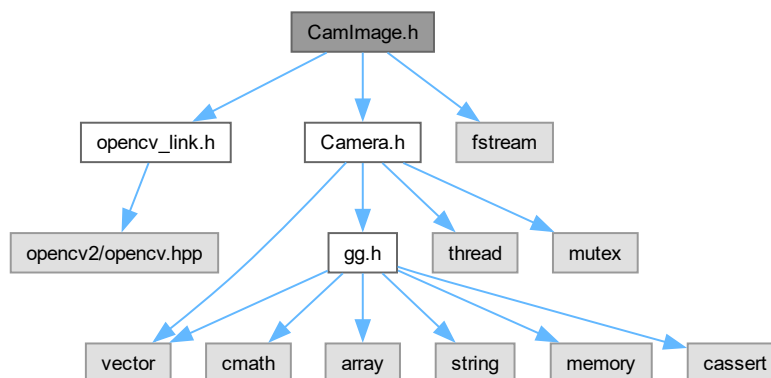
```

```
00223
00227 virtual void close()
00228 {
00229     // フレームを取得していないことにする
00230     captured = false;
00231 }
00232
00238 auto isRunning() const
00239 {
00240     return running;
00241 }
00242
00248 std::array<int, 2> getSize() const
00249 {
00250     return std::array<int, 2>{ width, height };
00251 }
00252
00258 auto getWidth() const
00259 {
00260     return width;
00261 }
00262
00268 auto getHeight() const
00269 {
00270     return height;
00271 }
00272
00278 auto getChannels() const
00279 {
00280     return channels;
00281 }
00282
00288 auto getFrames() const
00289 {
00290     return total;
00291 }
00292
00298 virtual double getFps() const
00299 {
00300     return 1000.0 / interval;
00301 }
00302
00306 virtual void increaseExposure()
00307 {
00308 }
00309
00313 virtual void decreaseExposure()
00314 {
00315 }
00316
00320 virtual void increaseGain()
00321 {
00322 }
00323
00327 virtual void decreaseGain()
00328 {
00329 }
00330 };
```

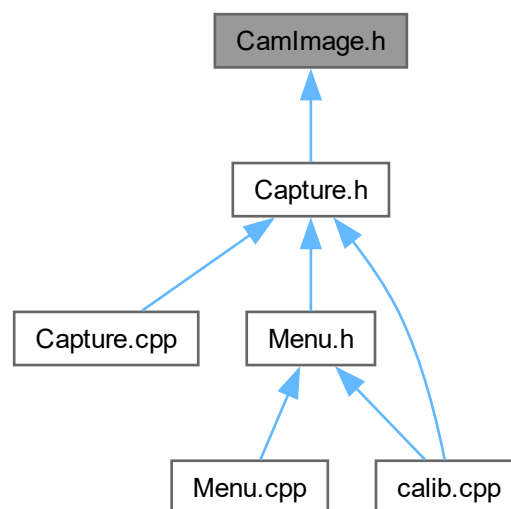
9.15 CamImage.h ファイル

```
#include "opencv_link.h"
#include "Camera.h"
#include "fstream"
```

CamImage.h の依存先関係図:



被依存関係図:



クラス

- class [CamImage](#)

9.15.1 詳解

OpenCV を使って画像ファイルを読み込むクラス

著者

Kohe Tokoi

日付

November 15, 2022

[CamImage.h](#) に定義があります。

9.16 CamImage.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // OpenCV へのリンクとインクルード
00012 #include "opencv_link.h"
00013
00014 // カメラ関連の処理
00015 #include "Camera.h"
00016
00017 // ファイル入出力
00018 #include "fstream"
00019
00023 class CamImage : public Camera
00024 {
00025 public:
00026
00030 CamImage()
00031 {
00032 }
00033
00040 CamImage(std::string& filename, bool flip = false)
00041 {
00042     open(filename, flip);
00043 }
00044
00048 virtual ~CamImage()
00049 {
00050 }
00051
00058 bool open(const std::string& filename, bool flip = false)
00059 {
00060     // 画像ファイルを読み込む
00061     cv::Mat cvFrame;
00062     if (!load(filename, cvFrame)) return false;
00063
00064     // 必要なら上下を反転する
00065     if (flip) cv::flip(cvFrame, cvFrame, 1);
00066
00067     // 読み出したデータを一時メモリにコピーする
00068     cv::Mat cvImage;
00069     cvFrame.copyTo(cvImage);
00070
00071     // 基底クラスのバッファとメンバを更新
00072     width = cvFrame.cols;
00073     height = cvFrame.rows;
00074     channels = cvFrame.channels();
00075     {
00076         const size_t size = cvFrame.total() * cvFrame.elemSize();
00077         frame.resize(size);
00078         memcpy(frame.data(), cvFrame.data, size);
00079         image.resize(size);
00080         memcpy(image.data(), cvImage.data, size);
00081     }
00082
00083     // 画像が読み込まれたことを記録する
00084     captured = true;
00085
00086     // 画像ファイルが開けた
00087     return true;
00088 }
00089
00095 bool isOpened() const
00096 {
00097     // 画像が読み込めていたら true
00098     return !image.empty();
00099 }
00100
00108 static bool load(const std::string& filename, cv::Mat& frame)
00109 {
00110     // 画像ファイルをバイナリモードで開いて読み込み位置を最後に移動する
00111     std::ifstream file(UTF8ToTChar(filename),
00112         std::ifstream::in |
00113         std::ifstream::binary |
00114         std::ifstream::ate);

```

```

00115
00116 // 画像ファイルが開けたら
00117 if (file.is_open())
00118 {
00119     // 画像ファイルを読み込むメモリを確保する
00120     std::vector<char> buffer(static_cast<std::vector<char>::size_type>(file.tellg()));
00121
00122     // 画像ファイルの先頭から全部読み込む
00123     file.seekg(0, std::ifstream::beg);
00124     file.read(buffer.data(), buffer.size());
00125
00126     // 画像ファイルを閉じる
00127     file.close();
00128
00129     // 画像ファイルが読み込めたら
00130     if (file.good())
00131     {
00132         // 読み込んだ画像データを復号して返す
00133         frame = cv::imdecode(buffer, cv::IMREAD_COLOR);
00134         return true;
00135     }
00136 }
00137
00138 // 読み込めなかった
00139 return false;
00140 }
00141
00149 static bool save(const std::string& filename, const cv::Mat& image)
00150 {
00151     // ファイル名の拡張子の場所を取り出す
00152     const auto pos{ filename.find_last_of('.') };
00153
00154     // 拡張子があればそれを取り出し、無ければ ".png" にする
00155     const std::string ext{ pos != std::string::npos ? filename.substr(pos) : ".png" };
00156
00157     // 画像の符号化に失敗したら戻る
00158     std::vector<uchar> buffer;
00159     if (!cv::imencode(ext, image, buffer)) return false;
00160
00161     // 画像ファイルをバイナリモードで開く
00162     std::ofstream file(Utf8ToTChar(filename),
00163         std::ifstream::out |
00164         std::ifstream::binary);
00165
00166     // 画像ファイルが開けなかったら戻る
00167     if (file.fail()) return false;
00168
00169     // 画像データをファイルに書き込む
00170     const auto ptr{ reinterpret_cast<char*>(buffer.data()) };
00171     file.write(ptr, buffer.size());
00172
00173     // 画像ファイルを閉じる
00174     file.close();
00175
00176     // 読み込めたかどうかを返す
00177     return file.good();
00178 }
00179 };

```

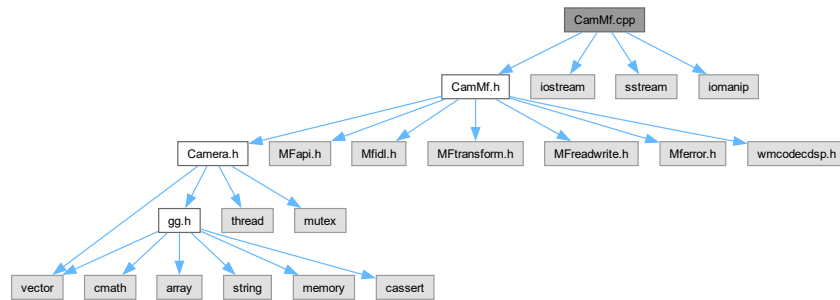
9.17 CamMf.cpp ファイル

```

#include "CamMf.h"
#include <iostream>
#include <sstream>
#include <iomanip>

```

CamMf.cpp の依存先関係図:



関数

- `template<class T>`
void [SafeRelease](#) (T **ppT)
- `std::string` [SubTypeToName](#) (const GUID &subType)
- HRESULT [HandleStreamChange](#) (IMFTransform *pDecoder)

9.17.1 詳解

Microsoft Media Foundation を使ったビデオキャプチャクラスの実装

著者

Kohe Tokoi

日付

December 24, 2024

[CamMf.cpp](#) に定義があります。

9.17.2 関数詳解

9.17.2.1 HandleStreamChange()

```
HRESULT HandleStreamChange (
    IMFTransform * pDecoder)
```

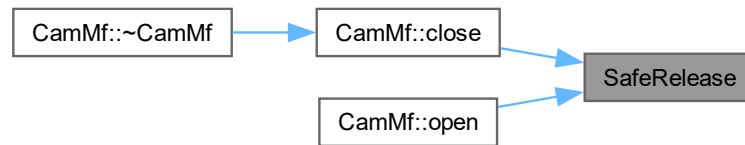
[CamMf.cpp](#) の 620 行目に定義があります。

9.17.2.2 SafeRelease()

```
template<class T>
void SafeRelease (
    T ** ppT)
```

[CamMf.cpp](#) の 26 行目に定義があります。

被呼び出し関係図:



9.17.2.3 SubTypeToName()

```
std::string SubTypeToName (
    const GUID & subType)
```

CamMf.cpp の 38 行目に定義があります。

9.18 CamMf.cpp

[詳解]

```

00001
00008 #include "CamMf.h"
00009 #include <iostream>
00010 #include <sstream>
00011 #include <iomanip>
00012
00013 // Microsoft Media Foundation
00014 #pragma comment(lib, "MF.lib")
00015 #pragma comment(lib, "MFplat.lib")
00016 #pragma comment(lib, "MFuuid.lib")
00017 #pragma comment(lib, "MFreadwrite.lib")
00018 #pragma comment(lib, "wmcodecdspuuid.lib")
00019
00020 // COM ライブラリの初期化と終了を行うオブジェクト
00021 CamMf::ComInitializer CamMf::ComInitializer::instance;
00022
00023 //
00024 // メモリの開放
00025 //
00026 template <class T> void SafeRelease(T** ppT)
00027 {
00028     if (*ppT)
00029     {
00030         (*ppT)->Release();
00031         *ppT = nullptr;
00032     }
00033 }
00034
00035 //
00036 // GUID から人間が読める形式の名前を返すヘルパー関数
00037 //
00038 std::string SubTypeToName(const GUID& subType)
00039 {
00040     if (subType == MFVideoFormat_YUY2) return "YUY2";
00041     if (subType == MFVideoFormat_NV12) return "NV12";
00042     if (subType == MFVideoFormat_MJPEG) return "MJPG";
00043     if (subType == MFVideoFormat_H264) return "H264";
00044
00045     // TODO: 他に使用するフォーマットがあればここに追加
00046     //if (subType == MFVideoFormat_RGB24) return "RGB24";
00047     if (subType == MFVideoFormat_RGB32) return "RGB32";
00048
00049     return "";
00050 }
00051
00052 //
00053 // COM ライブラリの初期化と終了を行うクラスのコンストラクタ
00054 //
00055 CamMf::ComInitializer::ComInitializer()
00056     : deviceList{}
  
```



```

00057     , ppSourceActivate{ nullptr }
00058     , cSourceActivate{ 0 }
00059     , coInitialized{ false }
00060     , mfStarted{ false }
00061 {
00062 }
00063
00064 //
00065 // COM ライブラリの初期化と終了を行うクラスのデストラクタ
00066 //
00067 CamMf::ComInitializer::~ComInitializer()
00068 {
00069     // メディアソースのリストを取得していれば
00070     if (ppSourceActivate)
00071     {
00072         // すべてのメディアソースを解放して
00073         for (DWORD i = 0; i < cSourceActivate; ++i) SafeRelease(&ppSourceActivate[i]);
00074
00075         // メディアソースのリストに使ったメモリを解放して
00076         CoTaskMemFree(ppSourceActivate);
00077         ppSourceActivate = nullptr;
00078
00079         // ビデオキャプチャデバイスの表示名のリストを空にする
00080         deviceList.clear();
00081     }
00082
00083     // Media Foundation が起動していればシャットダウンする
00084     if (mfStarted) MFShutdown();
00085
00086     // COM ライブラリが初期化されていれば終了する
00087     if (coInitialized) CoUninitialize();
00088 }
00089
00090 //
00091 // 初期化
00092 //
00093 const char* CamMf::ComInitializer::initialize()
00094 {
00095     // エラーメッセージ
00096     const char* message{ nullptr };
00097
00098     // 検索条件を保持する属性ストア
00099     IMFAttributes* pAttributes{ nullptr };
00100
00101     // COM ライブラリを初期化する
00102     if (FAILED(CoInitializeEx(nullptr, COINIT_MULTITHREADED)))
00103     {
00104         // COM ライブラリの初期化に失敗した
00105         message = "Failed to initialize COM library.";
00106         goto done;
00107     }
00108
00109     // COM ライブラリの初期化に成功した
00110     coInitialized = true;
00111
00112     // Media Foundation を起動する
00113     if (FAILED(MFStartup(MF_VERSION, MFSTARTUP_FULL)))
00114     {
00115         // Media Foundation の起動に失敗した
00116         message = "Failed to start Media Foundation.";
00117         goto done;
00118     }
00119
00120     // Media Foundation の起動に成功した
00121     mfStarted = true;
00122
00123     // 検索条件を保持する属性ストアを作成する
00124     if (FAILED(MFCreateAttributes(&pAttributes, 1)))
00125     {
00126         // 属性ストアの作成失敗した
00127         message = "Failed to create attribute store.";
00128         goto done;
00129     }
00130
00131     // 属性ストアにビデオキャプチャデバイスの属性を設定する
00132     if (FAILED(pAttributes->SetGUID(
00133         MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE,
00134         MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE_VIDCAP_GUID)))
00135     {
00136         // ビデオキャプチャデバイスの属性の設定に失敗した
00137         message = "Failed to set attribute for video capture device.";
00138         goto done;
00139     }
00140
00141     // メディアソースを列挙する
00142     if (FAILED(MFEnumDeviceSources(pAttributes,
00143         &ppSourceActivate, &cSourceActivate)))

```

```

00144 {
00145     // メディアソースの列挙に失敗した
00146     message = "Failed to enumerate media sources.";
00147     goto done;
00148 }
00149
00150 // すべてのメディアソースについて
00151 for (DWORD i = 0; i < cSourceActivate; ++i)
00152 {
00153     // メディアソースの表示名のリスト
00154     WCHAR* szFriendlyName{ nullptr };
00155
00156     // メディアソースの表示名の数
00157     UINT32 cFriendlyName{ 0 };
00158
00159     // メディアソースの表示名を取得する
00160     if (SUCCEEDED(ppSourceActivate[i]->GetAllocatedString(
00161         MF_DEVSOURCE_ATTRIBUTE_FRIENDLY_NAME, &szFriendlyName, &cFriendlyName)))
00162     {
00163         // ビデオキャプチャデバイス名を作る
00164         std::stringstream ss;
00165         ss << TCharToUtf8(szFriendlyName) << "##" << i;
00166
00167         // ビデオキャプチャデバイス名をリストに追加する
00168         deviceList.emplace_back(ss.str());
00169     }
00170
00171     // 表示名のリストに使ったメモリを解放する
00172     CoTaskMemFree(szFriendlyName);
00173 }
00174
00175 done:
00176
00177 // 属性ストアはもう使わないので解放する
00178 SafeRelease(&pAttributes);
00179
00180 // エラーメッセージを返す
00181 return message;
00182 }
00183
00184 //
00185 // COM ライブラリのシングルトンインスタンスを返す
00186 //
00187 const CamMf::ComInitializer& CamMf::ComInitializer::getInstance()
00188 {
00189     // COM ライブラリが初期化され Media Foundation が起動されていなければ
00190     if (!instance.ppSourceActivate)
00191     {
00192         // COM ライブラリを初期化して Media Foundation を起動する
00193         auto message{ instance.initialize() };
00194
00195         // COM ライブラリの初期化と Media Foundation の起動に失敗したら例外を投げる
00196         if (message) throw std::runtime_error(message);
00197     }
00198
00199     // COM ライブラリのシングルトンインスタンスへの参照を返す
00200     return instance;
00201 }
00202
00203 //
00204 // 有効化
00205 //
00206 bool CamMf::ComInitializer::activate(int device, IMFMediaSource** pMediaSource)
00207 {
00208     // メディアソースを作成して結果を返す
00209     return device >= 0 && static_cast<UINT32>(device) < instance.cSourceActivate
00210         && SUCCEEDED(instance.ppSourceActivate[device]->ActivateObject(IID_PPV_ARGS(pMediaSource)))
00211         && pMediaSource;
00212 }
00213
00214 //
00215 // ビデオキャプチャデバイスの表示名のリストを返す
00216 //
00217 const std::vector<std::string>& CamMf::ComInitializer::getDeviceList()
00218 {
00219     // ビデオキャプチャデバイスの表示名のリストを返す
00220     return getInstance().deviceList;
00221 }
00222
00223 //
00224 // 使用可能な解像度、フレームレート、コーデックのリストを作成する
00225 //
00226 bool CamMf::enumerateFormats()
00227 {
00228     // Source Reader が作成されていなければ戻る
00229     if (!pSourceReader) return false;
00230 }

```

```

00231 // 以前に作成したリストをクリアする
00232 availableFormats.clear();
00233 formatList.clear();
00234
00235 // ネイティブメディアタイプを一つずつ列挙する
00236 IMFMediaType* pMediaType{ nullptr };
00237 for (DWORD dwMediaTypeIndex = 0;
00238      SUCCEEDED(pSourceReader->GetNativeMediaType(MF_SOURCE_READER_FIRST_VIDEO_STREAM,
00239          dwMediaTypeIndex, &pMediaType));
00240      ++dwMediaTypeIndex
00241      )
00242 {
00243     // メディアタイプを取得する
00244     GUID majorType{};
00245     if (FAILED(pMediaType->GetGUID(MF_MT_MAJOR_TYPE, &majorType))) continue;
00246
00247     // メディアタイプがビデオで無ければ次へ
00248     if (majorType != MFMediaType_Video) continue;
00249
00250     // ビデオフォーマットを取得する
00251     GUID subType{};
00252     if (FAILED(pMediaType->GetGUID(MF_MT_SUBTYPE, &subType))) continue;
00253
00254     // 解像度を取得する
00255     UINT32 width{ 0 }, height{ 0 };
00256     if (FAILED(MFGetAttributeSize(pMediaType, MF_MT_FRAME_SIZE, &width, &height))) continue;
00257
00258     // フレームレートを取得する
00259     UINT32 numerator{ 0 }, denominator{ 0 };
00260     if (FAILED(MFGetAttributeRatio(pMediaType, MF_MT_FRAME_RATE, &numerator, &denominator))) continue;
00261
00262     // コーデックの文字列を取り出す
00263     const auto& codecName{ SubTypeToName(subType) };
00264
00265     // 対応できないコーデックかフレームレートに問題があれば次へ
00266     if (codecName.empty() || denominator <= 0 || numerator <= 0) continue;
00267
00268     // フレームレートを求める
00269     const double fps{ static_cast<double>(numerator) / static_cast<double>(denominator) };
00270
00271     // フレームレートが 5 未満なら次へ
00272     if (fps < 5.0) continue;
00273
00274     // 使用可能なビデオフォーマットの表示名を作成する
00275     std::stringstream ss;
00276     ss << width << " x " << height << " @ "
00277     << std::fixed << std::setprecision(2) << fps
00278     << " fps (" << codecName << ")###" << dwMediaTypeIndex;
00279
00280     // 使用可能なビデオフォーマットの表示名をリストに追加する
00281     formatList.emplace_back(ss.str());
00282
00283     // 使用可能なビデオフォーマットのリストに追加する
00284     availableFormats.emplace_back(width, height, numerator, denominator, subType);
00285
00286     // メディアタイプの取得に使ったメモリを解放する
00287     SafeRelease(&pMediaType);
00288 }
00289
00290 // リストが空でなければ成功
00291 return !formatList.empty();
00292 }
00293
00294 //
00295 // 指定されたサブタイプに対応するビデオデコーダを探す
00296 //
00297 HRESULT CamMf::findVideoDecoder(
00298     const GUID& subtype,
00299     IMFTransform** ppDecoder,
00300     BOOL bAllowAsync,
00301     BOOL bAllowHardware,
00302     BOOL bAllowTranscode
00303 ) const
00304 {
00305     // 結果
00306     HRESULT hr{ S_OK };
00307
00308     // 見つかったデコーダの数
00309     UINT32 count{ 0 };
00310
00311     // デコーダのアクティベーションオブジェクトのリスト
00312     IMFActivate** ppActivate{ nullptr };
00313
00314     // 検索条件
00315     MFT_REGISTER_TYPE_INFO inputInfo{ MFMediaType_Video, subtype };
00316     MFT_REGISTER_TYPE_INFO outputInfo{ MFMediaType_Video, MFVideoFormat_NV12 };
00317

```

```

00318 // 列挙のフラグ
00319 UINT32 unFlags
00320 {
00321     MFT_ENUM_FLAG_SYNCMFT
00322     | MFT_ENUM_FLAG_LOCALMFT
00323     | MFT_ENUM_FLAG_SORTANDFILTER
00324 };
00325
00326 // 非同期デコードも含めるなら
00327 if (bAllowAsync) unFlags |= MFT_ENUM_FLAG_ASYNCMFT;
00328
00329 // ハードウェアデコードも含めるなら
00330 if (bAllowHardware) unFlags |= MFT_ENUM_FLAG_HARDWARE;
00331
00332 // トランスコード専用のデコードも含めるなら
00333 if (bAllowTranscode) unFlags |= MFT_ENUM_FLAG_TRANSCODE_ONLY;
00334
00335 // 条件に合う MFT のアクティベーションオブジェクトを列挙する
00336 hr = MFTEnumEx(MFT_CATEGORY_VIDEO_DECODER,
00337     unFlags, &inputInfo, &outputInfo, &ppActivate, &count);
00338
00339 // デコーダが見つからなかったらエラー
00340 if (SUCCEEDED(hr) && count == 0) hr = MF_E_TOPO_CODEC_NOT_FOUND;
00341
00342 // リストの最初のデコーダをインスタンス化する
00343 if (SUCCEEDED(hr)) hr = ppActivate[0]->ActivateObject(IID_PPV_ARGS(ppDecoder));
00344
00345 // デコーダのアクティベーションオブジェクトのリストを解放する
00346 for (UINT32 i = 0; i < count; i++) ppActivate[i]->Release();
00347
00348 // デコーダのアクティベーションオブジェクトのリストに使ったメモリを解放する
00349 CoTaskMemFree(ppActivate);
00350
00351 // 結果を返す
00352 return hr;
00353 }
00354
00355 //
00356 // MFT のセットアップと接続を行う
00357 //
00358 HRESULT CamMf::setUpPipeline(IMFTransform* pTransform, const VideoFormat& format, const GUID& subType)
00359 {
00360     #if defined(_DEBUG)
00361         std::cerr << SubTypeToName(format.subType) << " -> " << SubTypeToName(subType) << std::endl;
00362     #endif
00363
00364     // 入力と出力のメディアタイプ
00365     IMFMediaType* pInputType{ nullptr };
00366     IMFMediaType* pOutputType{ nullptr };
00367
00368     // 結果
00369     HRESULT hr{ S_OK };
00370
00371     // MFT の入力タイプの設定
00372     hr = MFCreateMediaType(&pInputType);
00373     if (SUCCEEDED(hr)) hr = pInputType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00374     if (SUCCEEDED(hr)) hr = pInputType->SetGUID(MF_MT_SUBTYPE, format.subType);
00375     if (SUCCEEDED(hr)) hr = MFSetAttributeSize(pInputType, MF_MT_FRAME_SIZE, format.width,
format.height);
00376     if (SUCCEEDED(hr)) hr = MFSetAttributeRatio(pInputType, MF_MT_FRAME_RATE, format.fpsNum,
format.fpsDenom);
00377
00378     // MFT に入力タイプを設定する
00379     if (SUCCEEDED(hr)) hr = pTransform->SetInputType(0, pInputType, 0);
00380     if (FAILED(hr)) goto done;
00381
00382     // MFT の出力タイプの設定
00383     hr = MFCreateMediaType(&pOutputType);
00384     if (SUCCEEDED(hr)) hr = pOutputType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00385     if (SUCCEEDED(hr)) hr = pOutputType->SetGUID(MF_MT_SUBTYPE, subType);
00386     if (SUCCEEDED(hr)) hr = MFSetAttributeSize(pOutputType, MF_MT_FRAME_SIZE, format.width,
format.height);
00387     if (SUCCEEDED(hr)) hr = MFSetAttributeRatio(pOutputType, MF_MT_FRAME_RATE, format.fpsNum,
format.fpsDenom);
00388
00389     // MFT に出力タイプを設定する
00390     if (SUCCEEDED(hr)) hr = pTransform->SetOutputType(0, pOutputType, 0);
00391     if (FAILED(hr)) goto done;
00392
00393     // MFT をアクティブにする
00394     hr = pTransform->ProcessMessage(MFT_MESSAGE_COMMAND_FLUSH, NULL);
00395     if (SUCCEEDED(hr)) hr = pTransform->ProcessMessage(MFT_MESSAGE_NOTIFY_START_OF_STREAM, NULL);
00396     if (SUCCEEDED(hr)) hr = pTransform->ProcessMessage(MFT_MESSAGE_NOTIFY_BEGIN_STREAMING, NULL);
00397
00398 done:
00399

```

```

00400 // メディアタイプの取得に使ったメモリを解放する
00401 SafeRelease(&pInputType);
00402 SafeRelease(&pOutputType);
00403
00404 // 結果を返す
00405 return hr;
00406 }
00407
00408 //
00409 // MFT を解放する
00410 //
00411 void CamMf::cleanUpTransform(IMFTransform** pTransform) const
00412 {
00413     // MFT が作成されていれば
00414     if (*pTransform)
00415     {
00416         // MFT のストリーミングの終了を通知する
00417         (*pTransform)->ProcessMessage(MFT_MESSAGE_NOTIFY_END_STREAMING, NULL);
00418         (*pTransform)->ProcessMessage(MFT_MESSAGE_NOTIFY_END_OF_STREAM, NULL);
00419
00420         // MFT を解放する
00421         (*pTransform)->Release();
00422         *pTransform = nullptr;
00423     }
00424 }
00425
00426 //
00427 // Source Reader の出力フォーマットを設定し、基底クラスの frame を初期化する
00428 //
00429 bool CamMf::setFormat(int index)
00430 {
00431     // 使用可能なフォーマットのリストから選択されたフォーマットを取得する
00432     VideoFormat selectedFormat{ availableFormats[index] };
00433
00434     // デコード方法
00435     enum class DecodeMethod { None = 0, Convert, Decode }; decodeMethod
00436     {
00437         selectedFormat.subType == MFVideoFormat_MJPG ? DecodeMethod::Decode :
00438         selectedFormat.subType == MFVideoFormat_H264 ? DecodeMethod::Decode :
00439         selectedFormat.subType == MFVideoFormat_NV12 ? DecodeMethod::Convert :
00440         selectedFormat.subType == MFVideoFormat_YUY2 ? DecodeMethod::Convert :
00441         DecodeMethod::None
00442     };
00443
00444     // デコードとカラーコンバータを未設定にする
00445     cleanUpTransform(&pDecoder);
00446     cleanUpTransform(&pConverter);
00447
00448     // 結果
00449     HRESULT hr{ S_OK };
00450
00451     // 新しいメディアタイプを作成する
00452     IMFMediaType* pMediaType{ nullptr };
00453     hr = MFCreateMediaType(&pMediaType);
00454     if (FAILED(hr)) goto done;
00455
00456     // メジャータイプにビデオを指定する
00457     hr = pMediaType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00458     if (FAILED(hr)) goto done;
00459
00460     // ビデオフォーマットを指定する
00461     hr = pMediaType->SetGUID(MF_MT_SUBTYPE, selectedFormat.subType);
00462     if (FAILED(hr)) goto done;
00463
00464     // 解像度を設定する
00465     hr = MFSetAttributeSize(pMediaType, MF_MT_FRAME_SIZE, selectedFormat.width, selectedFormat.height);
00466     if (FAILED(hr)) goto done;
00467
00468     // フレームレートを設定する
00469     hr = MFSetAttributeRatio(pMediaType, MF_MT_FRAME_RATE, selectedFormat.fpsNum,
selectedFormat.fpsDenom);
00470     if (FAILED(hr)) goto done;
00471
00472     // Source Reader の出力メディアタイプを設定する
00473     hr = pSourceReader->SetCurrentMediaType(MF_SOURCE_READER_FIRST_VIDEO_STREAM, nullptr, pMediaType);
00474     if (FAILED(hr)) goto done;
00475
00476     // 基底クラスの解像度とチャンネル数を設定し、バッファサイズを設定する
00477     width = selectedFormat.width;
00478     height = selectedFormat.height;
00479     channels = 4;
00480     frame.resize(width * height * channels);
00481     image.resize(width * height * channels);
00482
00483     // インターバルを計算する
00484     interval = (selectedFormat.fpsDenom != 0)
? (1000.0 * selectedFormat.fpsDenom / selectedFormat.fpsNum)

```

```

00486         : 10.0;
00487
00488     // デコードまたはカラー変換が必要なとき
00489     if (decodeMethod != DecodeMethod::None)
00490     {
00491         // デコードが必要なら
00492         if (decodeMethod == DecodeMethod::Decode)
00493         {
00494             // 選択されたビデオフォーマットのデコーダを探す
00495             hr = findVideoDecoder(selectedFormat.subType, &pDecoder);
00496             if (FAILED(hr)) goto done;
00497
00498             // MFT デコーダのセットアップと接続を行う
00499             hr = setUpPipeline(pDecoder, selectedFormat, MFVideoFormat_NV12);
00500             if (FAILED(hr)) goto done;
00501
00502             // デコード後のビデオフォーマットは NV12 にしている
00503             selectedFormat.subType = MFVideoFormat_NV12;
00504
00505             // デコーダの出力バッファのサイズを計算する
00506             UINT32 cbDecoder{ 0 };
00507             MFCalculateImageSize(MFVideoFormat_NV12, width, height, &cbDecoder);
00508
00509             // デコーダの出力バッファを作成する
00510             hr = MFCreateMemoryBuffer(cbDecoder, &pDecoderBuffer);
00511             if (FAILED(hr)) goto done;
00512
00513             // 出力バッファのサイズを設定する
00514             hr = pDecoderBuffer->SetCurrentLength(static_cast<DWORD>(cbDecoder));
00515             if (FAILED(hr)) goto done;
00516         }
00517
00518         // MFT をインスタンス化する
00519         hr = CoCreateInstance(CLSID_CColorConvertDMO, nullptr, CLSCTX_INPROC_SERVER,
IID_PPV_ARGS(&Converter));
00520         if (FAILED(hr)) goto done;
00521
00522         // MFT カラーコンバータのセットアップと接続を行う
00523         hr = setUpPipeline(pConverter, selectedFormat, MFVideoFormat_RGB32);
00524         if (FAILED(hr)) goto done;
00525
00526         // カラーコンバータの出力バッファのサイズを計算する
00527         UINT32 cbConverter{ 0 };
00528         MFCalculateImageSize(MFVideoFormat_RGB32, width, height, &cbConverter);
00529
00530         // カラー変換用の出力バッファを作成する
00531         hr = MFCreateMemoryBuffer(cbConverter, &pConverterBuffer);
00532         if (FAILED(hr)) goto done;
00533
00534         // 出力バッファのサイズを設定する
00535         hr = pConverterBuffer->SetCurrentLength(static_cast<DWORD>(cbConverter));
00536         if (FAILED(hr)) goto done;
00537     }
00538
00539 done:
00540
00541     // メディアタイプはもう使わないので解放する
00542     SafeRelease(&pMediaType);
00543
00544     // 成功したら true を返す
00545     if (SUCCEEDED(hr)) return true;
00546
00547     // 失敗したときはカメラを閉じて
00548     close();
00549
00550     // false を返す
00551     return false;
00552 }
00553
00554 //
00555 // カメラを開く
00556 //
00557 bool CamMf::open(int device, bool setupFormat)
00558 {
00559     // メディアソースを作成する
00560     if (!ComInitializer::activate(device, &pMediaSource)) return false;
00561
00562     // 結果
00563     HRESULT hr{ S_OK };
00564
00565     // Source Reader の属性ストアを作成する
00566     IMFAttributes* pAttributes{ nullptr };
00567     hr = MFCreateAttributes(&pAttributes, 1);
00568     if (FAILED(hr)) goto done;
00569
00570     // Source Reader の属性ストアにデコード能力を設定する
00571     pAttributes->SetUINT32(MF_READWRITE_ENABLE_HARDWARE_TRANSFORMS, TRUE);

```

```

00572 pAttributes->SetUINT32(MF_READWRITE_DISABLE_CONVERTERS, FALSE);
00573
00574 // Source Reader の解放時に Media Source をシャットダウンするようにする
00575 pAttributes->SetUINT32(MF_SOURCE_READER_DISCONNECT_MEDIASOURCE_ON_SHUTDOWN, TRUE);
00576
00577 // Source Reader を作成する
00578 hr = MFCreateSourceReaderFromMediaSource(pMediaSource, pAttributes, &pSourceReader);
00579 if (FAILED(hr)) goto done;
00580
00581 // 属性ストアはもう必要ないので解放する
00582 SafeRelease(&pAttributes);
00583
00584 // 使用可能なフォーマットを列挙して最初のフォーマットを選択する
00585 if (enumerateFormats())
00586 {
00587     // 要求された場合のみ最初のフォーマットを設定する
00588     if (!setupFormat || setFormat(0)) return true;
00589 }
00590
00591 done:
00592
00593 // フォーマットの設定に失敗したら全てを解放して戻る
00594 SafeRelease(&pSourceReader);
00595 SafeRelease(&pAttributes);
00596 availableFormats.clear();
00597 formatList.clear();
00598
00599 return false;
00600 }
00601
00602 //
00603 // フォーマットを選択して設定する
00604 //
00605 bool CamMf::select(int index)
00606 {
00607     // カメラが開かれていないかインデックスが範囲外なら戻る
00608     if (!pSourceReader || index < 0 || index >= availableFormats.size()) return false;
00609
00610     // キャプチャスレッドが実行中であれば安全のために停止する
00611     stop();
00612
00613     // 選択されたフォーマットを設定する
00614     return setFormat(index);
00615 }
00616
00617 //
00618 // ストリームのフォーマット変更を処理する
00619 //
00620 HRESULT HandleStreamChange(IMFTransform* pDecoder)
00621 {
00622     // 現在のストリームを止める
00623     pDecoder->ProcessMessage(MFT_MESSAGE_NOTIFY_END_STREAMING, NULL);
00624     pDecoder->ProcessMessage(MFT_MESSAGE_NOTIFY_END_OF_STREAM, NULL);
00625
00626     HRESULT hr{ S_OK };
00627     IMFMediaType* pNewOutputType{ nullptr };
00628
00629     // 使用可能な出力タイプを探す
00630     for (DWORD typeIndex = 0; ++typeIndex)
00631     {
00632         // 出力タイプの候補を取得する
00633         hr = pDecoder->GetOutputAvailableType(0, typeIndex, &pNewOutputType);
00634
00635         // 取得に失敗したら終わる
00636         if (FAILED(hr)) break;
00637
00638         // 取得した出力タイプのビデオフォーマットを調べる
00639         GUID subtype{ 0 };
00640         pNewOutputType->GetGUID(MF_MT_SUBTYPE, &subtype);
00641
00642         #if defined(_DEBUG)
00643         if (subtype == MFVideoFormat_NV12)
00644             std::cerr << "Stream change to NV12" << std::endl;
00645         else if (subtype == MFVideoFormat_YUY2)
00646             std::cerr << "Stream change to YUY2" << std::endl;
00647         else
00648             std::cerr << "Stream change to other format" << std::endl;
00649         #endif
00650
00651         // ビデオフォーマットが NV12 または YUY2 フォーマットなら
00652         if (subtype == MFVideoFormat_NV12 || subtype == MFVideoFormat_YUY2)
00653         {
00654             // これを新しい出力タイプとして設定する
00655             hr = pDecoder->SetOutputType(0, pNewOutputType, 0);
00656             pNewOutputType->Release();
00657             break;
00658         }
00659     }

```

```

00659
00660     pNewOutputType->Release();
00661 }
00662
00663 // ストリームを再開する
00664 if (SUCCEEDED(hr)) pDecoder->ProcessMessage(MFT_MESSAGE_COMMAND_FLUSH, NULL);
00665 if (SUCCEEDED(hr)) hr = pDecoder->ProcessMessage(MFT_MESSAGE_NOTIFY_START_OF_STREAM, NULL);
00666 if (SUCCEEDED(hr)) hr = pDecoder->ProcessMessage(MFT_MESSAGE_NOTIFY_BEGIN_STREAMING, NULL);
00667
00668 // 結果を返す
00669 return hr;
00670 }
00671
00672 //
00673 // フレームをキャプチャする
00674 //
00675 void CamMf::capture()
00676 {
00677     // スレッドが実行可の間
00678     while (running)
00679     {
00680         // サンプルを読み出して
00681         DWORD dwStreamIndex{ 0 };
00682         DWORD dwStreamFlags{ 0 };
00683         LONGLONG llTimestamp{ 0 };
00684         IMFSample* pSample{ nullptr };
00685         if (FAILED(pSourceReader->ReadSample(MF_SOURCE_READER_FIRST_VIDEO_STREAM,
00686             0, &dwStreamIndex, &dwStreamFlags, &llTimestamp, &pSample))) continue;
00687
00688         // サンプルが取得できていなければ戻る
00689         if (!pSample) continue;
00690
00691         // サンプルから取り出したメディアバッファ
00692         IMFMediaBuffer* pBuffer{ nullptr };
00693
00694         // デコーダの出力フレームを保持するサンプル
00695         IMFSample* pDecodedSample{ nullptr };
00696
00697         // カラーコンバータの出力フレームを保持するサンプル
00698         IMFSample* pConvertedSample{ nullptr };
00699
00700         // ProcessOutput の呼び出しの状態
00701         DWORD dwStatus{ 0 };
00702
00703         // デコーダが設定されていれば (MJPG か H264 の場合)
00704         if (pDecoder)
00705         {
00706             // サンプルをデコーダに渡す
00707             HRESULT hr{ pDecoder->ProcessInput(0, pSample, 0) };
00708
00709             // デコーダへの入力に失敗したら
00710             if (FAILED(hr))
00711             {
00712 #if defined(_DEBUG)
00713                 std::cerr << "Decoder process input failed: ";
00714                 switch (hr)
00715                 {
00716                     case E_INVALIDARG:
00717                         std::cerr << "Invalid argument." << std::endl; break;
00718                     case E_UNEXPECTED:
00719                         std::cerr << "Unexpected error." << std::endl; break;
00720                     case E_FAIL:
00721                         std::cerr << "Unspecified error." << std::endl; break;
00722                     case MF_E_INVALIDSTREAMNUMBER:
00723                         std::cerr << "Invalid stream number." << std::endl; break;
00724                     case MF_E_NO_SAMPLE_DURATION:
00725                         std::cerr << "No sample duration." << std::endl; break;
00726                     case MF_E_NO_SAMPLE_TIMESTAMP:
00727                         std::cerr << "No sample timestamp." << std::endl; break;
00728                     case MF_E_NOTACCEPTING:
00729                         std::cerr << "Not accepting input." << std::endl; break;
00730                     case MF_E_TRANSFORM_TYPE_NOT_SET:
00731                         std::cerr << "Transform type not set." << std::endl; break;
00732                     case MF_E_UNSUPPORTED_D3D_TYPE:
00733                         std::cerr << "Unsupported D3D type." << std::endl; break;
00734                     default:
00735                         std::cerr << "Code: " << std::hex << hr << std::endl; break;
00736                 }
00737             }
00738 #endif
00739             goto done;
00740         }
00741
00742         // デコーダの出力サンプルを作成する
00743         hr = MFCreateSample(&pDecodedSample);
00744         if (FAILED(hr)) goto done;
00745
00746         // デコーダの出力サンプルにバッファを追加する

```



```

00746     hr = pDecodedSample->AddBuffer(pDecoderBuffer);
00747     if (FAILED(hr)) goto done;
00748
00749     // デコーダの出力バッファ
00750     MFT_OUTPUT_DATA_BUFFER decodedBuffer{ 0, pDecodedSample, 0, nullptr };
00751
00752     // デコード処理を実行
00753     hr = pDecoder->ProcessOutput(0, 1, &decodedBuffer, &dwStatus);
00754
00755     // ストリームのフォーマットが変化した場合の処理 (フレームのデコード完了?)
00756     if (hr == MF_E_TRANSFORM_STREAM_CHANGE)
00757     {
00758         #if defined(_DEBUG)
00759             std::cerr << "Transform stream change." << std::endl;
00760         #endif
00761         // 保留中のフレームをすべて処理する
00762         do
00763         {
00764             hr = pDecoder->ProcessOutput(0, 1, &decodedBuffer, &dwStatus);
00765             #if defined(_DEBUG)
00766                 if (hr == E_FAIL) std::cerr << "Failed to process output during stream change handling." <<
std::endl;
00767             #endif
00768         }
00769         while (hr != MF_E_TRANSFORM_NEED_MORE_INPUT);
00770
00771         // 現在のストリームを一旦止める
00772         pDecoder->ProcessMessage(MFT_MESSAGE_NOTIFY_END_STREAMING, NULL);
00773         //pDecoder->ProcessMessage(MFT_MESSAGE_NOTIFY_END_OF_STREAM, NULL);
00774
00775         // ストリームを再開する
00776         pDecoder->ProcessMessage(MFT_MESSAGE_COMMAND_FLUSH, NULL);
00777         //hr = pDecoder->ProcessMessage(MFT_MESSAGE_NOTIFY_START_OF_STREAM, NULL);
00778         //if (FAILED(hr)) goto done;
00779         hr = pDecoder->ProcessMessage(MFT_MESSAGE_NOTIFY_BEGIN_STREAMING, NULL);
00780         //if (FAILED(hr)) goto done;
00781     }
00782
00783     // デコード処理に失敗したら
00784     if (FAILED(hr))
00785     {
00786         #if defined(_DEBUG)
00787             std::cerr << "Decoder process output failed: ";
00788             switch (hr)
00789             {
00790             case E_INVALIDARG:
00791                 std::cerr << "Invalid argument." << std::endl; break;
00792             case E_UNEXPECTED:
00793                 std::cerr << "Unexpected error." << std::endl; break;
00794             case E_FAIL:
00795                 std::cerr << "Unspecified error." << std::endl; break;
00796             case MF_E_INVALID_STREAM_NUMBER:
00797                 std::cerr << "Invalid stream number." << std::endl; break;
00798             case MF_E_TRANSFORM_NEED_MORE_INPUT:
00799                 std::cerr << "Transform needs more input." << std::endl; break;
00800             case MF_E_TRANSFORM_TYPE_NOT_SET:
00801                 std::cerr << "Transform type not set." << std::endl; break;
00802             default:
00803                 std::cerr << "Code: " << std::hex << hr << std::endl; break;
00804             }
00805         #endif
00806         goto done;
00807     }
00808
00809     // 元のサンプルはもう使わないので解放する
00810     pSample->Release();
00811
00812     // デコードに成功したらデコードされたサンプルを使う
00813     pSample = pDecodedSample;
00814     pDecodedSample = nullptr;
00815 }
00816
00817 // カラーコンバータが設定されていれば (NV12 か YUY2 か MJPG か H264 の場合)
00818 if (pConverter)
00819 {
00820     // サンプルをカラーコンバータに渡す
00821     HRESULT hr{ pConverter->ProcessInput(0, pSample, 0) };
00822     if (FAILED(hr)) goto done;
00823
00824     // カラーコンバータの出力サンプルを作成する
00825     hr = MFCreateSample(&pConvertedSample);
00826     if (FAILED(hr)) goto done;
00827
00828     // カラーコンバータの出力サンプルにバッファを追加する
00829     hr = pConvertedSample->AddBuffer(pConverterBuffer);
00830     if (FAILED(hr)) goto done;
00831

```

```

00832 // カラーコンバータの出力バッファ
00833 MFT_OUTPUT_DATA_BUFFER convertedBuffer{ 0, pConvertedSample, 0, nullptr };
00834
00835 // カラー変換処理を実行
00836 hr = pConverter->ProcessOutput(0, 1, &convertedBuffer, &dwStatus);
00837 if (FAILED(hr)) goto done;
00838
00839 // 元のサンプルはもう使わないので解放する
00840 pSample->Release();
00841
00842 // コンバートに成功したらコンバートされたサンプルを使う
00843 pSample = pConvertedSample;
00844 pConvertedSample = nullptr;
00845 }
00846
00847 // サンプルからメディアバッファを取得して
00848 if (FAILED(pSample->GetBufferByIndex(0, &pBuffer))) goto done;
00849
00850 // メディアバッファが取得できていれば
00851 if (pBuffer)
00852 {
00853     // メディアバッファから取り出したデータ
00854     BYTE* pData{ nullptr };
00855
00856     // メディアバッファから取り出したデータの長さ
00857     DWORD cbDataLength{ 0 };
00858
00859     // メディアバッファからフレームの情報を取得できたら
00860     if (SUCCEEDED(pBuffer->Lock(&pData, nullptr, &cbDataLength)) && pData)
00861     {
00862         // 一時メモリをロックして
00863         mtx.lock();
00864
00865         // 基底クラスの frame バッファにデータをコピーする
00866         if (frame.size() < cbDataLength) frame.resize(cbDataLength);
00867         memcpy(frame.data(), pData, cbDataLength);
00868
00869         // キャプチャしたデータを一時メモリにコピーしたら
00870         if (image.size() < cbDataLength) image.resize(cbDataLength);
00871         memcpy(image.data(), frame.data(), cbDataLength);
00872
00873         // 新しいフレームがキャプチャされたことを通知して
00874         captured = true;
00875
00876         // 一時メモリロックを解除したら
00877         mtx.unlock();
00878
00879         // メディアバッファのロックを解除する
00880         pBuffer->Unlock();
00881     }
00882
00883     // メディアバッファを解放する
00884     pBuffer->Release();
00885     pBuffer = nullptr;
00886 }
00887
00888 done:
00889
00890 // デコーダの出力サンプルとバッファを解放する
00891 if (pDecodedSample) pDecodedSample->Release();
00892
00893 // カラーコンバータの出力サンプルバッファを解放する
00894 if (pConvertedSample) pConvertedSample->Release();
00895
00896 // サンプルを解放する
00897 pSample->Release();
00898 pSample = nullptr;
00899
00900 // 少し休む
00901 std::this_thread::sleep_for(std::chrono::milliseconds(static_cast<int>(interval)));
00902 }
00903 }
00904
00905 //
00906 // カメラを閉じる
00907 //
00908 void CamMf::close()
00909 {
00910     // MFT デコーダを解放する
00911     cleanUpTransform(&pDecoder);
00912
00913     // MFT カラーコンバータを解放する
00914     cleanUpTransform(&pConverter);
00915
00916     // MFT デコーダのバッファを解放する
00917     SafeRelease(&pDecoderBuffer);
00918

```

```

00919 // MFT カラーコンバータのバッファを解放する
00920 SafeRelease(&pConverterBuffer);
00921
00922 // Source Reader を解放する
00923 SafeRelease(&pSourceReader);
00924
00925 // Media Source 解放する
00926 SafeRelease(&pMediaSource);
00927
00928 // フォーマットリストをクリアする
00929 availableFormats.clear();
00930 formatList.clear();
00931
00932 // 基底クラスの close を呼び出す
00933 Camera::close();
00934 }

```

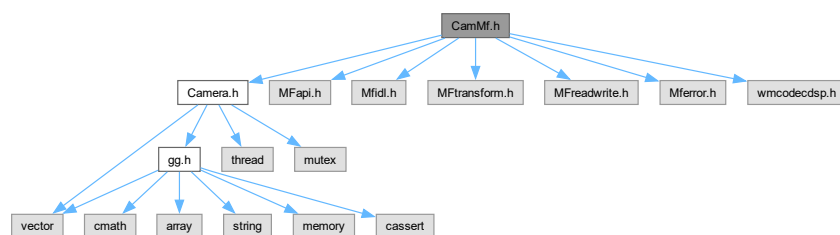
9.19 CamMf.h ファイル

```

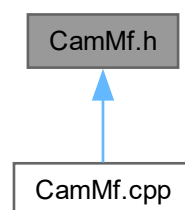
#include "Camera.h"
#include <MFapi.h>
#include <Mfidl.h>
#include <MFtransform.h>
#include <MFreadwrite.h>
#include <Mferror.h>
#include <wmcodecdsp.h>

```

CamMf.h の依存先関係図:



被依存関係図:



クラス

- class [CamMf](#)

9.19.1 詳解

Microsoft Media Foundation を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

December 24, 2024

[CamMf.h](#) に定義があります。

9.20 CamMf.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // カメラ関連の処理
00012 #include "Camera.h"
00013
00014 // Microsoft Media Foundation
00015 #include <MFapi.h>
00016 #include <Mfidl.h>
00017 #include <MFtransform.h>
00018 #include <MFreadwrite.h>
00019 #include <Mferror.h>
00020 #include <wmcodecdsp.h>
00021
00025 class CamMf : public Camera
00026 {
00030     struct VideoFormat
00031     {
00032         UINT32 width;
00033         UINT32 height;
00034         UINT32 fpsNum;
00035         UINT32 fpsDenom;
00036         GUID subType;
00037
00047         VideoFormat(UINT32 width, UINT32 height,
00048             UINT32 fpsNum, UINT32 fpsDenom, GUID subType)
00049             : width{ width }
00050             , height{ height }
00051             , fpsNum{ fpsNum }
00052             , fpsDenom{ fpsDenom }
00053             , subType{ subType }
00054     {
00055     };
00056 };
00057
00061 class ComInitializer
00062 {
00064     static ComInitializer instance;
00065
00067     IMFActivate** ppSourceActivate;
00068
00070     UINT32 cSourceActivate;
00071
00073     std::vector<std::string> deviceList;
00074
00076     bool coInitialized;
00077
00079     bool mfStarted;
00080
00084     ComInitializer();
00085
00089     ~ComInitializer();
00090
00096     const char* initialize();
00097
00098 public:
00099
00100     // シングルトンなのでコピーは作らせない
00101     ComInitializer(const ComInitializer& com) = delete;
00102     ComInitializer(ComInitializer&& com) = delete;
00103     ComInitializer& operator=(const ComInitializer& com) = delete;
00104     ComInitializer& operator=(ComInitializer&&) = delete;
00105 }
```

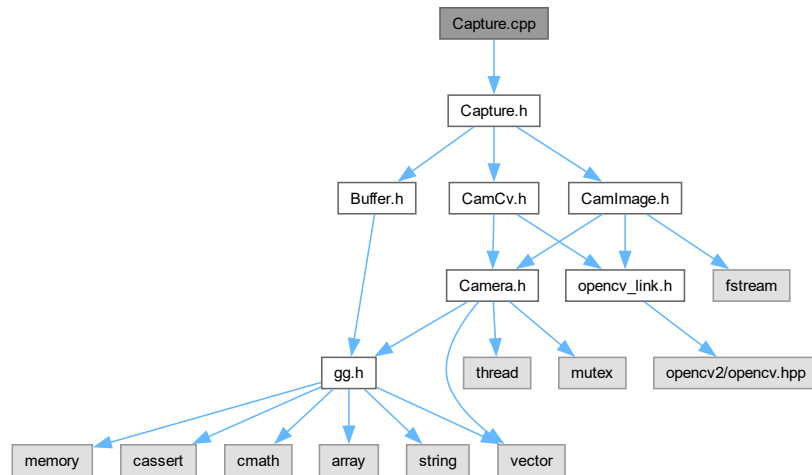
```

00111     static const ComInitializer& getInstance();
00112
00120     static bool activate(int device, IMFMediaSource** pMediaSource);
00121
00127     static const std::vector<std::string>& getDeviceList();
00128 };
00129
00131 IMFMediaSource* pMediaSource;
00132
00134 IMFSOURCEReader* pSourceReader;
00135
00137 IMFTransform* pDecoder;
00138
00140 IMFMediaBuffer* pDecoderBuffer;
00141
00143 IMFTransform* pConverter;
00144
00146 IMFMediaBuffer* pConverterBuffer;
00147
00149 std::vector<VideoFormat> availableFormats;
00150
00152 std::vector<std::string> formatList;
00153
00159 bool enumerateFormats();
00160
00171 HRESULT findVideoDecoder(
00172     const GUID& subtype,
00173     IMFTransform** ppDecoder,
00174     BOOL bAllowAsync = FALSE,
00175     BOOL bAllowHardware = FALSE,
00176     BOOL bAllowTranscode = FALSE
00177 ) const;
00178
00187 HRESULT setUpPipeline(IMFTransform* pTransform,
00188     const VideoFormat& format, const GUID& subType) const;
00189
00195 void cleanUpTransform(IMFTransform** pTransform) const;
00196
00203 bool setFormat(int index);
00204
00205 public:
00206
00210 CamMf()
00211     : pMediaSource{ nullptr }
00212     , pSourceReader{ nullptr }
00213     , pDecoder{ nullptr }
00214     , pDecoderBuffer{ nullptr }
00215     , pConverter{ nullptr }
00216     , pConverterBuffer{ nullptr }
00217     {
00218     }
00219
00223 virtual ~CamMf()
00224 {
00225     // カメラを閉じる
00226     close();
00227 }
00228
00234 static const auto& getDeviceList()
00235 {
00236     return ComInitializer::getDeviceList();
00237 }
00238
00246 bool open(int device, bool setupFormat = true);
00247
00253 const auto& getFormatList() const
00254 {
00255     return formatList;
00256 }
00257
00264 bool select(int index);
00265
00269 void capture();
00270
00274 void close();
00275 };

```

9.21 Capture.cpp ファイル

#include "Capture.h"
Capture.cpp の依存先関係図:



9.21.1 詳解

キャプチャクラスの実装

著者

Kohe Tokoi

日付

April 3, 2023

[Capture.cpp](#) に定義があります。

9.22 Capture.cpp

[詳解]

```

00001
00008 #include "Capture.h"
00009
00010 #if defined(_WIN32)
00012 const std::vector<std::string> Capture::emptyFormatList;
00013 #endif
00014
00015 //
00016 // 画像ファイルを開く
00017 //
00018 bool Capture::openImage(const std::string& filename)
00019 {
00020     // 新しいキャプチャデバイスを作成したら
00021     auto camImage{ std::make_unique<CamImage>() };
00022
00023     // キャプチャデバイスを開く
00024     if (camImage->open(filename))
00025     {
00026         // このキャプチャデバイスを使うことにする
00027         camera = std::move(camImage);
00028         return true;
00029     }
00030

```

```

00031 // 開けなかった
00032 return false;
00033 }
00034
00035 //
00036 // 動画ファイルを開く
00037 //
00038 bool Capture::openMovie(const std::string& filename,
00039 cv::VideoCaptureAPIs backend)
00040 {
00041 // 新しいキャプチャデバイスを作成したら
00042 auto camCv{ std::make_unique<CamCv>() };
00043
00044 // キャプチャデバイスを開く
00045 if (camCv->open(filename, 0, 0, 0.0, "", backend))
00046 {
00047 // このキャプチャデバイスを使うことにする
00048 camera = std::move(camCv);
00049 return true;
00050 }
00051
00052 // 開けなかった
00053 return false;
00054 }
00055
00056 #if defined(_WIN32)
00057 //
00058 // デバイスを開く (Windows用: MSMF)
00059 //
00060 bool Capture::openDevice(int deviceNumber)
00061 {
00062 // 既にカメラが有効なら一旦閉じる
00063 if (camera) camera->close();
00064
00065 // 新しいキャプチャデバイスを作成したら
00066 auto camMf{ std::make_unique<CamMf>() };
00067
00068 // このデバイスをデバイス番号で開いて
00069 if (camMf->open(deviceNumber, false))
00070 {
00071 // このキャプチャデバイスを使うことにする
00072 camera = std::move(camMf);
00073
00074 // 開けた
00075 return true;
00076 }
00077
00078 // カメラを無効にしておく
00079 camera.reset();
00080
00081 // 開けなかった
00082 return false;
00083 }
00084
00085 //
00086 // ビデオフォーマット選択
00087 //
00088 bool Capture::select(int index)
00089 {
00090 // カメラが有効でなければ戻る
00091 if (!camera) return false;
00092
00093 // バックエンドが Microsoft Media Foundation でなければ戻る
00094 auto camMf{ dynamic_cast<CamMf*>(camera.get()) };
00095 if (!camMf) return true; // Media Foundation 以外なら常に true
00096
00097 // ビデオフォーマットを選択する
00098 return camMf->select(index);
00099 }
00100
00101 void Capture::updateFormatList(int deviceNumber)
00102 {
00103 CamMf tempCam;
00104 if (tempCam.open(deviceNumber, false))
00105 {
00106 deviceFormatList = tempCam.getFormatList();
00107 tempCam.close();
00108 }
00109 else
00110 {
00111 deviceFormatList.clear();
00112 }
00113 }
00114
00115 #if defined(_WIN32)
00116 const std::vector<std::string>& Capture::getFormatList() const
00117 {

```

```

00118     auto camMf{ dynamic_cast<const CamMf*>(camera.get()) };
00119     return camMf ? camMf->getFormatList() : deviceFormatList;
00120 }
00121 #endif
00122 #else
00123 //
00124 // デバイスを開く (Windows以外用: OpenCV)
00125 //
00126 bool Capture::openDevice(int deviceNumber, std::array<int, 2>& size, double& fps,
00127     cv::VideoCaptureAPIs backend, char* fourcc)
00128 {
00129     // 既にカメラが有効なら一旦閉じる
00130     if (camera) camera->close();
00131
00132     // 新しいキャプチャデバイスを作成したら
00133     auto camCv{ std::make_unique<CamCv>() };
00134
00135     // このデバイスをデバイス番号で開いて
00136     if (camCv->open(deviceNumber, size[0], size[1], fps, fourcc, backend))
00137     {
00138         // 実際に開いた設定を書き戻す
00139         size[0] = camCv->getWidth();
00140         size[1] = camCv->getHeight();
00141         fps = camCv->getFps();
00142         camCv->getCodec(fourcc);
00143
00144         // このキャプチャデバイスを使うことにする
00145         camera = std::move(camCv);
00146         return true;
00147     }
00148
00149     // 開けなかった
00150     return false;
00151 }
00152 #endif
00153
00154 //
00155 // キャプチャ開始
00156 //
00157 void Capture::start()
00158 {
00159     // キャプチャデバイスが有効ならキャプチャスレッドを起動する
00160     if (camera) camera->start();
00161 }
00162
00163 //
00164 // キャプチャ終了
00165 //
00166 void Capture::stop()
00167 {
00168     // キャプチャデバイスが有効ならキャプチャスレッドを停止する
00169     if (camera) camera->stop();
00170 }
00171
00172 //
00173 // キャプチャデバイスを閉じる
00174 //
00175 void Capture::close()
00176 {
00177     // キャプチャデバイスが有効ならキャプチャスレッドを停止する
00178     if (camera)
00179     {
00180         camera->close();
00181         camera.reset();
00182     }
00183 }
00184
00185 //
00186 // キャプチャデバイスの解像度とフレームレートを取得する
00187 //
00188 std::array<int, 2> Capture::getSize() const
00189 {
00190     // キャプチャデバイスが有効ならその解像度を返す
00191     return camera ? camera->getSize() : std::array<int, 2>{ 0, 0 };
00192 }
00193
00194 //
00195 // キャプチャデバイスのフレームレートを取得する
00196 //
00197 double Capture::getFps() const
00198 {
00199     // キャプチャデバイスが有効ならそのフレームレートを返す
00200     return camera ? camera->getFps() : 0.0;
00201 }
00202
00203 //
00204 // フレームを取得する

```



```

00205 //
00206 void Capture::retrieve(Buffer& buffer)
00207 {
00208     // キャプチャデバイスが有効なら
00209     if (camera)
00210     {
00211         // バッファのサイズを取得したフレームのサイズに合わせて
00212         buffer.create(camera->getWidth(), camera->getHeight(), camera->getChannels());
00213
00214         // バッファのピクセルバッファオブジェクトにフレームを転送する
00215         camera->transmit(buffer.getBufferName());
00216     }
00217 }

```

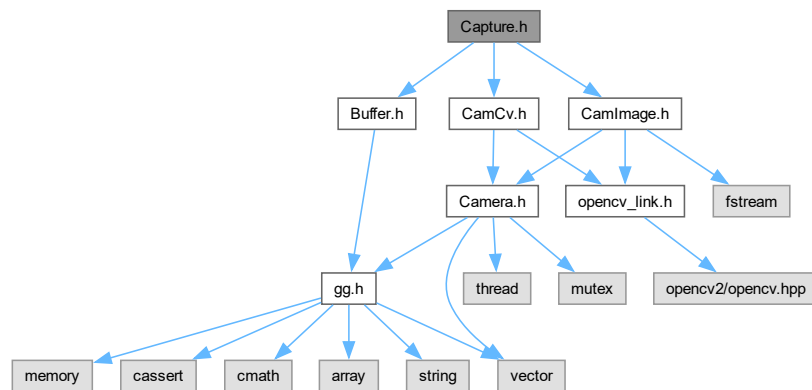
9.23 Capture.h ファイル

```

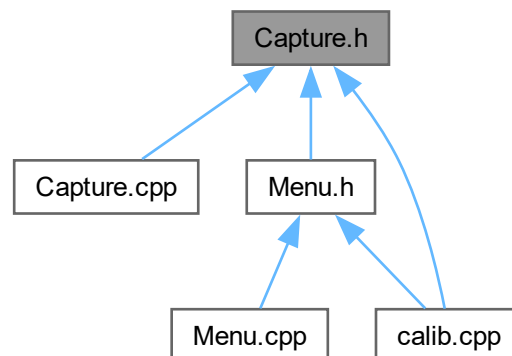
#include "Buffer.h"
#include "CamImage.h"
#include "CamCv.h"

```

Capture.h の依存先関係図:



被依存関係図:



クラス

- class [Capture](#)

9.23.1 詳解

キャプチャクラスの定義

著者

Kohe Tokoi

日付

April 3, 2023

[Capture.h](#) に定義があります。

9.24 Capture.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // バッファクラス
00012 #include "Buffer.h"
00013
00014 // OpenCV による画像ファイルの入力
00015 #include "CamImage.h"
00016
00017 // OpenCV による動画の入力
00018 #include "CamCv.h"
00019
00020 // Windows のみ Microsoft Media Foundation による動画の入力
00021 #if defined(_WIN32)
00022 #include "CamMf.h"
00023 #endif
00024
00028 class Capture
00029 {
00031     std::unique_ptr<Camera> camera;
00032
00033     #if defined(_WIN32)
00035         std::vector<std::string> deviceFormatList;
00036
00038         static const std::vector<std::string> emptyFormatList;
00039     #endif
00040
00041 public:
00042
00046     Capture()
00047     #if defined(_WIN32)
00048         : camera{ nullptr }
00049     #endif
00050     {
00051     }
00052
00058     Capture(const std::string& filename)
00059     #if defined(_WIN32)
00060         : camera{ nullptr }
00061     #endif
00062     {
00063         openImage(filename);
00064     }
00065
00069     virtual ~Capture()
00070     {
00071         close();
00072     }
00073
00080     bool openImage(const std::string& filename);
00081
00089     bool openMovie(const std::string& filename,
00090     #if defined(_WIN32)
00091         cv::VideoCaptureAPIs backend = cv::CAP_ANY);
00092     #else
00093         cv::VideoCaptureAPIs backend = cv::CAP_FFMPEG);
00094     #endif
```

```

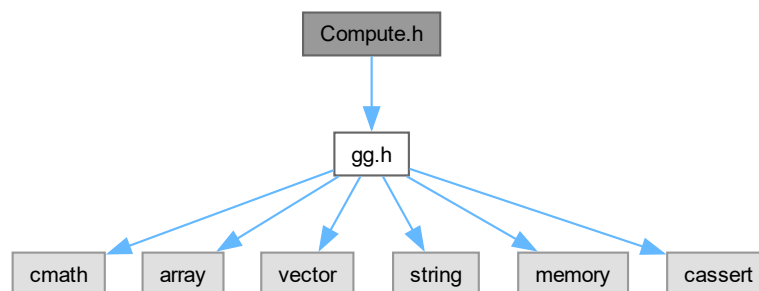
00095
00096 #if defined(_WIN32)
00103     bool openDevice(int deviceNumber);
00104
00111     const std::vector<std::string>& getFormatList() const;
00112
00118     bool select(int index);
00119
00125     void updateFormatList(int deviceNumber);
00126 #else
00137     bool openDevice(int deviceNumber,
00138                     std::array<int, 2>& size, double& fps,
00139                     cv::VideoCaptureAPIs backend,
00140                     char* fourcc);
00141 #endif
00142
00146     void start();
00147
00151     void stop();
00152
00156     void close();
00157
00163     explicit operator bool() const
00164     {
00165         return camera && camera->isRunning();
00166     }
00167
00171     bool isOpend() const
00172     {
00173         return bool(camera);
00174     }
00175
00179     bool isImage() const
00180     {
00181         return camera && dynamic_cast<const CamImage*>(camera.get()) != nullptr;
00182     }
00183
00189     std::array<int, 2> getSize() const;
00190
00196     double getFps() const;
00197
00203     void retrieve(Buffer& buffer);
00204 };

```

9.25 Compute.h ファイル

```
#include "gg.h"
```

Compute.h の依存先関係図:



クラス

- class [Compute](#)

9.25.1 詳解

画像処理クラスの定義 (コンピュートシェーダ版)

著者

Kohe Tokoi

日付

November 15, 2022

[Compute.h](#) に定義があります。

9.26 Compute.h

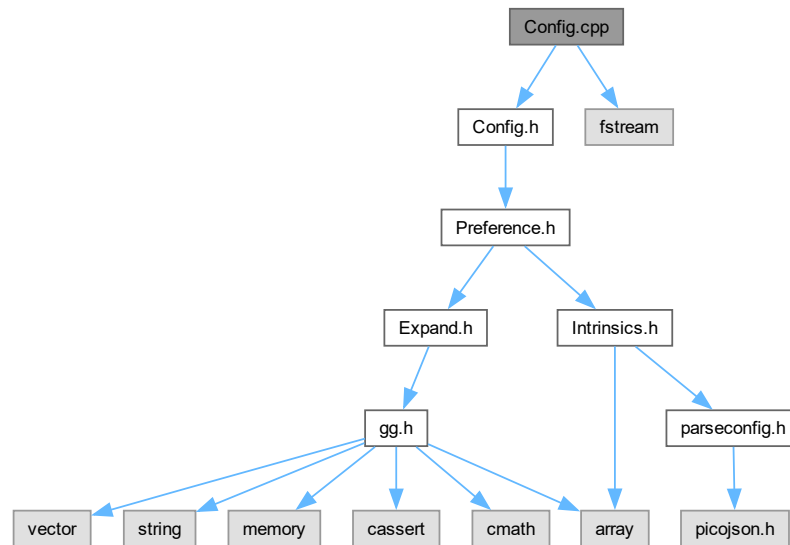
[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00018 class Compute
00019 {
00021     const GLuint program;
00022
00023 public:
00024
00030     Compute(const char* comp)
00031         : program(ggLoadComputeShader(comp))
00032     {
00033     }
00034
00038     virtual ~Compute()
00039     {
00040         // シェーダプログラムを削除する
00041         glDeleteShader(program);
00042     }
00043
00049     auto get() const
00050     {
00051         return program;
00052     }
00053
00057     void use() const
00058     {
00059         glUseProgram(program);
00060     }
00061
00070     void execute(GLuint width, GLuint height, GLuint local-size.x = 1, GLuint local-size.y = 1) const
00071     {
00072         glDispatchCompute((width + local-size.x - 1) / local-size.x, (height + local-size.y - 1) /
00073             local-size.y, 1);
00073     }
00074 };
```

9.27 Config.cpp ファイル

```
#include "Config.h"
#include <fstream>
```

Config.cpp の依存先関係図:



9.27.1 詳解

構成データクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Config.cpp](#) に定義があります。

9.28 Config.cpp

[詳解]

```

00001
00008 #include "Config.h"
00009
00010 // 標準ライブラリ
00011 #include <fstream>
00012
00013 #if defined(_WIN32)
00014 // Microsoft Media Foundation によるキャプチャ
00015 #include "CamMf.h"
00016 #endif
00017
00018 #if !defined(_DEBUG) && defined(_WIN32)
00019 // appData のパスを得るときに使う
00020 #include <shlobj_core.h>
00021 #endif
00022
00023 //
00024 // コンストラクタ
00025 //
00026 Config::Config(const std::string& filename)
00027 : title{ PROJECT_NAME }
00028 , windowSize{ 1280, 720 }
00029 , background{ 0.2f, 0.3f, 0.4f, 1.0f }
00030 , settings{ "DICT_4X4_50" }

```

```

00031     , menuFont{ "Mplus1-Regular.ttf" }
00032     , menuFontSize{ 20.0f }
00033 #if defined(WIN32)
00034     , deviceList{ CamMf::getDeviceList() }
00035 #endif
00036 {
00037     // 構成ファイルの保存場所を決定する
00038     #if defined(_DEBUG)
00039         const auto path{ Utf8ToTChar(filename) };
00040     #else
00041         # if defined(WIN32)
00042             // 構成ファイルの保存先のパス
00043             wchar_t appDataPath[MAX_PATH];
00044
00045             // AppData のパスを取得する
00046             SHGetSpecialFolderPathW(NULL, appDataPath, CSIDL_APPDATA, 0);
00047
00048             // AppData のパスに構成ファイル名を連結する
00049             const auto path{ CString(appDataPath) + TEXT("\\") + Utf8ToTChar(filename) };
00050         # else
00051             // パスワードファイルのエントリを取得する
00052             const passwd* pw{ getpwuid(getuid()) };
00053
00054             // ホームディレクトリのパスに構成ファイル名を連結する
00055             const auto path{ std::string(pw->pw_dir) + "/" + filename };
00056         # endif
00057     #endif
00058
00059     // 構成ファイルを読み込む
00060     if (!load(path))
00061     {
00062         // デフォルトの設定を追加する
00063         if (!load(Utf8ToTChar(filename))) preferenceList.emplace_back();
00064
00065         // デフォルトの設定を入れた構成ファイルを作成する
00066         save(path);
00067     }
00068 }
00069
00070 //
00071 // デストラクタ
00072 //
00073 Config::~Config()
00074 {
00075 }
00076
00077 //
00078 // 構成データを初期化する
00079 //
00080 void Config::initialize()
00081 {
00082     // 構成リストのすべて構成についてシェーダをビルドする
00083     for (auto& preference : preferenceList) preference.buildShader();
00084
00085     // 背景色を設定する
00086     glClearColor(background[0], background[1], background[2], background[3]);
00087 }
00088
00089 //
00090 // 構成ファイルを読み込む
00091 //
00092 bool Config::load(const pathString& filename)
00093 {
00094     // 構成ファイルの読み込み
00095     std::ifstream json{ filename };
00096     if (!json) return false;
00097
00098     // JSON の読み込み
00099     picojson::value value;
00100     json >> value;
00101     json.close();
00102
00103     // 構成内容の取り出し
00104     const auto& object{ value.get<picojson::object>() };
00105
00106     // オブジェクトが空だったらエラー
00107     if (object.empty()) return false;
00108
00109     // ウィンドウのサイズ
00110     getValue(object, "size", windowSize);
00111
00112     // ウィンドウの背景色
00113     getValue(object, "background", background);
00114
00115     // メッシュのサンプル数
00116     getValue(object, "samples", settings.samples);
00117

```

```

00118 // キャプチャデバイスの姿勢
00119 getValue(object, "pose", settings.euler);
00120
00121 // 描画時の焦点距離
00122 getValue(object, "focal", settings.focal);
00123
00124 // 描画時の焦点距離の範囲
00125 getValue(object, "range", settings.focalRange);
00126
00127 // ArUco Marker の辞書名
00128 getString(object, "dictionary", settings.dictionaryName);
00129
00130 // 初期表示画像
00131 getString(object, "initial", initialImage);
00132
00133 // GStreamer のパイプライン設定リスト
00134 getString(object, "gststreamer", gststreamerPipelines);
00135
00136 // キャプチャデバイスの構成を探す
00137 const auto& camera{ object.find("camera") };
00138
00139 // キャプチャデバイスの構成の配列が見つからなければエラー
00140 if (camera == object.end() || !camera->second.is<picojson::array>()) return false;
00141
00142 // 配列の個々の要素について
00143 for (const auto& value : camera->second.get<picojson::array>())
00144 {
00145     // キャプチャデバイスの構成のオブジェクトを取り出す
00146     const auto& preference{ value.get<picojson::object>() };
00147
00148     // キャプチャデバイスの構成をリストに追加
00149     preferenceList.emplace_back(preference);
00150 }
00151
00152 // キャプチャデバイスの構成が一つも読み取れなければエラー
00153 return !preferenceList.empty();
00154 }
00155
00156 //
00157 // 構成ファイルを保存する
00158 //
00159 bool Config::save(const pathString& filename) const
00160 {
00161     // 設定値を保存する
00162     std::ofstream config{ filename };
00163     if (!config) return false;
00164
00165     // オブジェクト
00166     picojson::object object;
00167
00168     // ウィンドウのサイズ
00169     setValue(object, "size", windowSize);
00170
00171     // ウィンドウの背景色
00172     setValue(object, "background", background);
00173
00174     // メッシュのサンプル数
00175     setValue(object, "samples", settings.samples);
00176
00177     // キャプチャデバイスの姿勢
00178     setValue(object, "pose", settings.euler);
00179
00180     // 描画時の焦点距離
00181     setValue(object, "focal", settings.focal);
00182
00183     // 描画時の焦点距離の範囲
00184     setValue(object, "range", settings.focalRange);
00185
00186     // ArUco Marker 辞書名
00187     setValue(object, "dictionary", settings.dictionaryName);
00188
00189     // 初期表示画像
00190     setValue(object, "initial", initialImage);
00191
00192     // GStreamer のパイプライン設定リスト
00193     setValue(object, "gststreamer", gststreamerPipelines);
00194
00195     // 配列
00196     picojson::array array;
00197
00198     // 構成リストのすべて構成について
00199     for (const auto& preference : preferenceList)
00200     {
00201         // 構成の要素
00202         picojson::object camera;
00203
00204         // 構成を JSON オブジェクトに格納する

```

```

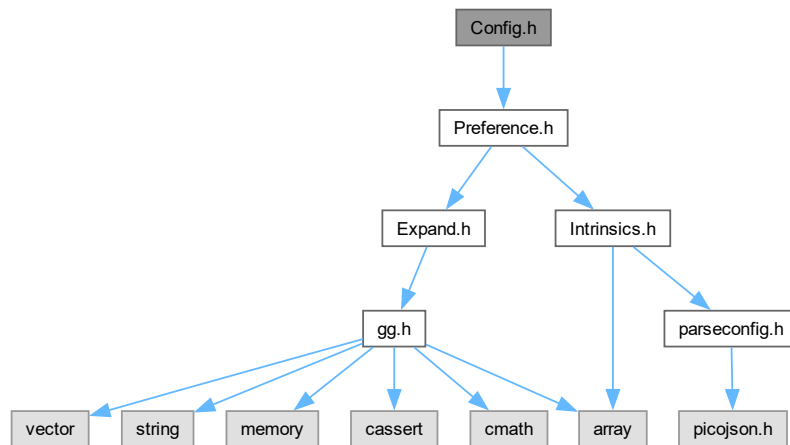
00205     preference.setPreference(camera);
00206
00207     // 要素を picojson::array に追加する
00208     array.emplace_back(picojson::value(camera));
00209 }
00210
00211 // オブジェクトに追加する
00212 object.emplace("camera", array);
00213
00214 // 構成をシリアライズして保存
00215 picojson::value v{ object };
00216 config << v.serialize(true);
00217 config.close();
00218
00219 // 構成データの書き込み成功
00220 return true;
00221 }

```

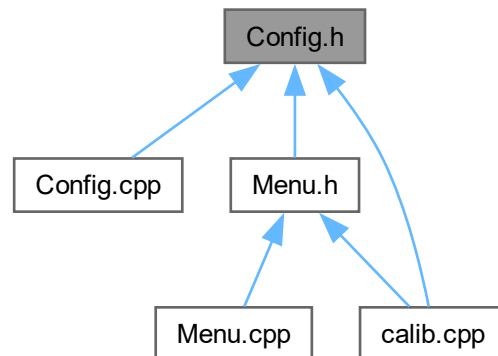
9.29 Config.h ファイル

#include "Preference.h"

Config.h の依存先関係図:



被依存関係図:



クラス

- struct [Settings](#)
- class [Config](#)

9.29.1 詳解

構成データクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Config.h](#) に定義があります。

9.30 Config.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // キャプチャデバイスの構成
00012 #include "Preference.h"
00013
00017 struct Settings
00018 {
00020     int samples;
00021
00023     std::array<float, 3> euler;
00024
00026     static constexpr decltype(euler) defaultEuler{ 0.0f, 0.0f, 0.0f };
00027
00029     float focal;
00030
00032     static constexpr decltype(focal) defaultFocal{ 50.0f };
00033
00035     std::array<float, 2> focalRange;
00036
00038     static constexpr decltype(focalRange) defaultFocalRange{ 10.0f, 200.0f };
00039
```

```

00041     std::string dictionaryName;
00042
00044     std::array<float, 2> checkerLength;
00045
00047     float markerLength;
00048
00052     Settings(const std::string& dictionaryName)
00053     : samples{ 57600 }
00054     , euler{ defaultEuler }
00055     , focal{ defaultFocal }
00056     , focalRange{ defaultFocalRange }
00057     , dictionaryName{ dictionaryName }
00058     , checkerLength{ 4.0f, 2.0f }
00059     , markerLength{ 5.0f }
00060     {}
00061
00065     auto getFocal() const
00066     {
00067         // 投影面の対角線長は 35mm (17.5mm × 2) とする
00068         return focal / 17.5f;
00069     }
00070 };
00071
00075 class Config
00076 {
00078     friend class Menu;
00079
00081     std::string title;
00082
00084     std::array<GLsizei, 2> windowSize;
00085
00087     std::array<GLfloat, 4> background;
00088
00090     Settings settings;
00091
00093     std::string menuFont;
00094
00096     float menuFontSize;
00097
00099     static std::string initialImage;
00100
00102     std::vector<std::string> gstreamerPipelines;
00103
00105     std::vector<Preference> preferenceList;
00106
00107 #if defined(_WIN32)
00109     const std::vector<std::string>& deviceList;
00110 #endif
00111
00112 public:
00113
00119     Config(const std::string& filename);
00120
00124     virtual ~Config();
00125
00131     void initialize();
00132
00139     bool load(const pathString& filename);
00140
00147     bool save(const pathString& filename) const;
00148
00154     const auto& getTitle() const
00155     {
00156         return title;
00157     }
00158
00164     auto getWidth() const
00165     {
00166         return windowSize[0];
00167     }
00168
00174     auto getHeight() const
00175     {
00176         return windowSize[1];
00177     }
00178
00184     const auto& getInitialImage() const
00185     {
00186         return initialImage;
00187     }
00188
00194     const auto& getDictionaryName() const
00195     {
00196         return settings.dictionaryName;
00197     }
00198
00204     const auto& getCheckerLength() const

```

```

00205 {
00206     return settings.checkerLength;
00207 }
00208
00214 auto getMarkerLength() const
00215 {
00216     return settings.markerLength;
00217 }
00218
00219 #if defined(_WIN32)
00225 const auto& getDeviceList() const
00226 {
00227     return deviceList;
00228 }
00229
00236 const auto& getDeviceName(int number) const
00237 {
00238     static const std::string empty{};
00239     return deviceList.empty() ? empty : deviceList[number];
00240 }
00241 #endif
00242 };

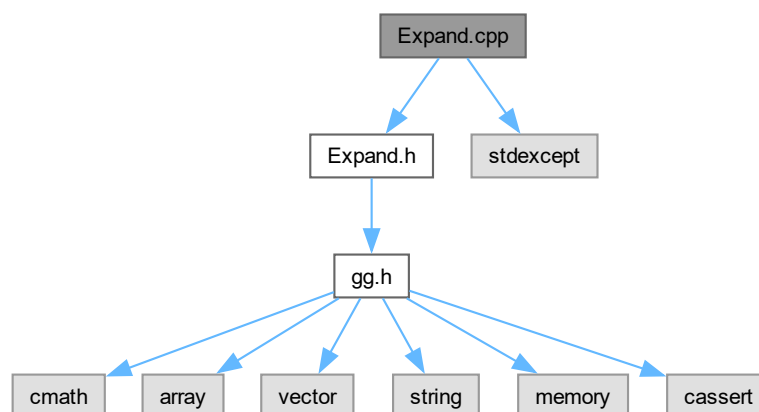
```

9.31 Expand.cpp ファイル

```

#include "Expand.h"
#include <stdexcept>
Expand.cpp の依存先関係図:

```



9.32 Expand.cpp

[詳解]

```

00001 //
00002 // 展開用シェーダ
00003 //
00004 #include "Expand.h"
00005
00006 // 標準ライブラリ
00007 #include <stdexcept>
00008
00009 //
00010 // コンストラクタ
00011 //
00012 Expand::Expand(const std::string& vert, const std::string& frag)
00013 : program{ gg::ggLoadShader(vert, frag) }
00014 , imageLoc{ glGetUniformLocation(program, "image") }
00015 , screenLoc{ glGetUniformLocation(program, "screen") }
00016 , focalLoc{ glGetUniformLocation(program, "focal") }
00017 , rotationLoc{ glGetUniformLocation(program, "rotation") }

```

```

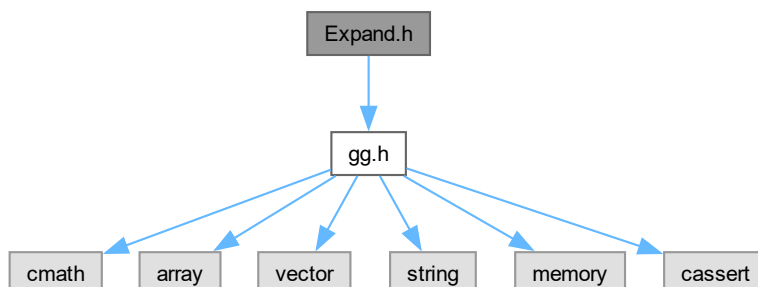
00018     , circleLoc{ glGetUniformLocation(program, "circle") }
00019     , borderLoc{ glGetUniformLocation(program, "border") }
00020     , gapLoc{ glGetUniformLocation(program, "gap") }
00021 {
00022     // プログラムオブジェクトが作れなかったら落とす
00023     if (program == 0) throw std::runtime.error("Cannot create one of the expand shader.");
00024 }
00025
00026 //
00027 // デストラクタ
00028 //
00029 Expand::~Expand()
00030 {
00031     glDeleteProgram(program);
00032 }
00033
00037 std::array<int, 2> Expand::setup(int samples, GLfloat aspect, const gg::GgMatrix& pose,
00038     const std::array<GLfloat, 2>& fov, const std::array<GLfloat, 2>& center, GLfloat focal,
00039     const std::array<GLfloat, 4>& border, int unit) const
00040 {
00041     // プログラムオブジェクトの指定
00042     glUseProgram(program);
00043
00044     // テクスチャユニットの指定
00045     glUniform1i(imageLoc, unit);
00046
00047     // 境界色
00048     glUniform4fv(borderLoc, 1, border.data());
00049
00050     // 投影像の画角 (度) と中心位置
00051     glUniform4f(circleLoc, fov[0], fov[1], center[0], center[1]);
00052
00053     // スクリーンのサイズと中心位置
00054     // screen[0] = (right - left) / 2
00055     // screen[1] = (top - bottom) / 2
00056     // screen[2] = (right + left) / 2
00057     // screen[3] = (top + bottom) / 2
00058     const GLfloat screen[] { aspect, 1.0f, 0.0f, 0.0f };
00059     glUniform4fv(screenLoc, 1, screen);
00060
00061     // レンズの主点とスクリーンの距離
00062     glUniform1f(focalLoc, focal);
00063
00064     // 背景に対する視線の回転行列
00065     glUniformMatrix4fv(rotationLoc, 1, GL_FALSE, pose.get());
00066
00067     // メッシュの横の格子点数
00068     // 標本点の数 (頂点数) samples = w * h とするとき、これに縦横比
00069     // aspect = w / h をかければ aspect * samples = w * w となるから、
00070     // w = sqrt(aspect * samples), h = samples / w で求められる
00071     const auto w{ static_cast<int>(sqrt(aspect * samples)) };
00072     const auto h{ samples / w };
00073
00074     // 格子間隔
00075     // クリッピング空間全体を埋める四角形は [-1, 1] の範囲すなわち
00076     // 縦横 2 の大きさだから、それを縦横の (格子数 - 1) で割る
00077     std::array<GLfloat, 2> gap{ 2.0f / (w - 1), 2.0f / (h - 1) };
00078     glUniform2fv(gapLoc, 1, gap.data());
00079
00080     // 描画するメッシュの横と縦の格子点数を返す
00081     return std::array<int, 2>{ w, h };
00082 }

```

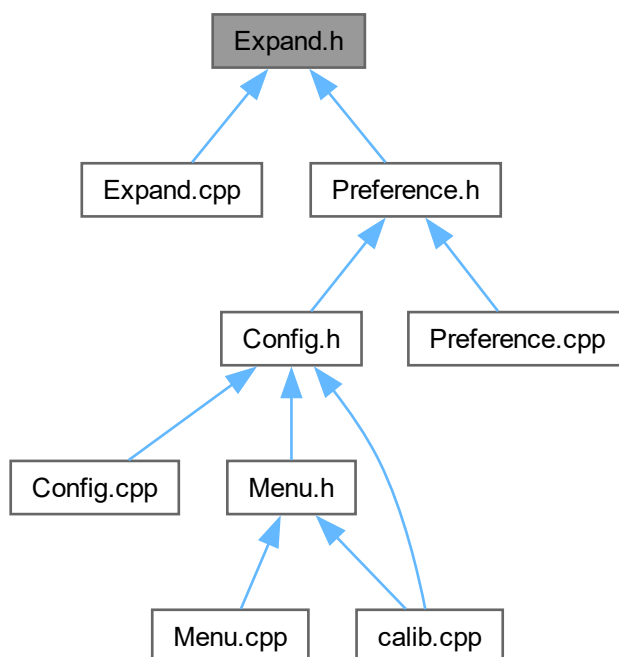
9.33 Expand.h ファイル

```
#include "gg.h"
```

Expand.h の依存先関係図:



被依存関係図:



クラス

- class [Expand](#)

9.33.1 詳解

展開用シェードクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Expand.h](#) に定義があります。

9.34 Expand.h

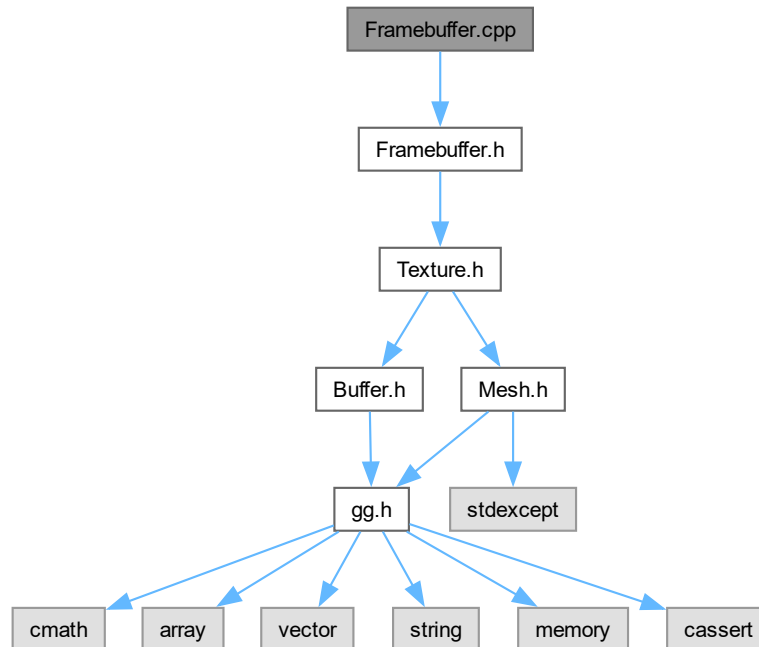
[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013
00017 class Expand
00018 {
00020     const GLuint program;
00021
00023     const GLint imageLoc;
00024
00026     const GLint screenLoc;
00027
00029     const GLint focalLoc;
00030
00032     const GLint rotationLoc;
00033
00035     const GLint circleLoc;
00036
00038     const GLint borderLoc;
00039
00041     const GLint gapLoc;
00042
00043 public:
00044
00051     Expand(const std::string& vert, const std::string& frag);
00052
00058     Expand(const Expand& shader) = delete;
00059
00063     virtual ~Expand();
00064
00071     Expand& operator=(const Expand& shader) = delete;
00072
00078     auto getProgram() const
00079     {
00080         return program;
00081     }
00082
00100     std::array<GLsizei, 2> setup(int samples, GLfloat aspect, const gg::GgMatrix& pose,
00101         const std::array<GLfloat, 2>& fov, const std::array<GLfloat, 2>& center, GLfloat focal,
00102         const std::array<GLfloat, 4>& border, int unit = 0) const;
00103 };
```

9.35 Framebuffer.cpp ファイル

```
#include "Framebuffer.h"
```

Framebuffer.cpp の依存先関係図:



9.35.1 詳解

フレームバッファオブジェクトクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Framebuffer.cpp](#) に定義があります。

9.36 Framebuffer.cpp

[詳解]

```

00001
00008 #include "Framebuffer.h"
00009
00010 //
00011 // フレームバッファオブジェクトを作成するコンストラクタ
00012 //
00013 Framebuffer::Framebuffer(GLsizei width, GLsizei height, int channels,
00014   GLenum attachment)
00015   : attachment{ attachment }
00016   {
00017   // フレームバッファオブジェクトを作る
00018   Framebuffer::create(width, height, channels);
00019   }

```

```

00020
00026 Framebuffer::Framebuffer(const Framebuffer& framebuffer)
00027 : attachment{ framebuffer.attachment }
00028 {
00029     // フレームバッファオブジェクトをコピーして作成する
00030     Framebuffer::copy(framebuffer);
00031 }
00032
00033 //
00034 // デストラクタ
00035 //
00036 Framebuffer::~Framebuffer()
00037 {
00038     // フレームバッファオブジェクトを破棄する
00039     Framebuffer::discard();
00040 }
00041
00042 //
00043 // 代入演算子
00044 //
00045 Framebuffer& Framebuffer::operator=(const Framebuffer& framebuffer)
00046 {
00047     // 代入元と代入先が同じでなければ
00048     if (&framebuffer != this)
00049     {
00050         // 引数のフレームバッファオブジェクトをコピーする
00051         Framebuffer::copy(framebuffer);
00052     }
00053
00054     // このフレームバッファオブジェクトを返す
00055     return *this;
00056 }
00057
00058 //
00059 // ムーブ代入演算子
00060 //
00061 Framebuffer& Framebuffer::operator=(Framebuffer&& framebuffer) noexcept
00062 {
00063     // ムーブ代入元とムーブ代入先が同じなら何もしない
00064     if (&framebuffer == this) return *this;
00065
00066     // ムーブ代入元の基底クラスをムーブする
00067     static_cast<Texture&>(*this) = std::move(framebuffer);
00068
00069     // ムーブ代入元のテクスチャのメンバをムーブする
00070     framebufferSize = framebuffer.framebufferSize;
00071     framebuffer.framebufferSize = { 0, 0 };
00072     framebufferChannels = framebuffer.framebufferChannels;
00073     framebuffer.framebufferChannels = 0;
00074     framebuffer.attachment = GL_COLOR_ATTACHMENT0;
00075
00076     // ムーブ代入先のフレームバッファを削除する
00077     glDeleteFramebuffers(1, &framebufferName);
00078
00079     // ムーブ代入元のフレームバッファ名をムーブ代入先に移す
00080     framebufferName = framebuffer.framebufferName;
00081
00082     // ムーブ代入元のデストラクタでフレームバッファが削除されないよう 0 にする
00083     framebuffer.framebufferName = 0;
00084
00085     // このフレームバッファオブジェクトを返す
00086     return *this;
00087 }
00088
00089 //
00090 // フレームバッファオブジェクトを破棄する
00091 //
00092 void Framebuffer::discard()
00093 {
00094     // デフォルトのフレームバッファオブジェクトに戻す
00095     glBindFramebuffer(GL_FRAMEBUFFER, 0);
00096
00097     // フレームバッファオブジェクトを削除する
00098     glDeleteFramebuffers(1, &framebufferName);
00099     framebufferName = 0;
00100
00101     // バッファオブジェクトのメンバを初期化する
00102     framebufferSize = std::array<int, 2>{ 0, 0 };
00103     framebufferChannels = 0;
00104     attachment = GL_COLOR_ATTACHMENT0;
00105 }
00106
00107 //
00108 // フレームバッファオブジェクトを作成する
00109 //
00110 void Framebuffer::create(GLsizei width, GLsizei height, int channels)
00111 {

```



```

00112 // 既存のテクスチャを破棄して新しいテクスチャを作成する
00113 Texture::create(width, height, channels);
00114
00115 // またメッシュの頂点配列オブジェクトが作られていなければ作る
00116 if (!mesh) mesh = std::make_shared<Mesh>();
00117
00118 // 指定したサイズがフレームバッファオブジェクトのサイズと同じなら何もしない
00119 if (width == framebufferSize[0] && height == framebufferSize[1]
00120     && channels == framebufferChannels) return;
00121
00122 // フレームバッファオブジェクトのサイズとチャンネル数を記録する
00123 framebufferSize = std::array<int, 2>{ width, height };
00124 framebufferChannels = channels;
00125
00126 // 以前のフレームバッファオブジェクトを削除する
00127 glDeleteFramebuffers(1, &framebufferName);
00128
00129 // フレームバッファオブジェクトを作成する
00130 glGenFramebuffers(1, &framebufferName);
00131 glBindFramebuffer(GL_FRAMEBUFFER, framebufferName);
00132 glFramebufferTexture2D(GL_FRAMEBUFFER, attachment, GL_TEXTURE_2D, getTextureName(), 0);
00133 glDrawBuffers(1, &attachment);
00134 glReadBuffer(attachment);
00135 glBindFramebuffer(GL_FRAMEBUFFER, 0);
00136 }
00137
00138 //
00139 // フレームバッファオブジェクトをコピーする
00140 //
00141 void Framebuffer::copy(const Buffer& framebuffer) noexcept
00142 {
00143     // コピー元と同じサイズの空のフレームバッファオブジェクトを作る
00144     Framebuffer::create(framebuffer.getWidth(), framebuffer.getHeight(),
00145         framebuffer.getChannels());
00146
00147     // 作成したフレームバッファオブジェクトのバッファにコピーする
00148     copyBuffer(framebuffer);
00149 }
00150
00151 //
00152 // レンダリング先をフレームバッファオブジェクトに切り替える
00153 //
00154 void Framebuffer::bindFramebuffer()
00155 {
00156     // 描画先をフレームバッファオブジェクトに切り替える
00157     glBindFramebuffer(GL_FRAMEBUFFER, framebufferName);
00158
00159     // ビューポートをフレームバッファオブジェクトに合わせる
00160     glViewport(0, 0, framebufferSize[0], framebufferSize[1]);
00161 }
00162
00163 //
00164 // レンダリング先を通常のフレームバッファに戻す
00165 //
00166 void Framebuffer::unbindFramebuffer() const
00167 {
00168     // 描画先を通常のフレームバッファに戻す
00169     glBindFramebuffer(GL_FRAMEBUFFER, 0);
00170
00171     // 読み書きを通常のフレームバッファのバックバッファに対して行う
00172     static constexpr GLenum bufs{
00173 #if defined(GL_GLES_PROTOTYPES)
00174         GL_BACK
00175 #else
00176         GL_BACK_LEFT
00177 #endif
00178     };
00179     glDrawBuffers(1, &bufs);
00180     glReadBuffer(GL_BACK);
00181 }
00182
00183 //
00184 // テクスチャを展開してフレームバッファオブジェクトを更新する
00185 //
00186 void Framebuffer::update(const std::array<int, 2>& size)
00187 {
00188     // 描画先をフレームバッファオブジェクトに切り替える
00189     bindFramebuffer();
00190
00191     // テクスチャをフレームバッファオブジェクトに展開する
00192     mesh->drawMesh(size);
00193
00194     // 描画先を通常のフレームバッファに戻す
00195     unbindFramebuffer();
00196 }
00197
00198 //

```

```

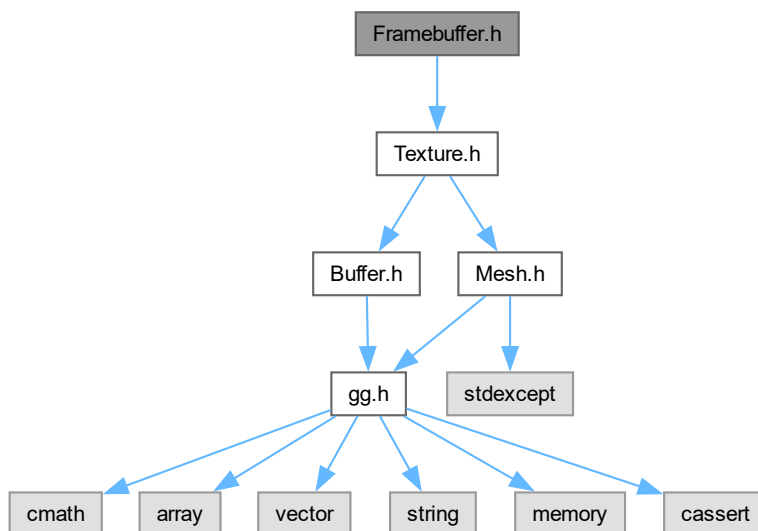
00199 // テクスチャを展開してフレームバッファオブジェクトを更新する
00200 //
00201 void Framebuffer::update(const std::array<int, 2>& size, const Texture& frame, int unit)
00202 {
00203     // 展開するするテクスチャを指定する
00204     frame.bindTexture(unit);
00205
00206     // テクスチャをフレームバッファオブジェクトに展開する
00207     update(size);
00208
00209     // 展開するするテクスチャの指定を解除する
00210     frame.unbindTexture();
00211 }
00212
00213 //
00214 // フレームバッファオブジェクトの表示
00215 //
00216 void Framebuffer::show(GLsizei width, GLsizei height) const
00217 {
00218     // フレームバッファオブジェクトの縦横比
00219     const auto f{ static_cast<float>(framebufferSize[0] * height) };
00220
00221     // ウィンドウの表示領域の縦横比
00222     const auto d{ static_cast<float>(framebufferSize[1] * width) };
00223
00224     // 実際に描画する領域
00225     GLint dx0, dy0, dx1, dy1;
00226
00227     // 表示領域の右上端の位置を求める
00228     --width;
00229     --height;
00230
00231     // フレームバッファオブジェクトの縦横比が大きかったら
00232     if (f > d)
00233     {
00234         // ウィンドウの表示領域の高さを求める
00235         const auto h{ static_cast<GLint>(d / getWidth() + 0.5f) };
00236
00237         // 表示が横長なので表示領域の横幅いっぱいに表示する
00238         dx0 = 0;
00239         dx1 = width;
00240
00241         // 高さは縦横比を維持して描画する領域の中央に描く
00242         dy0 = (height - h) / 2;
00243         dy1 = dy0 + h;
00244     }
00245     else
00246     {
00247         // ウィンドウの表示領域の幅を求める
00248         const auto w{ static_cast<GLint>(f / getHeight() + 0.5f) };
00249
00250         // 表示が縦長なので表示領域の高さいっぱいに表示する
00251         dy0 = 0;
00252         dy1 = height;
00253
00254         // 横幅は縦横比を維持して描画する領域の中央に描く
00255         dx0 = (width - w) / 2;
00256         dx1 = dx0 + w;
00257     }
00258
00259     // フレームバッファオブジェクトを読み込み元にする
00260     glBindFramebuffer(GL_READ_FRAMEBUFFER, framebufferName);
00261     glReadBuffer(attachment);
00262
00263     // 現在のフレームバッファを背景色で塗りつぶす
00264     glClear(GL_COLOR_BUFFER_BIT);
00265
00266     // フレームバッファオブジェクトの内容を通常のフレームバッファに書き込む
00267     glBlitFramebuffer(0, 0, getWidth() - 1, getHeight() - 1,
00268         dx0, dy1, dx1, dy0, GL_COLOR_BUFFER_BIT, GL_NEAREST);
00269
00270     // 読み込み元を通常のフレームバッファに戻す
00271     glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
00272     glReadBuffer(GL_BACK);
00273 }

```

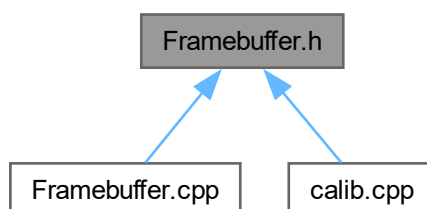
9.37 Framebuffer.h ファイル

```
#include "Texture.h"
```

Framebuffer.h の依存先関係図:



被依存関係図:



クラス

- class [Framebuffer](#)

9.37.1 詳解

フレームバッファオブジェクトクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Framebuffer.h](#) に定義があります。

9.38 Framebuffer.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // テクスチャ
00012 #include "Texture.h"
00013
00017 class Framebuffer : public Texture
00018 {
00020     std::array<int, 2> framebufferSize;
00021
00023     int framebufferChannels;
00024
00026     GLuint framebufferName;
00027
00029     GLenum attachment;
00030
00031 public:
00032
00036     Framebuffer()
00037     : Texture{
00038         , framebufferSize{ 0, 0 }
00039         , framebufferChannels{ 0 }
00040         , framebufferName{ 0 }
00041         , attachment{ GL_COLOR_ATTACHMENT0 }
00042     }
00043
00044
00053     Framebuffer(GLsizei width, GLsizei height, int channels = 3,
00054                 GLenum attachment = GL_COLOR_ATTACHMENT0);
00055
00061     Framebuffer(const Framebuffer& framebuffer);
00062
00068     Framebuffer(Framebuffer&& framebuffer) noexcept
00069     : Texture{ std::move(framebuffer) }
00070     , framebufferSize{ framebuffer.framebufferSize }
00071     , framebufferChannels{ framebuffer.framebufferChannels }
00072     , framebufferName{ framebuffer.framebufferName }
00073     , attachment{ framebuffer.attachment }
00074     {
00075         framebuffer.framebufferSize = { 0, 0 };
00076         framebuffer.framebufferChannels = 0;
00077
00078         // ムーブ元のデストラクタでフレームバッファが削除されないよう 0 にする
00079         framebuffer.framebufferName = 0;
00080         framebuffer.attachment = GL_COLOR_ATTACHMENT0;
00081     }
00082
00086     virtual ~Framebuffer();
00087
00094     Framebuffer& operator=(const Framebuffer& framebuffer);
00095
00102     Framebuffer& operator=(Framebuffer&& framebuffer) noexcept;
00103
00116     virtual void create(GLsizei width, GLsizei height, int channels);
00117
00131     virtual void copy(const Buffer& framebuffer) noexcept;
00132
00136     virtual void discard();
00137
00143     auto getFramebufferName() const
00144     {
00145         return framebufferName;
00146     }
00147
00153     virtual const std::array<GLsizei, 2>& getSize() const
00154     {
00155         return framebufferSize;
00156     }
00157
00163     virtual int getChannels() const
00164     {
00165         return framebufferChannels;
00166     }
00167

```

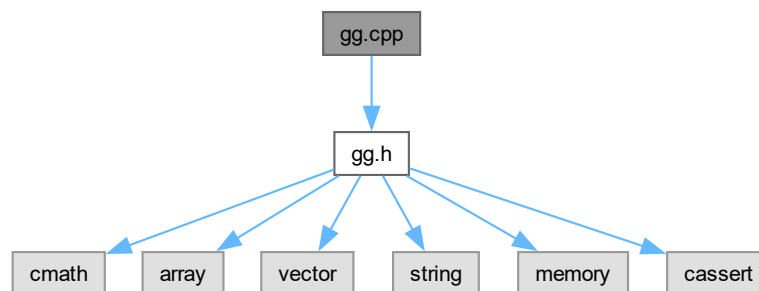
```

00177 void bindFramebuffer();
00178
00188 void unbindFramebuffer() const;
00189
00200 void update(const std::array<int, 2>& size);
00201
00217 void update(const std::array<int, 2>& size, const Texture& frame, int unit = 0);
00218
00225 void show(GLsizei width, GLsizei height) const;
00226 };

```

9.39 gg.cpp ファイル

#include "gg.h"
gg.cpp の依存先関係図:



9.39.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の定義.

著者

Kohe Tokoi

日付

July 17, 2025

[gg.cpp](#) に定義があります。

9.40 gg.cpp

[[詳解](#)]

```

00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,

```

```

00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00032 #include "gg.h"
00033
00035
00036 // 標準ライブラリ
00037 #include <cmath>
00038 #include <cstdlib>
00039 #include <iostream>
00040 #include <fstream>
00041 #include <sstream>
00042 #include <limits>
00043 #include <map>
00044
00046 #define READ_TEXTURE_COORDINATE_FROM_OBJ 0
00047
00048 // Windows (Visual Studio) のとき
00049 #if defined(_WIN32)
00050 // デバッグビルドかどうか調べて
00051 # if defined(_DEBUG)
00052 // デバッグビルドならそのことを示す記号定数を別に定義して
00053 #   define DEBUG
00054 // デバッグビルド用のライブラリをリンクする
00055 #   pragma comment(lib, "glfw3.lib")
00056 # else
00057 // リリースビルドならコンソールにメッセージを出さないようにして
00058 #   pragma comment(linker, "/subsystem:\"windows\" /entry:\"mainCRTStartup\"")
00059 // リリースビルド用のライブラリをリンクする
00060 #   pragma comment(lib, "glfw3.lib")
00061 # endif
00062
00063 //
00064 // For VC++ MFC Convert UTF-8 to TCHAR, or Convert TCHAR to UTF-8. VC++ MFC用 UTF-8⇔TCHARの変換処理
00065 //
00066 // Original author: mt-u
00067 // Copyright (c) 2013 mt-u
00068 // https://gist.github.com/mt-u/6878251
00069 //
00070 // Modified by: Kohe Tokoi
00071 //
00072
00073 //
00074 // UTF-8 文字列を CString に変換する
00075 //
00076 pathString Utf8ToTChar(const std::string& string)
00077 {
00078     // UTF-8 文字列を UTF-16 に変換した後の文字列の長さを求める
00079     const INT length{ MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, NULL, 0) };
00080
00081     // 変換結果の格納に必要な長さのメモリを確保する
00082     std::vector<WCHAR> utf16(length);
00083
00084     // UTF-8 文字列を UTF-16 に変換する
00085     MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, utf16.data(), length);
00086
00087     // 変換した文字列を CString にして返す
00088     return CString{ utf16.data(), static_cast<int>(wcslen(utf16.data())) };
00089 }
00090
00091 //
00092 // CString を UTF-8 文字列に変換する
00093 //
00094 std::string TCharToUtf8(const pathString& cstring)
00095 {
00096     // 与えられた文字列を一旦 UTF-16 に変換する
00097     CStringW cstringw{ cstring };
00098
00099     // UTF-16 を UTF-8 に変換した後の文字列の長さを求める
00100     const INT length{ WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, NULL, 0, NULL, NULL) };
00101
00102     // 変換結果の格納に必要な長さのメモリを確保する
00103     std::vector<CHAR> utf8(length);
00104
00105     // UTF-16 を UTF-8 に変換する
00106     WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, utf8.data(), length, NULL, NULL);
00107
00108     // 変換した文字列を std::string にして返す
00109     return std::string{ utf8.data(), strlen(utf8.data()) };
00110 }
00111 #endif
00112
00113 // OpenGL 3.2 の API のエントリポイント

```

```
00114 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00115 PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00116 PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00117 PFNGLACTIVETEXTUREPROC glActiveTexture;
00118 PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00119 PFNGLATTACHSHADERPROC glAttachShader;
00120 PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00121 PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00122 PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00123 PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00124 PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00125 PFNGLBEGINQUERYPROC glBeginQuery;
00126 PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00127 PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00128 PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00129 PFNGLBINDBUFFERPROC glBindBuffer;
00130 PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00131 PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00132 PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00133 PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00134 PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00135 PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00136 PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00137 PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00138 PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00139 PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00140 PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00141 PFNGLBINDSAMPLERPROC glBindSampler;
00142 PFNGLBINDSAMPLERSPROC glBindSamplers;
00143 PFNGLBINDTEXTUREPROC glBindTexture;
00144 PFNGLBINDTEXTURESPROC glBindTextures;
00145 PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00146 PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00147 PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00148 PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00149 PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00150 PFNGLBLENDARRIERKHRPROC glBlendBarrierKHR;
00151 PFNGLBLENDARRIERNVPROC glBlendBarrierNV;
00152 PFNGLBLENDCOLORPROC glBlendColor;
00153 PFNGLBLEND EQUATIONIARBPROC glBlendEquationiARB;
00154 PFNGLBLEND EQUATIONIPROC glBlendEquationi;
00155 PFNGLBLEND EQUATIONPROC glBlendEquation;
00156 PFNGLBLEND EQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00157 PFNGLBLEND EQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00158 PFNGLBLEND EQUATIONSEPARATEPROC glBlendEquationSeparate;
00159 PFNGLBLEND FUNC IARBPROC glBlendFunciARB;
00160 PFNGLBLEND FUNC IPROC glBlendFunci;
00161 PFNGLBLEND FUNC PROC glBlendFunc;
00162 PFNGLBLEND FUNC SEPARATEIARBPROC glBlendFuncSeparateiARB;
00163 PFNGLBLEND FUNC SEPARATEIPROC glBlendFuncSeparatei;
00164 PFNGLBLEND FUNC SEPARATEPROC glBlendFuncSeparate;
00165 PFNGLBLEND PARAMETER INVPROC glBlendParameteriNV;
00166 PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00167 PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00168 PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00169 PFNGLBUFFERDATAPROC glBufferData;
00170 PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00171 PFNGLBUFFERSTORAGEPROC glBufferStorage;
00172 PFNGLBUFFERSUBDATAPROC glBufferSubData;
00173 PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00174 PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00175 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC glCheckNamedFramebufferStatusEXT;
00176 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;
00177 PFNGLCLAMP COLORPROC glClampColor;
00178 PFNGLCLEARBUFFERDATAPROC glClearBufferData;
00179 PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00180 PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00181 PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00182 PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00183 PFNGLCLEARBUFFERUIVPROC glClearBufferuiv;
00184 PFNGLCLEAR COLORPROC glClearColor;
00185 PFNGLCLEAR DEPTH FPROC glClearDepthf;
00186 PFNGLCLEAR DEPTH PROC glClearDepth;
00187 PFNGLCLEARNAMEDBUFFERDATAEXTPROC glClearNamedBufferDataEXT;
00188 PFNGLCLEARNAMEDBUFFERDATAPROC glClearNamedBufferData;
00189 PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00190 PFNGLCLEARNAMEDBUFFERSUBDATAPROC glClearNamedBufferSubData;
00191 PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00192 PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00193 PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00194 PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00195 PFNGLCLEARPROC glClear;
00196 PFNGLCLEARSTENCILPROC glClearStencil;
00197 PFNGLCLEARTEXTIMAGEPROC glClearTexImage;
00198 PFNGLCLEARTEXTSUBIMAGEPROC glClearTexSubImage;
00199 PFNGLCLIENTATTRIBDEFAULTTEXTPROC glClientAttribDefaultEXT;
00200 PFNGLCLIENTWAITSYNCPROC glClientWaitSync;
```

```
00201 PFNGLCLIPCONTROLPROC glClipControl;
00202 PFNGLCOLORFORMATNVPROC glColorFormatNV;
00203 PFNGLCOLORMASKIPROC glColorMaski;
00204 PFNGLCOLORMASKPROC glColorMask;
00205 PFNGLCOMMANDLISTSEGMENTSNVPROC glCommandListSegmentsNV;
00206 PFNGLCOMPILECOMMANDLISTNVPROC glCompileCommandListNV;
00207 PFNGLCOMPILESHADERINCLUDEARBPROC glCompileShaderIncludeARB;
00208 PFNGLCOMPILESHADERPROC glCompileShader;
00209 PFNGLCOMPRESSEDMULTITEXIMAGE1DXTPROC glCompressedMultiTexImage1DXT;
00210 PFNGLCOMPRESSEDMULTITEXIMAGE2DXTPROC glCompressedMultiTexImage2DXT;
00211 PFNGLCOMPRESSEDMULTITEXIMAGE3DXTPROC glCompressedMultiTexImage3DXT;
00212 PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DXTPROC glCompressedMultiTexSubImage1DXT;
00213 PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DXTPROC glCompressedMultiTexSubImage2DXT;
00214 PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DXTPROC glCompressedMultiTexSubImage3DXT;
00215 PFNGLCOMPRESSEDTEXIMAGE1DPROC glCompressedTexImage1D;
00216 PFNGLCOMPRESSEDTEXIMAGE2DPROC glCompressedTexImage2D;
00217 PFNGLCOMPRESSEDTEXIMAGE3DPROC glCompressedTexImage3D;
00218 PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC glCompressedTexSubImage1D;
00219 PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC glCompressedTexSubImage2D;
00220 PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC glCompressedTexSubImage3D;
00221 PFNGLCOMPRESSEDTEXTUREIMAGE1DXTPROC glCompressedTextureImage1DXT;
00222 PFNGLCOMPRESSEDTEXTUREIMAGE2DXTPROC glCompressedTextureImage2DXT;
00223 PFNGLCOMPRESSEDTEXTUREIMAGE3DXTPROC glCompressedTextureImage3DXT;
00224 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DXTPROC glCompressedTextureSubImage1DXT;
00225 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00226 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DXTPROC glCompressedTextureSubImage2DXT;
00227 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00228 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DXTPROC glCompressedTextureSubImage3DXT;
00229 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00230 PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00231 PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00232 PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00233 PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00234 PFNGLCOPYMULTITEXIMAGE1DXTPROC glCopyMultiTexImage1DXT;
00235 PFNGLCOPYMULTITEXIMAGE2DXTPROC glCopyMultiTexImage2DXT;
00236 PFNGLCOPYMULTITEXSUBIMAGE1DXTPROC glCopyMultiTexSubImage1DXT;
00237 PFNGLCOPYMULTITEXSUBIMAGE2DXTPROC glCopyMultiTexSubImage2DXT;
00238 PFNGLCOPYMULTITEXSUBIMAGE3DXTPROC glCopyMultiTexSubImage3DXT;
00239 PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00240 PFNGLCOPYPATHNVPROC glCopyPathNV;
00241 PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00242 PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00243 PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00244 PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00245 PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00246 PFNGLCOPYTEXTUREIMAGE1DXTPROC glCopyTextureImage1DXT;
00247 PFNGLCOPYTEXTUREIMAGE2DXTPROC glCopyTextureImage2DXT;
00248 PFNGLCOPYTEXTURESUBIMAGE1DXTPROC glCopyTextureSubImage1DXT;
00249 PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00250 PFNGLCOPYTEXTURESUBIMAGE2DXTPROC glCopyTextureSubImage2DXT;
00251 PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00252 PFNGLCOPYTEXTURESUBIMAGE3DXTPROC glCopyTextureSubImage3DXT;
00253 PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00254 PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00255 PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00256 PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00257 PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00258 PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00259 PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00260 PFNGLCREATEBUFFERSPROC glCreateBuffers;
00261 PFNGLCREATECOMMANDLISTSNVPROC glCreateCommandListsNV;
00262 PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00263 PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00264 PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00265 PFNGLCREATEPROGRAMPROC glCreateProgram;
00266 PFNGLCREATEQUERIESPROC glCreateQueries;
00267 PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00268 PFNGLCREATESAMPLERSPROC glCreateSamplers;
00269 PFNGLCREATESHADERPROC glCreateShader;
00270 PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00271 PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00272 PFNGLCREATESTATESNVPROC glCreateStatesNV;
00273 PFNGLCREATESYNCFROMCLEVENTARBPROC glCreateSyncFromCLeventARB;
00274 PFNGLCREATETEXTURESPROC glCreateTextures;
00275 PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00276 PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00277 PFNGLCULLFACEPROC glCullFace;
00278 PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00279 PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00280 PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00281 PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00282 PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00283 PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00284 PFNGLDELETETEXTURESPROC glDeleteTextures;
00285 PFNGLDELETECOMMANDLISTSNVPROC glDeleteCommandListsNV;
00286 PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00287 PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
```



```
00288 PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00289 PFNGLDELETEPERFMONITORSSAMDPROC glDeletePerfMonitorsAMD;
00290 PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00291 PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00292 PFNGLDELETEPROGRAMPROC glDeleteProgram;
00293 PFNGLDELETEQUERIESPROC glDeleteQueries;
00294 PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00295 PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00296 PFNGLDELETESHADERPROC glDeleteShader;
00297 PFNGLDELETETEXTURESNVPROC glDeleteTexturesNV;
00298 PFNGLDELETESYNCPROC glDeleteSync;
00299 PFNGLDELETETEXTURESPROC glDeleteTextures;
00300 PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00301 PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00302 PFNGLDEPTHFUNCPROC glDepthFunc;
00303 PFNGLDEPTHMASKPROC glDepthMask;
00304 PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00305 PFNGLDEPTHRANGEPROC glDepthRange;
00306 PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00307 PFNGLDEPTHRANGEPROC glDepthRange;
00308 PFNGLDETACHSHADERPROC glDetachShader;
00309 PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00310 PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00311 PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00312 PFNGLDISABLEIPROC glDisablei;
00313 PFNGLDISABLEPROC glDisable;
00314 PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00315 PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00316 PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00317 PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00318 PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00319 PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00320 PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00321 PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00322 PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00323 PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00324 PFNGLDRAWARRAYSINSTANCEDEXTPROC glDrawArraysInstancedEXT;
00325 PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00326 PFNGLDRAWARRAYSPROC glDrawArrays;
00327 PFNGLDRAWBUFFERPROC glDrawBuffer;
00328 PFNGLDRAWBUFFERSPROC glDrawBuffers;
00329 PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00330 PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00331 PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00332 PFNGLDRAWCOMMANDSSTATESNVPROC glDrawCommandsStatesNV;
00333 PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00334 PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00335 PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00336 PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00337 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC glDrawElementsInstancedBaseVertexBaseInstance;
00338 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00339 PFNGLDRAWELEMENTSINSTANCEDEXTPROC glDrawElementsInstancedEXT;
00340 PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00341 PFNGLDRAWELEMENTSPROC glDrawElements;
00342 PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00343 PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00344 PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00345 PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00346 PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00347 PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00348 PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00349 PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00350 PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00351 PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00352 PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00353 PFNGLENABLEIPROC glEnablei;
00354 PFNGLENABLEPROC glEnable;
00355 PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00356 PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00357 PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00358 PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
00359 PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00360 PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00361 PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00362 PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00363 PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00364 PFNGLENDQUERYPROC glEndQuery;
00365 PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00366 PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00367 PFNGLFENCESYNCPROC glFenceSync;
00368 PFNGLFINISHPROC glFinish;
00369 PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00370 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00371 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00372 PFNGLFLUSHPROC glFlush;
00373 PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00374 PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
```

```
00375 PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00376 PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00377 PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00378 PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00379 PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00380 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00381 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00382 PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00383 PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00384 PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00385 PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00386 PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00387 PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00388 PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00389 PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00390 PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00391 PFNGLFRONTFACEPROC glFrontFace;
00392 PFNGLGENBUFFERSPROC glGenBuffers;
00393 PFNGLGENERATEMIPMAPPROC glGenerateMipmap;
00394 PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenerateMultiTexMipmapEXT;
00395 PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenerateTextureMipmapEXT;
00396 PFNGLGENERATETEXTUREMIPMAPPROC glGenerateTextureMipmap;
00397 PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00398 PFNGLGENPATHSNVPROC glGenPathsNV;
00399 PFNGLGENPERFMONITORSAMDPROC glGenPerfMonitorsAMD;
00400 PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00401 PFNGLGENQUERIESPROC glGenQueries;
00402 PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00403 PFNGLGENSAMPLERSPROC glGenSamplers;
00404 PFNGLGENTEXTURESPROC glGenTextures;
00405 PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00406 PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00407 PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00408 PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00409 PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00410 PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00411 PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00412 PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00413 PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00414 PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00415 PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00416 PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00417 PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00418 PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00419 PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00420 PFNGLGETBOOLEANIVPROC glGetBooleani_v;
00421 PFNGLGETBOOLEANVPROC glGetBooleanv;
00422 PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00423 PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00424 PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00425 PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00426 PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00427 PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00428 PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00429 PFNGLGETCOMPRESSEDTEXIMAGEPROC glGetCompressedTexImage;
00430 PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00431 PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00432 PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00433 PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00434 PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00435 PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00436 PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00437 PFNGLGETDOUBLEIVEXTPROC glGetDoublei_vEXT;
00438 PFNGLGETDOUBLEIVPROC glGetDoublei_v;
00439 PFNGLGETDOUBLEVPROC glGetDoublev;
00440 PFNGLGETERRORPROC glGetError;
00441 PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00442 PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00443 PFNGLGETFLOATIVEXTPROC glGetFloati_vEXT;
00444 PFNGLGETFLOATIVPROC glGetFloati_v;
00445 PFNGLGETFLOATVPROC glGetFloatv;
00446 PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00447 PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00448 PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00449 PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00450 PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00451 PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00452 PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00453 PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00454 PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00455 PFNGLGETINTEGER64IVPROC glGetInteger64i_v;
00456 PFNGLGETINTEGER64VPROC glGetInteger64v;
00457 PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00458 PFNGLGETINTEGERIVPROC glGetIntegeri_v;
00459 PFNGLGETINTEGERUI64IVNVPROC glGetIntegerui64i_vNV;
00460 PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00461 PFNGLGETINTEGERVPROC glGetIntegerv;
```

```
00462 PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00463 PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00464 PFNGLGETINTERNALFORMATSAMPLEIVNVPROC glGetInternalformatSampleivNV;
00465 PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00466 PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00467 PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00468 PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00469 PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00470 PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00471 PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00472 PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00473 PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00474 PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00475 PFNGLGETMULTITEXPARAMETERIIVEXTPROC glGetMultiTexParameterIivEXT;
00476 PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameterIuivEXT;
00477 PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00478 PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00479 PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00480 PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00481 PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00482 PFNGLGETNAMEDBUFFERPOINTERVEXTPROC glGetNamedBufferPointervEXT;
00483 PFNGLGETNAMEDBUFFERPOINTERVPROC glGetNamedBufferPointerv;
00484 PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00485 PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00486 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC glGetNamedFramebufferAttachmentParameterivEXT;
00487 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00488 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00489 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00490 PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00491 PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00492 PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00493 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameterIivEXT;
00494 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameterIuivEXT;
00495 PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00496 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00497 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00498 PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00499 PFNGLGETNAMEDSTRINGIVARBPROC glGetNamedStringivARB;
00500 PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00501 PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00502 PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00503 PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00504 PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00505 PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvARB;
00506 PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00507 PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvARB;
00508 PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00509 PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64vARB;
00510 PFNGLGETNUNIFORMIVARBPROC glGetnUniformivARB;
00511 PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00512 PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64vARB;
00513 PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivARB;
00514 PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00515 PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00516 PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00517 PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00518 PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00519 PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00520 PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00521 PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00522 PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00523 PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00524 PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00525 PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00526 PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00527 PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00528 PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00529 PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00530 PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00531 PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00532 PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00533 PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00534 PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00535 PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00536 PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00537 PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00538 PFNGLGETPOINTERIIVEXTPROC glGetPointerIivEXT;
00539 PFNGLGETPOINTERVPROC glGetPointerv;
00540 PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00541 PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00542 PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00543 PFNGLGETPROGRAMIVPROC glGetProgramiv;
00544 PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00545 PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00546 PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00547 PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00548 PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
```

```
00549 PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00550 PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00551 PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00552 PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00553 PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00554 PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00555 PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00556 PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00557 PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00558 PFNGLGETQUERYIVPROC glGetQueryiv;
00559 PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00560 PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00561 PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00562 PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00563 PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00564 PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00565 PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00566 PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00567 PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00568 PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00569 PFNGLGETSHADERIVPROC glGetShaderiv;
00570 PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00571 PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00572 PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00573 PFNGLGETSTRINGIPROC glGetStringi;
00574 PFNGLGETSTRINGPROC glGetString;
00575 PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00576 PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00577 PFNGLGETSYNCPROC glGetSynciv;
00578 PFNGLGETTEXIMAGEPROC glGetTexImage;
00579 PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00580 PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00581 PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00582 PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00583 PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00584 PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00585 PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00586 PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00587 PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00588 PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00589 PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00590 PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00591 PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00592 PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00593 PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00594 PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00595 PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00596 PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00597 PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00598 PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00599 PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00600 PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00601 PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00602 PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00603 PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00604 PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64v;
00605 PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00606 PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00607 PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00608 PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00609 PFNGLGETUNIFORMDVPROC glGetUniformdv;
00610 PFNGLGETUNIFORMFVPROC glGetUniformfv;
00611 PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00612 PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00613 PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00614 PFNGLGETUNIFORMIVPROC glGetUniformiv;
00615 PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00616 PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00617 PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00618 PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00619 PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00620 PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00621 PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00622 PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00623 PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayInteger_vEXT;
00624 PFNGLGETVERTEXARRAYIVPROC glGetVertexArrayiv;
00625 PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00626 PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointer_vEXT;
00627 PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00628 PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00629 PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribIiv;
00630 PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00631 PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00632 PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribLdv;
00633 PFNGLGETVERTEXATTRIBLI64VNVPROC glGetVertexAttribLi64vNV;
00634 PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00635 PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
```

```
00636 PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointerv;
00637 PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00638 PFNGLHINTPROC glHint;
00639 PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00640 PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00641 PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00642 PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00643 PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00644 PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00645 PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00646 PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
00647 PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
00648 PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00649 PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00650 PFNGLISBUFFERPROC glIsBuffer;
00651 PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00652 PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00653 PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00654 PFNGLISENABLEDIPROC glIsEnabledi;
00655 PFNGLISENABLEDPPROC glIsEnabled;
00656 PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00657 PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00658 PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00659 PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00660 PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00661 PFNGLISPATHTNVPROC glIsPathNV;
00662 PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00663 PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
00664 PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00665 PFNGLISPROGRAMPROC glIsProgram;
00666 PFNGLISQUERYPROC glIsQuery;
00667 PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00668 PFNGLISSAMPLERPROC glIsSampler;
00669 PFNGLISSHADERPROC glIsShader;
00670 PFNGLISSTATENVPROC glIsStateNV;
00671 PFNGLISSYNCPROC glIsSync;
00672 PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00673 PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00674 PFNGLISTEXTUREPROC glIsTexture;
00675 PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00676 PFNGLISVERTEXARRAYPROC glIsVertexArray;
00677 PFNGLLABELOBJECTEXTPROC glLabelObjectEXT;
00678 PFNGLLINEWIDTHPROC glLineWidth;
00679 PFNGLLINKPROGRAMPROC glLinkProgram;
00680 PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC glListDrawCommandsStatesClientNV;
00681 PFNGLLOGICOPPROC glLogicOp;
00682 PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00683 PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00684 PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00685 PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;
00686 PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00687 PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00688 PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00689 PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00690 PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00691 PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00692 PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00693 PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00694 PFNGLMAPBUFFERPROC glMapBuffer;
00695 PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00696 PFNGLMAPNAMEDBUFFEREXTPROC glMapNamedBufferEXT;
00697 PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00698 PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
00699 PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00700 PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
00701 PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00702 PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00703 PFNGLMATRIXLOADDEXTPROC glMatrixLoadEXT;
00704 PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00705 PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00706 PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00707 PFNGLMATRIXLOADTRANSPOSEDEXTPROC glMatrixLoadTransposedEXT;
00708 PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00709 PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00710 PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00711 PFNGLMATRIXMULTDEXTPROC glMatrixMultEXT;
00712 PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00713 PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00714 PFNGLMATRIXMULTTRANSPOSEDEXTPROC glMatrixMultTransposedEXT;
00715 PFNGLMATRIXMULTTRANSPOSEFEXTPROC glMatrixMultTransposefEXT;
00716 PFNGLMATRIXORTHOEXTPROC glMatrixOrthoEXT;
00717 PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00718 PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00719 PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00720 PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00721 PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00722 PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
```

```
00723 PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00724 PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatfEXT;
00725 PFNGLMAXSHADERCOMPILETHREADSARBPROC glMaxShaderCompilerThreadsARB;
00726 PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00727 PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00728 PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00729 PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00730 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00731 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00732 PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00733 PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00734 PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00735 PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00736 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00737 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00738 PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00739 PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00740 PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00741 PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00742 PFNGLMULTITEXCOORDPOINTEREXTPROC glMultiTexCoordPointerEXT;
00743 PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00744 PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00745 PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00746 PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00747 PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00748 PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00749 PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00750 PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00751 PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00752 PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00753 PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DEXT;
00754 PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00755 PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00756 PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00757 PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00758 PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00759 PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameterIivEXT;
00760 PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameterIuivEXT;
00761 PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00762 PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00763 PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1DEXT;
00764 PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00765 PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00766 PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00767 PFNGLNAMEDBUFFERDATAPROC glNamedBufferData;
00768 PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00769 PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00770 PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00771 PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00772 PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00773 PFNGLNAMEDBUFFERSUBDATAPROC glNamedBufferSubData;
00774 PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00775 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00776 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00777 PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00778 PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00779 PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00780 PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00781 PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00782 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00783 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00784 PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00785 PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
00786 PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00787 PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00788 PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00789 PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00790 PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00791 PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00792 PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00793 PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
00794 PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00795 PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00796 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00797 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00798 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00799 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00800 PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00801 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC glNamedProgramLocalParametersI4ivEXT;
00802 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00803 PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00804 PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00805 PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLECOVERAGEEXTPROC
    glNamedRenderbufferStorageMultisampleCoverageEXT;
00806 PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00807 PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00808 PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
```

```

00809 PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00810 PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00811 PFNGLOBJEC TLABELPROC glObjectLabel;
00812 PFNGLOBJEC TPTRLABELPROC glObjectPtrLabel;
00813 PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00814 PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00815 PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00816 PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00817 PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00818 PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00819 PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00820 PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00821 PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00822 PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00823 PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00824 PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00825 PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00826 PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00827 PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00828 PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00829 PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00830 PFNGLPATHSTRINGNVPROC glPathStringNV;
00831 PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00832 PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00833 PFNGLPAUSESETTRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00834 PFNGLPIXELSTOREFPROC glPixelStoref;
00835 PFNGLPIXELSTOREIPROC glPixelStorei;
00836 PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00837 PFNGLPOINTPARAMETERFPROC glPointParameterf;
00838 PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00839 PFNGLPOINTPARAMETERIPROC glPointParameteri;
00840 PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00841 PFNGLPOINTSIZEPROC glPointSize;
00842 PFNGLPOLYGONMODEPROC glPolygonMode;
00843 PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00844 PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00845 PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00846 PFNGLPOPGROUPMARKEREXTPROC glPopGroupMarkerEXT;
00847 PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00848 PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00849 PFNGLPROGRAMBINARYPROC glProgramBinary;
00850 PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00851 PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00852 PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00853 PFNGLPROGRAMUNIFORM1DTEXTPROC glProgramUniform1dEXT;
00854 PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00855 PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00856 PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00857 PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00858 PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00859 PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00860 PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00861 PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00862 PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00863 PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00864 PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00865 PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00866 PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00867 PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00868 PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00869 PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00870 PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00871 PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00872 PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00873 PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00874 PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00875 PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00876 PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00877 PFNGLPROGRAMUNIFORM2DTEXTPROC glProgramUniform2dEXT;
00878 PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00879 PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
00880 PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00881 PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00882 PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00883 PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00884 PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00885 PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00886 PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00887 PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00888 PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00889 PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00890 PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00891 PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00892 PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00893 PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00894 PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00895 PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;

```

```
00896 PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00897 PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00898 PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00899 PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00900 PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00901 PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00902 PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00903 PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00904 PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00905 PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00906 PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00907 PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
00908 PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00909 PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00910 PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00911 PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00912 PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00913 PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00914 PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00915 PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00916 PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00917 PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00918 PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00919 PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00920 PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00921 PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00922 PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00923 PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00924 PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00925 PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00926 PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00927 PFNGLPROGRAMUNIFORM4DVEXTPROC glProgramUniform4dvEXT;
00928 PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00929 PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00930 PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00931 PFNGLPROGRAMUNIFORM4FVEXTPROC glProgramUniform4fvEXT;
00932 PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00933 PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00934 PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00935 PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00936 PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00937 PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00938 PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00939 PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00940 PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00941 PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00942 PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00943 PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00944 PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00945 PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00946 PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00947 PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00948 PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00949 PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00950 PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00951 PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00952 PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00953 PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00954 PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00955 PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC glProgramUniformMatrix2fvEXT;
00956 PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00957 PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00958 PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00959 PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC glProgramUniformMatrix2x3fvEXT;
00960 PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00961 PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC glProgramUniformMatrix2x4dvEXT;
00962 PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00963 PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC glProgramUniformMatrix2x4fvEXT;
00964 PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00965 PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC glProgramUniformMatrix3dvEXT;
00966 PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00967 PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC glProgramUniformMatrix3fvEXT;
00968 PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00969 PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC glProgramUniformMatrix3x2dvEXT;
00970 PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00971 PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC glProgramUniformMatrix3x2fvEXT;
00972 PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00973 PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC glProgramUniformMatrix3x4dvEXT;
00974 PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00975 PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC glProgramUniformMatrix3x4fvEXT;
00976 PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00977 PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC glProgramUniformMatrix4dvEXT;
00978 PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00979 PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC glProgramUniformMatrix4fvEXT;
00980 PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00981 PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC glProgramUniformMatrix4x2dvEXT;
00982 PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
```



```
00983 PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC glProgramUniformMatrix4x2fvEXT;
00984 PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00985 PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC glProgramUniformMatrix4x3dvEXT;
00986 PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00987 PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC glProgramUniformMatrix4x3fvEXT;
00988 PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00989 PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00990 PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00991 PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00992 PFNGLPUSHCLIENTATTRIBDEFAULTEXTPROC glPushClientAttribDefaultEXT;
00993 PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00994 PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
00995 PFNGLQUERYCOUNTERPROC glQueryCounter;
00996 PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00997 PFNGLREADBUFFERPROC glReadBuffer;
00998 PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
00999 PFNGLREADNPIXELSPROC glReadnPixels;
01000 PFNGLREADPIXELSPROC glReadPixels;
01001 PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
01002 PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
01003 PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
01004 PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
01005 PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
01006 PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
01007 PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
01008 PFNGLSAMPLEMASKIPROC glSampleMaski;
01009 PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
01010 PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
01011 PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
01012 PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01013 PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01014 PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01015 PFNGLSCISSORARRAYVPROC glScissorArrayv;
01016 PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01017 PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01018 PFNGLSCISSORPROC glScissor;
01019 PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01020 PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01021 PFNGLSHADERBINARYPROC glShaderBinary;
01022 PFNGLSHADERSOURCEPROC glShaderSource;
01023 PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01024 PFNGLSIGNALVKFENCENVPROC glSignalVkFenceNV;
01025 PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01026 PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01027 PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01028 PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01029 PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01030 PFNGLSTENCILFUNCPROC glStencilFunc;
01031 PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01032 PFNGLSTENCILMASKPROC glStencilMask;
01033 PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01034 PFNGLSTENCILOPPROC glStencilOp;
01035 PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01036 PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01037 PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01038 PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01039 PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01040 PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01041 PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01042 PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01043 PFNGLTEXBUFFERARBPROC glTexBufferARB;
01044 PFNGLTEXBUFFERPROC glTexBuffer;
01045 PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01046 PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01047 PFNGLTEXIMAGE1DPROC glTexImage1D;
01048 PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01049 PFNGLTEXIMAGE2DPROC glTexImage2D;
01050 PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01051 PFNGLTEXIMAGE3DPROC glTexImage3D;
01052 PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01053 PFNGLTEXPARAMETERFPROC glTexParameterf;
01054 PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01055 PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01056 PFNGLTEXPARAMETERIPROC glTexParameteri;
01057 PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01058 PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01059 PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01060 PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01061 PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01062 PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01063 PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01064 PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01065 PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01066 PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01067 PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01068 PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01069 PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
```

```
01070 PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01071 PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01072 PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01073 PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01074 PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01075 PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01076 PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01077 PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01078 PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfvEXT;
01079 PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01080 PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01081 PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01082 PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameteriivEXT;
01083 PFNGLTEXTUREPARAMETERIIVPROC glTextureParameteriiv;
01084 PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01085 PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameteriuivEXT;
01086 PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameteriuiv;
01087 PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01088 PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01089 PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01090 PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DEXT;
01091 PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01092 PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DEXT;
01093 PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01094 PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01095 PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01096 PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DEXT;
01097 PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01098 PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01099 PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01100 PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DEXT;
01101 PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01102 PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DEXT;
01103 PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01104 PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DEXT;
01105 PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01106 PFNGLTEXTUREVIEWPROC glTextureView;
01107 PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01108 PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01109 PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01110 PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01111 PFNGLUNIFORM1DPROC glUniform1d;
01112 PFNGLUNIFORM1DVPROC glUniform1dv;
01113 PFNGLUNIFORM1FPROC glUniform1f;
01114 PFNGLUNIFORM1FVPROC glUniform1fv;
01115 PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01116 PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01117 PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01118 PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01119 PFNGLUNIFORM1IPROC glUniform1i;
01120 PFNGLUNIFORM1IVPROC glUniform1iv;
01121 PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01122 PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01123 PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01124 PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01125 PFNGLUNIFORM1UIPROC glUniform1ui;
01126 PFNGLUNIFORM1UIVPROC glUniform1uiv;
01127 PFNGLUNIFORM2DPROC glUniform2d;
01128 PFNGLUNIFORM2DVPROC glUniform2dv;
01129 PFNGLUNIFORM2FPROC glUniform2f;
01130 PFNGLUNIFORM2FVPROC glUniform2fv;
01131 PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01132 PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01133 PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01134 PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01135 PFNGLUNIFORM2IPROC glUniform2i;
01136 PFNGLUNIFORM2IVPROC glUniform2iv;
01137 PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01138 PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01139 PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01140 PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01141 PFNGLUNIFORM2UIPROC glUniform2ui;
01142 PFNGLUNIFORM2UIVPROC glUniform2uiv;
01143 PFNGLUNIFORM3DPROC glUniform3d;
01144 PFNGLUNIFORM3DVPROC glUniform3dv;
01145 PFNGLUNIFORM3FPROC glUniform3f;
01146 PFNGLUNIFORM3FVPROC glUniform3fv;
01147 PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01148 PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01149 PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01150 PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01151 PFNGLUNIFORM3IPROC glUniform3i;
01152 PFNGLUNIFORM3IVPROC glUniform3iv;
01153 PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01154 PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01155 PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01156 PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
```

```
01157 PFNGLUNIFORM3UIPROC glUniform3ui;
01158 PFNGLUNIFORM3UIVPROC glUniform3uiv;
01159 PFNGLUNIFORM4DPROC glUniform4d;
01160 PFNGLUNIFORM4DVPROC glUniform4dv;
01161 PFNGLUNIFORM4FPROC glUniform4f;
01162 PFNGLUNIFORM4FVPROC glUniform4fv;
01163 PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01164 PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01165 PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01166 PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01167 PFNGLUNIFORM4IPROC glUniform4i;
01168 PFNGLUNIFORM4IVPROC glUniform4iv;
01169 PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01170 PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01171 PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01172 PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01173 PFNGLUNIFORM4UIPROC glUniform4ui;
01174 PFNGLUNIFORM4UIVPROC glUniform4uiv;
01175 PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01176 PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01177 PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01178 PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01179 PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01180 PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01181 PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01182 PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01183 PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01184 PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01185 PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01186 PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01187 PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01188 PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01189 PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01190 PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01191 PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01192 PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01193 PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01194 PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01195 PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01196 PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01197 PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01198 PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01199 PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01200 PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01201 PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01202 PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01203 PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01204 PFNGLUSEPROGRAMPROC glUseProgram;
01205 PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01206 PFNGLUSESHADERPROGRAMEXTPROC glUseProgramProgramEXT;
01207 PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01208 PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01209 PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01210 PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01211 PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01212 PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01213 PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01214 PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01215 PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01216 PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01217 PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01218 PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01219 PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01220 PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01221 PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01222 PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01223 PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01224 PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01225 PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01226 PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01227 PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01228 PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01229 PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC glVertexArrayVertexAttribLFormatEXT;
01230 PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01231 PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01232 PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01233 PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01234 PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01235 PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01236 PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01237 PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01238 PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01239 PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01240 PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01241 PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01242 PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01243 PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
```

```
01244 PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01245 PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01246 PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01247 PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01248 PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01249 PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01250 PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01251 PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01252 PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01253 PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01254 PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01255 PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01256 PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01257 PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01258 PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01259 PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01260 PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01261 PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01262 PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01263 PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01264 PFNGLVERTEXATTRIB4NUBVPROC glVertexAttrib4Nubv;
01265 PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01266 PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01267 PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01268 PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01269 PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01270 PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01271 PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01272 PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01273 PFNGLVERTEXATTRIBBDIVISORARBPROC glVertexAttribDivisorARB;
01274 PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01275 PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01276 PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01277 PFNGLVERTEXATTRIBI1PROC glVertexAttribI1i;
01278 PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01279 PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01280 PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01281 PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01282 PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01283 PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01284 PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01285 PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01286 PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01287 PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01288 PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01289 PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01290 PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01291 PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01292 PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01293 PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01294 PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01295 PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01296 PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01297 PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01298 PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01299 PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01300 PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01301 PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01302 PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01303 PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01304 PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01305 PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01306 PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01307 PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01308 PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01309 PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01310 PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01311 PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01312 PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01313 PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01314 PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01315 PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01316 PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01317 PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01318 PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01319 PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01320 PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01321 PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01322 PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01323 PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01324 PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01325 PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01326 PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01327 PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01328 PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01329 PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01330 PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
```

```

01331 PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01332 PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01333 PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01334 PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01335 PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01336 PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01337 PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01338 PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01339 PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01340 PFNGLVIEWPORTARRAYVPROC glViewportArrayv;
01341 PFNGLVIEWPORTINDEXEDFPROC glViewportIndexdf;
01342 PFNGLVIEWPORTINDEXEDFVPROC glViewportIndexdfv;
01343 PFNGLVIEWPORTPOSITIONWSCALENVPROC glViewportPositionWScaleNV;
01344 PFNGLVIEWPORTPROC glViewport;
01345 PFNGLVIEWPORTSWIZZLENVPROC glViewportSwizzleNV;
01346 PFNGLWAITSYNCPROC glWaitSync;
01347 PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01348 PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01349 PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01350 #endif
01351
01353
01355 GLint gg::ggBufferAlignment(0);
01356
01357 //
01358 // ゲームグラフィックス特論の都合にもとづく初期化
01359 //
01360 void gg::ggInit()
01361 {
01362     //すでにこの関数が実行されていたら以降の処理を行わない
01363     if (ggBufferAlignment) return;
01364
01365     // macOS 以外で OpenGL 3.2 以降の API を取得する
01366     #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
01367         glActiveProgramEXT = PFNGLACTIVEPROGRAMEXTPROC(glwGetProcAddress("glActiveProgramEXT"));
01368         glActiveShaderProgram = PFNGLACTIVESHADERPROGRAMPROC(glwGetProcAddress("glActiveShaderProgram"));
01369         glActiveTexture = PFNGLACTIVETEXTUREPROC(glwGetProcAddress("glActiveTexture"));
01370         glApplyFramebufferAttachmentCMAAINTEL =
PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC(glwGetProcAddress("glApplyFramebufferAttachmentCMAAINTEL"));
01371         glAttachShader = PFNGLATTACHSHADERPROC(glwGetProcAddress("glAttachShader"));
01372         glBeginConditionalRender =
PFNGLBEGINCONDITIONALRENDERPROC(glwGetProcAddress("glBeginConditionalRender"));
01373         glBeginConditionalRenderNV =
PFNGLBEGINCONDITIONALRENDERNVPROC(glwGetProcAddress("glBeginConditionalRenderNV"));
01374         glBeginPerfMonitorAMD = PFNGLBEGINPERFMONITORAMDPROC(glwGetProcAddress("glBeginPerfMonitorAMD"));
01375         glBeginPerfQueryINTEL = PFNGLBEGINPERFQUERYINTELPROC(glwGetProcAddress("glBeginPerfQueryINTEL"));
01376         glBeginQuery = PFNGLBEGINQUERYPROC(glwGetProcAddress("glBeginQuery"));
01377         glBeginQueryIndexed = PFNGLBEGINQUERYINDEXEDPROC(glwGetProcAddress("glBeginQueryIndexed"));
01378         glBeginTransformFeedback =
PFNGLBEGINTRANSFORMFEEDBACKPROC(glwGetProcAddress("glBeginTransformFeedback"));
01379         glBindAttribLocation = PFNGLBINDATTRIBLOCATIONPROC(glwGetProcAddress("glBindAttribLocation"));
01380         glBindBuffer = PFNGLBINDBUFFERPROC(glwGetProcAddress("glBindBuffer"));
01381         glBindBufferBase = PFNGLBINDBUFFERBASEPROC(glwGetProcAddress("glBindBufferBase"));
01382         glBindBufferRange = PFNGLBINDBUFFERRANGEPROC(glwGetProcAddress("glBindBufferRange"));
01383         glBindBuffersBase = PFNGLBINDBUFFERSBASEPROC(glwGetProcAddress("glBindBuffersBase"));
01384         glBindBuffersRange = PFNGLBINDBUFFERSRANGEPROC(glwGetProcAddress("glBindBuffersRange"));
01385         glBindFragDataLocation =
PFNGLBINDFRAGDATALOCATIONPROC(glwGetProcAddress("glBindFragDataLocation"));
01386         glBindFragDataLocationIndexed =
PFNGLBINDFRAGDATALOCATIONINDEXEDPROC(glwGetProcAddress("glBindFragDataLocationIndexed"));
01387         glBindFramebuffer = PFNGLBINDFRAMEBUFFERPROC(glwGetProcAddress("glBindFramebuffer"));
01388         glBindImageTexture = PFNGLBINDIMAGETEXTUREPROC(glwGetProcAddress("glBindImageTexture"));
01389         glBindImageTextures = PFNGLBINDIMAGETEXTURESPROC(glwGetProcAddress("glBindImageTextures"));
01390         glBindMultiTextureEXT = PFNGLBINDMULTITEXTUREEXTPROC(glwGetProcAddress("glBindMultiTextureEXT"));
01391         glBindProgramPipeline = PFNGLBINDPROGRAMPIPELINEPROC(glwGetProcAddress("glBindProgramPipeline"));
01392         glBindRenderbuffer = PFNGLBINDRENDERBUFFERPROC(glwGetProcAddress("glBindRenderbuffer"));
01393         glBindSampler = PFNGLBINDSAMPLERPROC(glwGetProcAddress("glBindSampler"));
01394         glBindSamplers = PFNGLBINDSAMPLERSPROC(glwGetProcAddress("glBindSamplers"));
01395         glBindTexture = PFNGLBINDTEXTUREPROC(glwGetProcAddress("glBindTexture"));
01396         glBindTextureUnit = PFNGLBINDTEXTUREUNITPROC(glwGetProcAddress("glBindTextureUnit"));
01397         glBindTextures = PFNGLBINDTEXTURESPROC(glwGetProcAddress("glBindTextures"));
01398         glBindTransformFeedback =
PFNGLBINDTRANSFORMFEEDBACKPROC(glwGetProcAddress("glBindTransformFeedback"));
01399         glBindVertexArray = PFNGLBINDVERTEXARRAYPROC(glwGetProcAddress("glBindVertexArray"));
01400         glBindVertexBuffer = PFNGLBINDVERTEXBUFFERPROC(glwGetProcAddress("glBindVertexBuffer"));
01401         glBindVertexBuffers = PFNGLBINDVERTEXBUFFERSPROC(glwGetProcAddress("glBindVertexBuffers"));
01402         glBlendBarrierKHR = PFNGLBLENDBARRIERKHRPROC(glwGetProcAddress("glBlendBarrierKHR"));
01403         glBlendBarrierNV = PFNGLBLENDBARRIERNVPROC(glwGetProcAddress("glBlendBarrierNV"));
01404         glBlendColor = PFNGLBLENDCOLORPROC(glwGetProcAddress("glBlendColor"));
01405         glBlendEquation = PFNGLBLEND EQUATIONPROC(glwGetProcAddress("glBlendEquation"));
01406         glBlendEquationSeparate =
PFNGLBLEND EQUATIONSEPARATEPROC(glwGetProcAddress("glBlendEquationSeparate"));
01407         glBlendEquationSeparatei =
PFNGLBLEND EQUATIONSEPARATEIPROC(glwGetProcAddress("glBlendEquationSeparatei"));
01408         glBlendEquationSeparateiARB =
PFNGLBLEND EQUATIONSEPARATEIARBPROC(glwGetProcAddress("glBlendEquationSeparateiARB"));
01409         glBlendEquationi = PFNGLBLEND EQUATIONIPROC(glwGetProcAddress("glBlendEquationi"));

```

```

01410     glBlendEquationiARB = PFNGLBLENDSEPARATEIARBPROC (glfwGetProcAddress("glBlendEquationiARB"));
01411     glBlendFunc = PFNGLBLENDSEPARATEPROC (glfwGetProcAddress("glBlendFunc"));
01412     glBlendFuncSeparate = PFNGLBLENDSEPARATEPROC (glfwGetProcAddress("glBlendFuncSeparate"));
01413     glBlendFuncSeparatei = PFNGLBLENDSEPARATEIPROC (glfwGetProcAddress("glBlendFuncSeparatei"));
01414     glBlendFuncSeparateiARB =
PFNGLBLENDSEPARATEIARBPROC (glfwGetProcAddress("glBlendFuncSeparateiARB"));
01415     glBlendFunci = PFNGLBLENDSEPARATEIARBPROC (glfwGetProcAddress("glBlendFunci"));
01416     glBlendFunciARB = PFNGLBLENDSEPARATEIARBPROC (glfwGetProcAddress("glBlendFunciARB"));
01417     glBlendParameteriNV = PFNGLBLENDSEPARATEIARBPROC (glfwGetProcAddress("glBlendParameteriNV"));
01418     glBlitFramebuffer = PFNGLBLITFRAMEBUFFERPROC (glfwGetProcAddress("glBlitFramebuffer"));
01419     glBlitNamedFramebuffer =
PFNGLBLITNAMEDFRAMEBUFFERPROC (glfwGetProcAddress("glBlitNamedFramebuffer"));
01420     glBufferAddressRangeNV =
PFNGLBUFFERADDRESSRANGENVPROC (glfwGetProcAddress("glBufferAddressRangeNV"));
01421     glBufferData = PFNGLBUFFERDATAPROC (glfwGetProcAddress("glBufferData"));
01422     glBufferPageCommitmentARB =
PFNGLBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress("glBufferPageCommitmentARB"));
01423     glBufferStorage = PFNGLBUFFERSTORAGEPROC (glfwGetProcAddress("glBufferStorage"));
01424     glBufferSubData = PFNGLBUFFERSUBDATAPROC (glfwGetProcAddress("glBufferSubData"));
01425     glCallCommandListNV = PFNGLCALLCOMMANDLISTNVPROC (glfwGetProcAddress("glCallCommandListNV"));
01426     glCheckFramebufferStatus =
PFNGLCHECKFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckFramebufferStatus"));
01427     glCheckNamedFramebufferStatus =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckNamedFramebufferStatus"));
01428     glCheckNamedFramebufferStatusEXT =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckNamedFramebufferStatusEXT"));
01429     glClampColor = PFNGLCLAMPCOLORPROC (glfwGetProcAddress("glClampColor"));
01430     glClear = PFNGLCLEARPROC (glfwGetProcAddress("glClear"));
01431     glClearBufferData = PFNGLCLEARBUFFERDATAPROC (glfwGetProcAddress("glClearBufferData"));
01432     glClearBufferSubData = PFNGLCLEARBUFFERSUBDATAPROC (glfwGetProcAddress("glClearBufferSubData"));
01433     glClearBufferfi = PFNGLCLEARBUFFERFIPROC (glfwGetProcAddress("glClearBufferfi"));
01434     glClearBufferfv = PFNGLCLEARBUFFERFVPROC (glfwGetProcAddress("glClearBufferfv"));
01435     glClearBufferiv = PFNGLCLEARBUFFERFVPROC (glfwGetProcAddress("glClearBufferiv"));
01436     glClearBufferuiv = PFNGLCLEARBUFFERUIVPROC (glfwGetProcAddress("glClearBufferuiv"));
01437     glClearColor = PFNGLCLEARCOLORPROC (glfwGetProcAddress("glClearColor"));
01438     glClearDepth = PFNGLCLEARDEPTHPROC (glfwGetProcAddress("glClearDepth"));
01439     glClearDepthf = PFNGLCLEARDEPTHFPROC (glfwGetProcAddress("glClearDepthf"));
01440     glClearNamedBufferData =
PFNGLCLEARNAMEDBUFFERDATAPROC (glfwGetProcAddress("glClearNamedBufferData"));
01441     glClearNamedBufferDataEXT =
PFNGLCLEARNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress("glClearNamedBufferDataEXT"));
01442     glClearNamedBufferSubData =
PFNGLCLEARNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress("glClearNamedBufferSubData"));
01443     glClearNamedBufferSubDataEXT =
PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress("glClearNamedBufferSubDataEXT"));
01444     glClearNamedFramebuffersfi =
PFNGLCLEARNAMEDFRAMEBUFFERFIPROC (glfwGetProcAddress("glClearNamedFramebuffersfi"));
01445     glClearNamedFramebuffersfv =
PFNGLCLEARNAMEDFRAMEBUFFERFVPROC (glfwGetProcAddress("glClearNamedFramebuffersfv"));
01446     glClearNamedFramebuffersiv =
PFNGLCLEARNAMEDFRAMEBUFFERFVPROC (glfwGetProcAddress("glClearNamedFramebuffersiv"));
01447     glClearNamedFramebuffersuiv =
PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC (glfwGetProcAddress("glClearNamedFramebuffersuiv"));
01448     glClearStencil = PFNGLCLEARSTENCILPROC (glfwGetProcAddress("glClearStencil"));
01449     glClearTexImage = PFNGLCLEARTEXIMAGEPROC (glfwGetProcAddress("glClearTexImage"));
01450     glClearTexSubImage = PFNGLCLEARTEXSUBIMAGEPROC (glfwGetProcAddress("glClearTexSubImage"));
01451     glClientAttribDefaultEXT =
PFNGLCLIENTATTRIBDEFAULTPROC (glfwGetProcAddress("glClientAttribDefaultEXT"));
01452     glClientWaitSync = PFNGLCLIENTWAITSYNCPROC (glfwGetProcAddress("glClientWaitSync"));
01453     glClipControl = PFNGLCLIPCONTROLPROC (glfwGetProcAddress("glClipControl"));
01454     glColorFormatNV = PFNGLCOLORFORMATNVPROC (glfwGetProcAddress("glColorFormatNV"));
01455     glColorMask = PFNGLCOLORMASKPROC (glfwGetProcAddress("glColorMask"));
01456     glColorMaski = PFNGLCOLORMASKIPROC (glfwGetProcAddress("glColorMaski"));
01457     glCommandListSegmentsNV =
PFNGLCOMMANDLISTSEGMENTSNVPROC (glfwGetProcAddress("glCommandListSegmentsNV"));
01458     glCompileCommandListNV =
PFNGLCOMPILECOMMANDLISTNVPROC (glfwGetProcAddress("glCompileCommandListNV"));
01459     glCompileShader = PFNGLCOMPILESHADERPROC (glfwGetProcAddress("glCompileShader"));
01460     glCompileShaderIncludeARB =
PFNGLCOMPILESHADERINCLUDEARBPROC (glfwGetProcAddress("glCompileShaderIncludeARB"));
01461     glCompressedMultiTexImage1D =
PFNGLCOMPRESSEDMULTITEXIMAGE1DPROC (glfwGetProcAddress("glCompressedMultiTexImage1D"));
01462     glCompressedMultiTexImage2D =
PFNGLCOMPRESSEDMULTITEXIMAGE2DPROC (glfwGetProcAddress("glCompressedMultiTexImage2D"));
01463     glCompressedMultiTexImage3D =
PFNGLCOMPRESSEDMULTITEXIMAGE3DPROC (glfwGetProcAddress("glCompressedMultiTexImage3D"));
01464     glCompressedMultiTexSubImage1D =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DPROC (glfwGetProcAddress("glCompressedMultiTexSubImage1D"));
01465     glCompressedMultiTexSubImage2D =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DPROC (glfwGetProcAddress("glCompressedMultiTexSubImage2D"));
01466     glCompressedMultiTexSubImage3D =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DPROC (glfwGetProcAddress("glCompressedMultiTexSubImage3D"));
01467     glCompressedTexImage1D =
PFNGLCOMPRESSEDTEXIMAGE1DPROC (glfwGetProcAddress("glCompressedTexImage1D"));
01468     glCompressedTexImage2D =
PFNGLCOMPRESSEDTEXIMAGE2DPROC (glfwGetProcAddress("glCompressedTexImage2D"));
01469     glCompressedTexImage3D =

```



```

    PFNGLCOMPRESSEDTEXIMAGE3DPROC (glfwGetProcAddress ("glCompressedTexImage3D"));
01470    glCompressedTexSubImage1D =
    PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC (glfwGetProcAddress ("glCompressedTexSubImage1D"));
01471    glCompressedTexSubImage2D =
    PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC (glfwGetProcAddress ("glCompressedTexSubImage2D"));
01472    glCompressedTexSubImage3D =
    PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTexSubImage3D"));
01473    glCompressedTextureImage1DEXT =
    PFNGLCOMPRESSEDTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage1DEXT"));
01474    glCompressedTextureImage2DEXT =
    PFNGLCOMPRESSEDTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage2DEXT"));
01475    glCompressedTextureImage3DEXT =
    PFNGLCOMPRESSEDTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage3DEXT"));
01476    glCompressedTextureSubImage1D =
    PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCompressedTextureSubImage1D"));
01477    glCompressedTextureSubImage1DEXT =
    PFNGLCOMPRESSEDTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage1DEXT"));
01478    glCompressedTextureSubImage2D =
    PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCompressedTextureSubImage2D"));
01479    glCompressedTextureSubImage2DEXT =
    PFNGLCOMPRESSEDTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage2DEXT"));
01480    glCompressedTextureSubImage3D =
    PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTextureSubImage3D"));
01481    glCompressedTextureSubImage3DEXT =
    PFNGLCOMPRESSEDTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage3DEXT"));
01482    glConservativeRasterParameterfNV =
    PFNGLCONSERVATIVERASTERPARAMETERFNVPROC (glfwGetProcAddress ("glConservativeRasterParameterfNV"));
01483    glConservativeRasterParameteriNV =
    PFNGLCONSERVATIVERASTERPARAMETERINVPROC (glfwGetProcAddress ("glConservativeRasterParameteriNV"));
01484    glCopyBufferSubData = PFNGLCOPYBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyBufferSubData"));
01485    glCopyImageSubData = PFNGLCOPYIMAGESUBDATAPROC (glfwGetProcAddress ("glCopyImageSubData"));
01486    glCopyMultiTexImage1DEXT =
    PFNGLCOPYMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage1DEXT"));
01487    glCopyMultiTexImage2DEXT =
    PFNGLCOPYMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage2DEXT"));
01488    glCopyMultiTexSubImage1DEXT =
    PFNGLCOPYMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage1DEXT"));
01489    glCopyMultiTexSubImage2DEXT =
    PFNGLCOPYMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage2DEXT"));
01490    glCopyMultiTexSubImage3DEXT =
    PFNGLCOPYMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage3DEXT"));
01491    glCopyNamedBufferSubData =
    PFNGLCOPYNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyNamedBufferSubData"));
01492    glCopyPathNV = PFNGLCOPYPATHNVPROC (glfwGetProcAddress ("glCopyPathNV"));
01493    glCopyTexImage1D = PFNGLCOPYTEXIMAGE1DPROC (glfwGetProcAddress ("glCopyTexImage1D"));
01494    glCopyTexImage2D = PFNGLCOPYTEXIMAGE2DPROC (glfwGetProcAddress ("glCopyTexImage2D"));
01495    glCopyTexSubImage1D = PFNGLCOPYTEXSUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTexSubImage1D"));
01496    glCopyTexSubImage2D = PFNGLCOPYTEXSUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTexSubImage2D"));
01497    glCopyTexSubImage3D = PFNGLCOPYTEXSUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTexSubImage3D"));
01498    glCopyTextureImage1DEXT =
    PFNGLCOPYTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyTextureImage1DEXT"));
01499    glCopyTextureImage2DEXT =
    PFNGLCOPYTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyTextureImage2DEXT"));
01500    glCopyTextureSubImage1D =
    PFNGLCOPYTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTextureSubImage1D"));
01501    glCopyTextureSubImage1DEXT =
    PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage1DEXT"));
01502    glCopyTextureSubImage2D =
    PFNGLCOPYTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTextureSubImage2D"));
01503    glCopyTextureSubImage2DEXT =
    PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage2DEXT"));
01504    glCopyTextureSubImage3D =
    PFNGLCOPYTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTextureSubImage3D"));
01505    glCopyTextureSubImage3DEXT =
    PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage3DEXT"));
01506    glCoverFillPathInstancedNV =
    PFNGLCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glCoverFillPathInstancedNV"));
01507    glCoverFillPathNV = PFNGLCOVERFILLPATHNVPROC (glfwGetProcAddress ("glCoverFillPathNV"));
01508    glCoverStrokePathInstancedNV =
    PFNGLCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glCoverStrokePathInstancedNV"));
01509    glCoverStrokePathNV = PFNGLCOVERSTROKEPATHNVPROC (glfwGetProcAddress ("glCoverStrokePathNV"));
01510    glCoverageModulationNV =
    PFNGLCOVERAGEMODULATIONNVPROC (glfwGetProcAddress ("glCoverageModulationNV"));
01511    glCoverageModulationTableNV =
    PFNGLCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress ("glCoverageModulationTableNV"));
01512    glCreateBuffers = PFNGLCREATEBUFFERSPROC (glfwGetProcAddress ("glCreateBuffers"));
01513    glCreateCommandListsNV =
    PFNGLCREATECOMMANDLISTSNVPROC (glfwGetProcAddress ("glCreateCommandListsNV"));
01514    glCreateFramebuffers = PFNGLCREATEFRAMEBUFFERSPROC (glfwGetProcAddress ("glCreateFramebuffers"));
01515    glCreatePerfQueryINTEL =
    PFNGLCREATEPERFQUERYINTELPROC (glfwGetProcAddress ("glCreatePerfQueryINTEL"));
01516    glCreateProgram = PFNGLCREATEPROGRAMPROC (glfwGetProcAddress ("glCreateProgram"));
01517    glCreateProgramPipelines =
    PFNGLCREATEPROGRAMPIPELINESPROC (glfwGetProcAddress ("glCreateProgramPipelines"));
01518    glCreateQueries = PFNGLCREATEQUERIESPROC (glfwGetProcAddress ("glCreateQueries"));
01519    glCreateRenderbuffers = PFNGLCREATERENDERBUFFERSPROC (glfwGetProcAddress ("glCreateRenderbuffers"));
01520    glCreateSamplers = PFNGLCREATESAMPLERSPROC (glfwGetProcAddress ("glCreateSamplers"));

```

```
01521     glCreateShader = PFNGLCREATESHADERPROC (glfwGetProcAddress("glCreateShader"));
01522     glCreateShaderProgramEXT =
PFNGLCREATESHADERPROGRAMEXTPROC (glfwGetProcAddress("glCreateShaderProgramEXT"));
01523     glCreateShaderProgramv =
PFNGLCREATESHADERPROGRAMVPROC (glfwGetProcAddress("glCreateShaderProgramv"));
01524     glCreateStatesNV = PFNGLCREATESTATESNVPROC (glfwGetProcAddress("glCreateStatesNV"));
01525     glCreateSyncFromCLeventARB =
PFNGLCREATESYNCFROMCLEVENTARBPROC (glfwGetProcAddress("glCreateSyncFromCLeventARB"));
01526     glCreateTextures = PFNGLCREATETEXTURESPROC (glfwGetProcAddress("glCreateTextures"));
01527     glCreateTransformFeedbacks =
PFNGLCREATETRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glCreateTransformFeedbacks"));
01528     glCreateVertexArrays = PFNGLCREATEVERTEXARRAYSPROC (glfwGetProcAddress("glCreateVertexArrays"));
01529     glCullFace = PFNGLCULLFACEPROC (glfwGetProcAddress("glCullFace"));
01530     glDebugMessageCallback =
PFNGLDEBUGMESSAGECALLBACKPROC (glfwGetProcAddress("glDebugMessageCallback"));
01531     glDebugMessageCallbackARB =
PFNGLDEBUGMESSAGECALLBACKARBPROC (glfwGetProcAddress("glDebugMessageCallbackARB"));
01532     glDebugMessageControl = PFNGLDEBUGMESSAGECONTROLPROC (glfwGetProcAddress("glDebugMessageControl"));
01533     glDebugMessageControlARB =
PFNGLDEBUGMESSAGECONTROLARBPROC (glfwGetProcAddress("glDebugMessageControlARB"));
01534     glDebugMessageInsert = PFNGLDEBUGMESSAGEINSERTPROC (glfwGetProcAddress("glDebugMessageInsert"));
01535     glDebugMessageInsertARB =
PFNGLDEBUGMESSAGEINSERTARBPROC (glfwGetProcAddress("glDebugMessageInsertARB"));
01536     glDeleteBuffers = PFNGLDELETEBUFFERSPROC (glfwGetProcAddress("glDeleteBuffers"));
01537     glDeleteCommandListsNV =
PFNGLDELETECOMMANDLISTSNVPROC (glfwGetProcAddress("glDeleteCommandListsNV"));
01538     glDeleteFramebuffers = PFNGLDELETEFRAMEBUFFERSPROC (glfwGetProcAddress("glDeleteFramebuffers"));
01539     glDeleteNamedStringARB =
PFNGLDELETENAMEDSTRINGARBPROC (glfwGetProcAddress("glDeleteNamedStringARB"));
01540     glDeletePathsNV = PFNGLDELETEPATHSNVPROC (glfwGetProcAddress("glDeletePathsNV"));
01541     glDeletePerfMonitorsAMD =
PFNGLDELETEPERFMONITORSAMDPROC (glfwGetProcAddress("glDeletePerfMonitorsAMD"));
01542     glDeletePerfQueryINTEL =
PFNGLDELETEPERFQUERYINTELPROC (glfwGetProcAddress("glDeletePerfQueryINTEL"));
01543     glDeleteProgram = PFNGLDELETEPROGRAMPROC (glfwGetProcAddress("glDeleteProgram"));
01544     glDeleteProgramPipelines =
PFNGLDELETEPROGRAMPIPELINESPROC (glfwGetProcAddress("glDeleteProgramPipelines"));
01545     glDeleteQueries = PFNGLDELETEQUERIESPROC (glfwGetProcAddress("glDeleteQueries"));
01546     glDeleteRenderbuffers = PFNGLDELETERENDERBUFFERSPROC (glfwGetProcAddress("glDeleteRenderbuffers"));
01547     glDeleteSamplers = PFNGLDELETESAMPLERSPROC (glfwGetProcAddress("glDeleteSamplers"));
01548     glDeleteShader = PFNGLDELETESHADERPROC (glfwGetProcAddress("glDeleteShader"));
01549     glDeleteStatesNV = PFNGLDELETESTATESNVPROC (glfwGetProcAddress("glDeleteStatesNV"));
01550     glDeleteSync = PFNGLDELETESYNCPROC (glfwGetProcAddress("glDeleteSync"));
01551     glDeleteTextures = PFNGLDELETETEXTURESPROC (glfwGetProcAddress("glDeleteTextures"));
01552     glDeleteTransformFeedbacks =
PFNGLDELETETRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glDeleteTransformFeedbacks"));
01553     glDeleteVertexArrays = PFNGLDELETEVERTEXARRAYSPROC (glfwGetProcAddress("glDeleteVertexArrays"));
01554     glDepthFunc = PFNGLDEPTHFUNCPROC (glfwGetProcAddress("glDepthFunc"));
01555     glDepthMask = PFNGLDEPTHMASKPROC (glfwGetProcAddress("glDepthMask"));
01556     glDepthRange = PFNGLDEPTHRANGEPROC (glfwGetProcAddress("glDepthRange"));
01557     glDepthRangeArrayv = PFNGLDEPTHRANGEARRAYVPROC (glfwGetProcAddress("glDepthRangeArrayv"));
01558     glDepthRangeIndexed = PFNGLDEPTHRANGEINDEXEDPROC (glfwGetProcAddress("glDepthRangeIndexed"));
01559     glDepthRangef = PFNGLDEPTHRANGEFPROC (glfwGetProcAddress("glDepthRangef"));
01560     glDetachShader = PFNGLDETACHSHADERPROC (glfwGetProcAddress("glDetachShader"));
01561     glDisable = PFNGLDISABLEPROC (glfwGetProcAddress("glDisable"));
01562     glDisableClientStateIndexedEXT =
PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress("glDisableClientStateIndexedEXT"));
01563     glDisableClientStateiEXT =
PFNGLDISABLECLIENTSTATEIEXTPROC (glfwGetProcAddress("glDisableClientStateiEXT"));
01564     glDisableIndexedEXT = PFNGLDISABLEINDEXEDEXTPROC (glfwGetProcAddress("glDisableIndexedEXT"));
01565     glDisableVertexArrayAttrib =
PFNGLDISABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress("glDisableVertexArrayAttrib"));
01566     glDisableVertexArrayAttribEXT =
PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress("glDisableVertexArrayAttribEXT"));
01567     glDisableVertexArrayEXT =
PFNGLDISABLEVERTEXARRAYEXTPROC (glfwGetProcAddress("glDisableVertexArrayEXT"));
01568     glDisableVertexAttribArray =
PFNGLDISABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress("glDisableVertexAttribArray"));
01569     glDisablei = PFNGLDISABLEIPROC (glfwGetProcAddress("glDisablei"));
01570     glDispatchCompute = PFNGLDISPATCHCOMPUTEPROC (glfwGetProcAddress("glDispatchCompute"));
01571     glDispatchComputeGroupSizeARB =
PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC (glfwGetProcAddress("glDispatchComputeGroupSizeARB"));
01572     glDispatchComputeIndirect =
PFNGLDISPATCHCOMPUTEINDIRECTPROC (glfwGetProcAddress("glDispatchComputeIndirect"));
01573     glDrawArrays = PFNGLDRAWARRAYSPROC (glfwGetProcAddress("glDrawArrays"));
01574     glDrawArraysIndirect = PFNGLDRAWARRAYSINDIRECTPROC (glfwGetProcAddress("glDrawArraysIndirect"));
01575     glDrawArraysInstanced = PFNGLDRAWARRAYSINSTANCEDPROC (glfwGetProcAddress("glDrawArraysInstanced"));
01576     glDrawArraysInstancedARB =
PFNGLDRAWARRAYSINSTANCEDARBPROC (glfwGetProcAddress("glDrawArraysInstancedARB"));
01577     glDrawArraysInstancedBaseInstance =
PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress("glDrawArraysInstancedBaseInstance"));
01578     glDrawArraysInstancedEXT =
PFNGLDRAWARRAYSINSTANCEDEXTPROC (glfwGetProcAddress("glDrawArraysInstancedEXT"));
01579     glDrawBuffer = PFNGLDRAWBUFFERPROC (glfwGetProcAddress("glDrawBuffer"));
01580     glDrawBuffers = PFNGLDRAWBUFFERSPROC (glfwGetProcAddress("glDrawBuffers"));
01581     glDrawCommandsAddressNV =
PFNGLDRAWCOMMANDSADDRESSNVPROC (glfwGetProcAddress("glDrawCommandsAddressNV"));
```



```

01582     glDrawCommandsNV = PFNGLDRAWCOMMANDSNVPROC (glfwGetProcAddress ("glDrawCommandsNV"));
01583     glDrawCommandsStatesAddressNV =
PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC (glfwGetProcAddress ("glDrawCommandsStatesAddressNV"));
01584     glDrawCommandsStatesNV =
PFNGLDRAWCOMMANDSSTATESNVPROC (glfwGetProcAddress ("glDrawCommandsStatesNV"));
01585     glDrawElements = PFNGLDRAWELEMENTSPROC (glfwGetProcAddress ("glDrawElements"));
01586     glDrawElementsBaseVertex =
PFNGLDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress ("glDrawElementsBaseVertex"));
01587     glDrawElementsIndirect =
PFNGLDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress ("glDrawElementsIndirect"));
01588     glDrawElementsInstanced =
PFNGLDRAWELEMENTSINSTANCEDPROC (glfwGetProcAddress ("glDrawElementsInstanced"));
01589     glDrawElementsInstancedARB =
PFNGLDRAWELEMENTSINSTANCEDARBPROC (glfwGetProcAddress ("glDrawElementsInstancedARB"));
01590     glDrawElementsInstancedBaseInstance =
PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress ("glDrawElementsInstancedBaseInstance"));
01591     glDrawElementsInstancedBaseVertex =
PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC (glfwGetProcAddress ("glDrawElementsInstancedBaseVertex"));
01592     glDrawElementsInstancedBaseVertexBaseInstance =
PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC (glfwGetProcAddress ("glDrawElementsInstancedBaseVertexBaseInstance"));
01593     glDrawElementsInstancedEXT =
PFNGLDRAWELEMENTSINSTANCEDEXTPROC (glfwGetProcAddress ("glDrawElementsInstancedEXT"));
01594     glDrawRangeElements = PFNGLDRAWRANGEELEMENTSPROC (glfwGetProcAddress ("glDrawRangeElements"));
01595     glDrawRangeElementsBaseVertex =
PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC (glfwGetProcAddress ("glDrawRangeElementsBaseVertex"));
01596     glDrawTransformFeedback =
PFNGLDRAWTRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glDrawTransformFeedback"));
01597     glDrawTransformFeedbackInstanced =
PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC (glfwGetProcAddress ("glDrawTransformFeedbackInstanced"));
01598     glDrawTransformFeedbackStream =
PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC (glfwGetProcAddress ("glDrawTransformFeedbackStream"));
01599     glDrawTransformFeedbackStreamInstanced =
PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC (glfwGetProcAddress ("glDrawTransformFeedbackStreamInstanced"));
01600     glDrawVkImageNV = PFNGLDRAWVKIMAGENVPROC (glfwGetProcAddress ("glDrawVkImageNV"));
01601     glEdgeFlagFormatNV = PFNGLEDGEFLAGFORMATNVPROC (glfwGetProcAddress ("glEdgeFlagFormatNV"));
01602     glEnable = PFNGLENABLEPROC (glfwGetProcAddress ("glEnable"));
01603     glEnableClientStateIndexedEXT =
PFNGLENABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress ("glEnableClientStateIndexedEXT"));
01604     glEnableClientStateiEXT =
PFNGLENABLECLIENTSTATEIEXTPROC (glfwGetProcAddress ("glEnableClientStateiEXT"));
01605     glEnableIndexedEXT = PFNGLENABLEINDEXEDEXTPROC (glfwGetProcAddress ("glEnableIndexedEXT"));
01606     glEnableVertexArrayAttrib =
PFNGLENABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress ("glEnableVertexArrayAttrib"));
01607     glEnableVertexArrayAttribEXT =
PFNGLENABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress ("glEnableVertexArrayAttribEXT"));
01608     glEnableVertexArrayEXT =
PFNGLENABLEVERTEXARRAYEXTPROC (glfwGetProcAddress ("glEnableVertexArrayEXT"));
01609     glEnableVertexArrayAttribArray =
PFNGLENABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress ("glEnableVertexArrayAttribArray"));
01610     glEnablei = PFNGLENABLEIPROC (glfwGetProcAddress ("glEnablei"));
01611     glEndConditionalRender =
PFNGLENDCONDITIONALRENDERPROC (glfwGetProcAddress ("glEndConditionalRender"));
01612     glEndConditionalRenderNV =
PFNGLENDCONDITIONALRENDERNVPROC (glfwGetProcAddress ("glEndConditionalRenderNV"));
01613     glEndPerfMonitorAMD = PFNGLENDPERFMONITORAMDPROC (glfwGetProcAddress ("glEndPerfMonitorAMD"));
01614     glEndPerfQueryINTEL = PFNGLENDPERFQUERYINTELPROC (glfwGetProcAddress ("glEndPerfQueryINTEL"));
01615     glEndQuery = PFNGLENDQUERYPROC (glfwGetProcAddress ("glEndQuery"));
01616     glEndQueryIndexed = PFNGLENDQUERYINDEXEDPROC (glfwGetProcAddress ("glEndQueryIndexed"));
01617     glEndTransformFeedback =
PFNGLENDTRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glEndTransformFeedback"));
01618     glEvaluateDepthValuesARB =
PFNGLEVALUATEDEPTHVALUESARBPROC (glfwGetProcAddress ("glEvaluateDepthValuesARB"));
01619     glFenceSync = PFNGLFENCESYNCPROC (glfwGetProcAddress ("glFenceSync"));
01620     glFinish = PFNGLFINISHPROC (glfwGetProcAddress ("glFinish"));
01621     glFlush = PFNGLFLUSHPROC (glfwGetProcAddress ("glFlush"));
01622     glFlushMappedBufferRange =
PFNGLFLUSHMAPPEDBUFFERRANGEPROC (glfwGetProcAddress ("glFlushMappedBufferRange"));
01623     glFlushMappedNamedBufferRange =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC (glfwGetProcAddress ("glFlushMappedNamedBufferRange"));
01624     glFlushMappedNamedBufferRangeEXT =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress ("glFlushMappedNamedBufferRangeEXT"));
01625     glFogCoordFormatNV = PFNGLFOGCOORDFORMATNVPROC (glfwGetProcAddress ("glFogCoordFormatNV"));
01626     glFragmentCoverageColorNV =
PFNGLFRAGMENTCOVERAGECOLORNVPROC (glfwGetProcAddress ("glFragmentCoverageColorNV"));
01627     glFramebufferDrawBufferEXT =
PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC (glfwGetProcAddress ("glFramebufferDrawBufferEXT"));
01628     glFramebufferDrawBuffersEXT =
PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC (glfwGetProcAddress ("glFramebufferDrawBuffersEXT"));
01629     glFramebufferParameteri =
PFNGLFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress ("glFramebufferParameteri"));
01630     glFramebufferReadBufferEXT =
PFNGLFRAMEBUFFERREADBUFFEREXTPROC (glfwGetProcAddress ("glFramebufferReadBufferEXT"));
01631     glFramebufferRenderbuffer =
PFNGLFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress ("glFramebufferRenderbuffer"));
01632     glFramebufferSampleLocationsfvARB =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress ("glFramebufferSampleLocationsfvARB"));
01633     glFramebufferSampleLocationsfvNV =

```

```

        PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress ("glFramebufferSampleLocationsfvNV"));
01634     glFramebufferTexture = PFNGLFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress ("glFramebufferTexture"));
01635     glFramebufferTexture1D =
        PFNGLFRAMEBUFFERTEXTURE1DPROC (glfwGetProcAddress ("glFramebufferTexture1D"));
01636     glFramebufferTexture2D =
        PFNGLFRAMEBUFFERTEXTURE2DPROC (glfwGetProcAddress ("glFramebufferTexture2D"));
01637     glFramebufferTexture3D =
        PFNGLFRAMEBUFFERTEXTURE3DPROC (glfwGetProcAddress ("glFramebufferTexture3D"));
01638     glFramebufferTextureARB =
        PFNGLFRAMEBUFFERTEXTUREARBPROC (glfwGetProcAddress ("glFramebufferTextureARB"));
01639     glFramebufferTextureFaceARB =
        PFNGLFRAMEBUFFERTEXTUREFACEARBPROC (glfwGetProcAddress ("glFramebufferTextureFaceARB"));
01640     glFramebufferTextureLayer =
        PFNGLFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress ("glFramebufferTextureLayer"));
01641     glFramebufferTextureLayerARB =
        PFNGLFRAMEBUFFERTEXTURELAYERARBPROC (glfwGetProcAddress ("glFramebufferTextureLayerARB"));
01642     glFramebufferTextureMultiviewOVR =
        PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC (glfwGetProcAddress ("glFramebufferTextureMultiviewOVR"));
01643     glFrontFace = PFNGLFRONTFACEPROC (glfwGetProcAddress ("glFrontFace"));
01644     glGenBuffers = PFNGLGENBUFFERSPROC (glfwGetProcAddress ("glGenBuffers"));
01645     glGenFramebuffers = PFNGLGENFRAMEBUFFERSPROC (glfwGetProcAddress ("glGenFramebuffers"));
01646     glGenPathsNV = PFNGLGENPATHSNVPROC (glfwGetProcAddress ("glGenPathsNV"));
01647     glGenPerfMonitorsAMD = PFNGLGENPERFMONITORSAMDPROC (glfwGetProcAddress ("glGenPerfMonitorsAMD"));
01648     glGenProgramPipelines = PFNGLGENPROGRAMPIPELINESPROC (glfwGetProcAddress ("glGenProgramPipelines"));
01649     glGenQueries = PFNGLGENQUERIESPROC (glfwGetProcAddress ("glGenQueries"));
01650     glGenRenderbuffers = PFNGLGENRENDERBUFFERSPROC (glfwGetProcAddress ("glGenRenderbuffers"));
01651     glGenSamplers = PFNGLGENSAMPLERSPROC (glfwGetProcAddress ("glGenSamplers"));
01652     glGenTextures = PFNGLGENTEXTURESPROC (glfwGetProcAddress ("glGenTextures"));
01653     glGenTransformFeedbacks =
        PFNGLGENTRANSFORMFEEDBACKSPROC (glfwGetProcAddress ("glGenTransformFeedbacks"));
01654     glGenVertexArrays = PFNGLGENVERTEXARRAYSPROC (glfwGetProcAddress ("glGenVertexArrays"));
01655     glGenerateMipmap = PFNGLGENERATEMIPMAPPROC (glfwGetProcAddress ("glGenerateMipmap"));
01656     glGenerateMultiTexMipmapEXT =
        PFNGLGENERATEMULTITEXMIPMAPEXTPROC (glfwGetProcAddress ("glGenerateMultiTexMipmapEXT"));
01657     glGenerateTextureMipmap =
        PFNGLGENERATETEXTUREMIPMAPPROC (glfwGetProcAddress ("glGenerateTextureMipmap"));
01658     glGenerateTextureMipmapEXT =
        PFNGLGENERATETEXTUREMIPMAPEXTPROC (glfwGetProcAddress ("glGenerateTextureMipmapEXT"));
01659     glGetActiveAtomicCounterBufferiv =
        PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC (glfwGetProcAddress ("glGetActiveAtomicCounterBufferiv"));
01660     glGetActiveAttrib = PFNGLGETACTIVEATTRIBPROC (glfwGetProcAddress ("glGetActiveAttrib"));
01661     glGetActiveSubroutineName =
        PFNGLGETACTIVESUBROUTINENAMEPROC (glfwGetProcAddress ("glGetActiveSubroutineName"));
01662     glGetActiveSubroutineUniformName =
        PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC (glfwGetProcAddress ("glGetActiveSubroutineUniformName"));
01663     glGetActiveSubroutineUniformiv =
        PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC (glfwGetProcAddress ("glGetActiveSubroutineUniformiv"));
01664     glGetActiveUniform = PFNGLGETACTIVEUNIFORMPROC (glfwGetProcAddress ("glGetActiveUniform"));
01665     glGetActiveUniformBlockName =
        PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC (glfwGetProcAddress ("glGetActiveUniformBlockName"));
01666     glGetActiveUniformBlockiv =
        PFNGLGETACTIVEUNIFORMBLOCKIVPROC (glfwGetProcAddress ("glGetActiveUniformBlockiv"));
01667     glGetActiveUniformName =
        PFNGLGETACTIVEUNIFORMNAMEPROC (glfwGetProcAddress ("glGetActiveUniformName"));
01668     glGetActiveUniformsiv = PFNGLGETACTIVEUNIFORMSIVPROC (glfwGetProcAddress ("glGetActiveUniformsiv"));
01669     glGetAttachedShaders = PFNGLGETATTACHEDSHADERSPROC (glfwGetProcAddress ("glGetAttachedShaders"));
01670     glGetAttribLocation = PFNGLGETATTRIBLOCATIONPROC (glfwGetProcAddress ("glGetAttribLocation"));
01671     glGetBooleanIndexedvEXT =
        PFNGLGETBOOLEANINDEXEDVEXTPROC (glfwGetProcAddress ("glGetBooleanIndexedvEXT"));
01672     glGetBooleani_v = PFNGLGETBOOLEANI_VPROC (glfwGetProcAddress ("glGetBooleani_v"));
01673     glGetBooleani_v = PFNGLGETBOOLEANI_VPROC (glfwGetProcAddress ("glGetBooleani_v"));
01674     glGetBufferParameteri64v =
        PFNGLGETBUFFERPARAMETERI64VPROC (glfwGetProcAddress ("glGetBufferParameteri64v"));
01675     glGetBufferParameteriv =
        PFNGLGETBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetBufferParameteriv"));
01676     glGetBufferParameterui64vNV =
        PFNGLGETBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress ("glGetBufferParameterui64vNV"));
01677     glGetBufferPointerv = PFNGLGETBUFFERPOINTERVPROC (glfwGetProcAddress ("glGetBufferPointerv"));
01678     glGetBufferSubData = PFNGLGETBUFFERSUBDATAPROC (glfwGetProcAddress ("glGetBufferSubData"));
01679     glGetCommandHeaderNV = PFNGLGETCOMMANDHEADERNVPROC (glfwGetProcAddress ("glGetCommandHeaderNV"));
01680     glGetCompressedMultiTexImageEXT =
        PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC (glfwGetProcAddress ("glGetCompressedMultiTexImageEXT"));
01681     glGetCompressedTexImage =
        PFNGLGETCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress ("glGetCompressedTexImage"));
01682     glGetCompressedTextureImage =
        PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC (glfwGetProcAddress ("glGetCompressedTextureImage"));
01683     glGetCompressedTextureImageEXT =
        PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC (glfwGetProcAddress ("glGetCompressedTextureImageEXT"));
01684     glGetCompressedTextureSubImage =
        PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC (glfwGetProcAddress ("glGetCompressedTextureSubImage"));
01685     glGetCoverageModulationTableNV =
        PFNGLGETCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress ("glGetCoverageModulationTableNV"));
01686     glGetDebugMessageLog = PFNGLGETDEBUGMESSAGELOGPROC (glfwGetProcAddress ("glGetDebugMessageLog"));
01687     glGetDebugMessageLogARB =
        PFNGLGETDEBUGMESSAGELOGARBPROC (glfwGetProcAddress ("glGetDebugMessageLogARB"));
01688     glGetDoubleIndexedvEXT =
        PFNGLGETDOUBLEINDEXEDVEXTPROC (glfwGetProcAddress ("glGetDoubleIndexedvEXT"));

```

```

01689     glGetDoublei_v = PFNGLGETDOUBLEI_VPROC (glfwGetProcAddress ("glGetDoublei_v"));
01690     glGetDoublei_vEXT = PFNGLGETDOUBLEI_VEXTPROC (glfwGetProcAddress ("glGetDoublei_vEXT"));
01691     glGetDoublev = PFNGLGETDOUBLEVPROC (glfwGetProcAddress ("glGetDoublev"));
01692     glGetError = PFNGLGETERRORPROC (glfwGetProcAddress ("glGetError"));
01693     glGetFirstPerfQueryIdINTEL =
PFNGLGETFIRSTPERFQUERYIDINTELPROC (glfwGetProcAddress ("glGetFirstPerfQueryIdINTEL"));
01694     glGetFloatIndexedvEXT = PFNGLGETFLOATINDEXEDVEXTPROC (glfwGetProcAddress ("glGetFloatIndexedvEXT"));
01695     glGetFloati_v = PFNGLGETFLOATI_VPROC (glfwGetProcAddress ("glGetFloati_v"));
01696     glGetFloati_vEXT = PFNGLGETFLOATI_VEXTPROC (glfwGetProcAddress ("glGetFloati_vEXT"));
01697     glGetFloatv = PFNGLGETFLOATVPROC (glfwGetProcAddress ("glGetFloatv"));
01698     glGetFragDataIndex = PFNGLGETFRAGDATAINDEXPROC (glfwGetProcAddress ("glGetFragDataIndex"));
01699     glGetFragDataLocation = PFNGLGETFRAGDATALOCATIONPROC (glfwGetProcAddress ("glGetFragDataLocation"));
01700     glGetFramebufferAttachmentParameteriv =
PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress ("glGetFramebufferAttachmentParameteriv"));
01701     glGetFramebufferParameteriv =
PFNGLGETFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetFramebufferParameteriv"));
01702     glGetFramebufferParameterivEXT =
PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetFramebufferParameterivEXT"));
01703     glGetGraphicsResetStatus =
PFNGLGETGRAPHICSRESETSTATUSPROC (glfwGetProcAddress ("glGetGraphicsResetStatus"));
01704     glGetGraphicsResetStatusARB =
PFNGLGETGRAPHICSRESETSTATUSARBPROC (glfwGetProcAddress ("glGetGraphicsResetStatusARB"));
01705     glGetImageHandleARB = PFNGLGETIMAGEHANDLEARBPROC (glfwGetProcAddress ("glGetImageHandleARB"));
01706     glGetImageHandleNV = PFNGLGETIMAGEHANDLENVPROC (glfwGetProcAddress ("glGetImageHandleNV"));
01707     glGetInteger64i_v = PFNGLGETINTEGER64I_VPROC (glfwGetProcAddress ("glGetInteger64i_v"));
01708     glGetInteger64v = PFNGLGETINTEGER64VPROC (glfwGetProcAddress ("glGetInteger64v"));
01709     glGetIntegerIndexedvEXT =
PFNGLGETINTEGERINDEXEDVEXTPROC (glfwGetProcAddress ("glGetIntegerIndexedvEXT"));
01710     glGetIntegeri_v = PFNGLGETINTEGERI_VPROC (glfwGetProcAddress ("glGetIntegeri_v"));
01711     glGetIntegerui64i_vNV = PFNGLGETINTEGERUI64I_VNVPROC (glfwGetProcAddress ("glGetIntegerui64i_vNV"));
01712     glGetIntegerui64vNV = PFNGLGETINTEGERUI64VNVPROC (glfwGetProcAddress ("glGetIntegerui64vNV"));
01713     glGetIntegerv = PFNGLGETINTEGERVPROC (glfwGetProcAddress ("glGetIntegerv"));
01714     glGetInternalFormatSampleivNV =
PFNGLGETINTERNALFORMATSAMPLEIVNVPROC (glfwGetProcAddress ("glGetInternalFormatSampleivNV"));
01715     glGetInternalFormati64v =
PFNGLGETINTERNALFORMATI64VPROC (glfwGetProcAddress ("glGetInternalFormati64v"));
01716     glGetInternalFormativ = PFNGLGETINTERNALFORMATIVPROC (glfwGetProcAddress ("glGetInternalFormativ"));
01717     glGetMultiTexEnvfvEXT = PFNGLGETMULTITEXENVFVEXTPROC (glfwGetProcAddress ("glGetMultiTexEnvfvEXT"));
01718     glGetMultiTexEnvivEXT = PFNGLGETMULTITEXENVIVEXTPROC (glfwGetProcAddress ("glGetMultiTexEnvivEXT"));
01719     glGetMultiTexGendvEXT = PFNGLGETMULTITEXGENDVEXTPROC (glfwGetProcAddress ("glGetMultiTexGendvEXT"));
01720     glGetMultiTexGenfvEXT = PFNGLGETMULTITEXGENFVEXTPROC (glfwGetProcAddress ("glGetMultiTexGenfvEXT"));
01721     glGetMultiTexGenivEXT = PFNGLGETMULTITEXGENIVEXTPROC (glfwGetProcAddress ("glGetMultiTexGenivEXT"));
01722     glGetMultiTexImageEXT = PFNGLGETMULTITEXIMAGEEXTPROC (glfwGetProcAddress ("glGetMultiTexImageEXT"));
01723     glGetMultiTexLevelParameterfvEXT =
PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetMultiTexLevelParameterfvEXT"));
01724     glGetMultiTexLevelParameterivEXT =
PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetMultiTexLevelParameterivEXT"));
01725     glGetMultiTexParameterIivEXT =
PFNGLGETMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterIivEXT"));
01726     glGetMultiTexParameterIuivEXT =
PFNGLGETMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterIuivEXT"));
01727     glGetMultiTexParameterfvEXT =
PFNGLGETMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterfvEXT"));
01728     glGetMultiTexParameterivEXT =
PFNGLGETMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterivEXT"));
01729     glGetMultisamplefv = PFNGLGETMULTISAMPLEFVPROC (glfwGetProcAddress ("glGetMultisamplefv"));
01730     glGetNamedBufferParameteri64v =
PFNGLGETNAMEDBUFFERPARAMETERI64VPROC (glfwGetProcAddress ("glGetNamedBufferParameteri64v"));
01731     glGetNamedBufferParameteriv =
PFNGLGETNAMEDBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedBufferParameteriv"));
01732     glGetNamedBufferParameterivEXT =
PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedBufferParameterivEXT"));
01733     glGetNamedBufferParameterui64vNV =
PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress ("glGetNamedBufferParameterui64vNV"));
01734     glGetNamedBufferPointerv =
PFNGLGETNAMEDBUFFERPOINTERVPROC (glfwGetProcAddress ("glGetNamedBufferPointerv"));
01735     glGetNamedBufferPointervEXT =
PFNGLGETNAMEDBUFFERPOINTERVEXTPROC (glfwGetProcAddress ("glGetNamedBufferPointervEXT"));
01736     glGetNamedBufferSubData =
PFNGLGETNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glGetNamedBufferSubData"));
01737     glGetNamedBufferSubDataEXT =
PFNGLGETNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glGetNamedBufferSubDataEXT"));
01738     glGetNamedFramebufferAttachmentParameteriv =
PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedFramebufferAttachmentParameteriv"));
01739     glGetNamedFramebufferAttachmentParameterivEXT =
PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedFramebufferAttachmentParameterivEXT"));
01740     glGetNamedFramebufferParameteriv =
PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedFramebufferParameteriv"));
01741     glGetNamedFramebufferParameterivEXT =
PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedFramebufferParameterivEXT"));
01742     glGetNamedProgramLocalParameterIivEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterIivEXT"));
01743     glGetNamedProgramLocalParameterIuivEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterIuivEXT"));
01744     glGetNamedProgramLocalParameterdvEXT =
PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterdvEXT"));
01745     glGetNamedProgramLocalParameterfvEXT =

```

```

    PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterfvEXT"));
01746   glGetNamedProgramStringEXT =
    PFNGLGETNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress ("glGetNamedProgramStringEXT"));
01747   glGetNamedProgramivEXT =
    PFNGLGETNAMEDPROGRAMIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramivEXT"));
01748   glGetNamedRenderbufferParameteriv =
    PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedRenderbufferParameteriv"));
01749   glGetNamedRenderbufferParameterivEXT =
    PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedRenderbufferParameterivEXT"));
01750   glGetNamedStringARB = PFNGLGETNAMEDSTRINGARBPROC (glfwGetProcAddress ("glGetNamedStringARB"));
01751   glGetNamedStringivARB = PFNGLGETNAMEDSTRINGIVARBPROC (glfwGetProcAddress ("glGetNamedStringivARB"));
01752   glGetNextPerfQueryIdINTEL =
    PFNGLGETNEXTPERFQUERYIDINTELPROC (glfwGetProcAddress ("glGetNextPerfQueryIdINTEL"));
01753   glGetObjectLabel = PFNGLGETOBJECTLABELPROC (glfwGetProcAddress ("glGetObjectLabel"));
01754   glGetObjectLabelEXT = PFNGLGETOBJECTLABELEXTPROC (glfwGetProcAddress ("glGetObjectLabelEXT"));
01755   glGetObjectPtrLabel = PFNGLGETOBJECTPTRLABELPROC (glfwGetProcAddress ("glGetObjectPtrLabel"));
01756   glGetPathCommandsNV = PFNGLGETPATHCOMMANDSNVPROC (glfwGetProcAddress ("glGetPathCommandsNV"));
01757   glGetPathCoordsNV = PFNGLGETPATHCOORDSNVPROC (glfwGetProcAddress ("glGetPathCoordsNV"));
01758   glGetPathDashArrayNV = PFNGLGETPATHDASHARRAYNVPROC (glfwGetProcAddress ("glGetPathDashArrayNV"));
01759   glGetPathLengthNV = PFNGLGETPATHLENGTHNVPROC (glfwGetProcAddress ("glGetPathLengthNV"));
01760   glGetPathMetricRangeNV =
    PFNGLGETPATHMETRICRANGENVPROC (glfwGetProcAddress ("glGetPathMetricRangeNV"));
01761   glGetPathMetricsNV = PFNGLGETPATHMETRICSNVPROC (glfwGetProcAddress ("glGetPathMetricsNV"));
01762   glGetPathParameterfvNV =
    PFNGLGETPATHPARAMETERFVNVPROC (glfwGetProcAddress ("glGetPathParameterfvNV"));
01763   glGetPathParameterivNV =
    PFNGLGETPATHPARAMETERIVNVPROC (glfwGetProcAddress ("glGetPathParameterivNV"));
01764   glGetPathSpacingNV = PFNGLGETPATHSPACINGNVPROC (glfwGetProcAddress ("glGetPathSpacingNV"));
01765   glGetPerfCounterInfoINTEL =
    PFNGLGETPERFCOUNTERINFOINTELPROC (glfwGetProcAddress ("glGetPerfCounterInfoINTEL"));
01766   glGetPerfMonitorCounterDataAMD =
    PFNGLGETPERFMONITORCOUNTERDATAAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterDataAMD"));
01767   glGetPerfMonitorCounterInfoAMD =
    PFNGLGETPERFMONITORCOUNTERINFOAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterInfoAMD"));
01768   glGetPerfMonitorCounterStringAMD =
    PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterStringAMD"));
01769   glGetPerfMonitorCountersAMD =
    PFNGLGETPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCountersAMD"));
01770   glGetPerfMonitorGroupStringAMD =
    PFNGLGETPERFMONITORGROUPSTRINGAMDPROC (glfwGetProcAddress ("glGetPerfMonitorGroupStringAMD"));
01771   glGetPerfMonitorGroupsAMD =
    PFNGLGETPERFMONITORGROUPSAMDPROC (glfwGetProcAddress ("glGetPerfMonitorGroupsAMD"));
01772   glGetPerfQueryDataINTEL =
    PFNGLGETPERFQUERYDATAINTELPROC (glfwGetProcAddress ("glGetPerfQueryDataINTEL"));
01773   glGetPerfQueryIdByNameINTEL =
    PFNGLGETPERFQUERYIDBYNAMEINTELPROC (glfwGetProcAddress ("glGetPerfQueryIdByNameINTEL"));
01774   glGetPerfQueryInfoINTEL =
    PFNGLGETPERFQUERYINFOINTELPROC (glfwGetProcAddress ("glGetPerfQueryInfoINTEL"));
01775   glGetPointerIndexedvEXT =
    PFNGLGETPOINTERINDEXEDVEXTPROC (glfwGetProcAddress ("glGetPointerIndexedvEXT"));
01776   glGetPointerivEXT = PFNGLGETPOINTERIVEXTPROC (glfwGetProcAddress ("glGetPointerivEXT"));
01777   glGetPointerv = PFNGLGETPOINTERVPROC (glfwGetProcAddress ("glGetPointerv"));
01778   glGetProgramBinary = PFNGLGETPROGRAMBINARYPROC (glfwGetProcAddress ("glGetProgramBinary"));
01779   glGetProgramInfoLog = PFNGLGETPROGRAMINFOLOGPROC (glfwGetProcAddress ("glGetProgramInfoLog"));
01780   glGetProgramInterfaceiv =
    PFNGLGETPROGRAMINTERFACEIVPROC (glfwGetProcAddress ("glGetProgramInterfaceiv"));
01781   glGetProgramPipelineInfoLog =
    PFNGLGETPROGRAMPIPELINEINFOLOGPROC (glfwGetProcAddress ("glGetProgramPipelineInfoLog"));
01782   glGetProgramPipelineiv =
    PFNGLGETPROGRAMPIPELINEIVPROC (glfwGetProcAddress ("glGetProgramPipelineiv"));
01783   glGetProgramResourceIndex =
    PFNGLGETPROGRAMRESOURCEINDEXPROC (glfwGetProcAddress ("glGetProgramResourceIndex"));
01784   glGetProgramResourceLocation =
    PFNGLGETPROGRAMRESOURCELOCATIONPROC (glfwGetProcAddress ("glGetProgramResourceLocation"));
01785   glGetProgramResourceLocationIndex =
    PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC (glfwGetProcAddress ("glGetProgramResourceLocationIndex"));
01786   glGetProgramResourceName =
    PFNGLGETPROGRAMRESOURCENAMEPROC (glfwGetProcAddress ("glGetProgramResourceName"));
01787   glGetProgramResourcefvNV =
    PFNGLGETPROGRAMRESOURCEFVNVPROC (glfwGetProcAddress ("glGetProgramResourcefvNV"));
01788   glGetProgramResourceiv =
    PFNGLGETPROGRAMRESOURCEIVPROC (glfwGetProcAddress ("glGetProgramResourceiv"));
01789   glGetProgramStageiv = PFNGLGETPROGRAMSTAGEIVPROC (glfwGetProcAddress ("glGetProgramStageiv"));
01790   glGetProgramiv = PFNGLGETPROGRAMIVPROC (glfwGetProcAddress ("glGetProgramiv"));
01791   glGetQueryBufferObjecti64v =
    PFNGLGETQUERYBUFFEROBJECTI64VPROC (glfwGetProcAddress ("glGetQueryBufferObjecti64v"));
01792   glGetQueryBufferObjectiv =
    PFNGLGETQUERYBUFFEROBJECTIVPROC (glfwGetProcAddress ("glGetQueryBufferObjectiv"));
01793   glGetQueryBufferObjectui64v =
    PFNGLGETQUERYBUFFEROBJECTUI64VPROC (glfwGetProcAddress ("glGetQueryBufferObjectui64v"));
01794   glGetQueryBufferObjectuiv =
    PFNGLGETQUERYBUFFEROBJECTUIVPROC (glfwGetProcAddress ("glGetQueryBufferObjectuiv"));
01795   glGetQueryIndexediv = PFNGLGETQUERYINDEXEDIVPROC (glfwGetProcAddress ("glGetQueryIndexediv"));
01796   glGetQueryObjecti64v = PFNGLGETQUERYOBJECTI64VPROC (glfwGetProcAddress ("glGetQueryObjecti64v"));
01797   glGetQueryObjectiv = PFNGLGETQUERYOBJECTIVPROC (glfwGetProcAddress ("glGetQueryObjectiv"));
01798   glGetQueryObjectui64v = PFNGLGETQUERYOBJECTUI64VPROC (glfwGetProcAddress ("glGetQueryObjectui64v"));
01799   glGetQueryObjectuiv = PFNGLGETQUERYOBJECTUIVPROC (glfwGetProcAddress ("glGetQueryObjectuiv"));

```



```

01800     glGetQueryiv = PFNGLGETQUERYIVPROC (glfwGetProcAddress("glGetQueryiv"));
01801     glGetRenderbufferParameteriv =
PFNGLGETRENDERBUFFERPARAMETERIVPROC (glfwGetProcAddress("glGetRenderbufferParameteriv"));
01802     glGetSamplerParameterIiv =
PFNGLGETSAMPLERPARAMETERIIVPROC (glfwGetProcAddress("glGetSamplerParameterIiv"));
01803     glGetSamplerParameterIuiv =
PFNGLGETSAMPLERPARAMETERIUIVPROC (glfwGetProcAddress("glGetSamplerParameterIuiv"));
01804     glGetSamplerParameterfv =
PFNGLGETSAMPLERPARAMETERFVPROC (glfwGetProcAddress("glGetSamplerParameterfv"));
01805     glGetSamplerParameteriv =
PFNGLGETSAMPLERPARAMETERIVPROC (glfwGetProcAddress("glGetSamplerParameteriv"));
01806     glGetShaderInfoLog = PFNGLGETSHADERINFOLOGPROC (glfwGetProcAddress("glGetShaderInfoLog"));
01807     glGetShaderPrecisionFormat =
PFNGLGETSHADERPRECISIONFORMATPROC (glfwGetProcAddress("glGetShaderPrecisionFormat"));
01808     glGetShaderSource = PFNGLGETSHADERSOURCEPROC (glfwGetProcAddress("glGetShaderSource"));
01809     glGetShaderiv = PFNGLGETSHADERIVPROC (glfwGetProcAddress("glGetShaderiv"));
01810     glGetStageIndexNV = PFNGLGETSTAGEINDEXNVPROC (glfwGetProcAddress("glGetStageIndexNV"));
01811     glGetString = PFNGLGETSTRINGPROC (glfwGetProcAddress("glGetString"));
01812     glGetStringi = PFNGLGETSTRINGIPROC (glfwGetProcAddress("glGetStringi"));
01813     glGetSubroutineIndex = PFNGLGETSUBROUTINEINDEXPROC (glfwGetProcAddress("glGetSubroutineIndex"));
01814     glGetSubroutineUniformLocation =
PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC (glfwGetProcAddress("glGetSubroutineUniformLocation"));
01815     glGetSynciv = PFNGLGETSYNIVPROC (glfwGetProcAddress("glGetSynciv"));
01816     glGetTexImage = PFNGLGETTEXIMAGEPROC (glfwGetProcAddress("glGetTexImage"));
01817     glGetTexLevelParameterfv =
PFNGLGETTEXLEVELPARAMETERFVPROC (glfwGetProcAddress("glGetTexLevelParameterfv"));
01818     glGetTexLevelParameteriv =
PFNGLGETTEXLEVELPARAMETERIVPROC (glfwGetProcAddress("glGetTexLevelParameteriv"));
01819     glGetTexParameterIiv = PFNGLGETTEXPARAMETERIIVPROC (glfwGetProcAddress("glGetTexParameterIiv"));
01820     glGetTexParameterIuiv = PFNGLGETTEXPARAMETERIUIVPROC (glfwGetProcAddress("glGetTexParameterIuiv"));
01821     glGetTexParameterfv = PFNGLGETTEXPARAMETERFVPROC (glfwGetProcAddress("glGetTexParameterfv"));
01822     glGetTexParameteriv = PFNGLGETTEXPARAMETERIVPROC (glfwGetProcAddress("glGetTexParameteriv"));
01823     glGetTextureHandleARB = PFNGLGETTEXTUREHANDLEARBPROC (glfwGetProcAddress("glGetTextureHandleARB"));
01824     glGetTextureHandleNV = PFNGLGETTEXTUREHANDLENVPROC (glfwGetProcAddress("glGetTextureHandleNV"));
01825     glGetTextureImage = PFNGLGETTEXTUREIMAGEPROC (glfwGetProcAddress("glGetTextureImage"));
01826     glGetTextureImageEXT = PFNGLGETTEXTUREIMAGEEXTPROC (glfwGetProcAddress("glGetTextureImageEXT"));
01827     glGetTextureLevelParameterfv =
PFNGLGETTEXTURELEVELPARAMETERFVPROC (glfwGetProcAddress("glGetTextureLevelParameterfv"));
01828     glGetTextureLevelParameterfvEXT =
PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC (glfwGetProcAddress("glGetTextureLevelParameterfvEXT"));
01829     glGetTextureLevelParameteriv =
PFNGLGETTEXTURELEVELPARAMETERIVPROC (glfwGetProcAddress("glGetTextureLevelParameteriv"));
01830     glGetTextureLevelParameterivEXT =
PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC (glfwGetProcAddress("glGetTextureLevelParameterivEXT"));
01831     glGetTextureParameterIiv =
PFNGLGETTEXTUREPARAMETERIIVPROC (glfwGetProcAddress("glGetTextureParameterIiv"));
01832     glGetTextureParameterIivEXT =
PFNGLGETTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress("glGetTextureParameterIivEXT"));
01833     glGetTextureParameterIuiv =
PFNGLGETTEXTUREPARAMETERIUIVPROC (glfwGetProcAddress("glGetTextureParameterIuiv"));
01834     glGetTextureParameterIuivEXT =
PFNGLGETTEXTUREPARAMETERIUIVEXTPROC (glfwGetProcAddress("glGetTextureParameterIuivEXT"));
01835     glGetTextureParameterfv =
PFNGLGETTEXTUREPARAMETERFVPROC (glfwGetProcAddress("glGetTextureParameterfv"));
01836     glGetTextureParameterfvEXT =
PFNGLGETTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress("glGetTextureParameterfvEXT"));
01837     glGetTextureParameteriv =
PFNGLGETTEXTUREPARAMETERIVPROC (glfwGetProcAddress("glGetTextureParameteriv"));
01838     glGetTextureParameterivEXT =
PFNGLGETTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress("glGetTextureParameterivEXT"));
01839     glGetTextureSamplerHandleARB =
PFNGLGETTEXTURESAMPLERHANDLEARBPROC (glfwGetProcAddress("glGetTextureSamplerHandleARB"));
01840     glGetTextureSamplerHandleNV =
PFNGLGETTEXTURESAMPLERHANDLENVPROC (glfwGetProcAddress("glGetTextureSamplerHandleNV"));
01841     glGetTextureSubImage = PFNGLGETTEXTURESUBIMAGEPROC (glfwGetProcAddress("glGetTextureSubImage"));
01842     glGetTransformFeedbackVarying =
PFNGLGETTRANSFORMFEEDBACKVARYINGPROC (glfwGetProcAddress("glGetTransformFeedbackVarying"));
01843     glGetTransformFeedbacki64_v =
PFNGLGETTRANSFORMFEEDBACKI64_VPROC (glfwGetProcAddress("glGetTransformFeedbacki64_v"));
01844     glGetTransformFeedbacki_v =
PFNGLGETTRANSFORMFEEDBACKI_VPROC (glfwGetProcAddress("glGetTransformFeedbacki_v"));
01845     glGetTransformFeedbackiv =
PFNGLGETTRANSFORMFEEDBACKIVPROC (glfwGetProcAddress("glGetTransformFeedbackiv"));
01846     glGetUniformBlockIndex =
PFNGLGETUNIFORMBLOCKINDEXPROC (glfwGetProcAddress("glGetUniformBlockIndex"));
01847     glGetUniformIndices = PFNGLGETUNIFORMINDICESPROC (glfwGetProcAddress("glGetUniformIndices"));
01848     glGetUniformLocation = PFNGLGETUNIFORMLOCATIONPROC (glfwGetProcAddress("glGetUniformLocation"));
01849     glGetUniformSubroutineuiv =
PFNGLGETUNIFORMSUBROUTINEUIVPROC (glfwGetProcAddress("glGetUniformSubroutineuiv"));
01850     glGetUniformdv = PFNGLGETUNIFORMDVPROC (glfwGetProcAddress("glGetUniformdv"));
01851     glGetUniformfv = PFNGLGETUNIFORMFVPROC (glfwGetProcAddress("glGetUniformfv"));
01852     glGetUniformi64vARB = PFNGLGETUNIFORMI64VARBPROC (glfwGetProcAddress("glGetUniformi64vARB"));
01853     glGetUniformi64vNV = PFNGLGETUNIFORMI64VNVPROC (glfwGetProcAddress("glGetUniformi64vNV"));
01854     glGetUniformiv = PFNGLGETUNIFORMIVPROC (glfwGetProcAddress("glGetUniformiv"));
01855     glGetUniformui64vARB = PFNGLGETUNIFORMUI64VARBPROC (glfwGetProcAddress("glGetUniformui64vARB"));
01856     glGetUniformui64vNV = PFNGLGETUNIFORMUI64VNVPROC (glfwGetProcAddress("glGetUniformui64vNV"));
01857     glGetUniformuiv = PFNGLGETUNIFORMUIVPROC (glfwGetProcAddress("glGetUniformuiv"));

```

```

01858     glGetVertexArrayIndexed64iv =
PFNGLGETVERTEXARRAYINDEXED64IVPROC (glfwGetProcAddress("glGetVertexArrayIndexed64iv"));
01859     glGetVertexArrayIndexediv =
PFNGLGETVERTEXARRAYINDEXEDIVPROC (glfwGetProcAddress("glGetVertexArrayIndexediv"));
01860     glGetVertexArrayIntegerivEXT =
PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC (glfwGetProcAddress("glGetVertexArrayIntegeri_vEXT"));
01861     glGetVertexArrayIntegerivEXT =
PFNGLGETVERTEXARRAYINTEGERVEXTPROC (glfwGetProcAddress("glGetVertexArrayInteger_vEXT"));
01862     glGetVertexArrayPointerivEXT =
PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC (glfwGetProcAddress("glGetVertexArrayPointeri_vEXT"));
01863     glGetVertexArrayPointervEXT =
PFNGLGETVERTEXARRAYPOINTERVEXTPROC (glfwGetProcAddress("glGetVertexArrayPointer_vEXT"));
01864     glGetVertexArrayiv = PFNGLGETVERTEXARRAYIVPROC (glfwGetProcAddress("glGetVertexArrayiv"));
01865     glGetVertexAttribIiv = PFNGLGETVERTEXATTRIBIIVPROC (glfwGetProcAddress("glGetVertexAttribIiv"));
01866     glGetVertexAttribIuiv = PFNGLGETVERTEXATTRIBUIVPROC (glfwGetProcAddress("glGetVertexAttribIuiv"));
01867     glGetVertexAttribLdv = PFNGLGETVERTEXATTRIBLDVPROC (glfwGetProcAddress("glGetVertexAttribLdv"));
01868     glGetVertexAttribLi64vNV =
PFNGLGETVERTEXATTRIBLI64VNVPROC (glfwGetProcAddress("glGetVertexAttribLi64vNV"));
01869     glGetVertexAttribLui64vARB =
PFNGLGETVERTEXATTRIBLUI64VARBPROC (glfwGetProcAddress("glGetVertexAttribLui64vARB"));
01870     glGetVertexAttribLui64vNV =
PFNGLGETVERTEXATTRIBLUI64VNVPROC (glfwGetProcAddress("glGetVertexAttribLui64vNV"));
01871     glGetVertexAttribPointerv =
PFNGLGETVERTEXATTRIBPOINTERVPROC (glfwGetProcAddress("glGetVertexAttribPointer_v"));
01872     glGetVertexAttribdv = PFNGLGETVERTEXATTRIBDVPROC (glfwGetProcAddress("glGetVertexAttribdv"));
01873     glGetVertexAttribfv = PFNGLGETVERTEXATTRIBFVPROC (glfwGetProcAddress("glGetVertexAttribfv"));
01874     glGetVertexAttribiv = PFNGLGETVERTEXATTRIBIVPROC (glfwGetProcAddress("glGetVertexAttribiv"));
01875     glGetVkProcAddrNV = PFNGLGETVKPROCADDRNVPROC (glfwGetProcAddress("glGetVkProcAddrNV"));
01876     glGetnCompressedTexImage =
PFNGLGETNCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress("glGetnCompressedTexImage"));
01877     glGetnCompressedTexImageARB =
PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC (glfwGetProcAddress("glGetnCompressedTexImageARB"));
01878     glGetnTexImage = PFNGLGETNTEXIMAGEPROC (glfwGetProcAddress("glGetnTexImage"));
01879     glGetnTexImageARB = PFNGLGETNTEXIMAGEARBPROC (glfwGetProcAddress("glGetnTexImageARB"));
01880     glGetnUniformdv = PFNGLGETNUNIFORMDVPROC (glfwGetProcAddress("glGetnUniformdv"));
01881     glGetnUniformdvarb = PFNGLGETNUNIFORMDVARBPROC (glfwGetProcAddress("glGetnUniformdvarb"));
01882     glGetnUniformfv = PFNGLGETNUNIFORMFVPROC (glfwGetProcAddress("glGetnUniformfv"));
01883     glGetnUniformfvarb = PFNGLGETNUNIFORMFVARBPROC (glfwGetProcAddress("glGetnUniformfvarb"));
01884     glGetnUniformi64varb = PFNGLGETNUNIFORMI64VARBPROC (glfwGetProcAddress("glGetnUniformi64varb"));
01885     glGetnUniformiv = PFNGLGETNUNIFORMIVPROC (glfwGetProcAddress("glGetnUniformiv"));
01886     glGetnUniformivarb = PFNGLGETNUNIFORMIVARBPROC (glfwGetProcAddress("glGetnUniformivarb"));
01887     glGetnUniformui64varb = PFNGLGETNUNIFORMUI64VARBPROC (glfwGetProcAddress("glGetnUniformui64varb"));
01888     glGetnUniformuiv = PFNGLGETNUNIFORMUIVPROC (glfwGetProcAddress("glGetnUniformuiv"));
01889     glGetnUniformuivarb = PFNGLGETNUNIFORMUIVARBPROC (glfwGetProcAddress("glGetnUniformuivarb"));
01890     glHint = PFNGLHINTPROC (glfwGetProcAddress("glHint"));
01891     glIndexFormatNV = PFNGLINDEXFORMATNVPROC (glfwGetProcAddress("glIndexFormatNV"));
01892     glInsertEventMarkerEXT =
PFNGLINSERTEVENTMARKEREXTPROC (glfwGetProcAddress("glInsertEventMarkerEXT"));
01893     glInterpolatePathsNV = PFNGLINTERPOLATEPATHSNVPROC (glfwGetProcAddress("glInterpolatePathsNV"));
01894     glInvalidateBufferData =
PFNGLINVALIDATEBUFFERDATAPROC (glfwGetProcAddress("glInvalidateBufferData"));
01895     glInvalidateBufferSubData =
PFNGLINVALIDATEBUFFERSUBDATAPROC (glfwGetProcAddress("glInvalidateBufferSubData"));
01896     glInvalidateFramebuffer =
PFNGLINVALIDATEFRAMEBUFFERPROC (glfwGetProcAddress("glInvalidateFramebuffer"));
01897     glInvalidateNamedFramebufferData =
PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC (glfwGetProcAddress("glInvalidateNamedFramebufferData"));
01898     glInvalidateNamedFramebufferSubData =
PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC (glfwGetProcAddress("glInvalidateNamedFramebufferSubData"));
01899     glInvalidateSubFramebuffer =
PFNGLINVALIDATESUBFRAMEBUFFERPROC (glfwGetProcAddress("glInvalidateSubFramebuffer"));
01900     glInvalidateTexImage = PFNGLINVALIDATETEXIMAGEPROC (glfwGetProcAddress("glInvalidateTexImage"));
01901     glInvalidateTexSubImage =
PFNGLINVALIDATETEXSUBIMAGEPROC (glfwGetProcAddress("glInvalidateTexSubImage"));
01902     glIsBuffer = PFNGLISBUFFERPROC (glfwGetProcAddress("glIsBuffer"));
01903     glIsBufferResidentNV = PFNGLISBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsBufferResidentNV"));
01904     glIsCommandListNV = PFNGLISCOMMANDLISTNVPROC (glfwGetProcAddress("glIsCommandListNV"));
01905     glIsEnabled = PFNGLISENABLEDPROC (glfwGetProcAddress("glIsEnabled"));
01906     glIsEnabledIndexedEXT = PFNGLISENABLEDINDEXEDEXTPROC (glfwGetProcAddress("glIsEnabledIndexedEXT"));
01907     glIsEnabledi = PFNGLISENABLEDIPROC (glfwGetProcAddress("glIsEnabledi"));
01908     glIsFramebuffer = PFNGLISFRAMEBUFFERPROC (glfwGetProcAddress("glIsFramebuffer"));
01909     glIsImageHandleResidentARB =
PFNGLISIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsImageHandleResidentARB"));
01910     glIsImageHandleResidentNV =
PFNGLISIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsImageHandleResidentNV"));
01911     glIsNamedBufferResidentNV =
PFNGLISNAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsNamedBufferResidentNV"));
01912     glIsNamedStringARB = PFNGLISNAMEDSTRINGARBPROC (glfwGetProcAddress("glIsNamedStringARB"));
01913     glIsPathNV = PFNGLISPATHNVPROC (glfwGetProcAddress("glIsPathNV"));
01914     glIsPointInFillPathNV = PFNGLISPOINTINFILLPATHNVPROC (glfwGetProcAddress("glIsPointInFillPathNV"));
01915     glIsPointInStrokePathNV =
PFNGLISPOINTINSTROKEPATHNVPROC (glfwGetProcAddress("glIsPointInStrokePathNV"));
01916     glIsProgram = PFNGLISPROGRAMPROC (glfwGetProcAddress("glIsProgram"));
01917     glIsProgramPipeline = PFNGLISPROGRAMPIPELINEPROC (glfwGetProcAddress("glIsProgramPipeline"));
01918     glIsQuery = PFNGLISQUERYPROC (glfwGetProcAddress("glIsQuery"));
01919     glIsRenderbuffer = PFNGLISRENDERBUFFERPROC (glfwGetProcAddress("glIsRenderbuffer"));
01920     glIsSampler = PFNGLISSAMPLERPROC (glfwGetProcAddress("glIsSampler"));

```

```

01921     glIsShader = PFNGLISSHADERPROC (glfwGetProcAddress ("glIsShader"));
01922     glIsStateNV = PFNGLISSTATENVPROC (glfwGetProcAddress ("glIsStateNV"));
01923     glIsSync = PFNGLISSYNCPROC (glfwGetProcAddress ("glIsSync"));
01924     glIsTexture = PFNGLISTEXTUREPROC (glfwGetProcAddress ("glIsTexture"));
01925     glIsTextureHandleResidentARB =
PFNGLISTEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress ("glIsTextureHandleResidentARB"));
01926     glIsTextureHandleResidentNV =
PFNGLISTEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress ("glIsTextureHandleResidentNV"));
01927     glIsTransformFeedback = PFNGLISTRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glIsTransformFeedback"));
01928     glIsVertexArray = PFNGLISVERTEXARRAYPROC (glfwGetProcAddress ("glIsVertexArray"));
01929     glLabelObjectEXT = PFNGLLABELOBJECTTEXTPROC (glfwGetProcAddress ("glLabelObjectEXT"));
01930     glLineWidth = PFNGLLINEWIDTHPROC (glfwGetProcAddress ("glLineWidth"));
01931     glLinkProgram = PFNGLLINKPROGRAMPROC (glfwGetProcAddress ("glLinkProgram"));
01932     glListDrawCommandsStatesClientNV =
PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC (glfwGetProcAddress ("glListDrawCommandsStatesClientNV"));
01933     glLogicOp = PFNGLLOGICOPPROC (glfwGetProcAddress ("glLogicOp"));
01934     glMakeBufferNonResidentNV =
PFNGLMAKEBUFFERNONRESIDENTNVPROC (glfwGetProcAddress ("glMakeBufferNonResidentNV"));
01935     glMakeBufferResidentNV =
PFNGLMAKEBUFFERRESIDENTNVPROC (glfwGetProcAddress ("glMakeBufferResidentNV"));
01936     glMakeImageHandleNonResidentARB =
PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC (glfwGetProcAddress ("glMakeImageHandleNonResidentARB"));
01937     glMakeImageHandleNonResidentNV =
PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC (glfwGetProcAddress ("glMakeImageHandleNonResidentNV"));
01938     glMakeImageHandleResidentARB =
PFNGLMAKEIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress ("glMakeImageHandleResidentARB"));
01939     glMakeImageHandleResidentNV =
PFNGLMAKEIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress ("glMakeImageHandleResidentNV"));
01940     glMakeNamedBufferNonResidentNV =
PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC (glfwGetProcAddress ("glMakeNamedBufferNonResidentNV"));
01941     glMakeNamedBufferResidentNV =
PFNGLMAKENAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress ("glMakeNamedBufferResidentNV"));
01942     glMakeTextureHandleNonResidentARB =
PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC (glfwGetProcAddress ("glMakeTextureHandleNonResidentARB"));
01943     glMakeTextureHandleNonResidentNV =
PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC (glfwGetProcAddress ("glMakeTextureHandleNonResidentNV"));
01944     glMakeTextureHandleResidentARB =
PFNGLMAKETEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress ("glMakeTextureHandleResidentARB"));
01945     glMakeTextureHandleResidentNV =
PFNGLMAKETEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress ("glMakeTextureHandleResidentNV"));
01946     glMapBuffer = PFNGLMAPBUFFERPROC (glfwGetProcAddress ("glMapBuffer"));
01947     glMapBufferRange = PFNGLMAPBUFFERRANGEPROC (glfwGetProcAddress ("glMapBufferRange"));
01948     glMapNamedBuffer = PFNGLMAPNAMEDBUFFERPROC (glfwGetProcAddress ("glMapNamedBuffer"));
01949     glMapNamedBufferEXT = PFNGLMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress ("glMapNamedBufferEXT"));
01950     glMapNamedBufferRange = PFNGLMAPNAMEDBUFFERRANGEPROC (glfwGetProcAddress ("glMapNamedBufferRange"));
01951     glMapNamedBufferRangeEXT =
PFNGLMAPNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress ("glMapNamedBufferRangeEXT"));
01952     glMatrixFrustumEXT = PFNGLMATRIXFRUSTUMEXTPROC (glfwGetProcAddress ("glMatrixFrustumEXT"));
01953     glMatrixLoad3x2fNV = PFNGLMATRIXLOAD3X2FNVPROC (glfwGetProcAddress ("glMatrixLoad3x2fNV"));
01954     glMatrixLoad3x3fNV = PFNGLMATRIXLOAD3X3FNVPROC (glfwGetProcAddress ("glMatrixLoad3x3fNV"));
01955     glMatrixLoadIdentityEXT =
PFNGLMATRIXLOADIDENTITYEXTPROC (glfwGetProcAddress ("glMatrixLoadIdentityEXT"));
01956     glMatrixLoadTranspose3x3fNV =
PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC (glfwGetProcAddress ("glMatrixLoadTranspose3x3fNV"));
01957     glMatrixLoadTransposedEXT =
PFNGLMATRIXLOADTRANPOSEDEXTPROC (glfwGetProcAddress ("glMatrixLoadTransposedEXT"));
01958     glMatrixLoadTransposefEXT =
PFNGLMATRIXLOADTRANPOSEFEEXTPROC (glfwGetProcAddress ("glMatrixLoadTransposefEXT"));
01959     glMatrixLoadEXT = PFNGLMATRIXLOADDEXTPROC (glfwGetProcAddress ("glMatrixLoadEXT"));
01960     glMatrixLoadfEXT = PFNGLMATRIXLOADFEEXTPROC (glfwGetProcAddress ("glMatrixLoadfEXT"));
01961     glMatrixMult3x2fNV = PFNGLMATRIXMULT3X2FNVPROC (glfwGetProcAddress ("glMatrixMult3x2fNV"));
01962     glMatrixMult3x3fNV = PFNGLMATRIXMULT3X3FNVPROC (glfwGetProcAddress ("glMatrixMult3x3fNV"));
01963     glMatrixMultTranspose3x3fNV =
PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC (glfwGetProcAddress ("glMatrixMultTranspose3x3fNV"));
01964     glMatrixMultTransposedEXT =
PFNGLMATRIXMULTTRANPOSEDEXTPROC (glfwGetProcAddress ("glMatrixMultTransposedEXT"));
01965     glMatrixMultTransposefEXT =
PFNGLMATRIXMULTTRANPOSEFEEXTPROC (glfwGetProcAddress ("glMatrixMultTransposefEXT"));
01966     glMatrixMultdEXT = PFNGLMATRIXMULTDEXTPROC (glfwGetProcAddress ("glMatrixMultdEXT"));
01967     glMatrixMultfEXT = PFNGLMATRIXMULTFEEXTPROC (glfwGetProcAddress ("glMatrixMultfEXT"));
01968     glMatrixOrthoEXT = PFNGLMATRIXORTHOEXTPROC (glfwGetProcAddress ("glMatrixOrthoEXT"));
01969     glMatrixPopEXT = PFNGLMATRIXPOPEXTPROC (glfwGetProcAddress ("glMatrixPopEXT"));
01970     glMatrixPushEXT = PFNGLMATRIXPUSHEXTPROC (glfwGetProcAddress ("glMatrixPushEXT"));
01971     glMatrixRotatedEXT = PFNGLMATRIXROTATEDEXTPROC (glfwGetProcAddress ("glMatrixRotatedEXT"));
01972     glMatrixRotatefEXT = PFNGLMATRIXROTATEFEEXTPROC (glfwGetProcAddress ("glMatrixRotatefEXT"));
01973     glMatrixScaledEXT = PFNGLMATRIXSCALEDEXTPROC (glfwGetProcAddress ("glMatrixScaledEXT"));
01974     glMatrixScalefEXT = PFNGLMATRIXSCALEFEEXTPROC (glfwGetProcAddress ("glMatrixScalefEXT"));
01975     glMatrixTranslatedEXT = PFNGLMATRIXTRANSLATEDEXTPROC (glfwGetProcAddress ("glMatrixTranslatedEXT"));
01976     glMatrixTranslatefEXT = PFNGLMATRIXTRANSLATEFEEXTPROC (glfwGetProcAddress ("glMatrixTranslatefEXT"));
01977     glMaxShaderCompilerThreadsARB =
PFNGLMAXSHADERCOMPILERTHREADSARBPROC (glfwGetProcAddress ("glMaxShaderCompilerThreadsARB"));
01978     glMemoryBarrier = PFNGLMEMORYBARRIERPROC (glfwGetProcAddress ("glMemoryBarrier"));
01979     glMemoryBarrierByRegion =
PFNGLMEMORYBARRIERBYREGIONPROC (glfwGetProcAddress ("glMemoryBarrierByRegion"));
01980     glMinSampleShading = PFNGLMINSAMPLESHADINGPROC (glfwGetProcAddress ("glMinSampleShading"));
01981     glMinSampleShadingARB = PFNGLMINSAMPLESHADINGARBPROC (glfwGetProcAddress ("glMinSampleShadingARB"));
01982     glMultiDrawArrays = PFNGLMULTIDRAWARRAYSPROC (glfwGetProcAddress ("glMultiDrawArrays"));

```

```
01983     glMultiDrawArraysIndirect =
PFNGLMULTIDRAWARRAYSINDIRECTPROC (glfwGetProcAddress("glMultiDrawArraysIndirect"));
01984     glMultiDrawArraysIndirectBindlessCountNV =
PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress("glMultiDrawArraysIndirectBindlessCountNV"));
01985     glMultiDrawArraysIndirectBindlessNV =
PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC (glfwGetProcAddress("glMultiDrawArraysIndirectBindlessNV"));
01986     glMultiDrawArraysIndirectCountARB =
PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC (glfwGetProcAddress("glMultiDrawArraysIndirectCountARB"));
01987     glMultiDrawElements = PFNGLMULTIDRAWELEMENTSPROC (glfwGetProcAddress("glMultiDrawElements"));
01988     glMultiDrawElementsBaseVertex =
PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glMultiDrawElementsBaseVertex"));
01989     glMultiDrawElementsIndirect =
PFNGLMULTIDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress("glMultiDrawElementsIndirect"));
01990     glMultiDrawElementsIndirectBindlessCountNV =
PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress("glMultiDrawElementsIndirectBindlessCountNV"));
01991     glMultiDrawElementsIndirectBindlessNV =
PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC (glfwGetProcAddress("glMultiDrawElementsIndirectBindlessNV"));
01992     glMultiDrawElementsIndirectCountARB =
PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC (glfwGetProcAddress("glMultiDrawElementsIndirectCountARB"));
01993     glMultiTexBufferEXT = PFNGLMULTITEXBUFFEREXTPROC (glfwGetProcAddress("glMultiTexBufferEXT"));
01994     glMultiTexCoordPointerEXT =
PFNGLMULTITEXCOORDPOINTEREXTPROC (glfwGetProcAddress("glMultiTexCoordPointerEXT"));
01995     glMultiTexEnvfEXT = PFNGLMULTITEXENVFEXTPROC (glfwGetProcAddress("glMultiTexEnvfEXT"));
01996     glMultiTexEnvfvEXT = PFNGLMULTITEXENVFVEXTPROC (glfwGetProcAddress("glMultiTexEnvfvEXT"));
01997     glMultiTexEnviEXT = PFNGLMULTITEXENVIEXTPROC (glfwGetProcAddress("glMultiTexEnviEXT"));
01998     glMultiTexEnvivEXT = PFNGLMULTITEXENVIVEXTPROC (glfwGetProcAddress("glMultiTexEnvivEXT"));
01999     glMultiTexGendEXT = PFNGLMULTITEXGENDEXTPROC (glfwGetProcAddress("glMultiTexGendEXT"));
02000     glMultiTexGendvEXT = PFNGLMULTITEXGENDVEXTPROC (glfwGetProcAddress("glMultiTexGendvEXT"));
02001     glMultiTexGenfEXT = PFNGLMULTITEXGENFEXTPROC (glfwGetProcAddress("glMultiTexGenfEXT"));
02002     glMultiTexGenfvEXT = PFNGLMULTITEXGENFVEXTPROC (glfwGetProcAddress("glMultiTexGenfvEXT"));
02003     glMultiTexGeniEXT = PFNGLMULTITEXGENIEXTPROC (glfwGetProcAddress("glMultiTexGeniEXT"));
02004     glMultiTexGenivEXT = PFNGLMULTITEXGENIVEXTPROC (glfwGetProcAddress("glMultiTexGenivEXT"));
02005     glMultiTexImage1DEXT = PFNGLMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress("glMultiTexImage1DEXT"));
02006     glMultiTexImage2DEXT = PFNGLMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress("glMultiTexImage2DEXT"));
02007     glMultiTexImage3DEXT = PFNGLMULTITEXIMAGE3DEXTPROC (glfwGetProcAddress("glMultiTexImage3DEXT"));
02008     glMultiTexParameterIivEXT =
PFNGLMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress("glMultiTexParameterIivEXT"));
02009     glMultiTexParameterIuivEXT =
PFNGLMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress("glMultiTexParameterIuivEXT"));
02010     glMultiTexParameterfEXT =
PFNGLMULTITEXPARAMETERFEXTPROC (glfwGetProcAddress("glMultiTexParameterfEXT"));
02011     glMultiTexParameterfvEXT =
PFNGLMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress("glMultiTexParameterfvEXT"));
02012     glMultiTexParameteriEXT =
PFNGLMULTITEXPARAMETERIEXTPROC (glfwGetProcAddress("glMultiTexParameteriEXT"));
02013     glMultiTexParameterivEXT =
PFNGLMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress("glMultiTexParameterivEXT"));
02014     glMultiTexRenderbufferEXT =
PFNGLMULTITEXRENDERBUFFEREXTPROC (glfwGetProcAddress("glMultiTexRenderbufferEXT"));
02015     glMultiTexSubImage1DEXT =
PFNGLMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress("glMultiTexSubImage1DEXT"));
02016     glMultiTexSubImage2DEXT =
PFNGLMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress("glMultiTexSubImage2DEXT"));
02017     glMultiTexSubImage3DEXT =
PFNGLMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress("glMultiTexSubImage3DEXT"));
02018     glNamedBufferData = PFNGLNAMEDBUFFERDATAPROC (glfwGetProcAddress("glNamedBufferData"));
02019     glNamedBufferDataEXT = PFNGLNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress("glNamedBufferDataEXT"));
02020     glNamedBufferPageCommitmentARB =
PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress("glNamedBufferPageCommitmentARB"));
02021     glNamedBufferPageCommitmentEXT =
PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC (glfwGetProcAddress("glNamedBufferPageCommitmentEXT"));
02022     glNamedBufferStorage = PFNGLNAMEDBUFFERSTORAGEPROC (glfwGetProcAddress("glNamedBufferStorage"));
02023     glNamedBufferStorageEXT =
PFNGLNAMEDBUFFERSTORAGEEXTPROC (glfwGetProcAddress("glNamedBufferStorageEXT"));
02024     glNamedBufferSubData = PFNGLNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress("glNamedBufferSubData"));
02025     glNamedBufferSubDataEXT =
PFNGLNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress("glNamedBufferSubDataEXT"));
02026     glNamedCopyBufferSubDataEXT =
PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC (glfwGetProcAddress("glNamedCopyBufferSubDataEXT"));
02027     glNamedFramebufferDrawBuffer =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC (glfwGetProcAddress("glNamedFramebufferDrawBuffer"));
02028     glNamedFramebufferDrawBuffers =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC (glfwGetProcAddress("glNamedFramebufferDrawBuffers"));
02029     glNamedFramebufferParameteri =
PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress("glNamedFramebufferParameteri"));
02030     glNamedFramebufferParameteriEXT =
PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC (glfwGetProcAddress("glNamedFramebufferParameteriEXT"));
02031     glNamedFramebufferReadBuffer =
PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC (glfwGetProcAddress("glNamedFramebufferReadBuffer"));
02032     glNamedFramebufferRenderbuffer =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress("glNamedFramebufferRenderbuffer"));
02033     glNamedFramebufferRenderbufferEXT =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC (glfwGetProcAddress("glNamedFramebufferRenderbufferEXT"));
02034     glNamedFramebufferSampleLocationsfvARB =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress("glNamedFramebufferSampleLocationsfvARB"));
02035     glNamedFramebufferSampleLocationsfvNV =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress("glNamedFramebufferSampleLocationsfvNV"));
```



```

02036     glNamedFramebufferTexture =
PFNGLNAMEDFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress ("glNamedFramebufferTexture"));
02037     glNamedFramebufferTexture1DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture1DEXT"));
02038     glNamedFramebufferTexture2DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture2DEXT"));
02039     glNamedFramebufferTexture3DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture3DEXT"));
02040     glNamedFramebufferTextureEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureEXT"));
02041     glNamedFramebufferTextureFaceEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureFaceEXT"));
02042     glNamedFramebufferTextureLayer =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress ("glNamedFramebufferTextureLayer"));
02043     glNamedFramebufferTextureLayerEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureLayerEXT"));
02044     glNamedProgramLocalParameter4dEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4dEXT"));
02045     glNamedProgramLocalParameter4dvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4dvEXT"));
02046     glNamedProgramLocalParameter4fEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4fEXT"));
02047     glNamedProgramLocalParameter4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4fvEXT"));
02048     glNamedProgramLocalParameterI4iEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4iEXT"));
02049     glNamedProgramLocalParameterI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4ivEXT"));
02050     glNamedProgramLocalParameterI4uiEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4uiEXT"));
02051     glNamedProgramLocalParameterI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4uivEXT"));
02052     glNamedProgramLocalParameters4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameters4fvEXT"));
02053     glNamedProgramLocalParametersI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParametersI4ivEXT"));
02054     glNamedProgramLocalParametersI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParametersI4uivEXT"));
02055     glNamedProgramStringEXT =
PFNGLNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress ("glNamedProgramStringEXT"));
02056     glNamedRenderbufferStorage =
PFNGLNAMEDRENDERBUFFERSTORAGEPROC (glfwGetProcAddress ("glNamedRenderbufferStorage"));
02057     glNamedRenderbufferStorageEXT =
PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageEXT"));
02058     glNamedRenderbufferStorageMultisample =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisample"));
02059     glNamedRenderbufferStorageMultisampleCoverageEXT =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLECOVERAGEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisampleCoverageEXT"));
02060     glNamedRenderbufferStorageMultisampleEXT =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisampleEXT"));
02061     glNamedStringARB = PFNGLNAMEDSTRINGARBPROC (glfwGetProcAddress ("glNamedStringARB"));
02062     glNormalFormatNV = PFNGLNORMALFORMATNVPROC (glfwGetProcAddress ("glNormalFormatNV"));
02063     glObjectLabel = PFNGLOBJECTLABELPROC (glfwGetProcAddress ("glObjectLabel"));
02064     glObjectPtrLabel = PFNGLOBJECTPTRLABELPROC (glfwGetProcAddress ("glObjectPtrLabel"));
02065     glPatchParameterfv = PFNGLPATCHPARAMETERFVPROC (glfwGetProcAddress ("glPatchParameterfv"));
02066     glPatchParameteri = PFNGLPATCHPARAMETERIPROC (glfwGetProcAddress ("glPatchParameteri"));
02067     glPathCommandsNV = PFNGLPATHCOMMANDSNVPROC (glfwGetProcAddress ("glPathCommandsNV"));
02068     glPathCoordsNV = PFNGLPATHCOORDSNVPROC (glfwGetProcAddress ("glPathCoordsNV"));
02069     glPathCoverDepthFuncNV =
PFNGLPATHCOVERDEPTHFUNCNVPROC (glfwGetProcAddress ("glPathCoverDepthFuncNV"));
02070     glPathDashArrayNV = PFNGLPATHDASHARRAYNVPROC (glfwGetProcAddress ("glPathDashArrayNV"));
02071     glPathGlyphIndexArrayNV =
PFNGLPATHGLYPHINDEXARRAYNVPROC (glfwGetProcAddress ("glPathGlyphIndexArrayNV"));
02072     glPathGlyphIndexRangeNV =
PFNGLPATHGLYPHINDEXRANGENVPROC (glfwGetProcAddress ("glPathGlyphIndexRangeNV"));
02073     glPathGlyphRangeNV = PFNGLPATHGLYPHRANGENVPROC (glfwGetProcAddress ("glPathGlyphRangeNV"));
02074     glPathGlyphsNV = PFNGLPATHGLYPHSNVPROC (glfwGetProcAddress ("glPathGlyphsNV"));
02075     glPathMemoryGlyphIndexArrayNV =
PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC (glfwGetProcAddress ("glPathMemoryGlyphIndexArrayNV"));
02076     glPathParameterfNV = PFNGLPATHPARAMETERFNVPROC (glfwGetProcAddress ("glPathParameterfNV"));
02077     glPathParameterfvNV = PFNGLPATHPARAMETERFVNVPROC (glfwGetProcAddress ("glPathParameterfvNV"));
02078     glPathParameteriNV = PFNGLPATHPARAMETERINVPROC (glfwGetProcAddress ("glPathParameteriNV"));
02079     glPathParameterivNV = PFNGLPATHPARAMETERIVNVPROC (glfwGetProcAddress ("glPathParameterivNV"));
02080     glPathStencilDepthOffsetNV =
PFNGLPATHSTENCILDEPTHOFFSETNVPROC (glfwGetProcAddress ("glPathStencilDepthOffsetNV"));
02081     glPathStencilFuncNV = PFNGLPATHSTENCILFUNCNVPROC (glfwGetProcAddress ("glPathStencilFuncNV"));
02082     glPathStringNV = PFNGLPATHSTRINGNVPROC (glfwGetProcAddress ("glPathStringNV"));
02083     glPathSubCommandsNV = PFNGLPATHSUBCOMMANDSNVPROC (glfwGetProcAddress ("glPathSubCommandsNV"));
02084     glPathSubCoordsNV = PFNGLPATHSUBCOORDSNVPROC (glfwGetProcAddress ("glPathSubCoordsNV"));
02085     glPauseTransformFeedback =
PFNGLPAUSETRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glPauseTransformFeedback"));
02086     glPixelStoref = PFNGLPIXELSTOREFPROC (glfwGetProcAddress ("glPixelStoref"));
02087     glPixelStorei = PFNGLPIXELSTOREIPROC (glfwGetProcAddress ("glPixelStorei"));
02088     glPointAlongPathNV = PFNGLPOINTALONGPATHNVPROC (glfwGetProcAddress ("glPointAlongPathNV"));
02089     glPointParameterf = PFNGLPOINTPARAMETERFPROC (glfwGetProcAddress ("glPointParameterf"));
02090     glPointParameterfv = PFNGLPOINTPARAMETERFVPROC (glfwGetProcAddress ("glPointParameterfv"));
02091     glPointParameteri = PFNGLPOINTPARAMETERIPROC (glfwGetProcAddress ("glPointParameteri"));

```

```

02092     glPointParameteriv = PFNGLPOINTPARAMETERIVPROC (glfwGetProcAddress("glPointParameteriv"));
02093     glPointSize = PFNGLPOINTSIZEPROC (glfwGetProcAddress("glPointSize"));
02094     glPolygonMode = PFNGLPOLYGONMODEPROC (glfwGetProcAddress("glPolygonMode"));
02095     glPolygonOffset = PFNGLPOLYGONOFFSETPROC (glfwGetProcAddress("glPolygonOffset"));
02096     glPolygonOffsetClampEXT =
PFNGLPOLYGONOFFSETCLAMPEXTPROC (glfwGetProcAddress("glPolygonOffsetClampEXT"));
02097     glPopDebugGroup = PFNGLPOPDEBUGGROUPPROC (glfwGetProcAddress("glPopDebugGroup"));
02098     glPopGroupMarkerEXT = PFNGLPOPGROUPMARKEREXTPROC (glfwGetProcAddress("glPopGroupMarkerEXT"));
02099     glPrimitiveBoundingBoxARB =
PFNGLPRIMITIVEBOUNDINGBOXARBPROC (glfwGetProcAddress("glPrimitiveBoundingBoxARB"));
02100     glPrimitiveRestartIndex =
PFNGLPRIMITIVERESTARTINDEXPROC (glfwGetProcAddress("glPrimitiveRestartIndex"));
02101     glProgramBinary = PFNGLPROGRAMBINARYPROC (glfwGetProcAddress("glProgramBinary"));
02102     glProgramParameteri = PFNGLPROGRAMPARAMETERIPROC (glfwGetProcAddress("glProgramParameteri"));
02103     glProgramParameteriARB =
PFNGLPROGRAMPARAMETERIARBPROC (glfwGetProcAddress("glProgramParameteriARB"));
02104     glProgramPathFragmentInputGenNV =
PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC (glfwGetProcAddress("glProgramPathFragmentInputGenNV"));
02105     glProgramUniformld = PFNGLPROGRAMUNIFORMLDPROC (glfwGetProcAddress("glProgramUniformld"));
02106     glProgramUniformldEXT = PFNGLPROGRAMUNIFORMLDEXTPROC (glfwGetProcAddress("glProgramUniformldEXT"));
02107     glProgramUniformldv = PFNGLPROGRAMUNIFORMLDVPROC (glfwGetProcAddress("glProgramUniformldv"));
02108     glProgramUniformldvEXT =
PFNGLPROGRAMUNIFORMLDVEXTPROC (glfwGetProcAddress("glProgramUniformldvEXT"));
02109     glProgramUniformlf = PFNGLPROGRAMUNIFORMLFPROC (glfwGetProcAddress("glProgramUniformlf"));
02110     glProgramUniformlfEXT = PFNGLPROGRAMUNIFORMLFEXTPROC (glfwGetProcAddress("glProgramUniformlfEXT"));
02111     glProgramUniformlfv = PFNGLPROGRAMUNIFORMLFFVPROC (glfwGetProcAddress("glProgramUniformlfv"));
02112     glProgramUniformlfvEXT =
PFNGLPROGRAMUNIFORMLFFVEXTPROC (glfwGetProcAddress("glProgramUniformlfvEXT"));
02113     glProgramUniformli = PFNGLPROGRAMUNIFORMLIIPROC (glfwGetProcAddress("glProgramUniformli"));
02114     glProgramUniformli64ARB =
PFNGLPROGRAMUNIFORMLI64ARBPROC (glfwGetProcAddress("glProgramUniformli64ARB"));
02115     glProgramUniformli64NV =
PFNGLPROGRAMUNIFORMLI64NVPROC (glfwGetProcAddress("glProgramUniformli64NV"));
02116     glProgramUniformli64vARB =
PFNGLPROGRAMUNIFORMLI64VARBPROC (glfwGetProcAddress("glProgramUniformli64vARB"));
02117     glProgramUniformli64vNV =
PFNGLPROGRAMUNIFORMLI64VNVPROC (glfwGetProcAddress("glProgramUniformli64vNV"));
02118     glProgramUniformliEXT = PFNGLPROGRAMUNIFORMLIEXTPROC (glfwGetProcAddress("glProgramUniformliEXT"));
02119     glProgramUniformliv = PFNGLPROGRAMUNIFORMLIVPROC (glfwGetProcAddress("glProgramUniformliv"));
02120     glProgramUniformlivEXT =
PFNGLPROGRAMUNIFORMLIVEXTPROC (glfwGetProcAddress("glProgramUniformlivEXT"));
02121     glProgramUniformlui = PFNGLPROGRAMUNIFORMLUIPROC (glfwGetProcAddress("glProgramUniformlui"));
02122     glProgramUniformlui64ARB =
PFNGLPROGRAMUNIFORMLUI64ARBPROC (glfwGetProcAddress("glProgramUniformlui64ARB"));
02123     glProgramUniformlui64NV =
PFNGLPROGRAMUNIFORMLUI64NVPROC (glfwGetProcAddress("glProgramUniformlui64NV"));
02124     glProgramUniformlui64vARB =
PFNGLPROGRAMUNIFORMLUI64VARBPROC (glfwGetProcAddress("glProgramUniformlui64vARB"));
02125     glProgramUniformlui64vNV =
PFNGLPROGRAMUNIFORMLUI64VNVPROC (glfwGetProcAddress("glProgramUniformlui64vNV"));
02126     glProgramUniformluiEXT =
PFNGLPROGRAMUNIFORMLUIEXTPROC (glfwGetProcAddress("glProgramUniformluiEXT"));
02127     glProgramUniformluiv = PFNGLPROGRAMUNIFORMLUIVPROC (glfwGetProcAddress("glProgramUniformluiv"));
02128     glProgramUniformluivEXT =
PFNGLPROGRAMUNIFORMLUIVEXTPROC (glfwGetProcAddress("glProgramUniformluivEXT"));
02129     glProgramUniform2d = PFNGLPROGRAMUNIFORM2DPROC (glfwGetProcAddress("glProgramUniform2d"));
02130     glProgramUniform2dEXT = PFNGLPROGRAMUNIFORM2DEXTPROC (glfwGetProcAddress("glProgramUniform2dEXT"));
02131     glProgramUniform2dv = PFNGLPROGRAMUNIFORM2DVPROC (glfwGetProcAddress("glProgramUniform2dv"));
02132     glProgramUniform2dvEXT =
PFNGLPROGRAMUNIFORM2DVEXTPROC (glfwGetProcAddress("glProgramUniform2dvEXT"));
02133     glProgramUniform2f = PFNGLPROGRAMUNIFORM2FPROC (glfwGetProcAddress("glProgramUniform2f"));
02134     glProgramUniform2fEXT = PFNGLPROGRAMUNIFORM2FEXTPROC (glfwGetProcAddress("glProgramUniform2fEXT"));
02135     glProgramUniform2fv = PFNGLPROGRAMUNIFORM2FVPROC (glfwGetProcAddress("glProgramUniform2fv"));
02136     glProgramUniform2fvEXT =
PFNGLPROGRAMUNIFORM2FVEXTPROC (glfwGetProcAddress("glProgramUniform2fvEXT"));
02137     glProgramUniform2i = PFNGLPROGRAMUNIFORM2IPROC (glfwGetProcAddress("glProgramUniform2i"));
02138     glProgramUniform2i64ARB =
PFNGLPROGRAMUNIFORM2I64ARBPROC (glfwGetProcAddress("glProgramUniform2i64ARB"));
02139     glProgramUniform2i64NV =
PFNGLPROGRAMUNIFORM2I64NVPROC (glfwGetProcAddress("glProgramUniform2i64NV"));
02140     glProgramUniform2i64vARB =
PFNGLPROGRAMUNIFORM2I64VARBPROC (glfwGetProcAddress("glProgramUniform2i64vARB"));
02141     glProgramUniform2i64vNV =
PFNGLPROGRAMUNIFORM2I64VNVPROC (glfwGetProcAddress("glProgramUniform2i64vNV"));
02142     glProgramUniform2iEXT = PFNGLPROGRAMUNIFORM2IEXTPROC (glfwGetProcAddress("glProgramUniform2iEXT"));
02143     glProgramUniform2iv = PFNGLPROGRAMUNIFORM2IVPROC (glfwGetProcAddress("glProgramUniform2iv"));
02144     glProgramUniform2ivEXT =
PFNGLPROGRAMUNIFORM2IVEXTPROC (glfwGetProcAddress("glProgramUniform2ivEXT"));
02145     glProgramUniform2ui = PFNGLPROGRAMUNIFORM2UIPROC (glfwGetProcAddress("glProgramUniform2ui"));
02146     glProgramUniform2ui64ARB =
PFNGLPROGRAMUNIFORM2UI64ARBPROC (glfwGetProcAddress("glProgramUniform2ui64ARB"));
02147     glProgramUniform2ui64NV =
PFNGLPROGRAMUNIFORM2UI64NVPROC (glfwGetProcAddress("glProgramUniform2ui64NV"));
02148     glProgramUniform2ui64vARB =
PFNGLPROGRAMUNIFORM2UI64VARBPROC (glfwGetProcAddress("glProgramUniform2ui64vARB"));
02149     glProgramUniform2ui64vNV =
PFNGLPROGRAMUNIFORM2UI64VNVPROC (glfwGetProcAddress("glProgramUniform2ui64vNV"));

```

```

02150     glProgramUniform2uiEXT =
PFNGLPROGRAMUNIFORM2UIEXTPROC (glfwGetProcAddress ("glProgramUniform2uiEXT"));
02151     glProgramUniform2uiV = PFNGLPROGRAMUNIFORM2UIVPROC (glfwGetProcAddress ("glProgramUniform2uiV"));
02152     glProgramUniform2uiEXT =
PFNGLPROGRAMUNIFORM2UIEXTPROC (glfwGetProcAddress ("glProgramUniform2uiEXT"));
02153     glProgramUniform3d = PFNGLPROGRAMUNIFORM3DPROC (glfwGetProcAddress ("glProgramUniform3d"));
02154     glProgramUniform3dEXT = PFNGLPROGRAMUNIFORM3DEXTPROC (glfwGetProcAddress ("glProgramUniform3dEXT"));
02155     glProgramUniform3dv = PFNGLPROGRAMUNIFORM3DVPROC (glfwGetProcAddress ("glProgramUniform3dv"));
02156     glProgramUniform3dvEXT =
PFNGLPROGRAMUNIFORM3DVEXTPROC (glfwGetProcAddress ("glProgramUniform3dvEXT"));
02157     glProgramUniform3f = PFNGLPROGRAMUNIFORM3FPROC (glfwGetProcAddress ("glProgramUniform3f"));
02158     glProgramUniform3fEXT = PFNGLPROGRAMUNIFORM3FEXTPROC (glfwGetProcAddress ("glProgramUniform3fEXT"));
02159     glProgramUniform3fv = PFNGLPROGRAMUNIFORM3FVPROC (glfwGetProcAddress ("glProgramUniform3fv"));
02160     glProgramUniform3fvEXT =
PFNGLPROGRAMUNIFORM3FVEXTPROC (glfwGetProcAddress ("glProgramUniform3fvEXT"));
02161     glProgramUniform3i = PFNGLPROGRAMUNIFORM3IPROC (glfwGetProcAddress ("glProgramUniform3i"));
02162     glProgramUniform3i4ARB =
PFNGLPROGRAMUNIFORM3I4ARBPROC (glfwGetProcAddress ("glProgramUniform3i4ARB"));
02163     glProgramUniform3i4NV =
PFNGLPROGRAMUNIFORM3I4NVPROC (glfwGetProcAddress ("glProgramUniform3i4NV"));
02164     glProgramUniform3i4vARB =
PFNGLPROGRAMUNIFORM3I4VARBPROC (glfwGetProcAddress ("glProgramUniform3i4vARB"));
02165     glProgramUniform3i4vNV =
PFNGLPROGRAMUNIFORM3I4VNVPROC (glfwGetProcAddress ("glProgramUniform3i4vNV"));
02166     glProgramUniform3iEXT = PFNGLPROGRAMUNIFORM3IEXTPROC (glfwGetProcAddress ("glProgramUniform3iEXT"));
02167     glProgramUniform3iv = PFNGLPROGRAMUNIFORM3IVPROC (glfwGetProcAddress ("glProgramUniform3iv"));
02168     glProgramUniform3ivEXT =
PFNGLPROGRAMUNIFORM3IVEXTPROC (glfwGetProcAddress ("glProgramUniform3ivEXT"));
02169     glProgramUniform3ui = PFNGLPROGRAMUNIFORM3UIPROC (glfwGetProcAddress ("glProgramUniform3ui"));
02170     glProgramUniform3ui4ARB =
PFNGLPROGRAMUNIFORM3UI4ARBPROC (glfwGetProcAddress ("glProgramUniform3ui4ARB"));
02171     glProgramUniform3ui4NV =
PFNGLPROGRAMUNIFORM3UI4NVPROC (glfwGetProcAddress ("glProgramUniform3ui4NV"));
02172     glProgramUniform3ui4vARB =
PFNGLPROGRAMUNIFORM3UI4VARBPROC (glfwGetProcAddress ("glProgramUniform3ui4vARB"));
02173     glProgramUniform3ui4vNV =
PFNGLPROGRAMUNIFORM3UI4VNVPROC (glfwGetProcAddress ("glProgramUniform3ui4vNV"));
02174     glProgramUniform3uiEXT =
PFNGLPROGRAMUNIFORM3UIEXTPROC (glfwGetProcAddress ("glProgramUniform3uiEXT"));
02175     glProgramUniform3uiV = PFNGLPROGRAMUNIFORM3UIVPROC (glfwGetProcAddress ("glProgramUniform3uiV"));
02176     glProgramUniform3uiEXT =
PFNGLPROGRAMUNIFORM3UIEXTPROC (glfwGetProcAddress ("glProgramUniform3uiEXT"));
02177     glProgramUniform4d = PFNGLPROGRAMUNIFORM4DPROC (glfwGetProcAddress ("glProgramUniform4d"));
02178     glProgramUniform4dEXT = PFNGLPROGRAMUNIFORM4DEXTPROC (glfwGetProcAddress ("glProgramUniform4dEXT"));
02179     glProgramUniform4dv = PFNGLPROGRAMUNIFORM4DVPROC (glfwGetProcAddress ("glProgramUniform4dv"));
02180     glProgramUniform4dvEXT =
PFNGLPROGRAMUNIFORM4DVEXTPROC (glfwGetProcAddress ("glProgramUniform4dvEXT"));
02181     glProgramUniform4f = PFNGLPROGRAMUNIFORM4FPROC (glfwGetProcAddress ("glProgramUniform4f"));
02182     glProgramUniform4fEXT = PFNGLPROGRAMUNIFORM4FEXTPROC (glfwGetProcAddress ("glProgramUniform4fEXT"));
02183     glProgramUniform4fv = PFNGLPROGRAMUNIFORM4FVPROC (glfwGetProcAddress ("glProgramUniform4fv"));
02184     glProgramUniform4fvEXT =
PFNGLPROGRAMUNIFORM4FVEXTPROC (glfwGetProcAddress ("glProgramUniform4fvEXT"));
02185     glProgramUniform4i = PFNGLPROGRAMUNIFORM4IPROC (glfwGetProcAddress ("glProgramUniform4i"));
02186     glProgramUniform4i4ARB =
PFNGLPROGRAMUNIFORM4I4ARBPROC (glfwGetProcAddress ("glProgramUniform4i4ARB"));
02187     glProgramUniform4i4NV =
PFNGLPROGRAMUNIFORM4I4NVPROC (glfwGetProcAddress ("glProgramUniform4i4NV"));
02188     glProgramUniform4i4vARB =
PFNGLPROGRAMUNIFORM4I4VARBPROC (glfwGetProcAddress ("glProgramUniform4i4vARB"));
02189     glProgramUniform4i4vNV =
PFNGLPROGRAMUNIFORM4I4VNVPROC (glfwGetProcAddress ("glProgramUniform4i4vNV"));
02190     glProgramUniform4iEXT = PFNGLPROGRAMUNIFORM4IEXTPROC (glfwGetProcAddress ("glProgramUniform4iEXT"));
02191     glProgramUniform4iv = PFNGLPROGRAMUNIFORM4IVPROC (glfwGetProcAddress ("glProgramUniform4iv"));
02192     glProgramUniform4ivEXT =
PFNGLPROGRAMUNIFORM4IVEXTPROC (glfwGetProcAddress ("glProgramUniform4ivEXT"));
02193     glProgramUniform4ui = PFNGLPROGRAMUNIFORM4UIPROC (glfwGetProcAddress ("glProgramUniform4ui"));
02194     glProgramUniform4ui4ARB =
PFNGLPROGRAMUNIFORM4UI4ARBPROC (glfwGetProcAddress ("glProgramUniform4ui4ARB"));
02195     glProgramUniform4ui4NV =
PFNGLPROGRAMUNIFORM4UI4NVPROC (glfwGetProcAddress ("glProgramUniform4ui4NV"));
02196     glProgramUniform4ui4vARB =
PFNGLPROGRAMUNIFORM4UI4VARBPROC (glfwGetProcAddress ("glProgramUniform4ui4vARB"));
02197     glProgramUniform4ui4vNV =
PFNGLPROGRAMUNIFORM4UI4VNVPROC (glfwGetProcAddress ("glProgramUniform4ui4vNV"));
02198     glProgramUniform4uiEXT =
PFNGLPROGRAMUNIFORM4UIEXTPROC (glfwGetProcAddress ("glProgramUniform4uiEXT"));
02199     glProgramUniform4uiV = PFNGLPROGRAMUNIFORM4UIVPROC (glfwGetProcAddress ("glProgramUniform4uiV"));
02200     glProgramUniform4uiEXT =
PFNGLPROGRAMUNIFORM4UIEXTPROC (glfwGetProcAddress ("glProgramUniform4uiEXT"));
02201     glProgramUniformHandleui64ARB =
PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress ("glProgramUniformHandleui64ARB"));
02202     glProgramUniformHandleui64NV =
PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress ("glProgramUniformHandleui64NV"));
02203     glProgramUniformHandleui64vARB =
PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress ("glProgramUniformHandleui64vARB"));
02204     glProgramUniformHandleui64vNV =
PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress ("glProgramUniformHandleui64vNV"));

```

```

02205     glProgramUniformMatrix2dv =
PFNGLPROGRAMUNIFORMMATRIX2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2dv"));
02206     glProgramUniformMatrix2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2dvEXT"));
02207     glProgramUniformMatrix2fv =
PFNGLPROGRAMUNIFORMMATRIX2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2fv"));
02208     glProgramUniformMatrix2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2fvEXT"));
02209     glProgramUniformMatrix2x3dv =
PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3dv"));
02210     glProgramUniformMatrix2x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3dvEXT"));
02211     glProgramUniformMatrix2x3fv =
PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3fv"));
02212     glProgramUniformMatrix2x3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3fvEXT"));
02213     glProgramUniformMatrix2x4dv =
PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4dv"));
02214     glProgramUniformMatrix2x4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4dvEXT"));
02215     glProgramUniformMatrix2x4fv =
PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4fv"));
02216     glProgramUniformMatrix2x4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4fvEXT"));
02217     glProgramUniformMatrix3dv =
PFNGLPROGRAMUNIFORMMATRIX3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3dv"));
02218     glProgramUniformMatrix3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3dvEXT"));
02219     glProgramUniformMatrix3fv =
PFNGLPROGRAMUNIFORMMATRIX3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3fv"));
02220     glProgramUniformMatrix3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3fvEXT"));
02221     glProgramUniformMatrix3x2dv =
PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2dv"));
02222     glProgramUniformMatrix3x2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2dvEXT"));
02223     glProgramUniformMatrix3x2fv =
PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2fv"));
02224     glProgramUniformMatrix3x2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2fvEXT"));
02225     glProgramUniformMatrix3x4dv =
PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4dv"));
02226     glProgramUniformMatrix3x4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4dvEXT"));
02227     glProgramUniformMatrix3x4fv =
PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4fv"));
02228     glProgramUniformMatrix3x4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4fvEXT"));
02229     glProgramUniformMatrix4dv =
PFNGLPROGRAMUNIFORMMATRIX4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4dv"));
02230     glProgramUniformMatrix4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4dvEXT"));
02231     glProgramUniformMatrix4fv =
PFNGLPROGRAMUNIFORMMATRIX4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4fv"));
02232     glProgramUniformMatrix4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4fvEXT"));
02233     glProgramUniformMatrix4x2dv =
PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2dv"));
02234     glProgramUniformMatrix4x2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2dvEXT"));
02235     glProgramUniformMatrix4x2fv =
PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2fv"));
02236     glProgramUniformMatrix4x2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2fvEXT"));
02237     glProgramUniformMatrix4x3dv =
PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dv"));
02238     glProgramUniformMatrix4x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dvEXT"));
02239     glProgramUniformMatrix4x3fv =
PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fv"));
02240     glProgramUniformMatrix4x3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fvEXT"));
02241     glProgramUniformi64NV =
PFNGLPROGRAMUNIFORMUI64NVPROC (glfwGetProcAddress ("glProgramUniformui64NV"));
02242     glProgramUniformui64vNV =
PFNGLPROGRAMUNIFORMUI64VNVPROC (glfwGetProcAddress ("glProgramUniformui64vNV"));
02243     glProvokingVertex = PFNGLPROVOKINGVERTEXPROC (glfwGetProcAddress ("glProvokingVertex"));
02244     glPushClientAttribDefaultEXT =
PFNGLPUSHCLIENTATTRIBDEFAULTEXTPROC (glfwGetProcAddress ("glPushClientAttribDefaultEXT"));
02245     glPushDebugGroup = PFNGLPUSHDEBUGGROUPPROC (glfwGetProcAddress ("glPushDebugGroup"));
02246     glPushDebugMarkerEXT = PFNGLPUSHDEBUGGROUPMARKEREXTPROC (glfwGetProcAddress ("glPushDebugMarkerEXT"));
02247     glQueryCounter = PFNGLQUERYCOUNTERPROC (glfwGetProcAddress ("glQueryCounter"));
02248     glRasterSamplesEXT = PFNGLRASTERSAMPLESEXTPROC (glfwGetProcAddress ("glRasterSamplesEXT"));
02249     glReadBuffer = PFNGLREADBUFFERPROC (glfwGetProcAddress ("glReadBuffer"));
02250     glReadPixels = PFNGLREADPIXELSPROC (glfwGetProcAddress ("glReadPixels"));
02251     glReadnPixels = PFNGLREADNPIXELSPROC (glfwGetProcAddress ("glReadnPixels"));
02252     glReadnPixelsARB = PFNGLREADNPIXELSARBPROC (glfwGetProcAddress ("glReadnPixelsARB"));

```



```
02253     glReleaseShaderCompiler =
PFNGLRELEASESHADERCOMPILERPROC (glfwGetProcAddress("glReleaseShaderCompiler"));
02254     glRenderbufferStorage = PFNGLRENDERBUFFERSTORAGEPROC (glfwGetProcAddress("glRenderbufferStorage"));
02255     glRenderbufferStorageMultisample =
PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC (glfwGetProcAddress("glRenderbufferStorageMultisample"));
02256     glRenderbufferStorageMultisampleCoverageNV =
PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEENVPROC (glfwGetProcAddress("glRenderbufferStorageMultisampleCoverageNV"));
02257     glResolveDepthValuesNV =
PFNGLRESOLVEDEPTHVALUESNVPROC (glfwGetProcAddress("glResolveDepthValuesNV"));
02258     glResumeTransformFeedback =
PFNGLRESUMETRANSFORMFEEDBACKPROC (glfwGetProcAddress("glResumeTransformFeedback"));
02259     glSampleCoverage = PFNGLSAMPLECOVERAGEPROC (glfwGetProcAddress("glSampleCoverage"));
02260     glSampleMaski = PFNGLSAMPLEMASKIPROC (glfwGetProcAddress("glSampleMaski"));
02261     glSamplerParameterIiv = PFNGLSAMPLERPARAMETERIIVPROC (glfwGetProcAddress("glSamplerParameterIiv"));
02262     glSamplerParameterIuiv =
PFNGLSAMPLERPARAMETERIUIVPROC (glfwGetProcAddress("glSamplerParameterIuiv"));
02263     glSamplerParameterf = PFNGLSAMPLERPARAMETERFPROC (glfwGetProcAddress("glSamplerParameterf"));
02264     glSamplerParameterfv = PFNGLSAMPLERPARAMETERFVPROC (glfwGetProcAddress("glSamplerParameterfv"));
02265     glSamplerParameteri = PFNGLSAMPLERPARAMETERIPROC (glfwGetProcAddress("glSamplerParameteri"));
02266     glSamplerParameteriv = PFNGLSAMPLERPARAMETERIVPROC (glfwGetProcAddress("glSamplerParameteriv"));
02267     glScissor = PFNGLSCISSORPROC (glfwGetProcAddress("glScissor"));
02268     glScissorArrayv = PFNGLSCISSORARRAYVPROC (glfwGetProcAddress("glScissorArrayv"));
02269     glScissorIndexed = PFNGLSCISSORINDEXEDPROC (glfwGetProcAddress("glScissorIndexed"));
02270     glScissorIndexedv = PFNGLSCISSORINDEXEDVPROC (glfwGetProcAddress("glScissorIndexedv"));
02271     glSecondaryColorFormatNV =
PFNGLSECONDARYCOLORFORMATNVPROC (glfwGetProcAddress("glSecondaryColorFormatNV"));
02272     glSelectPerfMonitorCountersAMD =
PFNGLSELECTPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress("glSelectPerfMonitorCountersAMD"));
02273     glShaderBinary = PFNGLSHADERBINARYPROC (glfwGetProcAddress("glShaderBinary"));
02274     glShaderSource = PFNGLSHADERSOURCEPROC (glfwGetProcAddress("glShaderSource"));
02275     glShaderStorageBlockBinding =
PFNGLSHADERSTORAGEBLOCKBINDINGPROC (glfwGetProcAddress("glShaderStorageBlockBinding"));
02276     glSignalVkFenceNV = PFNGLSIGNALVKFENCEENVPROC (glfwGetProcAddress("glSignalVkFenceNV"));
02277     glSignalVkSemaphoreNV = PFNGLSIGNALVKSEMAPHORENVPROC (glfwGetProcAddress("glSignalVkSemaphoreNV"));
02278     glSpecializeShaderARB = PFNGLSPECIALIZESHADERARBPROC (glfwGetProcAddress("glSpecializeShaderARB"));
02279     glStateCaptureNV = PFNGLSTATECAPTURENVPROC (glfwGetProcAddress("glStateCaptureNV"));
02280     glStencilFillPathInstancedNV =
PFNGLSTENCILFILLPATHINSTANCEDNVPROC (glfwGetProcAddress("glStencilFillPathInstancedNV"));
02281     glStencilFillPathNV = PFNGLSTENCILFILLPATHNVPROC (glfwGetProcAddress("glStencilFillPathNV"));
02282     glStencilFunc = PFNGLSTENCILFUNCPROC (glfwGetProcAddress("glStencilFunc"));
02283     glStencilFuncSeparate = PFNGLSTENCILFUNCSEPARATEPROC (glfwGetProcAddress("glStencilFuncSeparate"));
02284     glStencilMask = PFNGLSTENCILMASKPROC (glfwGetProcAddress("glStencilMask"));
02285     glStencilMaskSeparate = PFNGLSTENCILMASKSEPARATEPROC (glfwGetProcAddress("glStencilMaskSeparate"));
02286     glStencilOp = PFNGLSTENCILOPPROC (glfwGetProcAddress("glStencilOp"));
02287     glStencilOpSeparate = PFNGLSTENCILOPSEPARATEPROC (glfwGetProcAddress("glStencilOpSeparate"));
02288     glStencilStrokePathInstancedNV =
PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress("glStencilStrokePathInstancedNV"));
02289     glStencilStrokePathNV = PFNGLSTENCILSTROKEPATHNVPROC (glfwGetProcAddress("glStencilStrokePathNV"));
02290     glStencilThenCoverFillPathInstancedNV =
PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress("glStencilThenCoverFillPathInstancedNV"));
02291     glStencilThenCoverFillPathNV =
PFNGLSTENCILTHENCOVERFILLPATHNVPROC (glfwGetProcAddress("glStencilThenCoverFillPathNV"));
02292     glStencilThenCoverStrokePathInstancedNV =
PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress("glStencilThenCoverStrokePathInstancedNV"));
02293     glStencilThenCoverStrokePathNV =
PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC (glfwGetProcAddress("glStencilThenCoverStrokePathNV"));
02294     glSubpixelPrecisionBiasNV =
PFNGLSUBPIXELPRECISIONBIASNVPROC (glfwGetProcAddress("glSubpixelPrecisionBiasNV"));
02295     glTexBuffer = PFNGLTEXBUFFERPROC (glfwGetProcAddress("glTexBuffer"));
02296     glTexBufferARB = PFNGLTEXBUFFERARBPROC (glfwGetProcAddress("glTexBufferARB"));
02297     glTexBufferRange = PFNGLTEXBUFFERRANGEPROC (glfwGetProcAddress("glTexBufferRange"));
02298     glTexCoordFormatNV = PFNGLTEXCOORDFORMATNVPROC (glfwGetProcAddress("glTexCoordFormatNV"));
02299     glTexImage1D = PFNGLTEXIMAGE1DPROC (glfwGetProcAddress("glTexImage1D"));
02300     glTexImage2D = PFNGLTEXIMAGE2DPROC (glfwGetProcAddress("glTexImage2D"));
02301     glTexImage2DMultisample =
PFNGLTEXIMAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexImage2DMultisample"));
02302     glTexImage3D = PFNGLTEXIMAGE3DPROC (glfwGetProcAddress("glTexImage3D"));
02303     glTexImage3DMultisample =
PFNGLTEXIMAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexImage3DMultisample"));
02304     glTexPageCommitmentARB =
PFNGLTEXPAGECOMMITMENTARBPROC (glfwGetProcAddress("glTexPageCommitmentARB"));
02305     glTexParameterIiv = PFNGLTEXPARAMETERIIVPROC (glfwGetProcAddress("glTexParameterIiv"));
02306     glTexParameterIuiv = PFNGLTEXPARAMETERIUIVPROC (glfwGetProcAddress("glTexParameterIuiv"));
02307     glTexParameterf = PFNGLTEXPARAMETERFPROC (glfwGetProcAddress("glTexParameterf"));
02308     glTexParameterfv = PFNGLTEXPARAMETERFVPROC (glfwGetProcAddress("glTexParameterfv"));
02309     glTexParameteri = PFNGLTEXPARAMETERIPROC (glfwGetProcAddress("glTexParameteri"));
02310     glTexParameteriv = PFNGLTEXPARAMETERIVPROC (glfwGetProcAddress("glTexParameteriv"));
02311     glTexStorage1D = PFNGLTEXSTORAGE1DPROC (glfwGetProcAddress("glTexStorage1D"));
02312     glTexStorage2D = PFNGLTEXSTORAGE2DPROC (glfwGetProcAddress("glTexStorage2D"));
02313     glTexStorage2DMultisample =
PFNGLTEXSTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage2DMultisample"));
02314     glTexStorage3D = PFNGLTEXSTORAGE3DPROC (glfwGetProcAddress("glTexStorage3D"));
02315     glTexStorage3DMultisample =
PFNGLTEXSTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage3DMultisample"));
02316     glTexSubImage1D = PFNGLTEXSUBIMAGE1DPROC (glfwGetProcAddress("glTexSubImage1D"));
02317     glTexSubImage2D = PFNGLTEXSUBIMAGE2DPROC (glfwGetProcAddress("glTexSubImage2D"));
02318     glTexSubImage3D = PFNGLTEXSUBIMAGE3DPROC (glfwGetProcAddress("glTexSubImage3D"));
```

```
02319     glTextureBarrier = PFNGLTEXTUREBARRIERPROC (glfwGetProcAddress("glTextureBarrier"));
02320     glTextureBarrierNV = PFNGLTEXTUREBARRIERNVPROC (glfwGetProcAddress("glTextureBarrierNV"));
02321     glTextureBuffer = PFNGLTEXTUREBUFFERPROC (glfwGetProcAddress("glTextureBuffer"));
02322     glTextureBufferEXT = PFNGLTEXTUREBUFFEREXTPROC (glfwGetProcAddress("glTextureBufferEXT"));
02323     glTextureBufferRange = PFNGLTEXTUREBUFFERRANGEPROC (glfwGetProcAddress("glTextureBufferRange"));
02324     glTextureBufferRangeEXT =
PFNGLTEXTUREBUFFERRANGEEXTPROC (glfwGetProcAddress("glTextureBufferRangeEXT"));
02325     glTextureImage1DEXT = PFNGLTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress("glTextureImage1DEXT"));
02326     glTextureImage2DEXT = PFNGLTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress("glTextureImage2DEXT"));
02327     glTextureImage3DEXT = PFNGLTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress("glTextureImage3DEXT"));
02328     glTexturePageCommitmentEXT =
PFNGLTEXTUREPAGECOMMITMENTEXTPROC (glfwGetProcAddress("glTexturePageCommitmentEXT"));
02329     glTextureParameterIiv = PFNGLTEXTUREPARAMETERIIVPROC (glfwGetProcAddress("glTextureParameterIiv"));
02330     glTextureParameterIivEXT =
PFNGLTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress("glTextureParameterIivEXT"));
02331     glTextureParameterIuiv =
PFNGLTEXTUREPARAMETERUIVPROC (glfwGetProcAddress("glTextureParameterIuiv"));
02332     glTextureParameterIuivEXT =
PFNGLTEXTUREPARAMETERUIVEXTPROC (glfwGetProcAddress("glTextureParameterIuivEXT"));
02333     glTextureParameterf = PFNGLTEXTUREPARAMETERFPROC (glfwGetProcAddress("glTextureParameterf"));
02334     glTextureParameterfEXT =
PFNGLTEXTUREPARAMETERFEXTPROC (glfwGetProcAddress("glTextureParameterfEXT"));
02335     glTextureParameterfv = PFNGLTEXTUREPARAMETERFVPROC (glfwGetProcAddress("glTextureParameterfv"));
02336     glTextureParameterfvEXT =
PFNGLTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress("glTextureParameterfvEXT"));
02337     glTextureParameteri = PFNGLTEXTUREPARAMETERIPROC (glfwGetProcAddress("glTextureParameteri"));
02338     glTextureParameteriEXT =
PFNGLTEXTUREPARAMETERIEXTPROC (glfwGetProcAddress("glTextureParameteriEXT"));
02339     glTextureParameteriv = PFNGLTEXTUREPARAMETERIVPROC (glfwGetProcAddress("glTextureParameteriv"));
02340     glTextureParameterivEXT =
PFNGLTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress("glTextureParameterivEXT"));
02341     glTextureRenderbufferEXT =
PFNGLTEXTURERENDERBUFFEREXTPROC (glfwGetProcAddress("glTextureRenderbufferEXT"));
02342     glTextureStorage1D = PFNGLTEXTURESTORAGE1DPROC (glfwGetProcAddress("glTextureStorage1D"));
02343     glTextureStorage1DEXT = PFNGLTEXTURESTORAGE1DEXTPROC (glfwGetProcAddress("glTextureStorage1DEXT"));
02344     glTextureStorage2D = PFNGLTEXTURESTORAGE2DPROC (glfwGetProcAddress("glTextureStorage2D"));
02345     glTextureStorage2DEXT = PFNGLTEXTURESTORAGE2DEXTPROC (glfwGetProcAddress("glTextureStorage2DEXT"));
02346     glTextureStorage2DMultisample =
PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage2DMultisample"));
02347     glTextureStorage2DMultisampleEXT =
PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage2DMultisampleEXT"));
02348     glTextureStorage3D = PFNGLTEXTURESTORAGE3DPROC (glfwGetProcAddress("glTextureStorage3D"));
02349     glTextureStorage3DEXT = PFNGLTEXTURESTORAGE3DEXTPROC (glfwGetProcAddress("glTextureStorage3DEXT"));
02350     glTextureStorage3DMultisample =
PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage3DMultisample"));
02351     glTextureStorage3DMultisampleEXT =
PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage3DMultisampleEXT"));
02352     glTextureSubImage1D = PFNGLTEXTURESUBIMAGE1DPROC (glfwGetProcAddress("glTextureSubImage1D"));
02353     glTextureSubImage1DEXT =
PFNGLTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress("glTextureSubImage1DEXT"));
02354     glTextureSubImage2D = PFNGLTEXTURESUBIMAGE2DPROC (glfwGetProcAddress("glTextureSubImage2D"));
02355     glTextureSubImage2DEXT =
PFNGLTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress("glTextureSubImage2DEXT"));
02356     glTextureSubImage3D = PFNGLTEXTURESUBIMAGE3DPROC (glfwGetProcAddress("glTextureSubImage3D"));
02357     glTextureSubImage3DEXT =
PFNGLTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress("glTextureSubImage3DEXT"));
02358     glTextureView = PFNGLTEXTUREVIEWPROC (glfwGetProcAddress("glTextureView"));
02359     glTransformFeedbackBufferBase =
PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC (glfwGetProcAddress("glTransformFeedbackBufferBase"));
02360     glTransformFeedbackBufferRange =
PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC (glfwGetProcAddress("glTransformFeedbackBufferRange"));
02361     glTransformFeedbackVaryings =
PFNGLTRANSFORMFEEDBACKVARYINGSPROC (glfwGetProcAddress("glTransformFeedbackVaryings"));
02362     glTransformPathNV = PFNGLTRANSFORMPATHNVPROC (glfwGetProcAddress("glTransformPathNV"));
02363     glUniform1d = PFNGLUNIFORM1DPROC (glfwGetProcAddress("glUniform1d"));
02364     glUniform1dv = PFNGLUNIFORM1DVPROC (glfwGetProcAddress("glUniform1dv"));
02365     glUniform1f = PFNGLUNIFORM1FPROC (glfwGetProcAddress("glUniform1f"));
02366     glUniform1fv = PFNGLUNIFORM1FVPROC (glfwGetProcAddress("glUniform1fv"));
02367     glUniform1i = PFNGLUNIFORM1IPROC (glfwGetProcAddress("glUniform1i"));
02368     glUniform1i64ARB = PFNGLUNIFORM1I64ARBPROC (glfwGetProcAddress("glUniform1i64ARB"));
02369     glUniform1i64NV = PFNGLUNIFORM1I64NVPROC (glfwGetProcAddress("glUniform1i64NV"));
02370     glUniform1i64vARB = PFNGLUNIFORM1I64VARBPROC (glfwGetProcAddress("glUniform1i64vARB"));
02371     glUniform1i64vNV = PFNGLUNIFORM1I64VNVPROC (glfwGetProcAddress("glUniform1i64vNV"));
02372     glUniform1iv = PFNGLUNIFORM1IVPROC (glfwGetProcAddress("glUniform1iv"));
02373     glUniform1ui = PFNGLUNIFORM1UIPROC (glfwGetProcAddress("glUniform1ui"));
02374     glUniform1ui64ARB = PFNGLUNIFORM1UI64ARBPROC (glfwGetProcAddress("glUniform1ui64ARB"));
02375     glUniform1ui64NV = PFNGLUNIFORM1UI64NVPROC (glfwGetProcAddress("glUniform1ui64NV"));
02376     glUniform1ui64vARB = PFNGLUNIFORM1UI64VARBPROC (glfwGetProcAddress("glUniform1ui64vARB"));
02377     glUniform1ui64vNV = PFNGLUNIFORM1UI64VNVPROC (glfwGetProcAddress("glUniform1ui64vNV"));
02378     glUniform1uiv = PFNGLUNIFORM1UIVPROC (glfwGetProcAddress("glUniform1uiv"));
02379     glUniform2d = PFNGLUNIFORM2DPROC (glfwGetProcAddress("glUniform2d"));
02380     glUniform2dv = PFNGLUNIFORM2DVPROC (glfwGetProcAddress("glUniform2dv"));
02381     glUniform2f = PFNGLUNIFORM2FPROC (glfwGetProcAddress("glUniform2f"));
02382     glUniform2fv = PFNGLUNIFORM2FVPROC (glfwGetProcAddress("glUniform2fv"));
02383     glUniform2i = PFNGLUNIFORM2IPROC (glfwGetProcAddress("glUniform2i"));
02384     glUniform2i64ARB = PFNGLUNIFORM2I64ARBPROC (glfwGetProcAddress("glUniform2i64ARB"));
02385     glUniform2i64NV = PFNGLUNIFORM2I64NVPROC (glfwGetProcAddress("glUniform2i64NV"));
```

```

02386   glUniform2i64vARB = PFNGLUNIFORM2I64VARBPROC (glfwGetProcAddress("glUniform2i64vARB"));
02387   glUniform2i64vNV = PFNGLUNIFORM2I64VNVPROC (glfwGetProcAddress("glUniform2i64vNV"));
02388   glUniform2iv = PFNGLUNIFORM2IVPROC (glfwGetProcAddress("glUniform2iv"));
02389   glUniform2ui = PFNGLUNIFORM2UIPROC (glfwGetProcAddress("glUniform2ui"));
02390   glUniform2ui64ARB = PFNGLUNIFORM2UI64ARBPROC (glfwGetProcAddress("glUniform2ui64ARB"));
02391   glUniform2ui64NV = PFNGLUNIFORM2UI64NVPROC (glfwGetProcAddress("glUniform2ui64NV"));
02392   glUniform2ui64vARB = PFNGLUNIFORM2UI64VARBPROC (glfwGetProcAddress("glUniform2ui64vARB"));
02393   glUniform2ui64vNV = PFNGLUNIFORM2UI64VNVPROC (glfwGetProcAddress("glUniform2ui64vNV"));
02394   glUniform2uiv = PFNGLUNIFORM2UIVPROC (glfwGetProcAddress("glUniform2uiv"));
02395   glUniform3d = PFNGLUNIFORM3DPROC (glfwGetProcAddress("glUniform3d"));
02396   glUniform3dv = PFNGLUNIFORM3DVPROC (glfwGetProcAddress("glUniform3dv"));
02397   glUniform3f = PFNGLUNIFORM3FPROC (glfwGetProcAddress("glUniform3f"));
02398   glUniform3fv = PFNGLUNIFORM3FVPROC (glfwGetProcAddress("glUniform3fv"));
02399   glUniform3i = PFNGLUNIFORM3IPROC (glfwGetProcAddress("glUniform3i"));
02400   glUniform3i64ARB = PFNGLUNIFORM3I64ARBPROC (glfwGetProcAddress("glUniform3i64ARB"));
02401   glUniform3i64NV = PFNGLUNIFORM3I64NVPROC (glfwGetProcAddress("glUniform3i64NV"));
02402   glUniform3i64vARB = PFNGLUNIFORM3I64VARBPROC (glfwGetProcAddress("glUniform3i64vARB"));
02403   glUniform3i64vNV = PFNGLUNIFORM3I64VNVPROC (glfwGetProcAddress("glUniform3i64vNV"));
02404   glUniform3iv = PFNGLUNIFORM3IVPROC (glfwGetProcAddress("glUniform3iv"));
02405   glUniform3ui = PFNGLUNIFORM3UIPROC (glfwGetProcAddress("glUniform3ui"));
02406   glUniform3ui64ARB = PFNGLUNIFORM3UI64ARBPROC (glfwGetProcAddress("glUniform3ui64ARB"));
02407   glUniform3ui64NV = PFNGLUNIFORM3UI64NVPROC (glfwGetProcAddress("glUniform3ui64NV"));
02408   glUniform3ui64vARB = PFNGLUNIFORM3UI64VARBPROC (glfwGetProcAddress("glUniform3ui64vARB"));
02409   glUniform3ui64vNV = PFNGLUNIFORM3UI64VNVPROC (glfwGetProcAddress("glUniform3ui64vNV"));
02410   glUniform3uiv = PFNGLUNIFORM3UIVPROC (glfwGetProcAddress("glUniform3uiv"));
02411   glUniform4d = PFNGLUNIFORM4DPROC (glfwGetProcAddress("glUniform4d"));
02412   glUniform4dv = PFNGLUNIFORM4DVPROC (glfwGetProcAddress("glUniform4dv"));
02413   glUniform4f = PFNGLUNIFORM4FPROC (glfwGetProcAddress("glUniform4f"));
02414   glUniform4fv = PFNGLUNIFORM4FVPROC (glfwGetProcAddress("glUniform4fv"));
02415   glUniform4i = PFNGLUNIFORM4IPROC (glfwGetProcAddress("glUniform4i"));
02416   glUniform4i64ARB = PFNGLUNIFORM4I64ARBPROC (glfwGetProcAddress("glUniform4i64ARB"));
02417   glUniform4i64NV = PFNGLUNIFORM4I64NVPROC (glfwGetProcAddress("glUniform4i64NV"));
02418   glUniform4i64vARB = PFNGLUNIFORM4I64VARBPROC (glfwGetProcAddress("glUniform4i64vARB"));
02419   glUniform4i64vNV = PFNGLUNIFORM4I64VNVPROC (glfwGetProcAddress("glUniform4i64vNV"));
02420   glUniform4iv = PFNGLUNIFORM4IVPROC (glfwGetProcAddress("glUniform4iv"));
02421   glUniform4ui = PFNGLUNIFORM4UIPROC (glfwGetProcAddress("glUniform4ui"));
02422   glUniform4ui64ARB = PFNGLUNIFORM4UI64ARBPROC (glfwGetProcAddress("glUniform4ui64ARB"));
02423   glUniform4ui64NV = PFNGLUNIFORM4UI64NVPROC (glfwGetProcAddress("glUniform4ui64NV"));
02424   glUniform4ui64vARB = PFNGLUNIFORM4UI64VARBPROC (glfwGetProcAddress("glUniform4ui64vARB"));
02425   glUniform4ui64vNV = PFNGLUNIFORM4UI64VNVPROC (glfwGetProcAddress("glUniform4ui64vNV"));
02426   glUniform4uiv = PFNGLUNIFORM4UIVPROC (glfwGetProcAddress("glUniform4uiv"));
02427   glUniformBlockBinding = PFNGLUNIFORMBLOCKBINDINGPROC (glfwGetProcAddress("glUniformBlockBinding"));
02428   glUniformHandleui64ARB =
PFNGLUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress("glUniformHandleui64ARB"));
02429   glUniformHandleui64NV = PFNGLUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress("glUniformHandleui64NV"));
02430   glUniformHandleui64vARB =
PFNGLUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress("glUniformHandleui64vARB"));
02431   glUniformHandleui64vNV =
PFNGLUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress("glUniformHandleui64vNV"));
02432   glUniformMatrix2dv = PFNGLUNIFORMMATRIX2DVPROC (glfwGetProcAddress("glUniformMatrix2dv"));
02433   glUniformMatrix2fv = PFNGLUNIFORMMATRIX2FVPROC (glfwGetProcAddress("glUniformMatrix2fv"));
02434   glUniformMatrix2x3dv = PFNGLUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress("glUniformMatrix2x3dv"));
02435   glUniformMatrix2x3fv = PFNGLUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress("glUniformMatrix2x3fv"));
02436   glUniformMatrix2x4dv = PFNGLUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress("glUniformMatrix2x4dv"));
02437   glUniformMatrix2x4fv = PFNGLUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress("glUniformMatrix2x4fv"));
02438   glUniformMatrix3dv = PFNGLUNIFORMMATRIX3DVPROC (glfwGetProcAddress("glUniformMatrix3dv"));
02439   glUniformMatrix3fv = PFNGLUNIFORMMATRIX3FVPROC (glfwGetProcAddress("glUniformMatrix3fv"));
02440   glUniformMatrix3x2dv = PFNGLUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress("glUniformMatrix3x2dv"));
02441   glUniformMatrix3x2fv = PFNGLUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress("glUniformMatrix3x2fv"));
02442   glUniformMatrix3x4dv = PFNGLUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress("glUniformMatrix3x4dv"));
02443   glUniformMatrix3x4fv = PFNGLUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress("glUniformMatrix3x4fv"));
02444   glUniformMatrix4dv = PFNGLUNIFORMMATRIX4DVPROC (glfwGetProcAddress("glUniformMatrix4dv"));
02445   glUniformMatrix4fv = PFNGLUNIFORMMATRIX4FVPROC (glfwGetProcAddress("glUniformMatrix4fv"));
02446   glUniformMatrix4x2dv = PFNGLUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress("glUniformMatrix4x2dv"));
02447   glUniformMatrix4x2fv = PFNGLUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress("glUniformMatrix4x2fv"));
02448   glUniformMatrix4x3dv = PFNGLUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress("glUniformMatrix4x3dv"));
02449   glUniformMatrix4x3fv = PFNGLUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress("glUniformMatrix4x3fv"));
02450   glUniformSubroutinesuiv =
PFNGLUNIFORMSUBROUTINESUIVPROC (glfwGetProcAddress("glUniformSubroutinesuiv"));
02451   glUniformui64NV = PFNGLUNIFORMUI64NVPROC (glfwGetProcAddress("glUniformui64NV"));
02452   glUniformui64vNV = PFNGLUNIFORMUI64VNVPROC (glfwGetProcAddress("glUniformui64vNV"));
02453   glUnmapBuffer = PFNGLUNMAPBUFFERPROC (glfwGetProcAddress("glUnmapBuffer"));
02454   glUnmapNamedBuffer = PFNGLUNMAPNAMEDBUFFERPROC (glfwGetProcAddress("glUnmapNamedBuffer"));
02455   glUnmapNamedBufferEXT = PFNGLUNMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress("glUnmapNamedBufferEXT"));
02456   glUseProgram = PFNGLUSEPROGRAMPROC (glfwGetProcAddress("glUseProgram"));
02457   glUseProgramStages = PFNGLUSEPROGRAMSTAGESPROC (glfwGetProcAddress("glUseProgramStages"));
02458   glUseProgramEXT = PFNGLUSESHADERPROGRAMEXTPROC (glfwGetProcAddress("glUseProgramEXT"));
02459   glValidateProgram = PFNGLVALIDATEPROGRAMPROC (glfwGetProcAddress("glValidateProgram"));
02460   glValidateProgramPipeline =
PFNGLVALIDATEPROGRAMPIPELINEPROC (glfwGetProcAddress("glValidateProgramPipeline"));
02461   glVertexArrayAttribBinding =
PFNGLVERTEXARRAYATTRIBBINDINGPROC (glfwGetProcAddress("glVertexArrayAttribBinding"));
02462   glVertexArrayAttribFormat =
PFNGLVERTEXARRAYATTRIBFORMATPROC (glfwGetProcAddress("glVertexArrayAttribFormat"));
02463   glVertexArrayAttribIFormat =
PFNGLVERTEXARRAYATTRIBIFORMATPROC (glfwGetProcAddress("glVertexArrayAttribIFormat"));
02464   glVertexArrayAttribLFormat =

```



```
PFNGVERTEXARRAYATTRIBIFORMATPROC (glfwGetProcAddress ("glVertexArrayAttribLFormat"));
02465 glVertexArrayBindVertexBufferEXT =
PFNGVERTEXARRAYBINDVERTEXBUFFEREXTPROC (glfwGetProcAddress ("glVertexArrayBindVertexBufferEXT"));
02466 glVertexArrayBindingDivisor =
PFNGVERTEXARRAYBINDINGDIVISORPROC (glfwGetProcAddress ("glVertexArrayBindingDivisor"));
02467 glVertexArrayColorOffsetEXT =
PFNGVERTEXARRAYCOLOROFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayColorOffsetEXT"));
02468 glVertexArrayEdgeFlagOffsetEXT =
PFNGVERTEXARRAYEDGEFLAGOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayEdgeFlagOffsetEXT"));
02469 glVertexArrayElementBuffer =
PFNGVERTEXARRAYELEMENTBUFFERPROC (glfwGetProcAddress ("glVertexArrayElementBuffer"));
02470 glVertexArrayFogCoordOffsetEXT =
PFNGVERTEXARRAYFOGCOORDOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayFogCoordOffsetEXT"));
02471 glVertexArrayIndexOffsetEXT =
PFNGVERTEXARRAYINDEXOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayIndexOffsetEXT"));
02472 glVertexArrayMultiTexCoordOffsetEXT =
PFNGVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayMultiTexCoordOffsetEXT"));
02473 glVertexArrayNormalOffsetEXT =
PFNGVERTEXARRAYNORMALOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayNormalOffsetEXT"));
02474 glVertexArraySecondaryColorOffsetEXT =
PFNGVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC (glfwGetProcAddress ("glVertexArraySecondaryColorOffsetEXT"));
02475 glVertexArrayTexCoordOffsetEXT =
PFNGVERTEXARRAYTEXCOORDOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayTexCoordOffsetEXT"));
02476 glVertexArrayVertexAttribBindingEXT =
PFNGVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribBindingEXT"));
02477 glVertexArrayVertexAttribDivisorEXT =
PFNGVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribDivisorEXT"));
02478 glVertexArrayVertexAttribFormatEXT =
PFNGVERTEXARRAYVERTEXATTRIBFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribFormatEXT"));
02479 glVertexArrayVertexAttribIFormatEXT =
PFNGVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribIFormatEXT"));
02480 glVertexArrayVertexAttribIOffsetEXT =
PFNGVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribIOffsetEXT"));
02481 glVertexArrayVertexAttribLFormatEXT =
PFNGVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribLFormatEXT"));
02482 glVertexArrayVertexAttribLOffsetEXT =
PFNGVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribLOffsetEXT"));
02483 glVertexArrayVertexAttribOffsetEXT =
PFNGVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribOffsetEXT"));
02484 glVertexArrayVertexBindingDivisorEXT =
PFNGVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC (glfwGetProcAddress ("glVertexArrayVertexBindingDivisorEXT"));
02485 glVertexArrayVertexBuffer =
PFNGVERTEXARRAYVERTEXBUFFERPROC (glfwGetProcAddress ("glVertexArrayVertexBuffer"));
02486 glVertexArrayVertexBuffers =
PFNGVERTEXARRAYVERTEXBUFFERSPROC (glfwGetProcAddress ("glVertexArrayVertexBuffers"));
02487 glVertexArrayVertexOffsetEXT =
PFNGVERTEXARRAYVERTEXOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexOffsetEXT"));
02488 glVertexAttrib1d = PFNGVERTEXATTRIB1DPROC (glfwGetProcAddress ("glVertexAttrib1d"));
02489 glVertexAttrib1dv = PFNGVERTEXATTRIB1DVPROC (glfwGetProcAddress ("glVertexAttrib1dv"));
02490 glVertexAttrib1f = PFNGVERTEXATTRIB1FPROC (glfwGetProcAddress ("glVertexAttrib1f"));
02491 glVertexAttrib1fv = PFNGVERTEXATTRIB1FVPROC (glfwGetProcAddress ("glVertexAttrib1fv"));
02492 glVertexAttrib1s = PFNGVERTEXATTRIB1SPROC (glfwGetProcAddress ("glVertexAttrib1s"));
02493 glVertexAttrib1sv = PFNGVERTEXATTRIB1SVPROC (glfwGetProcAddress ("glVertexAttrib1sv"));
02494 glVertexAttrib2d = PFNGVERTEXATTRIB2DPROC (glfwGetProcAddress ("glVertexAttrib2d"));
02495 glVertexAttrib2dv = PFNGVERTEXATTRIB2DVPROC (glfwGetProcAddress ("glVertexAttrib2dv"));
02496 glVertexAttrib2f = PFNGVERTEXATTRIB2FPROC (glfwGetProcAddress ("glVertexAttrib2f"));
02497 glVertexAttrib2fv = PFNGVERTEXATTRIB2FVPROC (glfwGetProcAddress ("glVertexAttrib2fv"));
02498 glVertexAttrib2s = PFNGVERTEXATTRIB2SPROC (glfwGetProcAddress ("glVertexAttrib2s"));
02499 glVertexAttrib2sv = PFNGVERTEXATTRIB2SVPROC (glfwGetProcAddress ("glVertexAttrib2sv"));
02500 glVertexAttrib3d = PFNGVERTEXATTRIB3DPROC (glfwGetProcAddress ("glVertexAttrib3d"));
02501 glVertexAttrib3dv = PFNGVERTEXATTRIB3DVPROC (glfwGetProcAddress ("glVertexAttrib3dv"));
02502 glVertexAttrib3f = PFNGVERTEXATTRIB3FPROC (glfwGetProcAddress ("glVertexAttrib3f"));
02503 glVertexAttrib3fv = PFNGVERTEXATTRIB3FVPROC (glfwGetProcAddress ("glVertexAttrib3fv"));
02504 glVertexAttrib3s = PFNGVERTEXATTRIB3SPROC (glfwGetProcAddress ("glVertexAttrib3s"));
02505 glVertexAttrib3sv = PFNGVERTEXATTRIB3SVPROC (glfwGetProcAddress ("glVertexAttrib3sv"));
02506 glVertexAttrib4Nbv = PFNGVERTEXATTRIB4NBVPROC (glfwGetProcAddress ("glVertexAttrib4Nbv"));
02507 glVertexAttrib4Niv = PFNGVERTEXATTRIB4NIVPROC (glfwGetProcAddress ("glVertexAttrib4Niv"));
02508 glVertexAttrib4Nsv = PFNGVERTEXATTRIB4NSVPROC (glfwGetProcAddress ("glVertexAttrib4Nsv"));
02509 glVertexAttrib4Nubv = PFNGVERTEXATTRIB4NUBPROC (glfwGetProcAddress ("glVertexAttrib4Nubv"));
02510 glVertexAttrib4Nuiv = PFNGVERTEXATTRIB4NUVPROC (glfwGetProcAddress ("glVertexAttrib4Nuiv"));
02511 glVertexAttrib4Nusv = PFNGVERTEXATTRIB4NUSVPROC (glfwGetProcAddress ("glVertexAttrib4Nusv"));
02512 glVertexAttrib4bv = PFNGVERTEXATTRIB4BVPROC (glfwGetProcAddress ("glVertexAttrib4bv"));
02513 glVertexAttrib4d = PFNGVERTEXATTRIB4DPROC (glfwGetProcAddress ("glVertexAttrib4d"));
02514 glVertexAttrib4dv = PFNGVERTEXATTRIB4DVPROC (glfwGetProcAddress ("glVertexAttrib4dv"));
02515 glVertexAttrib4f = PFNGVERTEXATTRIB4FPROC (glfwGetProcAddress ("glVertexAttrib4f"));
02516 glVertexAttrib4fv = PFNGVERTEXATTRIB4FVPROC (glfwGetProcAddress ("glVertexAttrib4fv"));
02517 glVertexAttrib4iv = PFNGVERTEXATTRIB4IVPROC (glfwGetProcAddress ("glVertexAttrib4iv"));
02518 glVertexAttrib4s = PFNGVERTEXATTRIB4SPROC (glfwGetProcAddress ("glVertexAttrib4s"));
02519 glVertexAttrib4sv = PFNGVERTEXATTRIB4SVPROC (glfwGetProcAddress ("glVertexAttrib4sv"));
02520 glVertexAttrib4ubv = PFNGVERTEXATTRIB4UBVPROC (glfwGetProcAddress ("glVertexAttrib4ubv"));
02521 glVertexAttrib4uiv = PFNGVERTEXATTRIB4UIVPROC (glfwGetProcAddress ("glVertexAttrib4uiv"));
02522 glVertexAttrib4usv = PFNGVERTEXATTRIB4USVPROC (glfwGetProcAddress ("glVertexAttrib4usv"));
02523 glVertexAttribBinding = PFNGVERTEXATTRIBBINDINGPROC (glfwGetProcAddress ("glVertexAttribBinding"));
02524 glVertexAttribDivisor = PFNGVERTEXATTRIBDIVISORPROC (glfwGetProcAddress ("glVertexAttribDivisor"));
02525 glVertexAttribDivisorARB =
PFNGVERTEXATTRIBDIVISORARBPROC (glfwGetProcAddress ("glVertexAttribDivisorARB"));
```



```

02527     glVertexAttribFormat = PFNGLVERTEXATTRIBFORMATPROC (glfwGetProcAddress("glVertexAttribFormat"));
02528     glVertexAttribFormatNV =
PFNGLVERTEXATTRIBFORMATNVPROC (glfwGetProcAddress("glVertexAttribFormatNV"));
02529     glVertexAttribI1i = PFNGLVERTEXATTRIBI1IPROC (glfwGetProcAddress("glVertexAttribI1i"));
02530     glVertexAttribI1iv = PFNGLVERTEXATTRIBI1IVPROC (glfwGetProcAddress("glVertexAttribI1iv"));
02531     glVertexAttribI1ui = PFNGLVERTEXATTRIBI1UIPROC (glfwGetProcAddress("glVertexAttribI1ui"));
02532     glVertexAttribI1uiv = PFNGLVERTEXATTRIBI1UIVPROC (glfwGetProcAddress("glVertexAttribI1uiv"));
02533     glVertexAttribI2i = PFNGLVERTEXATTRIBI2IPROC (glfwGetProcAddress("glVertexAttribI2i"));
02534     glVertexAttribI2iv = PFNGLVERTEXATTRIBI2IVPROC (glfwGetProcAddress("glVertexAttribI2iv"));
02535     glVertexAttribI2ui = PFNGLVERTEXATTRIBI2UIPROC (glfwGetProcAddress("glVertexAttribI2ui"));
02536     glVertexAttribI2uiv = PFNGLVERTEXATTRIBI2UIVPROC (glfwGetProcAddress("glVertexAttribI2uiv"));
02537     glVertexAttribI3i = PFNGLVERTEXATTRIBI3IPROC (glfwGetProcAddress("glVertexAttribI3i"));
02538     glVertexAttribI3iv = PFNGLVERTEXATTRIBI3IVPROC (glfwGetProcAddress("glVertexAttribI3iv"));
02539     glVertexAttribI3ui = PFNGLVERTEXATTRIBI3UIPROC (glfwGetProcAddress("glVertexAttribI3ui"));
02540     glVertexAttribI3uiv = PFNGLVERTEXATTRIBI3UIVPROC (glfwGetProcAddress("glVertexAttribI3uiv"));
02541     glVertexAttribI4bv = PFNGLVERTEXATTRIBI4BVPROC (glfwGetProcAddress("glVertexAttribI4bv"));
02542     glVertexAttribI4i = PFNGLVERTEXATTRIBI4IPROC (glfwGetProcAddress("glVertexAttribI4i"));
02543     glVertexAttribI4iv = PFNGLVERTEXATTRIBI4IVPROC (glfwGetProcAddress("glVertexAttribI4iv"));
02544     glVertexAttribI4sv = PFNGLVERTEXATTRIBI4SVPROC (glfwGetProcAddress("glVertexAttribI4sv"));
02545     glVertexAttribI4ubv = PFNGLVERTEXATTRIBI4UBVPROC (glfwGetProcAddress("glVertexAttribI4ubv"));
02546     glVertexAttribI4ui = PFNGLVERTEXATTRIBI4UIPROC (glfwGetProcAddress("glVertexAttribI4ui"));
02547     glVertexAttribI4uiv = PFNGLVERTEXATTRIBI4UIVPROC (glfwGetProcAddress("glVertexAttribI4uiv"));
02548     glVertexAttribI4usv = PFNGLVERTEXATTRIBI4USVPROC (glfwGetProcAddress("glVertexAttribI4usv"));
02549     glVertexAttribIFormat = PFNGLVERTEXATTRIBIFORMATPROC (glfwGetProcAddress("glVertexAttribIFormat"));
02550     glVertexAttribIFormatNV =
PFNGLVERTEXATTRIBIFORMATNVPROC (glfwGetProcAddress("glVertexAttribIFormatNV"));
02551     glVertexAttribIPointer =
PFNGLVERTEXATTRIBIPOINTERPROC (glfwGetProcAddress("glVertexAttribIPointer"));
02552     glVertexAttribL1d = PFNGLVERTEXATTRIBL1DPROC (glfwGetProcAddress("glVertexAttribL1d"));
02553     glVertexAttribL1dv = PFNGLVERTEXATTRIBL1DVPROC (glfwGetProcAddress("glVertexAttribL1dv"));
02554     glVertexAttribL1i64NV = PFNGLVERTEXATTRIBL1i64NVPROC (glfwGetProcAddress("glVertexAttribL1i64NV"));
02555     glVertexAttribL1i64vNV =
PFNGLVERTEXATTRIBL1i64vNVPROC (glfwGetProcAddress("glVertexAttribL1i64vNV"));
02556     glVertexAttribL1ui64ARB =
PFNGLVERTEXATTRIBL1UI64ARBPROC (glfwGetProcAddress("glVertexAttribL1ui64ARB"));
02557     glVertexAttribL1ui64NV =
PFNGLVERTEXATTRIBL1UI64NVPROC (glfwGetProcAddress("glVertexAttribL1ui64NV"));
02558     glVertexAttribL1ui64vARB =
PFNGLVERTEXATTRIBL1UI64vARBPROC (glfwGetProcAddress("glVertexAttribL1ui64vARB"));
02559     glVertexAttribL1ui64vNV =
PFNGLVERTEXATTRIBL1UI64vNVPROC (glfwGetProcAddress("glVertexAttribL1ui64vNV"));
02560     glVertexAttribL2d = PFNGLVERTEXATTRIBL2DPROC (glfwGetProcAddress("glVertexAttribL2d"));
02561     glVertexAttribL2dv = PFNGLVERTEXATTRIBL2DVPROC (glfwGetProcAddress("glVertexAttribL2dv"));
02562     glVertexAttribL2i64NV = PFNGLVERTEXATTRIBL2i64NVPROC (glfwGetProcAddress("glVertexAttribL2i64NV"));
02563     glVertexAttribL2i64vNV =
PFNGLVERTEXATTRIBL2i64vNVPROC (glfwGetProcAddress("glVertexAttribL2i64vNV"));
02564     glVertexAttribL2ui64NV =
PFNGLVERTEXATTRIBL2UI64NVPROC (glfwGetProcAddress("glVertexAttribL2ui64NV"));
02565     glVertexAttribL2ui64vNV =
PFNGLVERTEXATTRIBL2UI64vNVPROC (glfwGetProcAddress("glVertexAttribL2ui64vNV"));
02566     glVertexAttribL3d = PFNGLVERTEXATTRIBL3DPROC (glfwGetProcAddress("glVertexAttribL3d"));
02567     glVertexAttribL3dv = PFNGLVERTEXATTRIBL3DVPROC (glfwGetProcAddress("glVertexAttribL3dv"));
02568     glVertexAttribL3i64NV = PFNGLVERTEXATTRIBL3i64NVPROC (glfwGetProcAddress("glVertexAttribL3i64NV"));
02569     glVertexAttribL3i64vNV =
PFNGLVERTEXATTRIBL3i64vNVPROC (glfwGetProcAddress("glVertexAttribL3i64vNV"));
02570     glVertexAttribL3ui64NV =
PFNGLVERTEXATTRIBL3UI64NVPROC (glfwGetProcAddress("glVertexAttribL3ui64NV"));
02571     glVertexAttribL3ui64vNV =
PFNGLVERTEXATTRIBL3UI64vNVPROC (glfwGetProcAddress("glVertexAttribL3ui64vNV"));
02572     glVertexAttribL4d = PFNGLVERTEXATTRIBL4DPROC (glfwGetProcAddress("glVertexAttribL4d"));
02573     glVertexAttribL4dv = PFNGLVERTEXATTRIBL4DVPROC (glfwGetProcAddress("glVertexAttribL4dv"));
02574     glVertexAttribL4i64NV = PFNGLVERTEXATTRIBL4i64NVPROC (glfwGetProcAddress("glVertexAttribL4i64NV"));
02575     glVertexAttribL4i64vNV =
PFNGLVERTEXATTRIBL4i64vNVPROC (glfwGetProcAddress("glVertexAttribL4i64vNV"));
02576     glVertexAttribL4ui64NV =
PFNGLVERTEXATTRIBL4UI64NVPROC (glfwGetProcAddress("glVertexAttribL4ui64NV"));
02577     glVertexAttribL4ui64vNV =
PFNGLVERTEXATTRIBL4UI64vNVPROC (glfwGetProcAddress("glVertexAttribL4ui64vNV"));
02578     glVertexAttribLFormat = PFNGLVERTEXATTRIBLFORMATPROC (glfwGetProcAddress("glVertexAttribLFormat"));
02579     glVertexAttribLFormatNV =
PFNGLVERTEXATTRIBLFORMATNVPROC (glfwGetProcAddress("glVertexAttribLFormatNV"));
02580     glVertexAttribLPointer =
PFNGLVERTEXATTRIBLPOINTERPROC (glfwGetProcAddress("glVertexAttribLPointer"));
02581     glVertexAttribP1ui = PFNGLVERTEXATTRIBP1UIPROC (glfwGetProcAddress("glVertexAttribP1ui"));
02582     glVertexAttribP1uiv = PFNGLVERTEXATTRIBP1UIVPROC (glfwGetProcAddress("glVertexAttribP1uiv"));
02583     glVertexAttribP2ui = PFNGLVERTEXATTRIBP2UIPROC (glfwGetProcAddress("glVertexAttribP2ui"));
02584     glVertexAttribP2uiv = PFNGLVERTEXATTRIBP2UIVPROC (glfwGetProcAddress("glVertexAttribP2uiv"));
02585     glVertexAttribP3ui = PFNGLVERTEXATTRIBP3UIPROC (glfwGetProcAddress("glVertexAttribP3ui"));
02586     glVertexAttribP3uiv = PFNGLVERTEXATTRIBP3UIVPROC (glfwGetProcAddress("glVertexAttribP3uiv"));
02587     glVertexAttribP4ui = PFNGLVERTEXATTRIBP4UIPROC (glfwGetProcAddress("glVertexAttribP4ui"));
02588     glVertexAttribP4uiv = PFNGLVERTEXATTRIBP4UIVPROC (glfwGetProcAddress("glVertexAttribP4uiv"));
02589     glVertexAttribPPointer = PFNGLVERTEXATTRIBPOINTERPROC (glfwGetProcAddress("glVertexAttribPPointer"));
02590     glVertexBindingDivisor =
PFNGLVERTEXBINDINGDIVISORPROC (glfwGetProcAddress("glVertexBindingDivisor"));
02591     glVertexFormatNV = PFNGLVERTEXFORMATNVPROC (glfwGetProcAddress("glVertexFormatNV"));
02592     glViewport = PFNGLVIEWPORTPROC (glfwGetProcAddress("glViewport"));
02593     glViewportArrayv = PFNGLVIEWPORTARRAYVPROC (glfwGetProcAddress("glViewportArrayv"));

```

```

02594     glViewportIndexedf = PFNGLVIEWPORTINDEXEDFPROC(glfwGetProcAddress("glViewportIndexedf"));
02595     glViewportIndexedfv = PFNGLVIEWPORTINDEXEDFVPROC(glfwGetProcAddress("glViewportIndexedfv"));
02596     glViewportPositionWScaleNV =
PFNGLVIEWPORTPOSITIONWSCALENVPROC(glfwGetProcAddress("glViewportPositionWScaleNV"));
02597     glViewportSwizzleNV = PFNGLVIEWPORTSWIZZLENVPROC(glfwGetProcAddress("glViewportSwizzleNV"));
02598     glWaitSync = PFNGLWAITSYNCPROC(glfwGetProcAddress("glWaitSync"));
02599     glWaitVkSemaphoreNV = PFNGLWAITVKSEMAPHORENVPROC(glfwGetProcAddress("glWaitVkSemaphoreNV"));
02600     glWeightPathsNV = PFNGLWEIGHTPATHSNVPROC(glfwGetProcAddress("glWeightPathsNV"));
02601     glWindowRectanglesEXT = PFNGLWINDOWRECTANGLESEXTPROC(glfwGetProcAddress("glWindowRectanglesEXT"));
02602 #endif
02603
02604     // 使用している GPU のバッファアライメントを調べる
02605     glGetIntegerv(GL_UNIFORM_BUFFER_OFFSET_ALIGNMENT, &ggBufferAlignment);
02606 }
02607
02608 //
02609 // OpenGL のエラーをチェックする
02610 //
02611 //   OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する
02612 //
02613 //   msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
02614 //
02615 void gg::ggError(const std::string& name, unsigned int line)
02616 {
02617     const GLenum error{ glGetError() };
02618
02619     if (error != GL_NO_ERROR)
02620     {
02621         if (!name.empty())
02622         {
02623             std::cerr << name;
02624             if (line > 0) std::cerr << " (" << line << ")";
02625             std::cerr << ": ";
02626         }
02627
02628         switch (error)
02629         {
02630             case GL_INVALID_ENUM:
02631                 std::cerr << "An unacceptable value is specified for an enumerated argument" << std::endl;
02632                 break;
02633             case GL_INVALID_VALUE:
02634                 std::cerr << "A numeric argument is out of range" << std::endl;
02635                 break;
02636             case GL_INVALID_OPERATION:
02637                 std::cerr << "The specified operation is not allowed in the current state" << std::endl;
02638                 break;
02639             case GL_OUT_OF_MEMORY:
02640                 std::cerr << "There is not enough memory left to execute the command" << std::endl;
02641                 break;
02642             case GL_INVALID_FRAMEBUFFER_OPERATION:
02643                 std::cerr << "The specified operation is not allowed current frame buffer" << std::endl;
02644                 break;
02645             default:
02646                 std::cerr << "An OpenGL error has occurred: " << std::hex << std::showbase << error << std::endl;
02647                 break;
02648         }
02649     }
02650 }
02651
02652 //
02653 // FBO のエラーをチェックする
02654 //
02655 //   FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する
02656 //
02657 //   msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
02658 //
02659 void gg::ggFBOError(const std::string& name, unsigned int line)
02660 {
02661     const auto status{ glCheckFramebufferStatus(GL_FRAMEBUFFER) };
02662
02663     if (status != GL_FRAMEBUFFER_COMPLETE)
02664     {
02665         if (!name.empty())
02666         {
02667             std::cerr << name;
02668             if (line > 0) std::cerr << " (" << line << ")";
02669             std::cerr << ": ";
02670         }
02671
02672         switch (status)
02673         {
02674             case GL_FRAMEBUFFER_UNDEFINED:
02675                 std::cerr << "the default framebuffer does not exist" << std::endl;
02676                 break;
02677             case GL_FRAMEBUFFER_INCOMPLETE_ATTACHMENT:
02678                 std::cerr << "Framebuffer incomplete, duplicate attachment" << std::endl;
02679                 break;

```

```

02680     case GL_FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT:
02681         std::cerr << "Framebuffer incomplete, missing attachment" << std::endl;
02682         break;
02683     case GL_FRAMEBUFFER_UNSUPPORTED:
02684         std::cerr << "Unsupported framebuffer internal" << std::endl;
02685         break;
02686     case GL_FRAMEBUFFER_INCOMPLETE_MULTISAMPLE:
02687         std::cerr << "The value of GL_RENDERBUFFER_SAMPLES is not"
02688             " the same for all attached renderbuffers or,"
02689             " if the attached images are a mix of renderbuffers and textures,"
02690             " the value of GL_RENDERBUFFER_SAMPLES is not zero" << std::endl;
02691         break;
02692 #if !defined(GL_GLES_PROTOTYPES)
02693     case GL_FRAMEBUFFER_INCOMPLETE_LAYER_TARGETS:
02694         std::cerr << "Any framebuffer attachment is layered,"
02695             " and any populated attachment is not layered,"
02696             " or if all populated color attachments are not from textures"
02697             " of the same target" << std::endl;
02698         break;
02699     case GL_FRAMEBUFFER_INCOMPLETE_DRAW_BUFFER:
02700         std::cerr << "Framebuffer incomplete, missing draw buffer" << std::endl;
02701         break;
02702     case GL_FRAMEBUFFER_INCOMPLETE_READ_BUFFER:
02703         std::cerr << "Framebuffer incomplete, missing read buffer" << std::endl;
02704         break;
02705 #endif
02706     default:
02707         std::cerr << "Programming error; will fail on all hardware: " << std::hex << std::showbase <<
02708             status << std::endl;
02709         break;
02710     }
02711 }
02712
02713 //
02714 // 変換行列：行列とベクトルの積  $c \leftarrow a \times b$ 
02715 //
02716 void gg::GgMatrix::projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02717 {
02718     for (int i = 0; i < 4; ++i)
02719     {
02720         c[i] = a[0 + i] * b[0] + a[4 + i] * b[1] + a[8 + i] * b[2] + a[12 + i] * b[3];
02721     }
02722 }
02723
02724 //
02725 // 変換行列：行列と行列の積  $c \leftarrow a \times b$ 
02726 //
02727 void gg::GgMatrix::multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02728 {
02729     for (int i = 0; i < 16; ++i)
02730     {
02731         int j = i & 3, k = i & ~3;
02732         c[i] = a[0 + j] * b[k + 0] + a[4 + j] * b[k + 1] + a[8 + j] * b[k + 2] + a[12 + j] * b[k + 3];
02733     }
02734 }
02735
02736 //
02737 // 変換行列：単位行列を設定する
02738 //
02739 void gg::GgMatrix& gg::GgMatrix::loadIdentity()
02740 {
02741     *this =
02742     {
02743         {
02744             1.0f, 0.0f, 0.0f, 0.0f,
02745             0.0f, 1.0f, 0.0f, 0.0f,
02746             0.0f, 0.0f, 1.0f, 0.0f,
02747             0.0f, 0.0f, 0.0f, 1.0f
02748         };
02749     };
02750     return *this;
02751 }
02752
02753 //
02754 // 変換行列：平行移動変換行列を設定する
02755 //
02756 void gg::GgMatrix& gg::GgMatrix::loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02757 {
02758     *this =
02759     {
02760         {
02761             w, 0.0f, 0.0f, 0.0f,
02762             0.0f, w, 0.0f, 0.0f,
02763             0.0f, 0.0f, w, 0.0f,
02764             x, y, z, w
02765         };
02766     };
02767 }

```

```

02766     return *this;
02767 }
02768
02769 //
02770 // 変換行列：拡大縮小変換行列を設定する
02771 //
02772 gg::GgMatrix& gg::GgMatrix::loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02773 {
02774     *this =
02775     {
02776         x,    0.0f, 0.0f, 0.0f,
02777         0.0f, y,    0.0f, 0.0f,
02778         0.0f, 0.0f, z,    0.0f,
02779         0.0f, 0.0f, 0.0f, w
02780     };
02781
02782     return *this;
02783 }
02784
02785 //
02786 // 変換行列：x 軸中心の回転変換行列を設定する
02787 //
02788 gg::GgMatrix& gg::GgMatrix::loadRotateX(GLfloat a)
02789 {
02790     const auto c{ cosf(a) };
02791     const auto s{ sinf(a) };
02792
02793     *this =
02794     {
02795         1.0f, 0.0f, 0.0f, 0.0f,
02796         0.0f, c,    s,    0.0f,
02797         0.0f, -s,   c,    0.0f,
02798         0.0f, 0.0f, 0.0f, 1.0f
02799     };
02800
02801     return *this;
02802 }
02803
02804 //
02805 // 変換行列：y 軸中心の回転変換行列を設定する
02806 //
02807 gg::GgMatrix& gg::GgMatrix::loadRotateY(GLfloat a)
02808 {
02809     const auto c{ cosf(a) };
02810     const auto s{ sinf(a) };
02811
02812     *this =
02813     {
02814         c,    0.0f, -s,    0.0f,
02815         0.0f, 1.0f, 0.0f, 0.0f,
02816         s,    0.0f, c,    0.0f,
02817         0.0f, 0.0f, 0.0f, 1.0f
02818     };
02819
02820     return *this;
02821 }
02822
02823 //
02824 // 変換行列：z 軸中心の回転変換行列を設定する
02825 //
02826 gg::GgMatrix& gg::GgMatrix::loadRotateZ(GLfloat a)
02827 {
02828     const auto c{ cosf(a) };
02829     const auto s{ sinf(a) };
02830
02831     *this =
02832     {
02833         c,    s,    0.0f, 0.0f,
02834         -s,   c,    0.0f, 0.0f,
02835         0.0f, 0.0f, 1.0f, 0.0f,
02836         0.0f, 0.0f, 0.0f, 1.0f
02837     };
02838
02839     return *this;
02840 }
02841
02842 //
02843 // 変換行列：任意軸中心の回転変換行列を設定する
02844 //
02845 gg::GgMatrix& gg::GgMatrix::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
02846 {
02847     const auto d{ sqrtf(x * x + y * y + z * z) };
02848
02849     if (d > 0.0f)
02850     {
02851         const auto l1{ x / d }, m1{ y / d }, n1{ z / d };
02852         const auto l2{ 1 * 1 }, m2{ m * m }, n2{ n * n };

```

```

02853     const auto lm{ 1 * m }, mn{ m * n }, nl{ n * 1 };
02854     const auto c{ cosf(a) }, cl{ 1.0f - c };
02855     const auto s{ sinf(a) };
02856
02857     *this =
02858     {
02859         (1.0f - l2) * c + l2,
02860         lm * cl + n * s,
02861         nl * cl - m * s,
02862         0.0f,
02863
02864         lm * cl - n * s,
02865         (1.0f - m2) * c + m2,
02866         mn * cl + l * s,
02867         0.0f,
02868
02869         nl * cl + m * s,
02870         mn * cl - l * s,
02871         (1.0f - n2) * c + n2,
02872         0.0f,
02873
02874         0.0f,
02875         0.0f,
02876         0.0f,
02877         1.0f,
02878     };
02879 }
02880
02881 return *this;
02882 }
02883
02884 //
02885 // 変換行列：転置行列を設定する
02886 //
02887 gg::GgMatrix& gg::GgMatrix::loadTranspose(const GLfloat* marray)
02888 {
02889     *this =
02890     {
02891         marray[ 0],
02892         marray[ 4],
02893         marray[ 8],
02894         marray[12],
02895         marray[ 1],
02896         marray[ 5],
02897         marray[ 9],
02898         marray[13],
02899         marray[ 2],
02900         marray[ 6],
02901         marray[10],
02902         marray[14],
02903         marray[ 3],
02904         marray[ 7],
02905         marray[11],
02906         marray[15]
02907     };
02908
02909     return *this;
02910 }
02911
02912 //
02913 // 変換行列：逆行列を設定する
02914 //
02915 gg::GgMatrix& gg::GgMatrix::loadInvert(const GLfloat* marray)
02916 {
02917     GLfloat lu[20];
02918     GLfloat* plu[4];
02919
02920     // j 行の要素の値の絶対値の最大値を plu[j][4] に求める
02921     for (int j = 0; j < 4; ++j)
02922     {
02923         auto max{ fabs(*(plu[j] = lu + 5 * j) = *(marray++)) };
02924
02925         for (int i = 0; ++i < 4;)
02926         {
02927             auto a{ fabs(plu[j][i] = *(marray++)) };
02928             if (a > max) max = a;
02929         }
02930         if (max == 0.0f) return *this;
02931         plu[j][4] = 1.0f / max;
02932     }
02933
02934     // ピボットを考慮した LU 分解
02935     for (int j = 0; j < 4; ++j)
02936     {
02937         auto max{ fabs(plu[j][j] * plu[j][4]) };
02938         int i = j;
02939     }

```

```

02940     for (int k = j; ++k < 4;)
02941     {
02942         auto a{ fabs(plu[k][j] * plu[k][4]) };
02943         if (a > max)
02944         {
02945             max = a;
02946             i = k;
02947         }
02948     }
02949     if (i > j)
02950     {
02951         auto* t{ plu[j] };
02952         plu[j] = plu[i];
02953         plu[i] = t;
02954     }
02955     if (plu[j][j] == 0.0f) return *this;
02956     for (int k = j; ++k < 4;)
02957     {
02958         plu[k][j] /= plu[j][j];
02959         for (int i = j; ++i < 4;)
02960         {
02961             plu[k][i] -= plu[j][i] * plu[k][j];
02962         }
02963     }
02964 }
02965
02966 // LU 分解から逆行列を求める
02967 for (int k = 0; k < 4; ++k)
02968 {
02969     // array に単位行列を設定する
02970     for (int i = 0; i < 4; ++i)
02971     {
02972         (*this)[i * 4 + k] = (plu[i] == lu + k * 5) ? 1.0f : 0.0f;
02973     }
02974     // lu から逆行列を求める
02975     for (int i = 0; i < 4; ++i)
02976     {
02977         for (int j = i; ++j < 4;)
02978         {
02979             (*this)[j * 4 + k] -= (*this)[i * 4 + k] * plu[j][i];
02980         }
02981     }
02982     for (int i = 4; --i >= 0;)
02983     {
02984         for (int j = i; ++j < 4;)
02985         {
02986             (*this)[i * 4 + k] -= plu[i][j] * (*this)[j * 4 + k];
02987         }
02988         (*this)[i * 4 + k] /= plu[i][i];
02989     }
02990 }
02991
02992 return *this;
02993 }
02994
02995 //
02996 // 変換行列：法線変換行列を設定する
02997 //
02998 gg::GgMatrix& gg::GgMatrix::loadNormal(const GLfloat* marray)
02999 {
03000     *this =
03001     {
03002         marray[ 5] * marray[10] - marray[ 6] * marray[ 9],
03003         marray[ 6] * marray[ 8] - marray[ 4] * marray[10],
03004         marray[ 4] * marray[ 9] - marray[ 5] * marray[ 8],
03005         0.0f,
03006
03007         marray[ 9] * marray[ 2] - marray[10] * marray[ 1],
03008         marray[10] * marray[ 0] - marray[ 8] * marray[ 2],
03009         marray[ 8] * marray[ 1] - marray[ 9] * marray[ 0],
03010         0.0f,
03011
03012         marray[ 1] * marray[ 6] - marray[ 2] * marray[ 5],
03013         marray[ 2] * marray[ 4] - marray[ 0] * marray[ 6],
03014         marray[ 0] * marray[ 5] - marray[ 1] * marray[ 4],
03015         0.0f,
03016
03017         0.0f,
03018         0.0f,
03019         0.0f,
03020         1.0f
03021     };
03022
03023 return *this;
03024 }
03025
03026 //

```

```

03027 // 変換行列：ビュー変換行列を設定する
03028 //
03029 gg::GgMatrix& gg::GgMatrix::loadLookat(
03030     GLfloat ex, GLfloat ey, GLfloat ez,
03031     GLfloat tx, GLfloat ty, GLfloat tz,
03032     GLfloat ux, GLfloat uy, GLfloat uz
03033 )
03034 {
03035     // z 軸 = e - t
03036     const auto zx{ ex - tx };
03037     const auto zy{ ey - ty };
03038     const auto zz{ ez - tz };
03039
03040     // z 軸の長さ
03041     const auto z{ sqrtf(zx * zx + zy * zy + zz * zz) };
03042
03043     // x 軸 = u × z 軸
03044     const auto xx{ uy * zz - uz * zy };
03045     const auto xy{ uz * zx - ux * zz };
03046     const auto xz{ ux * zy - uy * zx };
03047
03048     // x 軸の長さ
03049     const auto x{ sqrtf(xx * xx + xy * xy + xz * xz) };
03050
03051     // y 軸 = z 軸 × x 軸
03052     const auto yx{ zy * xz - zz * xy };
03053     const auto yy{ zz * xx - zx * xz };
03054     const auto yz{ zx * xy - zy * xx };
03055
03056     // y 軸の長さ
03057     const auto y{ sqrtf(yx * yx + yy * yy + yz * yz) };
03058
03059     // y 軸の長さをチェック
03060     if (fabs(y) < std::numeric_limits<float>::epsilon()) return *this;
03061
03062     (*this)[ 0] = xx / x;
03063     (*this)[ 1] = yx / y;
03064     (*this)[ 2] = zx / z;
03065     (*this)[ 3] = 0.0f;
03066
03067     (*this)[ 4] = xy / x;
03068     (*this)[ 5] = yy / y;
03069     (*this)[ 6] = zy / z;
03070     (*this)[ 7] = 0.0f;
03071
03072     (*this)[ 8] = xz / x;
03073     (*this)[ 9] = yz / y;
03074     (*this)[10] = zz / z;
03075     (*this)[11] = 0.0f;
03076
03077     (*this)[12] = -(ex * (*this)[ 0] + ey * (*this)[ 4] + ez * (*this)[ 8]);
03078     (*this)[13] = -(ex * (*this)[ 1] + ey * (*this)[ 5] + ez * (*this)[ 9]);
03079     (*this)[14] = -(ex * (*this)[ 2] + ey * (*this)[ 6] + ez * (*this)[10]);
03080     (*this)[15] = 1.0f;
03081
03082     return *this;
03083 }
03084
03085 //
03086 // 変換行列：平行投影変換行列を設定する
03087 //
03088 gg::GgMatrix& gg::GgMatrix::loadOrthogonal(
03089     GLfloat left, GLfloat right,
03090     GLfloat bottom, GLfloat top,
03091     GLfloat zNear, GLfloat zFar
03092 )
03093 {
03094     const auto dx{ right - left };
03095     const auto dy{ top - bottom };
03096     const auto dz{ zFar - zNear };
03097
03098     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03099
03100     *this =
03101     {
03102         2.0f / dx,
03103         0.0f,
03104         0.0f,
03105         0.0f,
03106
03107         0.0f,
03108         2.0f / dy,
03109         0.0f,
03110         0.0f,
03111
03112         0.0f,
03113         0.0f,
03114     };

```

```

03114     -2.0f / dz,
03115     0.0f,
03116
03117     -(right + left) / dx,
03118     -(top + bottom) / dy,
03119     -(zFar + zNear) / dz,
03120     1.0f
03121 };
03122
03123 return *this;
03124 }
03125
03126 //
03127 // 変換行列：透視投影変換行列を設定する
03128 //
03129 gg::GgMatrix& gg::GgMatrix::loadFrustum(
03130     GLfloat left, GLfloat right,
03131     GLfloat bottom, GLfloat top,
03132     GLfloat zNear, GLfloat zFar
03133 )
03134 {
03135     const auto dx{ right - left };
03136     const auto dy{ top - bottom };
03137     const auto dz{ zFar - zNear };
03138
03139     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03140
03141     *this =
03142     {
03143         2.0f * zNear / dx,
03144         0.0f,
03145         0.0f,
03146         0.0f,
03147
03148         0.0f,
03149         2.0f * zNear / dy,
03150         0.0f,
03151         0.0f,
03152
03153         (right + left) / dx,
03154         (top + bottom) / dy,
03155         -(zFar + zNear) / dz,
03156         -1.0f,
03157
03158         0.0f,
03159         0.0f,
03160         -2.0f * zFar * zNear / dz,
03161         0.0f
03162     };
03163
03164     return *this;
03165 }
03166
03167 //
03168 // 変換行列：画角から透視投影変換行列を設定する
03169 //
03170 gg::GgMatrix& gg::GgMatrix::loadPerspective(
03171     GLfloat fovy, GLfloat aspect,
03172     GLfloat zNear, GLfloat zFar
03173 )
03174 {
03175     const auto dz{ zFar - zNear };
03176
03177     if (dz == 0.0f) return *this;
03178
03179     const auto f{ 1.0f / tanf(fovy * 0.5f) };
03180
03181     *this =
03182     {
03183         f / aspect,
03184         0.0f,
03185         0.0f,
03186         0.0f,
03187
03188         0.0f,
03189         f,
03190         0.0f,
03191         0.0f,
03192
03193         0.0f,
03194         0.0f,
03195         -(zFar + zNear) / dz,
03196         -1.0f,
03197
03198         0.0f,
03199         0.0f,
03200         -2.0f * zFar * zNear / dz,

```



```

03201     0.0f
03202 };
03203
03204 return *this;
03205 }
03206
03207 //
03208 // 四元数: GgQuaternion 型の四元数 p, q の積を r に求める
03209 //
03210 void gg::GgQuaternion::multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const
03211 {
03212     r[0] = p[1] * q[2] - p[2] * q[1] + p[0] * q[3] + p[3] * q[0];
03213     r[1] = p[2] * q[0] - p[0] * q[2] + p[1] * q[3] + p[3] * q[1];
03214     r[2] = p[0] * q[1] - p[1] * q[0] + p[2] * q[3] + p[3] * q[2];
03215     r[3] = p[3] * q[3] - p[0] * q[0] - p[1] * q[1] - p[2] * q[2];
03216 }
03217
03218 //
03219 // 四元数: GgQuaternion 型の四元数 q が表す変換行列を m に求める
03220 //
03221 void gg::GgQuaternion::toMatrix(GLfloat* m, const GLfloat* q) const
03222 {
03223     const auto xx{ q[0] * q[0] * 2.0f };
03224     const auto yy{ q[1] * q[1] * 2.0f };
03225     const auto zz{ q[2] * q[2] * 2.0f };
03226     const auto xy{ q[0] * q[1] * 2.0f };
03227     const auto yz{ q[1] * q[2] * 2.0f };
03228     const auto zx{ q[2] * q[0] * 2.0f };
03229     const auto xw{ q[0] * q[3] * 2.0f };
03230     const auto yw{ q[1] * q[3] * 2.0f };
03231     const auto zw{ q[2] * q[3] * 2.0f };
03232
03233     m[ 0] = 1.0f - yy - zz;
03234     m[ 1] = xy + zw;
03235     m[ 2] = zx - yw;
03236     m[ 4] = xy - zw;
03237     m[ 5] = 1.0f - zz - xx;
03238     m[ 6] = yz + xw;
03239     m[ 8] = zx + yw;
03240     m[ 9] = yz - xw;
03241     m[10] = 1.0f - xx - yy;
03242     m[ 3] = m[ 7] = m[11] = m[12] = m[13] = m[14] = 0.0f;
03243     m[15] = 1.0f;
03244 }
03245
03246 //
03247 // 四元数: 回転変換行列 a が表す四元数を q に求める
03248 //
03249 void gg::GgQuaternion::toQuaternion(GLfloat* q, const GLfloat* a) const
03250 {
03251     const auto tr{ a[0] + a[5] + a[10] + a[15] };
03252
03253     if (tr > 0.0f)
03254     {
03255         q[3] = sqrtf(tr) * 0.5f;
03256         q[0] = (a[6] - a[9]) * 0.25f / q[3];
03257         q[1] = (a[8] - a[2]) * 0.25f / q[3];
03258         q[2] = (a[1] - a[4]) * 0.25f / q[3];
03259     }
03260 }
03261
03262 //
03263 // 四元数: 球面線形補間 p に q と r を t で補間した四元数を求める
03264 //
03265 void gg::GgQuaternion::slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const
03266 {
03267     const auto qr{ ggDot3(q, r) };
03268     const auto ss{ 1.0f - qr * qr };
03269
03270     if (ss == 0.0f)
03271     {
03272         if (p != q)
03273         {
03274             p[0] = q[0];
03275             p[1] = q[1];
03276             p[2] = q[2];
03277             p[3] = q[3];
03278         }
03279     }
03280     else
03281     {
03282         const auto sp{ sqrtf(ss) };
03283         const auto ph{ acosf(qr) };
03284         const auto pt{ ph * t };
03285         const auto t1{ sinf(pt) / sp };
03286         const auto t0{ sinf(ph - pt) / sp };
03287     }

```

```

03288     p[0] = q[0] * t0 + r[0] * t1;
03289     p[1] = q[1] * t0 + r[1] * t1;
03290     p[2] = q[2] * t0 + r[2] * t1;
03291     p[3] = q[3] * t0 + r[3] * t1;
03292 }
03293 }
03294
03295 //
03296 // 四元数: (x, y, z) を軸とし角度 a 回転する四元数を求める
03297 //
03298 gg::GgQuaternion& gg::GgQuaternion::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03299 {
03300     const auto l{ x * x + y * y + z * z };
03301     const auto w{ cosf(a * 0.5f) };
03302
03303     if (fabs(l) > std::numeric_limits<float>::epsilon())
03304     {
03305         const auto s{ sinf(a) / sqrtf(l) };
03306
03307         *this = GgQuaternion{ x * s, y * s, z * s, w };
03308     }
03309     else
03310     {
03311         *this = GgQuaternion{ 0.0f, 0.0f, 0.0f, w };
03312     }
03313
03314     return *this;
03315 }
03316
03317 //
03318 // x 軸中心に角度 a 回転する四元数を格納する
03319 //
03320 gg::GgQuaternion& gg::GgQuaternion::loadRotateX(GLfloat a)
03321 {
03322     const auto t{ a * 0.5f };
03323
03324     *this = GgQuaternion{ sinf(t), 0.0f, 0.0f, cosf(t) };
03325
03326     return *this;
03327 }
03328
03329 //
03330 // y 軸中心に角度 a 回転する四元数を格納する
03331 //
03332 gg::GgQuaternion& gg::GgQuaternion::loadRotateY(GLfloat a)
03333 {
03334     const auto t{ a * 0.5f };
03335
03336     *this = GgQuaternion{ 0.0f, sinf(t), 0.0f, cosf(t) };
03337
03338     return *this;
03339 }
03340
03341 //
03342 // z 軸中心に角度 a 回転する四元数を格納する
03343 //
03344 gg::GgQuaternion& gg::GgQuaternion::loadRotateZ(GLfloat a)
03345 {
03346     const auto t{ a * 0.5f };
03347
03348     *this = GgQuaternion{ 0.0f, 0.0f, sinf(t), cosf(t) };
03349
03350     return *this;
03351 }
03352
03353 //
03354 // 四元数: オイラー角 (heading, pitch, roll) にもとづいて四元数を求める
03355 //
03356 gg::GgQuaternion& gg::GgQuaternion::loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll)
03357 {
03358     GgQuaternion h, p, r;
03359
03360     h.loadRotateY(heading);
03361     p.loadRotateX(pitch);
03362     r.loadRotateZ(roll);
03363
03364     *this = h * p * r;
03365
03366     return *this;
03367 }
03368
03369 //
03370 // 四元数: 正規化して格納する
03371 //
03372 gg::GgQuaternion& gg::GgQuaternion::loadNormalize(const GLfloat* a)
03373 {
03374     *this = a;

```

```

03375
03376     ggNormalize4(data());
03377
03378     return *this;
03379 }
03380
03381 //
03382 // 四元数：共役四元数を格納する
03383 //
03384 gg::GgQuaternion& gg::GgQuaternion::loadConjugate(const GLfloat* a)
03385 {
03386     // w 要素を反転する
03387     data()[0] = a[0];
03388     data()[1] = a[1];
03389     data()[2] = a[2];
03390     data()[3] = -a[3];
03391
03392     return *this;
03393 }
03394
03395 //
03396 // 四元数：逆元を格納する
03397 //
03398 gg::GgQuaternion& gg::GgQuaternion::loadInvert(const GLfloat* a)
03399 {
03400     // ノルムの二乗を求める
03401     const auto l(ggDot4(a, a));
03402
03403     if (l > 0.0f)
03404     {
03405         // 共役四元数を求める
03406         GgQuaternion r;
03407         r.loadConjugate(a);
03408
03409         // ノルムの二乗で割る
03410         data()[0] = r[0] / l;
03411         data()[1] = r[1] / l;
03412         data()[2] = r[2] / l;
03413         data()[3] = r[3] / l;
03414     }
03415
03416     return *this;
03417 }
03418
03419 //
03420 // 簡易トラックボール処理：リセット
03421 //
03422 void gg::GgTrackball::reset(const GgQuaternion& q)
03423 {
03424     // ドラッグ中ではない
03425     drag = false;
03426
03427     // 単位クォーターニオンに初期値を与える
03428     *this = cq = q;
03429
03430     // 回転行列を初期化する
03431     GgQuaternion::getMatrix(rt);
03432 }
03433
03434 //
03435 // 簡易トラックボール処理：トラックボールする領域の設定
03436 //
03437 //   Reshape コールバック (resize) の中で実行する
03438 //   (w, h) ウィンドウサイズ
03439 //
03440 void gg::GgTrackball::region(GLfloat w, GLfloat h)
03441 {
03442     // マウスポインタ位置のウィンドウ内の相対的位置への換算用
03443     scale[0] = 2.0f / w;
03444     scale[1] = 2.0f / h;
03445 }
03446
03447 //
03448 // 簡易トラックボール処理：ドラッグ開始時の処理
03449 //
03450 //   マウスボタンを押したときに実行する
03451 //   (x, y) 現在のマウス位置
03452 //
03453 void gg::GgTrackball::begin(GLfloat x, GLfloat y)
03454 {
03455     // ドラッグ開始
03456     drag = true;
03457
03458     // ドラッグ開始点を記録する
03459     start[0] = x;
03460     start[1] = y;
03461 }

```

```

03462
03463 //
03464 // 簡易トラックボール処理：ドラッグ中の処理
03465 //
03466 //   マウスのドラッグ中に実行する
03467 //   (x, y) 現在のマウス位置
03468 //
03469 void gg::GgTrackball::motion(GLfloat x, GLfloat y)
03470 {
03471     // ドラッグ中でなければ何もしない
03472     if (!drag) return;
03473
03474     // マウスポインタの位置のドラッグ開始位置からの変位
03475     const auto dx{ (x - start[0]) * scale[0] };
03476     const auto dy{ (y - start[1]) * scale[1] };
03477
03478     // マウスポインタの位置のドラッグ開始位置からの距離
03479     const auto a{ hypot(dx, dy) };
03480
03481     // マウスポインタの位置がドラッグ開始位置から移動していれば
03482     if (a > 0.0)
03483     {
03484         // 現在の回転の四元数に作った四元数を掛けて合成する
03485         *this = ggRotateQuaternion(dy, dx, 0.0f, a * 3.1415926536f) * cq;
03486
03487         // 合成した四元数から回転の変換行列を求める
03488         GgQuaternion::getMatrix(rt);
03489     }
03490 }
03491
03492 //
03493 // 簡易トラックボール処理：回転角の修正
03494 //
03495 //   現在の回転角を修正する
03496 //   q 修正分の回転角を表す四元数
03497 //
03498 void gg::GgTrackball::rotate(const GgQuaternion& q)
03499 {
03500     // ドラッグ中なら何もしない
03501     if (drag) return;
03502
03503     // 保存されている四元数に修正分の四元数を掛けて合成する
03504     *this = q * cq;
03505
03506     // 合成した四元数から回転の変換行列を求める
03507     GgQuaternion::getMatrix(rt);
03508
03509     // 誤差を吸収するために正規化して保存する
03510     cq = normalize();
03511 }
03512
03513 //
03514 // 簡易トラックボール処理：停止時の処理
03515 //
03516 //   マウスボタンを離れたときに実行する
03517 //   (x, y) 現在のマウス位置
03518 //
03519 void gg::GgTrackball::end(GLfloat x, GLfloat y)
03520 {
03521     // ドラッグ終了点における回転を求める
03522     motion(x, y);
03523
03524     // 誤差を吸収するために正規化して保存する
03525     cq = normalize();
03526
03527     // ドラッグ終了
03528     drag = false;
03529 }
03530
03531 //
03532 // 配列に格納された画像の内容を TGA ファイルに保存する
03533 //
03534 //   name ファイル名
03535 //   buffer 画像データ
03536 //   width 画像の横の画素数
03537 //   height 画像の縦の画素数
03538 //   depth 画像の 1 画素のバイト数
03539 //   戻り値 保存に成功すれば true, 失敗すれば false
03540 //
03541 bool gg::ggSaveTga(
03542     const std::string& name,
03543     const void* buffer,
03544     unsigned int width,
03545     unsigned int height,
03546     unsigned int depth
03547 )
03548 {

```

```

03549 // ファイルを開く
03550 std::ofstream file{ Utf8ToTChar(name), std::ios::binary };
03551
03552 // ファイルが開けなかったら戻る
03553 if (file.fail()) return false;
03554
03555 // 画像のヘッダ
03556 const unsigned char type{ static_cast<unsigned char>(depth == 0 ? 0 : depth < 3 ? 3 : 2) };
03557 const unsigned char alpha{ static_cast<unsigned char>(depth == 2 || depth == 4 ? 8 : 0) };
03558 const unsigned char header[18]
03559 {
03560     0, // ID length
03561     0, // Color map type (none)
03562     type, // Image Type (2:RGB, 3:Grayscale)
03563     0, 0, // Offset into the color map table
03564     0, 0, // Number of color map entries
03565     0, // Number of a color map entry bits per pixel
03566     0, 0, // Horizontal image position
03567     0, 0, // Vertical image position
03568     static_cast<unsigned char>(width & 0xff),
03569     static_cast<unsigned char>(width >> 8),
03570     static_cast<unsigned char>(height & 0xff),
03571     static_cast<unsigned char>(height >> 8),
03572     static_cast<unsigned char>(depth * 8),
03573     alpha // Image descriptor
03574 };
03575
03576 // ヘッダを書き込む
03577 file.write(reinterpret_cast<const char*>(header), sizeof header);
03578
03579 // ヘッダの書き込みに失敗したら戻る
03580 if (file.bad()) return false;
03581
03582 // データを書き込む
03583 const unsigned int size{ width * height * depth };
03584 if (type == 2)
03585 {
03586     // フルカラー
03587     std::vector<char> temp(size);
03588     for (GLuint i = 0; i < size; i += depth)
03589     {
03590         temp[i + 2] = static_cast<const char*>(buffer)[i + 0];
03591         temp[i + 1] = static_cast<const char*>(buffer)[i + 1];
03592         temp[i + 0] = static_cast<const char*>(buffer)[i + 2];
03593         if (depth == 4) temp[i + 3] = static_cast<const char*>(buffer)[i + 3];
03594     }
03595     file.write(temp.data(), size);
03596 }
03597 else if (type == 3)
03598 {
03599     // グレースケール
03600     file.write(static_cast<const char*>(buffer), size);
03601 }
03602
03603 // フッタを書き込む
03604 constexpr char footer[] = "\0\0\0\0\0\0\0TRUEVISION-XFILE.";
03605 file.write(footer, sizeof footer);
03606
03607 // データの書き込みに失敗していなければ true を返す
03608 return file.bad() != false;
03609 }
03610
03611 //
03612 // カラーバッファの内容を TGA ファイルに保存する
03613 //
03614 // name 保存するファイル名
03615 // 戻り値 保存に成功すれば true, 失敗すれば false
03616 //
03617 bool gg::ggSaveColor(const std::string& name)
03618 {
03619     // 現在のビューポートのサイズを得る
03620     GLint viewport[4];
03621     glGetIntegerv(GL_VIEWPORT, viewport);
03622
03623     // ビューポートのサイズ分のメモリを確保する
03624     std::vector<GLubyte> buffer(viewport[2] * viewport[3] * 3);
03625
03626     // 画面表示の完了を待つ
03627     glFinish();
03628
03629     // カラーバッファを読み込む
03630     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03631         GL_RGB, GL_UNSIGNED_BYTE, buffer.data());
03632
03633     // 読み込んだデータをファイルに書き込む
03634     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 3);
03635 }

```

```

03636
03637 //
03638 // デブスバッファの内容を TGA ファイルに保存する
03639 //
03640 // name 保存するファイル名
03641 // 戻り値 保存に成功すれば true, 失敗すれば false
03642 //
03643 bool gg::ggSaveDepth(const std::string& name)
03644 {
03645     // 現在のビューポートのサイズを得る
03646     GLint viewport[4];
03647     glGetIntegerv(GL_VIEWPORT, viewport);
03648
03649     // ビューポートのサイズ分のメモリを確保する
03650     std::vector<GLubyte> buffer(viewport[2] * viewport[3]);
03651
03652     // 画面表示の完了を待つ
03653     glFinish();
03654
03655     // デブスバッファを読み込む
03656     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03657                 GL_DEPTH_COMPONENT, GL_UNSIGNED_BYTE, buffer.data());
03658
03659     // 読み込んだデータをファイルに書き込む
03660     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 1);
03661 }
03662
03663 //
03664 // TGA ファイル (8/16/24/32bit) を読み込む
03665 //
03666 // name 読み込むファイル名
03667 // pWidth 読み込んだファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03668 // pHeight 読み込んだファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03669 // pFormat 読み込んだファイルのフォーマットの格納先のポインタ (nullptr なら格納しない)
03670 // image 読み込んだ画像を格納する vector
03671 // 戻り値 読み込みに成功すれば true, 失敗すれば false
03672 //
03673 bool gg::ggReadImage(
03674     const std::string& name,
03675     std::vector<GLubyte>& image,
03676     GLsizei* pWidth,
03677     GLsizei* pHeight,
03678     GLenum* pFormat
03679 )
03680 {
03681     // ファイルを開く
03682     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
03683
03684     // ファイルが開けなかったら戻る
03685     if (file.fail()) return false;
03686
03687     // ヘッダを読み込む
03688     unsigned char header[18];
03689     file.read(reinterpret_cast<char*>(header), sizeof header);
03690
03691     // ヘッダの読み込みに失敗したら戻る
03692     if (file.bad()) return false;
03693
03694     // 深度
03695     const auto depth{ header[16] / 8 };
03696     switch (depth)
03697     {
03698     case 1:
03699         *pFormat = GL_RED;
03700         break;
03701     case 2:
03702         *pFormat = GL_RG;
03703         break;
03704     case 3:
03705         *pFormat = GL_BGR;
03706         break;
03707     case 4:
03708         *pFormat = GL_BGRA;
03709         break;
03710     default:
03711         // 取り扱えないフォーマットだったら戻る
03712         return false;
03713     }
03714
03715     // 画像の縦横の画素数
03716     *pWidth = header[13] << 8 | header[12];
03717     *pHeight = header[15] << 8 | header[14];
03718
03719     // データサイズ
03720     const auto size{ *pWidth * *pHeight * depth };
03721
03722     // サイズが小さすぎたら戻る

```

```

03723     if (size < 2) return false;
03724
03725     // 読み込みに使うメモリを確保する
03726     image.resize(size);
03727
03728     // データを読み込む
03729     if (header[2] & 8)
03730     {
03731         // RLE
03732         int p{ 0 };
03733         char c;
03734         while (file.get(c))
03735         {
03736             if (c & 0x80)
03737             {
03738                 // run-length packet
03739                 const auto count{ (c & 0x7f) + 1 };
03740                 if (p + depth * count > size) break;
03741                 char temp[4];
03742                 file.read(temp, depth);
03743                 for (int i = 0; i < count; ++i)
03744                 {
03745                     for (int j = 0; j < depth; ++j) image[p++] = temp[j++];
03746                 }
03747             }
03748             else
03749             {
03750                 // raw packet
03751                 const auto count{ (c + 1) * depth };
03752                 if (p + count > size) break;
03753                 file.read(reinterpret_cast<char*>(image.data() + p), count);
03754                 p += count;
03755             }
03756         }
03757     }
03758     else
03759     {
03760         // 非圧縮
03761         file.read(reinterpret_cast<char*>(image.data()), size);
03762     }
03763
03764     // 読み込みに失敗していなければ true を返す
03765     return file.bad() != false;
03766 }
03767
03768 //
03769 // テクスチャを作成して画像を読み込む
03770 //
03771 // image 画像データ, nullptr ならメモリの確保だけを行う
03772 // width 画像の横の画素数
03773 // height 画像の縦の画素数
03774 // format 画像データのフォーマット
03775 // type 画像のデータ型
03776 // internal テクスチャの内部フォーマット
03777 // wrap テクスチャのラッピングモード
03778 // swizzle true ならテクスチャの赤と青を入れ替える
03779 // 戻り値 テクスチャ名
03780 //
03781 GLuint gg::ggLoadTexture(
03782     const GLvoid* image,
03783     GLsizei width,
03784     GLsizei height,
03785     GLenum format,
03786     GLenum type,
03787     GLenum internal,
03788     GLenum wrap,
03789     bool swizzle
03790 )
03791 {
03792     // テクスチャを作成する
03793     GLuint texture;
03794     glGenTextures(1, &texture);
03795
03796     // 作成したテクスチャを 2D テクスチャとしてバインドする
03797     glBindTexture(GL_TEXTURE_2D, texture);
03798
03799     // アルファチャンネルがついていれば 4 バイト境界に設定する
03800     glPixelStorei(GL_UNPACK_ALIGNMENT, format == GL_RGBA || format == GL_BGRA ? 4 : 1);
03801
03802     // テクスチャを割り当てる
03803     glTexImage2D(GL_TEXTURE_2D, 0, internal, width, height, 0, format, type, image);
03804
03805     // バイリニア（ミップマップなし）、エッジでクランプ
03806     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
03807     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
03808     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, wrap);
03809     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, wrap);

```

```

03810
03811     if (swizzle)
03812     {
03813         // テクスチャのサンプリング時に赤と青を入れ替える
03814         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, GL_BLUE);
03815         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, GL_RED);
03816     }
03817
03818     // テクスチャ名を返す
03819     return texture;
03820 }
03821
03822 //
03823 // テクスチャを作成して TGA フォーマットの画像ファイルを読み込む
03824 //
03825 //     name TGA ファイル名
03826 //     pWidth 読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03827 //     pHeight 読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03828 //     internal テクスチャの内部フォーマット
03829 //     wrap テクスチャのラッピングモード
03830 //     戻り値 テクスチャ名
03831 //
03832 GLuint gg::ggLoadImage(
03833     const std::string& name,
03834     GLsizei* pWidth,
03835     GLsizei* pHeight,
03836     GLenum internal,
03837     GLenum wrap
03838 )
03839 {
03840     // 画像データ
03841     std::vector<GLubyte> image;
03842
03843     // 画像サイズ
03844     GLsizei width, height;
03845
03846     // 画像フォーマット
03847     GLenum format;
03848
03849     // 画像を読み込む
03850     ggReadImage(name, image, &width, &height, &format);
03851
03852     // 画像が読み込めなかったら戻る
03853     if (image.empty()) return 0;
03854
03855     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
03856     if (internal == 0) internal = format;
03857
03858     // テクスチャに読み込む (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA)
03859     const auto tex{ ggLoadTexture(image.data(), width, height,
03860         format, GL_UNSIGNED_BYTE, internal, wrap, false) };
03861
03862     // 画像サイズを返す
03863     if (pWidth) *pWidth = width;
03864     if (pHeight) *pHeight = height;
03865
03866     // テクスチャ名を返す
03867     return tex;
03868 }
03869
03870 //
03871 // グレースケール画像 (8bit) から法線マップのデータを作成する
03872 //
03873 //     width 高さマップのグレースケール画像 hmap の横の画素数
03874 //     height 高さマップのグレースケール画像のデータ hmap の縦の画素数
03875 //     stride データの間隔
03876 //     hmap グレースケール画像のデータ
03877 //     nz 法線の z 成分の割合
03878 //     internal テクスチャの内部フォーマット
03879 //     nmap 法線マップを格納する vector
03880 //
03881 void gg::ggCreateNormalMap(
03882     const GLubyte* hmap,
03883     GLsizei width,
03884     GLsizei height,
03885     GLenum format,
03886     GLfloat nz,
03887     GLenum internal,
03888     std::vector<GgVector>& nmap
03889 )
03890 {
03891     // メモリサイズ
03892     const GLsizei size{ width * height };
03893
03894     // 法線マップのメモリを確保する
03895     nmap.resize(size);
03896

```



```

03897 // 画素のバイト数
03898 GLint stride;
03899 switch (format)
03900 {
03901 case GL_RED:
03902     stride = 1;
03903     break;
03904 case GL_RG:
03905     stride = 2;
03906     break;
03907 case GL_RGB:
03908     stride = 3;
03909     break;
03910 case GL_RGBA:
03911     stride = 4;
03912     break;
03913 default:
03914     stride = 1;
03915     break;
03916 }
03917
03918 // 法線マップの作成
03919 for (GLsizei i = 0; i < size; ++i)
03920 {
03921     const auto x{ i % width };
03922     const auto y{ i - x };
03923     const auto u0{ (y + (x - 1 + width) % width) * stride };
03924     const auto u1{ (y + (x + 1) % width) * stride };
03925     const auto v0{ ((y - width + size) % size + x) * stride };
03926     const auto v1{ ((y + width) % size + x) * stride };
03927
03928     // 隣接する画素との値の差を法線の成分に用いる
03929     nmap[i][0] = static_cast<GLfloat>(hmap[u0] - hmap[u1]);
03930     nmap[i][1] = static_cast<GLfloat>(hmap[v0] - hmap[v1]);
03931     nmap[i][2] = nz;
03932     nmap[i][3] = hmap[i * stride];
03933
03934     // 法線ベクトルを正規化する
03935     ggNormalize3(&nmap[i]);
03936 }
03937
03938 // 内部フォーマットが浮動小数点テクスチャでなければ [0,1] に正規化する
03939 if (
03940     internal != GL_RGB16F &&
03941     internal != GL_RGBA16F &&
03942     internal != GL_RGB32F &&
03943     internal != GL_RGBA32F
03944 )
03945 {
03946     for (GLsizei i = 0; i < size; ++i)
03947     {
03948         nmap[i][0] = nmap[i][0] * 0.5f + 0.5f;
03949         nmap[i][1] = nmap[i][1] * 0.5f + 0.5f;
03950         nmap[i][2] = nmap[i][2] * 0.5f + 0.5f;
03951         nmap[i][3] *= 0.0039215686f; // == 1/255
03952     }
03953 }
03954 }
03955
03956 //
03957 // TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する
03958 //
03959 //     name TGA ファイル名
03960 //     nz 作成した法線の z 成分の割合
03961 //     pWidth 読み込んだ画像の横の画素数の格納先のポインタ (nullptr なら格納しない)
03962 //     pHeight 読み込んだ画像の縦の画素数の格納先のポインタ (nullptr なら格納しない)
03963 //     internal テクスチャの内部フォーマット
03964 //     戻り値 テクスチャ名
03965 //
03966 GLuint gg::ggLoadHeight(
03967     const std::string& name,
03968     GLfloat nz,
03969     GLsizei* pWidth,
03970     GLsizei* pHeight,
03971     GLenum internal
03972 )
03973 {
03974     // 画像データ
03975     std::vector<GLubyte> hmap;
03976
03977     // 画像サイズ
03978     GLsizei width, height;
03979
03980     // 画像フォーマット
03981     GLenum format;
03982
03983     // 高さマップの画像を読み込む

```

```

03984     ggReadImage(name, hmap, &width, &height, &format);
03985
03986     // 画像が読み込めなかったら戻る
03987     if (hmap.empty()) return 0;
03988
03989     // 法線マップ
03990     std::vector<GgVector> nmap;
03991
03992     // 法線マップを作成する
03993     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
03994
03995     // 画像サイズを返す
03996     if (pWidth) *pWidth = width;
03997     if (pHeight) *pHeight = height;
03998
03999     // テクスチャを作成して返す
04000     return ggLoadTexture(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal, GL_REPEAT);
04001 }
04002
04003 //
04004 // テクスチャの赤と青を交換する
04005 //
04006 void gg::GgTexture::swapRandB(bool swap) const
04007 {
04008     const auto swizzle
04009     {
04010         swap ?
04011             std::pair<GLint, GLint>{ GL_BLUE, GL_RED } :
04012             std::pair<GLint, GLint>{ GL_RED, GL_BLUE }
04013     };
04014
04015     glBindTexture(GL_TEXTURE_2D, texture);
04016     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, swizzle.first);
04017     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, swizzle.second);
04018     glBindTexture(GL_TEXTURE_2D, 0);
04019 }
04020
04021 //
04022 // TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する
04023 //
04024 //     name 読み込むファイル名
04025 //     internal glTexImage2D() に指定するテクスチャの内部フォーマット. 0 なら外部フォーマットに合わせる
04026 //     戻り値 テクスチャの作成に成功すれば true, 失敗すれば false
04027 //
04028 void gg::GgColorTexture::load(
04029     const std::string& name,
04030     GLenum internal,
04031     GLenum wrap
04032 )
04033 {
04034     // 画像データ
04035     std::vector<GLubyte> image;
04036
04037     // 画像サイズ
04038     GLsizei width, height;
04039
04040     // 画像フォーマット
04041     GLenum format;
04042
04043     // 画像を読み込む
04044     ggReadImage(name, image, &width, &height, &format);
04045
04046     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
04047     if (internal == 0) internal = format;
04048
04049     // テクスチャを作成する (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA)
04050     texture = std::make_shared<GgTexture>(image.data(), width, height,
04051         format, GL_UNSIGNED_BYTE, internal, wrap, false);
04052 }
04053
04054 //
04055 // TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する
04056 //
04057 //     name 画像ファイル名
04058 //     width テクスチャとして用いる画像データの横幅
04059 //     height テクスチャとして用いる画像データの高さ
04060 //     format テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA)
04061 //     nz 法線マップの z 成分の値
04062 //     internal テクスチャの内部フォーマット
04063 //
04064 void gg::GgNormalTexture::load(
04065     const std::string& name,
04066     GLfloat nz,
04067     GLenum internal
04068 )
04069 {
04070     // 高さマップ

```

```

04071     std::vector<GLubyte> hmap;
04072
04073     // 画像サイズ
04074     GLsizei width, height;
04075
04076     // 画像フォーマット
04077     GLenum format;
04078
04079     // 高さマップの画像を読み込む
04080     ggReadImage(name, hmap, &width, &height, &format);
04081
04082     // 法線マップ
04083     std::vector<GgVector> nmap;
04084
04085     // 法線マップを作成する
04086     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
04087 }
04088
04090
04091 //
04092 // OBJ ファイルの読み込みに使うデータ型と関数
04093 //
04094 namespace gg
04095 {
04096     // GLfloat 型の 2 要素のベクトル
04097     using vec2 = std::array<GLfloat, 2>;
04098
04099     // GLfloat 型の 3 要素のベクトル
04100     using vec3 = std::array<GLfloat, 3>;
04101
04102     // 三角形データ
04103     struct fidx
04104     {
04105         GLuint p[3];           // 頂点座標番号
04106         GLuint n[3];           // 頂点法線番号
04107         GLuint t[3];           // テクスチャ座標番号
04108         bool smooth;           // スムーズシェーディングの有無
04109     };
04110
04111     // ポリゴングループ
04112     struct fgrp
04113     {
04114         GLuint nextgroup;      // 次のポリゴングループの最初の三角形番号
04115         GLuint mtlno;           // このポリゴングループの材質番号
04116
04117         // コンストラクタ
04118         fgrp(GLuint nextgroup, GLuint mtlno) :
04119             nextgroup(nextgroup),
04120             mtlno(mtlno)
04121         {
04122         }
04123     };
04124
04125     // デフォルトの材質
04126     constexpr GgSimpleShader::Material defaultMaterial
04127     {
04128         { 0.1f, 0.1f, 0.1f, 1.0f },
04129         { 0.6f, 0.6f, 0.6f, 0.0f },
04130         { 0.3f, 0.3f, 0.3f, 0.0f },
04131         60.0f
04132     };
04133
04134     // デフォルトの材質名
04135     constexpr char defaultMaterialName[] = "_default.";
04136
04137     //
04138     // Alias OBJ 形式の MTL ファイルを読み込む
04139     //
04140     // mtlpath MTL ファイルのパス名
04141     // mtl 読み込んだ材質名をキー、材質番号を値にした map
04142     // material 材質データ
04143     //
04144     static bool ggLoadMtl(const std::string& mtlpath,
04145         std::map<std::string, GLuint>& mtl,
04146         std::vector<GgSimpleShader::Material>& material)
04147     {
04148         // MTL ファイルが無ければ戻る
04149         std::ifstream mtlfile{ Utf8ToTChar(mtlpath), std::ios::binary };
04150         if (!mtlfile)
04151         {
04152             #if defined(DEBUG)
04153                 std::cerr << "Warning: Can't open MTL file: " << mtlpath << std::endl;
04154             #endif
04155             return false;
04156         }
04157
04158         // 一行読み込み用のバッファ

```

```

04159     std::string mtlname;
04160
04161     // 材質名（ループの外に置く）
04162     std::string mtlname{ defaultMaterialName };
04163
04164     // 現在の材質番号を登録する
04165     mtl[mtlname] = static_cast<GLuint>(material.size());
04166
04167     // 現在の材質にデフォルトの材質を設定する
04168     material.emplace_back(defaultMaterial);
04169
04170     // 材質データを読み込む
04171     while (std::getline(mtlfile, mtlname))
04172     {
04173         // 空行は読み飛ばす
04174         if (mtlname == "") continue;
04175
04176         // 最後の文字が '\r' なら
04177         if (*(mtlname.end() - 1) == '\r')
04178         {
04179             // 最後の文字を削除する
04180             mtlname.erase(mtlname.end() - 1, mtlname.end());
04181
04182             // 空行になったら読み飛ばす
04183             if (mtlname == "") continue;
04184         }
04185
04186         // 読み込んだ行を文字列ストリームにする
04187         std::istringstream mtlstr{ mtlname };
04188
04189         // オペレータ
04190         std::string mtlop;
04191
04192         // 文字列ストリームから材質パラメータの種類を取り出す
04193         mtlstr >> mtlop;
04194
04195         // '#' で始まる場合はコメントとして行末まで読み飛ばす
04196         if (mtlop[0] == '#') continue;
04197
04198         if (mtlop == "newmtl")
04199         {
04200             // 新規作成する材質名を取り出す
04201             mtlstr >> mtlname;
04202
04203             // 材質名が既に存在するかどうか調べる
04204             const auto m{ mtl.find(mtlname) };
04205             if (m == mtl.end())
04206             {
04207                 // 存在しないので新規作成する材質の番号をその材質名に割り当てる
04208                 mtl[mtlname] = static_cast<GLuint>(material.size());
04209
04210                 // 新規作成する材質にデフォルトの材質を設定しておく
04211                 material.emplace_back(defaultMaterial);
04212             }
04213
04214             #if defined(DEBUG)
04215                 std::cerr << "newmtl: " << mtlname << std::endl;
04216             #endif
04217         }
04218         else if (mtlop == "Ka")
04219         {
04220             // 環境光の反射係数を登録する
04221             mtlstr
04222                 >> material.back().ambient[0]
04223                 >> material.back().ambient[1]
04224                 >> material.back().ambient[2];
04225         }
04226         else if (mtlop == "Kd")
04227         {
04228             // 拡散反射係数を登録する
04229             mtlstr
04230                 >> material.back().diffuse[0]
04231                 >> material.back().diffuse[1]
04232                 >> material.back().diffuse[2];
04233         }
04234         else if (mtlop == "Ks")
04235         {
04236             // 鏡面反射係数を登録する
04237             mtlstr
04238                 >> material.back().specular[0]
04239                 >> material.back().specular[1]
04240                 >> material.back().specular[2];
04241         }
04242         else if (mtlop == "Ns")
04243         {
04244             // 輝き係数を登録する
04245             GLfloat shininess;

```

```

04246         mtlstr >> shininess;
04247         material.back().shininess = shininess * 0.1f;
04248     }
04249     else if (mtlop == "d")
04250     {
04251         // 不透明度を登録する
04252         mtlstr >> material.back().ambient[3];
04253     }
04254 }
04255
04256 // MTL ファイルの読み込みに失敗したら戻る
04257 if (mtlfile.bad())
04258 {
04259 #if defined(DEBUG)
04260     std::cerr << "Warning: Can't read MTL file: " << mtlpath << std::endl;
04261 #endif
04262     return false;
04263 }
04264
04265 // MTL ファイルの読み込みに成功した
04266 return true;
04267 }
04268
04269 //
04270 // Alias OBJ 形式のファイルを解析する
04271 //
04272 // name Alias OBJ 形式のファイルのファイル名
04273 // group 同じ材質を割り当てるポリゴングループ
04274 // mtl 読み込んだ材質名をキーにした map
04275 // pos 頂点の位置
04276 // norm 頂点の法線
04277 // tex 頂点のテクスチャ座標
04278 // face 三角形のデータ
04279 //
04280 static bool ggParseObj(
04281     const std::string& name,
04282     std::vector<fgrp>& group,
04283     std::vector<GgSimpleShader::Material>& material,
04284     std::vector<vec3>& pos,
04285     std::vector<vec3>& norm,
04286     std::vector<vec2>& tex,
04287     std::vector<fidx>& face,
04288     bool normalize
04289 )
04290 {
04291     // ファイルパスからディレクトリ名を取り出す
04292     const std::string path{ name };
04293     const size_t base{ path.find_last_of("/\\") };
04294     const std::string dirname{ (base == std::string::npos) ? "" : path.substr(0, base + 1) };
04295
04296     // OBJ ファイルを読み込む
04297     std::ifstream file{ Utf8ToTChar(path) };
04298
04299     // 読み込みに失敗したら戻る
04300     if (file.fail())
04301     {
04302 #if defined(DEBUG)
04303         std::cerr << "Error: Can't open OBJ file: " << path << std::endl;
04304 #endif
04305         return false;
04306     }
04307
04308     // ポリゴングループの最初の三角形番号
04309     GLsizei startgroup{ static_cast<GLsizei>(group.size()) };
04310
04311     // スムーズシェーディングのスイッチ
04312     bool smooth{ false };
04313
04314     // 材質のテーブル
04315     std::map<std::string, GLuint> mtl;
04316
04317     // 現在の材質名 (ループの外で宣言する)
04318     std::string mtlname;
04319
04320     // 座標値の最小値・最大値
04321     vec3 bmin{ FLT_MAX }, bmax{ -FLT_MAX };
04322
04323     // 一行読み込み用のバッファ
04324     std::string line;
04325
04326     // データの読み込み
04327     while (std::getline(file, line))
04328     {
04329         // 空行は読み飛ばす
04330         if (line == "") continue;
04331
04332         // 最後の文字が '\r' なら

```

```

04333     if (*(line.end() - 1) == '\\r')
04334     {
04335         // 最後の文字を削除する
04336         line.erase(line.end() - 1, line.end());
04337
04338         // 空行になったら読み飛ばす
04339         if (line == "") continue;
04340     }
04341
04342     // 一行を文字列ストリームに入れる
04343     std::istringstream str(line);
04344
04345     // 最初のトークンを命令 (op) とみなす
04346     std::string op;
04347     str >> op;
04348
04349     if (op[0] == '#') continue;
04350
04351     if (op == "v")
04352     {
04353         // 頂点位置
04354         vec3 v;
04355
04356         // 頂点位置はスペースで区切られている
04357         str >> v[0] >> v[1] >> v[2];
04358
04359         // 頂点位置を記録する
04360         pos.emplace_back(v);
04361
04362         // 頂点位置の最小値と最大値を求める (AABB)
04363         for (int i = 0; i < 3; ++i)
04364         {
04365             bmin[i] = std::min(bmin[i], v[i]);
04366             bmax[i] = std::max(bmax[i], v[i]);
04367         }
04368     }
04369     else if (op == "vt")
04370     {
04371         // テクスチャ座標
04372         vec2 t;
04373
04374         // 頂点位置はスペースで区切られている
04375         str >> t[0] >> t[1];
04376
04377         // テクスチャ座標を記録する
04378         tex.emplace_back(t);
04379     }
04380     else if (op == "vn")
04381     {
04382         // 頂点法線
04383         vec3 n;
04384
04385         // 頂点法線はスペースで区切られている
04386         str >> n[0] >> n[1] >> n[2];
04387
04388         // 頂点法線を記録する
04389         norm.emplace_back(n);
04390     }
04391     else if (op == "f")
04392     {
04393         // 三角形データ
04394         fidx f;
04395
04396         // スムースシェーディング
04397         f.smooth = smooth;
04398
04399         // 三頂点のそれぞれについて
04400         for (int i = 0; i < 3; ++i)
04401         {
04402             // 1項目取り出す
04403             std::string s;
04404             str >> s;
04405
04406             // 文字の位置
04407             auto c{ s.begin() };
04408
04409             // テクスチャ座標と法線の番号は未定義を表す 0 にしておく
04410             f.p[i] = f.t[i] = f.n[i] = 0;
04411
04412             // 項目の最初の要素は頂点座標番号
04413             while (c != s.end() && isdigit(*c)) f.p[i] = f.p[i] * 10 + *c++ - '0';
04414             if (c == s.end() || *c++ != '/') continue;
04415
04416             // 二つ目の項目はテクスチャ座標
04417             while (c != s.end() && isdigit(*c)) f.t[i] = f.t[i] * 10 + *c++ - '0';
04418             if (c == s.end() || *c++ != '/') continue;
04419

```

```

04420         // 三つ目の項目は法線番号
04421         while (c != s.end() && isdigit(*c)) f.n[i] = f.n[i] * 10 + *c++ - '0';
04422     }
04423
04424     // 三角形データを登録する
04425     face.emplace_back(f);
04426 }
04427 else if (op == "s")
04428 {
04429     // '1' だったらスムーズシェーディング有効
04430     std::string s;
04431     str >> s;
04432     smooth = s == "1";
04433 }
04434 else if (op == "usemtl")
04435 {
04436     // 次のポリゴングループの最初の三角形番号
04437     const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04438
04439     // ポリゴングループに三角形が存在すれば
04440     if (nextgroup > startgroup)
04441     {
04442         // ポリゴングループの三角形数と材質番号を記録する
04443         group.emplace_back(nextgroup, mtl[mtlname]);
04444
04445         // 次のポリゴングループの開始番号を保存しておく
04446         startgroup = nextgroup;
04447     }
04448
04449     // 次に usemtl が来るまで材質名を保持する
04450     str >> mtlname;
04451
04452     // 材質の存在チェック
04453     if (mtl.find(mtlname) == mtl.end())
04454     {
04455         #if defined(DEBUG)
04456         std::cerr << "Warning: Undefined material: " << mtlname << std::endl;
04457         #endif
04458
04459         // デフォルトの材質を割り当てておく
04460         mtlname = defaultMaterialName;
04461     }
04462     #if defined(DEBUG)
04463     else std::cerr << "usemtl: " << mtlname << std::endl;
04464     #endif
04465 }
04466 else if (op == "mtllib")
04467 {
04468     // MTL ファイルのパス名を作る
04469     str >> std::ws;
04470     std::string mtlpath;
04471     std::getline(str, mtlpath);
04472
04473     // MTL ファイルを読み込む
04474     ggLoadMtl(dirname + mtlpath, mtl, material);
04475 }
04476 }
04477
04478 // OBJ ファイルの読み込みに失敗したら戻る
04479 if (file.bad())
04480 {
04481     #if defined(DEBUG)
04482     std::cerr << "Error: Can't read OBJ file: " << path << std::endl;
04483     #endif
04484     return false;
04485 }
04486
04487 // 最後のポリゴングループの次の三角形番号
04488 const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04489 if (nextgroup > startgroup)
04490 {
04491     // 最後のポリゴングループの三角形数と材質を記録する
04492     group.emplace_back(nextgroup, static_cast<GLuint>(mtl[mtlname]));
04493 }
04494
04495 // スムーズシェーディングしない三角形の頂点を追加する
04496 for (auto& f : face)
04497 {
04498     if (!f.smooth)
04499     {
04500         // 三頂点のそれぞれについて
04501         for (int i = 0; i < 3; ++i)
04502         {
04503             // 新しい頂点座標を生成する (std::array の要素は emplace_back できない)
04504             pos.emplace_back(pos[f.p[i] - 1]);
04505             f.p[i] = static_cast<GLuint>(pos.size());
04506         }
04507     }
04508 }

```

```

04507         if (f.t[i] > 0)
04508         {
04509             // 新しいテクスチャ座標を生成する
04510             tex.emplace_back(tex[f.t[i] - 1]);
04511             f.t[i] = static_cast<GLuint>(tex.size());
04512         }
04513         if (f.n[i] > 0)
04514         {
04515             // 新しい法線を生成する
04516             norm.emplace_back(norm[f.n[i] - 1]);
04517             f.n[i] = static_cast<GLuint>(norm.size());
04518         }
04519     }
04520 }
04521 }
04522 }
04523
04524 // 法線データがなければ算出しておく
04525 if (norm.empty())
04526 {
04527     // 法線データ数の初期値は頂点数と同じでスムーズシェーディングのために初期値は 0
04528     norm.resize(pos.size(), { 0.0f, 0.0f, 0.0f });
04529
04530     // 面の法線の算出と頂点法線の算出
04531     for (auto& f : face)
04532     {
04533         // 頂点座標番号
04534         const auto v0{ f.p[0] - 1 };
04535         const auto v1{ f.p[1] - 1 };
04536         const auto v2{ f.p[2] - 1 };
04537
04538         // v1 - v0, v2 - v0 を求める
04539         const GLfloat d1[] { pos[v1][0] - pos[v0][0], pos[v1][1] - pos[v0][1], pos[v1][2] - pos[v0][2] };
04540
04541         const GLfloat d2[] { pos[v2][0] - pos[v0][0], pos[v2][1] - pos[v0][1], pos[v2][2] - pos[v0][2] };
04542     };
04543
04544     // 外積により面法線を求める
04545     vec3 n;
04546     ggCross(n.data(), d1, d2);
04547
04548     if (f.smooth)
04549     {
04550         // スムースシェーディングを行うときは
04551         for (int i = 0; i < 3; ++i)
04552         {
04553             // 面法線を頂点法線に積算する
04554             norm[v0][i] += n[i];
04555             norm[v1][i] += n[i];
04556             norm[v2][i] += n[i];
04557
04558             // 面の各頂点の法線番号は頂点番号と同じにする
04559             f.n[i] = f.p[i];
04560         }
04561     }
04562     else
04563     {
04564         // 面法線を最初の頂点に保存する
04565         norm[v0] = n;
04566         f.n[0] = f.p[0];
04567
04568         // 2 頂点追加
04569         for (int i = 1; i < 3; ++i)
04570         {
04571             norm.emplace_back(n);
04572             f.n[i] = static_cast<GLuint>(norm.size());
04573         }
04574     }
04575
04576     // 頂点の法線ベクトルを正規化する
04577     for (auto& n : norm) ggNormalize3(n.data());
04578 }
04579
04580 // 図形の正規化
04581 if (normalize)
04582 {
04583     // 図形の大きさ
04584     const auto sx{ bmax[0] - bmin[0] };
04585     const auto sy{ bmax[1] - bmin[1] };
04586     const auto sz{ bmax[2] - bmin[2] };
04587
04588     // 図形のスケール
04589     GLfloat s{ sx };
04590     if (sy > s) s = sy;
04591     if (sz > s) s = sz;
04592     const auto scale{ s != 0.0f ? 2.0f / s : 1.0f };

```



```

04592
04593 // 図形の中心位置
04594 const auto cx{ (bmax[0] + bmin[0]) * 0.5f };
04595 const auto cy{ (bmax[1] + bmin[1]) * 0.5f };
04596 const auto cz{ (bmax[2] + bmin[2]) * 0.5f };
04597
04598 // 図形の大きさと位置を正規化する
04599 for (auto& p : pos)
04600 {
04601     p[0] = (p[0] - cx) * scale;
04602     p[1] = (p[1] - cy) * scale;
04603     p[2] = (p[2] - cz) * scale;
04604 }
04605 }
04606
04607 #if defined(DEBUG)
04608     std::cerr
04609     << "[" << name << "]" \n(Parsed) Group: " << group.size() << ", Material: " << mtl.size()
04610     << ", Pos: " << pos.size() << ", Norm: " << norm.size() << ", Tex: " << tex.size()
04611     << ", Face: " << face.size() << "\n";
04612 #endif
04613
04614 // OBJ ファイルの読み込み成功
04615 return true;
04616 }
04617 }
04618
04619 //
04620 //
04621 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Arrays 形式)
04622 //
04623 // name 読み込むOBJ ファイル名
04624 // group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04625 // material 読み込んだデータのポリゴングループごとの材質
04626 // vert 読み込んだデータの頂点属性
04627 // normalize true ならサイズを正規化する
04628 // 戻り値 読み込みに成功したら true
04629 //
04630 bool gg::ggLoadSimpleObj(const std::string& name,
04631     std::vector<std::array<GLuint, 3>>& group,
04632     std::vector<GgSimpleShader::Material>& material,
04633     std::vector<GgVertex>& vert,
04634     bool normalize)
04635 {
04636     // 読み込み用の一時記憶領域
04637     std::vector<fgrp> tgroup;
04638     std::vector<vec3> tpos;
04639     std::vector<vec3> tnorm;
04640     std::vector<vec2> ttex;
04641     std::vector<fidx> tface;
04642
04643     // OBJ ファイルを解析する
04644     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04645
04646     // 頂点属性データのメモリを確保する
04647     vert.reserve(vert.size() + tface.size() * 3);
04648
04649     // ポリゴングループデータのメモリを確保する
04650     group.reserve(group.size() + tgroup.size());
04651     material.reserve(material.size() + tgroup.size());
04652
04653     // ポリゴングループの最初の三角形番号
04654     GLuint startgroup{ 0 };
04655
04656     // ポリゴングループデータの作成
04657     for (auto& g : tgroup)
04658     {
04659         // このポリゴングループの最初の頂点番号と頂点数・材質番号
04660         std::array<GLuint, 3> v;
04661
04662         // このポリゴングループの最初の頂点番号を保存する
04663         v[0] = static_cast<GLuint>(vert.size());
04664
04665         // 三角形ごとの頂点データの作成
04666         for (GLuint j = startgroup; j < g.nextgroup; ++j)
04667         {
04668             // 処理対象の三角形
04669             auto& f = tface[j];
04670
04671             // 三頂点のそれぞれについて
04672             for (int i = 0; i < 3; ++i)
04673             {
04674                 // テクスチャ座標
04675                 vec2 tex{ 0.0f, 0.0f };
04676                 if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04677
04678                 // 頂点法線
04679                 vec3 norm{ 0.0f, 0.0f, 0.0f };

```

```

04680         if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04681
04682         // 頂点属性の追加
04683         vert.emplace_back(tpos[f.p[i] - 1].data(), norm.data());
04684     }
04685 }
04686
04687 // このポリゴングループの頂点数を保存する
04688 v[1] = static_cast<GLuint>(vert.size()) - v[0];
04689 v[2] = g.mtln;
04690
04691 // このポリゴングループの最初の頂点番号と頂点数・材質番号を登録する
04692 group.emplace_back(v);
04693
04694 // 次のポリゴングループの最初の三角形番号を求める
04695 startgroup = g.nextgroup;
04696 }
04697
04698 #if defined(DEBUG)
04699     std::cerr
04700         << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04701         << ", Vertex: " << vert.size() << "\n";
04702 #endif
04703
04704 // OBJ ファイルの読み込み成功
04705 return true;
04706 }
04707
04708 //
04709 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Elements 形式)
04710 //
04711 //     name 読み込むOBJ ファイル名
04712 //     group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04713 //     material 読み込んだデータのポリゴングループごとの材質
04714 //     vert 読み込んだデータの頂点属性
04715 //     face 読み込んだデータの三角形の頂点インデックス
04716 //     normalize true ならサイズを正規化する
04717 //     戻り値 読み込みに成功したら true
04718 //
04719 bool gg::ggLoadSimpleObj(const std::string& name,
04720     std::vector<std::array<GLuint, 3>>& group,
04721     std::vector<GgSimpleShader::Material>& material,
04722     std::vector<GgVertex>& vert,
04723     std::vector<GLuint>& face,
04724     bool normalize)
04725 {
04726     // 読み込み用の一時記憶領域
04727     std::vector<fgrp> tgroup;
04728     std::vector<vec3> tpos;
04729     std::vector<vec3> tnorm;
04730     std::vector<vec2> ttex;
04731     std::vector<fidx> tface;
04732
04733     // OBJ ファイルを解析する
04734     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04735
04736     // 頂点属性データの最初の頂点番号
04737     const auto vertbase{ static_cast<GLuint>(vert.size()) };
04738
04739     // 頂点属性データのメモリを確保する
04740     vert.resize(vertbase + tpos.size());
04741
04742     // 三角形データのメモリを確保する
04743     face.reserve(face.size() + tface.size());
04744
04745     // ポリゴングループデータのメモリを確保する
04746     group.reserve(group.size() + tgroup.size());
04747     material.reserve(material.size() + tgroup.size());
04748
04749     // ポリゴングループの最初の三角形番号
04750     GLuint startgroup{ 0 };
04751
04752     // ポリゴングループデータの作成
04753     for (auto& g : tgroup)
04754     {
04755         // このポリゴングループの最初の頂点番号
04756         const auto first{ static_cast<GLuint>(face.size()) };
04757
04758         // 三角形ごとの頂点データの作成
04759         for (GLuint j = startgroup; j < g.nextgroup; ++j)
04760         {
04761             // 処理対象の三角形
04762             auto& f{ tface[j] };
04763
04764             // 三頂点のそれぞれについて
04765             for (int i = 0; i < 3; ++i)
04766             {

```

```

04767         // 追加する三角形データの頂点番号
04768         const auto q{ f.p[i] - 1 + vertbase };
04769
04770         // 三角形データの追加
04771         face.emplace_back(q);
04772
04773         // テクスチャ座標番号
04774         vec2 tex{ 0.0f, 0.0f };
04775         if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04776
04777         // 頂点法線番号
04778         vec3 norm{ 0.0f, 0.0f, 0.0f };
04779         if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04780
04781         // 頂点の格納
04782         vert[q] = GgVertex(tpos[f.p[i] - 1].data(), norm.data());
04783     }
04784 }
04785
04786     // このポリゴングループの最初の三角形番号と三角形数・材質番号を登録する
04787     group.emplace_back(std::array<GLuint, 3>{ first, static_cast<GLuint>(face.size()) - first, g.mtlno
});
04788
04789     // 次のポリゴングループの最初の三角形番号を求める
04790     startgroup = g.nextgroup;
04791 }
04792
04793 #if defined(DEBUG)
04794     std::cerr
04795         << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04796         << ", Vertex: " << vert.size() << ", Face: " << face.size() << "\n";
04797 #endif
04798
04799     // OBJ ファイルの読み込み成功
04800     return true;
04801 }
04802
04803 //
04804 // シェーダオブジェクトのコンパイル結果を表示する
04805 //
04806 static GLboolean printShaderInfoLog(GLuint shader, const std::string& str)
04807 {
04808     // コンパイル結果を取得する
04809     GLint status;
04810     glGetShaderiv(shader, GL_COMPILE_STATUS, &status);
04811     #if defined(DEBUG)
04812     if (status == GL_FALSE) std::cerr << "Compile Error in " << str << std::endl;
04813     #endif
04814
04815     // シェーダのコンパイル時のログの長さを取得する
04816     GLsizei bufSize;
04817     glGetShaderiv(shader, GL_INFO_LOG_LENGTH, &bufSize);
04818
04819     if (bufSize > 1)
04820     {
04821         // シェーダのコンパイル時のログの内容を取得する
04822         std::vector<GLchar> infoLog(bufSize);
04823         GLsizei length;
04824         glGetShaderInfoLog(shader, bufSize, &length, infoLog.data());
04825         #if defined(DEBUG)
04826         std::cerr << infoLog.data();
04827         #endif
04828     }
04829
04830     // コンパイル結果を返す
04831     return static_cast<GLboolean>(status);
04832 }
04833
04834 //
04835 // プログラムオブジェクトのリンク結果を表示する
04836 //
04837 static GLboolean printProgramInfoLog(GLuint program)
04838 {
04839     // リンク結果を取得する
04840     GLint status;
04841     glGetProgramiv(program, GL_LINK_STATUS, &status);
04842     #if defined(DEBUG)
04843     if (status == GL_FALSE) std::cerr << "Link Error." << std::endl;
04844     #endif
04845
04846     // シェーダのリンク時のログの長さを取得する
04847     GLsizei bufSize;
04848     glGetProgramiv(program, GL_INFO_LOG_LENGTH, &bufSize);
04849
04850     // シェーダのリンク時のログの内容を取得する
04851     if (bufSize > 1)
04852     {

```

```

04853     std::vector<GLchar> infoLog(bufSize);
04854     GLsizei length;
04855     glGetProgramInfoLog(program, bufSize, &length, infoLog.data());
04856 #if defined(DEBUG)
04857     std::cerr << infoLog.data() << std::endl;
04858 #endif
04859 }
04860
04861 // リンク結果を返す
04862 return static_cast<GLboolean>(status);
04863 }
04864
04865 //
04866 // シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
04867 //
04868 // vsrc パーテックスシェーダのソースプログラムの文字列
04869 // fsrc フラグメントシェーダのソースプログラムの文字列 (nullptr なら不使用)
04870 // gsrc ジオメトリシェーダのソースプログラムの文字列 (nullptr なら不使用)
04871 // nvarying フィードバックする varying 変数の数 (0 なら不使用)
04872 // varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
04873 // vtext パーテックスシェーダのコンパイル時のメッセージに追加する文字列
04874 // ftext フラグメントシェーダのコンパイル時のメッセージに追加する文字列
04875 // gtext ジオメトリシェーダのコンパイル時のメッセージに追加する文字列
04876 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
04877 //
04878 GLuint gg::ggCreateShader(
04879     const std::string& vsrc,
04880     const std::string& fsrc,
04881     const std::string& gsrc,
04882     GLint nvarying,
04883     const char* const* varyings,
04884     const std::string& vtext,
04885     const std::string& ftext,
04886     const std::string& gtext)
04887 {
04888     // シェーダプログラムの作成
04889     const auto program{ glCreateProgram() };
04890
04891     if (program > 0)
04892     {
04893         bool status = true;
04894
04895         if (!vsrc.empty())
04896         {
04897             // パーテックスシェーダのシェーダオブジェクトを作成する
04898             const auto vertShader{ glCreateShader(GL_VERTEX_SHADER) };
04899             const auto* vsrccp{ vsrc.c_str() };
04900             glShaderSource(vertShader, 1, &vsrccp, nullptr);
04901             glCompileShader(vertShader);
04902
04903             // パーテックスシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04904             if (printShaderInfoLog(vertShader, vtext))
04905                 glAttachShader(program, vertShader);
04906             else
04907                 status = false;
04908             glDeleteShader(vertShader);
04909         }
04910
04911         if (!fsrc.empty())
04912         {
04913             // フラグメントシェーダのシェーダオブジェクトを作成する
04914             const auto fragShader{ glCreateShader(GL_FRAGMENT_SHADER) };
04915             const auto* fsrccp{ fsrc.c_str() };
04916             glShaderSource(fragShader, 1, &fsrccp, nullptr);
04917             glCompileShader(fragShader);
04918
04919             // フラグメントシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04920             if (printShaderInfoLog(fragShader, ftext))
04921                 glAttachShader(program, fragShader);
04922             else
04923                 status = false;
04924             glDeleteShader(fragShader);
04925         }
04926
04927         if (!gsrc.empty())
04928         {
04929             #if defined(GL_GLES_PROTOTYPES)
04930             # if defined(DEBUG)
04931                 std::cerr << gtext << ": The geometry shader is not supported." << std::endl;
04932                 status = false;
04933             # endif
04934             #else
04935             // ジオメトリシェーダのシェーダオブジェクトを作成する
04936             const auto geomShader{ glCreateShader(GL_GEOMETRY_SHADER) };
04937             const auto* gsrccp{ gsrc.c_str() };
04938             glShaderSource(geomShader, 1, &gsrccp, nullptr);
04939             glCompileShader(geomShader);

```

```

04940
04941 // ジオメトリシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04942 if (printShaderInfoLog(geomShader, gtext))
04943     glAttachShader(program, geomShader);
04944 else
04945     status = false;
04946 glDeleteShader(geomShader);
04947 #endif
04948 }
04949
04950 // 全てのシェーダオブジェクトのコンパイルに成功したら
04951 if (status)
04952 {
04953     // feedback に使う varying 変数を指定する
04954     if (nvarying > 0)
04955         glTransformFeedbackVaryings(program, nvarying, varyings, GL_SEPARATE_ATTRIBS);
04956
04957     // シェーダプログラムをリンクする
04958     glLinkProgram(program);
04959
04960     // リンクに成功したらプログラムオブジェクト名を返す
04961     if (printProgramInfoLog(program) != GL_FALSE) return program;
04962 }
04963 }
04964
04965 // プログラムオブジェクトが作成できなかった
04966 glDeleteProgram(program);
04967 return 0;
04968 }
04969
04970 //
04971 // シェーダのソースファイルを読み込んだ vector を返す
04972 //
04973 // name ソースファイル名
04974 // src 読み込んだソースファイルの文字列
04975 // 戻り値 読み込みの成功すれば true, 失敗したら false
04976 //
04977 static bool readShaderSource(const std::string& name, std::string& src)
04978 {
04979     // ファイル名が nullptr ならそのまま戻る
04980     if (name.empty()) return true;
04981
04982     // ソースファイルを開く
04983     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
04984     if (file.fail())
04985     {
04986         // ファイルが開けなければエラーで戻る
04987         #if defined(DEBUG)
04988             std::cerr << "Error: Can't open source file: " << name << std::endl;
04989         #endif
04990         return false;
04991     }
04992
04993     // ファイル全体を文字列として読み込む
04994     std::istreambuf_iterator<char> it{ file };
04995     std::istreambuf_iterator<char> last;
04996     src = std::string(it, last);
04997
04998     // ファイルがうまく読み込めなければ戻る
04999     if (file.bad())
05000     {
05001         #if defined(DEBUG)
05002             std::cerr << "Error: Could not read source file: " << name << std::endl;
05003         #endif
05004         return false;
05005     }
05006
05007     // ファイルを閉じて戻る
05008     return true;
05009 }
05010
05011 //
05012 // シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05013 //
05014 // vert パーテックスシェーダのソースファイル名
05015 // frag フラグメントシェーダのソースファイル名 (nullptr なら不使用)
05016 // geom ジオメトリシェーダのソースファイル名 (nullptr なら不使用)
05017 // nvarying フィードバックする varying 変数の数 (0 なら不使用)
05018 // varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
05019 // 戻り値 シェーダプログラムのプログラム名 (作成できなければ 0)
05020 //
05021 GLuint gg::ggLoadShader(
05022     const std::string& vert,
05023     const std::string& frag,
05024     const std::string& geom,
05025     GLint nvarying,
05026     const char* const* varyings

```

```

05027 )
05028 {
05029     // 読み込んだシェーダのソースプログラム
05030     std::string vsrc, fsrc, gsrc;
05031
05032     // 読み込んだ結果
05033     bool status;
05034
05035     // シェーダのソースファイルを読み込む
05036     status = readShaderSource(vert, vsrc);
05037     status = readShaderSource(frag, fsrc) && status;
05038     status = readShaderSource(geom, gsrc) && status;
05039
05040     // 全てのソースファイルが読み込めていなかったらエラー
05041     if (!status) return 0;
05042
05043     // プログラムオブジェクトを作成する
05044     return ggCreateShader(vsrc, fsrc, gsrc, nvarying, varyings, vert, frag, geom);
05045 }
05046
05047 #if !defined(_APPLE_) && !defined(GL_GLES_PROTOTYPES)
05048 //
05049 // コンピュートシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
05050 //
05051 //     csrc コンピュートシェーダのソースプログラムの文字列
05052 //     戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05053 //
05054 GLuint gg::ggCreateComputeShader(const std::string& csrc, const std::string& ctext)
05055 {
05056     // シェーダプログラムの作成
05057     const auto program{ glCreateProgram() };
05058
05059     if (program > 0)
05060     {
05061         // ソースプログラムの文字列が空だったら 0 を返す
05062         if (csrc.empty()) return 0;
05063
05064         // コンピュートシェーダのシェーダオブジェクトを作成する
05065         const auto compShader{ glCreateShader(GL_COMPUTE_SHADER) };
05066         const auto* csrccp{ csrc.c_str() };
05067         glShaderSource(compShader, 1, &csrccp, nullptr);
05068         glCompileShader(compShader);
05069
05070         // コンピュートシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
05071         if (printShaderInfoLog(compShader, ctext)) glAttachShader(program, compShader);
05072         glDeleteShader(compShader);
05073
05074         // シェーダプログラムをリンクする
05075         glLinkProgram(program);
05076
05077         // プログラムオブジェクトが作成できなければ 0 を返す
05078         if (printProgramInfoLog(program) == GL_FALSE)
05079         {
05080             glDeleteProgram(program);
05081             return 0;
05082         }
05083     }
05084
05085     // プログラムオブジェクトを返す
05086     return program;
05087 }
05088
05089 //
05090 // コンピュートシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05091 //
05092 //     comp コンピュートシェーダのソースファイル名
05093 //     戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05094 //
05095 GLuint gg::ggLoadComputeShader(const std::string& comp)
05096 {
05097     // シェーダのソースファイルを読み込む
05098     std::string csrc;
05099     if (readShaderSource(comp, csrc))
05100     {
05101         // プログラムオブジェクトを作成する
05102         return ggCreateComputeShader(csrc.data(), comp);
05103     }
05104
05105     // プログラムオブジェクト作成失敗
05106     return 0;
05107 }
05108 #endif
05109
05110 //
05111 // 点：データ作成
05112 //
05113 void gg::GgPoints::load(const GgVector* pos, GLsizei count, GLenum usage)

```

```

05114 {
05115     // 頂点バッファオブジェクトを作成する
05116     position = std::make_shared<GgBuffer<GgVector>>(GL_ARRAY_BUFFER, pos,
static_cast<GLsizei>(sizeof(GgVector)), count, usage);
05117
05118     // このバッファオブジェクトは index == 0 の in 変数から入力する
05119     glVertexAttribPointer(0, static_cast<GLint>(pos->size()), GL_FLOAT, GL_FALSE, 0, 0);
05120     glEnableVertexAttribArray(0);
05121 }
05122
05123 //
05124 // 点: 描画
05125 //
05126 void gg::GgPoints::draw(GLint first, GLsizei count) const
05127 {
05128     // 頂点配列オブジェクトを指定する
05129     GgShape::draw(first, count);
05130
05131     // 図形を描画する
05132     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05133 }
05134
05135 //
05136 // 三角形: データ作成
05137 //
05138 void gg::GgTriangles::load(const GgVertex* vert, GLsizei count, GLenum usage)
05139 {
05140     // 頂点バッファオブジェクトを作成する
05141     vertex = std::make_shared<GgBuffer<GgVertex>>(GL_ARRAY_BUFFER, vert,
static_cast<GLsizei>(sizeof(GgVertex)), count, usage);
05142
05143     // 頂点の位置は index == 0 の in 変数から入力する
05144     glVertexAttribPointer(0, static_cast<GLint>(vert->position.size()), GL_FLOAT, GL_FALSE,
sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, position));
05145     glEnableVertexAttribArray(0);
05146
05147     // 頂点の法線は index == 1 の in 変数から入力する
05148     glVertexAttribPointer(1, static_cast<GLint>(vert->normal.size()), GL_FLOAT, GL_FALSE,
sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, normal));
05149     glEnableVertexAttribArray(1);
05150 }
05151
05152 //
05153 // 三角形: 描画
05154 //
05155 void gg::GgTriangles::draw(GLint first, GLsizei count) const
05156 {
05157     // 頂点配列オブジェクトを指定する
05158     GgShape::draw(first, count);
05159
05160     // 図形を描画する
05161     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05162 }
05163
05164 //
05165 // オブジェクト: 描画
05166 //
05167 void gg::GgElements::draw(GLint first, GLsizei count) const
05168 {
05169     // 頂点配列オブジェクトを指定する
05170     GgShape::draw(first, count);
05171
05172     // 図形を描画する
05173     glDrawElements(getMode(), count > 0 ? count : getIndexCount() - first,
GL_UNSIGNED_INT, static_cast<GLuint*>(0) + first);
05174 }
05175
05176 //
05177 // 点群を立方体状に生成する
05178 //
05179 void gg::GgPointsCube(GLsizei count, GLfloat length, GLfloat cx, GLfloat cy,
GLfloat cz)
05180 {
05181     // メモリを確保する
05182     std::vector<GgVector> pos(count);
05183
05184     // 点を生成する
05185     for (GLsizei v = 0; v < count; ++v)
05186     {
05187         const GgVector p
05188         {
05189             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cx,
05190             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cy,
05191             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cz,
05192             1.0f
05193         };
05194         pos[v] = p;
05195     }
05196 }
05197

```

```

05198     pos.emplace_back(p);
05199 }
05200
05201 // 点データの GgPoints オブジェクトを作成して返す
05202 return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05203 }
05204
05205 //
05206 // 点群を球状に生成する
05207 //
05208 std::shared_ptr<gg::GgPoints> gg::ggPointsSphere(GLsizei count, GLfloat radius,
05209 GLfloat cx, GLfloat cy, GLfloat cz)
05210 {
05211     // メモリを確保する
05212     std::vector<GgVector> pos(count);
05213
05214     // 点を生成する
05215     for (GLsizei v = 0; v < count; ++v)
05216     {
05217         const auto r{ radius * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) };
05218         const auto t{ 6.28318530718f * static_cast<GLfloat>(rand()) / (static_cast<GLfloat>(RAND_MAX) +
05219 1.0f) };
05220         const auto cp{ 2.0f * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 1.0f };
05221         const auto sp{ sqrtf(1.0f - cp * cp) };
05222         const auto ct{ cosf(t) };
05223         const auto st{ sinf(t) };
05224         const GgVector p{ r * sp * ct + cx, r * sp * st + cy, r * cp + cz, 1.0f };
05225
05226         pos.emplace_back(p);
05227     }
05228
05229 // 点データの GgPoints オブジェクトを作成して返す
05230 return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05231 }
05232 //
05233 // 矩形状に 2 枚の三角形を生成する
05234 //
05235 std::shared_ptr<gg::GgTriangles> gg::ggRectangle(GLfloat width, GLfloat height)
05236 {
05237     // 頂点属性
05238     std::array<gg::GgVertex, 4> vert;
05239
05240     // 頂点位置と法線を求める
05241     for (int v = 0; v < 4; ++v)
05242     {
05243         const auto x{ ((v & 1) * 2 - 1) * width };
05244         const auto y{ ((v & 2) - 1) * height };
05245
05246         vert[v] = gg::GgVertex{ x, y, 0.0f, 0.0f, 0.0f, 1.0f };
05247     }
05248
05249     // 矩形の GgTriangles オブジェクトを作成する
05250     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_STRIP);
05251 }
05252
05253 //
05254 // 楕円状に三角形を生成する
05255 //
05256 std::shared_ptr<gg::GgTriangles> gg::ggEllipse(GLfloat width, GLfloat height, GLuint slices)
05257 {
05258     // 楕円のスケール
05259     constexpr GLfloat scale{ 0.5f };
05260
05261     // 作業用のメモリ
05262     std::vector<gg::GgVertex> vert;
05263
05264     // 頂点位置と法線を求める
05265     for (GLuint v = 0; v < slices; ++v)
05266     {
05267         const auto t{ 6.28318530717f * static_cast<GLfloat>(v) / static_cast<GLfloat>(slices) };
05268         const auto x{ cosf(t) * width * scale };
05269         const auto y{ sinf(t) * height * scale };
05270
05271         vert.emplace_back(x, y, 0.0f, 0.0f, 0.0f, 1.0f);
05272     }
05273
05274     // GgTriangles オブジェクトを作成する
05275     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_FAN);
05276 }
05277
05278 //
05279 // Wavefront OBJ ファイルを読み込む (Arrays 形式)
05280 //
05281 std::shared_ptr<gg::GgTriangles> gg::ggArraysObj(const std::string& name, bool normalize)

```



```

05282 {
05283     std::vector<std::array<GLuint, 3>> group;
05284     std::vector<gg::GgSimpleShader::Material> material;
05285     std::vector<gg::GgVertex> vert;
05286
05287     // ファイルを読み込む
05288     if (!ggLoadSimpleObj(name, group, material, vert, normalize)) return nullptr;
05289
05290     // GgTriangles オブジェクトを作成する
05291     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLES);
05292 }
05293
05294 //
05295 // Wavefront OBJ ファイル を読み込む (Elements 形式)
05296 //
05297 std::shared_ptr<gg::GgElements> gg::ggElementsObj(const std::string& name, bool normalize)
05298 {
05299     std::vector<std::array<GLuint, 3>> group;
05300     std::vector<gg::GgSimpleShader::Material> material;
05301     std::vector<gg::GgVertex> vert;
05302     std::vector<GLuint> face;
05303
05304     // ファイルを読み込む
05305     if (!ggLoadSimpleObj(name, group, material, vert, face, normalize)) return nullptr;
05306
05307     // GgElements オブジェクトを作成する
05308     return std::make_shared<gg::GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05309 }
05310
05311 //
05312 //
05313 // メッシュ形状を作成する (Elements 形式)
05314 //
05315 std::shared_ptr<gg::GgElements> gg::ggElementsMesh(GLuint slices, GLuint stacks, const
GLfloat (*pos)[3], const GLfloat (*norm)[3])
05316 {
05317     // 頂点属性
05318     std::vector<gg::GgVertex> vert((slices + 1) * (stacks + 1));
05319
05320     // 頂点の法線を求める
05321     for (GLuint j = 0; j <= stacks; ++j)
05322     {
05323         for (GLuint i = 0; i <= slices; ++i)
05324         {
05325             // 処理対象の頂点番号
05326             const auto k{ j * (slices + 1) + i };
05327
05328             // 頂点の法線
05329             gg::GgVector tnorm;
05330             tnorm[3] = 0.0f;
05331
05332             if (norm)
05333             {
05334                 tnorm[0] = norm[k][0];
05335                 tnorm[1] = norm[k][1];
05336                 tnorm[2] = norm[k][2];
05337             }
05338             else
05339             {
05340                 // 処理対象の頂点の周囲の頂点番号
05341                 const auto kim{ i > 0 ? k - 1 : k };
05342                 const auto kip{ i < slices ? k + 1 : k };
05343                 const auto kjm{ j > 0 ? k - slices - 1 : k };
05344                 const auto kjp{ j < stacks ? k + slices + 1 : k };
05345
05346                 // 接線ベクトル
05347                 const std::array<GLfloat, 3> t
05348                 {
05349                     pos[kip][0] - pos[kim][0],
05350                     pos[kip][1] - pos[kim][1],
05351                     pos[kip][2] - pos[kim][2]
05352                 };
05353
05354                 // 従接線ベクトル
05355                 const std::array<GLfloat, 3> b
05356                 {
05357                     pos[kjp][0] - pos[kjm][0],
05358                     pos[kjp][1] - pos[kjm][1],
05359                     pos[kjp][2] - pos[kjm][2]
05360                 };
05361
05362                 // 法線
05363                 tnorm[0] = t[1] * b[2] - t[2] * b[1];
05364                 tnorm[1] = t[2] * b[0] - t[0] * b[2];
05365                 tnorm[2] = t[0] * b[1] - t[1] * b[0];
05366

```

```

05367         // 法線の正規化
05368         ggNormalize3(tnorm);
05369     }
05370
05371     // 頂点の位置
05372     const gg::GgVector tpos{ pos[k][0], pos[k][1], pos[k][2], 1.0f };
05373
05374     // 頂点属性の保存
05375     vert.emplace_back(tpos, tnorm);
05376 }
05377 }
05378
05379 // 頂点のインデックス (三角形データ)
05380 std::vector<GLuint> face;
05381
05382 // 頂点のインデックスを求める
05383 for (GLuint j = 0; j < stacks; ++j)
05384 {
05385     for (GLuint i = 0; i < slices; ++i)
05386     {
05387         // 処理対象のマス
05388         const auto k{ (slices + 1) * j + i };
05389
05390         // マスの上半分の三角形
05391         face.emplace_back(k);
05392         face.emplace_back(k + slices + 2);
05393         face.emplace_back(k + 1);
05394
05395         // マスの下半分の三角形
05396         face.emplace_back(k);
05397         face.emplace_back(k + slices + 1);
05398         face.emplace_back(k + slices + 2);
05399     }
05400 }
05401
05402 // GgElements オブジェクトを作成する
05403 return std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
05404     face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05405 }
05406
05407 //
05408 // 球状に三角形データを生成する (Elements 形式)
05409 //
05410 std::shared_ptr<gg::GgElements> gg::ggElementsSphere(GLfloat radius, int slices, int stacks)
05411 {
05412     // 頂点の位置と法線
05413     std::vector<GLfloat> p, n;
05414
05415     // 頂点の位置と法線を求める
05416     for (int j = 0; j <= stacks; ++j)
05417     {
05418         const auto t{ static_cast<GLfloat>(j) / static_cast<GLfloat>(stacks) };
05419         const auto ph{ 3.1415926536f * t };
05420         const auto y{ cosf(ph) };
05421         const auto r{ sinf(ph) };
05422
05423         for (int i = 0; i <= slices; ++i)
05424         {
05425             const auto s{ static_cast<GLfloat>(i) / static_cast<GLfloat>(slices) };
05426             const auto th{ -2.0f * 3.1415926536f * s };
05427             const auto x{ r * cosf(th) };
05428             const auto z{ r * sinf(th) };
05429
05430             // 頂点の座標値
05431             p.emplace_back(x * radius);
05432             p.emplace_back(y * radius);
05433             p.emplace_back(z * radius);
05434
05435             // 頂点の法線
05436             n.emplace_back(x);
05437             n.emplace_back(y);
05438             n.emplace_back(z);
05439         }
05440     }
05441
05442     // GgElements オブジェクトを作成する
05443     return ggElementsMesh(slices, stacks, reinterpret_cast<GLfloat(*)[3]>(p.data()),
05444         reinterpret_cast<GLfloat(*)[3]>(n.data()));
05445 }
05446
05447 //
05448 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05449 //
05450 // r 光源の強度の環境光成分の赤成分
05451 // g 光源の強度の環境光成分の緑成分
05452 // b 光源の強度の環境光成分の青成分
05453 // a 光源の強度の環境光成分の不透明度

```

```

05454 // first 値を設定する光源データの最初の番号
05455 // count 値を設定する光源データの数
05456 //
05457 void gg::GgSimpleShader::LightBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05458 GLint first, GLsizei count) const
05459 {
05460 // データを格納するバッファオブジェクトの先頭のポインタ
05461 const auto start{ static_cast<char*>(map(first, count)) };
05462 for (GLsizei i = 0; i < count; ++i)
05463 {
05464 // バッファオブジェクトの i 番目のブロックのポインタ
05465 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05466 // 光源の環境光成分を設定する
05467 light->ambient[0] = r;
05468 light->ambient[1] = g;
05469 light->ambient[2] = b;
05470 light->ambient[3] = a;
05471 }
05472 }
05473 unmap();
05474 }
05475 //
05476 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05477 //
05478 // ambient 光源の強度の環境光成分
05479 // first 値を設定する光源データの最初の番号
05480 // count 値を設定する光源データの数
05481 //
05482 //
05483 void gg::GgSimpleShader::LightBuffer::loadAmbient(const GgVector& ambient,
05484 GLint first, GLsizei count) const
05485 {
05486 // データを格納するバッファオブジェクトの先頭のポインタ
05487 const auto start{ static_cast<char*>(map(first, count)) };
05488 for (GLsizei i = 0; i < count; ++i)
05489 {
05490 // バッファオブジェクトの i 番目のブロックのポインタ
05491 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05492 // 光源の強度の環境光成分を設定する
05493 light->ambient = ambient;
05494 }
05495 }
05496 unmap();
05497 }
05498 //
05499 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05500 //
05501 //
05502 // r 光源の強度の拡散反射光成分の赤成分
05503 // g 光源の強度の拡散反射光成分の緑成分
05504 // b 光源の強度の拡散反射光成分の青成分
05505 // a 光源の強度の拡散反射光成分の不透明度
05506 // first 値を設定する光源データの最初の番号
05507 // count 値を設定する光源データの数
05508 //
05509 void gg::GgSimpleShader::LightBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05510 GLint first, GLsizei count) const
05511 {
05512 // データを格納するバッファオブジェクトの先頭のポインタ
05513 const auto start{ static_cast<char*>(map(first, count)) };
05514 for (GLsizei i = 0; i < count; ++i)
05515 {
05516 // バッファオブジェクトの i 番目のブロックのポインタ
05517 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05518 // 光源の拡散反射光成分を設定する
05519 light->diffuse[0] = r;
05520 light->diffuse[1] = g;
05521 light->diffuse[2] = b;
05522 light->diffuse[3] = a;
05523 }
05524 }
05525 unmap();
05526 }
05527 //
05528 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05529 //
05530 //
05531 // ambient 光源の強度の拡散反射光成分
05532 // first 値を設定する光源データの最初の番号
05533 // count 値を設定する光源データの数
05534 //
05535 void gg::GgSimpleShader::LightBuffer::loadDiffuse(const GgVector& diffuse,
05536 GLint first, GLsizei count) const
05537 {
05538 // データを格納するバッファオブジェクトの先頭のポインタ
05539 const auto start{ static_cast<char*>(map(first, count)) };
05540 for (GLsizei i = 0; i < count; ++i)

```

```

05541 {
05542     // バッファオブジェクトの i 番目のブロックのポインタ
05543     const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05544
05545     // 光源の強度の拡散反射光成分を設定する
05546     light->diffuse = diffuse;
05547 }
05548 unmap();
05549 }
05550
05551 //
05552 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05553 //
05554 //     r 光源の強度の鏡面反射光成分の赤成分
05555 //     g 光源の強度の鏡面反射光成分の緑成分
05556 //     b 光源の強度の鏡面反射光成分の青成分
05557 //     a 光源の強度の鏡面反射光成分の不透明度
05558 //     first 値を設定する光源データの最初の番号
05559 //     count 値を設定する光源データの数
05560 //
05561 void gg::GgSimpleShader::LightBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05562     GLint first, GLsizei count) const
05563 {
05564     // データを格納するバッファオブジェクトの先頭のポインタ
05565     const auto start{ static_cast<char*>(map(first, count)) };
05566     for (GLsizei i = 0; i < count; ++i)
05567     {
05568         // バッファオブジェクトの i 番目のブロックのポインタ
05569         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05570
05571         // 光源の鏡面反射光成分を設定する
05572         light->specular[0] = r;
05573         light->specular[1] = g;
05574         light->specular[2] = b;
05575         light->specular[3] = a;
05576     }
05577     unmap();
05578 }
05579
05580 //
05581 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05582 //
05583 //     ambient 光源の強度の鏡面反射光成分
05584 //     first 値を設定する光源データの最初の番号
05585 //     count 値を設定する光源データの数
05586 //
05587 void gg::GgSimpleShader::LightBuffer::loadSpecular(const GgVector& specular,
05588     GLint first, GLsizei count) const
05589 {
05590     // データを格納するバッファオブジェクトの先頭のポインタ
05591     const auto start{ static_cast<char*>(map(first, count)) };
05592     for (GLsizei i = 0; i < count; ++i)
05593     {
05594         // バッファオブジェクトの i 番目のブロックのポインタ
05595         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05596
05597         // 光源の強度の鏡面反射光成分を設定する
05598         light->specular = specular;
05599     }
05600     unmap();
05601 }
05602
05603 //
05604 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の色を設定するが位置は変更しない
05605 //
05606 //     material 光源の特性の GgSimpleShader::Light 構造体
05607 //     first 値を設定する光源データの最初の番号
05608 //     count 値を設定する光源データの数
05609 //
05610 void gg::GgSimpleShader::LightBuffer::loadColor(const Light& color,
05611     GLint first, GLsizei count) const
05612 {
05613     // データを格納するバッファオブジェクトの先頭のポインタ
05614     const auto start{ static_cast<char*>(map(first, count)) };
05615     for (GLsizei i = 0; i < count; ++i)
05616     {
05617         // バッファオブジェクトの i 番目のブロックのポインタ
05618         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05619
05620         // 光源の色を設定する
05621         light->ambient = color.ambient;
05622         light->diffuse = color.diffuse;
05623         light->specular = color.specular;
05624     }
05625     unmap();
05626 }
05627

```

```

05628 //
05629 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05630 //
05631 //   x 光源の位置の x 座標
05632 //   y 光源の位置の y 座標
05633 //   z 光源の位置の z 座標
05634 //   w 光源の位置の w 座標
05635 //   first 値を設定する光源データの最初の番号
05636 //   count 値を設定する光源データの数
05637 //
05638 void gg::GgSimpleShader::LightBuffer::loadPosition(GLfloat x, GLfloat y, GLfloat z, GLfloat w,
05639 GLint first, GLsizei count) const
05640 {
05641     // データを格納するバッファオブジェクトの先頭のポインタ
05642     const auto start{ static_cast<char*>(map(first, count)) };
05643     for (GLsizei i = 0; i < count; ++i)
05644     {
05645         // バッファオブジェクトの i 番目のブロックのポインタ
05646         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05647
05648         // 光源の位置を設定する
05649         light->position[0] = x;
05650         light->position[1] = y;
05651         light->position[2] = z;
05652         light->position[3] = w;
05653     }
05654     unmap();
05655 }
05656
05657 //
05658 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05659 //
05660 //   position 光源の位置
05661 //   first 値を設定する光源データの最初の番号
05662 //   count 値を設定する光源データの数
05663 //
05664 void gg::GgSimpleShader::LightBuffer::loadPosition(const GgVector& position,
05665 GLint first, GLsizei count) const
05666 {
05667     // データを格納するバッファオブジェクトの先頭のポインタ
05668     const auto start{ static_cast<char*>(map(first, count)) };
05669     for (GLsizei i = 0; i < count; ++i)
05670     {
05671         // バッファオブジェクトの i 番目のブロックのポインタ
05672         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05673
05674         // 光源の位置を設定する
05675         light->position = position;
05676     }
05677     unmap();
05678 }
05679
05680 //
05681 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05682 //
05683 //   r 環境光に対する反射係数の赤成分
05684 //   g 環境光に対する反射係数の緑成分
05685 //   b 環境光に対する反射係数の青成分
05686 //   a 環境光に対する反射係数の不透明度
05687 //   first 値を設定する材質データの最初の番号
05688 //   count 値を設定する材質データの数
05689 //
05690 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05691 GLint first, GLsizei count) const
05692 {
05693     // データを格納するバッファオブジェクトの先頭のポインタ
05694     const auto start{ static_cast<char*>(map(first, count)) };
05695     for (GLsizei i = 0; i < count; ++i)
05696     {
05697         // バッファオブジェクトの i 番目のブロックのポインタ
05698         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05699
05700         // 環境光に対する反射係数を設定する
05701         material->ambient[0] = r;
05702         material->ambient[1] = g;
05703         material->ambient[2] = b;
05704         material->ambient[3] = a;
05705     }
05706     unmap();
05707 }
05708
05709 //
05710 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05711 //
05712 //   ambient 環境光に対する反射係数
05713 //   first 値を設定する光源データの最初の番号
05714 //   count 値を設定する光源データの数

```

```

05715 //
05716 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(const GgVector& ambient,
05717 GLint first, GLsizei count) const
05718 {
05719     // データを格納するバッファオブジェクトの先頭のポインタ
05720     const auto start{ static_cast<char*>(map(first, count)) };
05721     for (GLsizei i = 0; i < count; ++i)
05722     {
05723         // バッファオブジェクトの i 番目のブロックのポインタ
05724         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05725
05726         // 光源の強度の環境光成分を設定する
05727         material->ambient = ambient;
05728     }
05729     unmap();
05730 }
05731
05732 //
05733 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05734 //
05735 //     r 拡散反射係数の赤成分
05736 //     g 拡散反射係数の緑成分
05737 //     b 拡散反射係数の青成分
05738 //     a 拡散反射係数の不透明度
05739 //     first 値を設定する材質データの最初の番号
05740 //     count 値を設定する材質データの数
05741 //
05742 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05743 GLint first, GLsizei count) const
05744 {
05745     // データを格納するバッファオブジェクトの先頭のポインタ
05746     const auto start{ static_cast<char*>(map(first, count)) };
05747     for (GLsizei i = 0; i < count; ++i)
05748     {
05749         // バッファオブジェクトの i 番目のブロックのポインタ
05750         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05751
05752         // 拡散反射係数を設定する
05753         material->diffuse[0] = r;
05754         material->diffuse[1] = g;
05755         material->diffuse[2] = b;
05756         material->diffuse[3] = a;
05757     }
05758     unmap();
05759 }
05760
05761 //
05762 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05763 //
05764 //     diffuse 拡散反射係数
05765 //     first 値を設定する光源データの最初の番号
05766 //     count 値を設定する光源データの数
05767 //
05768 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(const GgVector& diffuse,
05769 GLint first, GLsizei count) const
05770 {
05771     // データを格納するバッファオブジェクトの先頭のポインタ
05772     const auto start{ static_cast<char*>(map(first, count)) };
05773     for (GLsizei i = 0; i < count; ++i)
05774     {
05775         // バッファオブジェクトの i 番目のブロックのポインタ
05776         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05777
05778         // 光源の強度の環境光成分を設定する
05779         material->diffuse = diffuse;
05780     }
05781     unmap();
05782 }
05783
05784 //
05785 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05786 //
05787 //     r 環境光に対する反射係数と拡散反射係数の赤成分
05788 //     g 環境光に対する反射係数と拡散反射係数の緑成分
05789 //     b 環境光に対する反射係数と拡散反射係数の青成分
05790 //     a 環境光に対する反射係数と拡散反射係数の不透明度
05791 //     first 値を設定する材質データの最初の番号
05792 //     count 値を設定する材質データの数
05793 //
05794 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(GLfloat r, GLfloat g, GLfloat b,
05795 GLfloat a,
05796 GLint first, GLsizei count) const
05797 {
05798     // データを格納するバッファオブジェクトの先頭のポインタ
05799     const auto start{ static_cast<char*>(map(first, count)) };
05800     for (GLsizei i = 0; i < count; ++i)
05801     {

```

```

05801 // バッファオブジェクトの i 番目のブロックのポインタ
05802 const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05803
05804 // 環境光に対する反射係数と拡散反射係数を設定する
05805 material->ambient[0] = material->diffuse[0] = r;
05806 material->ambient[1] = material->diffuse[1] = g;
05807 material->ambient[2] = material->diffuse[2] = b;
05808 material->ambient[3] = material->diffuse[3] = a;
05809 }
05810 unmap();
05811 }
05812
05813 //
05814 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05815 //
05816 // color 環境光に対する反射係数と拡散反射係数
05817 // first 値を設定する光源データの最初の番号
05818 // count 値を設定する光源データの数
05819 //
05820 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GgVector& color,
05821 GLint first, GLsizei count) const
05822 {
05823 // データを格納するバッファオブジェクトの先頭のポインタ
05824 const auto start{ static_cast<char*>(map(first, count)) };
05825 for (GLsizei i = 0; i < count; ++i)
05826 {
05827 // バッファオブジェクトの i 番目のブロックのポインタ
05828 const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05829
05830 // 光源の強度の環境光成分を設定する
05831 material->ambient = material->diffuse = color;
05832 }
05833 unmap();
05834 }
05835
05836 //
05837 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05838 //
05839 // color 環境光に対する反射係数と拡散反射係数を格納した GLfloat 型の 4 要素の配列
05840 // first 値を設定する材質データの最初の番号
05841 // count 値を設定する材質データの数
05842 //
05843 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GLfloat* color,
05844 GLint first, GLsizei count) const
05845 {
05846 // ambient 要素のバイトオフセット
05847 constexpr auto ambientOffset{ offsetof(Material, ambient) };
05848
05849 // ambient 要素のバイト数
05850 constexpr auto ambientSize{ sizeof(Material::diffuse) };
05851
05852 // diffuse 要素のバイトオフセット
05853 constexpr auto diffuseOffset{ offsetof(Material, diffuse) };
05854
05855 // diffuse 要素のバイト数
05856 constexpr auto diffuseSize{ sizeof(Material::diffuse) };
05857
05858 // 元のデータの先頭
05859 const auto* source{ reinterpret_cast<const char*>(color) };
05860
05861 // first 番目のブロックから count 個の ambient 要素と diffuse 要素に値を設定する
05862 bind();
05863 for (GLsizei i = 0; i < count; ++i)
05864 {
05865 // 格納先
05866 const auto destination{ getStride() * (first + i) };
05867
05868 // first + i 番目のブロックの ambient 要素に値を設定する
05869 glBufferSubData(getTarget(), destination + ambientOffset, ambientSize, source + i * ambientSize);
05870
05871 // first + i 番目のブロックの diffuse 要素に値を設定する
05872 glBufferSubData(getTarget(), destination + diffuseOffset, diffuseSize, source + i * diffuseSize);
05873 }
05874 }
05875
05876 //
05877 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
05878 //
05879 // r 鏡面反射係数の赤成分
05880 // g 鏡面反射係数の緑成分
05881 // b 鏡面反射係数の青成分
05882 // a 鏡面反射係数の不透明度
05883 // first 値を設定する材質データの最初の番号
05884 // count 値を設定する材質データの数
05885 //
05886 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05887 GLint first, GLsizei count) const

```



```

05888 {
05889     // データを格納するバッファオブジェクトの先頭のポインタ
05890     const auto start{ static_cast<char*>(map(first, count)) };
05891     for (GLsizei i = 0; i < count; ++i)
05892     {
05893         // バッファオブジェクトの i 番目のブロックのポインタ
05894         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05895
05896         // 鏡面反射係数を設定する
05897         material->specular[0] = r;
05898         material->specular[1] = g;
05899         material->specular[2] = b;
05900         material->specular[3] = a;
05901     }
05902     unmap();
05903 }
05904
05905 //
05906 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
05907 //
05908 //     specular 鏡面反射係数
05909 //     first 値を設定する光源データの最初の番号
05910 //     count 値を設定する光源データの数
05911 //
05912 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(const GgVector& specular,
05913 GLint first, GLsizei count) const
05914 {
05915     // データを格納するバッファオブジェクトの先頭のポインタ
05916     const auto start{ static_cast<char*>(map(first, count)) };
05917     for (GLsizei i = 0; i < count; ++i)
05918     {
05919         // バッファオブジェクトの i 番目のブロックのポインタ
05920         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05921
05922         // 光源の強度の環境光成分を設定する
05923         material->specular = specular;
05924     }
05925     unmap();
05926 }
05927
05928 //
05929 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
05930 //
05931 //     shininess 輝き係数
05932 //     first 値を設定する材質データの最初の番号
05933 //     count 値を設定する材質データの数
05934 //
05935 void gg::GgSimpleShader::MaterialBuffer::loadShininess(GLfloat shininess,
05936 GLint first, GLsizei count) const
05937 {
05938     // データを格納するバッファオブジェクトの先頭のポインタ
05939     const auto start{ static_cast<char*>(map(first, count)) };
05940     for (GLsizei i = 0; i < count; ++i)
05941     {
05942         // バッファオブジェクトの i 番目のブロックのポインタ
05943         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05944         material->shininess = shininess;
05945     }
05946     unmap();
05947 }
05948
05949 //
05950 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
05951 //
05952 //     shininess 輝き係数
05953 //     first 値を設定する材質データの最初の番号
05954 //     count 値を設定する材質データの数
05955 //
05956 void gg::GgSimpleShader::MaterialBuffer::loadShininess(const GLfloat* shininess,
05957 GLint first, GLsizei count) const
05958 {
05959     // データを格納するバッファオブジェクトの先頭のポインタ
05960     const auto start{ static_cast<char*>(map(first, count)) };
05961     for (GLsizei i = 0; i < count; ++i)
05962     {
05963         // バッファオブジェクトの i 番目のブロックのポインタ
05964         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05965         material->shininess = shininess[i];
05966     }
05967     unmap();
05968 }
05969
05970 //
05971 // 三角形に単純な陰影付けを行うシェーダ：シェーダのソースファイルの読み込み
05972 //
05973 bool gg::GgSimpleShader::load(const std::string& vert, const std::string& frag, const std::string&
geom,

```



```

05974 GLint nvarying, const char* const* varyings)
05975 {
05976     if (!GgPointShader::load(vert, frag, geom, nvarying, varyings)) return false;
05977
05978     mnLoc = glGetUniformLocation(get(), "mn");
05979     const auto lightIndex{ glGetUniformLocation(get(), "Light") };
05980     glUniformBlockBinding(get(), lightIndex, LightBindingPoint);
05981     const auto materialIndex{ glGetUniformLocation(get(), "Material") };
05982     glUniformBlockBinding(get(), materialIndex, MaterialBindingPoint);
05983
05984     return true;
05985 }
05986
05987 //
05988 // Wavefront OBJ 形式のデータ：コンストラクタ
05989 //
05990 gg::GgSimpleObj::GgSimpleObj(const std::string& name, bool normalize)
05991 {
05992     // 作業用のメモリ
05993     std::vector<GgSimpleShader::Material> mat;
05994     std::vector<GgVertex> vert;
05995     std::vector<GLuint> face;
05996
05997     // グループのデータのメモリを確保する
05998     group = std::make_shared<std::vector<std::array<GLuint, 3>>>();
05999
06000     // ファイルを読み込む
06001     if (ggLoadSimpleObj(name, *group, mat, vert, face, normalize))
06002     {
06003         // 頂点バッファオブジェクトを作成する
06004         data = std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
06005             face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
06006
06007         // 描画するオブジェクトを切り替えるために頂点配列オブジェクトを閉じておく
06008         glBindVertexArray(0);
06009
06010         // 材質データを設定する
06011         material = std::make_shared<GgSimpleShader::MaterialBuffer>(mat.data(),
06012             static_cast<GLsizei>(mat.size()));
06013     }
06014
06015 //
06016 // Wavefront OBJ 形式のデータ：図形の描画
06017 //
06018 void gg::GgSimpleObj::draw(GLint first, GLsizei count) const
06019 {
06020     // 保持しているグループの数
06021     const auto ng{ static_cast<GLsizei>(group->size()) };
06022
06023     // 描画する最後のグループの次
06024     auto last{ count <= 0 ? ng : first + count };
06025     if (last > ng) last = ng;
06026
06027     for (GLsizei i = first; i < last; ++i)
06028     {
06029         // グループのデータ
06030         const auto& g{ (*group)[i] };
06031
06032         // 材質を設定する
06033         material->select(g[2]);
06034
06035         // 図形を描画する
06036         data->draw(g[0], g[1]);
06037     }
06038 }

```

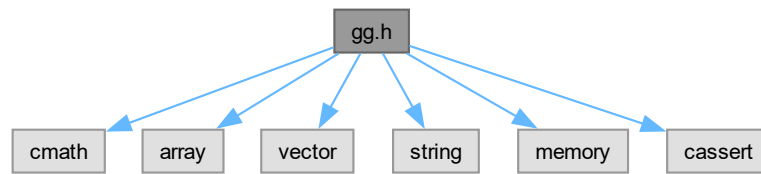
9.41 gg.h ファイル

```

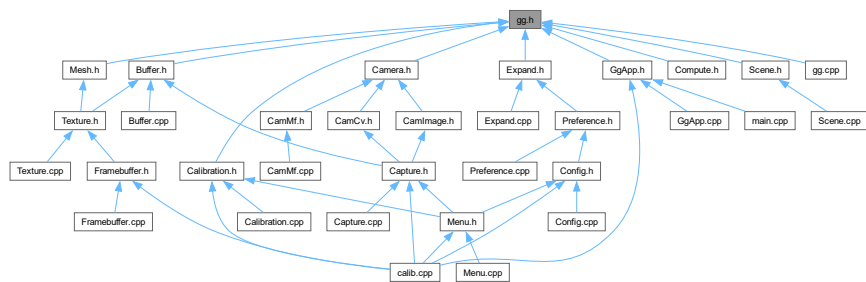
#include <cmath>
#include <array>
#include <vector>
#include <string>
#include <memory>
#include <cassert>

```

gg.h の依存先関係図:



被依存関係図:



クラス

- class gg::GgVector
- class gg::GgMatrix
- class gg::GgQuaternion
- class gg::GgTrackball
- class gg::GgTexture
- class gg::GgColorTexture
- class gg::GgNormalTexture
- class gg::GgBuffer< T >
- class gg::GgUniformBuffer< T >
- class gg::GgVertexArray
- class gg::GgShape
- class gg::GgPoints
- struct gg::GgVertex
- class gg::GgTriangles
- class gg::GgElements
- class gg::GgShader
- class gg::GgPointShader
- class gg::GgSimpleShader
- struct gg::GgSimpleShader::Light
- class gg::GgSimpleShader::LightBuffer
- struct gg::GgSimpleShader::Material
- class gg::GgSimpleShader::MaterialBuffer
- class gg::GgSimpleObj

名前空間

- namespace gg

マクロ定義

- #define `ggError()`
- #define `ggFBOError()`

型定義

- using `pathString` = `std::string`
- using `pathChar` = `char`

列挙型

- enum `gg::BindingPoints` { `gg::LightBindingPoint` = 0 , `gg::MaterialBindingPoint` }

関数

- `pathString Utf8ToTChar` (const `std::string` &string)
- `std::string TCharToUtf8` (const `pathString` &cstring)
- void `gg::ggInit` ()
- void `gg::ggError` (const `std::string` &name="", unsigned int line=0)
- void `gg::ggFBOError` (const `std::string` &name="", unsigned int line=0)
- void `gg::ggCross` (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggDot3` (const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggLength3` (const GLfloat *a)
- GLfloat `gg::ggDistance3` (const GLfloat *a, const GLfloat *b)
- void `gg::ggNormalize3` (const GLfloat *a, GLfloat *b)
- void `gg::ggNormalize3` (GLfloat *a)
- GLfloat `gg::ggDot4` (const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggLength4` (const GLfloat *a)
- GLfloat `gg::ggDistance4` (const GLfloat *a, const GLfloat *b)
- void `gg::ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `gg::ggNormalize4` (GLfloat *a)
- const `GgVector` & `gg::operator+` (const `GgVector` &v)
- `GgVector` `gg::operator+` (GLfloat a, const `GgVector` &b)
- const `GgVector` `gg::operator-` (const `GgVector` &v)
- `GgVector` `gg::operator-` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::operator*` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::operator/` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::ggCross` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `gg::ggDot3` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `gg::ggLength3` (const `GgVector` &a)
- GLfloat `gg::ggDistance3` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `gg::ggNormalize3` (const `GgVector` &a)
- void `gg::ggNormalize3` (`GgVector` *a)
- GLfloat `gg::ggDot4` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `gg::ggLength4` (const `GgVector` &a)
- GLfloat `gg::ggDistance4` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `gg::ggNormalize4` (const `GgVector` &a)
- void `gg::ggNormalize4` (`GgVector` *a)
- `GgMatrix` `gg::ggIdentity` ()
- `GgMatrix` `gg::ggTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `gg::ggTranslate` (const GLfloat *t)
- `GgMatrix` `gg::ggTranslate` (const `GgVector` &t)
- `GgMatrix` `gg::ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `gg::ggScale` (const GLfloat *s)
- `GgMatrix` `gg::ggScale` (const `GgVector` &s)

- [GgMatrix gg::ggRotateX](#) (GLfloat a)
- [GgMatrix gg::ggRotateY](#) (GLfloat a)
- [GgMatrix gg::ggRotateZ](#) (GLfloat a)
- [GgMatrix gg::ggRotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- [GgMatrix gg::ggRotate](#) (const GLfloat *r, GLfloat a)
- [GgMatrix gg::ggRotate](#) (const [GgVector](#) &r, GLfloat a)
- [GgMatrix gg::ggRotate](#) (const GLfloat *r)
- [GgMatrix gg::ggRotate](#) (const [GgVector](#) &r)
- [GgMatrix gg::ggLookat](#) (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- [GgMatrix gg::ggLookat](#) (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- [GgMatrix gg::ggLookat](#) (const [GgVector](#) &e, const [GgVector](#) &t, const [GgVector](#) &u)
- [GgMatrix gg::ggOrthogonal](#) (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- [GgMatrix gg::ggFrustum](#) (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- [GgMatrix gg::ggPerspective](#) (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- [GgMatrix gg::ggTranspose](#) (const [GgMatrix](#) &m)
- [GgMatrix gg::ggInvert](#) (const [GgMatrix](#) &m)
- [GgMatrix gg::ggNormal](#) (const [GgMatrix](#) &m)
- [GgQuaternion gg::ggIdentityQuaternion](#) ()
- [GgQuaternion gg::ggMatrixQuaternion](#) (const GLfloat *a)
- [GgQuaternion gg::ggMatrixQuaternion](#) (const [GgMatrix](#) &m)
- [GgMatrix gg::ggQuaternionMatrix](#) (const [GgQuaternion](#) &q)
- [GgMatrix gg::ggQuaternionTransposeMatrix](#) (const [GgQuaternion](#) &q)
- [GgQuaternion gg::ggRotateQuaternion](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- [GgQuaternion gg::ggRotateQuaternion](#) (const GLfloat *v, GLfloat a)
- [GgQuaternion gg::ggRotateQuaternion](#) (const GLfloat *v)
- [GgQuaternion gg::ggEulerQuaternion](#) (GLfloat heading, GLfloat pitch, GLfloat roll)
- [GgQuaternion gg::ggEulerQuaternion](#) (const GLfloat *e)
- [GgQuaternion gg::ggEulerQuaternion](#) (const [GgVector](#) &e)
- [GgQuaternion gg::ggSlerp](#) (const GLfloat *a, const GLfloat *b, GLfloat t)
- [GgQuaternion gg::ggSlerp](#) (const [GgQuaternion](#) &q, const [GgQuaternion](#) &r, GLfloat t)
- [GgQuaternion gg::ggSlerp](#) (const [GgQuaternion](#) &q, const GLfloat *a, GLfloat t)
- [GgQuaternion gg::ggSlerp](#) (const GLfloat *a, const [GgQuaternion](#) &q, GLfloat t)
- GLfloat [gg::ggNorm](#) (const [GgQuaternion](#) &q)
- [GgQuaternion gg::ggNormalize](#) (const [GgQuaternion](#) &q)
- [GgQuaternion gg::ggConjugate](#) (const [GgQuaternion](#) &q)
- [GgQuaternion gg::ggInvert](#) (const [GgQuaternion](#) &q)
- bool [gg::ggSaveTga](#) (const std::string &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- bool [gg::ggSaveColor](#) (const std::string &name)
- bool [gg::ggSaveDepth](#) (const std::string &name)
- bool [gg::ggReadImage](#) (const std::string &name, std::vector< GLubyte > &image, GLsizei *pWidth, GLsizei *pHeight, GLenum *pFormat)
- GLuint [gg::ggLoadTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_<← RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_<← EDGE, bool swizzle=false)
- GLuint [gg::ggLoadImage](#) (const std::string &name, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE)
- void [gg::ggCreateNormalMap](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format, GLfloat nz, GLenum internal, std::vector< [GgVector](#) > &nmap)
- GLuint [gg::ggLoadHeight](#) (const std::string &name, GLfloat nz, GLsizei *pWidth=nullptr, GLsizei *p_<← Height=nullptr, GLenum internal=GL_RGBA)
- GLuint [gg::ggCreateShader](#) (const std::string &vsr, const std::string &fsr="", const std::string &gsr="", GLint nvarying=0, const char *const *varyings=nullptr, const std::string &vtext="vertex shader", const std_<← ::string &ftext="fragment shader", const std::string >ext="geometry shader")

- GLuint [gg::ggLoadShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [gg::ggLoadShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [gg::ggCreateComputeShader](#) (const std::string &csrc, const std::string &ctext="compute shader")
- GLuint [gg::ggLoadComputeShader](#) (const std::string &comp)
- std::shared_ptr< [GgPoints](#) > [gg::ggPointsCube](#) (GLsizei countv, GLfloat length=1.0f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgPoints](#) > [gg::ggPointsSphere](#) (GLsizei countv, GLfloat radius=0.5f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgTriangles](#) > [gg::ggRectangle](#) (GLfloat width=1.0f, GLfloat height=1.0f)
- std::shared_ptr< [GgTriangles](#) > [gg::ggEllipse](#) (GLfloat width=1.0f, GLfloat height=1.0f, GLuint slices=16)
- std::shared_ptr< [GgTriangles](#) > [gg::ggArraysObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsMesh](#) (GLuint slices, GLuint stacks, const GLfloat(*pos)[3], const GLfloat(*norm)[3]=nullptr)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsSphere](#) (GLfloat radius=1.0f, int slices=16, int stacks=8)
- bool [gg::ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, bool normalize=false)
- bool [gg::ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint [gg::ggBufferAlignment](#)
使用している GPU のバッファアライメント。

9.41.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の宣言。

著者

Kohe Tokoi

日付

July 17, 2025

[gg.h](#) に定義があります。

9.41.2 マクロ定義詳解

9.41.2.1 ggError

```
#define ggError()
```

OpenGL のエラーの発生を検知したときにソースファイル名と行番号を表示する。

覚え書き

このマクロを置いた場所（より前）で OpenGL のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する。リリースビルド時には無視される。

[gg.h](#) の 1392 行目に定義があります。

9.41.2.2 ggFBOError

```
#define ggFBOError()
```

FBO のエラーの発生を検知したときにソースファイル名と行番号を表示する。

覚え書き

このマクロを置いた場所（より前）で FBO のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する。リリースビルド時には無視される。

gg.h の 1419 行目に定義があります。

9.41.3 型定義詳解

9.41.3.1 pathChar

```
using pathChar = char
```

gg.h の 78 行目に定義があります。

9.41.3.2 pathString

```
using pathString = std::string
```

gg.h の 77 行目に定義があります。

9.41.4 関数詳解

9.41.4.1 TCharToUtf8()

```
std::string TCharToUtf8 (
    const pathString & cstring) [inline]
```

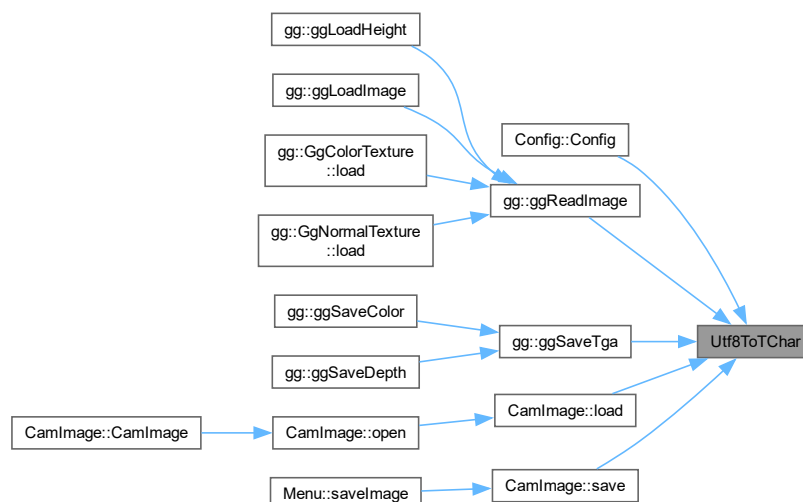
gg.h の 80 行目に定義があります。

9.41.4.2 Utf8ToTChar()

```
pathString Utf8ToTChar (
    const std::string & string) [inline]
```

gg.h の 79 行目に定義があります。

被呼び出し関係図:



9.42 gg.h

[詳解]

```

00001 #pragma once
00002
00026
00034
00035 // 標準ライブラリ
00036 #include <cmath>
00037 #include <array>
00038 #include <vector>
00039 #include <string>
00040 #include <memory>
00041 #include <cassert>
00042
00043 // Windows (Visual Studio) のとき
00044 #if defined(_WIN32)
00045 // 非推奨の警告を出さない
00046 # pragma warning(disable:4996)
00047 // 数学ライブラリの定数を使う
00048 # define _USE_MATH_DEFINES
00049 // MIN() / MAX マクロは使わない
00050 # define NOMINMAX
00051 // ファイルパスの文字コード
00052 #include <atlstr.h>
00053 using pathString = CString;
00054 using pathChar = wchar_t;
00055 extern pathString Utf8ToTChar(const std::string& string);
00056 extern std::string TCharToUtf8(const pathString& cstring);
00057 // デバッグビルドかどうか調べる
00058 # if defined(_DEBUG)
00059 #     if !defined(DEBUG)
00060 #         define DEBUG
00061 #     endif
00062 #     define GLFW3_CONFIGURATION "Debug"
00063 # else
00064 #     if !defined(NDEBUG)
00065 #         define NDEBUG
00066 #     endif
00067 #     define GLFW3_CONFIGURATION "Release"
00068 # endif
00069 // プラットフォームを調べる
00070 # if defined(_WIN64)
00071 #     define GLFW3_PLATFORM "x64"
00072 # else
00073 #     define GLFW3_PLATFORM "Win32"
00074 # endif
00075 #else
00076 // ファイルパスの文字コード
00077 using pathString = std::string;
00078 using pathChar = char;
00079 inline pathString Utf8ToTChar(const std::string& string) { return string; }
00080 inline std::string TCharToUtf8(const pathString& cstring) { return cstring; }
00081 #endif
00082
00084
00085 // macOS で "OpenGL deprecated" の警告を出さない
00086 #if defined(__APPLE__)
00087 # define GL_SILENCE_DEPRECATION
00088 #endif
00089
00090 // フレームワークに GLFW 3 を使う
00091 #if defined(IMGUI_IMPL_OPENGL_ES2)
00092 # define GLFW_INCLUDE_ES2
00093 #elif defined(IMGUI_IMPL_OPENGL_ES3)
00094 # define GLFW_INCLUDE_ES3
00095 #else
00096 # define GLFW_INCLUDE_GLCOREARB
00097 #endif
00098 #include <GLFW/glfw3.h>
00099
00100 // OpenGL 3.2 の API のエントリポイント
00101 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00102 extern PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00103 extern PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00104 extern PFNGLACTIVETEXTUREPROC glActiveTexture;
00105 extern PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00106 extern PFNGLATTACHSHADERPROC glAttachShader;
00107 extern PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00108 extern PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00109 extern PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00110 extern PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00111 extern PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00112 extern PFNGLBEGINQUERYPROC glBeginQuery;
00113 extern PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00114 extern PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;

```

```

00115 extern PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00116 extern PFNGLBINDBUFFERPROC glBindBuffer;
00117 extern PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00118 extern PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00119 extern PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00120 extern PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00121 extern PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00122 extern PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00123 extern PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00124 extern PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00125 extern PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00126 extern PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00127 extern PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00128 extern PFNGLBINDSAMPLERPROC glBindSampler;
00129 extern PFNGLBINDSAMPLERSPROC glBindSamplers;
00130 extern PFNGLBINDTEXTUREPROC glBindTexture;
00131 extern PFNGLBINDTEXTURESPROC glBindTextures;
00132 extern PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00133 extern PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00134 extern PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00135 extern PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00136 extern PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00137 extern PFNGLBLENDBARRIERKHRPROC glBlendBarrierKHR;
00138 extern PFNGLBLENDBARRIERNVPROC glBlendBarrierNV;
00139 extern PFNGLBLENDCOLORPROC glBlendColor;
00140 extern PFNGLBLEND EQUATIONIARBPROC glBlendEquationiARB;
00141 extern PFNGLBLEND EQUATIONIPROC glBlendEquationi;
00142 extern PFNGLBLEND EQUATIONPROC glBlendEquation;
00143 extern PFNGLBLEND EQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00144 extern PFNGLBLEND EQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00145 extern PFNGLBLEND EQUATIONSEPARATEPROC glBlendEquationSeparate;
00146 extern PFNGLBLEND FUNC IARBPROC glBlendFunciARB;
00147 extern PFNGLBLEND FUNC IPROC glBlendFunci;
00148 extern PFNGLBLEND FUNC PROC glBlendFunc;
00149 extern PFNGLBLEND FUNC SEPARATEIARBPROC glBlendFuncSeparateiARB;
00150 extern PFNGLBLEND FUNC SEPARATEIPROC glBlendFuncSeparatei;
00151 extern PFNGLBLEND FUNC SEPARATEPROC glBlendFuncSeparate;
00152 extern PFNGLBLEND PARAMETER INVPROC glBlendParameteriNV;
00153 extern PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00154 extern PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00155 extern PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00156 extern PFNGLBUFFERDATAPROC glBufferData;
00157 extern PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00158 extern PFNGLBUFFERSTORAGEPROC glBufferStorage;
00159 extern PFNGLBUFFERSUBDATAPROC glBufferSubData;
00160 extern PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00161 extern PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00162 extern PFNGLCHECKNAMEDFRAMEBUFFERSTATUS EXT PROC glCheckNamedFramebufferStatusEXT;
00163 extern PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;
00164 extern PFNGLCLAMP COLORPROC glClampColor;
00165 extern PFNGLCLEARBUFFERDATAPROC glClearBufferData;
00166 extern PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00167 extern PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00168 extern PFNGLCLEARBUFFERIVPROC glClearBufferiv;
00169 extern PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00170 extern PFNGLCLEARBUFFERUIVPROC glClearBufferuiv;
00171 extern PFNGLCLEARCOLORPROC glClearColor;
00172 extern PFNGLCLEARDEPTHFPROC glClearDepthf;
00173 extern PFNGLCLEARDEPTHPROC glClearDepth;
00174 extern PFNGLCLEARNAMEDBUFFERDATA EXT PROC glClearNamedBufferDataEXT;
00175 extern PFNGLCLEARNAMEDBUFFERDATAPROC glClearNamedBufferData;
00176 extern PFNGLCLEARNAMEDBUFFERSUBDATA EXT PROC glClearNamedBufferSubDataEXT;
00177 extern PFNGLCLEARNAMEDBUFFERSUBDATAPROC glClearNamedBufferSubData;
00178 extern PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00179 extern PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00180 extern PFNGLCLEARNAMEDFRAMEBUFFERIVPROC glClearNamedFramebufferiv;
00181 extern PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC glClearNamedFramebufferuiv;
00182 extern PFNGLCLEARPROC glClear;
00183 extern PFNGLCLEARSTENCILPROC glClearStencil;
00184 extern PFNGLCLEARTEXTIMAGEPROC glClearTexImage;
00185 extern PFNGLCLEARTEXTSUBIMAGEPROC glClearTexSubImage;
00186 extern PFNGLCLIENTATTRIBDEFAULT EXT PROC glClientAttribDefaultEXT;
00187 extern PFNGLCLIENTWAITSPROC glClientWaitSync;
00188 extern PFNGLCLIPCONTROLPROC glClipControl;
00189 extern PFNGLCOLORFORMATNVPROC glColorFormatNV;
00190 extern PFNGLCOLORMASKIPROC glColorMaski;
00191 extern PFNGLCOLORMASKPROC glColorMask;
00192 extern PFNGLCOMMANDLISTSEGMENTS NVPROC glCommandListSegmentsNV;
00193 extern PFNGLCOMPILECOMMANDLIST NVPROC glCompileCommandListNV;
00194 extern PFNGLCOMPILESHADER INCLUDE ARB PROC glCompileShaderIncludeARB;
00195 extern PFNGLCOMPILESHADERPROC glCompileShader;
00196 extern PFNGLCOMPRESSEDMULTITEXIMAGE1 DEXT PROC glCompressedMultiTexImage1DEXT;
00197 extern PFNGLCOMPRESSEDMULTITEXIMAGE2 DEXT PROC glCompressedMultiTexImage2DEXT;
00198 extern PFNGLCOMPRESSEDMULTITEXIMAGE3 DEXT PROC glCompressedMultiTexImage3DEXT;
00199 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE1 DEXT PROC glCompressedMultiTexSubImage1DEXT;
00200 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE2 DEXT PROC glCompressedMultiTexSubImage2DEXT;
00201 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE3 DEXT PROC glCompressedMultiTexSubImage3DEXT;

```



```
00202 extern PFNGLCOMPRESSEDTEXIMAGE1DPROC glCompressedTexImage1D;
00203 extern PFNGLCOMPRESSEDTEXIMAGE2DPROC glCompressedTexImage2D;
00204 extern PFNGLCOMPRESSEDTEXIMAGE3DPROC glCompressedTexImage3D;
00205 extern PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC glCompressedTexSubImage1D;
00206 extern PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC glCompressedTexSubImage2D;
00207 extern PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC glCompressedTexSubImage3D;
00208 extern PFNGLCOMPRESSEDTEXTUREIMAGE1DXTPROC glCompressedTextureImage1DEXT;
00209 extern PFNGLCOMPRESSEDTEXTUREIMAGE2DXTPROC glCompressedTextureImage2DEXT;
00210 extern PFNGLCOMPRESSEDTEXTUREIMAGE3DXTPROC glCompressedTextureImage3DEXT;
00211 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DXTPROC glCompressedTextureSubImage1DEXT;
00212 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00213 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DXTPROC glCompressedTextureSubImage2DEXT;
00214 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00215 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DXTPROC glCompressedTextureSubImage3DEXT;
00216 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00217 extern PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00218 extern PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00219 extern PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00220 extern PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00221 extern PFNGLCOPYMULTITEXIMAGE1DXTPROC glCopyMultiTexImage1DEXT;
00222 extern PFNGLCOPYMULTITEXIMAGE2DXTPROC glCopyMultiTexImage2DEXT;
00223 extern PFNGLCOPYMULTITEXSUBIMAGE1DXTPROC glCopyMultiTexSubImage1DEXT;
00224 extern PFNGLCOPYMULTITEXSUBIMAGE2DXTPROC glCopyMultiTexSubImage2DEXT;
00225 extern PFNGLCOPYMULTITEXSUBIMAGE3DXTPROC glCopyMultiTexSubImage3DEXT;
00226 extern PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00227 extern PFNGLCOPYPATHNVPROC glCopyPathNV;
00228 extern PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00229 extern PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00230 extern PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00231 extern PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00232 extern PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00233 extern PFNGLCOPYTEXTUREIMAGE1DXTPROC glCopyTextureImage1DEXT;
00234 extern PFNGLCOPYTEXTUREIMAGE2DXTPROC glCopyTextureImage2DEXT;
00235 extern PFNGLCOPYTEXTURESUBIMAGE1DXTPROC glCopyTextureSubImage1DEXT;
00236 extern PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00237 extern PFNGLCOPYTEXTURESUBIMAGE2DXTPROC glCopyTextureSubImage2DEXT;
00238 extern PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00239 extern PFNGLCOPYTEXTURESUBIMAGE3DXTPROC glCopyTextureSubImage3DEXT;
00240 extern PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00241 extern PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00242 extern PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00243 extern PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00244 extern PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00245 extern PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00246 extern PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00247 extern PFNGLCREATEBUFFERSPROC glCreateBuffers;
00248 extern PFNGLCREATECOMMANDLISTSNVPROC glCreateCommandListsNV;
00249 extern PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00250 extern PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00251 extern PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00252 extern PFNGLCREATEPROGRAMPROC glCreateProgram;
00253 extern PFNGLCREATEQUERIESPROC glCreateQueries;
00254 extern PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00255 extern PFNGLCREATESAMPLERSPROC glCreateSamplers;
00256 extern PFNGLCREATESHADERPROC glCreateShader;
00257 extern PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00258 extern PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00259 extern PFNGLCREATESTATESNVPROC glCreateStatesNV;
00260 extern PFNGLCREATESYNCFROMCLEVENTARBPROC glCreateSyncFromCLeventARB;
00261 extern PFNGLCREATETEXTURESPROC glCreateTextures;
00262 extern PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00263 extern PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00264 extern PFNGLCULLFACEPROC glCullFace;
00265 extern PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00266 extern PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00267 extern PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00268 extern PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00269 extern PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00270 extern PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00271 extern PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00272 extern PFNGLDELETECOMMANDLISTSNVPROC glDeleteCommandListsNV;
00273 extern PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00274 extern PFNGLDELETONAMEDSTRINGARBPROC glDeleteNamedStringARB;
00275 extern PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00276 extern PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00277 extern PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00278 extern PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00279 extern PFNGLDELETEPROGRAMPROC glDeleteProgram;
00280 extern PFNGLDELETEQUERIESPROC glDeleteQueries;
00281 extern PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00282 extern PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00283 extern PFNGLDELETESHADERPROC glDeleteShader;
00284 extern PFNGLDELETETESTATESNVPROC glDeleteStatesNV;
00285 extern PFNGLDELETESYNCPROC glDeleteSync;
00286 extern PFNGLDELETETEXTURESPROC glDeleteTextures;
00287 extern PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00288 extern PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
```

```

00289 extern PFNGLDEPTHFUNCPROC glDepthFunc;
00290 extern PFNGLDEPTHTMASKPROC glDepthMask;
00291 extern PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00292 extern PFNGLDEPTHRANGEFPROC glDepthRangef;
00293 extern PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00294 extern PFNGLDEPTHRANGEPROC glDepthRange;
00295 extern PFNGLDETACHSHADERPROC glDetachShader;
00296 extern PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00297 extern PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00298 extern PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00299 extern PFNGLDISABLEIPROC glDisablei;
00300 extern PFNGLDISABLEPROC glDisable;
00301 extern PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00302 extern PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00303 extern PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00304 extern PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00305 extern PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00306 extern PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00307 extern PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00308 extern PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00309 extern PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00310 extern PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00311 extern PFNGLDRAWARRAYSINSTANCEDEXTPROC glDrawArraysInstancedEXT;
00312 extern PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00313 extern PFNGLDRAWARRAYSPROC glDrawArrays;
00314 extern PFNGLDRAWBUFFERPROC glDrawBuffer;
00315 extern PFNGLDRAWBUFFERSPROC glDrawBuffers;
00316 extern PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00317 extern PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00318 extern PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00319 extern PFNGLDRAWCOMMANDSSTATESNVPROC glDrawCommandsStatesNV;
00320 extern PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00321 extern PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00322 extern PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00323 extern PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00324 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC
    glDrawElementsInstancedBaseVertexBaseInstance;
00325 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00326 extern PFNGLDRAWELEMENTSINSTANCEDEXTPROC glDrawElementsInstancedEXT;
00327 extern PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00328 extern PFNGLDRAWELEMENTSPROC glDrawElements;
00329 extern PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00330 extern PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00331 extern PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00332 extern PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00333 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00334 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00335 extern PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00336 extern PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00337 extern PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00338 extern PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00339 extern PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00340 extern PFNGLENABLEIPROC glEnablei;
00341 extern PFNGLENABLEPROC glEnable;
00342 extern PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00343 extern PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00344 extern PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00345 extern PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
00346 extern PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00347 extern PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00348 extern PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00349 extern PFNGLENDPERFQUERYINTELPROC glEndPerfQueryIntel;
00350 extern PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00351 extern PFNGLENDQUERYPROC glEndQuery;
00352 extern PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00353 extern PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00354 extern PFNGLFENCESYNCPROC glFenceSync;
00355 extern PFNGLFINISHPROC glFinish;
00356 extern PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00357 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00358 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00359 extern PFNGLFLUSHPROC glFlush;
00360 extern PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00361 extern PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00362 extern PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00363 extern PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00364 extern PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00365 extern PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00366 extern PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00367 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00368 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00369 extern PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00370 extern PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00371 extern PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00372 extern PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00373 extern PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00374 extern PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;

```

```
00375 extern PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00376 extern PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00377 extern PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00378 extern PFNGLFRONTFACEPROC glFrontFace;
00379 extern PFNGLGENBUFFERSPROC glGenBuffers;
00380 extern PFNGLGENERATEMIPMAPPROC glGenerateMipmap;
00381 extern PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenerateMultiTexMipmapEXT;
00382 extern PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenerateTextureMipmapEXT;
00383 extern PFNGLGENERATETEXTUREMIPMAPPROC glGenerateTextureMipmap;
00384 extern PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00385 extern PFNGLGENPATHSNVPROC glGenPathsNV;
00386 extern PFNGLGENPERFMONITORSAMDPROC glGenPerfMonitorsAMD;
00387 extern PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00388 extern PFNGLGENQUERIESPROC glGenQueries;
00389 extern PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00390 extern PFNGLGENSAMPLERSPROC glGenSamplers;
00391 extern PFNGLGENTEXTURESPROC glGenTextures;
00392 extern PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00393 extern PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00394 extern PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00395 extern PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00396 extern PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00397 extern PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00398 extern PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00399 extern PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00400 extern PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00401 extern PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00402 extern PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00403 extern PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00404 extern PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00405 extern PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00406 extern PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00407 extern PFNGLGETBOOLEANI_VPROC glGetBooleani_v;
00408 extern PFNGLGETBOOLEANVPROC glGetBooleanv;
00409 extern PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00410 extern PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00411 extern PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00412 extern PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00413 extern PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00414 extern PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00415 extern PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00416 extern PFNGLGETCOMPRESSEDTEXIMAGEPROC glGetCompressedTexImage;
00417 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00418 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00419 extern PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00420 extern PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00421 extern PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00422 extern PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00423 extern PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00424 extern PFNGLGETDOUBLEI_VEXTPROC glGetDoublei_vEXT;
00425 extern PFNGLGETDOUBLEI_VPROC glGetDoublei_v;
00426 extern PFNGLGETDOUBLEVPROC glGetDoublev;
00427 extern PFNGLGETERRORPROC glGetError;
00428 extern PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00429 extern PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00430 extern PFNGLGETFLOATI_VEXTPROC glGetFloati_vEXT;
00431 extern PFNGLGETFLOATI_VPROC glGetFloati_v;
00432 extern PFNGLGETFLOATVPROC glGetFloatv;
00433 extern PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00434 extern PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00435 extern PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00436 extern PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00437 extern PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00438 extern PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00439 extern PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00440 extern PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00441 extern PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00442 extern PFNGLGETINTEGER64I_VPROC glGetInteger64i_v;
00443 extern PFNGLGETINTEGER64VPROC glGetInteger64v;
00444 extern PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00445 extern PFNGLGETINTEGERI_VPROC glGetIntegeri_v;
00446 extern PFNGLGETINTEGERUI64I_VNVPROC glGetIntegerui64i_vNV;
00447 extern PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00448 extern PFNGLGETINTEGERVPROC glGetInteger;
00449 extern PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00450 extern PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00451 extern PFNGLGETINTERNALFORMATSAMPLEI_VNVPROC glGetInternalformatSamplei_vNV;
00452 extern PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00453 extern PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00454 extern PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00455 extern PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00456 extern PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00457 extern PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00458 extern PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00459 extern PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00460 extern PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00461 extern PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
```

```
00462 extern PFNGLGETMULTITEXPARAMETERIIVEXTPROC glGetMultiTexParameterIivEXT;
00463 extern PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameterIuivEXT;
00464 extern PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00465 extern PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00466 extern PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00467 extern PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00468 extern PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00469 extern PFNGLGETNAMEDBUFFERPOINTERVEXTPROC glGetNamedBufferPointervEXT;
00470 extern PFNGLGETNAMEDBUFFERPOINTERVPROC glGetNamedBufferPointerv;
00471 extern PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00472 extern PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00473 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC
    glGetNamedFramebufferAttachmentParameterivEXT;
00474 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00475 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00476 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00477 extern PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00478 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00479 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00480 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameterIivEXT;
00481 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameterIuivEXT;
00482 extern PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00483 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00484 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00485 extern PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00486 extern PFNGLGETNAMEDSTRINGIVARBPROC glGetNamedStringivARB;
00487 extern PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00488 extern PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00489 extern PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00490 extern PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00491 extern PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00492 extern PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvARB;
00493 extern PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00494 extern PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvARB;
00495 extern PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00496 extern PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64vARB;
00497 extern PFNGLGETNUNIFORMIVARBPROC glGetnUniformivARB;
00498 extern PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00499 extern PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64vARB;
00500 extern PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivARB;
00501 extern PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00502 extern PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00503 extern PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00504 extern PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00505 extern PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00506 extern PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00507 extern PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00508 extern PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00509 extern PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00510 extern PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00511 extern PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00512 extern PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00513 extern PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00514 extern PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00515 extern PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00516 extern PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00517 extern PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00518 extern PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00519 extern PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00520 extern PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00521 extern PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00522 extern PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00523 extern PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00524 extern PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00525 extern PFNGLGETPOINTERI.VEXTPROC glGetPointerivEXT;
00526 extern PFNGLGETPOINTERVPROC glGetPointerv;
00527 extern PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00528 extern PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00529 extern PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00530 extern PFNGLGETPROGRAMIVPROC glGetProgramiv;
00531 extern PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00532 extern PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00533 extern PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00534 extern PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00535 extern PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00536 extern PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00537 extern PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00538 extern PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00539 extern PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00540 extern PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00541 extern PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00542 extern PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00543 extern PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00544 extern PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00545 extern PFNGLGETQUERYIVPROC glGetQueryiv;
00546 extern PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00547 extern PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
```



```
00548 extern PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00549 extern PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00550 extern PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00551 extern PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00552 extern PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00553 extern PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00554 extern PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00555 extern PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00556 extern PFNGLGETSHADERIVPROC glGetShaderiv;
00557 extern PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00558 extern PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00559 extern PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00560 extern PFNGLGETSTRINGIPROC glGetStringi;
00561 extern PFNGLGETSTRINGPROC glGetString;
00562 extern PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00563 extern PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00564 extern PFNGLGETSYNCPROC glGetSynciv;
00565 extern PFNGLGETTEXIMAGEPROC glGetTexImage;
00566 extern PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00567 extern PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00568 extern PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00569 extern PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00570 extern PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00571 extern PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00572 extern PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00573 extern PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00574 extern PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00575 extern PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00576 extern PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00577 extern PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00578 extern PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00579 extern PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00580 extern PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00581 extern PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00582 extern PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00583 extern PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00584 extern PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00585 extern PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00586 extern PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00587 extern PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00588 extern PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00589 extern PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00590 extern PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00591 extern PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64v;
00592 extern PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00593 extern PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00594 extern PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00595 extern PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00596 extern PFNGLGETUNIFORMDVPROC glGetUniformdv;
00597 extern PFNGLGETUNIFORMFVPROC glGetUniformfv;
00598 extern PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00599 extern PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00600 extern PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00601 extern PFNGLGETUNIFORMIVPROC glGetUniformiv;
00602 extern PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00603 extern PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00604 extern PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00605 extern PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00606 extern PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00607 extern PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00608 extern PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00609 extern PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00610 extern PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayInteger_vEXT;
00611 extern PFNGLGETVERTEXARRAYIVPROC glGetVertexArrayiv;
00612 extern PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00613 extern PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointer_vEXT;
00614 extern PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00615 extern PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00616 extern PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribIiv;
00617 extern PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00618 extern PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00619 extern PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribldv;
00620 extern PFNGLGETVERTEXATTRIBLI64VNVPROC glGetVertexAttribLi64vNV;
00621 extern PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00622 extern PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00623 extern PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointerv;
00624 extern PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00625 extern PFNGLHINTPROC glHint;
00626 extern PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00627 extern PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00628 extern PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00629 extern PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00630 extern PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00631 extern PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00632 extern PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00633 extern PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
00634 extern PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
```

```

00635 extern PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00636 extern PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00637 extern PFNGLISBUFFERPROC glIsBuffer;
00638 extern PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00639 extern PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00640 extern PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00641 extern PFNGLISENABLEDIPROC glIsEnabledi;
00642 extern PFNGLISENABLEDPROC glIsEnabled;
00643 extern PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00644 extern PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00645 extern PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00646 extern PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00647 extern PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00648 extern PFNGLISPATHTNVPROC glIsPathNV;
00649 extern PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00650 extern PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
00651 extern PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00652 extern PFNGLISPROGRAMPROC glIsProgram;
00653 extern PFNGLISQUERYPROC glIsQuery;
00654 extern PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00655 extern PFNGLISSAMPLERPROC glIsSampler;
00656 extern PFNGLISSHADERPROC glIsShader;
00657 extern PFNGLISSTATENVPROC glIsStateNV;
00658 extern PFNGLISSYNCPROC glIsSync;
00659 extern PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00660 extern PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00661 extern PFNGLISTEXTUREPROC glIsTexture;
00662 extern PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00663 extern PFNGLISVERTEXARRAYPROC glIsVertexArray;
00664 extern PFNGLLABELOBJECTEXTPROC glLabelObjectEXT;
00665 extern PFNGLLINEWIDTHPROC glLineWidth;
00666 extern PFNGLLINKPROGRAMPROC glLinkProgram;
00667 extern PFNGLLISTDRAWCOMMANDSSTATESCIENTNVPROC glListDrawCommandsStatesClientNV;
00668 extern PFNGLLOGICOPPROC glLogicOp;
00669 extern PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00670 extern PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00671 extern PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00672 extern PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;
00673 extern PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00674 extern PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00675 extern PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00676 extern PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00677 extern PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00678 extern PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00679 extern PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00680 extern PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00681 extern PFNGLMAPBUFFERPROC glMapBuffer;
00682 extern PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00683 extern PFNGLMAPNAMEDBUFFEREXTPROC glMapNamedBufferEXT;
00684 extern PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00685 extern PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
00686 extern PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00687 extern PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
00688 extern PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00689 extern PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00690 extern PFNGLMATRIXLOADDEXTPROC glMatrixLoaddeEXT;
00691 extern PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00692 extern PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00693 extern PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00694 extern PFNGLMATRIXLOADTRANSPOSEDEXTPROC glMatrixLoadTransposedEXT;
00695 extern PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00696 extern PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00697 extern PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00698 extern PFNGLMATRIXMULTDEXTPROC glMatrixMultdeEXT;
00699 extern PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00700 extern PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00701 extern PFNGLMATRIXMULTTRANSPOSEDEXTPROC glMatrixMultTransposedEXT;
00702 extern PFNGLMATRIXMULTTRANSPOSEFEXTPROC glMatrixMultTransposefEXT;
00703 extern PFNGLMATRIXORTHOREXTPROC glMatrixOrthoEXT;
00704 extern PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00705 extern PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00706 extern PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00707 extern PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00708 extern PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00709 extern PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00710 extern PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00711 extern PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00712 extern PFNGLMAXSHADERCOMPILERTHREADSARBPROC glMaxShaderCompilerThreadsARB;
00713 extern PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00714 extern PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00715 extern PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00716 extern PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00717 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00718 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00719 extern PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00720 extern PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00721 extern PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;

```

```
00722 extern PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00723 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00724 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00725 extern PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00726 extern PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00727 extern PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00728 extern PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00729 extern PFNGLMULTITEXCOORDPOINTEREXTPROC glMultiTexCoordPointerEXT;
00730 extern PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00731 extern PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00732 extern PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00733 extern PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00734 extern PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00735 extern PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00736 extern PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00737 extern PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00738 extern PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00739 extern PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00740 extern PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DEXT;
00741 extern PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00742 extern PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00743 extern PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00744 extern PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00745 extern PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00746 extern PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameterIivEXT;
00747 extern PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameterIuivEXT;
00748 extern PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00749 extern PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00750 extern PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1DEXT;
00751 extern PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00752 extern PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00753 extern PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00754 extern PFNGLNAMEDBUFFERDATAPROC glNamedBufferData;
00755 extern PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00756 extern PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00757 extern PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00758 extern PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00759 extern PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00760 extern PFNGLNAMEDBUFFERSUBDATAPROC glNamedBufferSubData;
00761 extern PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00762 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00763 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00764 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00765 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00766 extern PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00767 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00768 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00769 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00770 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00771 extern PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00772 extern PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
00773 extern PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00774 extern PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00775 extern PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00776 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00777 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00778 extern PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00779 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00780 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
00781 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00782 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00783 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00784 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00785 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00786 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00787 extern PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00788 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC glNamedProgramLocalParametersI4ivEXT;
00789 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00790 extern PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00791 extern PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00792 extern PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLECOVERAGEXTPROC
    glNamedRenderbufferStorageMultisampleCoverageEXT;
00793 extern PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00794 extern PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00795 extern PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00796 extern PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00797 extern PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00798 extern PFNGLOBJEC TLABELPROC glObjectLabel;
00799 extern PFNGLOBJECTPTRLABELPROC glObjectPtrLabel;
00800 extern PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00801 extern PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00802 extern PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00803 extern PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00804 extern PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00805 extern PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00806 extern PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00807 extern PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
```

```
00808 extern PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00809 extern PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00810 extern PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00811 extern PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00812 extern PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00813 extern PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00814 extern PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00815 extern PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00816 extern PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00817 extern PFNGLPATHSTRINGNVPROC glPathStringNV;
00818 extern PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00819 extern PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00820 extern PFNGLPAUSESETTRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00821 extern PFNGLPIXELSTOREFPROC glPixelStoref;
00822 extern PFNGLPIXELSTOREIPROC glPixelStorei;
00823 extern PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00824 extern PFNGLPOINTPARAMETERFPROC glPointParameterf;
00825 extern PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00826 extern PFNGLPOINTPARAMETERIPROC glPointParameteri;
00827 extern PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00828 extern PFNGLPOINTSIZEPROC glPointSize;
00829 extern PFNGLPOLYGONMODEPROC glPolygonMode;
00830 extern PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00831 extern PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00832 extern PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00833 extern PFNGLPOPGROUPMARKEREXTPROC glPopGroupMarkerEXT;
00834 extern PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00835 extern PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00836 extern PFNGLPROGRAMBINARYPROC glProgramBinary;
00837 extern PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00838 extern PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00839 extern PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00840 extern PFNGLPROGRAMUNIFORM1DEXTPROC glProgramUniform1dEXT;
00841 extern PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00842 extern PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00843 extern PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00844 extern PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00845 extern PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00846 extern PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00847 extern PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00848 extern PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00849 extern PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00850 extern PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00851 extern PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00852 extern PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00853 extern PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00854 extern PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00855 extern PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00856 extern PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00857 extern PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00858 extern PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00859 extern PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00860 extern PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00861 extern PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00862 extern PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00863 extern PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00864 extern PFNGLPROGRAMUNIFORM2DEXTPROC glProgramUniform2dEXT;
00865 extern PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00866 extern PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
00867 extern PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00868 extern PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00869 extern PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00870 extern PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00871 extern PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00872 extern PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00873 extern PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00874 extern PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00875 extern PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00876 extern PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00877 extern PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00878 extern PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00879 extern PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00880 extern PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00881 extern PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00882 extern PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00883 extern PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00884 extern PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00885 extern PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00886 extern PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00887 extern PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00888 extern PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00889 extern PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00890 extern PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00891 extern PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00892 extern PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00893 extern PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00894 extern PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
```



```
00895 extern PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00896 extern PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00897 extern PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00898 extern PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00899 extern PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64vNV;
00900 extern PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00901 extern PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00902 extern PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00903 extern PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00904 extern PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00905 extern PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00906 extern PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00907 extern PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64vNV;
00908 extern PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00909 extern PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00910 extern PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00911 extern PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00912 extern PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00913 extern PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00914 extern PFNGLPROGRAMUNIFORM4DVECTPROC glProgramUniform4dvEXT;
00915 extern PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00916 extern PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00917 extern PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00918 extern PFNGLPROGRAMUNIFORM4FVECTPROC glProgramUniform4fvEXT;
00919 extern PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00920 extern PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00921 extern PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00922 extern PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00923 extern PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64vNV;
00924 extern PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00925 extern PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00926 extern PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00927 extern PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00928 extern PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00929 extern PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00930 extern PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00931 extern PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64vNV;
00932 extern PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00933 extern PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00934 extern PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00935 extern PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00936 extern PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00937 extern PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00938 extern PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00939 extern PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64vNV;
00940 extern PFNGLPROGRAMUNIFORMMATRIX2DVECTPROC glProgramUniformMatrix2dvEXT;
00941 extern PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00942 extern PFNGLPROGRAMUNIFORMMATRIX2FVECTPROC glProgramUniformMatrix2fvEXT;
00943 extern PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00944 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVECTPROC glProgramUniformMatrix2x3dvEXT;
00945 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00946 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVECTPROC glProgramUniformMatrix2x3fvEXT;
00947 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00948 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVECTPROC glProgramUniformMatrix2x4dvEXT;
00949 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00950 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVECTPROC glProgramUniformMatrix2x4fvEXT;
00951 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00952 extern PFNGLPROGRAMUNIFORMMATRIX3DVECTPROC glProgramUniformMatrix3dvEXT;
00953 extern PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00954 extern PFNGLPROGRAMUNIFORMMATRIX3FVECTPROC glProgramUniformMatrix3fvEXT;
00955 extern PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00956 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVECTPROC glProgramUniformMatrix3x2dvEXT;
00957 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00958 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVECTPROC glProgramUniformMatrix3x2fvEXT;
00959 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00960 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVECTPROC glProgramUniformMatrix3x4dvEXT;
00961 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00962 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVECTPROC glProgramUniformMatrix3x4fvEXT;
00963 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00964 extern PFNGLPROGRAMUNIFORMMATRIX4DVECTPROC glProgramUniformMatrix4dvEXT;
00965 extern PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00966 extern PFNGLPROGRAMUNIFORMMATRIX4FVECTPROC glProgramUniformMatrix4fvEXT;
00967 extern PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00968 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVECTPROC glProgramUniformMatrix4x2dvEXT;
00969 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00970 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVECTPROC glProgramUniformMatrix4x2fvEXT;
00971 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00972 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVECTPROC glProgramUniformMatrix4x3dvEXT;
00973 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00974 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVECTPROC glProgramUniformMatrix4x3fvEXT;
00975 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00976 extern PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00977 extern PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64vNV;
00978 extern PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00979 extern PFNGLPUSHCLIENTATTRIBDEFAULTTEXTPROC glPushClientAttribDefaultEXT;
00980 extern PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00981 extern PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
```

```

00982 extern PFNGLQUERYCOUNTERPROC glQueryCounter;
00983 extern PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00984 extern PFNGLREADBUFFERPROC glReadBuffer;
00985 extern PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
00986 extern PFNGLREADNPIXELSPROC glReadnPixels;
00987 extern PFNGLREADPIXELSPROC glReadPixels;
00988 extern PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
00989 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
00990 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
00991 extern PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
00992 extern PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
00993 extern PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
00994 extern PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
00995 extern PFNGLSAMPLEMASKIPROC glSampleMaski;
00996 extern PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
00997 extern PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
00998 extern PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
00999 extern PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01000 extern PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01001 extern PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01002 extern PFNGLSCISSORARRAYVPROC glScissorArrayv;
01003 extern PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01004 extern PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01005 extern PFNGLSCISSORPROC glScissor;
01006 extern PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01007 extern PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01008 extern PFNGLSHADERBINARYPROC glShaderBinary;
01009 extern PFNGLSHADERSOURCEPROC glShaderSource;
01010 extern PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01011 extern PFNGLSIGNALVKFENCENVPROC glSignalVkFenceNV;
01012 extern PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01013 extern PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01014 extern PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01015 extern PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01016 extern PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01017 extern PFNGLSTENCILFUNCPROC glStencilFunc;
01018 extern PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01019 extern PFNGLSTENCILMASKPROC glStencilMask;
01020 extern PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01021 extern PFNGLSTENCILOPPROC glStencilOp;
01022 extern PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01023 extern PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01024 extern PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01025 extern PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01026 extern PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01027 extern PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01028 extern PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01029 extern PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01030 extern PFNGLTEXBUFFERARBPROC glTexBufferARB;
01031 extern PFNGLTEXBUFFERPROC glTexBuffer;
01032 extern PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01033 extern PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01034 extern PFNGLTEXIMAGE1DPROC glTexImage1D;
01035 extern PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01036 extern PFNGLTEXIMAGE2DPROC glTexImage2D;
01037 extern PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01038 extern PFNGLTEXIMAGE3DPROC glTexImage3D;
01039 extern PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01040 extern PFNGLTEXPARAMETERFPROC glTexParameterf;
01041 extern PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01042 extern PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01043 extern PFNGLTEXPARAMETERIPROC glTexParameterI;
01044 extern PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01045 extern PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01046 extern PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01047 extern PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01048 extern PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01049 extern PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01050 extern PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01051 extern PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01052 extern PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01053 extern PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01054 extern PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01055 extern PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01056 extern PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01057 extern PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01058 extern PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01059 extern PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01060 extern PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01061 extern PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01062 extern PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01063 extern PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01064 extern PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01065 extern PFNGLTEXTUREPARAMETERFPROC glTextureParameterf;
01066 extern PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01067 extern PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01068 extern PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;

```

```
01069 extern PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameterIivEXT;
01070 extern PFNGLTEXTUREPARAMETERIIVPROC glTextureParameterIiv;
01071 extern PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01072 extern PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01073 extern PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameterIuiv;
01074 extern PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01075 extern PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01076 extern PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01077 extern PFNGLTEXTURESTORAGE1DXTPROC glTextureStorage1DEXT;
01078 extern PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01079 extern PFNGLTEXTURESTORAGE2DXTPROC glTextureStorage2DEXT;
01080 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01081 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01082 extern PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01083 extern PFNGLTEXTURESTORAGE3DXTPROC glTextureStorage3DEXT;
01084 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01085 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01086 extern PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01087 extern PFNGLTEXTURESUBIMAGE1DXTPROC glTextureSubImage1DEXT;
01088 extern PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01089 extern PFNGLTEXTURESUBIMAGE2DXTPROC glTextureSubImage2DEXT;
01090 extern PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01091 extern PFNGLTEXTURESUBIMAGE3DXTPROC glTextureSubImage3DEXT;
01092 extern PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01093 extern PFNGLTEXTUREVIEWPROC glTextureView;
01094 extern PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01095 extern PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01096 extern PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01097 extern PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01098 extern PFNGLUNIFORM1DPROC glUniform1d;
01099 extern PFNGLUNIFORM1DVPROC glUniform1dv;
01100 extern PFNGLUNIFORM1FPROC glUniform1f;
01101 extern PFNGLUNIFORM1FVPROC glUniform1fv;
01102 extern PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01103 extern PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01104 extern PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01105 extern PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01106 extern PFNGLUNIFORM1IPROC glUniform1i;
01107 extern PFNGLUNIFORM1IVPROC glUniform1iv;
01108 extern PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01109 extern PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01110 extern PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01111 extern PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01112 extern PFNGLUNIFORM1UIPROC glUniform1ui;
01113 extern PFNGLUNIFORM1UIVPROC glUniform1uiv;
01114 extern PFNGLUNIFORM2DPROC glUniform2d;
01115 extern PFNGLUNIFORM2DVPROC glUniform2dv;
01116 extern PFNGLUNIFORM2FPROC glUniform2f;
01117 extern PFNGLUNIFORM2FVPROC glUniform2fv;
01118 extern PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01119 extern PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01120 extern PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01121 extern PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01122 extern PFNGLUNIFORM2IPROC glUniform2i;
01123 extern PFNGLUNIFORM2IVPROC glUniform2iv;
01124 extern PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01125 extern PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01126 extern PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01127 extern PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01128 extern PFNGLUNIFORM2UIPROC glUniform2ui;
01129 extern PFNGLUNIFORM2UIVPROC glUniform2uiv;
01130 extern PFNGLUNIFORM3DPROC glUniform3d;
01131 extern PFNGLUNIFORM3DVPROC glUniform3dv;
01132 extern PFNGLUNIFORM3FPROC glUniform3f;
01133 extern PFNGLUNIFORM3FVPROC glUniform3fv;
01134 extern PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01135 extern PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01136 extern PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01137 extern PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01138 extern PFNGLUNIFORM3IPROC glUniform3i;
01139 extern PFNGLUNIFORM3IVPROC glUniform3iv;
01140 extern PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01141 extern PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01142 extern PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01143 extern PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01144 extern PFNGLUNIFORM3UIPROC glUniform3ui;
01145 extern PFNGLUNIFORM3UIVPROC glUniform3uiv;
01146 extern PFNGLUNIFORM4DPROC glUniform4d;
01147 extern PFNGLUNIFORM4DVPROC glUniform4dv;
01148 extern PFNGLUNIFORM4FPROC glUniform4f;
01149 extern PFNGLUNIFORM4FVPROC glUniform4fv;
01150 extern PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01151 extern PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01152 extern PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01153 extern PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01154 extern PFNGLUNIFORM4IPROC glUniform4i;
01155 extern PFNGLUNIFORM4IVPROC glUniform4iv;
```

```

01156 extern PFNGLUNIFORM4UI64ARBPROC glUniformui64ARB;
01157 extern PFNGLUNIFORM4UI64NVPROC glUniformui64NV;
01158 extern PFNGLUNIFORM4UI64VARBPROC glUniformui64vARB;
01159 extern PFNGLUNIFORM4UI64VNVPROC glUniformui64vNV;
01160 extern PFNGLUNIFORM4UIPROC glUniformui;
01161 extern PFNGLUNIFORM4UIVPROC glUniformuiiv;
01162 extern PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01163 extern PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01164 extern PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01165 extern PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01166 extern PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01167 extern PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01168 extern PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01169 extern PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01170 extern PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01171 extern PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01172 extern PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01173 extern PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01174 extern PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01175 extern PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01176 extern PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01177 extern PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01178 extern PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01179 extern PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01180 extern PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01181 extern PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01182 extern PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01183 extern PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01184 extern PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01185 extern PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01186 extern PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01187 extern PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01188 extern PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01189 extern PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01190 extern PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01191 extern PFNGLUSEPROGRAMPROC glUseProgram;
01192 extern PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01193 extern PFNGLUSESHADERPROGRAMEXTPROC glUseProgramProgramEXT;
01194 extern PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01195 extern PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01196 extern PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01197 extern PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01198 extern PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01199 extern PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01200 extern PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01201 extern PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01202 extern PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01203 extern PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01204 extern PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01205 extern PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01206 extern PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01207 extern PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01208 extern PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01209 extern PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01210 extern PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01211 extern PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01212 extern PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01213 extern PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01214 extern PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01215 extern PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01216 extern PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC glVertexArrayVertexAttribLFormatEXT;
01217 extern PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01218 extern PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01219 extern PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01220 extern PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01221 extern PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01222 extern PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01223 extern PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01224 extern PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01225 extern PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01226 extern PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01227 extern PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01228 extern PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01229 extern PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01230 extern PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01231 extern PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01232 extern PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01233 extern PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01234 extern PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01235 extern PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01236 extern PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01237 extern PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01238 extern PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01239 extern PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01240 extern PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01241 extern PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01242 extern PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;

```



```
01243 extern PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01244 extern PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01245 extern PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01246 extern PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01247 extern PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01248 extern PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01249 extern PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01250 extern PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01251 extern PFNGLVERTEXATTRIB4NUBVPROC glVertexAttrib4Nubv;
01252 extern PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01253 extern PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01254 extern PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01255 extern PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01256 extern PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01257 extern PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01258 extern PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01259 extern PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01260 extern PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01261 extern PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01262 extern PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01263 extern PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01264 extern PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01265 extern PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01266 extern PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01267 extern PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01268 extern PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01269 extern PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01270 extern PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01271 extern PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01272 extern PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01273 extern PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01274 extern PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01275 extern PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01276 extern PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01277 extern PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01278 extern PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01279 extern PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01280 extern PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01281 extern PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01282 extern PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01283 extern PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01284 extern PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01285 extern PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01286 extern PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01287 extern PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01288 extern PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01289 extern PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01290 extern PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01291 extern PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01292 extern PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01293 extern PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01294 extern PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01295 extern PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01296 extern PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01297 extern PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01298 extern PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01299 extern PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01300 extern PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01301 extern PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01302 extern PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01303 extern PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01304 extern PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01305 extern PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01306 extern PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01307 extern PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01308 extern PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01309 extern PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01310 extern PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01311 extern PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01312 extern PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01313 extern PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01314 extern PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01315 extern PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01316 extern PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01317 extern PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01318 extern PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01319 extern PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01320 extern PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01321 extern PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01322 extern PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01323 extern PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01324 extern PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01325 extern PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01326 extern PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01327 extern PFNGLVIEWPORTARRAYVPROC glVertexArrayv;
01328 extern PFNGLVIEWPORTINDEXEDFPROC glVertexIndexedf;
01329 extern PFNGLVIEWPORTINDEXEDFVPROC glVertexIndexedfv;
```

```

01330 extern PFNGLVIEWPORTPOSITIONWSCALENVPROC glViewportPositionWScaleNV;
01331 extern PFNGLVIEWPORTPROC glViewport;
01332 extern PFNGLVIEWPORTSWIZZLENVPROC glViewportSwizzleNV;
01333 extern PFNGLWAITSYNCPROC glWaitSync;
01334 extern PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01335 extern PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01336 extern PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01337 #endif
01338
01340
01344 namespace gg
01345 {
01349     enum BindingPoints
01350     {
01351         LightBindingPoint = 0,
01352         MaterialBindingPoint,
01353     };
01354
01358     extern GLint ggBufferAlignment;
01359
01366     extern void ggInit();
01367
01377     extern void _ggError(const std::string& name = "", unsigned int line = 0);
01378
01389 #if defined(DEBUG)
01390 # define ggError() gg::_ggError(__FILE__, __LINE__)
01391 #else
01392 # define ggError()
01393 #endif
01394
01404     extern void _ggFBOError(const std::string& name = "", unsigned int line = 0);
01405
01416 #if defined(DEBUG)
01417 # define ggFBOError() gg::_ggFBOError(__FILE__, __LINE__)
01418 #else
01419 # define ggFBOError()
01420 #endif
01421
01429     inline void ggCross(GLfloat* c, const GLfloat* a, const GLfloat* b)
01430     {
01431         c[0] = a[1] * b[2] - a[2] * b[1];
01432         c[1] = a[2] * b[0] - a[0] * b[2];
01433         c[2] = a[0] * b[1] - a[1] * b[0];
01434     }
01435
01443     inline GLfloat ggDot3(const GLfloat* a, const GLfloat* b)
01444     {
01445         return a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
01446     }
01447
01454     inline GLfloat ggLength3(const GLfloat* a)
01455     {
01456         return sqrtf(ggDot3(a, a));
01457     }
01458
01466     inline GLfloat ggDistance3(const GLfloat* a, const GLfloat* b)
01467     {
01468         const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], 0.0f };
01469         return ggLength3(c);
01470     }
01471
01478     inline void ggNormalize3(const GLfloat* a, GLfloat* b)
01479     {
01480         const GLfloat l { ggLength3(a) };
01481         assert(l > 0.0f);
01482         b[0] = a[0] / l;
01483         b[1] = a[1] / l;
01484         b[2] = a[2] / l;
01485     }
01486
01492     inline void ggNormalize3(GLfloat* a)
01493     {
01494         const GLfloat l { ggLength3(a) };
01495         assert(l > 0.0f);
01496         a[0] /= l;
01497         a[1] /= l;
01498         a[2] /= l;
01499     }
01500
01508     inline GLfloat ggDot4(const GLfloat* a, const GLfloat* b)
01509     {
01510         return a[0] * b[0] + a[1] * b[1] + a[2] * b[2] + a[3] * b[3];
01511     }
01512
01519     inline GLfloat ggLength4(const GLfloat* a)
01520     {
01521         return sqrtf(ggDot4(a, a));

```

```

01522     }
01523
01531 inline GLfloat ggDistance4(const GLfloat* a, const GLfloat* b)
01532 {
01533     const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], a[3] - b[3] };
01534     return ggLength4(c);
01535 }
01536
01543 inline void ggNormalize4(const GLfloat* a, GLfloat* b)
01544 {
01545     const GLfloat l { ggLength4(a) };
01546     assert(l > 0.0f);
01547     b[0] = a[0] / l;
01548     b[1] = a[1] / l;
01549     b[2] = a[2] / l;
01550     b[3] = a[3] / l;
01551 }
01552
01558 inline void ggNormalize4(GLfloat* a)
01559 {
01560     const GLfloat l { ggLength4(a) };
01561     assert(l > 0.0f);
01562     a[0] /= l;
01563     a[1] /= l;
01564     a[2] /= l;
01565     a[3] /= l;
01566 }
01567
01571 class GgVector : public std::array<GLfloat, 4>
01572 {
01573 public:
01574     GgVector() = default;
01578
01588 constexpr GgVector(GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3) :
01589     std::array<GLfloat, 4>{ v0, v1, v2, v3 }
01590 {
01591 }
01592
01598 constexpr GgVector(const GLfloat* a) :
01599     GgVector{ a[0], a[1], a[2], a[3] }
01600 {
01601 }
01602
01608 constexpr GgVector(GLfloat c) :
01609     GgVector{ c, c, c, c }
01610 {
01611 }
01612
01618 GgVector(const GgVector& v) = default;
01619
01625 GgVector(GgVector&& v) = default;
01626
01630 ~GgVector() = default;
01631
01635 GgVector& operator=(const GgVector& v) = default;
01636
01640 GgVector& operator=(GgVector&& v) = default;
01641
01648 inline GgVector operator+(const GgVector& v) const
01649 {
01650     return GgVector{ (*this)[0] + v[0], (*this)[1] + v[1], (*this)[2] + v[2], (*this)[3] + v[3] };
01651 }
01652
01659 inline GgVector operator+(GLfloat c) const
01660 {
01661     return *this + GgVector(c);
01662 }
01663
01670 inline GgVector& operator+=(const GgVector& v)
01671 {
01672     *this = *this + v;
01673     return *this;
01674 }
01675
01682 inline GgVector& operator+=(GLfloat c)
01683 {
01684     *this = *this + c;
01685     return *this;
01686 }
01687
01694 inline GgVector operator-(const GgVector& v) const
01695 {
01696     return GgVector{ (*this)[0] - v[0], (*this)[1] - v[1], (*this)[2] - v[2], (*this)[3] - v[3] };
01697 }
01698
01705 inline GgVector operator-(GLfloat c) const

```

```

01706     {
01707         return *this - GgVector(c);
01708     }
01709
01716     inline GgVector& operator--=(const GgVector& v)
01717     {
01718         *this = *this - v;
01719         return *this;
01720     }
01721
01728     inline GgVector& operator--=(GLfloat c)
01729     {
01730         *this = *this - c;
01731         return *this;
01732     }
01733
01740     inline GgVector operator*(const GgVector& v) const
01741     {
01742         return GgVector{ (*this)[0] * v[0], (*this)[1] * v[1], (*this)[2] * v[2], (*this)[3] * v[3] };
01743     }
01744
01751     inline GgVector operator*(GLfloat c) const
01752     {
01753         return *this * GgVector(c);
01754     }
01755
01761     inline GgVector& operator*=(const GgVector& v)
01762     {
01763         *this = *this * v;
01764         return *this;
01765     }
01766
01773     inline GgVector& operator*=(GLfloat c)
01774     {
01775         *this = *this * c;
01776         return *this;
01777     }
01778
01785     inline GgVector operator/(const GgVector& v) const
01786     {
01787         return GgVector{ (*this)[0] / v[0], (*this)[1] / v[1], (*this)[2] / v[2], (*this)[3] / v[3] };
01788     }
01789
01796     inline GgVector operator/(GLfloat c) const
01797     {
01798         return *this / GgVector(c);
01799     }
01800
01807     inline GgVector& operator/=(GgVector& v)
01808     {
01809         *this = *this / v;
01810         return *this;
01811     }
01812
01819     inline GgVector& operator/=(GLfloat c)
01820     {
01821         *this = *this / c;
01822         return *this;
01823     }
01824
01831     inline GLfloat dot3(const GgVector& v) const
01832     {
01833         return ggDot3(data(), v.data());
01834     }
01835
01841     inline GLfloat length3() const
01842     {
01843         return ggLength3(data());
01844     }
01845
01852     inline GLfloat distance3(const GgVector& v) const
01853     {
01854         return ggDistance3(data(), v.data());
01855     }
01856
01862     inline GgVector normalize3() const
01863     {
01864         GgVector b;
01865         ggNormalize3(data(), b.data());
01866         return b;
01867     }
01868
01875     inline GLfloat dot4(const GgVector& v) const
01876     {
01877         return ggDot4(data(), v.data());
01878     }
01879

```



```

01885     inline GLfloat length4() const
01886     {
01887         return ggLength4(data());
01888     }
01889
01896     inline GLfloat distance4(const GgVector& v) const
01897     {
01898         return ggDistance4(data(), v.data());
01899     }
01900
01906     inline GgVector normalize4() const
01907     {
01908         GgVector b;
01909         ggNormalize4(data(), b.data());
01910         return b;
01911     }
01912 };
01913
01920     inline const GgVector& operator+(const GgVector& v)
01921     {
01922         return v;
01923     }
01924
01932     inline GgVector operator+(GLfloat a, const GgVector& b)
01933     {
01934         return GgVector{ a + b[0], a + b[1], a + b[2], a + b[3] };
01935     }
01936
01943     inline const GgVector operator-(const GgVector& v)
01944     {
01945         return GgVector{ -v[0], -v[1], -v[2], -v[3] };
01946     }
01947
01955     inline GgVector operator-(GLfloat a, const GgVector& b)
01956     {
01957         return GgVector{ a - b[0], a - b[1], a - b[2], a - b[3] };
01958     }
01959
01967     inline GgVector operator*(GLfloat a, const GgVector& b)
01968     {
01969         return GgVector{ a * b[0], a * b[1], a * b[2], a * b[3] };
01970     }
01971
01979     inline GgVector operator/(GLfloat a, const GgVector& b)
01980     {
01981         return GgVector{ a / b[0], a / b[1], a / b[2], a / b[3] };
01982     }
01983
01994     inline GgVector ggCross(const GgVector& a, const GgVector& b)
01995     {
01996         GgVector c;
01997         ggCross(c.data(), a.data(), b.data());
01998         return c;
01999     }
02000
02008     inline GLfloat ggDot3(const GgVector& a, const GgVector& b)
02009     {
02010         return ggDot3(a.data(), b.data());
02011     }
02012
02019     inline GLfloat ggLength3(const GgVector& a)
02020     {
02021         return ggLength3(a.data());
02022     }
02023
02031     inline GLfloat ggDistance3(const GgVector& a, const GgVector& b)
02032     {
02033         return ggDistance3(a.data(), b.data());
02034     }
02035
02045     inline GgVector ggNormalize3(const GgVector& a)
02046     {
02047         GgVector b;
02048         ggNormalize3(a.data(), b.data());
02049         return b;
02050     }
02051
02060     inline void ggNormalize3(GgVector* a)
02061     {
02062         ggNormalize3(a->data());
02063     }
02064
02072     inline GLfloat ggDot4(const GgVector& a, const GgVector& b)
02073     {
02074         return ggDot4(a.data(), b.data());
02075     }
02076

```

```

02083 inline GLfloat ggLength4(const GgVector& a)
02084 {
02085     return ggLength4(a.data());
02086 }
02087
02095 inline GLfloat ggDistance4(const GgVector& a, const GgVector& b)
02096 {
02097     return ggDistance4(a.data(), b.data());
02098 }
02099
02106 inline GgVector ggNormalize4(const GgVector& a)
02107 {
02108     GgVector b;
02109     ggNormalize4(a.data(), b.data());
02110     return b;
02111 }
02112
02118 inline void ggNormalize4(GgVector* a)
02119 {
02120     ggNormalize4(a->data());
02121 }
02122
02126 class GgMatrix : public std::array<GLfloat, 16>
02127 {
02128     // 行列 a とベクトル b の積をベクトル c に格納する
02129     void projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02130
02131     // 行列 a と行列 b の積を行列 c に格納する
02132     void multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02133
02134 public:
02135
02139     GgMatrix() = default;
02140
02161     constexpr GgMatrix(
02162         GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03,
02163         GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13,
02164         GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23,
02165         GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33
02166     ) :
02167         std::array<GLfloat, 16>{ m00, m01, m02, m03, m10, m11, m12, m13, m20, m21, m22, m23, m30, m31,
m32, m33 }
02168     {
02169     }
02170
02176     constexpr GgMatrix(const GLfloat* a) :
02177         GgMatrix{ a[0], a[1], a[2], a[3], a[4], a[5], a[6], a[7], a[8], a[9], a[10], a[11], a[12],
a[13], a[14], a[15] }
02178     {
02179     }
02180
02186     constexpr GgMatrix(GLfloat c) :
02187         GgMatrix{ c, c, c, c, c, c, c, c, c, c, c, c, c, c, c, c }
02188     {
02189     }
02190
02196     GgMatrix(const GgMatrix& m) = default;
02197
02203     GgMatrix(GgMatrix&& m) = default;
02204
02208     ~GgMatrix() = default;
02209
02213     GgMatrix& operator=(const GgMatrix& m) = default;
02214
02218     GgMatrix& operator=(GgMatrix&& m) = default;
02219
02226     GgMatrix& operator=(const GLfloat* a)
02227     {
02228         *this = GgMatrix(a);
02229         return *this;
02230     }
02231
02238     GgMatrix operator+(const GLfloat* a) const
02239     {
02240         GgMatrix t;
02241         for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] + a[i];
02242         return t;
02243     }
02244
02251     GgMatrix operator+(const GgMatrix& m) const
02252     {
02253         return operator+(m.data());
02254     }
02255
02262     GgMatrix& operator+=(const GLfloat* a)
02263     {
02264         for (std::size_t i = 0; i < size(); ++i) data()[i] += a[i];

```

```

02265     return *this;
02266 }
02267
02274 GgMatrix& operator+=(const GgMatrix& m)
02275 {
02276     return operator+=(m.data());
02277 }
02278
02285 GgMatrix operator-(const GLfloat* a) const
02286 {
02287     GgMatrix t;
02288     for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] - a[i];
02289     return t;
02290 }
02291
02298 GgMatrix operator-(const GgMatrix& m) const
02299 {
02300     return operator-(m.data());
02301 }
02302
02309 GgMatrix& operator-=(const GLfloat* a)
02310 {
02311     for (std::size_t i = 0; i < size(); ++i) data()[i] -= a[i];
02312     return *this;
02313 }
02314
02321 GgMatrix& operator-=(const GgMatrix& m)
02322 {
02323     return operator-=(m.data());
02324 }
02325
02332 GgMatrix operator*(const GLfloat* a) const
02333 {
02334     GgMatrix t;
02335     multiply(t.data(), data(), a);
02336     return t;
02337 }
02338
02345 GgMatrix operator*(const GgMatrix& m) const
02346 {
02347     return operator*(m.data());
02348 }
02349
02356 GgMatrix& operator*=(const GLfloat* a)
02357 {
02358     *this = operator*(a);
02359     return *this;
02360 }
02361
02368 GgMatrix& operator*=(const GgMatrix& m)
02369 {
02370     return operator*=(m.data());
02371 }
02372
02379 GgMatrix operator/(const GLfloat* a) const
02380 {
02381     GgMatrix i, t;
02382     i.loadInvert(a);
02383     multiply(t.data(), data(), i.data());
02384     return t;
02385 }
02386
02393 GgMatrix operator/(const GgMatrix& m) const
02394 {
02395     return operator/(m.data());
02396 }
02397
02404 GgMatrix& operator/=(const GLfloat* a)
02405 {
02406     *this = operator/(a);
02407     return *this;
02408 }
02409
02416 GgMatrix& operator/=(const GgMatrix& m)
02417 {
02418     return operator/=(m.data());
02419 }
02420
02424 GgMatrix& loadIdentity();
02425
02435 GgMatrix& loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02436
02443 GgMatrix& loadTranslate(const GLfloat* t)
02444 {
02445     return loadTranslate(t[0], t[1], t[2]);
02446 }
02447

```

```

02454 GgMatrix& loadTranslate(const GgVector& t)
02455 {
02456     return loadTranslate(t[0], t[1], t[2], t[3]);
02457 }
02458
02468 GgMatrix& loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02469
02476 GgMatrix& loadScale(const GLfloat* s)
02477 {
02478     return loadScale(s[0], s[1], s[2]);
02479 }
02480
02487 GgMatrix& loadScale(const GgVector& s)
02488 {
02489     return loadScale(s[0], s[1], s[2], s[3]);
02490 }
02491
02498 GgMatrix& loadRotateX(GLfloat a);
02499
02506 GgMatrix& loadRotateY(GLfloat a);
02507
02514 GgMatrix& loadRotateZ(GLfloat a);
02515
02525 GgMatrix& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
02526
02534 GgMatrix& loadRotate(const GLfloat* r, GLfloat a)
02535 {
02536     return loadRotate(r[0], r[1], r[2], a);
02537 }
02538
02546 GgMatrix& loadRotate(const GgVector& r, GLfloat a)
02547 {
02548     return loadRotate(r[0], r[1], r[2], a);
02549 }
02550
02557 GgMatrix& loadRotate(const GLfloat* r)
02558 {
02559     return loadRotate(r[0], r[1], r[2], r[3]);
02560 }
02561
02568 GgMatrix& loadRotate(const GgVector& r)
02569 {
02570     return loadRotate(r[0], r[1], r[2], r[3]);
02571 }
02572
02587 GgMatrix& loadLookat(GLfloat ex, GLfloat ey, GLfloat ez,
02588     GLfloat tx, GLfloat ty, GLfloat tz,
02589     GLfloat ux, GLfloat uy, GLfloat uz);
02590
02599 GgMatrix& loadLookat(const GLfloat* e, const GLfloat* t, const GLfloat* u)
02600 {
02601     return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02602 }
02603
02612 GgMatrix& loadLookat(const GgVector& e, const GgVector& t, const GgVector& u)
02613 {
02614     return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02615 }
02616
02628 GgMatrix& loadOrthogonal(GLfloat left, GLfloat right,
02629     GLfloat bottom, GLfloat top,
02630     GLfloat zNear, GLfloat zFar);
02631
02643 GgMatrix& loadFrustum(GLfloat left, GLfloat right,
02644     GLfloat bottom, GLfloat top,
02645     GLfloat zNear, GLfloat zFar);
02646
02656 GgMatrix& loadPerspective(GLfloat fovy, GLfloat aspect,
02657     GLfloat zNear, GLfloat zFar);
02658
02665 GgMatrix& loadTranspose(const GLfloat* a);
02666
02673 GgMatrix& loadTranspose(const GgMatrix& m)
02674 {
02675     return loadTranspose(m.data());
02676 }
02677
02684 GgMatrix& loadInvert(const GLfloat* a);
02685
02692 GgMatrix& loadInvert(const GgMatrix& m)
02693 {
02694     return loadInvert(m.data());
02695 }
02696
02703 GgMatrix& loadNormal(const GLfloat* a);
02704
02711 GgMatrix& loadNormal(const GgMatrix& m)

```

```

02712     {
02713         return loadNormal(m.data());
02714     }
02715
02725 GgMatrix translate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02726 {
02727     GgMatrix m;
02728     return operator*(m.loadTranslate(x, y, z, w));
02729 }
02730
02737 GgMatrix translate(const GLfloat* t) const
02738 {
02739     return translate(t[0], t[1], t[2]);
02740 }
02741
02748 GgMatrix translate(const GgVector& t) const
02749 {
02750     return translate(t[0], t[1], t[2], t[3]);
02751 }
02752
02762 GgMatrix scale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02763 {
02764     GgMatrix m;
02765     return operator*(m.loadScale(x, y, z, w));
02766 }
02767
02774 GgMatrix scale(const GLfloat* s) const
02775 {
02776     return scale(s[0], s[1], s[2]);
02777 }
02778
02785 GgMatrix scale(const GgVector& s) const
02786 {
02787     return scale(s[0], s[1], s[2], s[3]);
02788 }
02789
02796 GgMatrix rotateX(GLfloat a) const
02797 {
02798     GgMatrix m;
02799     return operator*(m.loadRotateX(a));
02800 }
02801
02808 GgMatrix rotateY(GLfloat a) const
02809 {
02810     GgMatrix m;
02811     return operator*(m.loadRotateY(a));
02812 }
02813
02820 GgMatrix rotateZ(GLfloat a) const
02821 {
02822     GgMatrix m;
02823     return operator*(m.loadRotateZ(a));
02824 }
02825
02835 GgMatrix rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
02836 {
02837     GgMatrix m;
02838     return operator*(m.loadRotate(x, y, z, a));
02839 }
02840
02848 GgMatrix rotate(const GLfloat* r, GLfloat a) const
02849 {
02850     return rotate(r[0], r[1], r[2], a);
02851 }
02852
02860 GgMatrix rotate(const GgVector& r, GLfloat a) const
02861 {
02862     return rotate(r[0], r[1], r[2], a);
02863 }
02864
02871 GgMatrix rotate(const GLfloat* r) const
02872 {
02873     return rotate(r[0], r[1], r[2], r[3]);
02874 }
02875
02882 GgMatrix rotate(const GgVector& r) const
02883 {
02884     return rotate(r[0], r[1], r[2], r[3]);
02885 }
02886
02901 GgMatrix lookat(
02902     GLfloat ex, GLfloat ey, GLfloat ez,
02903     GLfloat tx, GLfloat ty, GLfloat tz,
02904     GLfloat ux, GLfloat uy, GLfloat uz
02905 ) const
02906 {
02907     GgMatrix m;

```

```

02908     return operator*(m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz));
02909 }
02910
02919 GgMatrix lookat(const GLfloat* e, const GLfloat* t, const GLfloat* u) const
02920 {
02921     return lookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02922 }
02923
02932 GgMatrix lookat(const GgVector& e, const GgVector& t, const GgVector& u) const
02933 {
02934     return lookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02935 }
02936
02948 GgMatrix orthogonal(
02949     GLfloat left, GLfloat right,
02950     GLfloat bottom, GLfloat top,
02951     GLfloat zNear, GLfloat zFar
02952 ) const
02953 {
02954     GgMatrix m;
02955     return operator*(m.loadOrthogonal(left, right, bottom, top, zNear, zFar));
02956 }
02957
02969 GgMatrix frustum(
02970     GLfloat left, GLfloat right,
02971     GLfloat bottom, GLfloat top,
02972     GLfloat zNear, GLfloat zFar
02973 ) const
02974 {
02975     GgMatrix m;
02976     return operator*(m.loadFrustum(left, right, bottom, top, zNear, zFar));
02977 }
02978
02988 GgMatrix perspective(
02989     GLfloat fovy, GLfloat aspect,
02990     GLfloat zNear, GLfloat zFar
02991 ) const
02992 {
02993     GgMatrix m;
02994     return operator*(m.loadPerspective(fovy, aspect, zNear, zFar));
02995 }
02996
03002 GgMatrix transpose() const
03003 {
03004     GgMatrix t;
03005     return t.loadTranspose(*this);
03006 }
03007
03013 GgMatrix invert() const
03014 {
03015     GgMatrix t;
03016     return t.loadInvert(*this);
03017 }
03018
03024 GgMatrix normal() const
03025 {
03026     GgMatrix t;
03027     return t.loadNormal(*this);
03028 }
03029
03036 void projection(GLfloat* c, const GLfloat* v) const
03037 {
03038     projection(c, data(), v);
03039 }
03040
03047 void projection(GLfloat* c, const GgVector& v) const
03048 {
03049     projection(c, v.data());
03050 }
03051
03058 void projection(GgVector& c, const GLfloat* v) const
03059 {
03060     projection(c.data(), v);
03061 }
03062
03069 void projection(GgVector& c, const GgVector& v) const
03070 {
03071     projection(c.data(), v.data());
03072 }
03073
03080 GgVector operator*(const GgVector& v) const
03081 {
03082     GgVector c;
03083     projection(c, v);
03084     return c;
03085 }
03086

```

```

03092     const GLfloat* get() const
03093     {
03094         return data();
03095     }
03096
03102     void get(GLfloat* a) const
03103     {
03104         std::copy(data(), data() + size(), a);
03105     }
03106 };
03107
03113 inline GgMatrix ggIdentity()
03114 {
03115     GgMatrix t;
03116     return t.loadIdentity();
03117 };
03118
03128 inline GgMatrix ggTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03129 {
03130     GgMatrix m;
03131     return m.loadTranslate(x, y, z, w);
03132 }
03133
03140 inline GgMatrix ggTranslate(const GLfloat* t)
03141 {
03142     GgMatrix m;
03143     return m.loadTranslate(t[0], t[1], t[2]);
03144 }
03145
03152 inline GgMatrix ggTranslate(const GgVector& t)
03153 {
03154     GgMatrix m;
03155     return m.loadTranslate(t[0], t[1], t[2], t[3]);
03156 }
03157
03167 inline GgMatrix ggScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03168 {
03169     GgMatrix m;
03170     return m.loadScale(x, y, z, w);
03171 }
03172
03179 inline GgMatrix ggScale(const GLfloat* s)
03180 {
03181     GgMatrix m;
03182     return m.loadScale(s[0], s[1], s[2]);
03183 }
03184
03191 inline GgMatrix ggScale(const GgVector& s)
03192 {
03193     GgMatrix m;
03194     return m.loadScale(s[0], s[1], s[2], s[3]);
03195 }
03196
03203 inline GgMatrix ggRotateX(GLfloat a)
03204 {
03205     GgMatrix m;
03206     return m.loadRotateX(a);
03207 }
03208
03215 inline GgMatrix ggRotateY(GLfloat a)
03216 {
03217     GgMatrix m;
03218     return m.loadRotateY(a);
03219 }
03220
03227 inline GgMatrix ggRotateZ(GLfloat a)
03228 {
03229     GgMatrix m;
03230     return m.loadRotateZ(a);
03231 }
03232
03242 inline GgMatrix ggRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03243 {
03244     GgMatrix m;
03245     return m.loadRotate(x, y, z, a);
03246 }
03247
03255 inline GgMatrix ggRotate(const GLfloat* r, GLfloat a)
03256 {
03257     GgMatrix m;
03258     return m.loadRotate(r[0], r[1], r[2], a);
03259 }
03260
03268 inline GgMatrix ggRotate(const GgVector& r, GLfloat a)
03269 {
03270     GgMatrix m;
03271     return m.loadRotate(r[0], r[1], r[2], a);

```

```

03272     }
03273
03280 inline GgMatrix ggRotate(const GLfloat* r)
03281 {
03282     GgMatrix m;
03283     return m.loadRotate(r[0], r[1], r[2], r[3]);
03284 }
03285
03292 inline GgMatrix ggRotate(const GgVector& r)
03293 {
03294     GgMatrix m;
03295     return m.loadRotate(r[0], r[1], r[2], r[3]);
03296 }
03297
03312 inline GgMatrix ggLookat(
03313     GLfloat ex, GLfloat ey, GLfloat ez,    // 視点の位置
03314     GLfloat tx, GLfloat ty, GLfloat tz,    // 目標点の位置
03315     GLfloat ux, GLfloat uy, GLfloat uz     // 上方向のベクトル
03316 )
03317 {
03318     GgMatrix m;
03319     return m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz);
03320 }
03321
03330 inline GgMatrix ggLookat(
03331     const GLfloat* e,                      // 視点の位置
03332     const GLfloat* t,                      // 目標点の位置
03333     const GLfloat* u                      // 上方向のベクトル
03334 )
03335 {
03336     GgMatrix m;
03337     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03338 }
03339
03348 inline GgMatrix ggLookat(
03349     const GgVector& e,                      // 視点の位置
03350     const GgVector& t,                      // 目標点の位置
03351     const GgVector& u                      // 上方向のベクトル
03352 )
03353 {
03354     GgMatrix m;
03355     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03356 }
03357
03369 inline GgMatrix ggOrthogonal(GLfloat left, GLfloat right,
03370     GLfloat bottom, GLfloat top,
03371     GLfloat zNear, GLfloat zFar)
03372 {
03373     GgMatrix m;
03374     return m.loadOrthogonal(left, right, bottom, top, zNear, zFar);
03375 }
03376
03388 inline GgMatrix ggFrustum(
03389     GLfloat left, GLfloat right,
03390     GLfloat bottom, GLfloat top,
03391     GLfloat zNear, GLfloat zFar
03392 )
03393 {
03394     GgMatrix m;
03395     return m.loadFrustum(left, right, bottom, top, zNear, zFar);
03396 }
03397
03407 inline GgMatrix ggPerspective(
03408     GLfloat fovy, GLfloat aspect,
03409     GLfloat zNear, GLfloat zFar
03410 )
03411 {
03412     GgMatrix m;
03413     return m.loadPerspective(fovy, aspect, zNear, zFar);
03414 }
03415
03422 inline GgMatrix ggTranspose(const GgMatrix& m)
03423 {
03424     return m.transpose();
03425 }
03426
03433 inline GgMatrix ggInvert(const GgMatrix& m)
03434 {
03435     return m.invert();
03436 }
03437
03444 inline GgMatrix ggNormal(const GgMatrix& m)
03445 {
03446     return m.normal();
03447 }
03448
03452 class GgQuaternion : public GgVector

```



```

03453 {
03454     // GgQuaternion 型の四元数 p と四元数 q の積を四元数 r に求める
03455     void multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const;
03456
03457     // GgQuaternion 型の四元数 q が表す回転の変換行列を m に求める
03458     void toMatrix(GLfloat* m, const GLfloat* q) const;
03459
03460     // 回転の変換行列 m が表す四元数を q に求める
03461     void toQuaternion(GLfloat* q, const GLfloat* m) const;
03462
03463     // 球面線形補間 q と r を t で補間した四元数を p に求める
03464     void slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const;
03465
03466 public:
03467
03471     GgQuaternion() = default;
03472
03481     constexpr GgQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat w) :
03482         GgVector{ x, y, z, w }
03483     {
03484     }
03485
03491     constexpr GgQuaternion(GLfloat c) :
03492         GgQuaternion{ c, c, c, c }
03493     {
03494     }
03495
03501     GgQuaternion(const GLfloat* a) :
03502         GgQuaternion{ a[0], a[1], a[2], a[3] }
03503     {
03504     }
03505
03511     GgQuaternion(const GgVector& v) :
03512         GgQuaternion{ v[0], v[1], v[2], v[3] }
03513     {
03514     }
03515
03521     GgQuaternion(const GgQuaternion& q) = default;
03522
03528     GgQuaternion(GgQuaternion&& q) = default;
03529
03533     ~GgQuaternion() = default;
03534
03541     GgQuaternion& operator=(const GgQuaternion& q) = default;
03542
03549     GgQuaternion& operator=(GgQuaternion&& q) = default;
03550
03556     GLfloat norm() const
03557     {
03558         return gglength4(*this);
03559     }
03560
03570     GgQuaternion& loadAdd(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03571     {
03572         (*this)[0] += x;
03573         (*this)[1] += y;
03574         (*this)[2] += z;
03575         (*this)[3] += w;
03576
03577         return *this;
03578     }
03579
03586     GgQuaternion& loadAdd(const GLfloat* a)
03587     {
03588         return loadAdd(a[0], a[1], a[2], a[3]);
03589     }
03590
03597     GgQuaternion& loadAdd(const GgVector& v)
03598     {
03599         return loadAdd(v[0], v[1], v[2], v[3]);
03600     }
03601
03608     GgQuaternion& loadAdd(const GgQuaternion& q)
03609     {
03610         return loadAdd(q[0], q[1], q[2], q[3]);
03611     }
03612
03622     GgQuaternion& loadSubtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03623     {
03624         (*this)[0] -= x;
03625         (*this)[1] -= y;
03626         (*this)[2] -= z;
03627         (*this)[3] -= w;
03628
03629         return *this;
03630     }
03631

```

```

03638 GgQuaternion& loadSubtract(const GLfloat* a)
03639 {
03640     return loadSubtract(a[0], a[1], a[2], a[3]);
03641 }
03642
03649 GgQuaternion& loadSubtract(const GgVector& v)
03650 {
03651     return loadSubtract(v[0], v[1], v[2], v[3]);
03652 }
03653
03660 GgQuaternion& loadSubtract(const GgQuaternion& q)
03661 {
03662     return loadSubtract(q[0], q[1], q[2], q[3]);
03663 }
03664
03674 GgQuaternion& loadMultiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03675 {
03676     const GLfloat a[]={ x, y, z, w };
03677     return loadMultiply(a);
03678 }
03679
03686 GgQuaternion& loadMultiply(const GLfloat* a)
03687 {
03688     return *this = multiply(a);
03689 }
03690
03697 GgQuaternion& loadMultiply(const GgVector& v)
03698 {
03699     return loadMultiply(v.data());
03700 }
03701
03708 GgQuaternion& loadMultiply(const GgQuaternion& q)
03709 {
03710     return loadMultiply(q.data());
03711 }
03712
03722 GgQuaternion& loadDivide(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03723 {
03724     const GLfloat a[]={ x, y, z, w };
03725     return loadDivide(a);
03726 }
03727
03734 GgQuaternion& loadDivide(const GLfloat* a)
03735 {
03736     return *this = divide(a);
03737 }
03738
03745 GgQuaternion& loadDivide(const GgVector& v)
03746 {
03747     return loadDivide(v.data());
03748 }
03749
03756 GgQuaternion& loadDivide(const GgQuaternion& q)
03757 {
03758     return loadDivide(q.data());
03759 }
03760
03770 GgQuaternion add(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03771 {
03772     GgQuaternion s{ *this };
03773     return s.loadAdd(x, y, z, w);
03774 }
03775
03782 GgQuaternion add(const GLfloat* a) const
03783 {
03784     return add(a[0], a[1], a[2], a[3]);
03785 }
03786
03793 GgQuaternion add(const GgVector& v) const
03794 {
03795     return add(v[0], v[1], v[2], v[3]);
03796 }
03797
03804 GgQuaternion add(const GgQuaternion& q) const
03805 {
03806     return add(q[0], q[1], q[2], q[3]);
03807 }
03808
03818 GgQuaternion subtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03819 {
03820     GgQuaternion s{ *this };
03821     return s.loadSubtract(x, y, z, w);
03822 }
03823
03830 GgQuaternion subtract(const GLfloat* a) const
03831 {
03832     return subtract(a[0], a[1], a[2], a[3]);

```

```

03833     }
03834
03841     GgQuaternion subtract(const GgVector& v) const
03842     {
03843         return subtract(v[0], v[1], v[2], v[3]);
03844     }
03845
03852     GgQuaternion subtract(const GgQuaternion& q) const
03853     {
03854         return subtract(q[0], q[1], q[2], q[3]);
03855     }
03856
03866     GgQuaternion multiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03867     {
03868         const GLfloat a[]={ x, y, z, w };
03869         return multiply(a);
03870     }
03871
03878     GgQuaternion multiply(const GLfloat* a) const
03879     {
03880         GgQuaternion s;
03881         multiply(s.data(), data(), a);
03882         return s;
03883     }
03884
03891     GgQuaternion multiply(const GgVector& v) const
03892     {
03893         return multiply(v.data());
03894     }
03895
03902     GgQuaternion multiply(const GgQuaternion& q) const
03903     {
03904         return multiply(q.data());
03905     }
03906
03916     GgQuaternion divide(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03917     {
03918         const GLfloat a[]={ x, y, z, w };
03919         return divide(a);
03920     }
03921
03928     GgQuaternion divide(const GLfloat* a) const
03929     {
03930         GgQuaternion s, ia;
03931         ia.loadInvert(a);
03932         multiply(s.data(), data(), ia.data());
03933         return s;
03934     }
03935
03942     GgQuaternion divide(const GgVector& v) const
03943     {
03944         return divide(v.data());
03945     }
03946
03953     GgQuaternion divide(const GgQuaternion& q) const
03954     {
03955         return divide(q.data());
03956     }
03957
03958     // 演算子
03959     GgQuaternion& operator=(const GLfloat* a)
03960     {
03961         return *this = GgQuaternion(a);
03962     }
03963     GgQuaternion& operator=(const GgVector& v)
03964     {
03965         return *this = GgQuaternion(v);
03966     }
03967     GgQuaternion& operator+=(const GLfloat* a)
03968     {
03969         return loadAdd(a);
03970     }
03971     GgQuaternion& operator+=(const GgVector& v)
03972     {
03973         return loadAdd(v);
03974     }
03975     GgQuaternion& operator+=(const GgQuaternion& q)
03976     {
03977         return loadAdd(q);
03978     }
03979     GgQuaternion& operator-=(const GLfloat* a)
03980     {
03981         return loadSubtract(a);
03982     }
03983     GgQuaternion& operator-=(const GgVector& v)
03984     {
03985         return loadSubtract(v);

```

```

03986     }
03987     GgQuaternion& operator--(const GgQuaternion& q)
03988     {
03989         return operator--(q.data());
03990     }
03991     GgQuaternion& operator*=(const GLfloat* a)
03992     {
03993         return loadMultiply(a);
03994     }
03995     GgQuaternion& operator*=(const GgVector& v)
03996     {
03997         return loadMultiply(v);
03998     }
03999     GgQuaternion& operator*=(const GgQuaternion& q)
04000     {
04001         return operator*=(q.data());
04002     }
04003     GgQuaternion& operator/=(const GLfloat* a)
04004     {
04005         return loadDivide(a);
04006     }
04007     GgQuaternion& operator/=(const GgVector& v)
04008     {
04009         return loadDivide(v);
04010     }
04011     GgQuaternion& operator/=(const GgQuaternion& q)
04012     {
04013         return operator/=(q.data());
04014     }
04015     GgQuaternion operator+(const GLfloat* a) const
04016     {
04017         return add(a);
04018     }
04019     GgQuaternion operator+(const GgVector& v) const
04020     {
04021         return add(v);
04022     }
04023     GgQuaternion operator+(const GgQuaternion& q) const
04024     {
04025         return operator+(q.data());
04026     }
04027     GgQuaternion operator-(const GLfloat* a) const
04028     {
04029         return add(a);
04030     }
04031     GgQuaternion operator-(const GgVector& v) const
04032     {
04033         return add(v);
04034     }
04035     GgQuaternion operator-(const GgQuaternion& q) const
04036     {
04037         return operator-(q.data());
04038     }
04039     GgQuaternion operator*(const GLfloat* a) const
04040     {
04041         return multiply(a);
04042     }
04043     GgQuaternion operator*(const GgVector& v) const
04044     {
04045         return multiply(v);
04046     }
04047     GgQuaternion operator*(const GgQuaternion& q) const
04048     {
04049         return operator*(q.data());
04050     }
04051     GgQuaternion operator/(const GLfloat* a) const
04052     {
04053         return divide(a);
04054     }
04055     GgQuaternion operator/(const GgVector& v) const
04056     {
04057         return divide(v);
04058     }
04059     GgQuaternion operator/(const GgQuaternion& q) const
04060     {
04061         return operator/(q.data());
04062     }
04063
04070     GgQuaternion& loadMatrix(const GLfloat* a)
04071     {
04072         toQuaternion(data(), a);
04073         return *this;
04074     }
04075
04082     GgQuaternion& loadMatrix(const GgMatrix& m)
04083     {
04084         return loadMatrix(m.get());

```

```

04085     }
04086
04092     GgQuaternion& loadIdentity()
04093     {
04094         return *this = GgQuaternion(0.0f, 0.0f, 0.0f, 1.0f);
04095     }
04096
04106     GgQuaternion& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
04107
04115     GgQuaternion& loadRotate(const GLfloat* v, GLfloat a)
04116     {
04117         return loadRotate(v[0], v[1], v[2], a);
04118     }
04119
04126     GgQuaternion& loadRotate(const GLfloat* v)
04127     {
04128         return loadRotate(v[0], v[1], v[2], v[3]);
04129     }
04130
04137     GgQuaternion& loadRotateX(GLfloat a);
04138
04145     GgQuaternion& loadRotateY(GLfloat a);
04146
04153     GgQuaternion& loadRotateZ(GLfloat a);
04154
04164     GgQuaternion rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
04165     {
04166         GgQuaternion q;
04167         return multiply(q.loadRotate(x, y, z, a));
04168     }
04169
04177     GgQuaternion rotate(const GLfloat* v, GLfloat a) const
04178     {
04179         return rotate(v[0], v[1], v[2], a);
04180     }
04181
04188     GgQuaternion rotate(const GLfloat* v) const
04189     {
04190         return rotate(v[0], v[1], v[2], v[3]);
04191     }
04192
04199     GgQuaternion rotateX(GLfloat a) const
04200     {
04201         return rotate(1.0f, 0.0f, 0.0f, a);
04202     }
04203
04210     GgQuaternion rotateY(GLfloat a) const
04211     {
04212         return rotate(0.0f, 1.0f, 0.0f, a);
04213     }
04214
04221     GgQuaternion rotateZ(GLfloat a) const
04222     {
04223         return rotate(0.0f, 0.0f, 1.0f, a);
04224     }
04225
04234     GgQuaternion& loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll);
04235
04242     GgQuaternion& loadEuler(const GLfloat* e)
04243     {
04244         return loadEuler(e[0], e[1], e[2]);
04245     }
04246
04255     GgQuaternion euler(GLfloat heading, GLfloat pitch, GLfloat roll) const
04256     {
04257         GgQuaternion r;
04258         return multiply(r.loadEuler(heading, pitch, roll));
04259     }
04260
04267     GgQuaternion euler(const GLfloat* e) const
04268     {
04269         return euler(e[0], e[1], e[2]);
04270     }
04271
04281     GgQuaternion euler(const GgVector& e) const
04282     {
04283         return euler(e[0], e[1], e[2]);
04284     }
04285
04294     GgQuaternion& loadSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04295     {
04296         slerp(data(), a, b, t);
04297         return *this;
04298     }
04299
04308     GgQuaternion& loadSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04309     {

```

```

04310     return loadSlerp(q.data(), r.data(), t);
04311 }
04312
04321 GgQuaternion& loadSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04322 {
04323     return loadSlerp(q.data(), a, t);
04324 }
04325
04334 GgQuaternion& loadSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04335 {
04336     return loadSlerp(a, q.data(), t);
04337 }
04338
04345 GgQuaternion& loadNormalize(const GLfloat* a);
04346
04353 GgQuaternion& loadNormalize(const GgQuaternion& q)
04354 {
04355     return loadNormalize(q.data());
04356 }
04357
04364 GgQuaternion& loadConjugate(const GLfloat* a);
04365
04372 GgQuaternion& loadConjugate(const GgQuaternion& q)
04373 {
04374     return loadConjugate(q.data());
04375 }
04376
04383 GgQuaternion& loadInvert(const GLfloat* a);
04384
04391 GgQuaternion& loadInvert(const GgQuaternion& q)
04392 {
04393     return loadInvert(q.data());
04394 }
04395
04403 GgQuaternion slerp(GLfloat* a, GLfloat t) const
04404 {
04405     GgQuaternion p;
04406     slerp(p.data(), data(), a, t);
04407     return p;
04408 }
04409
04417 GgQuaternion slerp(const GgQuaternion& q, GLfloat t) const
04418 {
04419     GgQuaternion p;
04420     slerp(p.data(), data(), q.data(), t);
04421     return p;
04422 }
04423
04429 GgQuaternion normalize() const
04430 {
04431     GgQuaternion q;
04432     q.loadNormalize(data());
04433     return q;
04434 }
04435
04441 GgQuaternion conjugate() const
04442 {
04443     GgQuaternion q;
04444     q.loadConjugate(data());
04445     return q;
04446 }
04447
04453 GgQuaternion invert() const
04454 {
04455     GgQuaternion q;
04456     q.loadInvert(data());
04457     return q;
04458 }
04459
04465 void get(GLfloat* a) const
04466 {
04467     a[0] = data()[0];
04468     a[1] = data()[1];
04469     a[2] = data()[2];
04470     a[3] = data()[3];
04471 }
04472
04478 void getMatrix(GLfloat* a) const
04479 {
04480     toMatrix(a, data());
04481 }
04482
04488 void getMatrix(GgMatrix& m) const
04489 {
04490     getMatrix(m.data());
04491 }
04492

```

```

04498     GgMatrix getMatrix() const
04499     {
04500         GgMatrix m;
04501         getMatrix(m);
04502         return m;
04503     }
04504
04510     void getConjugateMatrix(GLfloat* a) const
04511     {
04512         GgQuaternion c;
04513         c.loadConjugate(data());
04514         toMatrix(a, c.data());
04515     }
04516
04522     void getConjugateMatrix(GgMatrix& m) const
04523     {
04524         getConjugateMatrix(m.data());
04525     }
04526
04532     GgMatrix getConjugateMatrix() const
04533     {
04534         GgMatrix m;
04535         getConjugateMatrix(m);
04536         return m;
04537     }
04538 };
04539
04545     inline GgQuaternion ggIdentityQuaternion()
04546     {
04547         GgQuaternion q;
04548         return q.loadIdentity();
04549     }
04550
04557     inline GgQuaternion ggMatrixQuaternion(const GLfloat* a)
04558     {
04559         GgQuaternion q;
04560         return q.loadMatrix(a);
04561     }
04562
04569     inline GgQuaternion ggMatrixQuaternion(const GgMatrix& m)
04570     {
04571         return ggMatrixQuaternion(m.get());
04572     }
04573
04580     inline GgMatrix ggQuaternionMatrix(const GgQuaternion& q)
04581     {
04582         GLfloat m[16];
04583         q.getMatrix(m);
04584         return GgMatrix{ m };
04585     }
04586
04593     inline GgMatrix ggQuaternionTransposeMatrix(const GgQuaternion& q)
04594     {
04595         GLfloat m[16];
04596         q.getMatrix(m);
04597         GgMatrix t;
04598         return t.loadTranspose(m);
04599     }
04600
04610     inline GgQuaternion ggRotateQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
04611     {
04612         GgQuaternion q;
04613         return q.loadRotate(x, y, z, a);
04614     }
04615
04623     inline GgQuaternion ggRotateQuaternion(const GLfloat* v, GLfloat a)
04624     {
04625         return ggRotateQuaternion(v[0], v[1], v[2], a);
04626     }
04627
04634     inline GgQuaternion ggRotateQuaternion(const GLfloat* v)
04635     {
04636         return ggRotateQuaternion(v[0], v[1], v[2], v[3]);
04637     }
04638
04647     inline GgQuaternion ggEulerQuaternion(GLfloat heading, GLfloat pitch, GLfloat roll)
04648     {
04649         GgQuaternion q;
04650         return q.loadEuler(heading, pitch, roll);
04651     }
04652
04659     inline GgQuaternion ggEulerQuaternion(const GLfloat* e)
04660     {
04661         return ggEulerQuaternion(e[0], e[1], e[2]);
04662     }
04663
04670     inline GgQuaternion ggEulerQuaternion(const GgVector& e)

```

```

04671 {
04672     return ggEulerQuaternion(e[0], e[1], e[2]);
04673 }
04674
04683 inline GgQuaternion ggSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04684 {
04685     GgQuaternion r;
04686     return r.loadSlerp(a, b, t);
04687 }
04688
04697 inline GgQuaternion ggSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04698 {
04699     return ggSlerp(q.data(), r.data(), t);
04700 }
04701
04710 inline GgQuaternion ggSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04711 {
04712     return ggSlerp(q.data(), a, t);
04713 }
04714
04723 inline GgQuaternion ggSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04724 {
04725     return ggSlerp(a, q.data(), t);
04726 }
04727
04734 inline GLfloat ggNorm(const GgQuaternion& q)
04735 {
04736     return q.norm();
04737 }
04738
04745 inline GgQuaternion ggNormalize(const GgQuaternion& q)
04746 {
04747     return q.normalize();
04748 }
04749
04756 inline GgQuaternion ggConjugate(const GgQuaternion& q)
04757 {
04758     return q.conjugate();
04759 }
04760
04767 inline GgQuaternion ggInvert(const GgQuaternion& q)
04768 {
04769     return q.invert();
04770 }
04771
04775 class GgTrackball : public GgQuaternion
04776 {
04777     bool drag;          // ドラッグ中か否か
04778     GLfloat start[2];    // ドラッグ開始位置
04779     GLfloat scale[2];    // マウスの絶対位置→ウィンドウ内での相対位置の換算係数
04780     GgQuaternion cq;     // 回転の初期値 (四元数)
04781     GgMatrix rt;         // 回転の変換行列
04782
04783 public:
04784
04790     GgTrackball(const GgQuaternion& q = ggIdentityQuaternion())
04791     {
04792         reset(q);
04793     }
04794
04798     ~GgTrackball() = default;
04799
04805     GgTrackball& operator=(const GgQuaternion& q)
04806     {
04807         GgQuaternion::operator=(q);
04808         return *this;
04809     }
04810
04820     void region(GLfloat w, GLfloat h);
04821
04831     void region(int w, int h)
04832     {
04833         region(static_cast<GLfloat>(w), static_cast<GLfloat>(h));
04834     }
04835
04845     void begin(GLfloat x, GLfloat y);
04846
04856     void motion(GLfloat x, GLfloat y);
04857
04863     void rotate(const GgQuaternion& q);
04864
04874     void end(GLfloat x, GLfloat y);
04875
04881     void reset(const GgQuaternion& q = ggIdentityQuaternion());
04882
04888     const GLfloat* getStart() const
04889     {

```



```

04890     return start;
04891 }
04892
04896 const GLfloat& getStart(int direction) const
04897 {
04898     return start[direction];
04899 }
04900
04906 void getStart(GLfloat* position) const
04907 {
04908     position[0] = start[0];
04909     position[1] = start[1];
04910 }
04911
04917 const GLfloat* getScale() const
04918 {
04919     return scale;
04920 }
04921
04927 const GLfloat getScale(int direction) const
04928 {
04929     return scale[direction];
04930 }
04931
04937 void getScale(GLfloat* factor) const
04938 {
04939     factor[0] = scale[0];
04940     factor[1] = scale[1];
04941 }
04942
04948 const GgQuaternion& getQuaternion() const
04949 {
04950     return *this;
04951 }
04952
04958 const GgMatrix& getMatrix() const
04959 {
04960     return rt;
04961 }
04962
04968 const GLfloat* get() const
04969 {
04970     return rt.get();
04971 }
04972 };
04973
04984 extern bool ggSaveTga(
04985     const std::string& name,
04986     const void* buffer,
04987     unsigned int width,
04988     unsigned int height,
04989     unsigned int depth
04990 );
04991
04998 extern bool ggSaveColor(const std::string& name);
04999
05006 extern bool ggSaveDepth(const std::string& name);
05007
05018 extern bool ggReadImage(
05019     const std::string& name,
05020     std::vector<GLubyte>& image,
05021     GLsizei* pWidth,
05022     GLsizei* pHeight,
05023     GLenum* pFormat
05024 );
05025
05039 extern GLuint ggLoadTexture(
05040     const GLvoid* image,
05041     GLsizei width,
05042     GLsizei height,
05043     GLenum format = GL_RGB,
05044     GLenum type = GL_UNSIGNED_BYTE,
05045     GLenum internal = GL_RGB,
05046     GLenum wrap = GL_CLAMP_TO_EDGE,
05047     bool swizzle = false
05048 );
05049
05060 extern GLuint ggLoadImage(
05061     const std::string& name,
05062     GLsizei* pWidth = nullptr,
05063     GLsizei* pHeight = nullptr,
05064     GLenum internal = GL_RGBA,
05065     GLenum wrap = GL_CLAMP_TO_EDGE
05066 );
05067
05079 extern void ggCreateNormalMap(
05080     const GLubyte* hmap,

```

```

05081     GLsizei width,
05082     GLsizei height,
05083     GLenum format,
05084     GLfloat nz,
05085     GLenum internal,
05086     std::vector<GgVector>& nmap
05087 );
05088
05099 extern GLuint ggLoadHeight(
05100     const std::string& name,
05101     GLfloat nz,
05102     GLsizei* pWidth = nullptr,
05103     GLsizei* pHeight = nullptr,
05104     GLenum internal = GL_RGBA
05105 );
05106
05120 extern GLuint ggCreateShader(
05121     const std::string& vsrc,
05122     const std::string& fsrc = "",
05123     const std::string& gsrc = "",
05124     GLint nvarying = 0,
05125     const char* const* varyings = nullptr,
05126     const std::string& vtext = "vertex shader",
05127     const std::string& ftext = "fragment shader",
05128     const std::string& gtext = "geometry shader");
05129
05140 extern GLuint ggLoadShader(
05141     const std::string& vert,
05142     const std::string& frag = "",
05143     const std::string& geom = "",
05144     GLint nvarying = 0,
05145     const char* const* varyings = nullptr
05146 );
05147
05156 inline GLuint ggLoadShader(
05157     const std::array<std::string, 3>& files,
05158     GLint nvarying = 0,
05159     const char* const* varyings = nullptr
05160 )
05161 {
05162     return ggLoadShader(files[0], files[1], files[2], nvarying, varyings);
05163 }
05164
05165 #if !defined(__APPLE__)
05173 extern GLuint ggCreateComputeShader(
05174     const std::string& csrc,
05175     const std::string& ctext = "compute shader"
05176 );
05177
05184 extern GLuint ggLoadComputeShader(const std::string& comp);
05185 #endif
05186
05193 class GgTexture
05194 {
05195     // テクスチャ名
05196     GLuint texture;
05197
05198     // テクスチャの縦横の画素数
05199     GLsizei size[2];
05200
05201 public:
05202
05215     GgTexture(
05216         const GLvoid* image,
05217         GLsizei width,
05218         GLsizei height,
05219         GLenum format = GL_RGB,
05220         GLenum type = GL_UNSIGNED_BYTE,
05221         GLenum internal = GL_RGBA,
05222         GLenum wrap = GL_CLAMP_TO_EDGE,
05223         bool swizzle = false
05224     ) :
05225         texture{ ggLoadTexture(image, width, height, format, type, internal, wrap, swizzle) },
05226         size{ width, height }
05227     {
05228     }
05229
05235     GgTexture(const GgTexture& texture) = delete;
05236
05242     GgTexture(GgTexture&& texture) = default;
05243
05247     virtual ~GgTexture()
05248     {
05249         glBindTexture(GL_TEXTURE_2D, 0);
05250         glDeleteTextures(1, &texture);
05251     }
05252

```

```

05259     GgTexture& operator=(const GgTexture& texture) = delete;
05260
05267     GgTexture& operator=(GgTexture&& texture) = default;
05268
05272     void bind() const
05273     {
05274         glBindTexture(GL_TEXTURE_2D, texture);
05275     }
05276
05280     void unbind() const
05281     {
05282         glBindTexture(GL_TEXTURE_2D, 0);
05283     }
05284
05290     void swapRandB(bool swizzle) const;
05291
05297     const GLsizei& getWidth() const
05298     {
05299         return size[0];
05300     }
05301
05307     const GLsizei& getHeight() const
05308     {
05309         return size[1];
05310     }
05311
05317     void getSize(GLsizei* size) const
05318     {
05319         size[0] = getWidth();
05320         size[1] = getHeight();
05321     }
05322
05328     const GLsizei* getSize() const
05329     {
05330         return size;
05331     }
05332
05338     const GLuint& getTexture() const
05339     {
05340         return texture;
05341     }
05342 };
05343
05350 class GgColorTexture
05351 {
05352     // テクスチャ
05353     std::shared_ptr<GgTexture> texture;
05354
05355 public:
05356
05360     GgColorTexture()
05361     {
05362     }
05363
05376     GgColorTexture(
05377         const GLvoid* image,
05378         GLsizei width,
05379         GLsizei height,
05380         GLenum format = GL_RGB,
05381         GLenum type = GL_UNSIGNED_BYTE,
05382         GLenum internal = GL_RGB,
05383         GLenum wrap = GL_CLAMP_TO_EDGE,
05384         bool swizzle = true
05385     )
05386     {
05387         load(image, width, height, format, type, internal, wrap, swizzle);
05388     }
05389
05397     GgColorTexture(
05398         const std::string& name,
05399         GLenum internal = 0,
05400         GLenum wrap = GL_CLAMP_TO_EDGE
05401     )
05402     {
05403         load(name, internal, wrap);
05404     }
05405
05409     virtual ~GgColorTexture()
05410     {
05411     }
05412
05425     void load(
05426         const GLvoid* image,
05427         GLsizei width,
05428         GLsizei height,
05429         GLenum format = GL_RGB,
05430         GLenum type = GL_UNSIGNED_BYTE,

```

```

05431         GLenum internal = GL_RGB,
05432         GLenum wrap = GL_CLAMP_TO_EDGE,
05433         bool swizzle = true
05434     )
05435     {
05436         // テクスチャを作成する
05437         texture = std::make_shared<GgTexture>(image, width, height, format, type, internal, wrap,
swizzle);
05438     }
05439
05440     void load(
05441         const std::string& name,
05442         GLenum internal = 0,
05443         GLenum wrap = GL_CLAMP_TO_EDGE
05444     );
05445 };
05446
05447 class GgNormalTexture
05448 {
05449     // テクスチャ
05450     std::shared_ptr<GgTexture> texture;
05451
05452 public:
05453
05454     GgNormalTexture()
05455     {
05456     }
05457
05458     GgNormalTexture(
05459         const GLubyte* image,
05460         GLsizei width,
05461         GLsizei height,
05462         GLenum format = GL_RED,
05463         GLfloat nz = 1.0f,
05464         GLenum internal = GL_RGBA
05465     )
05466     {
05467         // 法線マップのテクスチャを作成する
05468         load(image, width, height, format, nz, internal);
05469     }
05470
05471     GgNormalTexture(
05472         const std::string& name,
05473         GLfloat nz = 1.0f,
05474         GLenum internal = GL_RGBA
05475     )
05476     {
05477         // 法線マップのテクスチャを作成する
05478         load(name, nz, internal);
05479     }
05480
05481     virtual ~GgNormalTexture()
05482     {
05483     }
05484
05485     void load(
05486         const GLubyte* hmap,
05487         GLsizei width,
05488         GLsizei height,
05489         GLenum format = GL_RED,
05490         GLfloat nz = 1.0f,
05491         GLenum internal = GL_RGBA
05492     )
05493     {
05494         // 法線マップ
05495         std::vector<GgVector> nmap;
05496
05497         // 法線マップを作成する
05498         ggCreateNormalMap(hmap, width, height, format, nz, internal, nmap);
05499
05500         // テクスチャを作成する
05501         texture = std::make_shared<GgTexture>(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal,
GL_REPEAT);
05502     }
05503
05504     void load(
05505         const std::string& name,
05506         GLfloat nz = 1.0f,
05507         GLenum internal = GL_RGBA
05508     );
05509 };
05510
05511 template <typename T>
05512 class GgBuffer
05513 {
05514     // ターゲット
05515     const GLenum target;

```

```

05575
05576 // バッファオブジェクトのアライメントを考慮したデータの間隔
05577 const GLsizei stride;
05578
05579 // データの数
05580 const GLsizei count;
05581
05582 // バッファオブジェクト
05583 const GLuint buffer;
05584
05585 public:
05586
05596 GgBuffer<T>(
05597     GLenum target,
05598     const T* data,
05599     GLsizei stride,
05600     GLsizei count,
05601     GLenum usage
05602 ) :
05603     target{ target },
05604     stride{ stride },
05605     count{ count },
05606     buffer{ [] { GLuint buffer; glGenBuffers(1, &buffer); return buffer; } () }
05607 {
05608     // バッファオブジェクトのメモリを確保してデータを転送する
05609     glBindBuffer(target, buffer);
05610     glBufferData(target, getStride() * count, data, usage);
05611 }
05612
05618 GgBuffer<T>(const GgBuffer<T>& buffer) = delete;
05619
05625 GgBuffer<T>(GgBuffer<T>&& buffer) = default;
05626
05630 virtual ~GgBuffer<T>()
05631 {
05632     // バッファオブジェクトを削除する
05633     glBindBuffer(target, 0);
05634     glDeleteBuffers(1, &buffer);
05635 }
05636
05643 GgBuffer<T>& operator=(const GgBuffer<T>& buffer) = delete;
05644
05651 GgBuffer<T>& operator=(GgBuffer<T>&& buffer) = default;
05652
05658 const GLuint& getTarget() const
05659 {
05660     return target;
05661 }
05662
05668 GLsizeiptr getStride() const
05669 {
05670     return static_cast<GLsizeiptr>(stride);
05671 }
05672
05678 const GLsizei& getCount() const
05679 {
05680     return count;
05681 }
05682
05688 const GLuint& getBuffer() const
05689 {
05690     return buffer;
05691 }
05692
05696 void bind() const
05697 {
05698     glBindBuffer(target, buffer);
05699 }
05700
05704 void unbind() const
05705 {
05706     glBindBuffer(target, 0);
05707 }
05708
05714 void* map() const
05715 {
05716     glBindBuffer(target, buffer);
05717 #if defined(GL_GLES_PROTOTYPES)
05718     return glMapBufferRange(target, 0, getStride() * count, GL_MAP_WRITE_BIT);
05719 #else
05720     return glMapBuffer(target, GL_WRITE_ONLY);
05721 #endif
05722 }
05723
05731 void* map(GLint first, GLsizei count) const
05732 {
05733     // count が 0 なら全データをマップする

```

```

05734     if (count == 0) count = getCount();
05735     if (first + count > getCount()) count = getCount() - first;
05736
05737     glBindBuffer(target, buffer);
05738     return glMapBufferRange(target, getStride() * first, getStride() * count, GL_MAP_WRITE_BIT);
05739 }
05740
05741 void unmap() const
05742 {
05743     glUnmapBuffer(target);
05744 }
05745
05746 void send(const T* data, GLint first, GLsizei count) const
05747 {
05748     // count が 0 なら全データを転送する
05749     if (count == 0) count = getCount();
05750     if (first + count > getCount()) count = getCount() - first;
05751
05752     // データを既存のバッファオブジェクトに転送する
05753     glBindBuffer(target, buffer);
05754     glBufferSubData(target, getStride() * first, getStride() * count, data);
05755 }
05756
05757 void read(T* data, GLint first, GLsizei count) const
05758 {
05759     // count が 0 なら全データを抽出する
05760     if (count == 0) count = getCount();
05761     if (first + count > getCount()) count = getCount() - first;
05762
05763     // データをバッファオブジェクトから抽出する
05764     glBindBuffer(target, buffer);
05765 #if defined(GL_GLES_PROTOTYPES)
05766     const GLsizeiptr begin{ getStride() * first };
05767     const GLsizeiptr range{ getStride() * count };
05768     T* const source{ glMapBufferRange(target, begin, range, GL_MAP_READ_BIT) };
05769     std::copy(source, source + count, data);
05770     glUnmapBuffer(target);
05771 #else
05772     glGetBufferSubData(target, getStride() * first, getStride() * count, data);
05773 #endif
05774 }
05775
05776 void copy(GLuint src_buffer, GLint src_first = 0, GLint dst_first = 0, GLsizei count = 0) const
05777 {
05778     // count が 0 なら全データを複写する
05779     if (count == 0) count = getCount();
05780     if (src_first + count > getCount()) count = getCount() - src_first;
05781     if (dst_first + count > getCount()) count = getCount() - dst_first;
05782
05783     // データの間隔
05784     const GLsizeiptr stride{ getStride() };
05785
05786     glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
05787     glBindBuffer(GL_COPY_WRITE_BUFFER, buffer);
05788     glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
05789         stride * src_first, stride * dst_first, stride * count);
05790     glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
05791     glBindBuffer(GL_COPY_READ_BUFFER, 0);
05792 }
05793 };
05794
05795 template <typename T>
05796 class GgUniformBuffer
05797 {
05798     // ユニフォームバッファオブジェクト
05799     std::shared_ptr<GgBuffer<T>> uniform;
05800
05801 public:
05802     GgUniformBuffer<T>()
05803     {
05804     }
05805
05806     GgUniformBuffer<T>(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
05807     {
05808         load(data, count, usage);
05809     }
05810
05811     GgUniformBuffer<T>(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
05812     {
05813         load(data, count, usage);
05814     }
05815
05816     virtual ~GgUniformBuffer<T>()
05817     {
05818     }
05819 }

```

```

05877     const GLuint& getTarget() const
05878     {
05879         return uniform->getTarget();
05880     }
05881
05887     GLsizeiptr getStride() const
05888     {
05889         return uniform->getStride();
05890     }
05891
05897     const GLsizei& getCount() const
05898     {
05899         return uniform->getCount();
05900     }
05901
05907     const GLuint& getBuffer() const
05908     {
05909         return uniform->getBuffer();
05910     }
05911
05915     void bind() const
05916     {
05917         uniform->bind();
05918     }
05919
05923     void unbind() const
05924     {
05925         uniform->unbind();
05926     }
05927
05933     void* map() const
05934     {
05935         return uniform->map();
05936     }
05937
05945     void* map(GLint first, GLsizei count) const
05946     {
05947         return uniform->map(first, count);
05948     }
05949
05953     void unmap() const
05954     {
05955         uniform->unmap();
05956     }
05957
05965     void load(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
05966     {
05967         // バッファオブジェクト上のデータの間の隔
05968         const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment )};
05969
05970         // ユニフォームバッファオブジェクトを確保する
05971         uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
05972
05973         // 確保したユニフォームバッファオブジェクトにデータを転送する
05974         if (data) send(data, 0, sizeof(T), 0, count);
05975     }
05976
05984     void load(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
05985     {
05986         // バッファオブジェクト上のデータの間の隔
05987         const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment )};
05988
05989         // ユニフォームバッファオブジェクトを確保する
05990         uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
05991
05992         // 確保したユニフォームバッファオブジェクトにデータを転送する
05993         fill(&data, 0, sizeof(T), 0, count);
05994     }
05995
06005     void send(
06006         const GLvoid* data,
06007         GLint offset = 0,
06008         GLsizei size = sizeof(T),
06009         GLint first = 0,
06010         GLsizei count = 0
06011     ) const
06012     {
06013         // count が 0 なら全データを転送する
06014         if (count == 0) count = getCount();
06015         if (first + count > getCount()) count = getCount() - first;
06016
06017         // 転送元のデータの先頭
06018         const char* source{ reinterpret_cast<const char*>(data) };
06019
06020         // ターゲット

```

```

06021     const GLuint target{ getTarget() };
06022
06023     // データの間隔
06024     const GLsizeiptr stride{ getStride() };
06025
06026     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06027     bind();
06028     for (GLsizei i = 0; i < count; ++i)
06029     {
06030         glBufferSubData(target, stride * (first + i) + offset, size, source + size * i);
06031     }
06032 }
06033
06043 void fill(
06044     const GLvoid* data,
06045     GLint offset = 0,
06046     GLsizei size = sizeof(T),
06047     GLint first = 0,
06048     GLsizei count = 0
06049 ) const
06050 {
06051     // count が 0 なら全データを転送する
06052     if (count == 0) count = getCount();
06053     if (first + count > getCount()) count = getCount() - first;
06054
06055     // ターゲット
06056     const GLuint target{ getTarget() };
06057
06058     // データの間隔
06059     const GLsizeiptr stride{ getStride() };
06060
06061     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06062     bind();
06063     for (GLsizei i = 0; i < count; ++i)
06064     {
06065         glBufferSubData(target, stride * (first + i) + offset, size, data);
06066     }
06067 }
06068
06078 void read(
06079     GLvoid* data,
06080     GLint offset = 0,
06081     GLsizei size = sizeof(T),
06082     GLint first = 0,
06083     GLsizei count = 0
06084 ) const
06085 {
06086     // count が 0 なら全データを転送する
06087     if (count == 0) count = getCount();
06088     if (first + count > getCount()) count = getCount() - first;
06089
06090     // 抽出先のデータの先頭
06091     char* const destination{ reinterpret_cast<char*>(data) };
06092
06093     // ターゲット
06094     const GLuint target{ getTarget() };
06095
06096     // データの間隔
06097     const GLsizeiptr stride{ getStride() };
06098
06099     // データをユニフォームバッファオブジェクトから抽出する
06100     bind();
06101     #if defined(GL_GLES_PROTOTYPES)
06102     const char* const source{ glMapBufferRange(target, stride * first, stride * count,
06103         GL_MAP_READ_BIT) };
06104     for (GLsizei i = 0; i < count; ++i)
06105     {
06106         const char* const begin{ source + stride * i + offset };
06107         const char* const end{ begin + size };
06108         std::copy(begin, end, destination + sizeof(T) * i);
06109     }
06110     glUnmapBuffer(target);
06111 #else
06112     for (GLsizei i = 0; i < count; ++i)
06113     {
06114         glGetBufferSubData(target, stride * (first + i) + offset, size, destination + sizeof(T) * i);
06115     }
06116 #endif
06117
06126 void copy(
06127     GLuint src_buffer,
06128     GLint src_first = 0,
06129     GLint dst_first = 0,
06130     GLsizei count = 0
06131 ) const
06132 {

```



```

06133         // count が 0 なら全データを複写する
06134         if (count == 0) count = getCount();
06135         if (src_first + count > getCount()) count = getCount() - src_first;
06136         if (dst_first + count > getCount()) count = getCount() - dst_first;
06137
06138         // データの間隔
06139         const GLsizeiptr stride{ getStride() };
06140
06141         // ユニフォームバッファオブジェクトではブロックごとに転送する
06142         glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
06143         glBindBuffer(GL_COPY_WRITE_BUFFER, getBuffer());
06144         for (GLsizei i = 0; i < count; ++i)
06145         {
06146             glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
06147                                 stride * (src_first + i), stride * (dst_first + i), sizeof(T));
06148         }
06149         glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
06150         glBindBuffer(GL_COPY_READ_BUFFER, 0);
06151     }
06152 };
06153
06154 class GgVertexArray
06155 {
06156     // 頂点配列オブジェクト
06157     const GLuint vao;
06158
06159 public:
06160
06161     GgVertexArray(GLenum mode = 0) :
06162         vao{ [] { GLuint vao; glGenVertexArrays(1, &vao); return vao; } () }
06163     {
06164         glBindVertexArray(vao);
06165     }
06166
06167     GgVertexArray(const GgVertexArray& array) = delete;
06168
06169     GgVertexArray(GgVertexArray&& array) = default;
06170
06171     virtual ~GgVertexArray()
06172     {
06173         glBindVertexArray(0);
06174         glDeleteVertexArrays(1, &vao);
06175     }
06176
06177     GgVertexArray& operator=(const GgVertexArray& array) = delete;
06178
06179     GgVertexArray& operator=(GgVertexArray&& array) = default;
06180
06181     const GLuint& get() const
06182     {
06183         return vao;
06184     }
06185
06186     void bind() const
06187     {
06188         glBindVertexArray(vao);
06189     }
06190 };
06191
06192 class GgShape
06193 {
06194     // 頂点配列オブジェクト
06195     std::shared_ptr<GgVertexArray> object;
06196
06197     // 基本図形の種類
06198     GLenum mode;
06199
06200 public:
06201
06202     GgShape(GLenum mode = 0) :
06203         object{ std::make_shared<GgVertexArray>() },
06204         mode{ mode }
06205     {
06206     }
06207
06208     virtual ~GgShape()
06209     {
06210     }
06211
06212     virtual explicit operator bool() const noexcept
06213     {
06214         return object.get() != nullptr;
06215     }
06216
06217     virtual bool operator!() const noexcept
06218     {
06219         return !static_cast<bool>(*this);
06220     }
06221 };

```

```

06286     }
06287
06293     const GLuint& get() const
06294     {
06295         return object->get();
06296     }
06297
06303     void setMode(GLenum mode)
06304     {
06305         this->mode = mode;
06306     }
06307
06313     const GLenum& getMode() const
06314     {
06315         return this->mode;
06316     }
06317
06324     virtual void draw(GLint first = 0, GLsizei count = 0) const
06325     {
06326         object->bind();
06327     }
06328 };
06329
06333 class GgPoints
06334 : public GgShape
06335 {
06336     // 頂点バッファオブジェクト
06337     std::shared_ptr<GgBuffer<GgVector>> position;
06338
06339 public:
06340
06344     GgPoints(GLenum mode = GL_POINTS) :
06345         GgShape(mode)
06346     {
06347     }
06348
06357     GgPoints(
06358         const GgVector* pos,
06359         GLsizei countv,
06360         GLenum mode = GL_POINTS,
06361         GLenum usage = GL_STATIC_DRAW
06362     ) :
06363         GgPoints(mode)
06364     {
06365         load(pos, countv, usage);
06366     }
06367
06371     virtual ~GgPoints()
06372     {
06373     }
06374
06380     explicit operator bool() const noexcept
06381     {
06382         return position.get() != nullptr;
06383     }
06384
06390     bool operator!() const noexcept
06391     {
06392         return !static_cast<bool>(*this);
06393     }
06394
06400     const GLsizei& getCount() const
06401     {
06402         return position->getCount();
06403     }
06404
06410     const GLuint& getBuffer() const
06411     {
06412         return position->getBuffer();
06413     }
06414
06422     void send(const GgVector* pos, GLint first = 0, GLsizei count = 0) const
06423     {
06424         position->send(pos, first, count);
06425     }
06426
06434     void load(const GgVector* pos, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06435
06442     virtual void draw(GLint first = 0, GLsizei count = 0) const;
06443 };
06444
06448 struct GgVertex
06449 {
06451     GgVector position;
06452
06454     GgVector normal;
06455

```

```

06459     GgVertex()
06460     {
06461     }
06462
06469     GgVertex(const GgVector& pos, const GgVector& norm) :
06470         position(pos),
06471         normal(norm)
06472     {
06473     }
06474
06485     GgVertex(
06486         GLfloat px, GLfloat py, GLfloat pz,
06487         GLfloat nx, GLfloat ny, GLfloat nz
06488     ) :
06489         position{ px, py, pz, 1.0f },
06490         normal{ nx, ny, nz, 0.0f }
06491     {
06492     }
06493
06500     GgVertex(const GLfloat* pos, const GLfloat* norm) :
06501         GgVertex(pos[0], pos[1], pos[2], norm[0], norm[1], norm[2])
06502     {
06503     }
06504 };
06505
06509     class GgTriangles
06510     : public GgShape
06511     {
06512     // 頂点属性
06513         std::shared_ptr<GgBuffer<GgVertex>> vertex;
06514
06515     public:
06516
06522         GgTriangles(GLenum mode = GL_TRIANGLES) :
06523             GgShape(mode)
06524         {
06525         }
06526
06535         GgTriangles(
06536             const GgVertex* vert,
06537             GLsizei count,
06538             GLenum mode = GL_TRIANGLES,
06539             GLenum usage = GL_STATIC_DRAW
06540         ) :
06541             GgTriangles(mode)
06542         {
06543             load(vert, count, usage);
06544         }
06545
06549         virtual ~GgTriangles()
06550         {
06551         }
06552
06558         const GLsizei& getCount() const
06559         {
06560             return vertex->getCount();
06561         }
06562
06568         const GLuint& getBuffer() const
06569         {
06570             return vertex->getBuffer();
06571         }
06572
06580         void send(const GgVertex* vert, GLint first = 0, GLsizei count = 0) const
06581         {
06582             vertex->send(vert, first, count);
06583         }
06584
06592         void load(const GgVertex* vert, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06593
06600         virtual void draw(GLint first = 0, GLsizei count = 0) const;
06601     };
06602
06606     class GgElements
06607     : public GgTriangles
06608     {
06609     // インデックスを格納する頂点バッファオブジェクト
06610         std::shared_ptr<GgBuffer<GLuint>> index;
06611
06612     public:
06613
06619         GgElements(GLenum mode = GL_TRIANGLES) :
06620             GgTriangles(mode)
06621         {
06622         }
06623
06634         GgElements(

```

```

06635     const GgVertex* vert,
06636     GLsizei countv,
06637     const GLuint* face,
06638     GLsizei countf,
06639     GLenum mode = GL_TRIANGLES,
06640     GLenum usage = GL_STATIC_DRAW
06641 ) :
06642     GgElements(mode)
06643 {
06644     load(vert, countv, face, countf, usage);
06645 }
06646
06650 virtual ~GgElements()
06651 {
06652 }
06653
06658 const GLsizei& getIndexCount() const
06659 {
06660     return index->getCount();
06661 }
06662
06668 const GLuint& getIndexBuffer() const
06669 {
06670     return index->getBuffer();
06671 }
06672
06683 void send(
06684     const GgVertex* vert,
06685     GLuint firstv,
06686     GLsizei countv,
06687     const GLuint* face = nullptr,
06688     GLuint firstf = 0,
06689     GLsizei countf = 0
06690 ) const
06691 {
06692     GgTriangles::send(vert, firstv, countv);
06693     if (face != nullptr && countf > 0) index->send(face, firstf, countf);
06694 }
06695
06705 void load(
06706     const GgVertex* vert,
06707     GLsizei countv,
06708     const GLuint* face,
06709     GLsizei countf,
06710     GLenum usage = GL_STATIC_DRAW
06711 )
06712 {
06713     // 頂点バッファオブジェクトを作成する
06714     GgTriangles::load(vert, countv, usage);
06715
06716     // インデックスの頂点バッファオブジェクトを作成する
06717     index = std::make_shared<GgBuffer<GLuint>>(GL_ELEMENT_ARRAY_BUFFER, face,
static_cast<GLsizei>(sizeof(GLuint)), countf, usage);
06718 }
06719
06726 virtual void draw(GLint first = 0, GLsizei count = 0) const;
06727 };
06728
06739 extern std::shared_ptr<GgPoints> ggPointsCube(
06740     GLsizei countv,
06741     GLfloat length = 1.0f,
06742     GLfloat cx = 0.0f,
06743     GLfloat cy = 0.0f,
06744     GLfloat cz = 0.0f
06745 );
06746
06757 extern std::shared_ptr<GgPoints> ggPointsSphere(
06758     GLsizei countv,
06759     GLfloat radius = 0.5f,
06760     GLfloat cx = 0.0f,
06761     GLfloat cy = 0.0f,
06762     GLfloat cz = 0.0f
06763 );
06764
06772 extern std::shared_ptr<GgTriangles> ggRectangle(
06773     GLfloat width = 1.0f,
06774     GLfloat height = 1.0f
06775 );
06776
06785 extern std::shared_ptr<GgTriangles> ggEllipse(
06786     GLfloat width = 1.0f,
06787     GLfloat height = 1.0f,
06788     GLuint slices = 16
06789 );
06790
06802 extern std::shared_ptr<GgTriangles> ggArraysObj(
06803     const std::string& name,

```

```

06804     bool normalize = false
06805 );
06806
06818 extern std::shared_ptr<GgElements> ggElementsObj(
06819     const std::string& name,
06820     bool normalize = false
06821 );
06822
06835 extern std::shared_ptr<GgElements> ggElementsMesh(
06836     GLuint slices,
06837     GLuint stacks,
06838     const GLfloat(*pos)[3],
06839     const GLfloat(*norm)[3] = nullptr
06840 );
06841
06852 extern std::shared_ptr<GgElements> ggElementsSphere(
06853     GLfloat radius = 1.0f,
06854     int slices = 16,
06855     int stacks = 8
06856 );
06857
06864 class GgShader
06865 {
06866     // プログラム名
06867     const GLuint program;
06868
06869 public:
06870
06880     GgShader(
06881         const std::string& vert,
06882         const std::string& frag = "",
06883         const std::string& geom = "",
06884         int nvarying = 0,
06885         const char* const* varyings = nullptr
06886     ) :
06887         program(ggLoadShader(vert, frag, geom, nvarying, varyings))
06888     {
06889     }
06890
06898     GgShader(
06899         const std::array<std::string, 3>& files,
06900         int nvarying = 0,
06901         const char* const* varyings = nullptr
06902     ) :
06903         GgShader(files[0], files[1], files[2], nvarying, varyings)
06904     {
06905     }
06906
06912     GgShader(const GgShader& shader) = delete;
06913
06919     GgShader(GgShader&& shader) = default;
06920
06924     virtual ~GgShader()
06925     {
06926         // 参照しているオブジェクトが一つだけならシェーダを削除する
06927         glUseProgram(0);
06928         glDeleteProgram(program);
06929     }
06930
06937     GgShader& operator=(const GgShader& shader) = delete;
06938
06945     GgShader& operator=(GgShader&& shader) = default;
06946
06950     void use() const
06951     {
06952         glUseProgram(program);
06953     }
06954
06958     void unuse() const
06959     {
06960         glUseProgram(0);
06961     }
06962
06968     GLuint get() const
06969     {
06970         return program;
06971     }
06972 };
06973
06977 class GgPointShader
06978 {
06979     // シェーダー
06980     std::shared_ptr<GgShader> shader;
06981
06982     // 投影変換行列の uniform 変数の場所
06983     GLint mpLoc;
06984

```

```

06985 // モデルビュー変換行列の uniform 変数の場所
06986 GLint mvLoc;
06987
06988 public:
06989
06993 GgPointShader() :
06994     mpLoc{ -1 },
06995     mvLoc{ -1 }
06996 {
06997 }
06998
07008 GgPointShader(
07009     const std::string& vert,
07010     const std::string& frag = "",
07011     const std::string& geom = "",
07012     GLint nvarying = 0,
07013     const char* const* varyings = nullptr
07014 ) :
07015     GgPointShader()
07016 {
07017     load(vert, frag, geom, nvarying, varyings);
07018 }
07019
07027 GgPointShader(
07028     const std::array<std::string, 3>& files,
07029     int nvarying = 0,
07030     const char* const* varyings = nullptr
07031 ) :
07032     GgPointShader(files[0], files[1], files[2], nvarying, varyings)
07033 {
07034 }
07035
07039 virtual ~GgPointShader()
07040 {
07041 }
07042
07053 bool load(
07054     const std::string& vert,
07055     const std::string& frag = "",
07056     const std::string& geom = "",
07057     GLint nvarying = 0,
07058     const char* const* varyings = nullptr
07059 )
07060 {
07061     // シェーダを作成する
07062     shader = std::make_shared<GgShader>(vert, frag, geom, nvarying, varyings);
07063
07064     // プログラム名を取り出す
07065     const GLuint program(shader->get());
07066
07067     // プログラムオブジェクトが作成できていなければ戻る
07068     if (program == 0) return false;
07069
07070     // 変換行列の uniform 変数の場所
07071     mpLoc = glGetUniformLocation(program, "mp");
07072     mvLoc = glGetUniformLocation(program, "mv");
07073
07074     // プログラムオブジェクトの作成に成功した
07075     return true;
07076 }
07077
07086 bool load(
07087     const std::array<std::string, 3>& files,
07088     GLint nvarying = 0,
07089     const char* const* varyings = nullptr
07090 )
07091 {
07092     return load(files[0], files[1], files[2], nvarying, varyings);
07093 }
07094
07100 virtual void loadProjectionMatrix(const GLfloat* mp) const
07101 {
07102     glUniformMatrix4fv(mpLoc, 1, GL_FALSE, mp);
07103 }
07104
07110 virtual void loadProjectionMatrix(const GgMatrix& mp) const
07111 {
07112     loadProjectionMatrix(mp.get());
07113 }
07114
07120 virtual void loadModelviewMatrix(const GLfloat* mv) const
07121 {
07122     glUniformMatrix4fv(mvLoc, 1, GL_FALSE, mv);
07123 }
07124
07130 virtual void loadModelviewMatrix(const GgMatrix& mv) const
07131 {

```

```

07132     loadModelviewMatrix(mv.get());
07133 }
07134
07141 virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07142 {
07143     loadProjectionMatrix(mp);
07144     loadModelviewMatrix(mv);
07145 }
07146
07153 virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07154 {
07155     loadMatrix(mp.get(), mv.get());
07156 }
07157
07161 virtual void use() const
07162 {
07163     shader->use();
07164 }
07165
07171 void use(const GLfloat* mp) const
07172 {
07173     use();
07174     loadProjectionMatrix(mp);
07175 }
07176
07182 void use(const GgMatrix& mp) const
07183 {
07184     use(mp.get());
07185 }
07186
07193 void use(const GLfloat* mp, const GLfloat* mv) const
07194 {
07195     use(mp);
07196     loadModelviewMatrix(mv);
07197 }
07198
07205 void use(const GgMatrix& mp, const GgMatrix& mv) const
07206 {
07207     use(mp.get(), mv.get());
07208 }
07209
07213 void unuse() const
07214 {
07215     shader->unuse();
07216 }
07217
07223 GLuint get() const
07224 {
07225     return shader->get();
07226 }
07227 };
07228
07232 class GgSimpleShader
07233 : public GgPointShader
07234 {
07235     // モデルビュー変換の法線変換行列の uniform 変数の場所
07236     GLint mnLoc;
07237
07238 public:
07239
07243 GgSimpleShader() :
07244     GgPointShader(),
07245     mnLoc{ -1 }
07246 {
07247 }
07248
07258 GgSimpleShader(
07259     const std::string& vert,
07260     const std::string& frag = "",
07261     const std::string& geom = "",
07262     GLint nvarying = 0,
07263     const char* const* varyings = nullptr
07264 )
07265 {
07266     load(vert, frag, geom, nvarying, varyings);
07267 }
07268
07276 GgSimpleShader(
07277     const std::array<std::string, 3>& files,
07278     GLint nvarying = 0,
07279     const char* const* varyings = nullptr
07280 ) :
07281     GgSimpleShader(files[0], files[1], files[2], nvarying, varyings)
07282 {
07283 }
07284
07288 GgSimpleShader(const GgSimpleShader& o) :

```

```

07289     GgPointShader(o),
07290     mnLoc{ o.mnLoc }
07291 {
07292 }
07293
07297 virtual ~GgSimpleShader()
07298 {
07299 }
07300
07307 GgSimpleShader& operator=(const GgSimpleShader& o)
07308 {
07309     if (&o != this)
07310     {
07311         GgPointShader::operator=(o);
07312         mnLoc = o.mnLoc;
07313     }
07314
07315     return *this;
07316 }
07317
07328 bool load(
07329     const std::string& vert,
07330     const std::string& frag = "",
07331     const std::string& geom = "",
07332     GLint nvarying = 0,
07333     const char* const* varyings = nullptr
07334 );
07335
07344 bool load(
07345     const std::array<std::string, 3>& files,
07346     GLint nvarying = 0,
07347     const char* const* varyings = nullptr
07348 )
07349 {
07350     return load(files[0], files[1], files[2], nvarying, varyings);
07351 }
07352
07359 virtual void loadModelviewMatrix(const GLfloat* mv, const GLfloat* mn) const
07360 {
07361     GgPointShader::loadModelviewMatrix(mv);
07362     glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07363 }
07364
07371 virtual void loadModelviewMatrix(const GgMatrix& mv, const GgMatrix& mn) const
07372 {
07373     loadModelviewMatrix(mv.get(), mn.get());
07374 }
07375
07381 virtual void loadModelviewMatrix(const GLfloat* mv) const
07382 {
07383     loadModelviewMatrix(mv, GgMatrix(mv).normal().get());
07384 }
07385
07391 virtual void loadModelviewMatrix(const GgMatrix& mv) const
07392 {
07393     loadModelviewMatrix(mv.get());
07394 }
07395
07403 virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
07404 {
07405     GgPointShader::loadMatrix(mp, mv);
07406     glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07407 }
07408
07416 virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
07417 {
07418     loadMatrix(mp.get(), mv.get(), mn.get());
07419 }
07420
07427 virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07428 {
07429     loadMatrix(mp, mv, GgMatrix(mv).normal());
07430 }
07431
07438 virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07439 {
07440     loadMatrix(mp, mv, mv.normal());
07441 }
07442
07446 struct Light
07447 {
07448     GgVector ambient;
07449     GgVector diffuse;
07450     GgVector specular;
07451     GgVector position;
07452 };
07453

```



```

07457     class LightBuffer
07458     : public GgUniformBuffer<Light>
07459     {
07460     public:
07461
07469         LightBuffer(
07470             const Light* light = nullptr,
07471             GLsizei count = 1,
07472             GLenum usage = GL_STATIC_DRAW
07473         ) :
07474             GgUniformBuffer<Light>(light, count, usage)
07475         {
07476         }
07477
07485         LightBuffer(
07486             const Light& light,
07487             GLsizei count = 1,
07488             GLenum usage = GL_STATIC_DRAW
07489         ) :
07490             GgUniformBuffer<Light>(light, count, usage)
07491         {
07492         }
07493
07504         LightBuffer(
07505             GgVector ambient,
07506             GgVector diffuse,
07507             GgVector specular,
07508             GgVector position,
07509             GLsizei count = 1,
07510             GLenum usage = GL_STATIC_DRAW
07511         ) :
07512             GgUniformBuffer<Light>(Light{ ambient, diffuse, specular, position }, count, usage)
07513         {
07514         }
07515
07519         virtual ~LightBuffer()
07520         {
07521         }
07522
07533         void loadAmbient(
07534             GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07535             GLint first = 0, GLsizei count = 1
07536         ) const;
07537
07545         void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
07546
07554         void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
07555         {
07556             // first 番目のブロックから count 個の ambient 要素に値を設定する
07557             send(ambient, offsetof(Light, ambient), sizeof(Light::ambient), first, count);
07558         }
07559
07570         void loadDiffuse(
07571             GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07572             GLint first = 0, GLsizei count = 1
07573         ) const;
07574
07582         void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
07583
07591         void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
07592         {
07593             // first 番目のブロックから count 個の diffuse 要素に値を設定する
07594             send(diffuse, offsetof(Light, diffuse), sizeof(Light::diffuse), first, count);
07595         }
07596
07607         void loadSpecular(
07608             GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07609             GLint first = 0, GLsizei count = 1
07610         ) const;
07611
07619         void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
07620
07628         void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
07629         {
07630             // first 番目のブロックから count 個の specular 要素に値を設定する
07631             send(specular, offsetof(Light, specular), sizeof(Light::specular), first, count);
07632         }
07633
07641         void loadColor(const Light& color, GLint first = 0, GLsizei count = 1) const;
07642
07653         void loadPosition(
07654             GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f,
07655             GLint first = 0, GLsizei count = 1
07656         ) const;
07657
07665         void loadPosition(const GgVector& position, GLint first = 0, GLsizei count = 1) const;
07666

```

```

07674 void loadPosition(const GLfloat* position, GLint first = 0, GLsizei count = 1) const
07675 {
07676     // first 番目のブロックから count 個の position 要素に値を設定する
07677     send(position, offsetof(Light, position), sizeof(Light::position), first, count);
07678 }
07679
07687 void loadPosition(const GgVector* position, GLint first = 0, GLsizei count = 1) const
07688 {
07689     loadPosition(position->data(), first, count);
07690 }
07691
07699 void load(const Light* light, GLint first = 0, GLsizei count = 1) const
07700 {
07701     send(light, 0, sizeof(Light), first, count);
07702 }
07703
07711 void load(const Light& light, GLint first = 0, GLsizei count = 1) const
07712 {
07713     load(&light, first, count);
07714 }
07715
07721 void select(GLint i = 0) const
07722 {
07723     // バッファオブジェクトの i 番目のブロックの位置
07724     const GLintptr offset(static_cast<GLintptr>(getStride()) * i);
07725     glBindBufferRange(getTarget(), LightBindingPoint, getBuffer(), offset, sizeof(Light));
07726 }
07727 };
07728
07732 struct Material
07733 {
07734     GgVector ambient;
07735     GgVector diffuse;
07736     GgVector specular;
07737     GLfloat shininess;
07738 };
07739
07743 class MaterialBuffer
07744     : public GgUniformBuffer<Material>
07745 {
07746 public:
07747
07755     MaterialBuffer(
07756         const Material* material = nullptr,
07757         GLsizei count = 1,
07758         GLenum usage = GL_STATIC_DRAW
07759     ) :
07760         GgUniformBuffer<Material>(material, count, usage)
07761     {
07762     }
07763
07771     MaterialBuffer(
07772         const Material& material,
07773         GLsizei count = 1,
07774         GLenum usage = GL_STATIC_DRAW
07775     ) :
07776         GgUniformBuffer<Material>(material, count, usage)
07777     {
07778     }
07779
07790     MaterialBuffer(
07791         GgVector ambient,
07792         GgVector diffuse,
07793         GgVector specular,
07794         GLfloat shininess,
07795         GLsizei count = 1,
07796         GLenum usage = GL_STATIC_DRAW
07797     ) :
07798         GgUniformBuffer<Material>(Material{ ambient, diffuse, specular, shininess }, count, usage)
07799     {
07800     }
07801
07805     virtual ~MaterialBuffer()
07806     {
07807     }
07808
07819 void loadAmbient(
07820     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07821     GLint first = 0, GLsizei count = 1
07822 ) const;
07823
07831 void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
07832
07840 void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
07841 {
07842     // first 番目のブロックから count 個のブロックの ambient 要素に値を設定する
07843     send(ambient, offsetof(Material, ambient), sizeof(Material::ambient), first, count);

```

```

07844     }
07845
07856 void loadDiffuse(
07857     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07858     GLint first = 0, GLsizei count = 1
07859 ) const;
07860
07868 void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
07869
07877 void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
07878 {
07879     // first 番目のブロックから count 個の diffuse 要素に値を設定する
07880     send(diffuse, offsetof(Material, diffuse), sizeof(Material::diffuse), first, count);
07881 }
07882
07893 void loadAmbientAndDiffuse(
07894     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07895     GLint first = 0, GLsizei count = 1
07896 ) const;
07897
07905 void loadAmbientAndDiffuse(const GgVector& color, GLint first = 0, GLsizei count = 1) const;
07906
07914 void loadAmbientAndDiffuse(const GLfloat* color, GLint first = 0, GLsizei count = 1) const;
07915
07926 void loadSpecular(
07927     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07928     GLint first = 0, GLsizei count = 1
07929 ) const;
07930
07938 void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
07939
07947 void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
07948 {
07949     // first 番目のブロックから count 個の specular 要素に値を設定する
07950     send(specular, offsetof(Material, specular), sizeof(Material::specular), first, count);
07951 }
07952
07960 void loadShininess(GLfloat shininess, GLint first = 0, GLsizei count = 1) const;
07961
07969 void loadShininess(const GLfloat* shininess, GLint first = 0, GLsizei count = 1) const;
07970
07978 void load(const Material* material, GLint first = 0, GLsizei count = 1) const
07979 {
07980     send(material, 0, sizeof(Material), first, count);
07981 }
07982
07990 void load(const Material& material, GLint first = 0, GLsizei count = 1) const
07991 {
07992     load(&material, first, count);
07993 }
07994
08000 void select(GLint i = 0) const
08001 {
08002     // バッファオブジェクトの i 番目のブロックの位置
08003     const GLintptr offset{ static_cast<GLintptr>(getStride()) * i };
08004     glBindBufferRange(getTarget(), MaterialBindingPoint, getBuffer(), offset, sizeof(Material));
08005 }
08006 };
08007
08011 void use() const
08012 {
08013     // プログラムオブジェクトは基底クラスで指定する
08014     GgPointShader::use();
08015 }
08016
08024 void use(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
08025 {
08026     // プログラムオブジェクトを指定する
08027     use();
08028
08029     // 変換行列を設定する
08030     loadMatrix(mp, mv, mn);
08031 }
08032
08040 void use(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
08041 {
08042     use(mp.get(), mv.get(), mn.get());
08043 }
08044
08051 void use(const GLfloat* mp, const GLfloat* mv) const
08052 {
08053     use(mp, mv, GgMatrix(mv).normal().get());
08054 }
08055
08062 void use(const GgMatrix& mp, const GgMatrix& mv) const
08063 {
08064     use(mp, mv, mv.normal());

```

```

08065     }
08066
08073 void use(const LightBuffer* light, GLint i = 0) const
08074 {
08075     // プログラムオブジェクトを指定する
08076     use();
08077
08078     // 光源を設定する
08079     light->select(i);
08080 }
08081
08088 void use(const LightBuffer& light, GLint i = 0) const
08089 {
08090     use(&light, i);
08091 }
08092
08102 void use(
08103     const GLfloat* mp,
08104     const GLfloat* mv,
08105     const GLfloat* mn,
08106     const LightBuffer* light,
08107     GLint i = 0
08108 ) const
08109 {
08110     // 光源を指定してプログラムオブジェクトを指定する
08111     use(light, i);
08112
08113     // 変換行列を設定する
08114     loadMatrix(mp, mv, mn);
08115 }
08116
08126 void use(
08127     const GgMatrix& mp,
08128     const GgMatrix& mv,
08129     const GgMatrix& mn,
08130     const LightBuffer& light,
08131     GLint i = 0
08132 ) const
08133 {
08134     use(mp.get(), mv.get(), mn.get(), &light, i);
08135 }
08136
08145 void use(
08146     const GLfloat* mp,
08147     const GLfloat* mv,
08148     const LightBuffer* light,
08149     GLint i = 0
08150 ) const
08151 {
08152     use(mp, mv, GgMatrix(mv).normal().get(), light, i);
08153 }
08154
08163 void use(
08164     const GgMatrix& mp,
08165     const GgMatrix& mv,
08166     const LightBuffer& light,
08167     GLint i = 0
08168 ) const
08169 {
08170     use(mp, mv, mv.normal(), light, i);
08171 }
08172
08180 void use(const GLfloat* mp, const LightBuffer* light, GLint i = 0) const
08181 {
08182     // 光源を指定してプログラムオブジェクトを指定する
08183     use(light, i);
08184
08185     // 投影変換行列を設定する
08186     loadProjectionMatrix(mp);
08187 }
08188
08196 void use(const GgMatrix& mp, const LightBuffer& light, GLint i = 0) const
08197 {
08198     // 光源を指定してプログラムオブジェクトを指定する
08199     use(mp.get(), &light, i);
08200 }
08201 };
08202
08213 extern bool ggLoadSimpleObj(
08214     const std::string& name,
08215     std::vector<std::array<GLuint, 3>>& group,
08216     std::vector<GgSimpleShader::Material>& material,
08217     std::vector<GgVertex>& vert,
08218     bool normalize = false
08219 );
08220
08232 extern bool ggLoadSimpleObj(

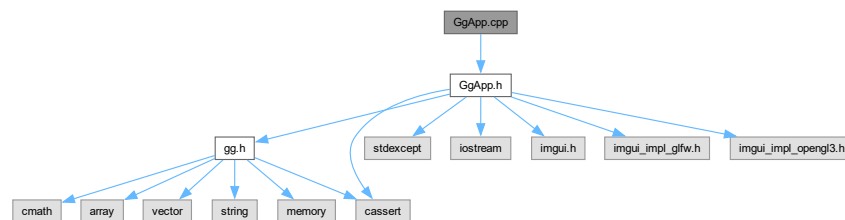
```

```

08233     const std::string& name,
08234     std::vector<std::array<GLuint, 3>>& group,
08235     std::vector<GgSimpleShader::Material>& material,
08236     std::vector<GgVertex>& vert,
08237     std::vector<GLuint>& face,
08238     bool normalize = false
08239 );
08240
08241 class GgSimpleObj
08242 {
08243     // 同じ材質を割り当てるポリゴングループごとの三角形数
08244     std::shared_ptr<std::vector<std::array<GLuint, 3>>> group;
08245
08246     // ポリゴングループごとの材質のユニフォームバッファ
08247     std::shared_ptr<GgSimpleShader::MaterialBuffer> material;
08248
08249     // この図形の形状データ
08250     std::shared_ptr<GgElements> data;
08251
08252 public:
08253     GgSimpleObj(const std::string& name, bool normalize = false);
08254
08255     virtual ~GgSimpleObj()
08256     {
08257     }
08258
08259     explicit operator bool() const noexcept
08260     {
08261         return data.get() != nullptr;
08262     }
08263
08264     bool operator!() const noexcept
08265     {
08266         return !static_cast<bool>(*this);
08267     }
08268
08269     const GgTriangles* get() const
08270     {
08271         return data.get();
08272     }
08273
08274     virtual void draw(GLint first = 0, GLsizei count = 0) const;
08275 };
08276
08277 }
```

9.43 GgApp.cpp ファイル

#include "GgApp.h"
GgApp.cpp の依存先関係図:



9.43.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの実装.

著者

Kohe Tokoi

日付

July 27, 2025

[GgApp.cpp](#) に定義があります。

9.44 GgApp.cpp

[詳解]

```
00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00032 #include "GgApp.h"
00033
00034 //
00035 // GLFW のエラー表示
00036 //
00037 static void glfwErrorCallback(int error, const char* description)
00038 {
00039     #if defined(__aarch64__)
00040         if (error == 65544) return;
00041     #endif
00042     throw std::runtime_error(description);
00043 }
00044
00045 //
00046 // GgApp クラスのコンストラクタ
00047 //
00048 GgApp::GgApp(int major, int minor)
00049 {
00050     // GLFW のエラー処理関数を登録する
00051     glfwSetErrorCallback(glfwErrorCallback);
00052
00053     // GLFW を初期化する
00054     if (glfwInit() == GLFW_FALSE) throw std::runtime_error("Can't initialize GLFW");
00055
00056     // OpenGL の major 番号が指定されていれば
00057     if (major > 0)
00058     {
00059         // OpenGL のバージョンを指定する
00060         glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, major);
00061         glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, minor);
00062
00063         #if defined(GL_GLES_PROTOTYPES)
00064             // OpenGL ES 3 のコンテキストを指定する
00065             glfwWindowHint(GLFW_CLIENT_API, GLFW_OPENGL_ES_API);
00066             glfwWindowHint(GLFW_CONTEXT_CREATION_API, GLFW_EGL_CONTEXT_API);
00067         #else
00068             // OpenGL Version 3.2 以降なら
00069             if (major * 10 + minor >= 32)
00070             {
00071                 // Core Profile を選択する (macOS の都合)
00072                 glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
00073                 glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
00074             }
00075         #endif
00076     }
00077
00078     #if defined(GG_USE_OCULUS_RIFT)
00079         // Oculus Rift では SRGB でレンダリングする
00080         glfwWindowHint(GLFW_SRGB_CAPABLE, GL_TRUE);
00081     #endif
}
```

```

00082
00083 #if defined(IMGUI_VERSION)
00084 // ImGui のバージョンをチェックする
00085 IMGUI_CHECKVERSION();
00086
00087 // ImGui のコンテキストを作成する
00088 ImGui::CreateContext();
00089 #endif
00090 }
00091
00092 //
00093 // デストラクタ
00094 //
00095 GgApp::~GgApp()
00096 {
00097 #if defined(IMGUI_VERSION)
00098 // Shutdown Platform/Renderer bindings
00099 ImGui_ImplOpenGL3_Shutdown();
00100 ImGui_ImplGlfw_Shutdown();
00101 ImGui::DestroyContext();
00102 #endif
00103
00104 // プログラム終了時に GLFW を終了する
00105 glfwTerminate();
00106 }
00107
00108 //
00109 // マウスや矢印キーによる平行移動量を初期化する
00110 //
00111 void GgApp::Window::HumanInterface::resetTranslation()
00112 {
00113 // 平行移動量を初期化する
00114 for (auto& t : translation)
00115 {
00116 std::fill(t.begin(), t.end(), GgVector{ 0.0f, 0.0f, 0.0f, 1.0f });
00117 }
00118
00119 // 矢印キーの設定値を初期化する
00120 std::fill(arrow.begin(), arrow.end(), std::array<int, 2>{ 0, 0 });
00121
00122 // マウスホイールの回転量を初期化する
00123 std::fill(wheel.begin(), wheel.end(), 0.0f);
00124 }
00125
00126 //
00127 // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00128 //
00129 void GgApp::Window::HumanInterface::calcTranslation(int button, const std::array<GLfloat, 3>&
velocity)
00130 {
00131 // マウスの相対変位
00132 assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00133 const auto dx{ (mouse[0] - rotation[button].getStart(0)) * rotation[button].getScale(0) };
00134 const auto dy{ (rotation[button].getStart(1) - mouse[1]) * rotation[button].getScale(1) };
00135
00136 // 平行移動量
00137 auto& t{ translation[button] };
00138
00139 // 平行移動量の更新
00140 t[1][0] = dx * velocity[0] + t[0][0];
00141 t[1][1] = dy * velocity[1] + t[0][1];
00142
00143 // 回転量の更新
00144 rotation[button].motion(mouse[0], mouse[1]);
00145 }
00146
00147 //
00148 // ウィンドウのサイズ変更時の処理
00149 //
00150 void GgApp::Window::resize(GLFWwindow* window, int width, int height)
00151 {
00152 // このインスタンスの this ポインタを得る
00153 auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00154
00155 if (instance)
00156 {
00157 // ウィンドウのサイズを保存する
00158 instance->size[0] = width;
00159 instance->size[1] = height;
00160
00161 // トラックボール処理の範囲を設定する
00162 for (auto& current_if : instance->interfaceData)
00163 {
00164 for (auto& t : current_if.rotation)
00165 {
00166 t.region(width, height);
00167 }

```

```

00168     }
00169
00170     // ビューポートを更新する
00171     instance->updateViewport();
00172
00173     // ユーザー定義のコールバック関数の呼び出し
00174     if (instance->resizeFunc) (*instance->resizeFunc)(instance, width, height);
00175 }
00176 }
00177
00178 //
00179 // キーボードをタイプした時の処理
00180 //
00181 void GgApp::Window::keyboard(GLFWwindow* window, int key, int scancode, int action, int mods)
00182 {
00183     #if defined(IMGUI_VERSION)
00184         // ImGui がキーボードを使うときはキーボードの処理を行わない
00185         if (ImGui::GetIO().WantCaptureKeyboard) return;
00186     #endif
00187
00188     // このインスタンスの this ポインタを得る
00189     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00190
00191     if (instance && action)
00192     {
00193         // ユーザー定義のコールバック関数の呼び出し
00194         if (instance->keyboardFunc) (*instance->keyboardFunc)(instance, key, scancode, action, mods);
00195
00196         // 対象のユーザインタフェース
00197         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00198
00199         switch (key)
00200         {
00201             case GLFW_KEY_HOME:
00202
00203                 // トラックボールを初期化する
00204                 instance->resetRotation();
00205                 [[fallthrough]];
00206
00207             case GLFW_KEY_END:
00208
00209                 // 平行移動量を初期化する
00210                 instance->resetTranslation();
00211                 break;
00212
00213             case GLFW_KEY_UP:
00214
00215                 if (mods & GLFW_MOD_SHIFT)
00216                     current_if.arrow[1][1]++;
00217                 else if (mods & GLFW_MOD_CONTROL)
00218                     current_if.arrow[2][1]++;
00219                 else if (mods & GLFW_MOD_ALT)
00220                     current_if.arrow[3][1]++;
00221                 else
00222                     current_if.arrow[0][1]++;
00223                 break;
00224
00225             case GLFW_KEY_DOWN:
00226
00227                 if (mods & GLFW_MOD_SHIFT)
00228                     current_if.arrow[1][1]--;
00229                 else if (mods & GLFW_MOD_CONTROL)
00230                     current_if.arrow[2][1]--;
00231                 else if (mods & GLFW_MOD_ALT)
00232                     current_if.arrow[3][1]--;
00233                 else
00234                     current_if.arrow[0][1]--;
00235                 break;
00236
00237             case GLFW_KEY_RIGHT:
00238
00239                 if (mods & GLFW_MOD_SHIFT)
00240                     current_if.arrow[1][0]++;
00241                 else if (mods & GLFW_MOD_CONTROL)
00242                     current_if.arrow[2][0]++;
00243                 else if (mods & GLFW_MOD_ALT)
00244                     current_if.arrow[3][0]++;
00245                 else
00246                     current_if.arrow[0][0]++;
00247                 break;
00248
00249             case GLFW_KEY_LEFT:
00250
00251                 if (mods & GLFW_MOD_SHIFT)
00252                     current_if.arrow[1][0]--;
00253                 else if (mods & GLFW_MOD_CONTROL)
00254                     current_if.arrow[2][0]--;

```



```

00255         else if (mods & GLFW_MOD_ALT)
00256             current_if.arrow[3][0]--;
00257         else
00258             current_if.arrow[0][0]--;
00259         break;
00260
00261     default:
00262         break;
00263     }
00264
00265     current_if.lastKey = key;
00266 }
00267 }
00268
00269 //
00270 // マウスボタンを操作したときの処理
00271 //
00272 void GgApp::Window::mouse(GLFWwindow* window, int button, int action, int mods)
00273 {
00274     #if defined(IMGUI_VERSION)
00275         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00276         if (ImGui::GetIO().WantCaptureMouse) return;
00277     #endif
00278
00279     // このインスタンスの this ポインタを得る
00280     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00281
00282     // マウスボタンの状態を記録する
00283     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG.BUTTON_COUNT);
00284     instance->status[button] = action != GLFW_RELEASE;
00285
00286     if (instance)
00287     {
00288         // ユーザー定義のコールバック関数の呼び出し
00289         if (instance->mouseFunc) (*instance->mouseFunc)(instance, button, action, mods);
00290
00291         // 対象のユーザインタフェース
00292         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00293
00294         // マウスの現在位置を得る
00295         const auto x{ current_if.mouse[0] };
00296         const auto y{ current_if.mouse[1] };
00297
00298         if (x < 0 || x >= instance->size[0] || y < 0 || y >= instance->size[1]) return;
00299
00300         if (action)
00301         {
00302             // ドラッグ開始
00303             current_if.rotation[button].begin(x, y);
00304         }
00305         else
00306         {
00307             // ドラッグ終了
00308             current_if.translation[button][0] = current_if.translation[button][1];
00309             current_if.rotation[button].end(x, y);
00310         }
00311     }
00312 }
00313
00314 //
00315 // マウスホイールを操作した時の処理
00316 //
00317 void GgApp::Window::wheel(GLFWwindow* window, double x, double y)
00318 {
00319     #if defined(IMGUI_VERSION)
00320         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00321         if (ImGui::GetIO().WantCaptureMouse) return;
00322     #endif
00323
00324     // このインスタンスの this ポインタを得る
00325     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00326
00327     if (instance)
00328     {
00329         // ユーザー定義のコールバック関数の呼び出し
00330         if (instance->wheelFunc) (*instance->wheelFunc)(instance, x, y);
00331
00332         // 対象のユーザインタフェース
00333         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00334
00335         // マウスホイールの回転量の保存
00336         current_if.wheel[0] += static_cast<GLfloat>(x);
00337         current_if.wheel[1] += static_cast<GLfloat>(y);
00338
00339         // マウスによる平行移動量の z 値の更新
00340         const auto z{ current_if.wheel[1] * instance->velocity[2] };
00341         for (auto& t : current_if.translation) t[1][2] = z;

```

```

00342     }
00343 }
00344
00345 //
00346 // Window クラスのコンストラクタ
00347 //
00348 GgApp::Window::Window(const std::string& title, int width, int height, int fullscreen, GLFWwindow*
share) :
00349     window{ nullptr },
00350     size{ width, height },
00351     fboSize{ width, height },
00352     #if defined(IMGUI_VERSION)
00353     menubarHeight{ 0 },
00354     #endif
00355     aspect{ 1.0f },
00356     velocity{ 1.0f, 1.0f, 0.1f },
00357     status{ false },
00358     interfaceNo{ 0 },
00359     userPointer{ nullptr },
00360     resizeFunc{ nullptr },
00361     keyboardFunc{ nullptr },
00362     mouseFunc{ nullptr },
00363     wheelFunc{ nullptr }
00364 {
00365     // ディスプレイの情報
00366     GLFWmonitor* monitor{ nullptr };
00367
00368     // フルスクリーン表示
00369     if (fullscreen > 0)
00370     {
00371         // 接続されているモニタの数を数える
00372         int mcount;
00373         auto** const monitors{ glfwGetMonitors(&mcount) };
00374
00375         // セカンダリモニタがあればそれを使う
00376         if (fullscreen > mcount) fullscreen = mcount;
00377         monitor = monitors[fullscreen - 1];
00378
00379         // モニタのモードを調べる
00380         const auto* mode{ glfwGetVideoMode(monitor) };
00381
00382         // ウィンドウのサイズをディスプレイのサイズにする
00383         width = mode->width;
00384         height = mode->height;
00385     }
00386
00387     // GLFW のウィンドウを作成する
00388     window = glfwCreateWindow(width, height, title.c_str(), monitor, share);
00389
00390     // ウィンドウが作成できなければエラー
00391     if (!window) throw std::runtime_error("Unable to open the GLFW window.");
00392
00393     // 現在のウィンドウを処理対象にする
00394     glfwMakeContextCurrent(window);
00395
00396     // ゲームグラフィックス特論の都合による初期化を行う
00397     ggInit();
00398
00399     // このインスタンスの this ポインタを記録しておく
00400     glfwSetWindowUserPointer(window, this);
00401
00402     // キーボードを操作した時の処理を登録する
00403     glfwSetKeyCallback(window, keyboard);
00404
00405     // マウスボタンを操作したときの処理を登録する
00406     glfwSetMouseButtonCallback(window, mouse);
00407
00408     // マウスホイール操作時に呼び出す処理を登録する
00409     glfwSetScrollCallback(window, wheel);
00410
00411     // ウィンドウのサイズ変更時に呼び出す処理を登録する
00412     glfwSetFramebufferSizeCallback(window, resize);
00413
00414     // 垂直同期タイミングに合わせる
00415     glfwSwapInterval(1);
00416
00417     // 実際のフレームバッファのサイズを取得する
00418     glfwGetFramebufferSize(window, &width, &height);
00419
00420     // ビューポートと投影変換行列を初期化する
00421     resize(window, width, height);
00422
00423     #if defined(IMGUI_VERSION)
00424     // 最初のウィンドウを開いたとき
00425     static bool firstTime{ true };
00426     if (firstTime)
00427     {

```

```

00428 // Setup Platform/Renderer bindings
00429 ImGui_ImplGlfw_InitForOpenGL(window, true);
00430 ImGui_ImplOpenGL3_Init(nullptr);
00431
00432 // 実行済みであることを記録する
00433 firstTime = false;
00434 }
00435 #endif
00436 }
00437
00438 //
00439 // イベントを取得してループを継続すべきかどうか調べる
00440 //
00441 GgApp::Window::operator bool()
00442 {
00443 // イベントを取り出す
00444 glfwPollEvents();
00445
00446 // ウィンドウを閉じるべきなら false を返す
00447 if (shouldClose()) return false;
00448
00449 // 対象のユーザインタフェース
00450 auto& current_if{ interfaceData[interfaceNo] };
00451
00452 #if defined(IMGUI_VERSION)
00453 // ImGui の新規フレームを作成する
00454 ImGui_ImplOpenGL3_NewFrame();
00455 ImGui_ImplGlfw_NewFrame();
00456 ImGui::NewFrame();
00457
00458 // ImGui の状態を取り出す
00459 const ImGuiIO& io{ ImGui::GetIO() };
00460
00461 // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00462 if (io.WantCaptureMouse) return true;
00463
00464 // マウスの位置を更新する
00465 current_if.mouse = std::array<GLfloat, 2>{ io.MousePos.x, io.MousePos.y };
00466 #else
00467 // マウスの現在位置を調べる
00468 double x, y;
00469 glfwGetCursorPos(window, &x, &y);
00470
00471 // マウスの位置を更新する
00472 current_if.mouse = std::array<GLfloat, 2>{ static_cast<GLfloat>(x), static_cast<GLfloat>(y) };
00473 #endif
00474
00475 // マウスドラッグ
00476 for (int button = GLFW_MOUSE_BUTTON_1; button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT; ++button)
00477 {
00478 // マウスボタンを押していたら
00479 if (status[button])
00480 {
00481 // 現在位置と平行移動量を更新する
00482 current_if.calcTranslation(button, velocity);
00483 }
00484 }
00485
00486 return true;
00487 }
00488
00489 //
00490 // カラーバッファを入れ替える
00491 //
00492 void GgApp::Window::swapBuffers() const
00493 {
00494 #if defined(IMGUI_VERSION)
00495 // ImGui の描画データがあればフレームをレンダリングする
00496 ImGui::Render();
00497 ImDrawData* data{ ImGui::GetDrawData() };
00498 if (data) ImGui_ImplOpenGL3_RenderDrawData(data);
00499 #endif
00500
00501 // エラーチェック
00502 ggError();
00503
00504 // カラーバッファを入れ替える
00505 glfwSwapBuffers(window);
00506 }
00507
00508 //
00509 // ビューポートのサイズを更新する
00510 //
00511 void GgApp::Window::updateViewport()
00512 {
00513 // フレームバッファの大きさを求める
00514 glfwGetFramebufferSize(window, &fboSize[0], &fboSize[1]);

```

```

00515
00516 #if defined(IMGUI_VERSION)
00517 // フレームバッファの高さからメニューバーの高さを減じる
00518 fboSize[1] -= menubarHeight;
00519 #endif
00520
00521 // ウィンドウの縦横比を保存する
00522 aspect = static_cast<GLfloat>(fboSize[0]) / static_cast<GLfloat>(fboSize[1]);
00523
00524 // ビューポートを設定する
00525 restoreViewport();
00526 }
00527
00528 #if defined(GG_USE_OCULUS_RIFT)
00529 # if OVR_PRODUCT_VERSION > 0
00530 //
00531 // グラフィックスカードのデフォルトの LUID を得る
00532 //
00533 ovrGraphicsLuid GgApp::Oculus::GetDefaultAdapterLuid()
00534 {
00535     ovrGraphicsLuid luid = ovrGraphicsLuid();
00536
00537     # if defined(_WIN32)
00538     IDXGIFactory* factory{ nullptr };
00539
00540     if (SUCCEEDED(CreateDXGIFactory(IID_PPV_ARGS(&factory))))
00541     {
00542         DXGIAdapter* adapter{ nullptr };
00543
00544         if (SUCCEEDED(factory->EnumAdapters(0, &adapter)))
00545         {
00546             DXGI_ADAPTER_DESC desc;
00547
00548             adapter->GetDesc(&desc);
00549             memcpy(&luid, &desc.AdapterLuid, sizeof luid);
00550             adapter->Release();
00551         }
00552
00553         factory->Release();
00554     }
00555     # endif
00556
00557     return luid;
00558 }
00559
00560 //
00561 // グラフィックスカードの LUID の比較
00562 //
00563 int GgApp::Oculus::Compare(const ovrGraphicsLuid& lhs, const ovrGraphicsLuid& rhs)
00564 {
00565     return memcmp(&lhs, &rhs, sizeof(ovrGraphicsLuid));
00566 }
00567 # endif
00568
00569 //
00570 // コンストラクタ
00571 //
00572 GgApp::Oculus::Oculus() :
00573     session{ nullptr },
00574     oculusFbo{ 0 },
00575     screen{ -1.0f, 1.0f, -1.0f, 1.0f, -1.0f, 1.0f, -1.0f, },
00576     mirrorFbo{ 0 },
00577     window{ nullptr },
00578     # if OVR_PRODUCT_VERSION > 0
00579     frameIndex{ 0LL },
00580     oculusDepth{ 0 },
00581     mirrorWidth{ 1280 },
00582     mirrorHeight{ 640 },
00583     # endif
00584     mirrorTexture{ nullptr }
00585 {
00586 }
00587
00588 //
00589 // Oculus Rift のセッションを作成する
00590 //
00591 GgApp::Oculus& GgApp::Oculus::initialize(const Window& window)
00592 {
00593     // Oculus Rift のコンテキスト
00594     static Oculus oculus;
00595
00596     // 既に Oculus Rift のセッションが作成されていたら参照を返す
00597     if (oculus.session) return oculus;
00598
00599     // 最初に呼び出したときだけ実行する
00600     static bool firstTime{ true };
00601     if (firstTime)

```

```

00602 {
00603     // Oculus Rift (LibOVR) を初期化する
00604     ovrInitParams initParams{ ovrInit_RequestVersion, OVR_MINOR_VERSION, NULL, 0, 0 };
00605     if (OVR_FAILURE(ovr_Initialize(&initParams)))
00606         throw std::runtime_error("Can't initialize LibOVR");
00607
00608     // アプリケーションの終了時に LibOVR を終了する
00609     atexit(ovr_Shutdown);
00610
00611     // 実行済みであることを記録する
00612     firstTime = false;
00613 }
00614
00615 // Oculus Rift のセッションを作成する
00616 ovrGraphicsLuid luid;
00617 if (OVR_FAILURE(ovr_Create(&oculus.session, &luid)))
00618     throw std::runtime_error("Can't create Oculus Rift session");
00619
00620 # if OVR_PRODUCT_VERSION > 0
00621 // デフォルトのグラフィックスアダプタが使われているか確かめる
00622 if (Compare(luid, GetDefaultAdapterLuid()))
00623     throw std::runtime_error("Graphics adapter is not default");
00624 # endif
00625
00626 // session が無効ならエラー
00627 if (!oculus.session) std::runtime_error("Unable to use the Oculus Rift.");
00628
00629 // ミラー表示を行うウィンドウを設定する
00630 oculus.window = &window;
00631
00632 // Oculus Rift の情報を取り出す
00633 oculus.hmdDesc = ovr_GetHmdDesc(oculus.session);
00634
00635 # if defined(_DEBUG)
00636 // Oculus Rift の情報を表示する
00637 std::cerr
00638     << "\nProduct name: " << oculus.hmdDesc.ProductName
00639     << "\nResolution: " << oculus.hmdDesc.Resolution.w << " x " << oculus.hmdDesc.Resolution.h
00640     << "\nDefault Fov: (" << oculus.hmdDesc.DefaultEyeFov[ovrEye_Left].LeftTan
00641     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Left].DownTan
00642     << ") - (" << oculus.hmdDesc.DefaultEyeFov[ovrEye_Left].RightTan
00643     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Left].UpTan
00644     << ") \n"
00645     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Right].LeftTan
00646     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Right].DownTan
00647     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Right].RightTan
00648     << ", " << oculus.hmdDesc.DefaultEyeFov[ovrEye_Right].UpTan
00649     << ") \nMaximum Fov: (" << oculus.hmdDesc.MaxEyeFov[ovrEye_Left].LeftTan
00650     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Left].DownTan
00651     << ") - (" << oculus.hmdDesc.MaxEyeFov[ovrEye_Left].RightTan
00652     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Left].UpTan
00653     << ") \n"
00654     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Right].LeftTan
00655     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Right].DownTan
00656     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Right].RightTan
00657     << ", " << oculus.hmdDesc.MaxEyeFov[ovrEye_Right].UpTan
00658     << ") \n" << std::endl;
00659 # endif
00660
00661 // Oculus Rift に転送する描画データを作成する
00662 # if OVR_PRODUCT_VERSION > 0
00663 oculus.layerData.Header.Type = ovrLayerType_EyeFov;
00664 # else
00665 oculus.layerData.Header.Type = ovrLayerType_EyeFovDepth;
00666 # endif
00667
00668 // OpenGL なので左下が原点
00669 oculus.layerData.Header.Flags = ovrLayerFlag_TextureOriginAtBottomLeft;
00670
00671 // Oculus Rift のレンダリングに使う FBO を作成する
00672 glGenFramebuffers(ovrEye_Count, oculus.oculusFbo);
00673
00674 // 全ての目について
00675 for (int eye = 0; eye < ovrEye_Count; ++eye)
00676 {
00677     // Oculus Rift の視野を取得する
00678     const auto& fov{ oculus.hmdDesc.DefaultEyeFov[ovrEyeType(eye)] };
00679
00680     // Oculus Rift 用の FBO のサイズを求める
00681     const auto textureSize{ ovr_GetFovTextureSize(oculus.session, ovrEyeType(eye), fov, 1.0f) };
00682
00683     // Oculus Rift のスクリーンのサイズを保存する
00684     oculus.screen[eye][0] = -fov.LeftTan;
00685     oculus.screen[eye][1] = fov.RightTan;
00686     oculus.screen[eye][2] = -fov.DownTan;
00687     oculus.screen[eye][3] = fov.UpTan;
00688
00689 # if OVR_PRODUCT_VERSION > 0
00690

```

```

00689 // 描画データに視野を設定する
00690 oculus.layerData.Fov[eye] = fov;
00691
00692 // 描画データにビューポートを設定する
00693 oculus.layerData.Viewport[eye].Pos = OVR::Vector2i(0, 0);
00694 oculus.layerData.Viewport[eye].Size = textureSize;
00695
00696 // Oculus Rift 用の FBO のカラーバッファとして使うテクスチャセットの特性
00697 const ovrTextureSwapChainDesc colorDesc
00698 {
00699     ovrTexture_2D, // Type
00700     OVR_FORMAT_R8G8B8A8_UNORM_SRGB, // Format
00701     1, // ArraySize
00702     textureSize.w, // Width
00703     textureSize.h, // Height
00704     1, // MipLevels
00705     1, // SampleCount
00706     ovrFalse, // StaticImage
00707     0, 0
00708 };
00709
00710 // Oculus Rift 用の FBO のレンダータargetとして使うテクスチャチェーンを作成する
00711 oculus.layerData.ColorTexture[eye] = nullptr;
00712 if (OVR_SUCCESS(ovr.CreateTextureSwapChainGL(oculus.session, &colorDesc,
&oculus.layerData.ColorTexture[eye])))
00713 {
00714     // 作成したテクスチャチェーンの長さを取得する
00715     int length(0);
00716     if (OVR_SUCCESS(ovr.GetTextureSwapChainLength(oculus.session, oculus.layerData.ColorTexture[eye],
&length)))
00717     {
00718         // テクスチャチェーンの個々の要素について
00719         for (int i = 0; i < length; ++i)
00720         {
00721             // テクスチャのパラメータを設定する
00722             GLuint texId{ 0 };
00723             ovr.GetTextureSwapChainBufferGL(oculus.session, oculus.layerData.ColorTexture[eye], i,
&texId);
00724             glBindTexture(GL_TEXTURE_2D, texId);
00725             glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
00726             glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
00727             glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
00728             glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
00729         }
00730     }
00731
00732 // Oculus Rift 用の FBO のデプスバッファとして使うテクスチャを作成する
00733 glGenTextures(1, &oculus.oculusDepth[eye]);
00734 glBindTexture(GL_TEXTURE_2D, oculus.oculusDepth[eye]);
00735 glTexImage2D(GL_TEXTURE_2D, 0, GL_DEPTH_COMPONENT32F, textureSize.w, textureSize.h, 0,
GL_DEPTH_COMPONENT, GL_FLOAT, NULL);
00736 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
00737 glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
00738 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
00739 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
00740 }
00741
00742 # else
00743
00744 // 描画データに視野を設定する
00745 oculus.layerData.EyeFov.Fov[eye] = fov;
00746
00747 // 描画データにビューポートを設定する
00748 oculus.layerData.EyeFov.Viewport[eye].Pos = OVR::Vector2i(0, 0);
00749 oculus.layerData.EyeFov.Viewport[eye].Size = textureSize;
00750
00751 // Oculus Rift 用の FBO のカラーバッファとして使うテクスチャセットを作成する
00752 ovrSwapTextureSet* colorTexture{ nullptr };
00753 ovr.CreateSwapTextureSetGL(oculus.session, GL_SRGB8_ALPHA8, textureSize.w, textureSize.h,
&colorTexture);
00754 oculus.layerData.EyeFov.ColorTexture[eye] = colorTexture;
00755
00756 // Oculus Rift 用の FBO のデプスバッファとして使うテクスチャセットを作成する
00757 ovrSwapTextureSet* depthTexture{ nullptr };
00758 ovr.CreateSwapTextureSetGL(oculus.session, GL_DEPTH_COMPONENT32F, textureSize.w, textureSize.h,
&depthTexture);
00759 oculus.layerData.EyeFovDepth.DepthTexture[eye] = depthTexture;
00760
00761 // Oculus Rift のレンズ補正等の設定値を取得する
00762 oculus.eyeRenderDesc[eye] = ovr.GetRenderDesc(oculus.session, ovrEyeType(eye), fov);
00763
00764 # endif
00765 }
00766
00767 # if OVR_PRODUCT_VERSION > 0
00768
00769 // 姿勢のトラッキングにおける床の高さを 0 に設定する

```

```

00770     ovr.SetTrackingOriginType(oculus.session, ovrTrackingOrigin.FloorLevel);
00771
00772     // ミラー表示用の FBO を作成する
00773     const GLsizei* size{ oculus.window->getSize() };
00774     const ovrMirrorTextureDesc mirrorDesc
00775     {
00776         OVR_FORMAT_R8G8B8A8_UNORM_SRGB, // Format
00777         oculus.mirrorWidth = size[0],    // Width
00778         oculus.mirrorHeight = size[1],   // Height
00779         0                                // Flags
00780     };
00781
00782     // ミラー表示用の FBO のカラーバッファとして使うテクスチャを作成する
00783     if (OVR_SUCCESS(ovr.CreateMirrorTextureGL(oculus.session, &mirrorDesc, &oculus.mirrorTexture)))
00784     {
00785         // 作成したテクスチャのテクスチャ名を得る
00786         GLuint texId{ 0 };
00787         if (OVR_SUCCESS(ovr.GetMirrorTextureBufferGL(oculus.session, oculus.mirrorTexture, &texId)))
00788         {
00789             // ミラー表示用の FBO を作成してテクスチャをカラーバッファとして組み込む
00790             glGenFramebuffers(1, &oculus.mirrorFbo);
00791             glBindFramebuffer(GL_READ_FRAMEBUFFER, oculus.mirrorFbo);
00792             glFramebufferTexture2D(GL_READ_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, texId, 0);
00793             glFramebufferRenderbuffer(GL_READ_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER, 0);
00794             glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
00795         }
00796     }
00797
00798 # else
00799
00800     // 作成したテクスチャのテクスチャ名を得る
00801     if (OVR_SUCCESS(ovr.CreateMirrorTextureGL(oculus.session, GL_SRGB8_ALPHA8, width, height,
00802     reinterpret_cast<ovrTexture*>(&mirrorTexture))))
00803     {
00804         // ミラー表示用の FBO を作成してテクスチャをカラーバッファとして組み込む
00805         oculus.mirrorFbo = 0;
00806         glGenFramebuffers(1, &oculus.mirrorFbo);
00807         glBindFramebuffer(GL_READ_FRAMEBUFFER, oculus.mirrorFbo);
00808         glFramebufferTexture2D(GL_READ_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D,
00809         mirrorTexture->OGL.TexId, 0);
00810         glFramebufferRenderbuffer(GL_READ_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_RENDERBUFFER, 0);
00811         glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
00812     }
00813 # endif
00814
00815     // Oculus Rift にレンダリングするときは sRGB カラースペースを使う
00816     glEnable(GL_FRAMEBUFFER_SRGB);
00817
00818     // フロントバッファに描く
00819     glDrawBuffer(GL_FRONT);
00820
00821     // Oculus Rift への表示では垂直同期タイミングに合わせない
00822     glfwSwapInterval(0);
00823
00824     return oculus;
00825 }
00826
00827 // Oculus Rift のセッションを破棄する
00828 //
00829 void GgApp::Oculus::terminate()
00830 {
00831     // session が無効なら何もしない
00832     if (!session) return;
00833
00834     // ミラー表示用の FBO を作っていたら削除する
00835     if (mirrorFbo)
00836     {
00837         glDeleteFramebuffers(1, &mirrorFbo);
00838         mirrorFbo = 0;
00839     }
00840
00841     // ミラー表示用の FBO のカラーバッファ用のテクスチャを作っていたら削除する
00842     if (mirrorTexture)
00843     {
00844         # if OVR_PRODUCT_VERSION > 0
00845             ovr_DestroyMirrorTexture(session, mirrorTexture);
00846         # else
00847             glDeleteTextures(1, &mirrorTexture->OGL.TexId);
00848             ovr_DestroyMirrorTexture(session, reinterpret_cast<ovrTexture*>(mirrorTexture));
00849         # endif
00850         mirrorTexture = nullptr;
00851     }
00852
00853     // 全ての目について
00854     for (int eye = 0; eye < ovrEye_Count; ++eye)

```

```

00855 {
00856     // Oculus Rift へのレンダリング用の FBO を削除する
00857     glDeleteFramebuffers(1, oculusFbo + eye);
00858     oculusFbo[eye] = 0;
00859
00860 # if OVR_PRODUCT_VERSION > 0
00861
00862     // レンダリングターゲットに使ったテクスチャを削除する
00863     if (layerData.ColorTexture[eye])
00864     {
00865         ovr_DestroyTextureSwapChain(session, layerData.ColorTexture[eye]);
00866         layerData.ColorTexture[eye] = nullptr;
00867     }
00868
00869     // デプスバッファとして使ったテクスチャを削除する
00870     glDeleteTextures(1, oculusDepth + eye);
00871     oculusDepth[eye] = 0;
00872
00873 # else
00874
00875     // レンダリングターゲットに使ったテクスチャを削除する
00876     auto* const colorTexture(layerData.EyeFov.ColorTexture[eye]);
00877     for (int i = 0; i < colorTexture->TextureCount; ++i)
00878     {
00879         const auto* const ctex(reinterpret_cast<ovrGLTexture*>(&colorTexture->Textures[i]));
00880         glDeleteTextures(1, &ctex->OGL.TexId);
00881     }
00882     ovr_DestroySwapTextureSet(session, colorTexture);
00883
00884     // デプスバッファとして使ったテクスチャを削除する
00885     auto* const depthTexture(layerData.EyeFovDepth.DepthTexture[eye]);
00886     for (int i = 0; i < depthTexture->TextureCount; ++i)
00887     {
00888         const auto* const dtex(reinterpret_cast<ovrGLTexture*>(&depthTexture->Textures[i]));
00889         glDeleteTextures(1, &dtex->OGL.TexId);
00890     }
00891     ovr_DestroySwapTextureSet(session, depthTexture);
00892
00893 # endif
00894 }
00895
00896 // Oculus Rift のセッションを破棄する
00897 ovr_Destroy(session);
00898 session = nullptr;
00899
00900 // カラースペースを元に戻す
00901 glDisable(GL_FRAMEBUFFER_SRGB);
00902
00903 // バックバッファに描く
00904 glDrawBuffer(GL_BACK);
00905
00906 // 垂直同期タイミングに合わせる
00907 glfwSwapInterval(1);
00908 }
00909
00910 //
00911 // Oculus Rift による描画開始
00912 //
00913 bool GgApp::Oculus::begin()
00914 {
00915 # if OVR_PRODUCT_VERSION > 0
00916
00917     // セッションの状態を取得する
00918     ovrSessionStatus sessionStatus;
00919     ovr_GetSessionStatus(session, &sessionStatus);
00920
00921     // アプリケーションが終了を要求しているときはウィンドウのクローズフラグを立てる
00922     if (sessionStatus.ShouldQuit) window->setClose(GLFW_TRUE);
00923
00924     // Oculus Rift に表示されていないときは戻る
00925     if (!sessionStatus.IsVisible) return false;
00926
00927     // 現在の状態をトラッキングの原点にする
00928     if (sessionStatus.ShouldRecenter) ovr_RecenterTrackingOrigin(session);
00929
00930     // HmdToEyeOffset などは実行時に変化するので毎フレーム ovr_GetRenderDesc() で ovrEyeRenderDesc を取得する
00931     const ovrEyeRenderDesc eyeRenderDesc[]
00932     {
00933         ovr_GetRenderDesc(session, ovrEyeType(0), hmdDesc.DefaultEyeFov[0]),
00934         ovr_GetRenderDesc(session, ovrEyeType(1), hmdDesc.DefaultEyeFov[1])
00935     };
00936
00937     // Oculus Rift のスクリーンのヘッドトラッキング位置からの変位を取得する
00938     const ovrPosef hmdToEyePose[]
00939     {
00940         eyeRenderDesc[0].HmdToEyePose,
00941         eyeRenderDesc[1].HmdToEyePose
00942     };

```



```

00942     };
00943
00944     // 視点の姿勢情報を取得する
00945     ovr_GetEyePoses(session, frameIndex, ovrTrue, hmdToEyePose, layerData.RenderPose,
&layerData.SensorSampleTime);
00946
00947 #   else
00948
00949     // フレームのタイミング計測開始
00950     const auto ftiming(ovr_GetPredictedDisplayTime(session, 0));
00951
00952     // sensorSampleTime の取得は可能な限り ovr_GetTrackingState() の近くで行う
00953     layerData.EyeFov.SensorSampleTime = ovr_GetTimeInSeconds();
00954
00955     // ヘッドトラッキングの状態を取得する
00956     const auto hmdState(ovr_GetTrackingState(session, ftiming, ovrTrue));
00957
00958     // Oculus Rift のスクリーンのヘッドトラッキング位置からの変位を取得する
00959     const ovrVector3f hmdToEyeViewOffset[]
00960     {
00961         eyeRenderDesc[0].HmdToEyeViewOffset,
00962         eyeRenderDesc[1].HmdToEyeViewOffset
00963     };
00964
00965     // 視点の姿勢情報を求める
00966     ovr_CalcEyePoses(hmdState.HeadPose.ThePose, hmdToEyeViewOffset, eyePose);
00967
00968 #   endif
00969
00970     return true;
00971 }
00972
00973 //
00974 // Oculus Rift の描画する目の指定
00975 //
00976 void GgApp::Oculus::select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation)
00977 {
00978     #   if OVR_PRODUCT_VERSION > 0
00979
00980     // Oculus Rift にレンダリングする FBO に切り替える
00981     if (layerData.ColorTexture[eye])
00982     {
00983         // FBO のカラーバッファに使う現在のテクスチャのインデックスを取得する
00984         int curIndex;
00985         ovr_GetTextureSwapChainCurrentIndex(session, layerData.ColorTexture[eye], &curIndex);
00986
00987         // FBO のカラーバッファに使うテクスチャを取得する
00988         GLuint curTexId;
00989         ovr_GetTextureSwapChainBufferGL(session, layerData.ColorTexture[eye], curIndex, &curTexId);
00990
00991         // FBO を設定する
00992         glBindFramebuffer(GL_FRAMEBUFFER, oculusFbo[eye]);
00993         glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, curTexId, 0);
00994         glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, oculusDepth[eye], 0);
00995
00996         // ビューポートを設定する
00997         const auto& vp{ layerData.Viewport[eye] };
00998         glViewport(vp.Pos.x, vp.Pos.y, vp.Size.w, vp.Size.h);
00999     }
01000
01001     // Oculus Rift の片目の位置と回転を取得する
01002     const auto& p{ layerData.RenderPose[eye].Position };
01003     const auto& o{ layerData.RenderPose[eye].Orientation };
01004
01005 #   else
01006
01007     // レンダーターゲットに描画する前にレンダーターゲットのインデックスをインクリメントする
01008     auto* const colorTexture{ layerData.EyeFov.ColorTexture[eye] };
01009     colorTexture->CurrentIndex = (colorTexture->CurrentIndex + 1) % colorTexture->TextureCount;
01010     auto* const depthTexture{ layerData.EyeFovDepth.DepthTexture[eye] };
01011     depthTexture->CurrentIndex = (depthTexture->CurrentIndex + 1) % depthTexture->TextureCount;
01012
01013     // レンダーターゲットを切り替える
01014     glBindFramebuffer(GL_FRAMEBUFFER, oculusFbo[eye]);
01015     const auto& ctex{
reinterpret_cast<ovrGLTexture*>(&colorTexture->Textures[colorTexture->CurrentIndex]) };
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, ctex->OGL.TexId, 0);
01016     const auto& dtex{
reinterpret_cast<ovrGLTexture*>(&depthTexture->Textures[depthTexture->CurrentIndex]) };
glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, dtex->OGL.TexId, 0);
01017
01018     // ビューポートを設定する
01019     const auto& vp{ layerData.EyeFov.Viewport[eye] };
01020     glViewport(vp.Pos.x, vp.Pos.y, vp.Size.w, vp.Size.h);
01021
01022     // Oculus Rift の片目の位置と回転を取得する
01023     const auto& p{ eyePose[eye].Position };

```

```

01026     const auto& o{ eyePose[eye].Orientation };
01027
01028 #   endif
01029
01030 // Oculus Rift のスクリーンの大きさを返す
01031 screen[0] = this->screen[eye][0];
01032 screen[1] = this->screen[eye][1];
01033 screen[2] = this->screen[eye][2];
01034 screen[3] = this->screen[eye][3];
01035
01036 // Oculus Rift の位置を返す
01037 position[0] = p.x;
01038 position[1] = p.y;
01039 position[2] = p.z;
01040
01041 // Oculus Rift の方向を返す
01042 orientation[0] = o.x;
01043 orientation[1] = o.y;
01044 orientation[2] = o.z;
01045 orientation[3] = o.w;
01046 }
01047
01048 //
01049 // Time Warp 処理に使う投影変換行列の成分の設定 (DK1, DK2)
01050 //
01051 void GgApp::Oculus::timewarp(const GgMatrix& projection)
01052 {
01053 #   if OVR_PRODUCT_VERSION < 1
01054     // TimeWarp に使う変換行列の成分を設定する
01055     auto& posTimewarpProjectionDesc{ layerData.EyeFovDepth.ProjectionDesc };
01056     posTimewarpProjectionDesc.Projection22 = (projection.get()[4 * 2 + 2] + projection.get()[4 * 3 + 2])
01057 * 0.5f;
01058     posTimewarpProjectionDesc.Projection23 = projection.get()[4 * 2 + 3] * 0.5f;
01059     posTimewarpProjectionDesc.Projection32 = projection.get()[4 * 3 + 2];
01060 #   endif
01061 }
01062 //
01063 // 図形の描画を完了する (CV1 以降)
01064 //
01065 void GgApp::Oculus::commit(int eye)
01066 {
01067 #   if OVR_PRODUCT_VERSION > 0
01068     // GL_COLOR_ATTACHMENT0 に割り当てられたテクスチャが wglDXUnlockObjectsNV() によって
01069     // アンロックされるために次のフレームの処理において無効な GL_COLOR_ATTACHMENT0 が
01070     // FBO に結合されるのを避ける
01071     glBindFramebuffer(GL_FRAMEBUFFER, oculusFbo[eye]);
01072     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, 0, 0);
01073     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_DEPTH_ATTACHMENT, GL_TEXTURE_2D, 0, 0);
01074
01075     // 保留中の変更を layerData.ColorTexture[eye] に反映しインデックスを更新する
01076     ovrCommitTextureSwapChain(session, layerData.ColorTexture[eye]);
01077 #   endif
01078 }
01079
01080 //
01081 // フレームを転送する
01082 //
01083 bool GgApp::Oculus::submit(bool mirror)
01084 {
01085     // エラーチェック
01086     ggError();
01087
01088 #   if OVR_PRODUCT_VERSION > 0
01089     // 描画データを Oculus Rift に転送する
01090     const auto* const layers{ &layerData.Header };
01091     if (OVR_FAILURE(ovrSubmitFrame(session, frameIndex++, nullptr, &layers, 1))) return false;
01092 #   else
01093     // Oculus Rift 上の描画位置と拡大率を求める
01094     ovrViewScaleDesc viewScaleDesc;
01095     viewScaleDesc.HmdSpaceToWorldScaleInMeters = 1.0f;
01096     viewScaleDesc.HmdToEyeViewOffset[0] = eyeRenderDesc[0].HmdToEyeViewOffset;
01097     viewScaleDesc.HmdToEyeViewOffset[1] = eyeRenderDesc[1].HmdToEyeViewOffset;
01098
01099     // 描画データを更新する
01100     layerData.EyeFov.RenderPose[0] = eyePose[0];
01101     layerData.EyeFov.RenderPose[1] = eyePose[1];
01102
01103     // 描画データを Oculus Rift に転送する
01104     const auto* const layers{ &layerData.Header };
01105     if (OVR_FAILURE(ovrSubmitFrame(session, 0, &viewScaleDesc, &layers, 1))) return false;
01106 #   endif
01107
01108     // ミラー表示
01109     if (mirror)
01110     {
01111 #   if OVR_PRODUCT_VERSION > 0

```

```

01112     const auto& sx1{ mirrorWidth };
01113     const auto& syl{ mirrorHeight };
01114 # else
01115     const auto& sx1{ mirrorTexture->OGL.Header.TextureSize.w };
01116     const auto& syl{ mirrorTexture->OGL.Header.TextureSize.h };
01117 # endif
01118
01119     // ミラー表示のウィンドウのサイズ
01120     GLsizei size[2];
01121     window->getSize(size);
01122
01123     // ミラー表示の表示領域
01124     GLint dx0{ 0 }, dx1{ size[0] }, dy0{ 0 }, dy1{ size[1] };
01125
01126     // ミラー表示がウィンドウからはみ出ないようにする
01127     if ((size[0] * sx1) < (size[1] * syl))
01128     {
01129         const GLint ty1{ size[0] / sx1 };
01130         dy0 = (dy1 - ty1) / 2;
01131         dy1 = dy0 + ty1;
01132     }
01133     else
01134     {
01135         const GLint tx1{ size[1] / syl };
01136         dx0 = (dx1 - tx1) / 2;
01137         dx1 = dx0 + tx1;
01138     }
01139
01140     // レンダリング結果をミラー表示用のフレームバッファにも転送する
01141     glBindFramebuffer(GL_READ_FRAMEBUFFER, mirrorFbo);
01142     glBindFramebuffer(GL_DRAW_FRAMEBUFFER, 0);
01143     glBlitFramebuffer(0, syl, sx1, 0, dx0, dy0, dx1, dy1, GL_COLOR_BUFFER_BIT, GL_NEAREST);
01144     glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
01145
01146     // 残っている OpenGL コマンドを実行する
01147     glFlush();
01148 }
01149
01150 return true;
01151 }
01152 #endif

```

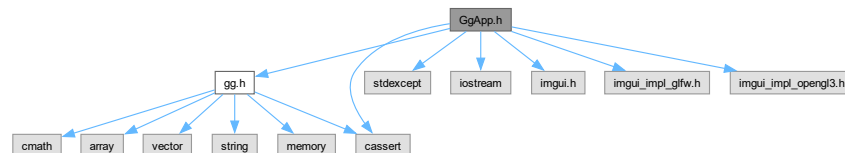
9.45 GgApp.h ファイル

```

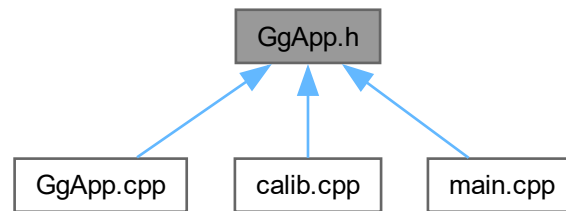
#include "gg.h"
#include <cassert>
#include <stdexcept>
#include <iostream>
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"

```

GgApp.h の依存先関係図:



被依存関係図:



クラス

- class [GgApp](#)
- class [GgApp::Window](#)

マクロ定義

- `#define` [GG_USE_IMGUI](#)
- `#define` [GG_BUTTON_COUNT](#) 3
- `#define` [GG_INTERFACE_COUNT](#) 5

9.45.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの定義.

著者

Kohe Tokoi

日付

July 17, 2025

[GgApp.h](#) に定義があります。

9.45.2 マクロ定義詳解

9.45.2.1 GG_BUTTON_COUNT

```
#define GG_BUTTON_COUNT 3
```

[GgApp.h](#) の 43 行目に定義があります。

9.45.2.2 GG_INTERFACE_COUNT

```
#define GG_INTERFACE_COUNT 5
```

[GgApp.h](#) の 48 行目に定義があります。

9.45.2.3 GG_USE_IMGUI

```
#define GG_USE_IMGUI
```

[GgApp.h](#) の 36 行目に定義があります。

9.46 GgApp.h

[詳解]

```

00001 #pragma once
00002
00003 /*
00004 ゲームグラフィックス特論用補助プログラム GLFW3 版
00005
00006 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00007
00008 Permission is hereby granted, free of charge, to any person obtaining a copy
00009 of this software and associated documentation files (the "Software"), to deal
00010 in the Software without restriction, including without limitation the rights
00011 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00012 copies or substantial portions of the Software.
00013
00014 The above copyright notice and this permission notice shall be included in
00015 all copies or substantial portions of the Software.
00016
00017 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00018 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00019 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00020 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00021 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00022 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00023
00024 */
00025
00026
00034
00035 // Dear ImGui を使うなら
00036 #define GG_USE_IMGUI
00037
00038 // Oculus Rift を使うなら
00039 // #define GG_USE_OCULUS_RIFT
00040
00041 // 使用するマウスのボタン数
00042 #if !defined(GG_BUTTON_COUNT)
00043 # define GG_BUTTON_COUNT 3
00044 #endif
00045
00046 // 使用するユーザインタフェースの数
00047 #if !defined(GG_INTERFACE_COUNT)
00048 # define GG_INTERFACE_COUNT 5
00049 #endif
00050
00051 // 補助プログラム
00052 #include "gg.h"
00053 using namespace gg;
00054
00055 // 標準ライブラリ
00056 #include <cassert>
00057 #include <stdexcept>
00058 #include <iostream>
00059
00060 // ImGui の組み込み
00061 #if defined(GG_USE_IMGUI)
00062 # include "imgui.h"
00063 # include "imgui_impl_glfw.h"
00064 # include "imgui_impl_opengl3.h"
00065 #endif
00066
00067 // Oculus Rift SDK ライブラリ (LibOVR) の組み込み
00068 #if defined(GG_USE_OCULUS_RIFT)
00069 # if defined(_WIN32)
00070 #   define GLFW_EXPOSE_NATIVE_WIN32
00071 #   define GLFW_EXPOSE_NATIVE_WGL
00072 #   include <GLFW/glfw3native.h>
00073 #   define OVR_OS_WIN32
00074 #   undef APIENTRY
00075 #   pragma comment(lib, "LibOVR.lib")
00076 #   endif
00077 #   include <OVR_CAPI_GL.h>
00078 #   include <Extras/OVRMath.h>
00079 #   if OVR_PRODUCT_VERSION > 0
00080 #     include <dxgi.h> // GetDefaultAdapterLuid のため
00081 #     pragma comment(lib, "dxgi.lib")
00082 #   endif
00083 #endif
00084
00088 class GgApp
00089 {
00090 public:
00091
00098     GgApp(int major = 0, int minor = 1);
00099

```

```

00105 GgApp(const GgApp& w) = delete;
00106
00112 GgApp(GgApp&& w) = default;
00113
00117 virtual ~GgApp();
00118
00125 GgApp& operator=(const GgApp& w) = delete;
00126
00133 GgApp& operator=(GgApp&& w) = default;
00134
00142 int main(int argc, const char* const* argv);
00143
00150 class Window
00151 {
00152     // ウィンドウの識別子
00153     GLFWwindow* window;
00154
00155     // ビューポートの横幅と高さ
00156     std::array<GLsizei, 2> size;
00157
00158     // フレームバッファの横幅と高さ
00159     std::array<GLsizei, 2> fboSize;
00160
00161     #if defined(IMGUI_VERSION)
00162     // メニューバーの高さ
00163     GLsizei menubarHeight;
00164     #endif
00165
00166     // ビューポートの縦横比
00167     GLfloat aspect;
00168
00169     // マウスの移動速度[X/Y/Z]
00170     std::array<GLfloat, 3> velocity;
00171
00172     // マウスボタンの状態
00173     std::array<bool, GG_BUTTON_COUNT> status;
00174
00175     // ユーザインタフェースのデータ構造
00176     struct HumanInterface
00177     {
00178         // 最後にタイプしたキー
00179         int lastKey;
00180
00181         // 矢印キー
00182         std::array<std::array<int, 2>, 4> arrow;
00183
00184         // マウスの現在位置
00185         std::array<GLfloat, 2> mouse;
00186
00187         // マウスホイールの回転量
00188         std::array<GLfloat, 2> wheel;
00189
00190         // 平行移動量[ボタン][直前/更新][X/Y/Z]
00191         std::array<std::array<GgVector, 2>, GG_BUTTON_COUNT> translation;
00192
00193         // トラックボール
00194         std::array<GgTrackball, GG_BUTTON_COUNT> rotation;
00195
00196         // コンストラクタ
00197         HumanInterface() :
00198             lastKey{ 0 },
00199             arrow{},
00200             mouse{},
00201             wheel{},
00202             translation{}
00203         {
00204             resetTranslation();
00205         }
00206
00207         //
00208         // マウスや矢印キーによる平行移動量を初期化する
00209         //
00210         void resetTranslation();
00211
00212         //
00213         // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00214         //
00215         void calcTranslation(int button, const std::array<GLfloat, 3>& velocity);
00216     };
00217
00218     // ヒューマンインタフェースデバイスのデータ
00219     std::array<HumanInterface, GG_INTERFACE_COUNT> interfaceData;
00220
00221     // ヒューマンインタフェースデバイスの番号
00222     int interfaceNo;
00223
00224     //

```

```

00225 // ユーザー定義のコールバック関数へのポインタ
00226 //
00227 void* userPointer;
00228 void (*resizeFunc)(const Window* window, int width, int height);
00229 void (*keyboardFunc)(const Window* window, int key, int scancode, int action, int mods);
00230 void (*mouseFunc)(const Window* window, int button, int action, int mods);
00231 void (*wheelFunc)(const Window* window, double x, double y);
00232
00233 //
00234 // ウィンドウのサイズ変更時の処理
00235 //
00236 static void resize(GLFWwindow* window, int width, int height);
00237
00238 //
00239 // キーボードをタイプした時の処理
00240 //
00241 static void keyboard(GLFWwindow* window, int key, int scancode, int action, int mods);
00242
00243 //
00244 // マウスボタンを操作したときの処理
00245 //
00246 static void mouse(GLFWwindow* window, int button, int action, int mods);
00247
00248 //
00249 // マウスホイールを操作した時の処理
00250 //
00251 static void wheel(GLFWwindow* window, double x, double y);
00252
00253 public:
00254
00255 Window(const std::string& title = "GLFW Window", int width = 640, int height = 480,
00256         int fullscreen = 0, GLFWwindow* share = nullptr);
00257
00258 Window(const Window& w) = delete;
00259
00260 Window(Window&& w) = default;
00261
00262 virtual ~Window()
00263 {
00264     // ウィンドウが作成されていなければ戻る
00265     if (!window) return;
00266
00267     // ウィンドウを破棄する
00268     glfwDestroyWindow(window);
00269 }
00270
00271 Window& operator=(const Window& w) = delete;
00272
00273 Window& operator=(Window&& w) = default;
00274
00275 auto* get() const
00276 {
00277     return window;
00278 }
00279
00280 void setClose(int flag = GLFW_TRUE) const
00281 {
00282     glfwSetWindowShouldClose(window, flag);
00283 }
00284
00285 bool shouldClose() const
00286 {
00287     // ウィンドウを閉じるべきなら true を返す
00288     return glfwWindowShouldClose(window) != GLFW_FALSE;
00289 }
00290
00291 explicit operator bool();
00292
00293 void swapBuffers() const;
00294
00295 void restoreViewport() const
00296 {
00297     if (!glfwGetWindowAttrib(window, GLFW_ICONIFIED)) glViewport(0, 0, fboSize[0], fboSize[1]);
00298 }
00299
00300 void updateViewport();
00301
00302 #if defined(IMGUI_VERSION)
00303 void setMenuBarHeight(GLsizei height)
00304 {
00305     // メニューバーの高さを保存する
00306     menubarHeight = height;
00307
00308     // ビューポートを復帰する
00309     updateViewport();
00310 }
00311 #endif

```

```

00380
00386     auto getWidth() const
00387     {
00388         return size[0];
00389     }
00390
00396     auto getHeight() const
00397     {
00398         return size[1];
00399     }
00400
00406     auto getFboWidth() const
00407     {
00408         return fboSize[0];
00409     }
00410
00416     auto getFboHeight() const
00417     {
00418         return fboSize[1];
00419     }
00420
00426     const auto& getSize() const
00427     {
00428         return size;
00429     }
00430
00436     void getSize(GLsizei* size) const
00437     {
00438         size[0] = getWidth();
00439         size[1] = getHeight();
00440     }
00441
00447     const auto& getFboSize() const
00448     {
00449         return fboSize;
00450     }
00451
00457     void getFboSize(GLsizei* fboSize) const
00458     {
00459         fboSize[0] = getFboWidth();
00460         fboSize[1] = getFboHeight();
00461     }
00462
00468     auto getAspect() const
00469     {
00470         return aspect;
00471     }
00472
00478     bool getKey(int key) const
00479     {
00480 #if defined(IMGUI_VERSION)
00481         // ImGui がキーボードを使うときはキーボードの処理を行わない
00482         if (ImGui::GetIO().WantCaptureKeyboard) return false;
00483 #endif
00484
00485         return glfwGetKey(window, key) != GLFW_RELEASE;
00486     }
00487
00493     void selectInterface(int no)
00494     {
00495         assert(static_cast<size_t>(no) < interfaceData.size());
00496         interfaceNo = no;
00497     }
00498
00506     void setVelocity(GLfloat vx, GLfloat vy, GLfloat vz = 0.1f)
00507     {
00508         velocity = std::array<GLfloat, 3>{ vx, vy, vz };
00509     }
00510
00516     int getLastKey()
00517     {
00518         auto& current_if{ interfaceData[interfaceNo] };
00519         const int key{ current_if.lastKey };
00520         current_if.lastKey = 0;
00521         return key;
00522     }
00523
00531     auto getArrow(int direction = 0, int mods = 0) const
00532     {
00533         const auto& current_if{ interfaceData[interfaceNo] };
00534         return static_cast<GLfloat>(current_if.arrow[mods & 3][direction & 1]);
00535     }
00536
00543     auto getArrowX(int mods = 0) const
00544     {
00545         return getArrow(0, mods);
00546     }

```



```

00547
00554     auto getArrowY(int mods = 0) const
00555     {
00556         return getArrow(1, mods);
00557     }
00558
00565     void getArrow(GLfloat* arrow, int mods = 0) const
00566     {
00567         arrow[0] = getArrowX(mods);
00568         arrow[1] = getArrowY(mods);
00569     }
00570
00576     auto getShiftArrowX() const
00577     {
00578         return getArrow(0, 1);
00579     }
00580
00586     auto getShiftArrowY() const
00587     {
00588         return getArrow(1, 1);
00589     }
00590
00596     void getShiftArrow(GLfloat* shift_arrow) const
00597     {
00598         shift_arrow[0] = getShiftArrowX();
00599         shift_arrow[1] = getShiftArrowY();
00600     }
00601
00607     auto getControlArrowX() const
00608     {
00609         return getArrow(0, 2);
00610     }
00611
00617     auto getControlArrowY() const
00618     {
00619         return getArrow(1, 2);
00620     }
00621
00627     void getControlArrow(GLfloat* control_arrow) const
00628     {
00629         control_arrow[0] = getControlArrowX();
00630         control_arrow[1] = getControlArrowY();
00631     }
00632
00638     auto getAltArrowX() const
00639     {
00640         return getArrow(0, 3);
00641     }
00642
00648     auto getAltArrowY() const
00649     {
00650         return getArrow(1, 3);
00651     }
00652
00658     void getAltArrow(GLfloat* alt_arrow) const
00659     {
00660         alt_arrow[0] = getAltArrowX();
00661         alt_arrow[1] = getAltArrowY();
00662     }
00663
00669     const auto* getMouse() const
00670     {
00671         const auto& current_if{ interfaceData[interfaceNo] };
00672         return current_if.mouse.data();
00673     }
00674
00680     void getMouse(GLfloat* position) const
00681     {
00682         const auto& current_if{ interfaceData[interfaceNo] };
00683         position[0] = current_if.mouse[0];
00684         position[1] = current_if.mouse[1];
00685     }
00686
00693     auto getMouse(int direction) const
00694     {
00695         const auto& current_if{ interfaceData[interfaceNo] };
00696         return current_if.mouse[direction & 1];
00697     }
00698
00704     auto getMouseX() const
00705     {
00706         const auto& current_if{ interfaceData[interfaceNo] };
00707         return current_if.mouse[0];
00708     }
00709
00715     auto getMouseY() const
00716     {

```

```

00717     const auto& current_if{ interfaceData[interfaceNo] };
00718     return current_if.mouse[1];
00719 }
00720
00726 const auto* getWheel() const
00727 {
00728     const auto& current_if{ interfaceData[interfaceNo] };
00729     return current_if.wheel.data();
00730 }
00731
00737 void getWheel(GLfloat* rotation) const
00738 {
00739     const auto& current_if{ interfaceData[interfaceNo] };
00740     rotation[0] = current_if.wheel[0];
00741     rotation[1] = current_if.wheel[1];
00742 }
00743
00750 auto getWheel(int direction) const
00751 {
00752     const auto& current_if{ interfaceData[interfaceNo] };
00753     return current_if.wheel[direction & 1];
00754 }
00755
00761 auto getWheelX() const
00762 {
00763     const auto& current_if{ interfaceData[interfaceNo] };
00764     return current_if.wheel[0];
00765 }
00766
00772 auto getWheelY() const
00773 {
00774     const auto& current_if{ interfaceData[interfaceNo] };
00775     return current_if.wheel[1];
00776 }
00777
00784 const auto& getTranslation(int button = GLFW_MOUSE_BUTTON_1) const
00785 {
00786     const auto& current_if{ interfaceData[interfaceNo] };
00787     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00788     return current_if.translation[button][1];
00789 }
00790
00797 auto getTranslationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00798 {
00799     const auto& current_if{ interfaceData[interfaceNo] };
00800     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00801     const auto& t{ current_if.translation[button][1] };
00802
00803     GgMatrix m
00804     {
00805         1.0f, 0.0f, 0.0f, 0.0f,
00806         0.0f, 1.0f, 0.0f, 0.0f,
00807         0.0f, 0.0f, 1.0f, 0.0f,
00808         t[0], t[1], t[2], 1.0f
00809     };
00810
00811     return m;
00812 }
00813
00820 auto getScrollMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00821 {
00822     const auto& current_if{ interfaceData[interfaceNo] };
00823     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00824     const auto& t{ current_if.translation[button][1] };
00825
00826     GgMatrix m
00827     {
00828         t[2] + 1.0f, 0.0f, 0.0f, 0.0f,
00829         0.0f, t[2] + 1.0f, 0.0f, 0.0f,
00830         0.0f, 0.0f, 1.0f, 0.0f,
00831         t[0], t[1], 0.0f, 1.0f
00832     };
00833
00834     return m;
00835 }
00836
00843 auto getRotation(int button = GLFW_MOUSE_BUTTON_1) const
00844 {
00845     const auto& current_if{ interfaceData[interfaceNo] };
00846     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00847     return current_if.rotation[button].getQuaternion();
00848 }
00849
00856 auto getRotationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00857 {
00858     const auto& current_if{ interfaceData[interfaceNo] };
00859     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);

```

```

00860     return current.if.rotation[button].getMatrix();
00861 }
00862
00866 void resetRotation()
00867 {
00868     // トラックボールを初期化する
00869     for (auto& tb : interfaceData[interfaceNo].rotation) tb.reset();
00870 }
00871
00875 void resetTranslation()
00876 {
00877     // 現在のインターフェースの平行移動量を初期化する
00878     interfaceData[interfaceNo].resetTranslation();
00879 }
00880
00884 void reset()
00885 {
00886     // トラックボール処理を初期化する
00887     resetRotation();
00888
00889     // 平行移動量を初期化する
00890     resetTranslation();
00891 }
00892
00898 void* getUserPointer() const
00899 {
00900     return userPointer;
00901 }
00902
00908 void setUserPointer(void* pointer)
00909 {
00910     userPointer = pointer;
00911 }
00912
00918 void setResizeFunc(void (*func)(const Window* window, int width, int height))
00919 {
00920     resizeFunc = func;
00921 }
00922
00928 void setKeyboardFunc(void (*func)(const Window* window, int key, int scancode, int action, int
00929 mods))
00930 {
00931     keyboardFunc = func;
00932 }
00933
00938 void setMouseFunc(void (*func)(const Window* window, int button, int action, int mods))
00939 {
00940     mouseFunc = func;
00941 }
00942
00948 void setWheelFunc(void (*func)(const Window* window, double x, double y))
00949 {
00950     wheelFunc = func;
00951 }
00952 };
00953
00954 #if defined(GG_USE_OCULUS_RIFT)
00961 class Oculus
00962 {
00963     // Oculus Rift のセッション
00964     ovrSession session;
00965
00966     // Oculus Rift の状態
00967     ovrHmdDesc hmdDesc;
00968
00969     // Oculus Rift へのレンダリングに使う FBO
00970     GLuint oculusFbo[ovrEye_Count];
00971
00972     // Oculus Rift のスクリーンのサイズ
00973     GLfloat screen[ovrEye_Count][4];
00974
00975     // ミラー表示用の FBO
00976     GLuint mirrorFbo;
00977
00978     // Oculus Rift のミラー表示を行うウィンドウ
00979     const Window* window;
00980
00981 # if OVR_PRODUCT_VERSION > 0
00982
00983     // Oculus Rift に送る描画データ
00984     ovrLayerEyeFov layerData;
00985
00986     // Oculus Rift にレンダリングするフレームの番号
00987     long long frameIndex;
00988
00989     // Oculus Rift へのレンダリングに使う FBO のデプステクスチャ
00990     GLuint oculusDepth[ovrEye_Count];

```

```

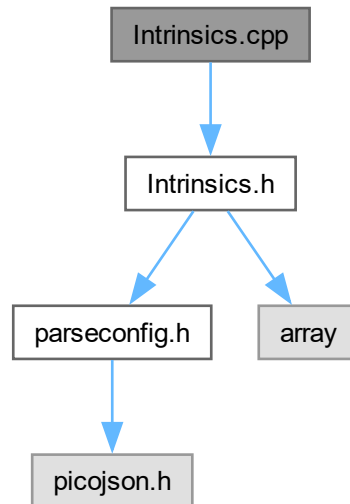
00991
00992 // ミラー表示用の FBO のサイズ
00993 int mirrorWidth, mirrorHeight;
00994
00995 // ミラー表示用の FBO のカラーテクスチャ
00996 ovrMirrorTexture mirrorTexture;
00997
00998 //
00999 // グラフィックスカードのデフォルトの LUID を得る
01000 //
01001 static ovrGraphicsLuid GetDefaultAdapterLuid();
01002
01003 //
01004 // グラフィックスカードの LUID の比較
01005 //
01006 static int Compare(const ovrGraphicsLuid& lhs, const ovrGraphicsLuid& rhs);
01007
01008 # else
01009
01010 // Oculus Rift に送る描画データ
01011 ovrLayerUnion layerData;
01012
01013 // Oculus Rift のレンダリング情報
01014 ovrEyeRenderDesc eyeRenderDesc[ovrEye_Count];
01015
01016 // Oculus Rift の視点情報
01017 ovrPosef eyePose[ovrEye_Count];
01018
01019 // ミラー表示用の FBO のカラーテクスチャ
01020 ovrGLTexture* mirrorTexture;
01021
01022 # endif
01023
01024 //
01025 // コンストラクタ
01026 //
01027 Oculus();
01028
01029 //
01030 // デストラクタ
01031 //
01032 virtual ~Oculus() = default;
01033
01034 public:
01035
01036 // シングルトンなのでコピー・ムーブ禁止
01037 Oculus(const Oculus&) = delete;
01038 Oculus& operator=(const Oculus&) = delete;
01039 Oculus(Oculus&&) = delete;
01040 Oculus& operator=(Oculus&&) = delete;
01041
01042 static Oculus& initialize(const Window& window);
01043
01044 void terminate();
01045
01046 bool begin();
01047
01048 void select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation);
01049
01050 void timewarp(const GgMatrix& projection);
01051
01052 void commit(int eye);
01053
01054 bool submit(bool mirror = true);
01055 };
01056 #endif
01057 };

```

9.47 Intrinsic.cpp ファイル

```
#include "Intrinsic.h"
```

Intrinsic.cpp の依存先関係図:



9.47.1 詳解

キャプチャデバイス固有のパラメータクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Intrinsic.cpp](#) に定義があります。

9.48 Intrinsic.cpp

[詳解]

```
00001
00008 #include "Intrinsic.h"
00009
00010 //
00011 // 構成ファイルを読み込むときに使う
00012 // キャプチャデバイス固有のパラメータの構造体のコンストラクタ
00013 //
00014 Intrinsic::Intrinsic(const picojson::object& object)
00015 {
00016     // キャプチャデバイスの画角
00017     getValue(object, "fov", fov);
00018
00019     // キャプチャデバイスの中心位置
00020     getValue(object, "center", center);
00021
00022     // キャプチャデバイスの解像度
00023     getValue(object, "size", size);
00024 }
```

```

00025 // キャプチャデバイスの周波数
00026 getValue(object, "fps", fps);
00027 }
00028 //
00029 // スクリーンの対角線長をもとにしてキャプチャデバイスのレンズの縦横の画角を変更する
00030 //
00031 //
00032 void Intrinsic::setFov(float focal)
00033 {
00034 // 撮像面の画素数の対角線長×2
00035 const auto d{ hypotf(static_cast<float>(size[0]), static_cast<float>(size[1])) * 2.0f };
00036 // 撮像面の画素数の対角線長に対するスクリーンの幅と高さの比の 1/2 (d は対角線長の2倍だから)
00037 const auto w{ size[0] / d };
00038 const auto h{ size[1] / d };
00039 // 焦点距離が1のときの撮像面の対角線長
00040 const auto t{ sensorSize / focal };
00041 // 焦点距離が1のときの撮像面の対角線長をこの比で横と縦に振り分ける
00042 const auto u{ t * w };
00043 const auto v{ t * h };
00044 // 画角を求め単位を度に換算して設定する
00045 constexpr auto s{ 360.0f / 3.14159265359f };
00046 setFov(atan(u) * s, atan(v) * s);
00047 }

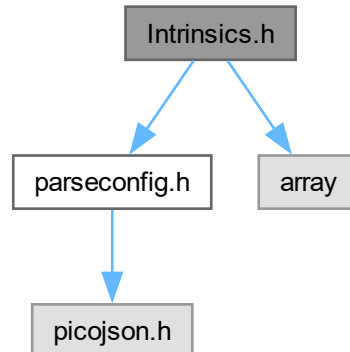
```

9.49 Intrinsic.h ファイル

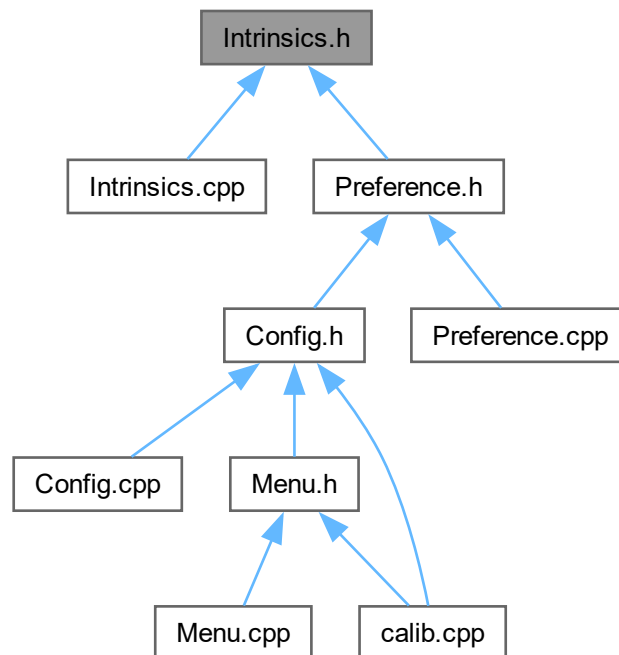
```
#include "parseconfig.h"
```

```
#include <array>
```

Intrinsic.h の依存先関係図:



被依存関係図:



クラス

- struct [Intrinsics](#)

9.49.1 詳解

キャプチャデバイス固有のパラメータクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Intrinsics.h](#) に定義があります。

9.50 Intrinsics.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 構成ファイルの読み取り補助
00012 #include "parseconfig.h"
00013
00014 // 標準ライブラリ
00015 #include <array>
00016
00020 struct Intrinsics

```

```

00021 {
00023     std::array<float, 2> fov;
00024
00026     std::array<float, 2> center;
00027
00029     std::array<int, 2> size;
00030
00032     double fps;
00033
00039     Intrinsic()
00040     : Intrinsic{ { 50.03f, 38.58f }, { 0.0f, 0.0f }, { 640, 480 }, 30.0 }
00041     {
00042     }
00043
00053     Intrinsic(const std::array<float, 2>& fov, const std::array<float, 2>& center,
00054               const std::array<int, 2> size, double fps)
00055     : fov{ fov }
00056       , center{ center }
00057       , size{ size }
00058       , fps{ fps }
00059     {
00060     }
00061
00066     Intrinsic(const picojson::object& object);
00067
00069     static constexpr auto sensorSize{ 35.0f };
00070
00076     void setFov(float focal);
00077
00084     void setFov(float fovx, float fovy)
00085     {
00086         // キャプチャデバイスのレンズの縦横の画角を設定する
00087         fov[0] = fovx;
00088         fov[1] = fovy;
00089     }
00090
00097     void setCenter(float x, float y)
00098     {
00099         // キャプチャデバイスのレンズの中心の位置を設定する
00100         center[0] = x;
00101         center[1] = y;
00102     }
00103
00104
00111     void setSize(int width, int height)
00112     {
00113         // 装置の解像度を設定する
00114         size[0] = width;
00115         size[1] = height;
00116     }
00117
00118
00124     void setFps(double frequency)
00125     {
00126         // 装置のフレームレートを設定する
00127         fps = frequency;
00128     }
00129 };

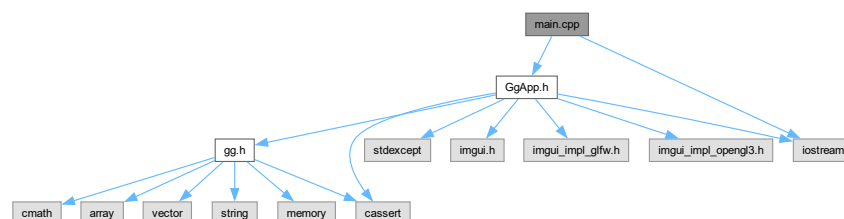
```

9.51 main.cpp ファイル

```

#include "GgApp.h"
#include <iostream>
main.cpp の依存先関係図:

```



マクロ定義

- `#define HEADER_STR "ゲームグラフィックス特論"`

関数

- `int main (int argc, const char *const *argv)`

9.51.1 詳解

ゲームグラフィックス特論宿題アプリケーション

著者

Kohe Tokoi

日付

February 20, 2024

`main.cpp` に定義があります。

9.51.2 マクロ定義詳解

9.51.2.1 HEADER_STR

`#define HEADER_STR "ゲームグラフィックス特論"`
`main.cpp` の 18 行目に定義があります。

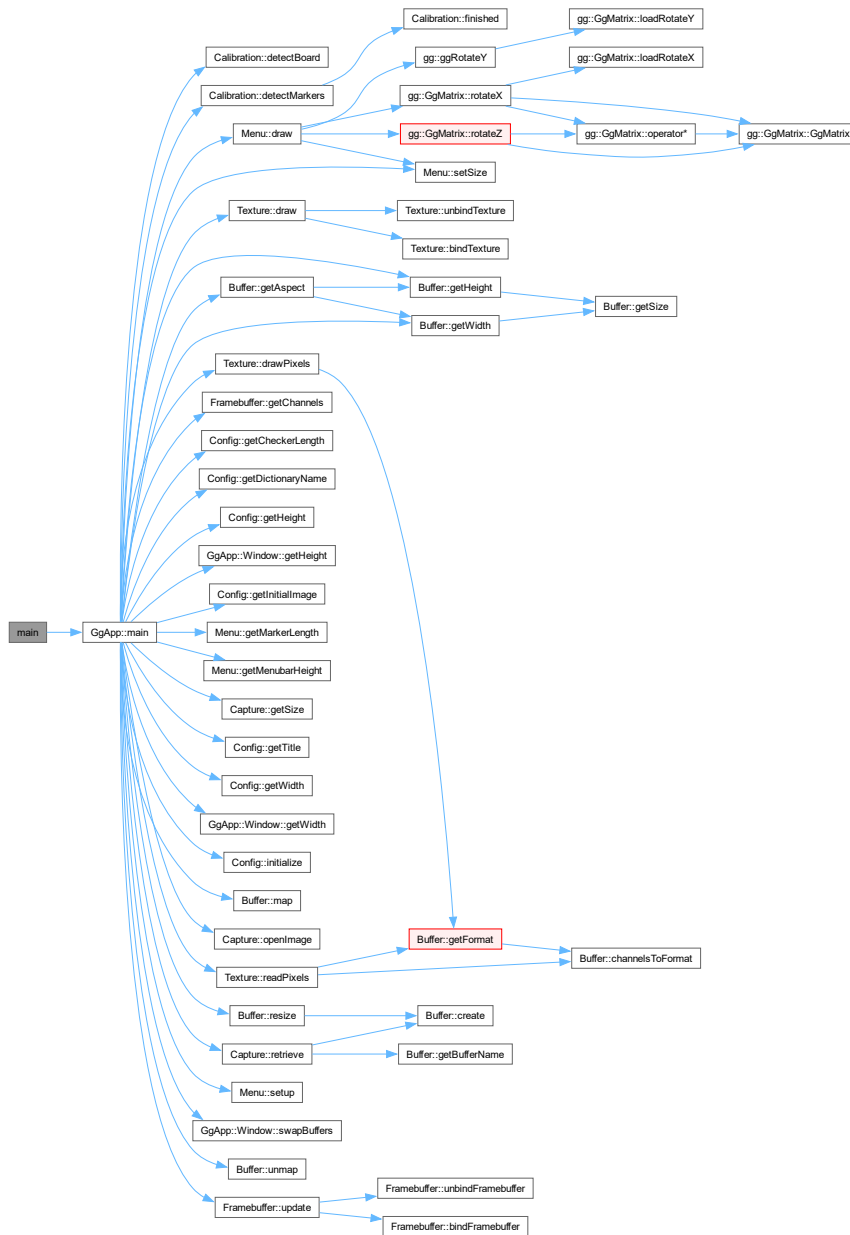
9.51.3 関数詳解

9.51.3.1 main()

```
int main (  
    int argc,  
    const char *const * argv)
```

`main.cpp` の 23 行目に定義があります。

呼び出し関係図:



9.52 main.cpp

[詳解]

```

00001
00008 #include "GgApp.h"
00009
00010 // MessageBox の準備
00011 #if defined(_WIN32)
00012 # include <atlstr.h>
00013 #elif defined(__APPLE__)
00014 # include <CoreFoundation/CoreFoundation.h>
00015 #else
00016 # include <iostream>
00017 #endif
00018 #define HEADER_STR "ゲームグラフィックス特論"
00019
00020 //

```

```

00021 // メインプログラム
00022 //
00023 int main(int argc, const char* const* argv) try
00024 {
00025     // アプリケーションのオブジェクトを生成する
00026     #if defined(GL_GLES_PROTOTYPES)
00027         GgApp app(3, 1);
00028     #else
00029         GgApp app(4, 1);
00030     #endif
00031
00032     // アプリケーションを実行する
00033     return app.main(argc, argv);
00034 }
00035 catch (const std::runtime_error &e)
00036 {
00037     // エラーメッセージを表示する
00038     #if defined(WIN32)
00039         MessageBox(NULL, CString(e.what()), TEXT(HEADER_STR), MB_ICONERROR);
00040     #elif defined(__APPLE__)
00041         // the following code is copied from
00042         // http://blog.jorgearimany.com/2010/05/messagebox-from-windows-to-mac.html
00043         // convert the strings from char* to CFStringRef
00044         CFStringRef msg_ref = CFStringCreateWithCString(NULL, e.what(), kCFStringEncodingUTF8);
00045         // result code from the message box
00046         CFOptionFlags result;
00047
00048         // launch the message box
00049         CFUserNotificationDisplayAlert(
00050             0, // no timeout
00051             kCFUserNotificationNoteAlertLevel, // change it depending message-type flags ( MB_ICONASTERISK....
00052             NULL, // icon url, use default, you can change it depending
00053             message_type_flags, // not used
00054             NULL, // localization of strings
00055             CFSTR(HEADER_STR), // header text
00056             msg_ref, // message text
00057             NULL, // default "ok" text in button
00058             NULL, // alternate button title
00059             NULL, // other button title, null--> no other button
00060             &result // response flags
00061         );
00062
00063         // Clean up the strings
00064         CFRelease(msg_ref);
00065     #else
00066         std::cerr << HEADER_STR << ": " << e.what() << '\n';
00067     #endif
00068
00069     // プログラムを終了する
00070     return EXIT_FAILURE;
00071 }

```

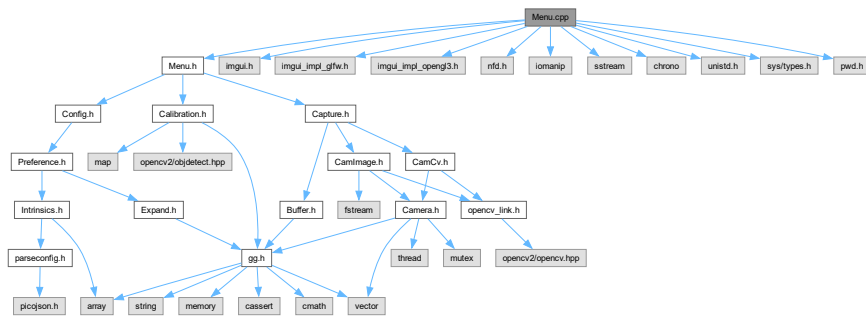
9.53 Menu.cpp ファイル

```

#include "Menu.h"
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"
#include "nfd.h"
#include <iomanip>
#include <sstream>
#include <chrono>
#include <unistd.h>
#include <sys/types.h>
#include <pwd.h>

```

Menu.cpp の依存先関係図:



関数

- void [getAnyList](#) (std::vector< std::string > &list)

変数

- constexpr nfdfilteritem_t [jsonFilter](#) [] { { "JSON", "json" } }
- constexpr nfdfilteritem_t [imageFilter](#) [] { { "Images", "png,jpg,jpeg,jfif,bmp,dib" } }
- constexpr nfdfilteritem_t [movieFilter](#) [] { { "Movies", "mp4,m4v,mpg,mov,avi,ogg,mkv" } }

9.53.1 詳解

メニューの描画クラスの実装

著者

Kohe Tokoi

日付

November 15, 2022

[Menu.cpp](#) に定義があります。

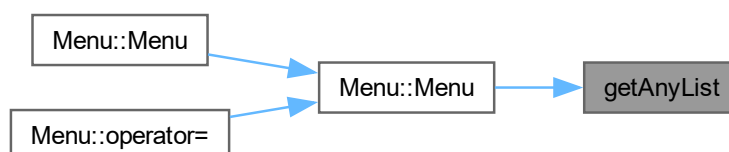
9.53.2 関数詳解

9.53.2.1 getAnyList()

```
void getAnyList (
    std::vector< std::string > & list)
```

[Menu.cpp](#) の 65 行目に定義があります。

被呼び出し関係図:



9.53.3 変数詳解

9.53.3.1 imageFilter

nfdfilteritem_t imageFilter[] { "Images", "png,jpg,jpeg,jfif,bmp,dib" } [constexpr]
Menu.cpp の 22 行目に定義があります。

9.53.3.2 jsonFilter

nfdfilteritem_t jsonFilter[] { { "JSON", "json" } } [constexpr]
Menu.cpp の 19 行目に定義があります。

9.53.3.3 movieFilter

nfdfilteritem_t movieFilter[] { "Movies", "mp4,m4v,mpg,mov,avi,ogg,mkv" } [constexpr]
Menu.cpp の 25 行目に定義があります。

9.54 Menu.cpp

[詳解]

```
00001
00008 #include "Menu.h"
00009
00010 // ImGui
00011 #include "imgui.h"
00012 #include "imgui_impl_glfw.h"
00013 #include "imgui_impl_opengl3.h"
00014
00015 // ファイルダイアログ
00016 #include "nfd.h"
00017
00018 // JSON ファイル名のフィルタ
00019 constexpr nfdfilteritem_t jsonFilter[] { { "JSON", "json" } };
00020
00021 // 画像ファイル名のフィルタ
00022 constexpr nfdfilteritem_t imageFilter[] { "Images", "png,jpg,jpeg,jfif,bmp,dib" };
00023
00024 // 動画ファイル名のフィルタ
00025 constexpr nfdfilteritem_t movieFilter[] { "Movies", "mp4,m4v,mpg,mov,avi,ogg,mkv" };
00026
00027 // 初期表示の画像ファイル名
00028 std::string Config::initialImage{ "initial.jpg" };
00029
00030 // 標準ライブラリ
00031 #include <iomanip>
00032 #include <sstream>
00033 #include <chrono>
00034
00035 #if !defined(_WIN32)
00036 // バックエンドのリスト
00037 const std::map<cv::VideoCaptureAPIs, const char*> Menu::backendList
00038 {
00039     { cv::CAP_ANY, "(any)" },
00040     #if defined(__APPLE__)
00041     { cv::CAP_AVFOUNDATION, "AV Foundation" },
00042     #endif
00043     { cv::CAP_GSTREAMER, "GStreamer" },
00044     { cv::CAP_FFMPEG, u8"動画ファイル履歴" }
00045 };
00046
00047 // コーデックのリスト
00048 const std::vector<const char*> Menu::codecList
00049 {
00050     "(any)",
00051     "MJPG",
00052     "H264",
00053     "BGR3",
00054     "YUY2",
00055     "I420",
00056     "NV12"
00057 };
00058
00059 // キャプチャデバイスのリスト
00060 std::map<cv::VideoCaptureAPIs, std::vector<std::string>> Menu::deviceList;
00061
00062 //
00063 // デフォルトのビデオデバイスの一覧を作る
00064 //
```

```

00065 void getAnyList (std::vector<std::string>& list)
00066 {
00067     list.emplace_back ("any");
00068     list.emplace_back ("Device 1");
00069     list.emplace_back ("Device 2");
00070     list.emplace_back ("Device 3");
00071     list.emplace_back ("Device 4");
00072     list.emplace_back ("Device 5");
00073     list.emplace_back ("Device 6");
00074     list.emplace_back ("Device 7");
00075 }
00076
00077 # if defined(__APPLE__)
00078 //
00079 // macOS のビデオデバイスの一覧を作る
00080 //
00081 void getAvFoundationList (std::vector<std::string>& list)
00082 {
00083     getAnyList (list);
00084 }
00085 # endif
00086
00087 // パスワードのエントリからホームディレクトリの場所を得るときに使う
00088 # include <unistd.h>
00089 # include <sys/types.h>
00090 # include <pwd.h>
00091
00092 #endif // !defined(_WIN32)
00093
00094 //
00095 // キャプチャデバイスを開く
00096 //
00097 bool Menu::openDevice ()
00098 {
00099     #if defined(_WIN32)
00100         // 何のデバイスも接続されていなければ戻る
00101         if (deviceNumber < 0) return false;
00102
00103         // キャプチャスレッドが動いていたら止める
00104         capture.stop();
00105
00106         // 前に開いていたキャプチャデバイスを閉じる
00107         capture.close();
00108
00109         // 選択したキャプチャデバイスを開く
00110         if (capture.openDevice(deviceNumber))
00111         {
00112             if (formatNumber < 0) formatNumber = 0;
00113         }
00114     #if defined(_WIN32)
00115         updateFormatDropdowns();
00116     #endif
00117
00118     // フォーマットを指定して開始できるように準備する
00119     if (capture.select(formatNumber))
00120     {
00121         // 構成データの解像度と画角を開いた画像に合わせる
00122         setSize (capture.getSize());
00123         return true;
00124     }
00125 }
00126
00127 // 開けなかった
00128 errorMessage = u8"デバイスが開けません";
00129 return false;
00130 #else
00131 // バックエンドが GStreamer なら
00132 if (backend == cv::CAP_GSTREAMER)
00133 {
00134     // バイブライン設定の取り出し
00135     const auto& pipeline{ config.gstreamerPipelines[deviceNumber] };
00136
00137     // ダイアログで指定したバイブラインが開けなかったら
00138     if (!capture.openMovie(pipeline, backend))
00139     {
00140         // 開けなかった
00141         errorMessage = u8"バイブラインが開けません";
00142         return false;
00143     }
00144
00145     // GStreamer が使える
00146     return true;
00147 }
00148
00149 // コーデック
00150 char codec[5]{};
00151 if (codecNumber > 0) strncpy(codec, codecList[codecNumber], 5);

```

```

00152
00153 // ダイアログで指定したキャプチャデバイスが開けなかったら
00154 if (!capture.openDevice(deviceNumber,
00155     intrinsics.size, intrinsics.fps, backend, codec))
00156 {
00157     // 開けなかった
00158     errorMessage = u8"デバイスが開けません";
00159     return false;
00160 }
00161
00162 // 使うことになったコーデックの番号を調べる
00163 for (size_t i = 0; i < codecList.size(); ++i)
00164 {
00165     if (strncmp(codec, codecList[i], 4) == 0)
00166     {
00167         // コーデックが分かった
00168         codecNumber = static_cast<int>(i);
00169         return true;
00170     }
00171 }
00172
00173 // コーデックが分からない
00174 codecNumber = 0;
00175 return true;
00176 #endif
00177 }
00178
00179 //
00180 // 画像ファイルを開く
00181 //
00182 void Menu::openImage()
00183 {
00184     // ファイルダイアログから得るパス
00185     nfdchar_t* filepath;
00186
00187     // ファイルダイアログを開く
00188     if (NFD_OpenDialog(&filepath, imageFilter, 1, NULL) == NFD_OKAY)
00189     {
00190         // スレッドが動作中なら停止する
00191         capture.stop();
00192
00193         // ダイアログで指定した画像ファイルが開けたら
00194         if (capture.openImage(filepath))
00195         {
00196             // 構成データの解像度と画角を開いた画像に合わせる
00197             setSize(capture.getSize());
00198         }
00199         else
00200         {
00201             // 開けなかった
00202             errorMessage = u8"画像ファイルが開けません";
00203         }
00204
00205         // ファイルパスの取り出しに使ったメモリを開放する
00206         NFD_FreePath(filepath);
00207     }
00208 }
00209
00210 //
00211 // 動画ファイルを開く
00212 //
00213 void Menu::openMovie()
00214 {
00215     // ファイルダイアログから得るパス
00216     nfdchar_t* filepath;
00217
00218     // ファイルダイアログを開く
00219     if (NFD_OpenDialog(&filepath, movieFilter, 1, NULL) == NFD_OKAY)
00220     {
00221         // スレッドが動作中なら停止する
00222         capture.stop();
00223
00224 #if defined(_WIN32)
00225         // ダイアログで指定した動画ファイルが開けたら
00226         if (capture.openMovie(filepath))
00227         {
00228             // 構成データの解像度と画角を開いた画像に合わせる
00229             setSize(capture.getSize());
00230         }
00231         else
00232         {
00233             errorMessage = u8"動画ファイルが開けません";
00234         }
00235 #else
00236         // 入力特性をファイルに切り替えて
00237         backend = cv::CAP_FFMPEG;
00238

```

```

00239 // ファイルのリストを取り出し
00240 const auto fileListLength{ static_cast<int>(fileHistory.size()) };
00241
00242 // ファイルのリストの各ファイルについて
00243 for (deviceNumber = 0; deviceNumber < fileListLength; ++deviceNumber)
00244 {
00245     // 選択したファイルと同じものがあればそれを選択する
00246     if (fileHistory[deviceNumber] == filepath) break;
00247 }
00248
00249 // 選択したファイルがファイルのリストの中になければ
00250 if (deviceNumber == fileListLength)
00251 {
00252     // その先頭にファイルパスを挿入して
00253     fileHistory.insert(fileHistory.begin(), filepath);
00254
00255     // そのエントリを選択する
00256     deviceNumber = 0;
00257 }
00258
00259 // ダイアログで指定した動画ファイルが開けたら
00260 if (capture.openMovie(filepath, backend))
00261 {
00262     // 構成データの解像度と画角を開いた画像に合わせる
00263     setSize(capture.getSize());
00264 }
00265 else
00266 {
00267     // 開けなかった
00268     errorMessage = u8"動画ファイルが開けません";
00269 }
00270 #endif
00271
00272 // ファイルパスの取り出しに使ったメモリを開放する
00273 NFD_FreePath(filepath);
00274 }
00275 }
00276
00277 //
00278 // 構成ファイルを読み込む
00279 //
00280 void Menu::loadConfig()
00281 {
00282     // ファイルダイアログから得るパス
00283     nfdchar_t* filepath;
00284
00285     // ファイルダイアログを開く
00286     if (NFD_OpenDialog(&filepath, jsonFilter, 1, NULL) == NFD_OKAY)
00287     {
00288         // 現在の構成を構成ファイルの内容にする
00289         if (const_cast<Config&>(config).load(filepath))
00290         {
00291             // 読み込んだ構成の数が現在の選択よりも少ないときは最初の項目の構成にする
00292             if (preferenceNumber > static_cast<int>(config.preferenceList.size()))
00293                 preferenceNumber = 0;
00294
00295             // 現在の設定に反映する
00296             settings = config.settings;
00297         }
00298         else
00299         {
00300             // 読み込めなかった
00301             errorMessage = u8"構成ファイルが読み込めません";
00302         }
00303
00304         // ファイルパスの取り出しに使ったメモリを開放する
00305         NFD_FreePath(filepath);
00306     }
00307 }
00308
00309 //
00310 // 構成ファイルを保存する
00311 //
00312 void Menu::saveConfig() const
00313 {
00314     // ファイルダイアログから得るパス
00315     nfdchar_t* filepath;
00316
00317     // ファイルダイアログを開く
00318     if (NFD_SaveDialog(&filepath, jsonFilter, 1, NULL, "*.json") == NFD_OKAY)
00319     {
00320         // 現在の設定で構成を更新する
00321         const_cast<Config&>(config).settings = settings;
00322
00323         // 現在の構成を保存する
00324         if (!config.save(filepath))
00325         {

```



```

00326         // 保存できなかった
00327         errorMessage = u8"構成ファイルが保存できません";
00328     }
00329
00330     // ファイルバスの取り出しに使ったメモリを開放する
00331     NFD_FreePath(filepath);
00332 }
00333 }
00334
00335 //
00336 // 校正ファイルを読み込む
00337 //
00338 void Menu::loadParameters() const
00339 {
00340     // ファイルダイアログから得るパス
00341     nfdchar_t* filepath;
00342
00343     // ファイルダイアログを開く
00344     if (NFD_OpenDialog(&filepath, jsonFilter, 1, NULL) == NFD_OKAY)
00345     {
00346         // 現在のキャリブレーションパラメータを校正ファイルの内容にする
00347         if (!calibration.loadParameters(filepath))
00348         {
00349             // 読み込めなかった
00350             errorMessage = u8"校正ファイルが読み込めません";
00351         }
00352
00353         // ファイルバスの取り出しに使ったメモリを開放する
00354         NFD_FreePath(filepath);
00355     }
00356 }
00357
00358 //
00359 // 校正ファイルを保存する
00360 //
00361 void Menu::saveParameters() const
00362 {
00363     // キャリブレーションが完了していなければ戻る
00364     if (!calibration.finished()) return;
00365
00366     // 現在時刻の取得
00367     const auto now{ std::chrono::system_clock::to_time_t(std::chrono::system_clock::now()) };
00368     const std::tm* const localtime{ std::localtime(&now) };
00369
00370     // 時刻を文字列に変換
00371     const auto timeString{ std::put_time(localtime, "cal_%Y%m%d_%H%M%S.json") };
00372     const auto pathString{ static_cast<std::ostringstream&&>(std::ostringstream() << timeString).str() };
00373 };
00374
00375 // ファイルダイアログから得るパス
00376 nfdchar_t* filepath;
00377
00378 // ファイルダイアログを開く
00379 if (NFD_SaveDialog(&filepath, jsonFilter, 1, NULL, pathString.c_str()) == NFD_OKAY)
00380 {
00381     // 現在のキャリブレーションパラメータを構成ファイルに保存する
00382     if (!calibration.saveParameters(filepath))
00383     {
00384         // 保存できなかった
00385         errorMessage = u8"校正ファイルが保存できません";
00386     }
00387
00388     // ファイルバスの取り出しに使ったメモリを開放する
00389     NFD_FreePath(filepath);
00390 }
00391
00392 //
00393 // 校正用の画像ファイルを取得する (複数選択)
00394 //
00395 void Menu::recordFileCorners() const
00396 {
00397     // ファイルダイアログから得るパス
00398     const nfdpathset_t* outPaths;
00399
00400     // ファイルダイアログを開く
00401     if (NFD_OpenDialogMultiple(&outPaths, imageFilter, 1, NULL) == NFD_OKAY)
00402     {
00403         // ファイルバスの一覧を得る
00404         nfdpathsetenum_t enumerator;
00405         NFD_PathSet_GetEnum(outPaths, &enumerator);
00406
00407         // ファイルバスの一覧からファイルバスを一つずつ取り出して
00408         for (nfdchar_t* path = NULL; NFD_PathSet_EnumNext(&enumerator, &path) && path;)
00409         {
00410             // 画像の読み出し
00411             CamImage image;

```

```

00412
00413 // 画像ファイルを読み出せたら
00414 if (image.open(path))
00415 {
00416     // データのコピー先
00417     cv::Mat frame;
00418
00419     // データをコピーして
00420     image.transmit(frame);
00421
00422     // ボードを検出して
00423     calibration.detectBoard(frame);
00424
00425     // コーナーを記録する
00426     calibration.recordCorners();
00427
00428     // 画像の読み出しを終わる
00429     image.close();
00430 }
00431
00432 // ファイルパスの取り出しに使ったメモリを開放する
00433 NFD.PathSet_FreePath(path);
00434 }
00435
00436 // ファイルパスの一覧に使ったメモリを開放する
00437 NFD.PathSet_FreeEnum(&enumerator);
00438
00439 // フォルダのパスに使ったメモリを開放する
00440 NFD.PathSet_Free(outPaths);
00441 }
00442 }
00443
00444 //
00445 // 校正用の ChArUco Board を作成する
00446 //
00447 void Menu::createCharuco() const
00448 {
00449     // ChArUco Board の画像を作成する
00450     cv::Mat boardImage;
00451     calibration.drawBoard(boardImage, 980, 692);
00452
00453     // ファイルに保存する
00454     saveImage(boardImage, "ChArUcoBoard.png");
00455 }
00456
00457 //
00458 // コンストラクタ
00459 //
00460 Menu::Menu(const Config& config, Capture& capture, Calibration& calibration)
00461 : config{ config }
00462 , settings{ config.settings }
00463 , capture{ capture }
00464 , calibration{ calibration }
00465 , deviceNumber{ 0 }
00466 #if defined(WIN32)
00467 , formatNumber{ 0 }
00468 , lastDeviceNumber{ -1 }
00469 #else
00470 , codecNumber{ 0 }
00471 , backend{ cv::CAP_ANY }
00472 #endif
00473 , preferenceNumber{ 0 }
00474 , pose{ ggIdentity() }
00475 , menubarHeight{ 0 }
00476 , showInputPanel{ true }
00477 , showCalibrationPanel{ true }
00478 , quit{ false }
00479 , errorMessage{ nullptr }
00480 , detectMarker{ false }
00481 , detectBoard{ false }
00482 {
00483     // ファイルダイアログ (Native File Dialog Extended) を初期化する
00484     NFD.Init();
00485
00486     // Dear ImGui の入力デバイス
00487     //io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard; // キーボードコントロールを使う
00488     //io.ConfigFlags |= ImGuiConfigFlags_NavEnableGamepad; // ゲームパッドを使う
00489
00490     // Dear ImGui のスタイル
00491     //ImGui::StyleColorsDark(); // 暗めのスタイル
00492     //ImGui::StyleColorsClassic(); // 以前のスタイル
00493
00494     // 日本語を表示できるメニューフォントを読み込む
00495     if (!ImGui::GetIO().Fonts->AddFontFromFileTTF(config.menuFont.c_str(), config.menuFontSize,
00496         nullptr, ImGui::GetIO().Fonts->GetGlyphRangesJapanese()))
00497     {
00498         // メニューフォントが読み込めなかったらエラーにする

```

```

00499     throw std::runtime_error("Cannot find any menu fonts.");
00500 }
00501
00502 #if defined(_WIN32)
00503 // 初期状態で最初のデバイスのフォーマットリストを取得しておく
00504 if (!config.getDeviceList().empty())
00505 {
00506     capture.updateFormatList(deviceNumber);
00507 }
00508 #else
00509 // バックエンドごとのキャプチャデバイスの一覧を初期化する
00510 for (auto& [api, name] : backendList)
00511 {
00512     // バックエンドごとに空のリストを追加する
00513     deviceList.emplace(api, std::vector<std::string>());
00514 }
00515
00516 // キャプチャデバイスの一覧を作る
00517 getAnyList(deviceList.at(cv::CAP_ANY));
00518 #if defined(_MSC_VER)
00519 getDirectShowList(deviceList.at(cv::CAP_DSHOW));
00520 getMediaFoundationList(deviceList.at(cv::CAP_MSMF));
00521 #elif defined(__APPLE__)
00522 getAvFoundationList(deviceList.at(cv::CAP_AVFOUNDATION));
00523 #endif
00524 #endif
00525 }
00526
00527 //
00528 // デストラクタ
00529 //
00530 Menu::~Menu()
00531 {
00532     // キャプチャスレッドが動いていたら止める
00533     capture.stop();
00534
00535     // 前に開いていたキャプチャデバイスを閉じる
00536     capture.close();
00537
00538     // ファイルダイアログ (Native File Dialog Extended) を終了する
00539     NFD_Quit();
00540 }
00541
00542 //
00543 // 解像度の初期値を設定する
00544 //
00545 void Menu::setSize(const std::array<int, 2>& size)
00546 {
00547     // 解像度の調整値を設定する
00548     intrinsics.size = size;
00549
00550     // 投影像の画角と中心位置を設定する
00551     intrinsics.setFov(settings.focal);
00552     intrinsics.setCenter(0.0f, 0.0f);
00553 }
00554
00555 #if defined(_WIN32)
00556 //
00557 // 解像度、フレームレート、コーデックの選択リストを更新する
00558 //
00559 void Menu::updateFormatDropdowns()
00560 {
00561     const auto& formatList{ capture.getFormatList() };
00562
00563     parsedFormats.clear();
00564     uniqueResolutions.clear();
00565     uniqueFpsList.clear();
00566     uniqueCodecs.clear();
00567
00568     // 各フォーマット文字列をパースする
00569     for (int i = 0; i < static_cast<int>(formatList.size()); ++i)
00570     {
00571         const std::string& fmt = formatList[i];
00572         size_t at_pos = fmt.find(" @ ");
00573         size_t fps_pos = fmt.find(" fps ");
00574         size_t close_pos = fmt.find(")");
00575         if (at_pos != std::string::npos && fps_pos != std::string::npos && close_pos != std::string::npos)
00576         {
00577             FormatInfo info;
00578             info.resolution = fmt.substr(0, at_pos);
00579             info.fps = fmt.substr(at_pos + 3, fps_pos - (at_pos + 3));
00580             info.codec = fmt.substr(fps_pos + 6, close_pos - (fps_pos + 6));
00581             info.index = i;
00582             parsedFormats.push_back(info);
00583
00584             if (std::find(uniqueResolutions.begin(), uniqueResolutions.end(), info.resolution) ==
                uniqueResolutions.end())

```

```

00585         uniqueResolutions.push_back(info.resolution);
00586         if (std::find(uniqueFpsList.begin(), uniqueFpsList.end(), info.fps) == uniqueFpsList.end())
00587             uniqueFpsList.push_back(info.fps);
00588         if (std::find(uniqueCodecs.begin(), uniqueCodecs.end(), info.codec) == uniqueCodecs.end())
00589             uniqueCodecs.push_back(info.codec);
00590     }
00591 }
00592
00593 // 現在の formatNumber のフォーマットに同期する
00594 if (formatNumber >= 0 && formatNumber < static_cast<int>(parsedFormats.size()))
00595 {
00596     currentRes = parsedFormats[formatNumber].resolution;
00597     currentFps = parsedFormats[formatNumber].fps;
00598     currentCodec = parsedFormats[formatNumber].codec;
00599 }
00600 else
00601 {
00602     currentRes.clear();
00603     currentFps.clear();
00604     currentCodec.clear();
00605 }
00606 lastDeviceNumber = deviceNumber;
00607 }
00608 #endif
00609
00610 //
00611 // シェーダを設定する
00612 //
00613 std::array<GLsizei, 2> Menu::setup(GLfloat aspect) const
00614 {
00615     // シェーダを設定する
00616     return config.preferenceList[preferenceNumber].getShader().setup(settings.samples, aspect,
00617         pose, intrinsics.fov, intrinsics.center, settings.getFocal(), config.background);
00618 }
00619
00620 //
00621 // メニューの描画
00622 //
00623 void Menu::draw()
00624 {
00625     // メインメニューバー
00626     if (ImGui::BeginMainMenuBar())
00627     {
00628         // ファイルメニュー
00629         if (ImGui::BeginMenu(u8"ファイル"))
00630         {
00631             // 画像ファイルを開く
00632             if (ImGui::MenuItem(u8"画像ファイルを開く")) openImage();
00633
00634             // 動画ファイルを開く
00635             if (ImGui::MenuItem(u8"動画ファイルを開く")) openMovie();
00636
00637             // 構成ファイルを開く
00638             if (ImGui::MenuItem(u8"構成ファイルを開く")) loadConfig();
00639
00640             // 構成ファイルを保存する
00641             if (ImGui::MenuItem(u8"構成ファイルを保存")) saveConfig();
00642
00643             // キャリブレーションパラメータファイルを開く
00644             if (ImGui::MenuItem(u8"校正ファイルを開く")) loadParameters();
00645
00646             // キャリブレーションパラメータファイルを保存する
00647             if (ImGui::MenuItem(u8"校正ファイルを保存")) saveParameters();
00648
00649             // フォルダ内の画像ファイルを使って校正する
00650             if (ImGui::MenuItem(u8"校正用画像から取得")) recordFileCorners();
00651
00652             // ChArUco Board の作成
00653             if (ImGui::MenuItem(u8"ChArUco 画像作成")) createCharuco();
00654
00655             // 終了
00656             quit = ImGui::MenuItem(u8"終了");
00657
00658             // File メニュー修了
00659             ImGui::EndMenu();
00660         }
00661
00662         // ウィンドウメニュー
00663         if (ImGui::BeginMenu(u8"ウィンドウ"))
00664         {
00665             // 入力パネルの表示
00666             ImGui::MenuItem(u8"入力", NULL, &showInputPanel);
00667
00668             // 校正パネルの表示
00669             ImGui::MenuItem(u8"校正", NULL, &showCalibrationPanel);
00670
00671             // File メニュー修了

```

```

00672     ImGui::EndMenu();
00673 }
00674
00675 // メニューバーの高さを保存しておく
00676 menubarHeight = static_cast<GLsizei>(ImGui::GetWindowHeight());
00677
00678 // メインメニューバー終了
00679 ImGui::EndMainMenuBar();
00680 }
00681
00682 // 入力パネル
00683 if (showInputPanel)
00684 {
00685     // ウィンドウの位置とサイズ
00686     ImGui::SetNextWindowPos(ImVec2(2.0f, 2.0f + menubarHeight), ImGuiCond_Once);
00687 #if defined(_WIN32)
00688     ImGui::SetNextWindowSize(ImVec2(231, 487), ImGuiCond_Once);
00689 #else
00690     ImGui::SetNextWindowSize(ImVec2(231, 517), ImGuiCond_Once);
00691 #endif
00692     ImGui::Begin(u8"入力", &showInputPanel);
00693
00694     // 投影方式の選択
00695     if (ImGui::BeginCombo(u8"投影方式", getPreference().getDescription().c_str()))
00696     {
00697         // すべての投影方式について
00698         for (int i = 0; i < static_cast<int>(config.preferenceList.size()); ++i)
00699         {
00700             // その投影方式が選択されていれば真
00701             const bool selected{ i == preferenceNumber };
00702
00703             // 投影方式を（それが現在の投影方式ならハイライトして）コンボボックスに表示する
00704             if (ImGui::Selectable(getPreference(i).getDescription().c_str(), selected))
00705             {
00706                 // 表示した投影方式が選択されていたらそれを現在の選択とする
00707                 preferenceNumber = i;
00708
00709                 // 選択した投影方式のキャプチャデバイス固有のパラメータをコピーする
00710                 intrinsics = getPreference().getIntrinsics();
00711             }
00712
00713             // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00714             if (selected) ImGui::SetItemDefaultFocus();
00715         }
00716     }
00717     ImGui::EndCombo();
00718
00719     // レンズの画角を設定する
00720     ImGui::DragFloat2(u8"画角", intrinsics.fov.data(), 0.1f, -360.0f, 360.0f, "%.2f");
00721
00722     // レンズの中心位置
00723     ImGui::DragFloat2(u8"中心", intrinsics.center.data(), 0.001f, -1.0f, 1.0f, "%.4f");
00724
00725     // キャプチャデバイス固有のパラメータを元に戻す
00726     if (ImGui::Button(u8"回復"))
00727     {
00728         // 選択した投影方式のキャプチャデバイス固有のパラメータを回復する
00729         intrinsics = getPreference().getIntrinsics();
00730     }
00731
00732     // 姿勢
00733     ImGui::SliderAngle(u8"方位", &settings.euler[1], -180.0f, 180.0f, "%.2f");
00734     ImGui::SliderAngle(u8"仰角", &settings.euler[0], -180.0f, 180.0f, "%.2f");
00735     ImGui::SliderAngle(u8"傾斜", &settings.euler[2], -180.0f, 180.0f, "%.2f");
00736     pose = ggRotateY(settings.euler[1]).rotateX(settings.euler[0]).rotateZ(settings.euler[2]);
00737
00738     // 焦点距離
00739     ImGui::SliderFloat(u8"焦点距離", &settings.focal, settings.focalRange[0], settings.focalRange[1],
00740         "%.1f");
00741
00742     // 姿勢を元に戻す
00743     if (ImGui::Button(u8"復帰"))
00744     {
00745         settings.euler = config.settings.euler;
00746         settings.focal = config.settings.focal;
00747         settings.focalRange = config.settings.focalRange;
00748     }
00749     ImGui::Separator();
00750
00751 #if defined(_WIN32)
00752     // キャプチャデバイスが存在するとき
00753     if (!config.getDeviceList().empty())
00754     {
00755         // 装置関連項目
00756         ImGui::Text("%s", u8"以下の変更は [開始] で反映します");
00757     }
00758 }

```

```

00758 // キャプチャデバイスの選択コンボボックス
00759 if (ImGui::BeginCombo(u8"装置", config.getDeviceName(deviceNumber).c_str()))
00760 {
00761     // すべてのキャプチャデバイスについて
00762     for (int i = 0; i < static_cast<int>(config.getDeviceList().size()); ++i)
00763     {
00764         // キャプチャデバイス名を (それを選択していればハイライトして) コンボボックスに表示する
00765         if (ImGui::Selectable(config.getDeviceName(i).c_str(), i == deviceNumber))
00766         {
00767             // キャプチャデバイスが変わったら
00768             if (deviceNumber != i)
00769             {
00770                 // キャプチャスレッドが動いていたら止める
00771                 capture.stop();
00772
00773                 // 前に開いていたキャプチャデバイスを閉じる
00774                 capture.close();
00775
00776                 // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
00777                 deviceNumber = i;
00778
00779                 // キャプチャデバイスが変わったので最初のビデオフォーマットを選択する
00780                 formatNumber = 0;
00781
00782                 // 開始ボタンが押されるまでは、フォーマットリストだけを一時取得して更新する
00783                 capture.updateFormatList(deviceNumber);
00784
00785                 // 選択可能なドロップダウンのリストを更新する
00786                 updateFormatDropdowns();
00787             }
00788
00789             // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00790             ImGui::SetItemDefaultFocus();
00791         }
00792     }
00793     ImGui::EndCombo();
00794 }
00795
00796 // 使用可能なビデオフォーマットの表示名のリスト
00797 const auto& formatList{ capture.getFormatList() };
00798
00799 // 使用可能なビデオフォーマットが存在するなら
00800 if (!formatList.empty())
00801 {
00802     // 必要な場合 (デバイス番号の不一致やリスト未作成時) にリストを更新する
00803     if (lastDeviceNumber != deviceNumber || parsedFormats.empty())
00804     {
00805         updateFormatDropdowns();
00806     }
00807
00808     // 1. 解像度の選択コンボボックス
00809     if (ImGui::BeginCombo(u8"解像度", currentRes.c_str()))
00810     {
00811         for (const auto& res : uniqueResolutions)
00812         {
00813             if (ImGui::Selectable(res.c_str(), currentRes == res))
00814             {
00815                 currentRes = res;
00816             }
00817         }
00818         ImGui::EndCombo();
00819     }
00820
00821     // 2. フレームレートの選択コンボボックス
00822     std::string fpsLabel = currentFps + " fps";
00823     if (ImGui::BeginCombo(u8"コマ数", fpsLabel.c_str()))
00824     {
00825         for (const auto& fpsVal : uniqueFpsList)
00826         {
00827             std::string valLabel = fpsVal + " fps";
00828             if (ImGui::Selectable(valLabel.c_str(), currentFps == fpsVal))
00829             {
00830                 currentFps = fpsVal;
00831             }
00832         }
00833         ImGui::EndCombo();
00834     }
00835
00836     // 3. コーデックの選択コンボボックス
00837     if (ImGui::BeginCombo(u8"符号化", currentCodec.c_str()))
00838     {
00839         for (const auto& cod : uniqueCodecs)
00840         {
00841             if (ImGui::Selectable(cod.c_str(), currentCodec == cod))
00842             {
00843                 currentCodec = cod;
00844             }
00845         }
00846     }

```

```

00845     }
00846     ImGui::EndCombo();
00847 }
00848
00849 // 選択された組み合わせが parsedFormats に存在するか探す
00850 int foundIndex = -1;
00851 for (const auto& info : parsedFormats)
00852 {
00853     if (info.resolution == currentRes && info.fps == currentFps && info.codec == currentCodec)
00854     {
00855         foundIndex = info.index;
00856         break;
00857     }
00858 }
00859
00860 bool formatExists = (foundIndex != -1);
00861 if (formatExists)
00862 {
00863     // 存在する場合は、必要なら formatNumber を更新する
00864     if (formatNumber != foundIndex)
00865     {
00866         capture.stop();
00867         formatNumber = foundIndex;
00868     }
00869 }
00870
00871 // キャプチャの開始と停止
00872 if (capture)
00873 {
00874     // キャプチャスレッドが動いているので止める
00875     if (ImGui::Button(u8"停止")) capture.stop();
00876     ImGui::SameLine();
00877     ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"取得中");
00878 }
00879 else
00880 {
00881     if (formatExists)
00882     {
00883         // 「開始」 ボタンをクリックしたときデバイスが選択されているとき
00884         if (ImGui::Button(u8"開始") && deviceNumber >= 0)
00885         {
00886             // もしすでにデバイスが開いていないか、画像が開かれているなら openDevice を呼ぶ
00887             if (!capture.isOpend() || capture.isImage())
00888             {
00889                 capture.openDevice(deviceNumber);
00890             }
00891
00892             // ビデオフォーマットを指定できたら
00893             if (capture.select(formatNumber))
00894             {
00895                 // 解像度を合わせる
00896                 setSize(capture.getSize());
00897                 // キャプチャスレッドを動かす
00898                 capture.start();
00899             }
00900             else
00901             {
00902                 // ビデオフォーマットが選択できなかった
00903                 errorMessage = u8"ビデオフォーマットが選択できません";
00904             }
00905         }
00906         ImGui::SameLine();
00907         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"停止中");
00908     }
00909     else
00910     {
00911         // 存在しない組み合わせの時はメッセージを表示する
00912         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"フォーマットが存在しません");
00913     }
00914 }
00915 }
00916 else
00917 {
00918     // キャプチャデバイスが開けなかった
00919     ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが開けません");
00920 }
00921 }
00922 else
00923 {
00924     // キャプチャデバイスが存在しないとき
00925     ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが見つかりません");
00926 }
00927 #else
00928 // 装置関連項目
00929 ImGui::Text("%s", u8"以下の変更は「開始」で反映します");
00930
00931 // デバイスプリファレンスを選択する

```

```

00932     if (ImGui::BeginCombo(u8"装置特性", backendList.at(backend)))
00933     {
00934         // すべての表示方式について
00935         for (auto& [apiId, apiName] : backendList)
00936         {
00937             // その表示方式が選択されていれば真
00938             const bool selected{ apiId == backend };
00939
00940             // 装置特性を（それが現在の装置特性ならハイライトして）コンボボックスに表示する
00941             if (ImGui::Selectable(apiName, selected))
00942             {
00943                 // 表示した装置特性が選択されていたらそれを現在の選択とする
00944                 backend = apiId;
00945
00946                 // 切り替え前の装置特性のデバイスが存在しなければ最初のデバイスの番号を選択する
00947                 if (deviceNumber < 0) deviceNumber = 0;
00948
00949                 // 選択されているデバイスの番号が継続されたキャプチャデバイスの数を超えないようにする
00950                 const int count{ getDeviceCount(backend) };
00951                 if (deviceNumber >= count) deviceNumber = count - 1;
00952             }
00953
00954             // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00955             if (selected) ImGui::SetItemDefaultFocus();
00956         }
00957         ImGui::EndCombo();
00958     }
00959
00960     // キャプチャデバイスが存在すれば
00961     if (deviceNumber >= 0)
00962     {
00963         // キャプチャデバイスの選択コンボボックス
00964         if (ImGui::BeginCombo(u8"入力源", getDeviceName(backend, deviceNumber).c_str()))
00965         {
00966             // すべてのキャプチャデバイスについて
00967             for (int i = 0; i < static_cast<int>(getDeviceList(backend).size()); ++i)
00968             {
00969                 // キャプチャデバイス名を（それを選択してあればハイライトして）コンボボックスに表示する
00970                 if (ImGui::Selectable(getDeviceName(backend, i).c_str(), i == deviceNumber))
00971                 {
00972                     // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
00973                     deviceNumber = i;
00974
00975                     // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00976                     ImGui::SetItemDefaultFocus();
00977                 }
00978             }
00979             ImGui::EndCombo();
00980         }
00981     }
00982     else
00983     {
00984         // 使えるキャプチャデバイスがない
00985         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが見つかりません");
00986     }
00987
00988     // キャプチャするサイズとフレームレート
00989     ImGui::InputInt2(u8"解像度", intrinsics.size.data());
00990     ImGui::InputDouble(u8"周波数", &intrinsics.fps, 1.0f, 1.0f, "%.1f");
00991
00992     // コーデックを選択する
00993     if (ImGui::BeginCombo(u8"符号化", codecList[codecNumber]))
00994     {
00995         // すべてのコーデックについて
00996         for (int i = 0; i < static_cast<int>(codecList.size()); ++i)
00997         {
00998             // コーデックを（それを選択してあればハイライトして）コンボボックスに表示する
00999             if (ImGui::Selectable(codecList[i], i == codecNumber))
01000             {
01001                 // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
01002                 codecNumber = i;
01003
01004                 // この選択を次にコンボボックスを開いたときのデフォルトにしておく
01005                 ImGui::SetItemDefaultFocus();
01006             }
01007         }
01008         ImGui::EndCombo();
01009     }
01010
01011     // キャプチャの開始と停止
01012     if (capture)
01013     {
01014         // キャプチャスレッドが動いているので止める
01015         if (ImGui::Button(u8"停止")) capture.stop();
01016         ImGui::SameLine();
01017         ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"取得中");
01018     }

```



```

01019     else
01020     {
01021         // キャプチャスレッドが止まっているので
01022         if (ImGui::Button(u8"開始") && deviceNumber >= 0)
01023         {
01024             // キャプチャデバイスが開いたらキャプチャスレッドを動かす
01025             if (openDevice()) capture.start();
01026         }
01027         ImGui::SameLine();
01028         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"停止中");
01029     }
01030 #endif
01031     ImGui::End();
01032 }
01033
01034 // 校正パネル
01035 if (showCalibrationPanel)
01036 {
01037     // ウィンドウの位置とサイズ
01038     ImGui::SetNextWindowPos(ImVec2(235.0f, 2.0f + menubarHeight), ImGuiCond.Once);
01039     ImGui::SetNextWindowSize(ImVec2(218, 329), ImGuiCond.Once);
01040     ImGui::Begin(u8"校正", &showCalibrationPanel);
01041
01042     // 辞書の選択
01043     if (ImGui::BeginCombo(u8"辞書", settings.dictionaryName.c_str()))
01044     {
01045         // すべての辞書について
01046         for (auto d = calibration.dictionaryList.begin(); d != calibration.dictionaryList.end(); ++d)
01047         {
01048             // その設定が現在選択されている設定なら真
01049             const bool selected(d->first == settings.dictionaryName);
01050
01051             // 設定を（それが現在の設定ならハイライトして）コンボボックスに表示する
01052             if (ImGui::Selectable(d->first.c_str(), d->first == settings.dictionaryName))
01053             {
01054                 // 表示した設定が選択されていたらそれを現在の選択とする
01055                 settings.dictionaryName = d->first;
01056
01057                 // 選択した ArUco Marker の辞書を設定する
01058                 calibration.setDictionary(settings.dictionaryName, settings.checkerLength);
01059             }
01060
01061             // この選択を次にコンボボックスを開いたときのデフォルトにしておく
01062             if (selected) ImGui::SetItemDefaultFocus();
01063         }
01064     }
01065     ImGui::EndCombo();
01066
01067     ImGui::Separator();
01068
01069     // ArUco Marker の検出
01070     if (ImGui::Checkbox(u8"ArUco Marker 検出", &detectMarker) && detectMarker) detectBoard = false;
01071
01072     // ArUco Marker の大きさ
01073     ImGui::InputFloat(u8"マーカ長", &settings.markerLength, 0.0f, 0.0f, "%.2f cm");
01074
01075     ImGui::Separator();
01076
01077     // 校正
01078     if (ImGui::Checkbox(u8"ChArUco Board 検出", &detectBoard) && detectBoard) detectMarker = false;
01079
01080     // ChArUco Board の大きさ
01081     if (ImGui::InputFloat2(u8"升目長", settings.checkerLength.data(), "%.2f cm"))
01082     {
01083         // ChArUco Board を作り直す
01084         calibration.createBoard(settings.checkerLength);
01085     }
01086
01087     // 「取得」ボタンをクリックしたとき ChArUco Board の検出中なら
01088     if (ImGui::Button(u8"取得") && detectBoard)
01089     {
01090         // 検出したコーナーを記録する
01091         calibration.recordCorners();
01092     }
01093
01094     // 1つでも標本を取得していれば
01095     if (calibration.getSampleCount() > 0)
01096     {
01097         // 標本の「消去」ボタンを表示する
01098         ImGui::SameLine();
01099         if (ImGui::Button(u8"消去")) calibration.discardCorners();
01100
01101         // 標本を6つ以上取得していれば
01102         if (calibration.getSampleCount() >= 6)
01103         {
01104             // 「校正」ボタンを表示する
01105             ImGui::SameLine();

```

```

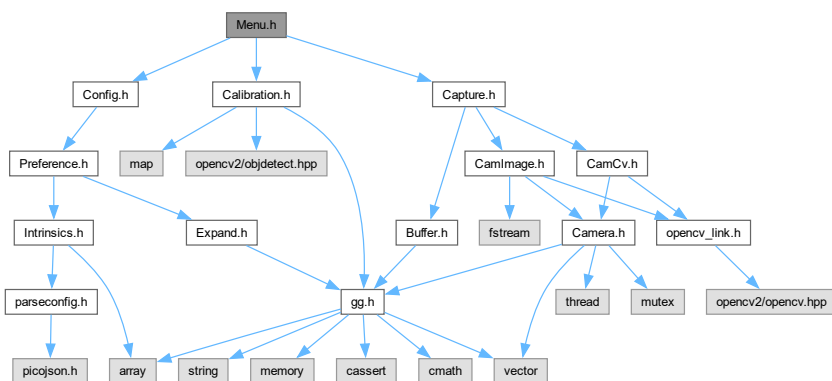
01106         if (ImGui::Button(u8"校正") && !calibration.calibrate())
01107         {
01108             // 校正失敗
01109             errorMessage = u8"校正に失敗しました";
01110         }
01111
01112         // 校正が完了していれば
01113         if (calibration.finished())
01114         {
01115             // 「完了」を表示する
01116             ImGui::SameLine();
01117             ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"完了");
01118         }
01119     }
01120 }
01121
01122 ImGui::Separator();
01123
01124 // フレームレートの表示
01125 ImGui::Text(u8"フレームレート: %6.2f fps", ImGui::GetIO().Framerate);
01126
01127 // 検出数の表示
01128 ImGui::Text(u8"コーナー検出数: %d", calibration.getCornersCount());
01129
01130 // 標本数の表示
01131 ImGui::Text(u8"サンプル取得数: %d (%d)", calibration.getSampleCount(), calibration.getTotalCount());
01132
01133 // 再投影誤差の表示
01134 ImGui::Text(u8"再投影誤差: %.4f", calibration.getReprojectionError());
01135
01136 ImGui::End();
01137 }
01138
01139 // エラーメッセージが設定されていたら
01140 if (errorMessage)
01141 {
01142     // ウィンドウの位置・サイズとタイトル
01143     ImGui::SetNextWindowPos(ImVec2(60, 60), ImGuiCond_Once);
01144     ImGui::SetNextWindowSize(ImVec2(240, 92), ImGuiCond_Always);
01145
01146     // ウィンドウを表示するとき true
01147     bool status{ true };
01148
01149     // エラーメッセージウィンドウを表示する
01150     ImGui::Begin(u8"エラー", &status);
01151
01152     // エラーメッセージの表示
01153     ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", errorMessage);
01154
01155     // クローズボックスか「閉じる」ボタンをクリックしたら
01156     if (!status || ImGui::Button(u8"閉じる"))
01157     {
01158         // エラーメッセージを消去する
01159         errorMessage = nullptr;
01160     }
01161     ImGui::End();
01162 }
01163
01164 // ChArUco Board の検出中にスペースバーをタイプしたなら
01165 if (detectBoard && ImGui::IsKeyPressed(ImGuiKey_Space))
01166 {
01167     // 検出したコーナーを記録する
01168     calibration.recordCorners();
01169 }
01170 }
01171
01172 //
01173 // 画像の保存
01174 //
01175 void Menu::saveImage(const cv::Mat& image, const std::string& filename) const
01176 {
01177     // 画像ファイル名のフィルタ
01178     constexpr nfdfilteritem_t imageFilter[]{ "Images", "png,jpg,jpeg,jfif,bmp,dib" };
01179
01180     // ファイルダイアログから得るパス
01181     nfdchar_t* filepath;
01182
01183     // ファイルダイアログを開く
01184     if (NFD_SaveDialog(&filepath, imageFilter, 1, NULL, filename.c_str()) == NFD_OKAY)
01185     {
01186         // ファイルに保存する
01187         if (!CamImage::save(filepath, image)) errorMessage = u8"ファイルが保存できませんでした";
01188
01189         // ファイルパスの取り出しに使ったメモリを開放する
01190         NFD_FreePath(filepath);
01191     }
01192 }

```

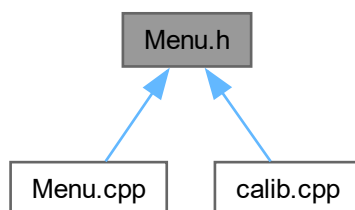
9.55 Menu.h ファイル

```
#include "Config.h"
#include "Capture.h"
#include "Calibration.h"
```

Menu.h の依存先関係図:



被依存関係図:



クラス

- class [Menu](#)

9.55.1 詳解

メニューの描画クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Menu.h](#) に定義があります。

9.56 Menu.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 構成データ
00012 #include "Config.h"
00013
00014 // キャプチャデバイス
00015 #include "Capture.h"
00016
00017 // 校正オブジェクト
00018 #include "Calibration.h"
00019
00023 class Menu
00024 {
00026     const Config& config;
00027
00029     Settings settings;
00030
00031 #if !defined(_WIN32)
00033     static const std::map<cv::VideoCaptureAPIs, const char*> backendList;
00034
00036     static const std::vector<const char*> codecList;
00037
00039     static std::map <cv::VideoCaptureAPIs, std::vector<std::string>> deviceList;
00040
00042     std::vector<std::string> fileHistory;
00043
00050     const auto& getDeviceList (cv::VideoCaptureAPIs api) const
00051     {
00052         return deviceList.at(api);
00053     }
00054
00061     auto getDeviceCount (cv::VideoCaptureAPIs api) const
00062     {
00063         return static_cast<int>(deviceList.at(api).size());
00064     }
00065
00073     const auto& getDeviceName (cv::VideoCaptureAPIs api, int number) const
00074     {
00075         static const std::string empty{};
00076         const auto& list{ deviceList.at(api) };
00077         return list.empty() ? empty : list[number];
00078     }
00079 #endif
00080
00082     Intrinsics intrinsics;
00083
00085     Capture& capture;
00086
00088     Calibration& calibration;
00089
00091     int deviceNumber;
00092
00093 #if defined(_WIN32)
00095     int formatNumber;
00096
00098     struct FormatInfo
00099     {
00100         std::string resolution; // "640 x 480"
00101         std::string fps;        // "30.00"
00102         std::string codec;      // "NV12"
00103         int index;              // formatList のインデックス
00104     };
00105
00107     std::vector<FormatInfo> parsedFormats;
00108
00110     std::vector<std::string> uniqueResolutions;
00111
00113     std::vector<std::string> uniqueFpsList;
00114
00116     std::vector<std::string> uniqueCodecs;
00117
00119     std::string currentRes;
00120
00122     std::string currentFps;
00123
00125     std::string currentCodec;
00126
00128     int lastDeviceNumber;
00129
00131     void updateFormatDropdowns();
00132 #else
00134     int codecNumber;

```

```

00135
00137     cv::VideoCaptureAPIs backend;
00138 #endif
00139
00141     int preferenceNumber;
00142
00144     GgMatrix pose;
00145
00147     GLsizei menubarHeight;
00148
00150     bool showInputPanel;
00151
00153     bool showCalibrationPanel;
00154
00156     bool quit;
00157
00159     mutable const char* errorMessage;
00160
00164     bool openDevice();
00165
00169     void openImage();
00170
00174     void openMovie();
00175
00179     void loadConfig();
00180
00184     void saveConfig() const;
00185
00189     void loadParameters() const;
00190
00194     void saveParameters() const;
00195
00199     void recordFileCorners() const;
00200
00204     void createCharuco() const;
00205
00211     const auto& getPreference(int i) const
00212     {
00213         return config.preferenceList[i];
00214     }
00215
00219     const auto& getPreference() const
00220     {
00221         return getPreference(preferenceNumber);
00222     }
00223
00224 public:
00225
00227     bool detectMarker;
00228
00230     bool detectBoard;
00231
00239     Menu(const Config& config, Capture& capture, Calibration& calibration);
00240
00246     Menu(const Menu& menu) = delete;
00247
00251     virtual ~Menu();
00252
00259     Menu& operator=(const Menu& menu) = delete;
00260
00266     explicit operator bool() const
00267     {
00268         return !quit;
00269     }
00270
00276     const auto& getPose() const
00277     {
00278         return pose;
00279     }
00280
00286     auto getMenubarHeight() const
00287     {
00288         return menubarHeight;
00289     }
00290
00301     void setSize(const std::array<int, 2>& size);
00302
00308     const auto& getCheckerLength() const
00309     {
00310         return settings.checkerLength;
00311     }
00312
00318     auto getMarkerLength() const
00319     {
00320         return settings.markerLength;
00321     }
00322

```

```

00332     std::array<GLsizei, 2> setup(GLfloat aspect) const;
00333
00337     void draw();
00338
00345     void saveImage(const cv::Mat& image, const std::string& filename = "*.jpg") const;
00346 };

```

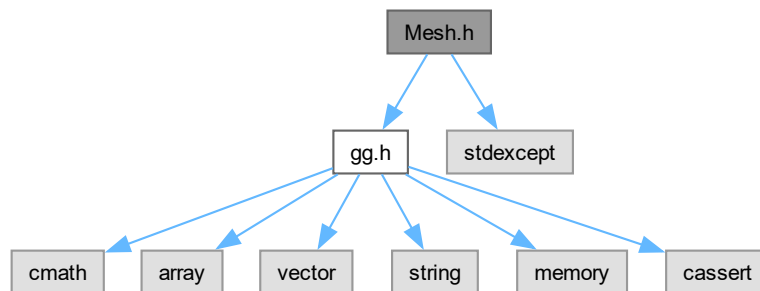
9.57 Mesh.h ファイル

```

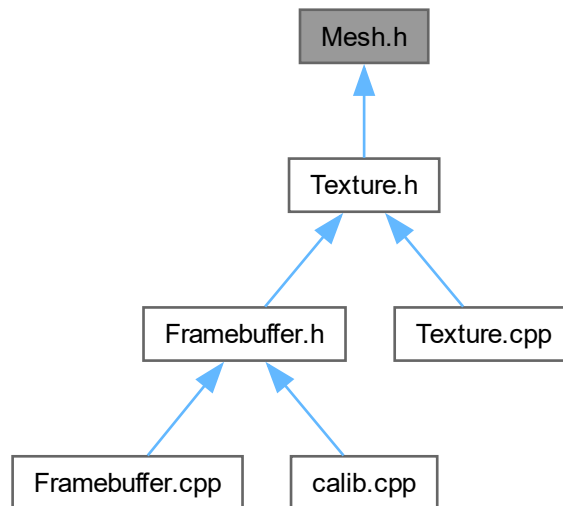
#include "gg.h"
#include <stdexcept>

```

Mesh.h の依存先関係図:



被依存関係図:



クラス

- class `Mesh`

9.57.1 詳解

メッシュの描画クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Mesh.h](#) に定義があります。

9.58 Mesh.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013
00014 // 標準ライブラリ
00015 #include <stdexcept>
00016
00020 class Mesh
00021 {
00023     const GLuint array;
00024
00026     const GLuint program;
00027
00029     const GLint imageLoc;
00030
00032     const GLint scaleLoc;
00033
00034 public:
00035
00039     Mesh()
00040     : array{ [] { GLuint array; glGenVertexArrays(1, &array); return array; }() }
00041     , program{ gg::ggLoadShader("draw.vert", "draw.frag") }
00042     , imageLoc{ glGetUniformLocation(program, "image") }
00043     , scaleLoc{ glGetUniformLocation(program, "scale") }
00044     {
00045         // 頂点配列オブジェクトが作れなかったら落とす
00046         assert(array);
00047
00048         // シェーダが作れなかったら落とす
00049         if (program == 0) throw std::runtime_error("Cannot create the shader for the mesh.");
00050     }
00051
00055     Mesh(const Mesh& mesh) = delete;
00056
00060     virtual ~Mesh()
00061     {
00062         glDeleteVertexArrays(1, &array);
00063         glDeleteProgram(program);
00064     }
00065
00072     Mesh& operator=(const Mesh& mesh) = delete;
00073
00084     void draw(const std::array<GLfloat, 2>& scale, GLint unit = 0) const
00085     {
00086         // 表示領域を背景色で塗りつぶす
00087         glClear(GL_COLOR_BUFFER_BIT);
00088
00089         // シェーダの使用開始
00090         glUseProgram(program);
00091
00092         // サンプラの指定
00093         glUniform1i(imageLoc, unit);
00094
00095         // スケールの設定
00096         glUniform2fv(scaleLoc, 1, scale.data());
00097
00098         // 描画
00099         glBindVertexArray(array);
00100         glDrawArrays(GL_TRIANGLES, 0, 4);
00101         glBindVertexArray(0);
00102     }
```

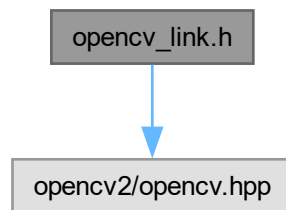
```
00103     // シェーダの仕様終了
00104     glUseProgram(0);
00105 }
00106
00116 void drawMesh(const std::array<GLsizei, 2>& size) const
00117 {
00118     // 描画
00119     glBindVertexArray(array);
00120     glDrawArraysInstanced(GL_TRIANGLES_STRIP, 0, size[0] * 2, size[1] - 1);
00121     glBindVertexArray(0);
00122 }
00123 };
```

9.59 opencv_link.h ファイル

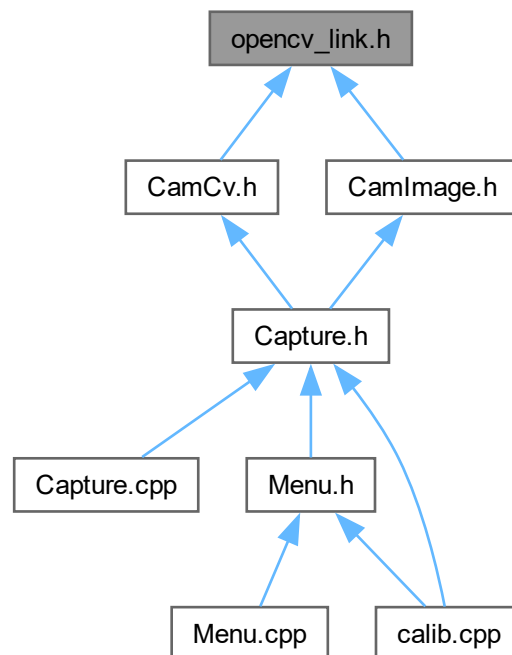
Msvc環境用のOpenCV自動リンクおよびインクルード一括管理ヘッダー

```
#include <opencv2/opencv.hpp>
```

opencv_link.h の依存先関係図:



被依存関係図:



9.59.1 詳解

MSVC環境用のOpenCV自動リンクおよびインクルード一括管理ヘッダー [opencv_link.h](#) に定義があります。

9.60 opencv_link.h

[詳解]

```

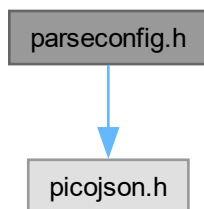
00001 #pragma once
00002
00007
00008 // OpenCV
00009 #pragma warning(disable:4819)
00010 #include <opencv2/opencv.hpp>
00011
00012 #if defined(_WIN32)
00013 #   define CV_VERSION_STR CVAUX_STR(CV_MAJOR_VERSION) CVAUX_STR(CV_MINOR_VERSION)
00014 #   define CV_EXT_STR "d.lib"
00015 #   define CV_EXT_STR ".lib"
00016 #   define CV_EXT_STR ".lib"
00017 #   define CV_EXT_STR ".lib"
00018 #   define CV_EXT_STR ".lib"
00019 #   pragma comment(lib, "opencv_world" CV_VERSION_STR CV_EXT_STR)
00020 #endif

```

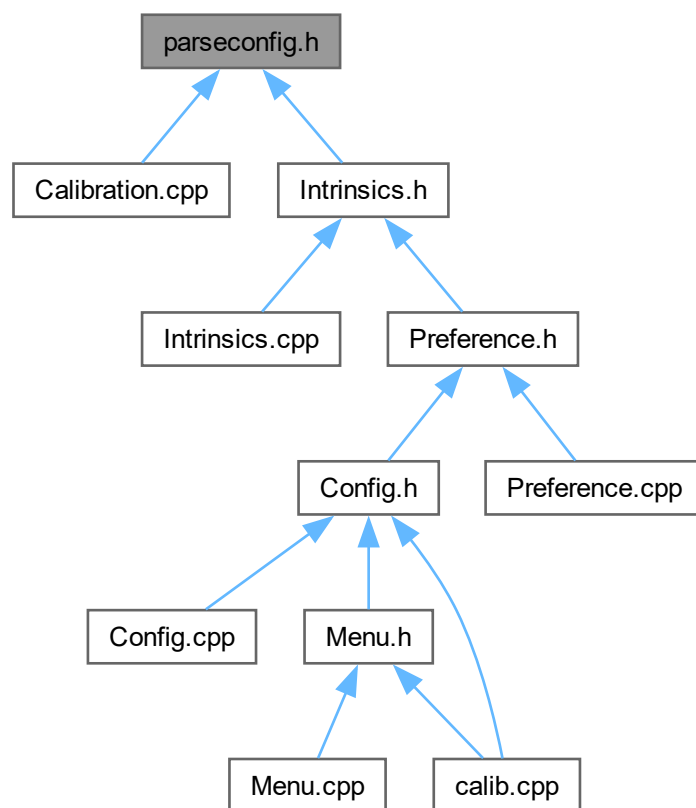
9.61 parseconfig.h ファイル

```
#include "picojson.h"
```

parseconfig.h の依存先関係図:



被依存関係図:



関数

- `template<typename T>`
`bool getValue (const picojson::object &object, const std::string &key, T &scalar)`
- `template<typename T, std::size_t U>`
`bool getValue (const picojson::object &object, const std::string &key, std::array< T, U > &vector)`
- `bool getString (const picojson::object &object, const std::string &key, std::string &string)`
- `template<std::size_t U>`
`bool getString (const picojson::object &object, const std::string &key, std::array< std::string, U > &strings)`
- `bool getString (const picojson::object &object, const std::string &key, std::vector< std::string > &strings)`
- `template<typename T>`
`void setValue (picojson::object &object, const std::string &key, const T &scalar)`
- `template<typename T, std::size_t U>`
`void setValue (picojson::object &object, const std::string &key, const std::array< T, U > &vector)`
- `void setString (picojson::object &object, const std::string &key, const std::string &string)`
- `template<std::size_t U>`
`void setString (picojson::object &object, const std::string &key, const std::array< std::string, U > &strings)`
- `void setString (picojson::object &object, const std::string &key, const std::vector< std::string > &strings)`

9.61.1 詳解

構成ファイルの読み取り補助

著者

Kohe Tokoi

日付

November 15, 2022

[parseconfig.h](#) に定義があります。

9.61.2 関数詳解

9.61.2.1 getString() [1/3]

```
template<std::size_t U>
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::array< std::string, U > & strings)
```

構成ファイルの JSON オブジェクトから文字列の配列を取得する

テンプレート引数

<i>U</i>	構成ファイルから取得する配列の要素の文字列の数
----------	-------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>strings</i>	取得した文字列の配列を格納する変数

[parseconfig.h](#) の 105 行目に定義があります。

9.61.2.2 getString() [2/3]

```
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::string & string) [inline]
```

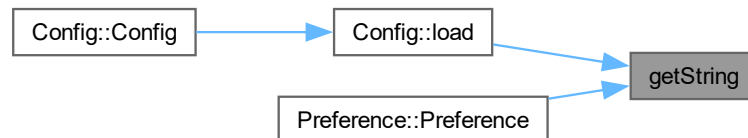
構成ファイルの JSON オブジェクトから文字列を取得する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>string</i>	取得した文字列を格納する変数

[parseconfig.h](#) の 81 行目に定義があります。

被呼び出し関係図:



9.61.2.3 getString() [3/3]

```
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::vector< std::string > & strings) [inline]
```

構成ファイルの JSON オブジェクトから文字列のベクトルを取得する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>strings</i>	取得した文字列の配列を格納する変数

[parseconfig.h](#) の 138 行目に定義があります。

9.61.2.4 getValue() [1/2]

```
template<typename T, std::size_t U>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    std::array< T, U > & vector)
```

構成ファイルの JSON オブジェクトから数値の配列を取得する

テンプレート引数

<i>T</i>	構成ファイルから取得する配列の要素の数値のデータ型
<i>U</i>	構成ファイルから取得する配列の要素の数値の数

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>vector</i>	取得した数値の配列を格納する変数

[parseconfig.h](#) の 48 行目に定義があります。

9.61.2.5 getValue() [2/2]

```
template<typename T>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    T & scalar)
```

構成ファイルの JSON オブジェクトから数値を取得する

テンプレート引数

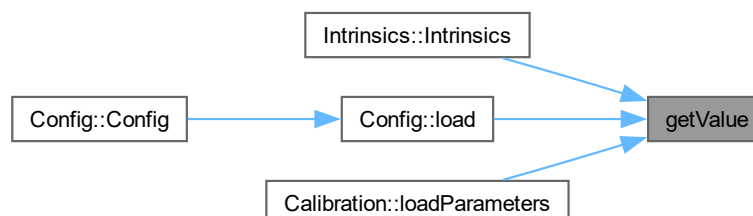
<i>T</i>	構成ファイルから取得する数値のデータ型
----------	---------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>scalar</i>	取得した JSON オブジェクトの数値を格納する変数

[parseconfig.h](#) の 23 行目に定義があります。

被呼び出し関係図:



9.61.2.6 setString() [1/3]

```
template<std::size_t U>
void setString (
    picojson::object & object,
    const std::string & key,
    const std::array< std::string, U > & strings)
```

構成ファイルの JSON オブジェクトに文字列の配列を設定する

テンプレート引数

<i>U</i>	構成ファイルに設定する配列の要素の文字列の数
----------	------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>strings</i>	設定する文字列の配列

[parseconfig.h](#) の 225 行目に定義があります。

9.61.2.7 setString() [2/3]

```
void setString (
    picojson::object & object,
    const std::string & key,
    const std::string & string) [inline]
```

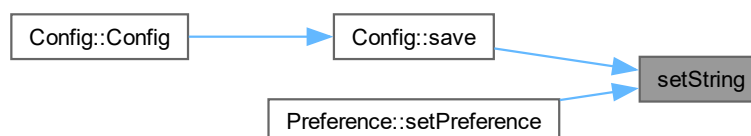
構成ファイルの JSON オブジェクトに文字列を設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>string</i>	設定する文字列

[parseconfig.h](#) の 210 行目に定義があります。

被呼び出し関係図:



9.61.2.8 setString() [3/3]

```
void setString (
```

```
picojson::object & object,
const std::string & key,
const std::vector< std::string > & strings) [inline]
```

構成ファイルの JSON オブジェクトに文字列のベクトルを設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>strings</i>	設定する文字列の配列

[parseconfig.h](#) の 250 行目に定義があります。

9.61.2.9 setValue() [1/2]

```
template<typename T, std::size_t U>
void setValue (
    picojson::object & object,
    const std::string & key,
    const std::array< T, U > & vector)
```

構成ファイルの JSON オブジェクトに数値の配列を設定する

テンプレート引数

<i>T</i>	構成ファイルに設定する配列の要素の数値のデータ型
<i>U</i>	構成ファイルに設定する配列の要素の数値の数

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>vector</i>	設定する数値の配列

[parseconfig.h](#) の 185 行目に定義があります。

9.61.2.10 setValue() [2/2]

```
template<typename T>
void setValue (
    picojson::object & object,
    const std::string & key,
    const T & scalar)
```

構成ファイルの JSON オブジェクトに数値を設定する

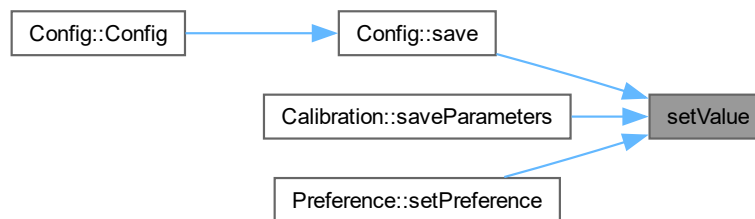
テンプレート引数

<i>T</i>	構成ファイルに設定する数値のデータ型
----------	--------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>scalar</i>	設定する数値

[parseconfig.h](#) の 169 行目に定義があります。
被呼び出し関係図:



9.62 parseconfig.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // JSON
00012 #include "picojson.h"
00013
00022 template <typename T>
00023 bool getValue(const picojson::object& object,
00024             const std::string& key, T& scalar)
00025 {
00026     // key に一致するオブジェクトを探す
00027     const auto&& value{ object.find(key) };
00028
00029     // オブジェクトが無いか数値でなかったら戻る
00030     if (value == object.end() || !value->second.is<double>()) return false;
00031
00032     // 数値として格納する
00033     scalar = static_cast<T>(value->second.get<double>());
00034
00035     return true;
00036 }
00037
00047 template <typename T, std::size_t U>
00048 bool getValue(const picojson::object& object,
00049             const std::string& key, std::array<T, U>& vector)
00050 {
00051     // key に一致するオブジェクトを探す
00052     const auto&& value{ object.find(key) };
00053
00054     // オブジェクトが無いか配列でなかったら戻る
00055     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00056
00057     // 配列を取り出す
00058     const auto& array{ value->second.get<picojson::array>() };
00059
00060     // 配列の要素数とデータの格納先の要素数の少ない方の数
00061     const auto n{ std::min(U, array.size()) };
00062
00063     // 配列の要素について
00064     for (std::size_t i = 0; i < n; ++i)
00065     {
00066         // 要素が数値なら格納する
00067         if (array[i].is<double>()) vector[i] = static_cast<T>(array[i].get<double>());
00068     }
00069
00070     return true;
00071 }
00072

```



```

00080 inline
00081 bool getString(const picojson::object& object,
00082               const std::string& key, std::string& string)
00083 {
00084     // key に一致するオブジェクトを探す
00085     const auto&& value{ object.find(key) };
00086     // オブジェクトが無いか文字列でなかったら戻る
00087     if (value == object.end() || !value->second.is<std::string>()) return false;
00088     // 文字列として格納する
00089     string = value->second.get<std::string>();
00090     return true;
00091 }
00092
00093 template <std::size_t U>
00094 bool getString(const picojson::object& object,
00095               const std::string& key, std::array<std::string, U>& strings)
00096 {
00097     // key に一致するオブジェクトを探す
00098     const auto&& value{ object.find(key) };
00099     // オブジェクトが無いか配列でなかったら戻る
00100     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00101     // 配列を取り出す
00102     const auto& array{ value->second.get<picojson::array>() };
00103     // 配列の要素数とデータの格納先の要素数の少ない方の数
00104     const auto n{ std::min(U, array.size()) };
00105     // 配列の要素について
00106     for (std::size_t i = 0; i < n; ++i)
00107     {
00108         // 要素が文字列なら文字列として格納する
00109         strings[i] = array[i].is<std::string>() ? array[i].get<std::string>() : "";
00110     }
00111     return true;
00112 }
00113
00114 inline
00115 bool getString(const picojson::object& object,
00116               const std::string& key, std::vector<std::string>& strings)
00117 {
00118     // key に一致するオブジェクトを探す
00119     const auto&& value{ object.find(key) };
00120     // オブジェクトが無いか配列でなかったら戻る
00121     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00122     // 配列を取り出す
00123     const auto& array{ value->second.get<picojson::array>() };
00124     // 配列のすべての要素について
00125     for (auto& element : array)
00126     {
00127         // 要素が文字列なら文字列として格納する
00128         strings.emplace_back(element.is<std::string>() ? element.get<std::string>() : "");
00129     }
00130     return true;
00131 }
00132
00133 template <typename T>
00134 void setValue(picojson::object& object,
00135              const std::string& key, const T& scalar)
00136 {
00137     object.emplace(key, picojson::value(static_cast<double>(scalar)));
00138 }
00139
00140 template <typename T, std::size_t U>
00141 void setValue(picojson::object& object,
00142              const std::string& key, const std::array<T, U>& vector)
00143 {
00144     // picojson の配列
00145     picojson::array array;
00146     // 配列のすべての要素について
00147     for (const auto& element : vector)
00148     {
00149         // 要素を picojson::array に追加する
00150         array.emplace_back(picojson::value(static_cast<double>(element)));
00151     }
00152     // オブジェクトに追加する

```

```

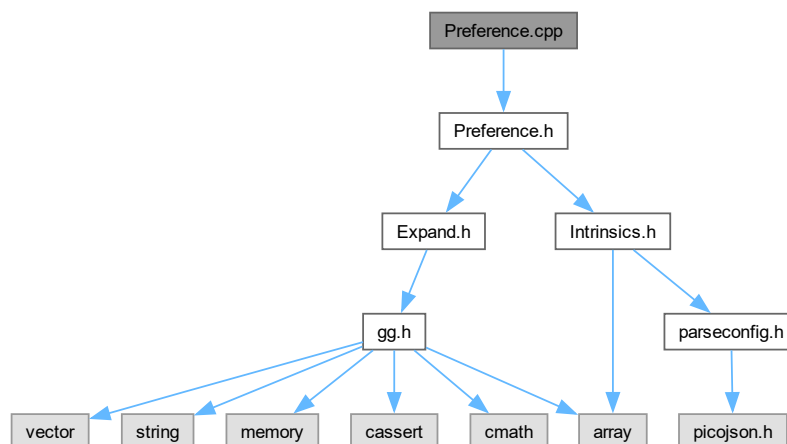
00199     object.emplace(key, array);
00200 }
00201
00209 inline
00210 void setString(picojson::object& object,
00211               const std::string& key, const std::string& string)
00212 {
00213     object.emplace(key, picojson::value(string));
00214 }
00215
00224 template <std::size_t U>
00225 void setString(picojson::object& object,
00226               const std::string& key, const std::array<std::string, U>& strings)
00227 {
00228     // picojson の配列
00229     picojson::array array;
00230
00231     // 配列のすべての要素について
00232     for (const auto& string : strings)
00233     {
00234         // 要素を picojson::array に追加する
00235         array.emplace_back(picojson::value(string));
00236     }
00237
00238     // オブジェクトに追加する
00239     object.emplace(key, array);
00240 }
00241
00249 inline
00250 void setString(picojson::object& object,
00251               const std::string& key, const std::vector<std::string>& strings)
00252 {
00253     // picojson の配列
00254     picojson::array array;
00255
00256     // 配列のすべての要素について
00257     for (auto& string : strings)
00258     {
00259         // 要素を picojson::array に追加する
00260         array.emplace_back(picojson::value(string));
00261     }
00262
00263     // オブジェクトに追加する
00264     object.emplace(key, array);
00265 }

```

9.63 Preference.cpp ファイル

#include "Preference.h"

Preference.cpp の依存先関係図:



9.63.1 詳解

キャプチャデバイスの構成データクラスの実装

著者

Kohe Tokoi

日付

January 23, 2025

[Preference.cpp](#) に定義があります。

9.64 Preference.cpp

[[詳解](#)]

```
00001
00008 #include "Preference.h"
00009
00010 //
00011 // キャプチャデバイスの構成データのデフォルトコンストラクタ
00012 //
00013 Preference::Preference()
00014 : Preference{ "Default", "orthographic.vert", "normal.frag" }
00015 {
00016 }
00017
00018 //
00019 // シェーダのソースファイルを指定するときに使う
00020 // キャプチャデバイスの構成データのコンストラクタ
00021 //
00022 Preference::Preference(const std::string& description,
00023     const std::string& vert, const std::string& frag,
00024     const Intrinsic& intrinsics)
00025 : description{ description }
00026 , source{ vert, frag }
00027 , intrinsics{ intrinsics }
00028 , shader{ nullptr }
00029 {
00030 }
00031
00032 //
00033 // 構成ファイルを読み込むときに使う
00034 // キャプチャデバイスの構成データのコンストラクタ
00035 //
00036 Preference::Preference(const picojson::object& object)
00037 : intrinsics{ object }
00038 , shader{ nullptr }
00039 {
00040     // 説明の文字列
00041     getString(object, "description", description);
00042     // 展開用シェーダのファイル名
00043     getString(object, "shader", source);
00044 }
00045 }
00046
00047 //
00048 // 構成データのデストラクタ
00049 //
00050 Preference::~Preference()
00051 {
00052     // static メンバにしている std::map の中身を先に消去しておく
00053     shaderList.clear();
00054 }
00055
00056 //
00057 // シェーダをビルドする
00058 //
00059 void Preference::buildShader()
00060 {
00061     // ソースファイル名をつないでシェーダを検索するキーを作る
00062     const auto key{ source[0] + "\\t" + source[1] };
00063     // キーがシェーダリストに無ければシェーダを構築して追加する
00064     shader = &shaderList.try_emplace(key, source[0], source[1]).first->second;
00065 }
00066 }
00067
00068 //
00069 // 構成を JSON オブジェクトに格納する
```

```

00070 //
00071 void Preference::setPreference(picojson::object& object) const
00072 {
00073     // 説明の文字列
00074     setString(object, "description", description);
00075
00076     // キャプチャデバイスのレンズの縦横の画角
00077     setValue(object, "fov", intrinsics.fov);
00078
00079     // キャプチャデバイスのレンズの中心（主点）位置
00080     setValue(object, "center", intrinsics.center);
00081
00082     // キャプチャデバイスの解像度
00083     setValue(object, "resolution", intrinsics.size);
00084
00085     // キャプチャデバイスのフレームレート
00086     setValue(object, "fps", intrinsics.fps);
00087
00088     // 展開用シェーダのファイル名
00089     setString(object, "shader", source);
00090 }
00091
00092 // すべての構成のシェーダーのリスト
00093 std::map<std::string, Expand> Preference::shaderList;

```

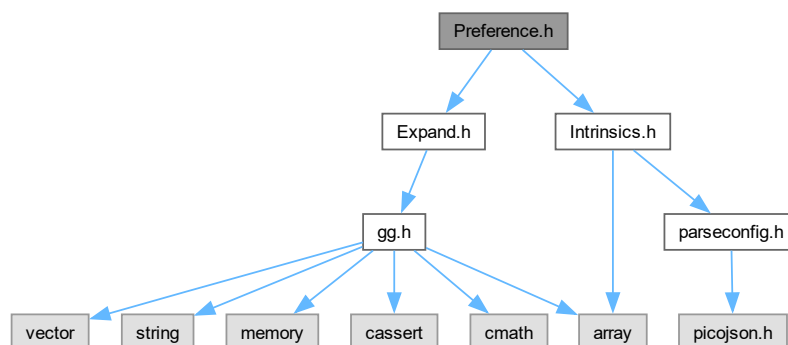
9.65 Preference.h ファイル

```

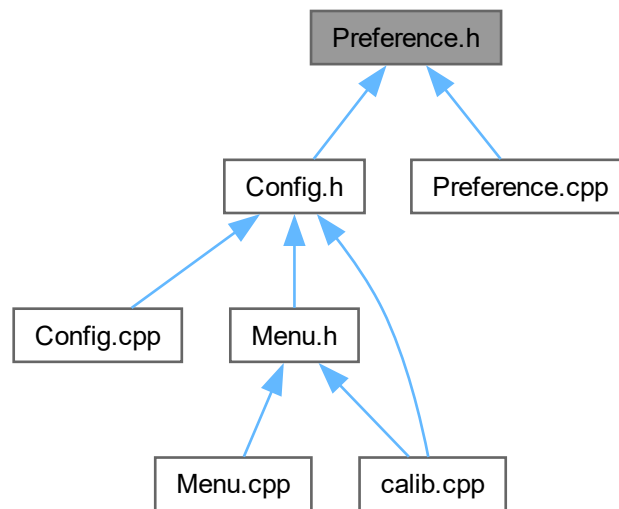
#include "Expand.h"
#include "Intrinsics.h"

```

Preference.h の依存先関係図:



被依存関係図:



クラス

- class [Preference](#)

9.65.1 詳解

キャプチャデバイスの構成データクラスの定義

著者

Kohe Tokoi

日付

January 23, 2025

[Preference.h](#) に定義があります。

9.66 Preference.h

[[詳解](#)]

```
00001 #pragma once
00002
00010
00011 // 平面展開用のシェード
00012 #include "Expand.h"
00013
00014 // キャプチャデバイス固有のパラメータ
00015 #include "Intrinsics.h"
00016
00024 class Preference
00025 {
00027     std::string description;
00028
00030     std::array<std::string, 2> source;
00031
00033     const Intrinsics intrinsics;
00034 }
```

```

00036     const Expand* shader;
00037
00039     static std::map<std::string, Expand> shaderList;
00040
00041 public:
00042
00048     Preference();
00049
00059     Preference(const std::string& description,
00060               const std::string& vert, const std::string& frag,
00061               const Intrinsic& intrinsics = Intrinsic{});
00062
00067     Preference(const picojson::object& object);
00068
00072     virtual ~Preference();
00073
00077     void buildShader();
00078
00084     const auto& getDescription() const
00085     {
00086         return description;
00087     }
00088
00094     const auto& getIntrinsic() const
00095     {
00096         return intrinsics;
00097     }
00098
00104     const auto& getShader() const
00105     {
00106         return *shader;
00107     }
00108
00114     void setPreference(picojson::object& object) const;
00115 };

```

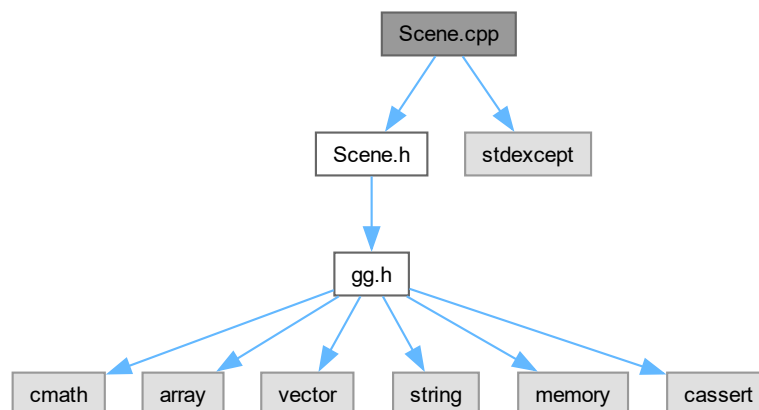
9.67 REQUESTS.md ファイル

9.68 Scene.cpp ファイル

```

#include "Scene.h"
#include <stdexcept>
Scene.cpp の依存先関係図:

```



9.68.1 詳解

シーンの描画クラスの実装

著者

Kohe Tokoi

日付

November 15, 2022

[Scene.cpp](#) に定義があります。

9.69 Scene.cpp

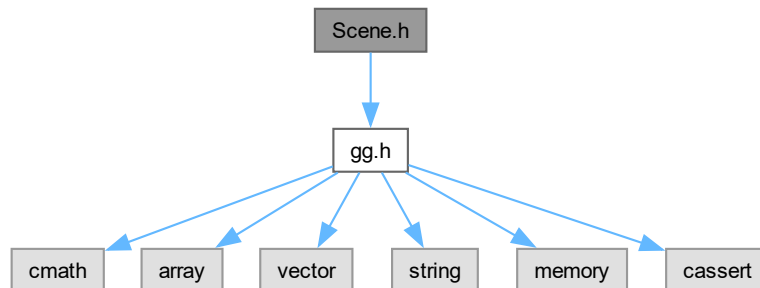
[詳解]

```
00001
00008 #include "Scene.h"
00009
00010 // 標準ライブラリ
00011 #include <stdexcept>
00012
00013 // 光源データ
00014 static constexpr GgSimpleShader::Light defaultLight
00015 {
00016     { 0.8f, 0.8f, 0.8f, 1.0f },
00017     { 0.8f, 0.8f, 0.8f, 0.0f },
00018     { 0.2f, 0.2f, 0.2f, 0.0f },
00019     { 3.0f, 4.0f, 5.0f, 1.0f }
00020 };
00021
00022 //
00023 // コンストラクタ
00024 //
00025 Scene::Scene()
00026 : shader{ "simple.vert", "simple.frag" }
00027 , model{ std::make_unique<const GgSimpleObj>("axis.obj") }
00028 , light{ defaultLight }
00029 {
00030     // シェーダが作れなかったら落とす
00031     if (shader.get() == 0) throw std::runtime_error("Cannot create the simple shader.");
00032
00033     // 図形が作れなかったら落とす
00034     if (!model) throw std::runtime_error("Cannot load the axis object.");
00035 }
00036
00037 //
00038 // デストラクタ
00039 //
00040 Scene::~Scene()
00041 {
00042 }
00043
00044 //
00045 // モデルを選択する
00046 //
00047 void Scene::set(const std::string& filename)
00048 {
00049     model = std::make_unique<const GgSimpleObj>(filename);
00050 }
00051
00052 //
00053 // 描画する
00054 //
00055 void Scene::draw(const GgMatrix& mp, const GgMatrix& mv) const
00056 {
00057     // シェーダプログラムを指定する
00058     shader.use(mp, mv, light);
00059
00060     // 図形を描画する
00061     model->draw();
00062 }
```

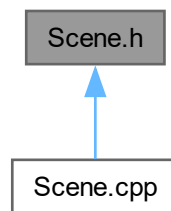
9.70 Scene.h ファイル

```
#include "gg.h"
```

Scene.h の依存先関係図:



被依存関係図:



クラス

- class [Scene](#)

9.70.1 詳解

シーンの描画クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Scene.h](#) に定義があります。

9.71 Scene.h

[\[詳解\]](#)

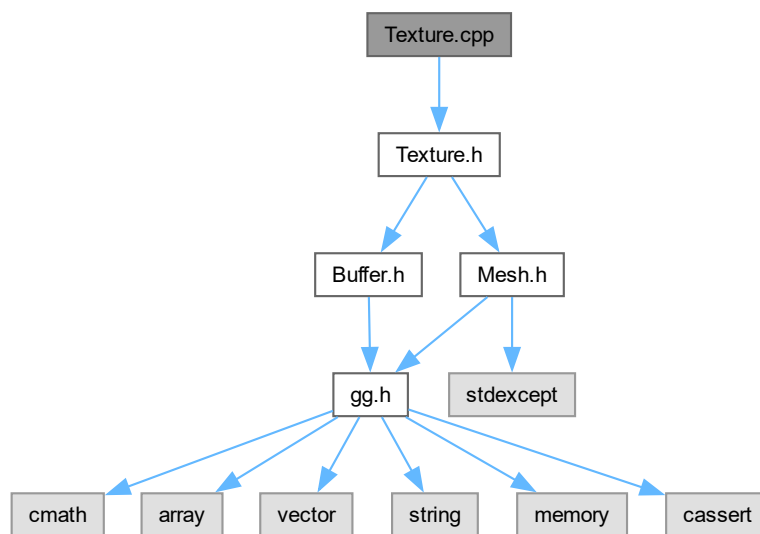

```

00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00018 class Scene
00019 {
00020     // シェーダ
00021     const GgSimpleShader shader;
00022
00023     // モデル
00024     std::unique_ptr<const GgSimpleObj> model;
00025
00026 public:
00027
00028     // 光源
00029     GgSimpleShader::LightBuffer light;
00030
00034     Scene();
00035
00039     virtual ~Scene();
00040
00046     void set(const std::string& filename);
00047
00054     void draw(const GgMatrix& mp, const GgMatrix& mv) const;
00055 };

```

9.72 Texture.cpp ファイル

#include "Texture.h"
Texture.cpp の依存先関係図:



9.72.1 詳解

テクスチャクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Texture.cpp](#) に定義があります。

9.73 Texture.cpp

[詳解]

```
00001
00008 #include "Texture.h"
00009
00010 //
00011 // テクスチャを作成するコンストラクタ
00012 //
00013 Texture::Texture(GLsizei width, GLsizei height, int channels)
00014 {
00015     // テクスチャを作る
00016     Texture::create(width, height, channels);
00017 }
00018
00019 //
00020 // コピーコンストラクタ
00021 //
00022 Texture::Texture(const Texture& texture)
00023 {
00024     // テクスチャをコピーして作成する
00025     Texture::copy(texture);
00026 }
00027
00028 //
00029 // デストラクタ
00030 //
00031 Texture::~Texture()
00032 {
00033     // テクスチャを破棄する
00034     Texture::discard();
00035 }
00036
00037 //
00038 // 代入演算子
00039 //
00040 Texture& Texture::operator=(const Texture& texture)
00041 {
00042     // 代入元と代入先が同じなら何もしない
00043     if (&texture == this) return *this;
00044
00045     // 引数のテクスチャをこのテクスチャにコピーする
00046     Texture::copy(texture);
00047
00048     // このテクスチャを返す
00049     return *this;
00050 }
00051
00052 //
00053 // ムーブ代入演算子
00054 //
00055 Texture& Texture::operator=(Texture&& texture) noexcept
00056 {
00057     // ムーブ代入元とムーブ代入先が同じなら何もしない
00058     if (&texture == this) return *this;
00059
00060     // ムーブ代入元の基底クラスをムーブする
00061     static.cast<Buffer&>(*this) = std::move(texture);
00062
00063     // ムーブ代入元のテクスチャのメンバをムーブする
00064     textureSize = texture.textureSize;
00065     texture.textureSize = { 0, 0 };
00066     textureChannels = texture.textureChannels;
00067     texture.textureChannels = 0;
00068
00069     // ムーブ代入先のテクスチャを削除する
00070     glDeleteTextures(1, &textureName);
00071
00072     // ムーブ代入元のテクスチャ名をムーブ代入先に移す
00073     textureName = texture.textureName;
00074
00075     // ムーブ代入元のデストラクタでテクスチャが削除されないよう 0 にする
00076     texture.textureName = 0;
00077
00078     // このテクスチャを返す
00079     return *this;
00080 }
```

```

00081
00082 //
00083 // テクスチャを作成する
00084 //
00085 void Texture::create(GLsizei width, GLsizei height, int channels)
00086 {
00087     // 既存のバッファを破棄して新しいバッファを作成する
00088     Buffer::create(width, height, channels);
00089
00090     // まだメッシュの頂点配列オブジェクトが作られていなければ作る
00091     if (!mesh) mesh = std::make_shared<Mesh>();
00092
00093     // 指定したサイズが保持しているテクスチャのサイズと同じなら何もしない
00094     if (width == textureSize[0] && height == textureSize[1]
00095         && channels == textureChannels) return;
00096
00097     // テクスチャのサイズとチャンネル数を記録する
00098     textureSize = std::array<int, 2>{ width, height };
00099     textureChannels = channels;
00100
00101     // 以前のテクスチャを削除する
00102     glDeleteTextures(1, &textureName);
00103
00104     // テクスチャを作成する
00105     glGenTextures(1, &textureName);
00106
00107     // テクスチャのメモリを確保してデータを転送する
00108     glBindTexture(GL_TEXTURE_2D, textureName);
00109     glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
00110         channelsToFormat(channels), GL_UNSIGNED_BYTE, nullptr);
00111     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
00112     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
00113     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
00114     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
00115     glBindTexture(GL_TEXTURE_2D, 0);
00116 }
00117
00118 //
00119 // テクスチャをコピーする
00120 //
00121 void Texture::copy(const Buffer& texture) noexcept
00122 {
00123     // コピー元と同じサイズの空のテクスチャを作る
00124     Texture::create(texture.getWidth(), texture.getHeight(),
00125         texture.getChannels());
00126
00127     // 作成したテクスチャのバッファにコピーする
00128     copyBuffer(texture);
00129 }
00130
00131 //
00132 // テクスチャを破棄する
00133 //
00134 void Texture::discard()
00135 {
00136     // デフォルトのテクスチャに戻す
00137     glBindTexture(GL_TEXTURE_2D, 0);
00138
00139     // テクスチャを削除する
00140     glDeleteTextures(1, &textureName);
00141     textureName = 0;
00142
00143     // テクスチャのサイズを 0 にする
00144     textureSize = std::array<int, 2>{ 0, 0 };
00145     textureChannels = 0;
00146 }
00147
00148 //
00149 // このテクスチャをマッピングしてメッシュを描画する
00150 //
00151 void Texture::draw(GLsizei width, GLsizei height, int unit) const
00152 {
00153     // テクスチャの縦横比
00154     const auto t{ static_cast<float>(textureSize[0] * height)};
00155
00156     // 表示領域の縦横比
00157     const auto d{ static_cast<float>(textureSize[1] * width)};
00158
00159     // テクスチャのスケール
00160     const std::array<GLfloat, 2> scale{ t > d // テクスチャの縦横比の方が
00161         ? std::array<GLfloat, 2>{ 1.0f, t / d } // 大きければ横方向いっぱい描く
00162         : std::array<GLfloat, 2>{ d / t, 1.0f } // それ以外は縦方向いっぱい描く
00163     };
00164
00165     // このオブジェクトのテクスチャを指定する
00166     glBindTexture(unit);
00167

```

```

00168 // メッシュを描画する
00169 mesh->draw(scale, unit);
00170
00171 // テクスチャの指定を解除する
00172 unbindTexture();
00173 }
00174
00175 //
00176 // テクスチャからピクセルバッファオブジェクトにデータをコピーする
00177 //
00178 void Texture::readPixels(
00179 #if defined(USE_PIXEL_BUFFER_OBJECT)
00180     GLuint buffer) const
00181 #else
00182     std::vector<GLubyte>& buffer)
00183 #endif
00184 {
00185     // 読み出し元のテクスチャを結合する
00186     glBindTexture(GL_TEXTURE_2D, textureName);
00187
00188 #if defined(USE_PIXEL_BUFFER_OBJECT)
00189     // 書き込み先のピクセルバッファオブジェクトを指定する
00190     glBindBuffer(GL_PIXEL_PACK_BUFFER, buffer);
00191
00192 # if defined(GL_GLES_PROTOTYPES)
00193     // 現在使っているフレームバッファオブジェクトを調べる
00194     GLuint currentFbo;
00195     glGetIntegerv(GL_FRAMEBUFFER_BINDING, reinterpret_cast<GLint*>(&currentFbo));
00196
00197     // テクスチャをカラーバッファに持ってフレームバッファオブジェクトを作る
00198     GLuint fbo;
00199     glGenFramebuffers(1, &fbo);
00200     glBindFramebuffer(GL_FRAMEBUFFER, fbo);
00201     static const GLenum attachment{ GL_COLOR_ATTACHMENT0 };
00202     glFramebufferTexture2D(GL_FRAMEBUFFER, attachment, GL_TEXTURE_2D, textureName, 0);
00203     glDrawBuffers(1, &attachment);
00204     glReadBuffer(attachment);
00205
00206     // フレームバッファオブジェクトからピクセルバッファオブジェクトに読み出す
00207     glReadPixels(0, 0, textureSize[0], textureSize[1], channelsToFormat(textureChannels),
00208         GL_UNSIGNED_BYTE, 0);
00209
00210     // 元のフレームバッファオブジェクトに戻す
00211     glBindFramebuffer(GL_FRAMEBUFFER, currentFbo);
00212
00213     // 作ったフレームバッファオブジェクトは削除する
00214     glDeleteFramebuffers(1, &fbo);
00215 # else
00216     // テクスチャの内容をピクセルバッファオブジェクトに書き込む
00217     glGetTexImage(GL_TEXTURE_2D, 0, getFormat(), GL_UNSIGNED_BYTE, 0);
00218 # endif
00219
00220     // 書き込み先のピクセルバッファオブジェクトの結合を解除する
00221     glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00222
00223 #else
00224
00225     // テクスチャの内容をピクセルバッファオブジェクトに書き込む
00226     glGetTexImage(GL_TEXTURE_2D, 0, getFormat(), GL_UNSIGNED_BYTE, buffer.data());
00227
00228 #endif
00229
00230     // 読み出し元のテクスチャの結合を解除する
00231     glBindTexture(GL_TEXTURE_2D, 0);
00232 }
00233
00234 //
00235 // ピクセルバッファオブジェクトからテクスチャにデータをコピーする
00236 //
00237 void Texture::drawPixels(
00238 #if defined(USE_PIXEL_BUFFER_OBJECT)
00239     GLuint
00240 #else
00241     const std::vector<GLubyte>&
00242 #endif
00243     buffer) const
00244 {
00245     // 書き込み先のテクスチャを結合する
00246     glBindTexture(GL_TEXTURE_2D, textureName);
00247
00248 #if defined(USE_PIXEL_BUFFER_OBJECT)
00249
00250     // 読み出し元のピクセルバッファオブジェクトを指定する
00251     glBindBuffer(GL_PIXEL_UNPACK_BUFFER, buffer);
00252
00253     // ピクセルバッファオブジェクトの内容をテクスチャに書き込む
00254     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, textureSize[0], textureSize[1],

```

```

00255     getFormat(), GL_UNSIGNED_BYTE, 0);
00256
00257     // 読み出し元のピクセルバッファオブジェクトの結合を解除する
00258     glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
00259
00260 #else
00261
00262     // ピクセルバッファオブジェクトの内容をテクスチャに書き込む
00263     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, textureSize[0], textureSize[1],
00264         getFormat(), GL_UNSIGNED_BYTE, buffer.data());
00265
00266 #endif
00267
00268     // 書き込み先のテクスチャの結合を解除する
00269     glBindTexture(GL_TEXTURE_2D, 0);
00270 }
00271
00272 // 展開に用いるメッシュの頂点配列オブジェクト
00273 std::shared_ptr<Mesh> Texture::mesh;

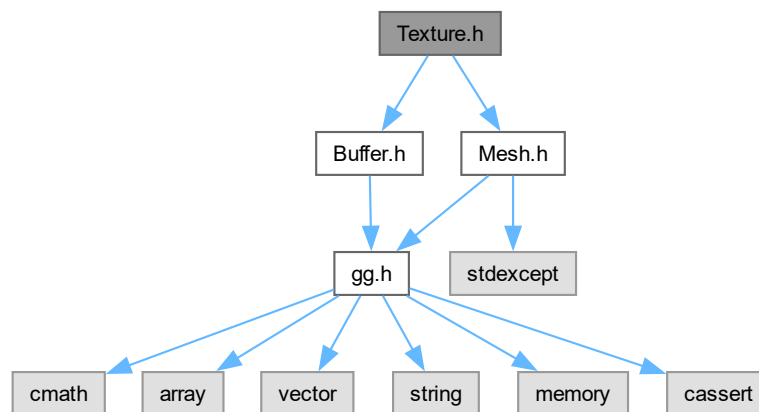
```

9.74 Texture.h ファイル

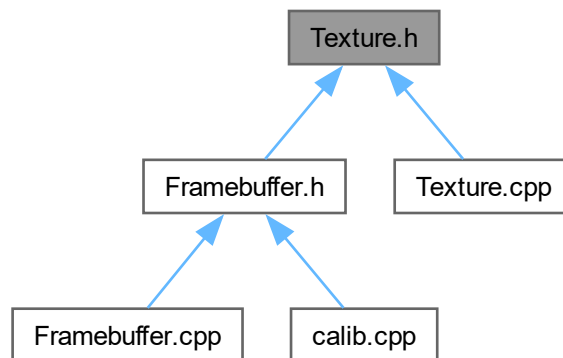
```

#include "Buffer.h"
#include "Mesh.h"
Texture.h の依存先関係図:

```



被依存関係図:



クラス

- class [Texture](#)

9.74.1 詳解

テクスチャクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Texture.h](#) に定義があります。

9.75 Texture.h

[[詳解](#)]

```
00001 #pragma once
00002
00010
00011 // バッファクラス
00012 #include "Buffer.h"
00013
00014 // テクスチャの展開に用いるメッシュ
00015 #include "Mesh.h"
00016
00020 class Texture : public Buffer
00021 {
00023     std::array<int, 2> textureSize;
00024
00026     int textureChannels;
00027
00029     GLuint textureName;
00030
00031 protected:
00032
00034     static std::shared_ptr<Mesh> mesh;
00035
00036 public:
00037
00041     Texture()
```

```

00042     : Buffer{
00043     , textureSize{ 0, 0 }
00044     , textureChannels{ 0 }
00045     , textureName{ 0 }
00046     {
00047     }
00048
00056     Texture(GLsizei width, GLsizei height, int channels);
00057
00063     Texture(const Texture& texture);
00064
00070     Texture(Texture&& texture) noexcept
00071     : Buffer{ std::move(texture) }
00072     , textureSize{ texture.textureSize }
00073     , textureChannels{ texture.textureChannels }
00074     , textureName{ texture.textureName }
00075     {
00076     texture.textureSize = { 0, 0 };
00077     texture.textureChannels = 0;
00078
00079     // ムーブ元のデストラクタでテクスチャが削除されないよう 0 にする
00080     texture.textureName = 0;
00081     }
00082
00086     virtual ~Texture();
00087
00094     Texture& operator=(const Texture& texture);
00095
00102     Texture& operator=(Texture&& texture) noexcept;
00103
00115     virtual void create(GLsizei width, GLsizei height, int channels);
00116
00129     virtual void copy(const Buffer& texture) noexcept;
00130
00134     virtual void discard();
00135
00141     auto getTextureName() const
00142     {
00143     return textureName;
00144     }
00145
00149     virtual const std::array<GLsizei, 2>& getSize() const
00150     {
00151     return textureSize;
00152     }
00153
00157     virtual int getChannels() const
00158     {
00159     return textureChannels;
00160     }
00161
00171     void bindTexture(int unit = 0) const
00172     {
00173     // テクスチャユニットを指定する
00174     glActiveTexture(GL_TEXTURE0 + unit);
00175
00176     // テクスチャを指定する
00177     glBindTexture(GL_TEXTURE_2D, textureName);
00178     }
00179
00183     void unbindTexture() const
00184     {
00185     // デフォルトのテクスチャユニットに戻す
00186     glActiveTexture(GL_TEXTURE0);
00187
00188     // デフォルトのテクスチャに戻す
00189     glBindTexture(GL_TEXTURE_2D, 0);
00190     }
00191
00204     void draw(GLsizei width, GLsizei height, int unit = 0) const;
00205
00214     void readPixels(
00215     #if defined(USE_PIXEL_BUFFER_OBJECT)
00216     GLuint buffer) const
00217     #else
00218     std::vector<GLubyte>& buffer)
00219     #endif
00220     ;
00221
00230     void readPixels(Buffer& buffer)
00231     #if defined(USE_PIXEL_BUFFER_OBJECT)
00232     const
00233     #endif
00234     {
00235     // このテクスチャのバッファから引数のオブジェクトのバッファにコピーする
00236     readPixels();
00237

```

```
00238     // 引数のオブジェクトのサイズをこのテクスチャに合わせてコピーする
00239     buffer.Buffer::copy(*this);
00240 }
00241
00245 void readPixels()
00246 #if defined(USE_PIXEL_BUFFER_OBJECT)
00247     const
00248 #endif
00249 {
00250     // このテクスチャからこのバッファにコピーする
00251     readPixels(getBufferName());
00252 }
00253
00262 void drawPixels(
00263 #if defined(USE_PIXEL_BUFFER_OBJECT)
00264     GLuint
00265 #else
00266     const std::vector<GLubyte>&
00267 #endif
00268     buffer) const;
00269
00278 void drawPixels(const Texture& texture)
00279 {
00280     // このテクスチャのサイズを引数のオブジェクトのサイズに合わせてコピーする
00281     copy(texture);
00282
00283     // バッファからこのテクスチャにコピーする
00284     drawPixels();
00285 }
00286
00295 void drawPixels(const Buffer& buffer)
00296 {
00297     // このテクスチャのサイズを引数のオブジェクトのサイズに合わせてコピーする
00298     copy(buffer);
00299
00300     // バッファからこのテクスチャにコピーする
00301     drawPixels();
00302 }
00303
00307 void drawPixels()
00308 #if defined(USE_PIXEL_BUFFER_OBJECT)
00309     const
00310 #endif
00311 {
00312     // このバッファからこのテクスチャにコピーする
00313     drawPixels(getBufferName());
00314 }
00315 };
```


Index

- .ggError
 - gg, [20](#)
- .ggFBOError
 - gg, [20](#)
- ~Buffer
 - Buffer, [83](#)
- ~Calibration
 - Calibration, [97](#)
- ~CamCv
 - CamCv, [108](#)
- ~CamImage
 - CamImage, [125](#)
- ~CamMf
 - CamMf, [129](#)
- ~Camera
 - Camera, [115](#)
- ~Capture
 - Capture, [133](#)
- ~Compute
 - Compute, [137](#)
- ~Config
 - Config, [139](#)
- ~Expand
 - Expand, [146](#)
- ~Framebuffer
 - Framebuffer, [152](#)
- ~GgApp
 - GgApp, [161](#)
- ~GgBuffer
 - gg::GgBuffer< T >, [166](#)
- ~GgColorTexture
 - gg::GgColorTexture, [174](#)
- ~GgElements
 - gg::GgElements, [178](#)
- ~GgMatrix
 - gg::GgMatrix, [187](#)
- ~GgNormalTexture
 - gg::GgNormalTexture, [234](#)
- ~GgPointShader
 - gg::GgPointShader, [243](#)
- ~GgPoints
 - gg::GgPoints, [238](#)
- ~GgQuaternion
 - gg::GgQuaternion, [259](#)
- ~GgShader
 - gg::GgShader, [318](#)
- ~GgShape
 - gg::GgShape, [321](#)
- ~GgSimpleObj
 - gg::GgSimpleObj, [324](#)
- ~GgSimpleShader
 - gg::GgSimpleShader, [329](#)
- ~GgTexture
 - gg::GgTexture, [348](#)
- ~GgTrackball
 - gg::GgTrackball, [356](#)
- ~GgTriangles
 - gg::GgTriangles, [365](#)
- ~GgUniformBuffer
 - gg::GgUniformBuffer< T >, [369](#)
- ~GgVector
 - gg::GgVector, [383](#)
- ~GgVertexArray
 - gg::GgVertexArray, [400](#)
- ~LightBuffer
 - gg::GgSimpleShader::LightBuffer, [411](#)
- ~MaterialBuffer
 - gg::GgSimpleShader::MaterialBuffer, [428](#)
- ~Menu
 - Menu, [442](#)
- ~Mesh
 - Mesh, [448](#)
- ~Preference
 - Preference, [451](#)
- ~Scene
 - Scene, [453](#)
- ~Texture
 - Texture, [461](#)
- ~Window
 - GgApp::Window, [476](#)
- add
 - gg::GgQuaternion, [259](#), [260](#)
- ambient
 - gg::GgSimpleShader::Light, [407](#)
 - gg::GgSimpleShader::Material, [424](#)
- begin
 - gg::GgTrackball, [356](#)
- bind
 - gg::GgBuffer< T >, [166](#)
 - gg::GgTexture, [348](#)
 - gg::GgUniformBuffer< T >, [370](#)
 - gg::GgVertexArray, [400](#)
- bindBuffer
 - Buffer, [83](#)
- bindFramebuffer
 - Framebuffer, [152](#)
- BindingPoints

- gg, 20
- bindTexture
 - Texture, 461
- Buffer, 79
 - ~Buffer, 83
 - bindBuffer, 83
 - Buffer, 80–82
 - channelsToFormat, 83
 - copy, 84
 - copyBuffer, 85
 - create, 86
 - discard, 87
 - formatToChannels, 87
 - getAspect, 87
 - getBufferName, 88
 - getChannels, 88
 - getFormat, 89
 - getHeight, 89
 - getSize, 90
 - getWidth, 91
 - map, 91
 - operator=, 92
 - resize, 93, 94
 - unbindBuffer, 95
 - unmap, 95
- Buffer.cpp, 499
- Buffer.h, 502
 - USE_PIXEL_BUFFER_OBJECT, 503
- buildShader
 - Preference, 451
- calib.cpp, 505
 - CONFIG_FILE, 506
- calibrate
 - Calibration, 98
- Calibration, 96
 - ~Calibration, 97
 - calibrate, 98
 - Calibration, 97
 - createBoard, 98
 - detectBoard, 98
 - detectMarkers, 99
 - dictionaryList, 105
 - discardCorners, 99
 - drawBoard, 100
 - finished, 100
 - getAllMarkerPoses, 100
 - getCameraMatrix, 101
 - getCornersCount, 101
 - getDistortionCoefficients, 101
 - getReprojectionError, 101
 - getSampleCount, 102
 - getTotalCount, 102
 - loadParameters, 102
 - operator=, 102
 - recordCorners, 103
 - RvecTvecToPose, 103
 - saveParameters, 104
 - setDictionary, 104
- Calibration.cpp, 508
 - USE_RODRIGUES, 508
- Calibration.h, 514
- CamCv, 106
 - ~CamCv, 108
 - CamCv, 108
 - close, 108
 - decreaseExposure, 108
 - decreaseGain, 108
 - getCodec, 109
 - getFps, 110
 - getPosition, 110
 - increaseExposure, 110
 - increaseGain, 110
 - open, 111
 - setExposure, 112
 - setGain, 112
 - setPosition, 113
- CamCv.h, 517
- Camera, 113
 - ~Camera, 115
 - Camera, 115
 - capture, 116
 - captured, 121
 - channels, 121
 - close, 116
 - decreaseExposure, 117
 - decreaseGain, 117
 - frame, 121
 - getChannels, 117
 - getFps, 117
 - getFrames, 117
 - getHeight, 117
 - getSize, 118
 - getWidth, 118
 - height, 121
 - image, 121
 - in, 121
 - increaseExposure, 118
 - increaseGain, 118
 - interval, 121
 - isRunning, 118
 - mtx, 121
 - operator=, 118
 - out, 121
 - running, 121
 - start, 119
 - stop, 119
 - thr, 121
 - total, 122
 - transmit, 120
 - width, 122
- Camera.h, 521
- CamImage, 122
 - ~CamImage, 125
 - CamImage, 124
 - isOpened, 125
 - load, 125

- open, 126
- save, 126
- CamImage.h, 525
- CamMf, 127
 - ~CamMf, 129
 - CamMf, 129
 - capture, 130
 - close, 130
 - getDeviceList, 130
 - getFormatList, 131
 - open, 131
 - select, 131
- CamMf.cpp, 528
 - HandleStreamChange, 529
 - SafeRelease, 529
 - SubTypeToName, 530
- CamMf.h, 541
- Capture, 132
 - ~Capture, 133
 - Capture, 133
 - close, 133
 - getFps, 133
 - getSize, 134
 - isImage, 134
 - isOpend, 134
 - openDevice, 134
 - openImage, 135
 - openMovie, 135
 - operator bool, 135
 - retrieve, 136
 - start, 136
 - stop, 136
- capture
 - Camera, 116
 - CamMf, 130
- Capture.cpp, 544
- Capture.h, 547
- captured
 - Camera, 121
- center
 - Intrinsics, 406
- channels
 - Camera, 121
- channelsToFormat
 - Buffer, 83
- checkerLength
 - Settings, 455
- close
 - CamCv, 108
 - Camera, 116
 - CamMf, 130
 - Capture, 133
- Compute, 137
 - ~Compute, 137
 - Compute, 137
 - execute, 138
 - get, 138
 - use, 138
- Compute.h, 549
- Config, 138
 - ~Config, 139
 - Config, 139
 - getCheckerLength, 140
 - getDictionaryName, 140
 - getHeight, 140
 - getInitialImage, 141
 - getMarkerLength, 141
 - getTitle, 141
 - getWidth, 142
 - initialize, 142
 - load, 142
 - Menu, 144
 - save, 143
- Config.cpp, 550
- Config.h, 554
- CONFIG.FILE
 - calib.cpp, 506
- conjugate
 - gg::GgQuaternion, 261
- copy
 - Buffer, 84
 - Framebuffer, 152
 - gg::GgBuffer< T >, 166
 - gg::GgUniformBuffer< T >, 370
 - Texture, 462
- copyBuffer
 - Buffer, 85
- create
 - Buffer, 86
 - Framebuffer, 153
 - Texture, 463
- createBoard
 - Calibration, 98
- decreaseExposure
 - CamCv, 108
 - Camera, 117
- decreaseGain
 - CamCv, 108
 - Camera, 117
- defaultEuler
 - Settings, 455
- defaultFocal
 - Settings, 455
- defaultFocalRange
 - Settings, 455
- detectBoard
 - Calibration, 98
 - Menu, 446
- detectMarker
 - Menu, 446
- detectMarkers
 - Calibration, 99
- dictionaryList
 - Calibration, 105
- dictionaryName
 - Settings, 456

- diffuse
 - gg::GgSimpleShader::Light, [407](#)
 - gg::GgSimpleShader::Material, [424](#)
- discard
 - Buffer, [87](#)
 - Framebuffer, [154](#)
 - Texture, [464](#)
- discardCorners
 - Calibration, [99](#)
- distance3
 - gg::GgVector, [383](#)
- distance4
 - gg::GgVector, [384](#)
- divide
 - gg::GgQuaternion, [262](#), [263](#)
- dot3
 - gg::GgVector, [384](#)
- dot4
 - gg::GgVector, [385](#)
- draw
 - gg::GgElements, [178](#)
 - gg::GgPoints, [238](#)
 - gg::GgShape, [321](#)
 - gg::GgSimpleObj, [324](#)
 - gg::GgTriangles, [365](#)
 - Menu, [442](#)
 - Mesh, [448](#)
 - Scene, [454](#)
 - Texture, [465](#)
- drawBoard
 - Calibration, [100](#)
- drawMesh
 - Mesh, [448](#)
- drawPixels
 - Texture, [465–467](#)
- end
 - gg::GgTrackball, [356](#)
- euler
 - gg::GgQuaternion, [264](#), [265](#)
 - Settings, [456](#)
- execute
 - Compute, [138](#)
- Expand, [144](#)
 - ~Expand, [146](#)
 - Expand, [145](#)
 - getProgram, [146](#)
 - operator=, [146](#)
 - setup, [146](#)
- Expand.cpp, [557](#)
- Expand.h, [559](#)
- fill
 - gg::GgUniformBuffer< T >, [371](#)
- finished
 - Calibration, [100](#)
- focal
 - Settings, [456](#)
- focalRange
 - Settings, [456](#)
- formatToChannels
 - Buffer, [87](#)
- fov
 - Intrinsics, [406](#)
- fps
 - Intrinsics, [406](#)
- frame
 - Camera, [121](#)
- Framebuffer, [147](#)
 - ~Framebuffer, [152](#)
 - bindFramebuffer, [152](#)
 - copy, [152](#)
 - create, [153](#)
 - discard, [154](#)
 - Framebuffer, [150](#), [151](#)
 - getChannels, [154](#)
 - getFramebufferName, [155](#)
 - getSize, [155](#)
 - operator=, [155](#), [156](#)
 - show, [156](#)
 - unbindFramebuffer, [157](#)
 - update, [157](#), [158](#)
- Framebuffer.cpp, [561](#)
- Framebuffer.h, [565](#)
- frustum
 - gg::GgMatrix, [187](#)
- get
 - Compute, [138](#)
 - gg::GgMatrix, [187](#), [188](#)
 - gg::GgPointShader, [243](#)
 - gg::GgQuaternion, [266](#)
 - gg::GgShader, [318](#)
 - gg::GgShape, [321](#)
 - gg::GgSimpleObj, [324](#)
 - gg::GgTrackball, [357](#)
 - gg::GgVertexArray, [400](#)
 - GgApp::Window, [476](#)
- getAllMarkerPoses
 - Calibration, [100](#)
- getAltArrowX
 - GgApp::Window, [476](#)
- getAltArrowY
 - GgApp::Window, [477](#)
- getAltArrow
 - GgApp::Window, [477](#)
- getAnyList
 - Menu.cpp, [734](#)
- getArrow
 - GgApp::Window, [478](#)
- getArrowX
 - GgApp::Window, [479](#)
- getArrowY
 - GgApp::Window, [480](#)
- getAspect
 - Buffer, [87](#)
 - GgApp::Window, [480](#)
- getBuffer

- gg::GgBuffer< T >, 166
- gg::GgPoints, 239
- gg::GgTriangles, 366
- gg::GgUniformBuffer< T >, 372
- getBufferName
 - Buffer, 88
- getCameraMatrix
 - Calibration, 101
- getChannels
 - Buffer, 88
 - Camera, 117
 - Framebuffer, 154
 - Texture, 468
- getCheckerLength
 - Config, 140
 - Menu, 443
- getCodec
 - CamCv, 109
- getConjugateMatrix
 - gg::GgQuaternion, 267, 268
- getControlArrow
 - GgApp::Window, 481
- getControlArrowX
 - GgApp::Window, 481
- getControlArrowY
 - GgApp::Window, 482
- getCornersCount
 - Calibration, 101
- getCount
 - gg::GgBuffer< T >, 167
 - gg::GgPoints, 239
 - gg::GgTriangles, 366
 - gg::GgUniformBuffer< T >, 373
- getDescription
 - Preference, 451
- getDeviceList
 - CamMf, 130
- getDictionaryName
 - Config, 140
- getDistortionCoefficients
 - Calibration, 101
- getFboHeight
 - GgApp::Window, 482
- getFboSize
 - GgApp::Window, 483
- getFboWidth
 - GgApp::Window, 483
- getFocal
 - Settings, 455
- getFormat
 - Buffer, 89
- getFormatList
 - CamMf, 131
- getFps
 - CamCv, 110
 - Camera, 117
 - Capture, 133
- getFramebufferName
 - Framebuffer, 155
- getFrames
 - Camera, 117
- getHeight
 - Buffer, 89
 - Camera, 117
 - Config, 140
 - gg::GgTexture, 348
 - GgApp::Window, 484
- getIndexBuffer
 - gg::GgElements, 179
- getIndexCount
 - gg::GgElements, 179
- getInitialImage
 - Config, 141
- getIntrinsics
 - Preference, 451
- getKey
 - GgApp::Window, 484
- getLastKey
 - GgApp::Window, 484
- getMarkerLength
 - Config, 141
 - Menu, 443
- getMatrix
 - gg::GgQuaternion, 268, 269
 - gg::GgTrackball, 357
- getMenubarHeight
 - Menu, 443
- getMode
 - gg::GgShape, 322
- getMouse
 - GgApp::Window, 484, 485
- getMouseX
 - GgApp::Window, 485
- getMouseY
 - GgApp::Window, 485
- getPose
 - Menu, 444
- getPosition
 - CamCv, 110
- getProgram
 - Expand, 146
- getQuaternion
 - gg::GgTrackball, 357
- getReprojectionError
 - Calibration, 101
- getRotation
 - GgApp::Window, 485
- getRotationMatrix
 - GgApp::Window, 486
- getSampleCount
 - Calibration, 102
- getScale
 - gg::GgTrackball, 357, 358
- getScrollMatrix
 - GgApp::Window, 486
- getShader

- Preference, 451
- getShiftArrow
 - GgApp::Window, 486
- getShiftArrowX
 - GgApp::Window, 487
- getShiftArrowY
 - GgApp::Window, 487
- getSize
 - Buffer, 90
 - Camera, 118
 - Capture, 134
 - Framebuffer, 155
 - gg::GgTexture, 348
 - GgApp::Window, 488
 - Texture, 468
- getStart
 - gg::GgTrackball, 358
- getStride
 - gg::GgBuffer< T >, 167
 - gg::GgUniformBuffer< T >, 373
- getString
 - parseconfig.h, 757, 758
- getTarget
 - gg::GgBuffer< T >, 168
 - gg::GgUniformBuffer< T >, 374
- getTexture
 - gg::GgTexture, 349
- getTextureName
 - Texture, 468
- getTitle
 - Config, 141
- getTotalCount
 - Calibration, 102
- getTranslation
 - GgApp::Window, 489
- getTranslationMatrix
 - GgApp::Window, 489
- getUserPointer
 - GgApp::Window, 489
- getValue
 - parseconfig.h, 758, 759
- getWheel
 - GgApp::Window, 489, 490
- getWheelX
 - GgApp::Window, 490
- getWheelY
 - GgApp::Window, 490
- getWidth
 - Buffer, 91
 - Camera, 118
 - Config, 142
 - gg::GgTexture, 349
 - GgApp::Window, 490
- gg, 17
 - _ggError, 20
 - _ggFBOError, 20
 - BindingPoints, 20
 - ggArraysObj, 20
 - ggBufferAlignment, 77
 - ggConjugate, 21
 - ggCreateComputeShader, 21
 - ggCreateNormalMap, 22
 - ggCreateShader, 23
 - ggCross, 24
 - ggDistance3, 25
 - ggDistance4, 26, 27
 - ggDot3, 27, 28
 - ggDot4, 29
 - ggElementsMesh, 30
 - ggElementsObj, 31
 - ggElementsSphere, 32
 - ggEllipse, 32
 - ggEulerQuaternion, 33, 34
 - ggFrustum, 35
 - ggIdentity, 35
 - ggIdentityQuaternion, 36
 - ggInit, 36
 - ggInvert, 37
 - ggLength3, 38
 - ggLength4, 39, 40
 - ggLoadComputeShader, 40
 - ggLoadHeight, 41
 - ggLoadImage, 42
 - ggLoadShader, 43
 - ggLoadSimpleObj, 44, 45
 - ggLoadTexture, 45
 - ggLookat, 46, 47
 - ggMatrixQuaternion, 48, 49
 - ggNorm, 49
 - ggNormal, 50
 - ggNormalize, 50
 - ggNormalize3, 51–53
 - ggNormalize4, 53–55
 - ggOrthogonal, 55
 - ggPerspective, 56
 - ggPointsCube, 56
 - ggPointsSphere, 57
 - ggQuaternionMatrix, 57
 - ggQuaternionTransposeMatrix, 58
 - ggReadImage, 58
 - ggRectangle, 59
 - ggRotate, 60–62
 - ggRotateQuaternion, 62, 63
 - ggRotateX, 64
 - ggRotateY, 65
 - ggRotateZ, 65
 - ggSaveColor, 66
 - ggSaveDepth, 66
 - ggSaveTga, 67
 - ggScale, 68, 69
 - ggSlerp, 70, 71
 - ggTranslate, 72, 73
 - ggTranspose, 74
 - LightBindingPoint, 20
 - MaterialBindingPoint, 20
 - operator+, 75

- operator-, 76
- operator/, 76
- operator*, 75
- gg.cpp, 567
- gg.h, 643
 - ggError, 647
 - ggFBOError, 647
 - pathChar, 648
 - pathString, 648
 - TCharToUtf8, 648
 - Utf8ToTChar, 648
- gg::GgBuffer< T >, 164
 - ~GgBuffer, 166
 - bind, 166
 - copy, 166
 - getBuffer, 166
 - getCount, 167
 - getStride, 167
 - getTarget, 168
 - GgBuffer, 165
 - map, 168, 169
 - operator=, 169, 170
 - read, 170
 - send, 171
 - unbind, 172
 - unmap, 172
- gg::GgColorTexture, 172
 - ~GgColorTexture, 174
 - GgColorTexture, 173
 - load, 174, 175
- gg::GgElements, 176
 - ~GgElements, 178
 - draw, 178
 - getIndexBuffer, 179
 - getIndexCount, 179
 - GgElements, 177, 178
 - load, 180
 - send, 181
- gg::GgMatrix, 181
 - ~GgMatrix, 187
 - frustum, 187
 - get, 187, 188
 - GgMatrix, 184–186
 - invert, 189
 - loadFrustum, 189
 - loadIdentity, 190
 - loadInvert, 190, 191
 - loadLookat, 191–193
 - loadNormal, 194
 - loadOrthogonal, 195
 - loadPerspective, 196
 - loadRotate, 196–199
 - loadRotateX, 200
 - loadRotateY, 200
 - loadRotateZ, 201
 - loadScale, 201, 202
 - loadTranslate, 203, 204
 - loadTranspose, 205, 206
 - lookat, 206, 207
 - normal, 208
 - operator+, 212, 213
 - operator+=, 213, 214
 - operator-, 215
 - operator-=, 216
 - operator/, 217, 218
 - operator/=: 218, 219
 - operator=, 220
 - operator*, 209, 210
 - operator*=: 211
 - orthogonal, 221
 - perspective, 221
 - projection, 222, 223
 - rotate, 223–225
 - rotateX, 226
 - rotateY, 227
 - rotateZ, 227
 - scale, 228, 229
 - translate, 230, 231
 - transpose, 232
- gg::GgNormalTexture, 233
 - ~GgNormalTexture, 234
 - GgNormalTexture, 233, 234
 - load, 234, 235
- gg::GgPoints, 236
 - ~GgPoints, 238
 - draw, 238
 - getBuffer, 239
 - getCount, 239
 - GgPoints, 237
 - load, 239
 - operator bool, 240
 - operator!, 240
 - send, 240
- gg::GgPointShader, 241
 - ~GgPointShader, 243
 - get, 243
 - GgPointShader, 242, 243
 - load, 244
 - loadMatrix, 245, 246
 - loadModelviewMatrix, 247
 - loadProjectionMatrix, 248
 - unuse, 249
 - use, 249–252
- gg::GgQuaternion, 252
 - ~GgQuaternion, 259
 - add, 259, 260
 - conjugate, 261
 - divide, 262, 263
 - euler, 264, 265
 - get, 266
 - getConjugateMatrix, 267, 268
 - getMatrix, 268, 269
 - GgQuaternion, 256–258
 - invert, 270
 - loadAdd, 271, 272
 - loadConjugate, 273, 274

- loadDivide, 274–276
- loadEuler, 277
- loadIdentity, 278
- loadInvert, 279
- loadMatrix, 280, 281
- loadMultiply, 281–283
- loadNormalize, 284
- loadRotate, 285, 286
- loadRotateX, 287
- loadRotateY, 287
- loadRotateZ, 288
- loadSlerp, 289–291
- loadSubtract, 292, 293
- multiply, 294–296
- norm, 296
- normalize, 297
- operator+, 300
- operator+=, 301
- operator-, 302
- operator-=, 303
- operator/, 304
- operator/=: 305, 306
- operator=, 306–308
- operator*, 298
- operator*=, 299
- rotate, 308, 309
- rotateX, 310
- rotateY, 311
- rotateZ, 311
- slerp, 312
- subtract, 313, 314
- gg::GgShader, 315
 - ~GgShader, 318
 - get, 318
 - GgShader, 316–318
 - operator=, 318, 319
 - unuse, 319
 - use, 319
- gg::GgShape, 320
 - ~GgShape, 321
 - draw, 321
 - get, 321
 - getMode, 322
 - GgShape, 320
 - operator bool, 322
 - operator!, 322
 - setMode, 323
- gg::GgSimpleObj, 323
 - ~GgSimpleObj, 324
 - draw, 324
 - get, 324
 - GgSimpleObj, 323
 - operator bool, 325
 - operator!, 325
- gg::GgSimpleShader, 325
 - ~GgSimpleShader, 329
 - GgSimpleShader, 327–329
 - load, 329, 330
 - loadMatrix, 331–333
 - loadModelviewMatrix, 334, 335
 - operator=, 336
 - use, 337–344
- gg::GgSimpleShader::Light, 406
 - ambient, 407
 - diffuse, 407
 - position, 407
 - specular, 408
- gg::GgSimpleShader::LightBuffer, 408
 - ~LightBuffer, 411
 - LightBuffer, 409, 410
 - load, 411, 412
 - loadAmbient, 412–414
 - loadColor, 415
 - loadDiffuse, 415, 416
 - loadPosition, 417–419
 - loadSpecular, 420, 421
 - select, 422
- gg::GgSimpleShader::Material, 423
 - ambient, 424
 - diffuse, 424
 - shininess, 424
 - specular, 425
- gg::GgSimpleShader::MaterialBuffer, 425
 - ~MaterialBuffer, 428
 - load, 429
 - loadAmbient, 430, 431
 - loadAmbientAndDiffuse, 432, 433
 - loadDiffuse, 434, 435
 - loadShininess, 436, 437
 - loadSpecular, 437–439
 - MaterialBuffer, 427, 428
 - select, 439
- gg::GgTexture, 345
 - ~GgTexture, 348
 - bind, 348
 - getHeight, 348
 - getSize, 348
 - getTexture, 349
 - getWidth, 349
 - GgTexture, 346, 347
 - operator=, 349, 350
 - swapRandB, 350
 - unbind, 351
- gg::GgTrackball, 351
 - ~GgTrackball, 356
 - begin, 356
 - end, 356
 - get, 357
 - getMatrix, 357
 - getQuaternion, 357
 - getScale, 357, 358
 - getStart, 358
 - GgTrackball, 355
 - motion, 359
 - operator=, 359
 - region, 360

- reset, 361
- rotate, 362
- gg::GgTriangles, 362
 - ~GgTriangles, 365
 - draw, 365
 - getBuffer, 366
 - getCount, 366
 - GgTriangles, 364
 - load, 366
 - send, 367
- gg::GgUniformBuffer< T >, 367
 - ~GgUniformBuffer, 369
 - bind, 370
 - copy, 370
 - fill, 371
 - getBuffer, 372
 - getCount, 373
 - getStride, 373
 - getTarget, 374
 - GgUniformBuffer, 368, 369
 - load, 374, 375
 - map, 376
 - read, 376
 - send, 377
 - unbind, 378
 - unmap, 378
- gg::GgVector, 379
 - ~GgVector, 383
 - distance3, 383
 - distance4, 384
 - dot3, 384
 - dot4, 385
 - GgVector, 380–383
 - length3, 385
 - length4, 386
 - normalize3, 386
 - normalize4, 387
 - operator+, 389
 - operator+=", 390
 - operator-, 391
 - operator-=, 392
 - operator/, 393
 - operator/=", 394
 - operator=, 395
 - operator*, 387
 - operator*=", 388
- gg::GgVertex, 396
 - GgVertex, 396, 397
 - normal, 398
 - position, 398
- gg::GgVertexArray, 398
 - ~GgVertexArray, 400
 - bind, 400
 - get, 400
 - GgVertexArray, 399, 400
 - operator=, 401
- GG_BUTTON_COUNT
 - GgApp.h, 718
- GG_INTERFACE_COUNT
 - GgApp.h, 718
- GG_USE_IMGUI
 - GgApp.h, 718
- GgApp, 159
 - ~GgApp, 161
 - GgApp, 159, 160
 - main, 161
 - operator=, 163
- GgApp.cpp, 703
- GgApp.h, 717
 - GG_BUTTON_COUNT, 718
 - GG_INTERFACE_COUNT, 718
 - GG_USE_IMGUI, 718
- GgApp::Window, 472
 - ~Window, 476
 - get, 476
 - getAltArrowX, 476
 - getAltArrowY, 477
 - getAltArrow, 477
 - getArrow, 478
 - getArrowX, 479
 - getArrowY, 480
 - getAspect, 480
 - getControlArrow, 481
 - getControlArrowX, 481
 - getControlArrowY, 482
 - getFboHeight, 482
 - getFboSize, 483
 - getFboWidth, 483
 - getHeight, 484
 - getKey, 484
 - getLastKey, 484
 - getMouse, 484, 485
 - getMouseX, 485
 - getMouseY, 485
 - getRotation, 485
 - getRotationMatrix, 486
 - getScrollMatrix, 486
 - getShiftArrow, 486
 - getShiftArrowX, 487
 - getShiftArrowY, 487
 - getSize, 488
 - getTranslation, 489
 - getTranslationMatrix, 489
 - getUserPointer, 489
 - getWheel, 489, 490
 - getWheelX, 490
 - getWheelY, 490
 - getWidth, 490
 - operator bool, 491
 - operator=, 491, 492
 - reset, 492
 - resetRotation, 492
 - resetTranslation, 493
 - restoreViewport, 493
 - selectInterface, 493
 - setClose, 494

- setKeyboardFunc, [494](#)
- setMouseFunc, [494](#)
- setResizeFunc, [495](#)
- setUserPointer, [495](#)
- setVelocity, [495](#)
- setWheelFunc, [496](#)
- shouldClose, [496](#)
- swapBuffers, [496](#)
- updateViewport, [497](#)
- Window, [474](#), [475](#)
- ggArraysObj
 - gg, [20](#)
- GgBuffer
 - gg::GgBuffer< T >, [165](#)
- ggBufferAlignment
 - gg, [77](#)
- GgColorTexture
 - gg::GgColorTexture, [173](#)
- ggConjugate
 - gg, [21](#)
- ggCreateComputeShader
 - gg, [21](#)
- ggCreateNormalMap
 - gg, [22](#)
- ggCreateShader
 - gg, [23](#)
- ggCross
 - gg, [24](#)
- ggDistance3
 - gg, [25](#)
- ggDistance4
 - gg, [26](#), [27](#)
- ggDot3
 - gg, [27](#), [28](#)
- ggDot4
 - gg, [29](#)
- GgElements
 - gg::GgElements, [177](#), [178](#)
- ggElementsMesh
 - gg, [30](#)
- ggElementsObj
 - gg, [31](#)
- ggElementsSphere
 - gg, [32](#)
- ggEllipse
 - gg, [32](#)
- ggError
 - gg.h, [647](#)
- ggEulerQuaternion
 - gg, [33](#), [34](#)
- ggFBOError
 - gg.h, [647](#)
- ggFrustum
 - gg, [35](#)
- ggIdentity
 - gg, [35](#)
- ggIdentityQuaternion
 - gg, [36](#)
- ggInit
 - gg, [36](#)
- ggInvert
 - gg, [37](#)
- ggLength3
 - gg, [38](#)
- ggLength4
 - gg, [39](#), [40](#)
- ggLoadComputeShader
 - gg, [40](#)
- ggLoadHeight
 - gg, [41](#)
- ggLoadImage
 - gg, [42](#)
- ggLoadShader
 - gg, [43](#)
- ggLoadSimpleObj
 - gg, [44](#), [45](#)
- ggLoadTexture
 - gg, [45](#)
- ggLookat
 - gg, [46](#), [47](#)
- GgMatrix
 - gg::GgMatrix, [184–186](#)
- ggMatrixQuaternion
 - gg, [48](#), [49](#)
- ggNorm
 - gg, [49](#)
- ggNormal
 - gg, [50](#)
- ggNormalize
 - gg, [50](#)
- ggNormalize3
 - gg, [51–53](#)
- ggNormalize4
 - gg, [53–55](#)
- GgNormalTexture
 - gg::GgNormalTexture, [233](#), [234](#)
- ggOrthogonal
 - gg, [55](#)
- ggPerspective
 - gg, [56](#)
- GgPoints
 - gg::GgPoints, [237](#)
- ggPointsCube
 - gg, [56](#)
- GgPointShader
 - gg::GgPointShader, [242](#), [243](#)
- ggPointsSphere
 - gg, [57](#)
- GgQuaternion
 - gg::GgQuaternion, [256–258](#)
- ggQuaternionMatrix
 - gg, [57](#)
- ggQuaternionTransposeMatrix
 - gg, [58](#)
- ggReadImage
 - gg, [58](#)

- ggRectangle
 - gg, [59](#)
- ggRotate
 - gg, [60–62](#)
- ggRotateQuaternion
 - gg, [62](#), [63](#)
- ggRotateX
 - gg, [64](#)
- ggRotateY
 - gg, [65](#)
- ggRotateZ
 - gg, [65](#)
- ggSaveColor
 - gg, [66](#)
- ggSaveDepth
 - gg, [66](#)
- ggSaveTga
 - gg, [67](#)
- ggScale
 - gg, [68](#), [69](#)
- GgShader
 - gg::GgShader, [316–318](#)
- GgShape
 - gg::GgShape, [320](#)
- GgSimpleObj
 - gg::GgSimpleObj, [323](#)
- GgSimpleShader
 - gg::GgSimpleShader, [327–329](#)
- ggSlerp
 - gg, [70](#), [71](#)
- GgTexture
 - gg::GgTexture, [346](#), [347](#)
- GgTrackball
 - gg::GgTrackball, [355](#)
- ggTranslate
 - gg, [72](#), [73](#)
- ggTranspose
 - gg, [74](#)
- GgTriangles
 - gg::GgTriangles, [364](#)
- GgUniformBuffer
 - gg::GgUniformBuffer< T >, [368](#), [369](#)
- GgVector
 - gg::GgVector, [380–383](#)
- GgVertex
 - gg::GgVertex, [396](#), [397](#)
- GgVertexArray
 - gg::GgVertexArray, [399](#), [400](#)
- HandleStreamChange
 - CamMf.cpp, [529](#)
- HEADER_STR
 - main.cpp, [731](#)
- height
 - Camera, [121](#)
- image
 - Camera, [121](#)
- imageFilter
 - Menu.cpp, [735](#)
- in
 - Camera, [121](#)
- increaseExposure
 - CamCv, [110](#)
 - Camera, [118](#)
- increaseGain
 - CamCv, [110](#)
 - Camera, [118](#)
- initialize
 - Config, [142](#)
- interval
 - Camera, [121](#)
- Intrinsics, [402](#)
 - center, [406](#)
 - fov, [406](#)
 - fps, [406](#)
 - Intrinsics, [403](#)
 - sensorSize, [406](#)
 - setCenter, [404](#)
 - setFov, [404](#), [405](#)
 - setFps, [405](#)
 - setSize, [405](#)
 - size, [406](#)
- Intrinsics.cpp, [727](#)
- Intrinsics.h, [728](#)
- invert
 - gg::GgMatrix, [189](#)
 - gg::GgQuaternion, [270](#)
- isImage
 - Capture, [134](#)
- isOpend
 - Capture, [134](#)
- isOpened
 - CamImage, [125](#)
- isRunning
 - Camera, [118](#)
- jsonFilter
 - Menu.cpp, [735](#)
- length3
 - gg::GgVector, [385](#)
- length4
 - gg::GgVector, [386](#)
- light
 - Scene, [454](#)
- LightBindingPoint
 - gg, [20](#)
- LightBuffer
 - gg::GgSimpleShader::LightBuffer, [409](#), [410](#)
- load
 - CamImage, [125](#)
 - Config, [142](#)
 - gg::GgColorTexture, [174](#), [175](#)
 - gg::GgElements, [180](#)
 - gg::GgNormalTexture, [234](#), [235](#)
 - gg::GgPoints, [239](#)
 - gg::GgPointShader, [244](#)

- gg::GgSimpleShader, 329, 330
- gg::GgSimpleShader::LightBuffer, 411, 412
- gg::GgSimpleShader::MaterialBuffer, 429
- gg::GgTriangles, 366
- gg::GgUniformBuffer< T >, 374, 375
- loadAdd
 - gg::GgQuaternion, 271, 272
- loadAmbient
 - gg::GgSimpleShader::LightBuffer, 412–414
 - gg::GgSimpleShader::MaterialBuffer, 430, 431
- loadAmbientAndDiffuse
 - gg::GgSimpleShader::MaterialBuffer, 432, 433
- loadColor
 - gg::GgSimpleShader::LightBuffer, 415
- loadConjugate
 - gg::GgQuaternion, 273, 274
- loadDiffuse
 - gg::GgSimpleShader::LightBuffer, 415, 416
 - gg::GgSimpleShader::MaterialBuffer, 434, 435
- loadDivide
 - gg::GgQuaternion, 274–276
- loadEuler
 - gg::GgQuaternion, 277
- loadFrustum
 - gg::GgMatrix, 189
- loadIdentity
 - gg::GgMatrix, 190
 - gg::GgQuaternion, 278
- loadInvert
 - gg::GgMatrix, 190, 191
 - gg::GgQuaternion, 279
- loadLookat
 - gg::GgMatrix, 191–193
- loadMatrix
 - gg::GgPointShader, 245, 246
 - gg::GgQuaternion, 280, 281
 - gg::GgSimpleShader, 331–333
- loadModelviewMatrix
 - gg::GgPointShader, 247
 - gg::GgSimpleShader, 334, 335
- loadMultiply
 - gg::GgQuaternion, 281–283
- loadNormal
 - gg::GgMatrix, 194
- loadNormalize
 - gg::GgQuaternion, 284
- loadOrthogonal
 - gg::GgMatrix, 195
- loadParameters
 - Calibration, 102
- loadPerspective
 - gg::GgMatrix, 196
- loadPosition
 - gg::GgSimpleShader::LightBuffer, 417–419
- loadProjectionMatrix
 - gg::GgPointShader, 248
- loadRotate
 - gg::GgMatrix, 196–199
- gg::GgQuaternion, 285, 286
- loadRotateX
 - gg::GgMatrix, 200
 - gg::GgQuaternion, 287
- loadRotateY
 - gg::GgMatrix, 200
 - gg::GgQuaternion, 287
- loadRotateZ
 - gg::GgMatrix, 201
 - gg::GgQuaternion, 288
- loadScale
 - gg::GgMatrix, 201, 202
- loadShininess
 - gg::GgSimpleShader::MaterialBuffer, 436, 437
- loadSlerp
 - gg::GgQuaternion, 289–291
- loadSpecular
 - gg::GgSimpleShader::LightBuffer, 420, 421
 - gg::GgSimpleShader::MaterialBuffer, 437–439
- loadSubtract
 - gg::GgQuaternion, 292, 293
- loadTranslate
 - gg::GgMatrix, 203, 204
- loadTranspose
 - gg::GgMatrix, 205, 206
- lookat
 - gg::GgMatrix, 206, 207
- main
 - GgApp, 161
 - main.cpp, 731
- main.cpp, 730
 - HEADER_STR, 731
 - main, 731
- map
 - Buffer, 91
 - gg::GgBuffer< T >, 168, 169
 - gg::GgUniformBuffer< T >, 376
- markerLength
 - Settings, 456
- MaterialBindingPoint
 - gg, 20
- MaterialBuffer
 - gg::GgSimpleShader::MaterialBuffer, 427, 428
- Menu, 440
 - ~Menu, 442
 - Config, 144
 - detectBoard, 446
 - detectMarker, 446
 - draw, 442
 - getCheckerLength, 443
 - getMarkerLength, 443
 - getMenubarHeight, 443
 - getPose, 444
 - Menu, 441
 - operator bool, 444
 - operator=, 444
 - savelmage, 445
 - setSize, 445

- setup, 446
- Menu.cpp, 733
 - getAnyList, 734
 - imageFilter, 735
 - jsonFilter, 735
 - movieFilter, 735
- Menu.h, 749
- Mesh, 447
 - ~Mesh, 448
 - draw, 448
 - drawMesh, 448
 - Mesh, 447
 - operator=, 448
- mesh
 - Texture, 472
- Mesh.h, 752
- motion
 - gg::GgTrackball, 359
- movieFilter
 - Menu.cpp, 735
- mtx
 - Camera, 121
- multiply
 - gg::GgQuaternion, 294–296
- norm
 - gg::GgQuaternion, 296
- normal
 - gg::GgMatrix, 208
 - gg::GgVertex, 398
- normalize
 - gg::GgQuaternion, 297
- normalize3
 - gg::GgVector, 386
- normalize4
 - gg::GgVector, 387
- open
 - CamCv, 111
 - CamImage, 126
 - CamMf, 131
- opencv_link.h, 754
- openDevice
 - Capture, 134
- openImage
 - Capture, 135
- openMovie
 - Capture, 135
- operator bool
 - Capture, 135
 - gg::GgPoints, 240
 - gg::GgShape, 322
 - gg::GgSimpleObj, 325
 - GgApp::Window, 491
 - Menu, 444
- operator!
 - gg::GgPoints, 240
 - gg::GgShape, 322
 - gg::GgSimpleObj, 325
- operator+
 - gg, 75
 - gg::GgMatrix, 212, 213
 - gg::GgQuaternion, 300
 - gg::GgVector, 389
- operator+=
 - gg::GgMatrix, 213, 214
 - gg::GgQuaternion, 301
 - gg::GgVector, 390
- operator-
 - gg, 76
 - gg::GgMatrix, 215
 - gg::GgQuaternion, 302
 - gg::GgVector, 391
- operator-=
 - gg::GgMatrix, 216
 - gg::GgQuaternion, 303
 - gg::GgVector, 392
- operator/
 - gg, 76
 - gg::GgMatrix, 217, 218
 - gg::GgQuaternion, 304
 - gg::GgVector, 393
- operator/=
 - gg::GgMatrix, 218, 219
 - gg::GgQuaternion, 305, 306
 - gg::GgVector, 394
- operator=
 - Buffer, 92
 - Calibration, 102
 - Camera, 118
 - Expand, 146
 - Framebuffer, 155, 156
 - gg::GgBuffer< T >, 169, 170
 - gg::GgMatrix, 220
 - gg::GgQuaternion, 306–308
 - gg::GgShader, 318, 319
 - gg::GgSimpleShader, 336
 - gg::GgTexture, 349, 350
 - gg::GgTrackball, 359
 - gg::GgVector, 395
 - gg::GgVertexArray, 401
 - GgApp, 163
 - GgApp::Window, 491, 492
 - Menu, 444
 - Mesh, 448
 - Texture, 469
- operator*
 - gg, 75
 - gg::GgMatrix, 209, 210
 - gg::GgQuaternion, 298
 - gg::GgVector, 387
- operator*=
 - gg::GgMatrix, 211
 - gg::GgQuaternion, 299
 - gg::GgVector, 388
- orthogonal
 - gg::GgMatrix, 221

- out
 - Camera, 121
- parseconfig.h, 756
 - getString, 757, 758
 - getValue, 758, 759
 - setString, 759, 760
 - setValue, 761
- pathChar
 - gg.h, 648
- pathString
 - gg.h, 648
- perspective
 - gg::GgMatrix, 221
- position
 - gg::GgSimpleShader::Light, 407
 - gg::GgVertex, 398
- Preference, 449
 - ~Preference, 451
 - buildShader, 451
 - getDescription, 451
 - getIntrinsics, 451
 - getShader, 451
 - Preference, 450
 - setPreference, 451
- Preference.cpp, 764
- Preference.h, 766
- projection
 - gg::GgMatrix, 222, 223
- read
 - gg::GgBuffer< T >, 170
 - gg::GgUniformBuffer< T >, 376
- readPixels
 - Texture, 470, 471
- recordCorners
 - Calibration, 103
- region
 - gg::GgTrackball, 360
- REQUESTS.md, 768
- REQUESTS.md - アプリケーションビルドおよび開発構成定義書, 3
- reset
 - gg::GgTrackball, 361
 - GgApp::Window, 492
- resetRotation
 - GgApp::Window, 492
- resetTranslation
 - GgApp::Window, 493
- resize
 - Buffer, 93, 94
- restoreViewport
 - GgApp::Window, 493
- retrieve
 - Capture, 136
- rotate
 - gg::GgMatrix, 223–225
 - gg::GgQuaternion, 308, 309
 - gg::GgTrackball, 362
- rotateX
 - gg::GgMatrix, 226
 - gg::GgQuaternion, 310
- rotateY
 - gg::GgMatrix, 227
 - gg::GgQuaternion, 311
- rotateZ
 - gg::GgMatrix, 227
 - gg::GgQuaternion, 311
- running
 - Camera, 121
- RvecTvecToPose
 - Calibration, 103
- SafeRelease
 - CamMf.cpp, 529
- samples
 - Settings, 456
- save
 - CamImage, 126
 - Config, 143
- savelmage
 - Menu, 445
- saveParameters
 - Calibration, 104
- scale
 - gg::GgMatrix, 228, 229
- Scene, 452
 - ~Scene, 453
 - draw, 454
 - light, 454
 - Scene, 453
 - set, 454
- Scene.cpp, 768
- Scene.h, 770
- select
 - CamMf, 131
 - gg::GgSimpleShader::LightBuffer, 422
 - gg::GgSimpleShader::MaterialBuffer, 439
- selectInterface
 - GgApp::Window, 493
- send
 - gg::GgBuffer< T >, 171
 - gg::GgElements, 181
 - gg::GgPoints, 240
 - gg::GgTriangles, 367
 - gg::GgUniformBuffer< T >, 377
- sensorSize
 - Intrinsics, 406
- set
 - Scene, 454
- setCenter
 - Intrinsics, 404
- setClose
 - GgApp::Window, 494
- setDictionary
 - Calibration, 104
- setExposure
 - CamCv, 112

- setFov
 - Intrinsics, [404](#), [405](#)
- setFps
 - Intrinsics, [405](#)
- setGain
 - CamCv, [112](#)
- setKeyboardFunc
 - GgApp::Window, [494](#)
- setMode
 - gg::GgShape, [323](#)
- setMouseFunc
 - GgApp::Window, [494](#)
- setPosition
 - CamCv, [113](#)
- setPreference
 - Preference, [451](#)
- setResizeFunc
 - GgApp::Window, [495](#)
- setSize
 - Intrinsics, [405](#)
 - Menu, [445](#)
- setString
 - parseconfig.h, [759](#), [760](#)
- Settings, [454](#)
 - checkerLength, [455](#)
 - defaultEuler, [455](#)
 - defaultFocal, [455](#)
 - defaultFocalRange, [455](#)
 - dictionaryName, [456](#)
 - euler, [456](#)
 - focal, [456](#)
 - focalRange, [456](#)
 - getFocal, [455](#)
 - markerLength, [456](#)
 - samples, [456](#)
 - Settings, [455](#)
- setup
 - Expand, [146](#)
 - Menu, [446](#)
- setUserPointer
 - GgApp::Window, [495](#)
- setValue
 - parseconfig.h, [761](#)
- setVelocity
 - GgApp::Window, [495](#)
- setWheelFunc
 - GgApp::Window, [496](#)
- shininess
 - gg::GgSimpleShader::Material, [424](#)
- shouldClose
 - GgApp::Window, [496](#)
- show
 - Framebuffer, [156](#)
- size
 - Intrinsics, [406](#)
- slerp
 - gg::GgQuaternion, [312](#)
- specular
 - gg::GgSimpleShader::Light, [408](#)
 - gg::GgSimpleShader::Material, [425](#)
- start
 - Camera, [119](#)
 - Capture, [136](#)
- stop
 - Camera, [119](#)
 - Capture, [136](#)
- subtract
 - gg::GgQuaternion, [313](#), [314](#)
- SubTypeToName
 - CamMf.cpp, [530](#)
- swapBuffers
 - GgApp::Window, [496](#)
- swapRandB
 - gg::GgTexture, [350](#)
- TCharToUtf8
 - gg.h, [648](#)
- Texture, [456](#)
 - ~Texture, [461](#)
 - bindTexture, [461](#)
 - copy, [462](#)
 - create, [463](#)
 - discard, [464](#)
 - draw, [465](#)
 - drawPixels, [465–467](#)
 - getChannels, [468](#)
 - getSize, [468](#)
 - getTextureName, [468](#)
 - mesh, [472](#)
 - operator=, [469](#)
 - readPixels, [470](#), [471](#)
 - Texture, [458–460](#)
 - unbindTexture, [472](#)
- Texture.cpp, [771](#)
- Texture.h, [775](#)
- thr
 - Camera, [121](#)
- total
 - Camera, [122](#)
- translate
 - gg::GgMatrix, [230](#), [231](#)
- transmit
 - Camera, [120](#)
- transpose
 - gg::GgMatrix, [232](#)
- unbind
 - gg::GgBuffer< T >, [172](#)
 - gg::GgTexture, [351](#)
 - gg::GgUniformBuffer< T >, [378](#)
- unbindBuffer
 - Buffer, [95](#)
- unbindFramebuffer
 - Framebuffer, [157](#)
- unbindTexture
 - Texture, [472](#)
- unmap

- Buffer, [95](#)
- gg::GgBuffer< T >, [172](#)
- gg::GgUniformBuffer< T >, [378](#)
- unuse
 - gg::GgPointShader, [249](#)
 - gg::GgShader, [319](#)
- update
 - Framebuffer, [157](#), [158](#)
- updateViewport
 - GgApp::Window, [497](#)
- use
 - Compute, [138](#)
 - gg::GgPointShader, [249–252](#)
 - gg::GgShader, [319](#)
 - gg::GgSimpleShader, [337–344](#)
- USE_PIXEL_BUFFER_OBJECT
 - Buffer.h, [503](#)
- USE_RODRIGUES
 - Calibration.cpp, [508](#)
- Utf8ToTChar
 - gg.h, [648](#)
- width
 - Camera, [122](#)
- Window
 - GgApp::Window, [474](#), [475](#)
- ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版., [1](#)